

บันทึกภายในทีม

ความผิดพลาดทั้งหมด ที่เราทำระหว่างสร้าง Constistant และ Destrier

รวมทุกเรื่องผิดพลาด ตั้งแต่วันแรกจนวันชนะ — เก็บไว้ครบทุกข้อ เพื่อไม่ให้พลาด
ซ้ำอีก

รวบรวมจากไดอารี่ข้างทั้งหมด 71 ไฟล์ · ช่วงเวลา 2026-07-01 ถึง 2026-09-02 · ทั้งหมด 1063 เรื่อง

เอกสารนี้เกิดขึ้นหลังทีมชนะเลิศ STECON Construction Innovation Challenge 4 — และ
เหตุผลจริงที่ชนะ ไม่ใช่เพราะไม่มีอะไรผิดพลาด แต่เพราะเกือบทุกอย่างเคยพังมาแล้วอย่างน้อยหนึ่งครั้ง
ถูกจัดไว้ แล้วถูกแก้ ทุกข้อในนี้เก็บไว้ครบโดยตั้งใจ ไม่มีการตัดข้อที่ดูแย่ออก เพราะข้อที่ดูแย่ที่สุด คือ
ข้อที่ยังมีประโยชน์เหลืออยู่มากที่สุด

วิธีใช้เอกสารนี้

เอกสารนี้ไม่ได้มีไว้อ่านรวดเดียวจบ มีไว้เปิดหาตอนกำลังจะทำอะไรที่เคยพลาดมาแล้ว

- ก่อนเริ่มงานที่เสียเงินหรือเสียเวลาเยอะ (เทรนโมเดล เข้าคาร์ดจอ ย้ายข้อมูลชุดใหญ่) — อ่านหัวข้อ “กฎ 31 ข้อ” ก่อน เป็นเนื้อหาทั้งหมดที่บีบมาเหลือไม่กี่หน้า
- ตอนมีอะไรพัง — ค้นหาการในเล่มนี้ก่อนจะเริ่มเดาสาเหตุ ความผิดพลาดจำนวนมากในนี้คือครั้งที่สองและสามของเรื่องเดิม
- ตอนทำงานเสร็จ — ถ้าเจออะไรที่ยังไม่มีในเล่ม ให้เพิ่มเข้าไป รูปแบบตั้งใจให้เหมือนกันหมด: เกิดอะไรขึ้น · ต้นเหตุจริง · เสียหายอะไร · แก้ยังไง · ได้กฎอะไรกลับมา

ภาพรวมทั้งหมด

ระดับความรุนแรง	จำนวน
วิกฤต เสียเงิน เสียข้อมูล หรือเสียงานทั้งรอบ	147
สูง ตัวเลขผิดพลาดออกไป หรือเสียเวลาเป็นวัน	346
กลาง จับได้ก่อนเสียหาย แต่เป็นเรื่องจริง	422
ต่ำ เรื่องเล็กและเกือบพลาด	148
รวมทั้งหมด	1063

แบ่งตามหมวด

หมวด	ทั้งหมด	วิกฤต
การตรวจสอบ และ ความมั่นใจผิดๆ	83	10
พึงเสียบ ไม่มีใครรู้	22	7
วิธีวัดผล และ การประเมิน	94	23
คุณภาพข้อมูล ความถูกต้อง และ ข้อมูลหาย	143	26
สเปกข้อมูล และ ข้อตกลงที่ค่อยๆ เปลี่ยน	40	4
กัปดักการตั้งค่า สภาพแวดล้อม และ โลบารรี	87	18
เครื่อง การ์ดจอเช่า และ ค่าใช้จ่าย	124	7
กัปดักของเครื่องมือ และ พฤติกรรมของบริการภายนอก	58	2
การเทรนโมเดล และ วิธีทำ ML	75	18
บั๊กในโค้ด และ ตรรกะผิด	53	6

Git ความสะอาดของ repo และ ของที่ทำแล้วหาย	19	2
เอกสาร การตั้งชื่อ และ การสื่อสาร	34	3
กระบวนการทำงาน การวางแผน และ งานที่เสียเปล่า	231	21

กฎ 31 ข้อ

กลั่นมาจาก 147 เรื่องที่รุนแรงระดับวิกฤต เรียงตามว่ากันความเสียหายได้มากแค่ไหน ไม่ได้เรียงตามว่าข้อไหนฟังดูเท่

พิสูจน์ อย่าเดา

1. “รันผ่าน ไม่มี error” ไม่ได้แปลว่ามันทำงานถูก

หลังทุกขั้นตอนที่แปลงของ — merge, convert, quantize, export, ย้ายข้อมูล — ต้องวัดของที่ได้ออกมาจริงๆ เทียบน้ำหนักโมเดล นับจำนวนแถว อ่านไฟล์กลับมาดู ขั้นตอนที่ไม่ขึ้น error ยังไม่ได้พิสูจน์อะไรเลยสักอย่างเกี่ยวกับผลลัพธ์ของมัน

2. ตัวกันที่ยังไม่เคยเห็นมันจับอะไรได้ ไม่นับว่าเป็นตัวกัน

ก่อนจะเชื่อการตรวจสอบใดๆ ให้ป้อนของที่ผิดแน่ๆ เข้าไปแล้วดูว่ามันร้องจริงไหม การตรวจที่ของฟังผ่านไปได้ แยกว่าไม่ตรวจเลย เพราะมันสร้างความมั่นใจปลอมๆ ให้เรา

3. ห้ามบอกว่าเสร็จ จนกว่าจะรันจบทั้งเส้นด้วยข้อมูลจริงที่ใช้งานจริง

ไม่มี error ไม่ใช่หลักฐานว่าใช้ได้ ข้อมูลตัวอย่างที่เราเขียนเองมีข้อสันนิษฐานของเราฝังอยู่ และมันจะบังบั๊กตัวที่ทดสอบมีไว้จับพอดิ ให้สร้างข้อมูลทดสอบจากของจริงเสมอ

4. ตัวจำลองต้องเรียกโค้ดจริง ห้ามเขียนขึ้นใหม่ว่า “มันน่าจะทำงานแบบนี้”

ทางลัดทุกอันในตัวจำลอง จะกลายเป็นตัวเลขที่เกินจริงในผลลัพธ์ และจะมีคนเอาตัวเลขนั้น ไปตัดสินใจต่อ

5. เรนเดอร์สิ่งที่เครื่องเห็นจริงๆ ออกมาด้วยตาตัวเอง

ครอบรูปมาเทียบกัน 5 นาที ตีกว่านั่งคำนวณ token เป็นชั่วโมง วิธีนี้เคยช่วยประหยัดค่าการ์ดจอทั้งรอบมาแล้วหลายครั้ง

ห้ามฟังเสียง

6. ข้อมูลที่ขาดหาย ห้ามแสดงผลออกมาเป็นศูนย์เด็ดขาด

ทุกเส้นทางที่คิดราคา ปริมาณ หรือคะแนน ต้องแยกให้ออกระหว่าง “วัดได้จริงว่าเป็นศูนย์” กับ “ไม่รู้” แล้วส่งเสียงดังกับอย่างหลัง เพราะบรรทัดที่เป็นศูนย์เสียงๆ บนหน้าจอ แยกไม่ออกจากบรรทัดที่ถูกต้องจริง

7. อะไรที่หึง ต้องบอกว่าหึง ห้ามหายไปเงียบๆ

ค่าที่ไม่รู้จัก ฟิลด์ที่อ่านไม่ออก แถวที่ถูกกรองทิ้ง รูปแบบที่ไม่คุ้น ทุกอันต้องบอกว่าหึงอะไรไปและเพราะอะไร ตัวอ่านข้อมูลที่คืนค่าว่างเปล่าเมื่อเจอรูปแบบแปลกๆ คือบั๊กข้อมูลหายชัดๆ

8. ทุกขั้นตอนที่ทำข้อมูลหายได้ ต้องมีการนับยอดกระทบกันที่ปลายทาง

นับจำนวนที่เข้า เทียบกับจำนวนที่ถูกใช้จริง แล้ว assert ว่าหมวดที่ควรมีของต้องไม่ว่าง คิวที่มีแต่คนนับ แต่ไม่มีใครเอาไปใช้ คือช่องทางข้อมูลหายที่ปลอมตัวมาเป็นความคืบหน้า

9. ห้ามเชื่อค่าตั้งต้นที่เราไม่ได้เลือกเอง

พิมพ์ค่าจริงออกมาและ assert ตอนเริ่มงานเสมอ — จำนวน token ต่อรูป ความยาว sequence ขนาดรูป timeout ค่า decoding ค่า default ของไลบรารีที่เราไม่ได้เลือก คือบั๊ก config ที่รื้อวันแพ่งที่สุดของมันอยู่

10. ขาวดีเกินคาด ให้ถือว่าเป็นอาการผิดปกติ

แรมใช้น้อยผิดปกติ เทสผ่านฉลุยผิดปกติ คะแนนสวยเกินจริง — ให้ไปตรวจก่อนดีใจ ทุกอันที่ว่ามาเคยชอนบั๊กจริงในโปรเจกต์นี้มาแล้วทั้งนั้น

วัดให้ตรงกับสิ่งที่อยากรู้

11. ห้ามรายงานคะแนนโดยไม่บอกเพดานที่เป็นไปได้จริง

เปอร์เซ็นต์ที่เท่ากันสองตัว อาจเป็นคนละปัญหาก็ได้สุดซั่ว ตัววัดที่มีเพดานแค่ 4% ไม่ได้กำลังวัดคุณภาพโมเดล แต่กำลังวัดคุณภาพการ label ของเราเอง

12. ตัวเลขความแม่นยำทุกตัว ต้องมี “วัดจากอะไร” ติดมาด้วยเสมอ

ชุดทดสอบต้องโหลดไฟล์ตัวเดียวกับที่ใช้งานจริง ถ้าตอนวัดกับตอนใช้โหลดคนละไฟล์กัน ตัวเลขนั้นกำลังอธิบายคนละระบบกัน

13. คะแนนเรื่องรูปแบบ ไม่ใช่คะแนนเรื่องคุณภาพ

JSON ถูกต้อง 100% ได้ ในขณะที่ความหมายถูกแค่ 0% ทุกครั้งที่วัดผล ต้องมีคนเปิดผลลัพธ์จริงดูด้วยตา และต้องรายงานตัววัดที่สนใจจริงแยกออกมาต่างหาก

14. ห้ามมีค่าเฉลี่ยโผล่อยู่ในคำถามเด็ดขาด

และต้อง assert ว่าป้อนข้อมูลต่างกันแล้วต้องได้คำตอบต่างกัน ถ้าตอบเหมือนกันหมดทุกอัน นั่นคือมันลอคคำถาม ไม่ใช่ความสามารถ

15. ต้องตรวจว่าชุดเทรนกับชุดวัดผลไม่ปนกัน ทุกครั้ง ทุกรอบ

ห้ามเชื่อชุดแบ่งข้อมูลของรอบก่อนโดยไม่ตรวจซ้ำ และต้องมีแหล่งเดียวที่ตัดสินว่าข้อมูลชิ้นไหนอยู่ชุดไหน รายชื่อที่ hardcode ไว้เป็นชุดที่สอง คือรอยร้าวที่รอเวลาปะทุ

16. ทดสอบเคสที่สำคัญที่สุดก่อน

ถ้าครบรอบแล้วความสามารถหลักที่เราอยากรู้ยังไม่ถูกทดสอบเลย แปลว่ารอบนั้นวัดผิดสิ่งไปทั้งรอบ

ความจริงต้องมีที่มาที่เดียว

17. แก่ schema ยังไม่จบ จนกว่าทุกคนที่อ่านมัน และตัวตรวจของมัน จะถูกแก้ไขใน commit เดียวกัน

คำว่า “validator ผ่านแล้ว” ไม่มีความหมายเลย ถ้าเราชี้ไม่ได้ว่าตัวไหนคือตัวที่ควรจับได้ก่อนที่จะแก้

18. เจอบั๊กแล้ว ต้อง grep หาทุกจุดที่เรียกใช้ ก่อนปิดงาน

ที่ที่ควรแก้คือฟังก์ชันกลางที่ทุกคนใช้ร่วมกัน ไม่ใช่ไฟล์ที่เราบังเอิญเปิดอยู่ตอนนั้น การแก้ฝาแฝดไปแค่ข้างเดียว คือบั๊กที่รอคิวของอีกข้างอยู่

19. ค่าเดียวกันที่เก็บไว้สองที่ จะเพี้ยนจากกันแน่นอน ให้ถือไว้ก่อนเลย

timeout ฝั่ง client กับฝั่ง server, config ที่ก๊อปกันมา, รายชื่อที่ hardcode ซ้ำ — ให้ดึงมาจากที่เดียว และ assert ความสัมพันธ์ระหว่างกันด้วย (เช่น timeout ฝั่ง server ต้องน้อยกว่าฝั่ง client เสมอ)

20. ค่าคงที่ทางวิศวกรรมทุกตัว ต้องมีมาตรฐานและเลขช้อยกำกับไว้

เพื่อให้ค่าที่ไม่ตรงมองเห็นได้ตั้งแต่ตอนอ่าน ไม่ใช่มารู้อีกหลายปีให้หลัง และเมื่อค่าไหนเปลี่ยน ต้องคำนวณทุกอย่างที่ขึ้นอยู่กับค่าของมันใหม่ในรอบเดียวกัน

เรื่องที่ย้อนกลับไม่ได้ ต้องหยุดให้สนิท

21. ไฟล์ที่อยู่บนเครื่องเรา ไม่ใช่ข้อมูลสำรอง

ก่อนเปิดหรือคืนเครื่องทุกครั้ง ต้องไปไล่ดูรายชื่อไฟล์ที่ปลายทางนอกเครื่องว่ามีอยู่จริง และต้องทดสอบปลายทางไว้ก่อน ไม่ใช่รอถึงเวลาที่ต้องใช้จริงค่อยรู้ว่ามันใช้ไม่ได้

22. การสำรองข้อมูลคือด่านบังคับที่ข้ามไม่ได้ ไม่ใช่โน้ตเตือนความจำ

และห้ามเอาไปแขวนไว้ในขั้นตอนที่ข้ามได้ ก่อนจะข้ามขั้นตอนไหนก็ตาม ต้องไล่ดูก่อนว่าขั้นตอนนั้นแบกอะไรไว้บ้าง

23. คำเตือนเรื่องข้อมูลหายถาวร ต้องพูดแยกออกมาต่างหาก เป็นภาษาคน พร้อมคำว่า “หยุด”

ห้ามซุกไว้ในรายการงานว่า “เดี๋ยวค่อยทำ” ถ้าผลที่ตามมาคือกินไม่ได้ ให้ปฏิเสธที่จะทำต่อ ดีกว่าทำต่อแล้วแปะโน้ตไว้เฉยๆ

24. งานที่กำลังรันอยู่ ให้ถือว่ามันจองทั้งเครื่องและทั้ง environment ไว้แล้ว

ห้าม pip install ห้ามใช้ interpreter รวมกัน ห้ามแทรกงานอื่นเข้าไป ให้แยก venv ออกมา หรือรอให้งานเดิมจบสนิทก่อน

ขนาดและขีดจำกัด ต้องมาจากของจริงที่วัดแล้ว

25. ทุกค่าเพดาน ขีดจำกัด และความยาว ต้องมาจากค่าสูงสุดที่วัดจากข้อมูลจริง

ห้ามมาจากตัวอย่างไม่กี่ชิ้น ห้ามมาจากค่ากลาง ห้ามมาจากการเดา และการตัดข้อมูลทั้งต้อง error ดังๆ เพราะการตัดเจี๊ยบๆ ทำลาย label ไปโดยไม่มีอะไรป้องกัน

26. ห้ามสรุปลักษณะข้อมูลทั้งหมด จากตัวอย่างชิ้นเดียว

ประโยคแบบ “ข้อมูลมันหน้าตาแบบนี้” ต้องมาจากสคริปต์ที่วัดทุกชิ้นแล้วรายงานค่าต่ำสุด ค่าสูงสุด และจำนวน

27. พิสูจน์เส้นทางขึ้นใช้งานทั้งเส้นบนโมเดลตั้งต้นก่อน แล้วค่อยจ่ายเงินเทรน

โมเดลที่ export ไม่ได้ โหลดไม่ได้ เสิร์ฟไม่ได้ มีค่าเท่ากับศูนย์ ไม่ว่าคะแนนจะสวยแค่ไหนก็ตาม

บทเรียนต้องกลายเป็นเครื่องจักร

28. ความผิดพลาดที่เกิดซ้ำ คือด่านอัตโนมัติที่ยังไม่มี ไม่ใช่โชคร้าย

พอเห็นอาการเดิมเป็นครั้งที่สอง ให้หยุดแก้เป็นรายครั้ง แล้วติดตั้งการตรวจถ่วงก่อนรันแทน ในโปรเจกต์นี้บั๊กคลาสเดียวกันเกิดซ้ำถึง 3 ครั้ง ทั้งที่บทเรียนของมันถูกเขียนไว้เรียบร้อยแล้วเป็นตัวแทนสื่อ

29. บทเรียนที่จดไว้ แต่ไม่ได้เอาไปใช้ คือบทเรียนที่ยังไม่ได้เรียน

วันที่เขียนลงไดอารี่ว่า “X เป็นปัญหา” ให้ grep ทั้ง repo หา X แล้วแก้ให้หมดในวันเดียวกันนั้นเลย

30. โค้ดที่ไม่เคยรัน คือโค้ดที่ยังไม่ได้ตรวจ และมันเน่าไปตามเวลา

รันทุกสคริปตัวอย่างน้อยหนึ่งครั้งกับข้อมูลเล็กๆ ก่อนเอาไปใช้กับงานที่เสียเงิน โค้ดที่ทิ้งไว้ไม่ได้รันนานๆ เสี่ยงกว่าโค้ดที่ยังไม่เคยมีอยู่เสียอีก

31. ห้ามเขียนต้นเหตุที่ยังพิสูจน์ไม่ได้

ต้องพิสูจน์ด้วยการอ่านโค้ดจริง หรือทำซ้ำให้เห็นสักรบรทัด ถ้าหลักฐานยังไม่พอ ให้เขียนตรงๆ ว่า “ยังไม่รู้ต้นเหตุ สงสัย A หรือ B” เพราะต้นเหตุผิดที่เขียนอย่างมั่นใจ พาทุกคนไปผิดทางกันหมด

การตรวจสอบ และ ความมั่นใจผิดๆ

บอกว่าเสร็จ/ถูกต้อง/ตรวจแล้ว ทั้งที่ยังไม่ได้ตรวจจริง

ทั้งหมด 83 เรื่อง · วิกฤต 10 · สูง 24 · กลาง 43 · ต่ำ 6

เรื่องที่ต้องอ่านให้ครบ (34)

วิกฤต 2026-07-09

Tool call รายงานว่า 'ถูกปฏิเสธ' แต่ .gitignore ถูกแก้ไปแล้วจริงๆ บนดิสก์

เกิดอะไรขึ้น: การพยายามอัปเดต 8 บรรทัดที่เกี่ยวกับ raw/ ใน .gitignore ถูกรายงานกลับมาจากระบบว่าเป็น tool use ที่ถูกปฏิเสธ แต่พอตรวจสอบรอบสุดท้ายกลับพบว่าเนื้อหาไฟล์ถูกเปลี่ยนเป็นชื่อโฟลเดอร์ใหม่ไปแล้วจริงๆ — ผลลัพธ์ที่รายงานไม่ตรงกับสภาพจริงบนดิสก์

ต้นเหตุจริง: สัญญาณ 'ปฏิเสธ' จากระบบเชื่อไม่ได้ว่าไม่มีการเขียนไฟล์เกิดขึ้น แต่กระบวนการของทิมกลับถือผลลัพธ์ของ tool เป็นข้อสรุป แทนที่จะไปตรวจตัวไฟล์จริง

เสียหายอะไร: ถ้าไม่ได้ตรวจสอบสุดท้าย .gitignore จะค้างซีไปชื่อโฟลเดอร์ที่ไม่มีอยู่แล้วอย่างถาวร ทำให้ข้อมูล 533 MB หลุดออกจากการ ignore แบบเสียๆ

แก้ยังไง: ไปอ่านไฟล์จริงบนดิสก์ คำนวณ .gitignore กลับเป็น raw/ แล้วยืนยันด้วยการ grep ทั้ง repo ได้ผลลัพธ์ 0 จุดสำหรับชื่อที่เล็กใช้

กฎที่ได้: อย่าเชื่อว่ารายงาน 'ถูกปฏิเสธ'/'ล้มเหลว' ของ tool แปลว่าไม่มีอะไรเปลี่ยน — ให้ไปอ่านไฟล์จริงกลับมาดูหลังทุกครั้งที่มีการเขียนล้มหรือถูกปฏิเสธ โดยเฉพาะก่อนจะสรุปว่าย้อนกลับสำเร็จแล้ว

วิกฤต 2026-07-13

ค่ากริดสมมติ 8.5 ที่ผิด ถูกยืนยันด้วยหลักฐานที่จริงๆ แล้วคือการอ้างอิงกลับมาที่ตัวเอง

เกิดอะไรขึ้น: พอถูกถามว่ากริดสมมติ `3` ควรเป็น 7.6 ไม่ใช่ 8.5 หรือเปล่า Claude ตอบผิดว่ายืนยันแล้วว่าเป็น 8.5 โดยอ้างไจระยะในหน้า 06 กับช่วงคาน B4X ในหน้า 19 เป็นหลักฐานยืนยันอิสระ แต่ช่วงของ B4X มี `span\_source: "grid\_table"` แปลว่ามันถูกคำนวณย้อนกลับมาจากค่า 8.5 ในกริดมันนั่นเอง

ต้นเหตุจริง: เอาค่าที่คำนวณต่อมา (derived) ไปนับเป็นหลักฐานยืนยันอิสระ ทั้งที่ `confidence\_flags` ในไฟล์นั้นเองก็เขียนไว้แล้วว่า `uses\_dummy\_grid\_3\_from\_page00\_master` แต่ไม่มีใครดูตรงนั้นก่อนจะเอาไปนับเป็นแหล่งที่สอง

เสียหายอะไร: ค่ากริดที่ผิดซึ่งสืบทอดมาตั้งแต่รอบ t3/t4 ลามไปที่ช่วงคาน B4X สองตัว (เป็น 1.5 แทนที่จะเป็น 0.6) และคำตอบที่ผิดถูกยืนยันผ่านการถามซ้ำสองรอบ จนผู้ใช้ต้องบังคับให้ไปตรวจใหม่กับภาพผังบริเวณ

แก้ยังไง: ไปดูหน้า A-03 (ผังบริเวณ) ซึ่งระบุชัดว่าตัวอาคารยื่นเลยกริด 3 ออกไป 0.60 ม. แก่ `3` จาก 8.5 เป็น 7.6 และแก้ช่วงคาน B4X ทั้งสองตัวจาก 1.5 เป็น 0.6 พร้อมเขียนคำเตือนบันทึกเรื่องการอ้างอิงไว้

กฎที่ได้: ก่อนจะนับค่าไหนเป็นหลักฐานยืนยันอิสระ ให้ดู `span\_source`/`confidence\_flags` ของมันก่อน — ค่าที่คำนวณมาจากกริดแม่ ไม่มีทางใช้ยืนยันกริดแม่ได้ และให้เขียนกติกาข้อนี้ลงใน schema ให้ชัด

เสาถูกทำโครงสร้างข้อมูลแบบคาน มีเหล็กบน/ล่าง ทำให้นับเหล็กแกนหลัก 4 เส้นกลายเป็น 8

เกิดอะไรขึ้น: ไฟล์ `หน้า21\_view2\_schedule.json` บันทึกเสา C1 ไว้ด้วย `main\_bar.top: 4` และ `main\_bar.bottom: 4` รวมเป็น 8 ทั้งที่ในแบบมีเหล็กมุมชุดเดียว 4 เส้น (4-Ø12mm) เสาไม่มีหน้าบน/ล่าง

ต้นเหตุจริง: เอาโครงสร้างสเปกของคานมาใช้กับเสาซ้ำๆ โดยไม่ได้ตรวจว่าแนวคิดมันย้ายมาใช้ได้จริงไหม บางไฟล์ถึงกับกลบอาการด้วยการเพิ่มช่อง `total\_main\_bar\_count` แทนที่จะไปแก้โครงสร้างให้ถูก

เสียหายอะไร: นับเหล็กแกนหลักของเสาเกินจริง 100% ทุกจุดที่เกิดขึ้น เป็นความผิดพลาดที่กระทบปริมาณเหล็กและต้นทุนโดยตรง เจอทั้งในบ้าน 1, บ้าน 2 (หน้า 29), บ้าน 3 (หน้า 35) และต่อมาเจออีก 6 ไฟล์ในเสาร์แรกของบ้าน 1/2/3

แก้ยังไง: เปลี่ยนเป็น `main\_bar: {count: 4}` ชุดเดียว ยืนยันทั้งโปรเจกต์บนบ้าน 2 แล้ววันที่ 2026-07-20 ทำการกวาดทั้งโปรเจกต์ เจอและแก้อีก 6 ไฟล์ที่มีบั๊กเดียวกัน แล้วยืนยันว่าไม่เหลือจุดไหนอีก

กฎที่ได้: อย่าเอาโครงสร้างสเปกของ element ชนิดหนึ่งไปใช้กับอีกชนิดหนึ่งโดยไม่ตรวจว่าแนวคิดมันย้ายมาได้จริง และเมื่อไหร่ที่เห็นช่องแก้ชดอย่าง `total\_main\_bar\_count` โพลมา ให้ถือว่ามันคือรายงานว่ามีบั๊กเชิงโครงสร้างที่ยังไม่ได้แก้ แล้วกวาดทั้งโปรเจกต์ทันที

แผนโครงสร้างของบ้าน 3 สกัดข้อมูลขาดไป 60% — หน้า 32 จาก 20 element กลายเป็น 32

เกิดอะไรขึ้น: การตรวจภาพแบบเต็มแผ่นของบ้าน 3 หน้า 31 (S-04) และ 32 (S-05) พบว่าหน้า 32 ขาดแนวคานแนวนอนทั้งแนวตามแถว D, ขาดรูปตัด B3X ที่แถว C, ขาดคานแนวตั้งสามตัว, ขาดคาน B1 ที่แนว B', ขาด BX และขาดจุดสังเกต S2 — จาก 20 element กลายเป็น 32 ส่วนหน้า 31 ต้องแก้ 7 จุด รวมถึงป้ายผิด (B3X→B3 สองที่, B2→B1, B3→B2) และการยึดกริดผิด (D1-C1→D1'-C1')

ต้นเหตุจริง: การสกัดข้อมูลรอบแรกไม่ได้กวาดทั้งแผ่นอย่างเป็นระบบ แต่บันทึก element ตามที่บังเอิญเจอ ทั้งแนวคานจึงหายไปโดยไม่มีใครสังเกต และการอ่านชื่อ mark ก็ไม่ได้ไปตรวจกับรูปตัดในสเปก หลายตัวจึงถูกติดป้ายผิด

เสียหายอะไร: นับ element ขาดไป 60% บนแผนโครงสร้างหลัก เป็นช่องว่างการสกัดข้อมูลที่ใหญ่ที่สุดเท่าที่วัดได้ในโปรเจกต์นี้ และชิ้นส่วนที่ต้องใช้ต่อเรื่อง B' (คาน B1 ที่แนว B') ก็เป็นหนึ่งในตัวที่หายไปด้วย

แก้ยังไง: ตั้งมาตราส่วนพิกเซลที่ปรับเทียบแล้ว (เส้นศูนย์กลางกริด x=1434.3px, กริด 2 x=1707.7px → 78.1 px/m, แนว E y=756) แล้วสกัดข้อมูลทั้งสองแผ่นใหม่ทั้งหมดโดยอิงมาตราส่วนนี้

กฎที่ได้: การสกัดข้อมูลจากผังโครงสร้าง ต้องกวาดอย่างเป็นระบบเทียบกับมาตราส่วนที่ปรับเทียบแล้ว ไม่ใช่หยิบเก็บตามที่บังเอิญเจอ ให้ตรวจว่าทุกแนวแถวและแนวเสามีชิ้นส่วนต่อเนื่องครบ และตรวจทุก mark กับสเปกในรูปตัด ไม่ใช่อ่านแค่ป้าย

ประกาศว่าโมเดล "พร้อมใช้แล้ว" ทั้งที่ยังอ่านรูปไม่ได้เลย — ซึ่งเป็นงานทั้งหมดของโปรเจกต์

เกิดอะไรขึ้น: วันที่ 2026-07-21 มีการรายงานว่าโมเดลที่จนแล้วพร้อมใช้งาน แต่ GGUF ที่ได้มาอ่านได้แค่ข้อความ ส่วนที่อ่านรูป (mmproj) แตกไฟล์ออกมาไม่สำเร็จ ทั้งที่การอ่านแบบก่อสร้างคือจุดประสงค์เดียวของโปรเจกต์นี้

ต้นเหตุจริง: นิยามคำว่า "เสร็จ" ด้วยการที่ pipeline รันจบโดยไม่มี error แทนที่จะดูว่าของที่ส่งมอบทำหน้าที่หลักได้จริงไหม ไม่มีการทดสอบตั้งแต่ต้นจนจบ (ป้อนแบบจริง ได้คำตอบจริง) มาก่อนก่อนจะเคลมว่า "พร้อม"

เสียหายอะไร: เกิดสถานะ "เสร็จแล้ว" ปลอมๆ — เป็นที่มาของฉายา "DAY OF SHAME" ของทีม ที่ถูกอ้างถึงซ้ำๆ ในเซสชันต่อๆ มา ในฐานะความผิดพลาดที่กฎกระบวนการทั้งหมดหลังจากนั้นมีไว้เพื่อป้องกัน

แก้ยังง: เพิ่มการทดสอบแบบซ้อมบนเครื่องตัวเอง (Phase 0.3) — ดาวนี่โหลด mmproj ตัวทางมารมา รัน llama.cpp กับหน้าแบบจริงของบ้าน house01 แล้วยืนยันว่าอ่านข้อความไทย เส้นกริด และป้ายฐานรากได้ถูกต้อง ก่อนจะเคลมว่าพร้อม

กฎที่ได้: อย่าเพิ่งบอกว่าของพร้อมใช้ จนกว่าจะได้ลองใช้จริงตั้งแต่ต้นจนจบกับข้อมูลจริงที่ใช้งานจริง การไม่มี error ไม่ใช่หลักฐานว่ามันทำงานได้

บันทึกสนหลังคาหลักที่ไม่มีอยู่จริง เพราะแบบสองแผ่นดูเหมือนยืนยันตรงกัน ทั้งที่อ่านคนละเส้นที่หน้าตาคลายกัน

เกิดอะไรขึ้น: แบบ S-06 กับ A-07 ถูกถือว่ายืนยันตรงกันอย่างเป็นอิสระว่ามีสนหลังคาหลักบนก้อนอาคาร 11.4×11.4 แต่จริงๆ แล้วก่อนหน้านี้เป็นหลังคาปั้นหยาทรงสี่เหลี่ยมจัตุรัสที่มีเส้นสี่เส้นมาบรรจบกันที่จุดเดียว — ไม่มีสนเลย S-06 อ่านเส้นจันทันกลาง ส่วน A-07 อ่านเส้นตัด A-A สองเส้นที่หน้าตาคลายกันคนละอันซึ่งบังเอิญให้คำตอบตรงกัน

ต้นเหตุจริง: ยอมรับว่าแบบสองแผ่นตรงกันเท่ากับตรวจสอบแล้ว โดยไม่ได้เช็ควาทะทั้งสองแผ่นกำลังอ่านของชิ้นเดียวกันจริงไหม การอ่านผิดสองครั้งบนวัตถุคนละชิ้นที่ดันไปลงเอยที่คำตอบผิดอันเดียวกัน คือการยืนยันเท็จที่หนักที่สุดเท่าที่จะเป็นไปได้

เสียหายอะไร: มีการบันทึกชิ้นส่วนโครงสร้างที่ไม่มีอยู่จริง แถมยังได้คะแนนความมั่นใจสูงขึ้นจากการยืนยันเท็จนั้น — เป็นความผิดพลาดชนิดที่อันตรายที่สุด เพราะมันดูเหมือนผ่านการตรวจสอบมาแล้ว

แก้ยังง: อ่านแบบทั้งสองแผ่นใหม่เทียบกับรูปทรงจริง แล้วลบสนหลังคาออก บันทึกเป็นหลักการที่มีชื่อเรียกไว้ในไดอารี่

กฎที่ได้: การที่แบบสองแผ่นให้ข้อมูลตรงกัน จะนับเป็นการตรวจสอบได้ก็ต่อเมื่อพิสูจน์ได้ว่าทั้งสองแผ่นอ่านของชิ้นเดียวกัน การที่คำอ่านผิดสองอันตรงกันไม่ใช่การตรวจสอบ — ให้ระบุข้อผิดพลาดจริงที่แต่ละแหล่งกำลังอ่านอยู่ก่อนเสมอ ก่อนจะถือว่ามันเป็นแหล่งอิสระต่อกัน

sanity check ของ GGUF นั้นโมเดลที่ไม่ได้ tune เลยก็ผ่านได้สบาย

เกิดอะไรขึ้น: export_gguf.py บรรทัด 107 กันการ merge สัมแบบเสียบๆ ด้วยการเช็คค่า output ขึ้นต้นด้วย '{' และมีคำใดคำหนึ่งใน "png"/"views"/"pattern"/"doc_page" มันขึ้นเครื่องหมายถูกสีเขียวเมื่อวันที่ 24 ก.ค. และทุกคนก็เชื่อมาตลอด แต่ตัว prompt เองสั่งให้โมเดลพ่นคำว่า "views" และ "pattern" อยู่แล้ว โมเดลไหนที่ทำตาม prompt เป็นก็ผ่านหมด

ต้นเหตุจริง: ตัวตรวจไปเช็คค่าที่ prompt ป้อนให้โมเดลเอง ไม่ใช่ค่าที่มีแต่การเทรนเท่านั้นที่ผลิตได้ มันถูกเขียนขึ้นโดยไม่ได้ถามตัวเองว่า 'ถ้าของมันพังจริงๆ ตัวนี้จะร้องไหม'

เสียหายอะไร: ความมั่นใจจอมปลอมใน GGUF ของ t01 นานถึง 5 วัน ความล้มเหลวชนิดที่ตัวกันนี้มิไว้ดักกลับหลุดไปโดยไม่มีใครรู้ ส่วนตัวกันของ t02 ที่เข้มกว่า (เทียบชื่อ key จริง) จับบักตัวเดียวกันได้ทันที ความต่างระหว่าง 'จับได้' กับ 'จับไม่ได้' คือคุณภาพของตัวกันล้วนๆ

แก้ยังง: ยังไม่แก้ — เป็นงานลำดับ 2 ของเซสชันถัดไป: เขียนตัวกันใหม่ให้เทียบชื่อ key จริงของ gold (png/doc_page/discipline/sheet_code) แบบเดียวกับตัวกันของ t02

กฎที่ได้: ตัวกันทุกตัวต้องเอาไปทดสอบกับ input ที่รู้อยู่แล้วว่าผิด ก่อนจะไว้ใจมัน ห้ามตรวจสอบด้วยข้อความที่ปรากฏอยู่ใน prompt เด็ดขาด — ให้ตรวจเฉพาะข้อความที่มีอยู่ในคำตอบสำหรับเทรนเท่านั้น ตัวตรวจที่ไม่มีทางล้มได้แยกว่าไม่มีตัวตรวจเลย เพราะมันผลิตความมั่นใจปลอมๆ ขึ้นมา

ประตูดตรวจว่าโมเดล tune แล้วหรือยังใน export_gguf.py ไปตรวจค่าที่ prompt ป้อนให้เอง โมเดลที่ไม่ได้ tune จึงผ่านฉลุย

เกิดอะไรขึ้น: export_gguf.py บรรทัด 107 ตัดสินว่าโมเดล 'tune แล้ว' ถ้าคำตอบขึ้นต้นด้วย { และมีคำใดคำหนึ่งใน "png", "views", "pattern", "doc_page" แต่ตัว prompt เองสั่งให้โมเดลพ่น "views" และ "pattern" อยู่แล้ว โมเดลไหนที่ทำตามคำสั่งเป็นก็ผ่านหมด ประตุนี้ขึ้นเขียวตั้งแต่ 24 ก.ค. และทุกคนก็เชื่อมาตลอด

ต้นเหตุจริง: ตัวตรวจถูกเขียนจากมุมมองว่า 'คำตอบที่ดีหน้าตาเป็นยังง' ไม่ใช่จากมุมมองว่า 'อะไรที่มีแต่โมเดลที่ tune แล้วเท่านั้นจะผลิตได้' ค่าที่แยกแยะได้จริง (png/doc_page) มีอยู่แค่ในข้อมูลจริงและไม่เคยถูกเอ่ยใน prompt เลย แต่การใช้ OR กับ 4 คำทำให้ค่าที่ prompt ป้อนให้ทำให้เงื่อนไขผ่านได้

เสียหายอะไร: ไฟเขียวปลอมนี้บังความล้มเหลวเสียบๆ ของ merge_and_unload ไว้ถึง 5 วัน และทำให้กล้าเอา GGUF ไปใช้กับงานสกัดข้อมูลจริง ส่วนประตูของ t02 ที่เข้มกว่า (เทียบชื่อ key จริง) จับบักตัวเดียวกันได้ทันที — บักเดียวกัน ต่างกันแค่คุณภาพของประตู

แก้ยังง: วางแผนแก้แบบไม่มีค่าใช้จ่ายไว้แล้ว: เปลี่ยนประตูให้เทียบชื่อ key จริงของข้อมูลจริงแบบที่ t02 ทำ โดยตรวจเฉพาะค่าที่ไม่มีอยู่ใน prompt

กฎที่ได้: ออกแบบประตูตรวจสอบทุกตัวด้วยการถามว่า 'ถ้าของมันพัง ตัวนี้จะจับได้ไหม' ตัวตรวจที่ของพังๆ ผ่านได้ แยกว่าไม่มีตัวตรวจเลย เพราะมันผลิตความมั่นใจปลอมๆ ขึ้นมา

ตัว format checker ขึ้น ALL CHECKS PASS ทั้งที่ 345 ไฟล์สะกดคำ discipline ผิด

เกิดอะไรขึ้น: tools/check_format.py ขึ้น "ALL CHECKS PASS" ทั้งชุดข้อมูล ทั้งที่จริงๆ มี 345 ไฟล์ใช้คำว่า `architecture` แทนที่จะเป็น `architectural` ตามสเปก บวกอีก 16 ค่าที่นอกคำศัพท์ที่กำหนด รวมทั้งหมด 485 จุดผิดใน 423 ไฟล์ ครบ 49 หลัง ที่หลุดผ่าน validator ที่ขึ้นเขียวไปได้

ต้นเหตุจริง: สเปก schema ถูกแก้ 3 รอบในวันที่ 08-21 แต่ตัว validator ที่ใช้ตรวจตามสเปกไม่เคยถูกอัปเดตตามในคราวเดียวกัน สเปกกับตัวตรวจกลายเป็นแหล่งความจริงคนละที่ที่ไม่มีอะไรผูกกันไว้ ตัวตรวจเลยยังคงตรวจตาม contract เก่าที่ล้าสมัยไปแล้วโดยไม่รู้ตัว

เสียหายอะไร: ข้อมูล training 485 จุดผิด / 423 ไฟล์ / 49 หลัง ถูกส่งออกไปในสถานะ "ผ่านการตรวจแล้ว" ทุกตัวที่ปลายทางที่กรองด้วยคำ `discipline` จะทิ้งไฟล์เหล่านี้ไปเสียๆ หรือส่งไปผิดที่ ต้องมาไล่ซ่อมทั้งหมด 485 จุด ครอบคลุม 4,366 ไฟล์

แก้ยังไง: เพิ่ม gate ใหม่ 5 ตัวเข้า check_format.py (คำศัพท์ discipline, `source\_image`, key แปลกใน notes, roof-framing โครงสร้างต้องเป็น `plan`, gridmaster ที่ขาด 3 array) แล้วซ่อมทั้ง 485 จุดเป็นชุด ยืนยันว่าไฟล์ทั้ง 4,366 ไฟล์ parse ผ่านสะอาดหมด

กฎที่ได้: แก้ schema แล้วยังไม่จบ ต้องแก้ validator ที่ใช้ตรวจตามสเปกในคอมมิตเดียวกันด้วย อย่าเชื่อว่า "validator ผ่าน" มีความหมาย ถ้าชี้ไม่ได้ว่า gate ตัวไหนจะจับความผิดพลาดแบบที่กังวลอยู่ได้จริง

sanitize_elements() ลบทิ้งทุก element ที่มีค่า level ติดลบแบบเสียๆ ทำให้คานาคอดินหายไปจาก BOQ

เกิดอะไรขึ้น: ฟังก์ชัน `sanitize\_elements()` ใน `worker.py` ใช้ regex `\_NEG\_DIM\_KEYS = r"(\_mm|\_m|\_cm)\$"` มองว่าคีย์ไหนลงท้ายด้วยหน่วยวัดแล้วมีค่าติดลบคือ 'ขยะ' ทั้งที่ค่าติดลบเป็นค่าที่ถูกต้องสำหรับ `level\_m` (คานาคอดินที่อยู่ต่ำกว่าระดับอ้างอิง เขียนแบบไทยว่า "GB1(-0.50)"), `pos\_m` (เส้นกริดที่อยู่ก่อนจุดกำเนิด) และ `level\_step\_mm` (พื้นสำเร็จรูปแบบขั้นบันไดของห้องน้ำ) ผลคือ element ทั้งชุดถูกทิ้งก่อนจะไปถึง BOQ และข้อความเตือนกลับโทษว่าเป็นความผิดของโมเดล ('generation หลุด') ทั้งที่จริงๆ โมเดลตอบถูกต้องแล้ว ยืนยันด้วย GT จริง: บ้าน 07บ้าน_ใหญ่_2ชั้น_01 หน้า 35 element หายไป 3 จาก 11 ตัว

ต้นเหตุจริง: เป็น denylist แบบ regex ที่ตัดสินจาก 'ท้ายชื่อคีย์' แทนที่จะเป็น allowlist ของมิติที่รู้แน่ชัดแล้วว่าติดลบไม่ได้จริงๆ ตัวตรวจสอบเข้ารหัสข้อสันนิษฐานที่ผิด ('ค่าวัดไม่มีวันติดลบ') ซึ่งไม่จริงสำหรับฟิลต์ที่เป็นระดับความสูงหรือระยะ offset และความผิดพลาดนี้เสียและโทษผิดคน เพราะข้อความเตือนชี้ไปที่การ generate ของโมเดลแทนที่จะชี้ไปที่ตัวกรองเอง

เสียหายอะไร: คานาคอดินหายไปจาก BOQ ทั้งหมดในทุกบ้านที่โดนผลกระทบ — เป็นการหายไปของ element โครงสร้างจริงในงานถอดปริมาณก่อสร้าง และยังทำให้ทีมเสียเวลาไล่ตื้บกผิดทาง เพราะข้อความเตือนชี้ไปที่โมเดล

แก้ยังไง: เปลี่ยนจาก denylist ตามท้ายชื่อ มาเป็น allowlist ของมิติหน้าตัดที่ติดลบไม่ได้จริงๆ (`width|height|thickness|depth|dia|spacing|length|cover`) พร้อมเพิ่ม regression test ยืนยันด้วยการรัน regex เก่ากับใหม่เทียบกัน: `level\_m`/`level\_step\_mm`/`pos\_m` เปลี่ยนจากถูกทิ้งเป็นถูกเก็บไว้ ขณะที่ `width\_mm`/`span\_length\_m`/`dia\_mm`/`depth\_mm` ยังถูกทิ้งเหมือนเดิม

กฎที่ได้: ตรวจสอบข้อมูลด้วย allowlist ของฟิลต์ที่เรายืนยันแล้วว่าต้องมีจำกัดจริง อย่าใช้ denylist ที่จับคู่จากรูปแบบชื่อ และเมื่อตัวกรองทิ้งข้อมูล ข้อความเตือนต้องระบุตัวกรองว่าเป็นตัวการเสมอ — ห้ามโทษ component ต้นทางที่ยังไม่ได้ตรวจสอบให้แน่ใจ

11 หน้าถูกสกัดข้อมูลด้วยความมั่นใจต่ำแล้วปล่อยเข้า dataset โดยไม่มีใครตรวจสอบซ้ำ

เกิดอะไรขึ้น: มี 11 หน้าที่ถูกตั้งธงว่าต้องมีคนซูมเข้าไปดูภาพแล้วตรวจสอบซ้ำก่อนจะเอาไปใช้จริง ได้แก่ หน้า 19 view2 (ความหมายสัญลักษณ์ SI/SO/ST ยังไม่รู้), หน้า 21 (การแยกเหล็กบน/ล่างของ B5/B5X), 11, 14, 05, 06, 09, 35, 36 (สงสัยขนาดสายไฟไม่ตรงกันระหว่าง BTU 12,000 กับ 18,000) และ 51/53 (จับคู่แถว BOQ ไม่แน่ใจ)

ต้นเหตุจริง: การสกัดข้อมูลเดิมนั้นจบโดยไม่สนใจ confidence เลย ความมั่นใจต่ำถูกบันทึกเป็นแค่ metadata ไม่ได้ใช้เป็นด่านกัน บวกกับไม่มีเครื่องมือซูม ความไม่แน่ใจเลยกลายเป็นผลลัพธ์ปกติแทนที่จะเป็นข้อบกพร่อง

เสียหายอะไร: ประมาณ 1 ใน 5 ของหน้าในบ้านหลังนี้มีข้อมูลที่ทีมเองยังบอกว่ายังใช้เป็น ground truth ไม่ได้ — แต่ไฟล์พวกนี้ดันอยู่ในโฟลเดอร์เดียวกัน รูปแบบเดียวกันกับข้อมูลที่ตรวจสอบแล้ว จนถึงสิ้นวันถัดไปงานตรวจสอบซ้ำก็ยังค้างอยู่

แก้ยังไม: ยังไม่ได้แก้ — ถูกจัดเป็นรายการค้างทั้งวันที่ 2026-07-05 และ 2026-07-06

กฎที่ได้: ข้อมูลที่ความมั่นใจต่ำต้องแยกออกจากกันจริงๆ ทางกายภาพ (แยกโฟลเดอร์ก็กกัน หรือมีฟิลด์สถานะให้ consumer กรองได้) ไม่ใช่แค่ติดป้ายทิ้งไว้ อะไรที่ต่ำกว่าเกณฑ์ ห้ามให้อยู่โฟลเดอร์เดียวกับข้อมูลที่ผ่านการยอมรับแล้วเด็ดขาด

สามในสี่ dummy gridline ถูกเขียนลง grid master ด้วยความมั่นใจแค่ 0.5-0.6 และไม่เคยถูกตรวจสอบยืนยัน

เกิดอะไรขึ้น: ไฟล์ grid master ใหม่บันทึก dummy gridline ไว้ 4 เส้น มีแค่ 1' ที่ความมั่นใจสูง (0.85 จากที่เห็นเส้นยาวเต็มความสูงจริงบนหน้า 06) ส่วน 3', A' และ A'' ถูกใส่ด้วยความมั่นใจแค่ 0.5-0.6 โดยอนุมานจากโน้ต 'unnamed_extension' ของ Gen 2 บวกกับตำแหน่งของคาน B4X/B3X/B5X — ตัวเลขระยะที่พิมพ์เป็นของจริง แต่ไม่เคยยืนยันว่ามีคานจริงอยู่ที่ตำแหน่งเหล่านั้น

ต้นเหตุจริง: การอนุมานที่ความมั่นใจต่ำถูกเลื่อนชั้นเข้าไปในไฟล์ master ซึ่งเป็นไฟล์ที่ทุกหน้าอื่นต้องอ้างอิงถึง การเป็นไฟล์ master สื่อถึงความน่าเชื่อถือ ความไม่แน่ใจต้นทางจึงกลายเป็นความแน่นอนที่ปลายทางไปเสียๆ

เสียหายอะไร: กริดไลน์ 3 เส้นที่ยังไม่ยืนยันว่ามีอยู่จริง กลายเป็นข้อมูลอ้างอิงกริดมาตรฐานของทั้งบ้าน และหน้า 19 view2 ในรอบ t2 ก็ถูกสกัดโดยอ้างอิงถึงมันแล้ว ความผิดพลาดใดๆ จะกระจายไปยังทุกหน้าที่อ้างอิง master ตัวนี้

แก้ยังไม: ยังไม่ได้แก้ ตั้งธงว่าต้องเทียบกับแปลนคานหน้า 19/20 อีกครั้งเมื่อสกัดใหม่ — ค่าความมั่นใจถูกเก็บไว้ตามจริงในไฟล์ ไม่ได้ป้อนให้สูงเกินจริง

กฎที่ได้: ไฟล์ master/reference ต้องมีแต่รายการที่ยืนยันแล้วเท่านั้น เก็บสิ่งที่ยังไม่ยืนยันไว้ในรายการแยกต่างหากจนกว่าจะยืนยันได้จริง — พอไหนก็ตามที่การอนุมานถูกเขียนลงไฟล์ที่มีอำนาจอ้างอิง ค่า confidence ของมันก็จะไม่มีใครอ่านอีกเลย

สูง 2026-07-06

ทำงาน Label Studio เต็มวันแต่จบลงโดยไม่มีอะไรถูกนำเข้า เผยแพร่ หรือรีวิวได้จริงเลย

เกิดอะไรขึ้น: สคริปต์, XML config สามตัว และไฟล์ task JSON (33 elements tasks, material_list 1, single 22) ถูกสร้างและปั่นใหม่หลายรอบ แต่ไดอารี่บันทึกว่า 'ยังไม่ได้นำเข้า Label Studio Cloud จริง' — ไม่มีการสร้างโปรเจกต์ ไม่มีการเผยแพร่ ไม่มีการเชิญผู้ร่วมงาน สคริปต์ฝั่งรับ label-studio-import-*.js สำหรับแปลงผลส่งออกกลับเป็น annotated/*.json ก็ไม่มีอยู่เลยด้วยซ้ำ

ต้นเหตุจริง: งานถูกสร้างขยายออกจากตัวสร้าง task โดยไม่เคยทำให้ครบวงจรแบบบางๆ สักรอบเดียวก่อน แต่รอบแก้ทั้งห้ารอบของวันนั้นเกิดจากความล้มเหลวที่การลองครบวงจรตั้งแต่ต้นครั้งเดียวจะเผยให้เห็นได้ทันที

เสียหายอะไร: งานเครื่องมือทั้งวันได้ผลลัพธ์ที่ถูกรีวิวเป็นศูนย์ เส้นทางไป-กลับไม่ใช่แค่ยังไม่เสร็จ แต่ยังไม่เคยพิสูจน์ได้เลยว่าทำงานได้จริง — ทิศทางส่งออก→annotated ไม่มีโค้ดเลย จึงไม่รู้ด้วยซ้ำว่าจะเอาข้อมูลที่รีวิวแล้วกลับออกมาได้จริงหรือไม่

แก้ยังไง: ยังไม่ได้แก้ — ยกไปเป็นรายการค้าง #1, #8 และ #9 (นำเข้าแล้วยืนยัน, เขียนสคริปต์ฝั่งนำเข้า, สร้างและเผยแพร่โปรเจกต์จริง)

กฎที่ได้: สร้างเส้นทางครบวงจรแบบบางๆ สักหนึ่งเส้นทางก่อน — หนึ่ง task นำเข้า รีวิว ส่งออกกลับมาที่ดิสก์ — ก่อนจะสร้าง task เป็นสิบๆ อันหรือขีดเคลาหน้าตา จนกว่าจะพิสูจน์ได้ว่าเส้นทางไป-กลับใช้ได้จริง งานสร้างตัวสร้าง task ทั้งหมดจะยังไม่รู้ด้วยซ้ำว่าใช้งานได้จริงหรือเปล่า

สูง 2026-07-07

เข้าใจผิดว่า 'ส่วนที่ยื่นเลยกริดหลัก' เป็นเรื่องเดียว ทั้งที่จริงคือ 3 ระบบที่ไม่เกี่ยวกันเลย

เกิดอะไรขึ้น: ความสับสนเรื่องระยะที่ยื่นเลยกริดหลักคลี่คลายเมื่อพบว่า เป็น 3 ระบบที่แยกกันจริงๆ คือ ส่วนต่อขยายห้องชั้นล่าง 1.50ม. (dummy grid 3'/A' ของจริง), มุมคานคอดิน 0.60/1.10ม., และชายคาหลังคา 1.30ม. ส่วน confidence ของ dummy grid A'' ถูกปรับลดลง เพราะสงสัยว่าอาจไม่ใช่เส้นกริดเลย แต่เป็นพื้นที่ปรับระดับดิน

ต้นเหตุจริง: offset หลายจุดที่มีรูปแบบภาพคล้ายกัน คือ 'ระยะที่เลยเส้นกริดสุดท้าย' ถูกรวบยอดเป็นคำอธิบายเดียวกันหมด เพราะ grid master ไม่มีช่องบอกว่า offset แต่ละจุดคือกายภาพอะไรจริงๆ

เสียหายอะไร: grid master มีเส้นกริด (A'') ที่อาจเป็นเรื่องแต่งขึ้นเองปนอยู่ ซึ่งจะทำให้ตำแหน่ง element ทุกตัวที่อ้างอิงเส้นนี้ผิดเพี้ยนไปหมด และเกิดความสับสนเข้ามาหลายรอบก่อนหน้านี้

แก้ยังไง: อัปเดต grid master ให้แยกโมเดล 3 ระบบออกจากกัน เพิ่ม confidence ของ 1'/3'/A' และลด confidence ของ A''

กฎที่ได้: ห้ามรวบยอดเรขาคณิตที่หน้าตาคล้ายกันเป็นคำอธิบายเดียว — ให้บันทึกว่า offset แต่ละจุดคือกายภาพอะไรจริงๆ และลด confidence แทนการเก็บเส้นกริดที่ยังไม่ยืนยันไว้ที่ confidence เต็ม

สูง 2026-07-07

ประกาศว่าผลลัพธ์จาก generator ของ t3 'ตรวจสอบแล้ว' โดยเทียบ UI กับ task JSON ที่สร้างจากตัวมันเอง ซึ่งเป็นไฟล์ที่มีบั๊กอยู่แล้ว

เกิดอะไรขึ้น: หลังรัน label-studio-tasks-makham.js กับ t3 การตรวจสอบที่ทำได้คือ 'ทดสอบแล้วว่าหน้า 19 (footing_plan + beam_plan) ตรงกับที่เมฆามเห็นจริงใน Label Studio' ซึ่งผ่าน แต่วันถัดมาการตรวจแบบละเอียดกว่าพบว่า generator ไม่เคยถูกอัปเดตให้รองรับรูปแบบฟิลด์ของ Gen 4 เลย ทำให้ grid_ref และค่าหลักหลักของทุกคานว่างเปล่าในไฟล์เดียวกันนั้น — การตรวจสอบที่ผ่านคือการเทียบ UI กับไฟล์ที่ฟังแล้วเทียบกันเองพบว่าสอดคล้องกัน

ต้นเหตุจริง: วิธีการตรวจสอบเปรียบเทียบสองสิ่งที่อยู่ปลายทางเดียวกันของบั๊ก (JSON ที่สร้างขึ้น ↔ UI ที่แสดงผล) แทนที่จะเทียบปลายทางกลับไปต้นทาง (raw JSON ↔ UI ที่แสดงผล) ความสอดคล้องกันระหว่างสองสิ่งที่อยู่ downstream ไม่ได้พิสูจน์อะไรเรื่องความถูกต้องเลย

เสียหายอะไร: ความมั่นใจเกี่ยวกับ pipeline Label Studio ของ t3 ทั้งวันเป็นเรื่องเทีจ ถ้า task set ของ t3 ถูก import เข้า Label Studio Cloud ไปแล้ว annotator จะติดป้ายคานที่ไม่มี grid reference และไม่มีข้อมูลหลักเลย โดยไม่มีอะไรบนหน้าจอบอกว่าข้อมูลหายไป

แก้ยังง: แก้ววันถัดมา: สอน generator ให้รู้จักทั้งรูปแบบฟิลด์ใหม่ (grid_ref_start/end/span_length_m, main_bar.top/bottom) และแบบเก่า, เพิ่มกล่องใน XML config ให้ตรงกัน, แล้วสร้าง task JSON ใหม่ หลังจากนั้นเพิ่มการตรวจสอบสามทาง source_image / ไฟล์บนดิสก์ / URL ที่ถอดรหัสแล้ว

กฎที่ได้: ต้องตรวจสอบกับต้นทาง ไม่ใช่กับ artifact ตัวกลาง 'UI แสดงตรงกับที่ไฟล์บอก' ไม่ใช่การทดสอบ — เลือกค่าที่มองเห็นได้บนแบบต้นฉบับจริงแล้วไล่ตามมันจนถึงหน้าจอ

สูง 2026-07-16

เอาระยะสองช่วงที่ติดกันมาบวกกันเป็นชายคายื่น 1.50 ม. ที่ไม่มีอยู่จริง

เกิดอะไรขึ้น: ในหน้า page06 ของบ้านหลัง 2 เลข '0.60' กับ '0.90' ในแถวเส้นบอกระยะของแนว F ถูกอ่านเป็นชายคายื่นยาวต่อเนื่อง 1.50 ม. พอซูมเข้าไปดูถึงเห็นว่าเป็นคนละจุดกัน (0.60 = ด้านนอก, 0.90 = พนักกันด้านใน) แต่มีจุดรวมกันตรงเส้นกริด F เท่านั้น

ต้นเหตุจริง: เอาตัวเลขที่อยู่ติดกันในเส้นบอกระยะมาบวกกันโดยไม่เช็คว่าแต่ละตัววัดไปจบที่อะไร รูปแบบการอ่านผิดแบบเดียวกันนี้เคยทำให้ได้ค่า '3' ผิดในบ้านหลัง 1 มาแล้ว

เสียหายอะไร: เกิดแถบพื้นที่ 1.50 ม. ที่ถูกสร้างขึ้นมาลอยๆ ในผังพื้น ทำให้ต้องกำหนดขอบเขตห้องซักผ้า ห้องครัว การขอยห้องนอน 1 และห้องน้ำ 1 ใหม่หมด แถมย่อน้ำคำเตือนทั้งย่อน้ำที่สรุปว่า "ไม่ต้องมี dummy grid" ก็ต้องเขียนใหม่เพราะผิด

แก้ยังง: แยกออกเป็น 'F-F' x 1-2 (ห้องซักผ้า) กับ 'F'-E x 1-2 (ห้องครัว), ตรงจุดแบ่งห้องนอน 1 ไว้ที่ 3' (6.4+1.30=7.7 พอตีเป๊ะ) แล้วเขียนย่อน้ำคำเตือนใหม่

กฎที่ได้: ห้ามเอาตัวเลขที่อยู่ติดกันในเส้นบอกระยะมาบวกกันโดยไม่ยืนยันก่อนว่าแต่ละตัวไปจบที่อะไร จุดปลายที่ใช้ร่วมกันทำให้ระยะสองค่าดูเหมือนต่อเนื่องกันทั้งที่วัดคนละของ

สูง 2026-07-20

เอาสเกลที่คาลิเบรตจากหน้า 31/32 มาใช้กับหน้า 33 โดยไม่เช็คจุดอ้างอิงของหน้า 33 เอง

เกิดอะไรขึ้น: ข้อสรุปที่ว่าแถบ D-C ในหน้า 33 ไม่มีส่วนต่อขยาย 1'/'3' มาจากการเอาค่าคาลิเบรตของหน้า 31/32 มาใช้ซ้ำ ผู้ใช้แก้ไขให้ตรงๆ ว่าหน้า 33 มีส่วนต่อขยาย 1'/'3' จริง และต้องเพิ่มคานสั้นอีก 4 ตัว (D1'-D1, C1'-C1, D3-D3', C3-C3')

ต้นเหตุจริง: การคาลิเบรตเป็นเรื่องเฉพาะของแต่ละแผ่นแบบ เพราะขึ้นกับจุดอ้างอิงและสเกลของแผ่นนั่นเอง แต่กลับถูกยกไปใช้ข้ามแผ่นเหมือนเป็นค่าคงที่ของทั้งโปรเจกต์

เสียหายอะไร: มีข้อสรุปเชิงลบที่ผิดถูกเขียนลงไปใน schema_note และ warnings ของไฟล์ คานหายไป 4 ตัว และต้องให้ผู้ใช้เป็นคนจับได้เอง

แก้ยังไง: ตรวจหน้า 33 ใหม่โดยเทียบกับจุดอ้างอิงของหน้านั้นเอง เพิ่มคานทั้ง 4 ตัวด้วย confidence 1 และแก้ schema_note/warnings ที่เคยระบุตรงกันข้าม

กฎที่ได้: หาค่าสเกลพิกเซลและจุดอ้างอิงใหม่ทุกแผ่นแบบ ห้ามยกค่าคาลิเบรตข้ามแผ่นเด็ดขาด — และเวลาข้อสรุปถูกกลับด้าน ต้องแก้ note กับ warnings ที่เคยยืนยันข้อสรุปนั้นด้วย ไม่ใช่แค่แก้ตัวข้อมูล

สูง 2026-07-24

ไม่เคยอ่านสคริปต์เรนเดอร์หัดต่อรหัดเทียบกับตัวเลขจริงของชุดข้อมูล

เกิดอะไรขึ้น: บั๊ก MAX_LENGTH มีอยู่ตั้งแต่ก่อนเพิ่มบ้านหลังที่ 05 เข้ามา และรอดผ่านทุกรอบก่อนหน้า เพราะไม่เคยมีใครเทียบค่าคงที่ในสคริปต์กับคุณสมบัติจริงที่วัดได้ของชุดข้อมูลเลย

ต้นเหตุจริง: การรีวิวนสคริปต์เป็นแค่การไล่หาข้อผิดพลาดที่เห็นชัดๆ ไม่ใช่การเทียบข้อจำกัดที่ตั้งไว้แต่ละตัวกับสถิติที่คำนวณจริงจากข้อมูลที่จะประมวลผล

เสียหายอะไร: บั๊กที่ทำให้คำตอบถูกตัดทอนไปแฝงอยู่และผ่านการเรนเดอร์ที่เสียเงินไปอย่างน้อยหนึ่งรอบโดยไม่มีใครรู้

แก้ยังไง: กำหนดให้ Phase 0.4 เป็นขั้นตอนบังคับ: อ่านสคริปต์อย่างจริงจังเทียบกับตัวเลขจริงของชุดข้อมูลก่อนเข้าเครื่องทุกครั้ง

กฎที่ได้: รีวิว config ด้วยการคำนวณสถิติจริงจากข้อมูลจริงมาเทียบกัน ไม่ใช่แค่อ่านตัวเลขแล้วรู้สึกเอาเองว่า 'ดูสมเหตุสมผล'

สูง 2026-07-28

บ้าน07 S-04: ตำแหน่งถูกกะด้วยสายตาว่า ~2.00 ม. ทั้งที่แบบพิมพ์ไว้ชัด 1.50 ม. และวัดระยะจากแถวกริดผิดแถว

เกิดอะไรขึ้น: ระยะจาก CS2 ตัวที่สองถึงกริด 3 ถูกบันทึกว่าประมาณ ~2.00 ม. แต่แบบพิมพ์ไว้ 1.50 ม. อีกจุดคือระยะ 1.60 ม. มุมล่างขวาถูกวัดจากแถว B ทั้งที่จริงต้องวัดจากแถว A — พอแก้แล้วมันตรงกับ GB1X ใน S-02 พอดี

ต้นเหตุจริง: ค่ากะด้วยสายตาถูกบันทึกลงในช่องที่อ่านเหมือนเป็นค่าวัดจริง โดยไม่มีเครื่องหมายแยกสองอย่างนี้ออกจากกัน แถวอ้างอิงของระยะถูกสันนิษฐานเอาเองแทนที่จะไล่ตามเส้นต่อขยายจริง

เสียหายอะไร: ตำแหน่งคลาดเคลื่อน 0.50 ม. (ผิด 33%) บวกกับแถวอ้างอิงผิด ทั้งสองอย่างไหลเข้าไปในตำแหน่ง element และการอ้างอิงกริด ความผิดพลาดเรื่องแถวอ้างอิงนี้เองที่ทำให้ S-02 กับ S-04 ดูเหมือนขัดแย้งกัน

แก้ยังไง: ล็อกตำแหน่งใหม่ตามชุดระยะที่พิมพ์บนแผ่นจริง แถวอ้างอิงที่แก้แล้วทำให้ S-04 ตรงกับ S-02

กฎที่ได้: ห้ามเขียนค่าประมาณลงในช่องที่เป็นค่าวัด — ถ้าไม่มีชุดระยะพิมพ์ไว้จริง ต้องทำเครื่องหมายว่ายังไม่สามารถระบุได้ และต้องไล่ระยะกลับไปหาเส้นต่อขยายเสมอเพื่อยืนยันว่าเส้นกริดไหนคือแถวอ้างอิง

'GGUF ผ่าน' ของ Phase 10 เช็คว่ามันออก JSON ได้และไม่พังเท่านั้น

เกิดอะไรขึ้น: การทดสอบรับผล Phase 10 ของ t01 เมื่อวันที่ 25 ก.ค. เช็แค่ (a) ตอบเป็น JSON ใหม่ — ผ่าน (b) แงค์หรือพังใหม่ — ผ่าน ไม่เคยเช็คเลยว่าเนื้อหาที่ตอบถูกต้องหรือเปล่า เอกสารเองก็เตือนไว้ว่า 'อย่าอ่านว่านี่คือการพิสูจน์ว่าเสร็จสมบูรณ์' แต่พอเวลาผ่านไป ทุกคนจำได้แค่ 'Phase 10 ผ่านแล้ว'

ต้นเหตุจริง: มีการทดสอบแค่ว่าโมเดลยังทำงานอยู่ (liveness) แต่ไม่ได้ทดสอบความถูกต้อง (correctness) และคำเตือนที่แนบไว้อยู่ในรูปข้อความ ไม่ได้อยู่ในผลตัดสิน คนจึงจำแต่ผลตัดสินว่า 'ผ่าน' ส่วนคำเตือนหายไปจากความทรงจำ

เสียหายอะไร: การทดสอบที่รู้อยู่แล้วว่ายังไม่สมบูรณ์ กลายเป็นความเชื่อฝังหัวของทีมว่า GGUF ผ่านการตรวจสอบแล้ว ทำให้กว่าจะเจอบั๊กเรื่อง merge ต้องรอไปอีกหลายวัน

แก้ยังง: ยังไม่ได้แก้ — เกณฑ์ของ Phase 10 ยังต้องเพิ่มการตรวจสอบความถูกต้องของเนื้อหาเข้าไปด้วย

กฎที่ได้: การทดสอบที่พิสูจน์แค่ 'มันรันได้' ต้องเรียกให้ตรงตามนั้น (เช่น 'smoke test') ห้ามบันทึกว่าเป็นผลผ่านเด็ดขาด คำเตือนที่แนบมากับผลตัดสินต้องเข้ารหัสไว้ในผลตัดสินเองเลย (เช่น FAIL หรือ PARTIAL) เพราะข้อความเตือนแบบร้อยแก้วจะเลื่อนหายไปตามเวลา แต่ผลตัดสินจะไม่เลื่อน

'Phase 10 GGUF verified' แต่เช็คว่าผลิต JSON ออกมาได้และไม่พัง — ไม่เคยเช็คเนื้อหาว่าถูกใหม่

เกิดอะไรขึ้น: การทดสอบ Phase 10 GGUF วันที่ 25 ก.ค. ผ่านด้วยเกณฑ์แค่สองข้อ คือผลลัพธ์เป็น JSON และไม่ค้าง/ไม่พัง ไม่เคยทดสอบว่าเนื้อหาถูกต้องหรือไม่ ตัวเอกสารเองเขียนคำเตือนไว้ว่า 'อย่าอ่านว่าพิสูจน์แล้วว่าเสร็จสมบูรณ์' แต่เมื่อเวลาผ่านไปทุกคนจำได้แค่ 'Phase 10 ผ่านแล้ว'

ต้นเหตุจริง: เกณฑ์การยอมรับวัดแค่ว่าโมเดลยังทำงานอยู่ ไม่ได้วัดความถูกต้อง และคำเตือนที่เขียนไว้ก็เสื่อมสลายไปเพราะมันอยู่แค่ในข้อความบรรยายข้างเครื่องหมายถูกสีเขียว ไม่ได้อยู่ในเกณฑ์ผ่าน/ไม่ผ่านเอง

เสียหายอะไร: artifact ที่พังถูกติดป้าย 'verified' อยู่นานถึง 4 วัน และถูกใช้ทำงาน extraction จริงกับข้อมูลบ้านหลังที่ 01 ได้ผลลัพธ์ที่เสีย

แก้ยังง: ปรับกรอบใหม่สำหรับรอบต่อไป: การตรวจความถูกต้องของเนื้อหาต้องเป็นส่วนหนึ่งของเกณฑ์ยอมรับ GGUF โดยใช้การเทียบกับคีย์คำตอบจริง (ground truth)

กฎที่ได้: เกณฑ์การยอมรับต้องวัดความสามารถที่ต้องการจริงๆ ไม่ใช่แค่ว่ากระบวนการรันผ่านไปได้ และห้ามฝากคำเตือนไว้แค่ในข้อความข้างผลผ่านสีเขียวเด็ดขาด — ต้องฝังมันไว้ในเงื่อนไขผ่านเอง ไม่งั้นมันจะถูกกลืน

ประกาศ 'ผ่านทุกด้าน' ทั้ง 11 บ้าน แล้วการตรวจอิสระเจอ id หายไป ~500 ตัว, ฟอรัมแตกเป็น 5 แบบ และเนื้อหาถอยหลังซ่อนอยู่ข้างล่าง

เกิดอะไรขึ้น: รอบ 4 ปิดงานด้วย check_format.py รายงาน ALL CHECKS PASS ครบทั้ง 11 บ้าน และไฟล์ในโฟลเดอร์ทำงานกับโฟลเดอร์ raw ต่างกัน 0 ไฟล์ พอรอบ 5 ทำการตรวจอิสระที่ใจไม่สนตัวเช็คตัวนั้น แล้วเทียบชนิดข้อมูล JSON ของทุก key ใน 2,366 ไฟล์แทน — เจอ element ~500 ตัวไม่มี element_id, ไฟล์ BOQ ของบ้าน 04 อยู่ผิด pattern, ตารางราคาแบบ series ถูกเก็บไว้ 5 แบบต่างกัน, หน้าเครดิตแตกเป็น 4 pattern, ค่าที่ควรเป็นตัวเลขกลับเป็น string หรือ array และเจอตัวเลขราคาของบ้าน 05 ที่ถอยหลังไปจากของเดิมจริงๆ ผลคือตัวเช็คขยายจาก 14 เป็น 19 ข้อ

ต้นเหตุจริง: ตัวเช็คบังคับได้แค่กฎที่มีคนเขียนไว้แล้วเท่านั้น การเห็นไฟเขียวถูกอ่านว่า "ข้อมูลถูกต้อง" ทั้งที่จริงมันแปลได้แค่ "กฎ 14 ข้อที่รู้จักผ่านหมด" ส่วนความถูกต้องของชนิดข้อมูลและ id ไม่ได้อยู่ใน 14 ข้อนั้นเลย

เสียหายอะไร: ปิดงานทั้งเซสชันด้วยความเชื่อว่าชุดข้อมูลสะอาดและล็อกแล้ว ทุกอย่างในรอบ 5 เจอจะไหลเข้าไปเป็น label สำหรับเทรนโดยไม่มีใครรู้ รวมถึงตัวเลขราคาที่ผิด ซึ่งเคยมีบันทึกไว้ว่าแก้ไปแล้วด้วยซ้ำ

แก้ยังไม: รันการตรวจแบบเทียบชนิดข้อมูลที่แยกจากตัวเช็คเดิม แล้วเอาสิ่งที่เจอกลับมาใส่เป็นกฎใหม่ 5 ข้อ (14 → 19)

กฎที่ได้: ตัวเช็คที่ผ่านคือหลักฐานเรื่องตัวเช็ค ไม่ใช่หลักฐานเรื่องข้อมูล ทุกครั้งที่จะเคลมว่า "ล็อกและตรวจสอบแล้ว" ต้องมีการตรวจอย่างน้อยหนึ่งครั้งที่ใช้วิธีคนละแบบกับเครื่องมือที่ให้คำเคลมนั้นมา

ตัดสินใจ **site_plan/side_profile** ออกจากแหล่งข้อมูลเส้นกริดผิด ด้วยเหตุผลว่ารูปด้านไม่มีแกนกริด — และเดาผิดแบบเดียวกันซ้ำสองครั้ง

เกิดอะไรขึ้น: การออกแบบชั้น t03 ตอนแรกตัด site_plan และ side_profile ออกจากงานย่อยเรื่องเส้นกริด ด้วยเหตุผลว่ารูปด้านไม่มีแกนกริดแบบมองจากบน มะขามเอาแบบรูปด้านจริง (A-05, รูปด้าน 1/2) มาให้ดู พบว่ามันพิมพ์เครื่องหมายกริด 1/2/3 และ D/C/B/A พร้อมสายมิติครบชุด เหมือนแบบแปลนเป๊ะ จึงต้องยกเลิกการตัดออกนั้น ต่อมาในเซสชันเดียวกัน บันทึกกระบวนวิธีคือครั้งที่สองแล้วที่การสันนิษฐานว่า side_profile/รูปด้านมีคุณค่าต่ำนั้นผิด — มันคือทีเดียวที่พิมพ์ระดับความสูง (z-level) เอาไว้เลย

ต้นเหตุจริง: การตัดสินใจออกแบบถูกทำจากภาพในหัวว่าแบบรูปด้านมีอะไรอยู่บ้าง แทนที่จะเปิดแบบจริงในชุดข้อมูลมาดูก่อน แล้วการสันนิษฐานผิดแบบเดียวกันก็เกิดซ้ำอีกในรูปแบบอื่นภายในเซสชันเดียวกัน

เสียหายอะไร: ถ้าไม่ถูกจับได้ จะตัดแหล่งข้อมูลเส้นกริดจริงออกจากงานย่อย gridline ของ t03 อย่างถาวร และ (ครั้งที่สอง) จะทำให้แกนแนวตั้งทั้งแกนสกดข้อมูลไม่ได้เลย ทั้งสองครั้งมะขามเป็นคนจับได้ ไม่ใช่ตัวตรวจสอบใดๆ

แก้ยังไม: ยกเลิกการตัดออกนั้น และภายหลังเพิ่ม z_levels[], dimension_chains[] สำหรับแกน z และ unassigned_dimensions[] เข้า schema §4

กฎที่ได้: ห้ามตัดรูปแบบแบบใดๆ ออกจากแหล่งข้อมูลโดยอาศัยแค่ข้อสันนิษฐานว่ามันมีอะไรอยู่ ต้องเปิดแบบจริงสักสามแผ่นของรูปแบบนั้นก่อนเสมอ และเมื่อการสันนิษฐานแบบเดียวกันถูกพิสูจน์ว่าผิดสองครั้ง ให้ปฏิบัติเป็นกฎ ('รูปด้านมีข้อมูลที่แปลนไม่มี') ไม่ใช่แค่แก้ไขทีละครั้ง

สูง 2026-08-22

ตอบคำถามเรื่องความสามารถของโมเดลจากความจำ แล้วตอบผิด

เกิดอะไรขึ้น: มะขามถามว่า Qwen ออกตำแหน่งพิกเซลของคานมาเพื่อเอาไปเช็คไขว้กับ grid master ได้ไหม คำตอบแรกคือ ไม่ ค่ะ โดยอ้างเอกสารงานวิจัยใน repo (FLOORPLANVLM.md) ที่บอกว่าต้องใช้ตัวอย่างที่จัดพิกเซลตรงกัน 300K ตัว บวก RL มะขามยืนยันให้ไปหาข้อมูลจากเว็บจริง ๆ แล้วพบว่า Qwen3-VL มี 2D grounding ในตัวอยู่แล้ว (bbox ในสเกล normalize 0-1000) ตั้งแต่ base model ไม่ต้อง fine-tune ขนาดนั้นเลย

ต้นเหตุจริง: เอาเอกสารวิจัยใน repo ฉบับเดียวมาถือว่าเป็นสถานะล่าสุดของวงการ ทั้งที่มันเทียบคนละตระกูลโมเดล และไม่เคยเอาไปเช็คกับเอกสารของผู้ผลิตอีกเลย ข้อมูลความสามารถของ VLM ซึ่งเปลี่ยนเร็วมากจะเก่าภายในไม่กี่สัปดาห์

เสียหายอะไร: แนวทาง QA ที่ใช้ได้จริง (เช็คไขว้ bbox กับ gridmaster) เกือบถูกปิดตายไปทั้งต้น คำตอบที่ถูกโผล่มาได้เพราะผู้ยืนยันกรานให้ไปหาข้อมูลสดเท่านั้น

แก้ยังไ: ไปค้นเว็บจริง เจอทั้ง native grounding และแพตเทิร์น GroundCount แบบ detection-แล้วค่อย-VLM แล้วเขียนข้อค้นพบที่แก้ไขแล้วลง tune_ai/t03/research_beam_grounding_vs_gridmaster.md

กฎที่ได้: ห้ามตอบคำถามแนว โมเดล X ทำ Y ได้ไหม จากความจำ หรือจากเอกสารใน repo ฉบับเดียวที่แคชไว้ ให้ไปเปิด model card หรือเอกสารปัจจุบันของผู้ผลิตสดๆ ก่อนจะฟันธงว่าทำไม่ได้

สูง 2026-08-25

เขียน MAX_PIXELS 7,680 + MAX_LENGTH 47,104 ลงสคริปต์เทรน ก่อนรันเช็ค VRAM 5 step ที่บังคับไว้

เกิดอะไรขึ้น: แก่ทั้งสองค่าใน train_t03.py แล้วบันทึกว่า 'ทำจริงแล้ว' ซึ่งทำให้ VRAM วางลดจาก 7.5GB เหลือ 4.0GB — ทั้งที่ไดอารี่เองบันทึกกฎไว้ว่าต้องรันสั้น 5 step ยืนยัน config ก่อนรันเต็มทุกครั้ง เพราะรอบก่อน OOM ตอนเหลือ 470MB

ต้นเหตุจริง: แก่ค่าโดยอิงจากการประมาณ VRAM สูงสุดบนสเปกชิต (90.9GB) ไม่ได้อิงจากการรันวัดจริง แกรมการประมาณนั้นมาจากวิธีคิดชุดเดียวกับที่เคยประมาณความจุรูปผิดมาแล้วครั้งหนึ่ง

เสียหายอะไร: รอบเทรนเต็มรอบหน้าอาจ OOM ตอนรันไปแล้วหลายชั่วโมง แล้วเผาค่า GPU ทั้งรอบทั้ง ในจังหวะที่เหลือเครดิตแค่ \$8.91

แก้ยังไ: ยังไม่ได้แก้ — ติดธง ⚠ ไว้ในไดอารี่ว่า 'ต้องรันทดสอบสั้น 5 step ยืนยันก่อนรันเต็มทุกครั้ง' แต่ยังไม่ได้รับ

กฎที่ได้: การแก้ config ยังไม่นับว่า 'เสร็จ' จนกว่า smoke test ที่ใช้ยืนยันมันจะรันแล้ว การเขียนค่ากับการพิสูจน์ค่าเป็นสองงานคนละงาน มีแต่งงานที่สองเท่านั้นที่กินเงินไม่ให้ไหม้

สูง 2026-08-25

มีแค่ฐานรากที่วัดเทียบกับจำนวนจริง คานกับเสาใช้แค่คาด แต่สรุปว่าใช้ได้ทั้งสามอย่าง

เกิดอะไรขึ้น: ตารางผลให้ฐานราก 14/14 และ 12/12 เทียบกับจำนวนจริง แต่คานรายงานเป็นจำนวนดิบ ('17 แถบ', '19 แถบ') ไม่มีตัวหาร มีแค่หมายเหตุว่า 'คลุมโครงคานเกือบครบ' ส่วนเสารายงาน 14 กับ 13 ก็ไม่มีค่าจริงเทียบ (ที่หลังพบว่าเลข 13 มีตัวซ้ำอยู่) แต่สรุปประจำวันกลับเขียนว่าเครื่องมือ 'มาร์ค เสา/คาน/ฐานราก ได้จริง'

ต้นเหตุจริง: ฐานรากเป็นไอคอนแยกชิ้นนับได้ง่าย เลยหา ground truth ได้ ส่วนคานเป็นเส้นยาวไม่เท่ากันและไม่มีหน่วยที่ชัดเจน จึงไม่ได้ตั้ง ground truth แล้วเอาการมองด้วยตามาแทนช่องว่างนั้น

เสียหายอะไร: ความแม่นยำของการตรวจจับคานไม่มีใครรู้ ซึ่งสำคัญที่สุด เพราะคานคือ subtask (plan_beam 0%) ที่เป็นเหตุผลของการสร้างเครื่องมือนี้ทั้งตัว เครื่องมือที่ไม่รู้ recall ใช้เป็นตัวเช็คว่า 'AI พลาดอะไรไปไหม' ตามที่เสนอในหัวข้อ 6.2(ข) ไม่ได้

แก้ยังไ: ยังไม่ได้แก้ — ไม่มีการนับจำนวนคานจริงเป็น ground truth

กฎที่ได้: subtask ที่เป็นเหตุผลของการสร้างเครื่องมือ คือตัวที่ต้องมีตัวเลข ไม่ใช่ตัวที่นับง่าย คำว่า 'เกือบครบ' ไม่ใช่การวัด

สูง 2026-08-25

การแก้ roof_plan→plan วันที่ 08-24 ทำไม่ครบ เหลืออีก 7 ไฟล์ที่ยังซ่อนบั๊กคานหลังคาหายจาก BOQ

เกิดอะไรขึ้น: การแก้เมื่อวันก่อนที่เปลี่ยนแท็ก roof_plan เป็น plan (ซึ่งปิดบั๊กที่คานหลังคาหายจาก BOQ) ครอบคลุมแค่บางส่วนของ dataset ยังเหลืออีก 7 ไฟล์ที่ยังติด pattern ผิดอยู่ ต้องมาแก้ซ้ำอีกรอบในวันถัดมา

ต้นเหตุจริง: รอบวันที่ 08-24 แก้เฉพาะ 11 บ้านที่ทำงานอยู่วันนั้น ไม่ได้กวาดทั้ง dataset หาข้อบกพร่องประเภทเดียวกัน บั๊กเดิมจึงยังอยู่ในบ้านที่ไม่ได้อยู่ในขอบเขตวันนั้น

เสียหายอะไร: คานหลังคาหายจาก BOQ ของ 7 ไฟล์นั้นแบบเสียบๆ อีกหนึ่งวัน และต้องทำงานเดิมซ้ำในรอบกวาดครั้งที่สองที่ใหญ่กว่า

แก้ยังไง: เปลี่ยนแท็กอีก 7 ไฟล์ที่เหลือจาก roof_plan เป็น plan ภายในรอบ normalization 485 จุด

กฎที่ได้: พอเจอข้อบกพร่องประเภทหนึ่ง ให้ grep ทั้ง dataset แล้วแก้ทุกจุดในรอบเดียว การแก้แค่ไฟล์ที่อยู่ตรงหน้าการันตีว่าบั๊กเดิมจะกลับมาตอนเปิดไฟล์ถัดไป

สูง 2026-08-25

template รูป '+' ขนาด 22px ตัวเดียวทำให้ฐานราก (footing) ผิขึ้นมา 636 จุดในหน้าที่ไม่มีฐานรากเลย

เกิดอะไรขึ้น: template ที่เก็บมาจากบ้าน 11 เป็นรูปกากบาท '+' ขนาด 22px มันไปแมตช์กับทุกจุดตัดเส้นในแบบแปลน ทำให้ยอดตรวจจับฐานรากพุ่งไป 636 จุดในหน้าที่ไม่มีฐานรากอยู่เลยแม้แต่จุดเดียว

ต้นเหตุจริง: ขั้นตอนการเก็บ template ยอมรับ contour ขนาดเท่าไอคอนที่ซ้ำกันโดยไม่มีการกรอง ทั้งที่กากบาทเป็นองค์ประกอบพื้นฐานของแบบก่อสร้างทั่วไป ไม่ใช่สัญลักษณ์เฉพาะเจาะจง ไม่มีการกำหนดขนาดขั้นต่ำหรือความเฉพาะตัวเป็นเงื่อนไขก่อนรับ template เข้าฐานข้อมูล

เสียหายอะไร: ทำให้ยอดนับของหน้าที่เกี่ยวข้องผิดเพี้ยนไปหมด และถ้าไม่จับได้ก็จะทำให้ทุกการวัดที่สร้างจากฐาน template นี้เสียหายไปด้วย

แก้ยังไง: ลบ template ตัวนั้นทิ้ง แล้วเพิ่มด่านกรองใน 'load_templates': ซ้ำม template ที่เล็กกว่า 28px พร้อมแจ้งเตือน

กฎที่ได้: template ที่เล็กกว่าไม่กี่สิบพิกเซลมักเป็นองค์ประกอบพื้นฐานของแบบ ไม่ใช่สัญลักษณ์จริง ต้องกำหนดขนาดขั้นต่ำและความเฉพาะตัวก่อนรับ template เข้าฐาน และต้องทดสอบ template ใหม่ทุกตัวกับหน้าที่ควรได้คะแนนศูนย์เสมอ

template ที่ครอบแน่นเกินไป 2 ตัวให้คะแนนขย่ะสูงพอกๆ กับของจริง ทำให้กำหนดเกณฑ์ **threshold** ไม่ได้เลย

เกิดอะไรขึ้น: template ทึบขนาด 31px จากบ้าน 05/33 ให้คะแนน 0.87–1.00 กับของจริงในบ้านของตัวเอง แต่ให้คะแนนขย่ะในบ้านอื่นสูงถึง 0.90 — ช่วงคะแนนทับซ้อนกัน จึงไม่มี threshold ไหนแยกออกได้เลย ตารางคะแนนแยกตาม template แสดงว่า 2 template นี้ตัวเดียวสร้างขย่ะถึง 224 จุด ในขณะที่อีก 9 ตัวไม่มีขย่ะเลยสักจุด

ต้นเหตุจริง: template ถูกครอบแน่นชิดตัวไอคอนเกินไป ทั้งพื้นที่ว่างรอบๆ ออกไป ไอคอนจริงจะลอยอยู่คนเดียวนบนพื้นขาว แต่ตัวอักษรไทยจะอยู่ชิดติดกันเป็นกลุ่ม — บริบทรอบๆ นี่แหละคือตัวแยกความต่างที่แท้จริง พอครอบทิ้งไปก็เท่ากับทิ้งสัญญาณเดียวที่แยกสองอย่างนี้ออกจากกันได้

เสียหายอะไร: เกิดการตรวจจับผิดพลาด 224 จุดทั่วชุดข้อมูล และ threshold ก็แก้ไม่ได้จนกว่าจะไปแก้ที่ต้นเหตุจริง

แก้ยังง: วินิจฉัยด้วยตารางคะแนนแยกตาม template ก่อนแก้อะไรทั้งนั้น จากนั้นแก้ที่ต้นตอ: ครอบใหม่ให้มีขอบขาวรอบไอคอน (68px แทน 31px) ลบตัวที่ซ้ำออก (05/33 คือสัญลักษณ์เดียวกัน คะแนนเท่ากันเป๊ะ) และปรับ FOOTING_THRESH จาก 0.60 เป็น 0.72

กฎที่ได้: เมื่อ threshold แยกสัญญาณจริงกับขย่ะไม่ได้ ตัว template นั้นแหละที่ผิด อย่าไปปรับ threshold วินิจฉัยเป็นรายตัว template ก่อนแต่ค่าคงที่ระดับรวม และครอบ template ให้เก็บบริบทรอบๆ ไว้ด้วย เพราะบริบทมักเป็นตัวแยกความต่างที่แท้จริง

เครดิต vast.ai ที่มีอยู่ (\$8.91) ไม่พอแม้แต่จะรัน H200 ให้จบรอบเดียว (\$17.45)

เกิดอะไรขึ้น: เส้นทาง training ความละเอียดเต็ม (MAX_PIXELS 16,384) ต้องใช้ H200 ที่ราคา \$3.293/ชม. $\approx$ \$17.45 ต่อการรัน 3 epoch แต่ปัญหามีเครดิตแค่ \$8.91 ถ้าเริ่มรันด้วยยอดนี้ งานจะตายกลางคันแน่นอน

ต้นเหตุจริง: ไม่เคยเช็คยอดเครดิตคงเหลือก่อนจะเสนอเปลี่ยน config ที่ต้องใช้การ์ดจอแพงขึ้น

เสียหายอะไร: เส้นทาง 16,384 ถูกบล็อกสนิทจนกว่าจะมีเงินเต็มเข้าไป ถ้าไม่เช็คก่อน จะมีการเช่าและรันจริงแล้วต้องถูกฆ่างานกลางคัน เสียทั้งเครดิตที่ใช้ไปแล้วเวลา

แก้ยังง: บอกตรงๆ กับมะขามหันทีว่าเป็นอุปสรรคที่ต้องแก้ (ตามบทเรียน Day of Shame ข้อ 3) แทนที่จะเก็บไว้เป็นแค่รายการ to-do — ไม่มีการเช่าหรือใช้เงินใดๆ ในเชสชันนี้

กฎที่ได้: เช็คยอดเงินคงเหลือเทียบกับต้นทุนเต็มของการรันที่วางแผนไว้ ก่อนเสนอ config ที่ต้องใช้มัน และพูดตรงๆ เรื่องอุปสรรคด้านงบประมาณ อย่าซ่อนไว้ในรายการ to-do

ปรับ MAX_PIXELS/MAX_LENGTH ขึ้นจากการคำนวณด้วยมือ โดยยังไม่ได้ยืนยันยืนยันบังคับด้วย TEST_STEPS=5

เกิดอะไรขึ้น: ปรับ MAX_PIXELS จาก 5120→7680 และ MAX_LENGTH จาก 32768→47104 ทำให้ margin VRAM แคลลง จาก 7.5 GB เหลือ 4.0 GB ตัวเลข peak 90.9 GB เป็นแค่การประมาณด้วยการคำนวณ ไม่ใช่ค่าที่วัดได้จริง และยังไม่ได้ยืนยันยืนยันบังคับด้วย TEST_STEPS=5

ต้นเหตุจริง: ขั้นตอนยืนยันต้องเข้า GPU ซึ่งไม่ได้ทำในเซสชันนี้ ทำให้การเปลี่ยน config ถูกส่งออกไปก่อนที่จะผ่านด้านตรวจของตัวเอง

เสียหายอะไร: ถ้าค่าประมาณผิด การรันเต็มรอบครั้งถัดไปจะ OOM (หน่วยความจำไม่พอ) — รอบก่อนหน้านี้เคย OOM ทั้งที่เหลือพื้นที่ว่างแค่ 470 MiB ดังนั้น margin ที่ประมาณไว้ 4.0 GB ก็ไม่ได้สบายใจเท่าไร การรันเต็มรอบที่ล้มเหลวจะเสียค่าเช่าทั้งรอบ

แก้ยังไง: บันทึกไว้เป็นด้านบังคับก่อนรัน (rule_of_tune ข้อ 4) พร้อมลำดับสำรองที่ชัดเจน: ลดจำนวนภาพเส้นกริดจาก 4 เหลือ 3 ก่อน (peak 89.2) แล้วค่อยลด MAX_PIXELS — ห้ามลด MAX_LENGTH เดี่ยวๆ โดยไม่ลด MAX_PIXELS ตามไปด้วย เพราะจะทำให้ label ถูกตัดกลาง JSON (บั๊กคลาสเดียวกับที่เจอใน t01 §0.4)

กฎที่ได้: การเปลี่ยน config ที่กระทบหน่วยความจำยังไม่จบแค่คำนวณ ต้องรันสั้นๆ ยืนยัน peak บนฮาร์ดแวร์จริงก่อน และต้องส่งการเปลี่ยนแปลงพร้อมลำดับแผนสำรองที่เขียนไว้แล้ว ให้ถือว่าการยืนยันเป็นด้านบังคับ ไม่ใช่แค่ to-do

กฎ span-scaling ใหม่ถูกยัดเข้าสคิมาโดยไม่เคยตรวจสอบเลยว่า VLM วัดพิกเซลได้แม่นยำจริงหรือไม่

เกิดอะไรขึ้น: มีการรับกฎใหม่: มิติ span/dummy ที่ไม่มีตัวเลขพิมพ์กำกับ จะวัดเป็นพิกเซลเทียบกับเส้นกริดสองเส้นที่รู้ pos_m แล้วแปลงตามสัดส่วน พร้อม enum ใหม่ `span\_source: scaled\_from\_grid` ถูกดันเข้าไปใน 8+ ไฟล์ (สคิมา, template แผน + ไฟล์ที่ render แล้ว 4 ไฟล์, gridline, pass3, _common, และ SPAN_SOURCE_VALUES ใน schema.js ของ Constistant) — ก่อนที่จะมีใครตรวจสอบว่า VLM วัดระยะพิกเซลได้แม่นยำหรือไม่เลย

ต้นเหตุจริง: กลไกที่ดูดีถูกออกแบบและกระจายเข้า 8+ ไฟล์ตามความรู้สึกว่ามันถูกต้องเชิงแนวคิด โดยไม่มีการทดสอบความเป็นไปได้ราคาถูกก่อน แม้แต่ prompt เองก็ยังไม่ส่งเสริมความแข็งแกร่งไม่ให้โมเดลปลอมค่าที่วัดโดยแปะป้ายไว้แต่ไม่ใส่ตัวเลข

เสียหายอะไร: ถ้า VLM วัดพิกเซลไม่ได้แม่นยำเลย enum นี้ทั้งหมด ส่วน prompt ตัวอย่างที่ทำมือ และการเปลี่ยนสคิมาข้ามสองรีโปจะเสียเปล่า และโมเดลจะถูกเทรนให้ตอบค่าที่ผิดแบบมั่นใจเกินจริง

แก้ยังไง: ยังไม่ได้ยืนยันความถูกต้อง สิ่งที่ยังบรรเทาไปแล้ว: prompt บังคับให้ใส่ตัวเลขพิกเซลลงในแฟล็ก (`scaled\_between:1,3 P:380 U:142` — ถ้าไม่มี P/U แปลว่ายังไม่ได้วัดจริง ต้องตอบ `unresolved` แทน); เพิ่ม `--hide-grid-lines` ใน infer_house_t03.py (สำหรับ eval เท่านั้น) เพื่อซ่อน pos_m ที่รู้อยู่แล้วบนหน้า val แล้ววัด |ค่าที่วัดได้ - GT| ตรงๆ โดยไม่ต้องสร้างชุดข้อมูลใหม่ ระบุชัดว่ารอบ GPU รอบแรกต้องเทียบค่า scaled_from_grid ทุกค่ากับ GT ก่อนที่จะมีใครเชื่อได้

กฎที่ได้: ต้องทดสอบความสามารถใหม่ของโมเดลกับตัวอย่างสักรูปมือก่อน แล้วค่อยขยายการเปลี่ยนสคิมาข้ามสองรีโป ออกแบบวิธีพิสูจน์ว่าผิดพร้อมกับกลไกตั้งแต่แรก — และบังคับให้โมเดลใช้วิธีคิดของมัน เพื่อให้ตรวจจับคำตอบที่แต่งขึ้นมาได้

สูง 2026-08-31

คิดว่า **dry-run** ไม่ต้องใช้ **HF token** แต่จริงๆ **push_to_hub** เรียก **init_hf_repo** ตั้งแต่ตอนสร้าง **SFTTrainer**

เกิดอะไรขึ้น: Dry-run ตายด้วย error 401 ตั้งแต่ตอนสร้าง SFTTrainer เพราะ push_to_hub=True จะสร้าง repo บน hub แทนที่ตอนสร้าง object ไม่ใช่ตอนถึง save_steps อย่างที่เข้าใจไว้

ต้นเหตุจริง: library ทำงานที่เกี่ยวกับ network/auth เร็วกว่าที่ mental model ของเราคิดไว้

เสียหายอะไร: ถ้าไม่เจอตอนนี้ รันจริงโดยไม่มี HF_TOKEN จะพังทันทีตั้งแต่วันแรกที่แรก หรือแย่กว่านั้นคือถูกมองว่าปลอดภัยไปจนถึงชั่วโมงที่ 15

แก้ยังง: ตั้งให้ dry-run ปิด push อัปเดตโน้ต และบันทึกไว้ว่ารันจริงโดยไม่มี HF_TOKEN จะพังทันที ไม่ใช่พังช้า

กฎที่ได้: ต้องตรวจให้แน่ใจว่า library ทำ auth/network เมื่อไรจริงๆ อย่าเดาเอาเองว่าทำตอนใช้งานครั้งแรก เพราะ side effect ตอนสร้าง object เกิดขึ้นบ่อยและเปลี่ยนขอบเขตที่ dry-run ต้องครอบคลุม

สูง 2026-09-01

Destrier (t05) เป็นโมเดลที่รอดจนถึงตอนนี้ แต่ยังไม่เคยรันครบทั้งสายพานบนการ์ดจอจริงสักครั้ง

เกิดอะไรขึ้น: Destrier ฝึกเสร็จ, soup แล้ว, อัปเดต HuggingFace แล้ว แต่ผลวัด recall มีแค่ครั้งเดียวประมาณ 43% ภายใต้การจับคู่ข้อแบบหลวมๆ (forgiving) และยังไม่เคยรันทั้ง pipeline บน GPU จริงสักครั้งเลย

ต้นเหตุจริง: การตรวจสอบถูกเลื่อนไปหลังสองสายที่ล้มเหลว โมเดลที่รอดมาได้จึงสืบทอดสถานะ 'ใช้ได้ไปก่อน' โดยไม่เคยผ่านการเช็คทั้ง pipeline เลย

เสียหายอะไร: โมเดลตัวเดียวที่ทั้งโปรเจกต์พึ่งพาอยู่ มีตัวเลขความแม่นยำที่ยังไม่ยืนยัน วัดจากตัวอย่างเดียว และให้คะแนนแบบหลวมๆ แผนงานใดๆ ที่วางบนตัวเลข 43% นี่คือการอยู่บนทราย

แก้ยังง: ยังไม่ได้แก้ — บันทึกไว้ชัดเจนว่าเป็นงานค้าง: 'ยังไม่เคยรันครบทั้งสายพานกับการ์ดจอจริงสักครั้ง'

กฎที่ได้: โมเดลจะยังไม่ถือว่าผ่านการตรวจสอบ จนกว่าจะได้รัน production pipeline ทั้งหมดอย่างน้อยหนึ่งครั้งบนฮาร์ดแวร์จริง และตัวเลขหลักที่อ้างต้องยืนยันซ้ำด้วยการจับคู่แบบเข้มงวด (ไม่ใช่แบบหลวม) — ทุกครั้งที่พูดถึงตัวเลขนี้ ต้องบอกทั้งสองเงื่อนไขนี้ให้ชัด

เรื่องที่เหลือในหมวดนี้ (49)

grid_ref ถูกเก็บในรูปแบบสำหรับเครื่องอ่าน ({D1D2,4}) ซึ่งคนตรวจอ่านไม่ออก

กลาง 2026-07-06

หน้าจอที่ให้คนตรวจ ต้องมีการแสดงผลแบบที่คนอ่านรู้เรื่องสำหรับทุกฟิลด์ที่เป็นรูปแบบเครื่อง โดยแสดงคู่กันไป (ไม่ใช่แทนที่) ค่าดิบที่แก้ไขได้ ถ้าคนตรวจกวาดตาแล้วอ่านฟิลด์นั้นไม่ออก เท่ากับฟิลด์นั้นไม่ได้ถูกตรวจเลย

ชิ้นส่วนคานย่อยของ mark เดียวกันกระจัดกระจายในรายการตรวจ แทนที่จะถูกจัดกลุ่มไว้ด้วยกัน

กลาง 2026-07-06

ทุกครั้งที่การแก้ schema ทำให้จำนวนแถวเพิ่มขึ้นเป็นทวีคูณ ให้แก้การแสดงผลหน้าตรวจไปพร้อมกันในรอบเดียวกัน จัดกลุ่มแถวตามคีย์ที่คนใช้คิดจริง (คือ mark) และใช้การเรียงแบบ stable เพื่อให้ลำดับภายในกลุ่มยังคงความหมายเดิมไว้

ไฟล์ PNG ของ BOQ หนึ่งไฟล์ จริงๆ มีแบบสองแผ่นซ้อนกัน หมุน 90° และแยกซ้าย-ขวา

กลาง 2026-07-06

ให้ตรวจสอบสมมติฐาน 'หนึ่งหน้า = หนึ่งรูป' กับตัวอย่างของทุกชุดแบบก่อนเริ่มสกัดข้อมูล และบันทึกความแปลกของรูปแบบไฟล์ต้นทางไว้ในโน้ตของบ้านหลังนั้นทันที เพื่อไม่ให้คนถัดไปต้องมาค้นพบซ้ำ ถ้าสองคนเจอความแปลกอันเดียวกันแยกกัน แปลว่าคนแรกไม่ได้เขียนไว้ในที่ที่คนที่สองจะเห็น

สคริปต์ตรวจสอบรายงานว่าไฟล์รูปหายทั้ง 61 ไฟล์ ทั้งที่ตัว path ต่างหากที่เป็นบั๊ก ไม่ใช่ไฟล์

กลาง 2026-07-08

เวลาสคริปต์ตรวจสอบบอกว่าทุกอย่างพัง 100% ให้สงสัยสคริปต์ก่อนสงสัยข้อมูล และห้าม hardcode path แบบ shell ลงใน runtime ปกติบน Windows เด็ดขาด

ไม่เคยยืนยันเลยว่าโปรเจกต์ Material List ใน Label Studio ถูกสร้างขึ้นจริงหรือเปล่า

กลาง 2026-07-09

สถานะของระบบภายนอกต้องยืนยันจากระบบภายนอกนั่นเอง (เอา project id หรือ URL มา) ไม่ใช่จากสกรีนช็อตของไฟล์ในเครื่องที่แค่เกี่ยวข้องกับมัน

ชุด BOQ ของฐานรากแผ่กับฐานรากเสาเข็มที่หาคู่ขนานกัน ถูกวินิจฉัยผิดว่าเป็นแบบพิมพ์ซ้ำ

กลาง 2026-07-12

ตัวเลขที่เหมือนกันบนแบบ BOQ สองแผ่น ไม่ใช่หลักฐานว่าซ้ำกันโดยตัวมันเอง ให้อ่านหัวแผ่นก่อนเสมอ (ฐานรากแผ่ vs ฐานรากเสาเข็ม) ก่อนจะฟันธงว่าซ้ำ และบันทึกความสัมพันธ์ไว้ในไฟล์อ้างอิงไว้ให้ชัด แทนที่จะลบหรือติดธงทิ้งไว้

เกือบจะลากค่าช่วงกริดที่เดามาไปใส่ WC1 โดยไม่ได้เช็คแบบขยายเฉพาะของมันก่อน

กลาง 2026-07-14

ก่อนจะลากการเปลี่ยนกริดไปใส่ห้องหรือ element ใดๆ ให้เช็คก่อนว่ามีแบบขยายเฉพาะของมันอยู่หรือเปล่า แบบขยายขณะการอนุมานที่ลากมาจากผังภาพรวมเสมอ

ช่วงคาน B4X สองช่วงถูกย้อนกลับไปใช้ dummy grid label ผิดหลังจากแก้ถูกไปแล้วรอบหนึ่ง

กลาง 2026-07-15

หลังแก้ไฟล์แบบใดๆ ต้อง validate ทุกช่วงกับ grid master ใหม่เสมอ มาร์คเดียวกันแต่ค่า span ไม่ตรงกันคือสัญญาณของ regression ต้องมองเป็นบั๊ก ไม่ใช่มองเป็นข้อมูลปกติ

คานคอดินถูกบันทึกเป็น RB1A ทั้งที่ในแบบพิมพ์ RB1

กลาง 2026-07-15

จุดที่มาร์คเปลี่ยนกลางเส้น ให้ชุมจุดดูต่อและเทียบความสมมาตรกับฝั่งตรงข้าม เพราะจุดเปลี่ยนมาร์คคือจุดที่การอ่านผิดชอบซ้อนตัวพอดี

การตรวจ pre-flight ผ่านครบทั้ง 5 ข้อ แต่ไม่จับปัญหาที่ทำให้ session พังจริงได้เลยสักข้อ

กลาง 2026-07-21

pre-flight ต้องตรวจสอบสิ่งที่เคยฟังมาก่อน: พื้นที่ดิสก์ว่าง $\geq$ จุดสูงสุดที่ pipeline ใช้, เวอร์ชันของ transformers/torch ที่ pin ไว้แน่นอน (บันทึกและยืนยันซ้ำทุกครั้ง), และการรันจริง 1 ตัวอย่างผ่าน code path จริง (รวม setting ของ image-processor ด้วย) ไม่ใช่แค่ตรวจว่า import ได้

ค่า hash รวมของชุดข้อมูลออกมาไม่ตรงกับที่จดไว้ ทำให้ตกใจว่าข้อมูลเปลี่ยน ทั้งที่ไม่เคยจดสูตรที่ใช้คำนวณไว้เลย

กลาง 2026-07-28

ห้ามจด checksum, hash หรือค่าที่คำนวณมา โดยไม่จดคำสั่งหรือสูตรที่ผลิตมันด้วย ตัวเลขตรวจสอบที่ทำซ้ำไม่ได้ แยกว่าไม่มีเลย เพราะมันสร้างสัญญาณเตือนปลอม

เกือบ 'แก้' max_seq_length ที่หายไปใน eval_fields.py ซึ่งจะทำให้เทียบผลข้ามรอบไม่ได้ ทั้งที่ไม่มีเหตุผลต้องแก้

กลาง 2026-07-28

ก่อนจะแก้บั๊กที่สงสัยในโค้ดที่ใช้วัดผลรวมกัน ให้หาหลักฐานจากผลจริงก่อนว่าบั๊กนั้นทำงานจริงไหม ถ้าได้ผลลัพธ์ที่เป็นไปไม่ได้เลย ถ้าบั๊กมีอยู่ นั่นคือหลักฐานว่ามันไม่มี และการแก้บั๊กที่ใส่ร่วมกันโดยไม่จำเป็น แยกว่าตัวบั๊กเอง

ยืนยันว่าโมเดลรองรับ โดยอ้างเลข PR ที่ merge แล้ว แทนที่จะดูจากตัว binary ที่อยู่บนเครื่องจริง

กลาง 2026-07-28

พิสูจน์ความสามารถจากของจริงบนเครื่องจริง เปิดดู binary เปิดอ่านไฟล์ config ตัวจริง เลขเวอร์ชันหรือเลข PR เป็นแค่ข้อสันนิษฐาน ไม่ใช่หลักฐาน

ตัวเลข VRAM ~74GB ที่ใช้ตัดสินใจเลือกเครื่อง ไม่เคยวัดจริง แต่คาดจาก t01 กับการทดลองรัน 5 step

กลาง 2026-07-28

พอรันเต็มรอบสำเร็จ ให้เก็บค่า VRAM/RAM สูงสุดทันที เพราะนั่นคือจังหวะเดียวที่จะได้ตัวเลขจริงมาฟรีๆ ตัวเลขจากการรัน 5 step ไม่ใช่ตัวเลขความจุที่ใช้อ้างอิงได้

มีคนสั่งทำลายเครื่อง GPU เขาสองครั้งกลางคัน ก่อนที่จะดาวน์โหลดผลลัพธ์ลงมา เก็บข้อมูลหาย

กลาง 2026-08-20

ทำให้การทำลายเครื่องเกิดขึ้นไม่ได้ถ้ายังไม่ได้ sync โดยห่อขั้นตอนปิดเครื่องไว้ในสคริปต์ที่ rsync ผลลัพธ์ลงมาและนับไฟล์ในเครื่องยืนยันก่อน วินัยแบบนี้ไม่ควรฝากไว้กับความจำของคนที่กำลังเหนื่อย

การ import แบบครบวงจรถูกตรวจสอบด้วยการรัน adapter ใน Node เท่านั้น ไม่เคยผ่านหน้าจอบเอร์วเซอร์จริง

กลาง 2026-08-21

การตรวจระดับ Node พิสูจน์ได้แค่เครื่องยนต์ ไม่ใช่ตัวผลิตภัณฑ์ วงจรจะยังไม่ถือว่าครบจนกว่าจะกด import จริงหนึ่งครั้งผ่าน UI จริง และได้ารต้องบอกตรงๆ ว่าทำแบบไหนไป

เดโม 'ครบวงจร' เข้า Constant ใช้ grid master ที่คนทำ ไม่ใช่ของโมเดล Qwen ไม่เคยรันหน้าเส้นกริดเลย

กลาง 2026-08-21

เวลาเดโม pipeline แบบครบวงจร ต้องบอกให้ชัดว่าขั้นไหนโมเดลทำจริง ขั้นไหนเอาข้อมูลคนมาแทน เพราะการเอาข้อมูลคนมาใส่แทนตั้งแต่ขั้นแรก ทำให้คำว่า 'ครบวงจร' ใช้ไม่ได้แล้ว ต่อให้ขั้นหลังจากนั้นจะจริงทั้งหมดก็ตาม

สร้างข้อเสนอความสามารถของโมเดลที่ไม่เคยพิสูจน์กับ adapter ที่ fine-tune แล้วจริงๆ

กลาง 2026-08-22

ก่อนออกแบบขั้นตอนใน pipeline โดยฟังความสามารถของโมเดลตัวตั้งต้น ให้อยู่สบายๆ \$0.10 ทดสอบก่อนว่า adapter ที่ fine-tune แล้วยังทำได้อยู่ไหม และให้เขียนข้อสันนิษฐานที่ยังไม่ทดสอบไว้ชัดๆ ในเอกสาร ไม่ใช่ซุกไว้

ข้อสันนิษฐานที่ยังไม่พิสูจน์ข้อที่สองในข้อเสนอเรื่อง grounding: chat template อาจไม่ปล่อย box token ออกมาเลยด้วยซ้ำ

กลาง 2026-08-22

การทดสอบความสามารถต้องรันผ่านเส้นทางการเสิร์ฟจริงทั้งเส้น (template + tokenizer + decode) ไม่ใช่แค่ถามว่า 'โมเดลรู้วิธีทำไหม' ให้ไล่รายชื่อทุกชั้นระหว่าง prompt กับผลลัพธ์ที่ parse แล้ว ก่อนจะบอกว่าการทดสอบนี้ถูก

ข้อสันนิษฐานของทีปรึกษาที่ว่า 'ก็ใช้ Qwen ตัวใหญ่ขึ้นสิ' มันหมดทางไปแล้ว

กลาง 2026-08-25

ก่อนรับข้อเสนอ 'ใช้โมเดลใหญ่ขึ้น' มาเป็นงานที่ต้องทำ ให้ตรวจก่อนว่าขนาดไหนบ้างที่ดาวน์โหลดได้และเทรนได้จริง และให้คิดราคาของการเปลี่ยนโครงสร้างพื้นฐานทั้งหมด ไม่ใช่แค่ค่า GPU ต่อชั่วโมง

template ของฐานราก F2 ยิ่งผลบวกปลอม 3 จุดบนแปลนคาน ซึ่งไม่มีฐานรากอยู่เลยสักตัว

กลาง 2026-08-25

เมื่อตัวตรวจจับสับสนระหว่างสองคลาส ให้เลือกใช้เงื่อนไขเชิงเงื่อนไขที่พิสูจน์ได้ว่าไม่ทำอะไรกับเคสที่ถูกอยู่แล้ว ดีกว่าไปชนเกณฑ์ให้แน่นขึ้น ซึ่งแค่แลกผลบวกปลอมเป็นผลลบปลอม ไม่ได้กำจัดอะไรสักอย่าง

pattern_recognition.py ทดสอบกับบ้านแค่ 2 หลัง ซึ่งมาจากแบบชุดเดียวกัน

กลาง 2026-08-25

ทุกครั้งที่ยำงานตัวเลขความแม่นยำ ต้องบอกด้วยว่าชุดทดสอบหลากหลายแค่ไหน ตัวอย่าง 2 ตัวจากแหล่งเดียวกันคือตัวอย่างของแหล่งนั้น 1 ตัว ไม่ใช่ 2

เครื่องมือตรวจจับไม่เคยถูกทดสอบกับหน้าปก หน้า section หรือตาราง BOQ เลย อาจไปมาร์กมั่วอย่างมั่นใจได้

กลาง 2026-08-25

detector ที่ใช้ได้กับ input แคบางชนิด ต้องทำอย่างใดอย่างหนึ่ง คือตรวจจับแล้วปฏิเสธชนิดที่ไม่รองรับ หรือห้ามส่งให้ผู้ใช้จนกว่าจะทดสอบครบทุกชนิดหน้าที่เขาจะป้อนเข้ามา

ปล่อยฟิกซ์ MAX_PIXELS ออกไปทั้งที่รู้ว่ารูป 30% ยังถูกย่ออยู่ และรูปใหญ่สุด 116 รูปยังเสียรายละเอียดไป 67%

กลาง 2026-08-25

ถ้าแก้ได้แค่บางส่วน ให้ออกส่วนที่ยังเหลืออยู่ในประโยคเดียวกับผลที่ดีขึ้น "ดีขึ้น 32%" กับ "รูป 30% ยังแยอยู่" เป็นการวัดชุดเดียวกัน ต้องเดินคู่กันเสมอ

เกือบพลาด: วิธีแก้แรกที่คิดไว้สำหรับ false positive ของ F2 จะลบฐานรากจริงทิ้งไปเสียๆ

กลาง 2026-08-25

ปุมปรับแบบ global แก้ error ที่ขึ้นกับบริบทไม่ได้ มันแค่เปลี่ยนความพึงแบบดั่งๆ ให้กลายเป็นความพึงแบบเสียๆ ถ้าคำตอบผิด เพราะตำแหน่งที่มันอยู่ วิธีแก้คือกฎเกี่ยวกับตำแหน่ง ไม่ใช่การปรับตัวเลข

"ก็แค่เพิ่ม B ลี" คิดว่าขนาดโมเดลเป็นปุมที่หมุนได้ฟรี ทั้งที่ตระกูลนี้ชนเพดาน open-weight ไปแล้ว

กลาง 2026-08-25

ก่อนจะเสนอให้ใช้โมเดลใหญ่ขึ้น ให้ไล่ดูก่อนว่า open-weight ที่ปล่อยออกมาจริงมีขนาดไหนบ้าง แล้วตีราคาฮาร์ดแวร์ของแต่ละขนาด "ใหญ่ขึ้น" เป็นการตัดสินใจเรื่องฮาร์ดแวร์กับ pipeline ไม่ใช่แค่ปรับ hyperparameter

แปล prompt ทั้งก่อนเป็นอังกฤษได้คะแนนแยกว่า prompt ภาษาแม่ราว 8% เป็นกับดักที่ไอเดียทำ dictionary เกือบเดินเข้าไป

กลาง 2026-08-25

เวลาเอาเทคนิคที่ได้จากที่ประชุมมาใช้ ให้หาทั้งเคสที่มันล้มเหลวและเคสที่มันสำเร็จ แล้วระบุให้ชัดว่ามันแก้ปัญหาข้อไหนกันแน่ ก่อนจะลงมือเขียนอะไรสักบรรทัด

สมมติฐานแรกผิด: ฐานราก F1 กับ F2 ต่างกันที่รูปทรง ไม่ใช่ขนาด matching หลาย scale เลยหา F1 ไม่เจอสักตัว

กลาง 2026-08-25

ก่อนเลือกกลยุทธ์ invariance (scale, rotation) ให้เอาตัวแปรจริงมาวางเทียบกันดูด้วยตาก่อน ถ้ามันเป็นสัญลักษณ์คนละตัว ต่อให้ย่อขยายยังไงก็หาไม่เจอ ต้องทำ template แยกต่อหนึ่งแบบ ตามที่ตำราบอกไว้เป๊ะ

template ฐานราก F2 ทรงสี่เหลี่ยมสูง ไปอิงโดนแบบแปลนคานผิดๆ

กลาง 2026-08-25

detector ที่อาศัยรูปทรงต้องมีตัวกรองบริบท ไม่ใช่แค่ขึ้น threshold ให้สูงขึ้น ใช้สัญญาณที่อยู่หน้าเดียวกันและเกิดพร้อมกันไม่ได้ (ในเคสนี้คือคานกับฐานรากไม่เคยอยู่หน้าเดียวกัน) มาตัด false positive ข้ามบริบทออก

matching หลาย scale บน template เสาที่เล็กแค่ 28x34 ทำให้จำนวนที่ตรวจเจอพองจาก 14 เป็น 21 ด้วย noise

กลาง 2026-08-25

อย่าใช้กลยุทธ์ invariance แบบเดียวกันกับทุก template ตัว template เล็กๆ มีสัญญาณน้อยเกินไปสำหรับ multi-scale ทุกครั้งที่เปลี่ยนกลยุทธ์ ให้ทดสอบกับหน้าที่รู้จำนวนที่ถูกต้องอยู่แล้วก่อนจะเก็บไว้ใช้

tpl_footing9 กว้างเกินไป ทำให้ยิงมั่วในบ้านหลังอื่น ทั้งที่คะแนนกับบ้านตัวเองก็ยังแย

กลาง 2026-08-25

ตัดสิน template ใหม่จากผลที่แย่ที่สุดของมันเมื่อเจอทุกบ้าน ไม่ใช่ผลที่ดีที่สุดกับบ้านที่มันเกิดมา ยอมมีช่องโหว่ที่ซื่อสัตย์ ดีกว่าใช้ template ที่คะแนนของชยะกับคะแนนของจริงหับกันจนแยกไม่ออก

Molmo-72B ที่ HuggingFace ระบุ 293GB นั้นเป็น fp32 ไม่ใช่ bf16 ขนาดจริงเล็กกว่าเกือบครึ่ง

กลาง 2026-08-25

ก่อนใช้ขนาดที่ repo ของโมเดลระบุไว้ ให้เอา bytes หารด้วยจำนวนพารามิเตอร์เพื่อดูว่าเป็น dtype อะไร repo ที่ปล่อยเป็น fp32 จะดูใหญ่กว่าขนาดจริงแบบ bf16 ราวสองเท่า

โมเดล InternVL ที่โฆษณาว่าไม่ใช่ Qwen จริงๆ ข้างในหุ้ม LLM backbone ของ Qwen อยู่

กลาง 2026-08-25

ตรวจสอบสายพันธุ์ของโมเดลจาก backbone ที่ config ประกาศไว้ อย่าดูจากชื่อแบรนด์ ตัว vision tower กับ LLM ของ VLM ตัวหนึ่งมักมาจากคนละองค์กรกัน

งานวิจัยรอบหนึ่งเข้าใจไปเองว่าการเทรนใช้ 4-bit QLoRA ทั้งที่ LOAD_IN_4BIT เป็น False อยู่แล้ว (คือ bf16)

กลาง 2026-08-25

อ่าน config ครึ่งก่อนจะไปคิดเรื่องการใช้ทรัพยากร ข้อสันนิษฐานเกี่ยวกับ setup ของตัวเองเป็นอย่างไรที่ตรวจสอบง่ายที่สุด และแพงที่สุดถ้าเข้าใจผิด

pattern recognition ตรวจสอบกับบ้านแค่ 2 หลังที่มาจากแบบชุดเดียวกัน

กลาง 2026-08-25

ถ้าชุดทดสอบมาจากแหล่งเดียวกัน สิ่งที่คุณวัดคือความแม่นยำในแหล่งนั้น ไม่ใช่ความแม่นยำจริงๆ ให้ระบุความหลากหลายของแหล่งที่มาไว้ข้างตัวเลขความแม่นยำทุกครั้ง

บ้านสองหลังตรวจเจอฐานรากเกินจริงราวสองเท่า น่าจะเพราะ template ซ้ำมุดไปแมตช์กับภาพขยายในแบบ

กลาง 2026-08-25

การตรวจเจอเกินก็เป็นความล้มเหลวพอๆ กับการตรวจไม่เจอ แกรมมองข้ามง่ายกว่าเพราะมันดูเหมือนครอบคลุมดี ให้วัดทั้งสองทิศทางเสมอ และไล่เคสที่ตรวจเกินด้วยแรงเท่ากับที่ไล่เคสที่พลาด

self-harvest กู้คืนได้แค่ประเภท element ที่มีอยู่ใน template bank แล้วเท่านั้น นี่คือเพดานจริงของมัน ซึ่งเจอจากการวัด

กลาง 2026-08-26

เพดานของรูป bootstrap คือคลาสที่มันมี seed อยู่แล้วอย่างน้อยหนึ่งตัว ให้วัดอัตราการกู้คืนแยกรายคลาส ไม่ใช่ดูตัวเลขรวม ไม่จันทัวเลขรวมจะบังไว้ว่ามีทั้งคลาสที่กู้คืนไม่ได้เลยสักตัว

กฎลอกเครื่องหมายวรรคตอนแบบเหมารวมจะทำให้ ป. (ปลอก) กลายเป็น ป แล้วชนกับ ประตุ

กลาง 2026-08-29

กฎที่กวาดอักขระเหมารวมบนข้อความไทยต้องมีรายการยกเว้นที่สร้างจากการ grep การใช้งานจริงก่อน เคสอันตรายคือเคสที่ลอกแล้วยังได้ token ที่ดูเหมือนถูกต้อง

เคลมว่า Qwen3.5-122B-A10B เป็นโมเดลข้อความล้วน — จริงๆ มันมี mmproj และเป็น multimodal

กลาง 2026-08-30

ห้ามพูดว่า 'X ไม่รองรับ Y' โดยไม่เปิดแหล่งอ้างอิงจริงในจังหวะเดียวกัน — model card, รายการไฟล์ หรือขอร์สของไลบรารี

ยืนยันว่า 'เอา LoRA มาเฉลี่ยรวมกันไม่ได้' — ทั้งที่ PEFT มี 'add_weighted_adapter' และเคสนี้เหมาะสมมาก

กลาง 2026-08-30

ทุกคำเคลมเชิงปฏิเสธความสามารถต้องมีแหล่งที่ตรวจแล้วรองรับก่อนพูดออกมา พูดผิดแบบนี้สองครั้งในวันเดียวคือรูปแบบพฤติกรรม ไม่ใช่อุบัติเหตุ ให้ถือว่า 'X ทำ Y ไม่ได้' เป็นงานที่ต้องไปค้น ไม่ใช่เรื่องที่นึกเอาจากความจำ

เดาว่าหน้ารายการวัสดุ ปร.4 วางซ้อนบน-ล่าง จริงๆ มันวางเคียงกันและหมุน 90 องศา

กลาง 2026-08-31

ข้อสันนิษฐานเรื่องเลย์เอาต์ ต้องตรวจกับตัวอย่างจริงอย่างน้อยสองอันจากคนละแหล่งก่อนเอาไปใช้ทั้งชุด และบันทึกระดับความมั่นใจให้ตรงกับจำนวนที่ตรวจจริง

พูดมั่นใจเรื่องไวยากรณ์เครื่องมือภายนอกที่ยังไม่ได้ตรวจสอบจริง

กลาง

อย่าพูดมั่นใจในสิ่งที่ยังไม่ได้ตรวจสอบ (เช่น ไวยากรณ์เครื่องมือภายนอกที่ยังไม่ยืนยัน) — บอกระดับความมั่นใจจริงและเสนอขั้นตอนตรวจสอบแทนการเดาเสียๆ

อ้างกฎแล้วลงมือทำในข้อความเดียวกันเลย แทนที่จะหยุดรอคำตอบจริง

กลาง

การอ้างกฎต้องตามด้วยการหยุดรอคำตอบจริงจากผู้ให้เสมอ — ห้ามอ้างกฎแล้วลงมือทำทันทีในข้อความเดียวกัน

เกือบใช้ className ใน <Style> ของ Label Studio กับ Text ทั้งที่เอกสารยืนยันแค่ View/Choices เท่านั้น

ต่ำ 2026-07-06

ถ้าเอกสารของเครื่องมือยืนยัน feature กับบาง element แต่ไม่พูดถึง element อื่นเลย ให้ถือว่า element ที่ไม่พูดถึงนั้น 'ไม่รองรับ' แล้วใช้ config แบบที่พิสูจน์แล้วว่าใช้ได้จริงในโปรเจกต์นี้ — อย่าเดาต่อยอดจากเอกสารที่บอกไม่ครบ

ระบบตีงว่าการสำรองข้อมูลล้มเหลว ทั้งที่ robocopy exit code 1 หมายถึงสำเร็จ

ต้า 2026-07-10

ก่อนเชื่อผลแดง/เขียวของเครื่องมือใดๆ ต้องเช็คความหมายของ exit code ก่อน สำหรับ robocopy โดยเฉพาะ ค่าที่ต่ำกว่า 8 คือ สำเร็จ ส่วนตัวเลข FAILED ใน log ต่างหากคือคำตอบที่แท้จริง

ตรวจจับเสาเข้าที่กริด (B,2) — นับได้ 13 กลอง ทั้งที่เสาจริงมี 12 ต้น

ต้า 2026-08-25

ให้รายงานจำนวนที่ตัวตรวจจับนับได้เทียบกับค่าจริงที่ตรวจด้วยมือทุกหน้าที่ใช้ validate ไม่ใช่แค่หน้าหลัก ตัวเลขที่ไม่เคยตรวจสอบไม่ใช่หลักฐาน และการนับเกินเล็กๆ อย่างเป็นระบบมักซ่อนอยู่ในตัวเลขที่ดู 'ใกล้เคียง'

ขนาดจริงของ Molmo-72B แบบ bf16 กินพื้นที่ราว 147GB ซึ่งไม่ตรงกับที่คำว่า '72B' บอกเป็นนัย — คอลัมน์ขนาดโมเดลกับการจัดจ่อที่ต้องเข้าจริง เป็นคนละตัวเลขกัน

ต้า 2026-08-26

ต้องคัดกรองตัวเลือกโมเดลจากจำนวนไบต์จริงที่ precision ที่จะใช้รันจริง ไม่ใช่จากจำนวนพารามิเตอร์ที่ประกาศขาย — และให้จดไว้ทุกแถว ไม่ใช่แค่แถวที่บังเอิญมีคนไปเช็คให้

แผนงานสันนิษฐานว่า grid master เกือบกิดพิกเซลไว้ — จริงๆ มีแค่ pos_m

ต้า 2026-08-26

ต้องเปิดไฟล์ข้อมูลจริงดูก่อนออกแบบอะไรที่ต้องใช้มัน แ่อ่านหัววินาทีดีกว่ามานั่งทำงานซ้ำทั้งปายเพราะสันนิษฐานผิดพลาด

ไอคอนเสาในบ้าน 01/02 มีขนาดต่ำกว่าเกณฑ์ขั้นต่ำ 28px ที่สันนิษฐานไว้ ไม่มีใครสังเกตจนกว่าจะครอบใหม่พร้อมบริบทรอบๆ

ต้า 2026-08-28

ขนาดขั้นต่ำในตัวตรวจจับมักถูกกำหนดตามชุดแบบชุดแรกๆ ที่เห็น พอบ้านไหนให้ผลลัพธ์เป็นศูนย์เลย ให้สงสัยว่าเป็นบั๊กของเครื่องมือแล้วไปตรวจสอบ ไม่ใช่สรุปเอาว่า 'บ้านนั้นไม่มี'

พังเจียบ ไม่มีใครรู้

โค้ด/ไปป์ไลน์ที่พังหรือข้ามงานไปเจียบๆ ไม่มี error ให้เห็น ตัวเลขออกมาดูปกติแต่ผิด

ทั้งหมด 22 เรื่อง · วิกฤต 7 · สูง 8 · กลาง 6 · ต่ำ 1

เรื่องที่ต้องอ่านให้ครบ (15)

วิกฤต 2026-07-05

Agent เบื้องหลังค้างสองรอบ ถูกสั่งหยุดกลางคัน และหน้า 41-47 ของ BOQ ฐานรากแผ่ไม่เคยถูกสกัดเลย

เกิดอะไรขึ้น: Agent 4 ถูกส่งไปทำหน้าที่ 38-47 (BOQ ฐานรากแผ่) มันหยุดตัวเองกลางทางสองครั้ง ต้องปลุกด้วย SendMessage สองรอบ ทำเสร็จแค่หน้า 38, 39, 40 แล้วก็ถูกม้ามสั่งหยุดตอนกำลังทำหน้าที่ 41 หน้า 41-47 จึงไม่เคยถูกสกัดเลย — ทั้งในรอบ t1 และรอบ t2 (เพราะ t2 กำหนดขอบเขตไว้แค่หน้า 1-37)

ต้นเหตุจริง: Agent ที่รันท้ายกับงานซ้ำๆ จำนวนหน้าเยอะ หยุดตัวเองซ้ำๆ โดยไม่มีระบบ resume อัตโนมัติ และไม่มีการทำเครื่องหมายจุดเช็คต่อหน่วยงาน ที่จะให้คนอื่นมารับช่วงต่อที่หน้า 41 ได้สะดวกๆ ช่องว่างที่เหลืออกนอกขอบเขตรอบถัดไป เพราะ t2 กำหนดขอบเขตด้วยช่วงเลขหน้า ไม่ใช่ด้วย 'อะไรที่ยังขาดอยู่'

เสียหายอะไร: ข้อมูลจริง BOQ หายถาวร 7 หน้าของบ้าน 01 ข้ามการสกัดสองรอบเต็ม ค้างเป็นงานค้างตั้งแต่ 2026-07-05 ยังค้างอยู่จนจบ 2026-07-06 และทิ้งไฟล์ scratch กำพร้าไว้ด้วย

แก้ง่าย: ยังไม่ได้แก้ ขึ้นเป็นงานค้างข้อ 5 ท้ายวัน 2026-07-06: 'หน้า 41-47 (BOQ ฐานรากแผ่) ยังไม่ถูกสกัดในรอบไหนเลย'

กฎที่ได้: งาน agent ที่รันท้ายต้องแบ่งเป็นหน่วยงานเล็กๆ ที่ชัดเจน พร้อมมีเครื่องหมาย resume ต่อหน่วย และห้ามกำหนดขอบเขตรอบถัดไปด้วยช่วงเลขหน้า — ให้กำหนดจากส่วนต่างเทียบกับสิ่งที่เสร็จจริง ช่องว่างที่รู้อยู่แล้วจะได้ไม่หลุดออกนอกขอบเขตรอบถัดไปแบบเจียบๆ

วิกฤต 2026-07-28

บั๊กที่ 2: ค่าเริ่มต้น `resize='min'` ของ `collator` ตกไปที่ 512px เจียบๆ — ความละเอียดหายไป 25 เทา ขณะที่ VRAM ดูสบายๆ จนล่อลวงตา

เกิดอะไรขึ้น: vision collator ใช้ค่าเริ่มต้น `resize='min'` ซึ่งไปอ่านคีย์ `image_size` ของโมเดล แต่ Qwen3-VL ไม่มีคีย์ `image_size` เลย collator เลยตกไปใช้ 512px เจียบๆ — ความละเอียดหายไป 25 เทาจากที่ตั้งใจไว้ ส่วนการใช้ VRAM ก็ดูสบายๆ ซึ่งอ่านแล้วเหมือนงานกำลังไปได้สวย ไม่เหมือนงานที่พัง

ต้นเหตุจริง: ค่าเริ่มต้นของไลบรารีที่ลดคุณภาพลงเจียบๆ เมื่อหาคีย์ไม่เจอ บวกกับข้อเท็จจริงที่ว่าอาการเดียวที่สังเกตได้ (VRAM ต่ำ) ดันดูเหมือนความสำเร็จ และไม่มีอะไรคอยตรวจว่าความละเอียดไปถึงโมเดลจริงๆ เป็นเท่าไร

เสียหายอะไร: จะเรนโมเดลด้วยรูปที่หยาบกว่าที่ตั้งใจ 25 เทา โดยที่ตัวชี้วัดทุกตัวบนหน้าจาดูปกติดี บั๊กตัวเดียวกันนี้ที่ไม่มีใครจับได้ในฝั่ง t01 กลายเป็นตัวที่ทำให้การเปรียบเทียบ A/B ทั้งหมดใช้ไม่ได้ในภายหลัง

แก้ง่าย: บังคับ `resize='max'` ไปเลยตรงๆ พร้อมใส่ assertion ตรวจความละเอียดของ batch แรก

กฎที่ได้: อย่าไว้ใจค่าเริ่มต้นของไลบรารีที่ต้องไปหาค่าจากคีย์ที่มีหรือไม่มีก็ได้ ให้ assert ค่าที่ใช้จริงตั้งแต่ batch แรก — และให้ถือว่าการใช้ทรัพยากรน้อยผิดปกติเป็นสัญญาณอันตราย ไม่ใช่เรื่องน่าสบายใจ

ตัวกันบั๊กที่แพงที่สุดของโปรเจกต์หลวมเกินไป 4 เท่า จนปล่อยผ่านบั๊กนั้นได้

เกิดอะไรขึ้น: assert ตัวแรกของ train_t03.py มีคอมเมนต์ว่า "ตัวอย่างแรก (ภาพเดียว)" และเขียนว่า ``_tok = (grid[:,1]*grid[:,2]).sum() // 4`` แล้ว ``assert _tok > 1000`` แต่ ``train_ds[0]`` ในชุดข้อมูล op04 มี 4 ภาพ และ ``sum()`` รวมค่าทุกภาพเข้าด้วยกัน ทั้งที่ threshold ตั้งไว้เป็นต่อภาพ จำลองสถานการณ์แล้วพบว่าที่ระดับความผิดพลาดเดียวกับ t01 (266 token/ภาพ) $\times 4 \text{ ภาพ} = 1,064 > 1,000$ assert เลยผ่าน

ต้นเหตุจริง: ตัวกันนี้ถูกเขียนตอนที่ตัวอย่างแรกมีภาพเดียว พอรูปแบบข้อมูลเปลี่ยนเป็นหลายภาพต่อตัวอย่างในภายหลัง ไม่มีใครกลับมาทบทวน guard ตัวนี้ คอมเมนต์ใน assert ก็ยังบันทึกข้อสันนิษฐานเดิมไว้

เสียหายอะไร: ตัวกันเดียวที่มีไว้ป้องกันไม่ให้เกิดซ้ำแบบ t01 (เทรนที่ 266 token/ภาพโดยไม่มีใครรู้ตัว บั๊กที่แพงที่สุดในประวัติศาสตร์โปรเจกต์) ไม่ทำงานจริง ถ้าเกิดหายนะแบบ t01 อีกครั้งจะรันจนจบโดยไม่มีใครจับได้ เผาเงินค่าเช่า GPU ทั้งเปล่าๆ

แก้ยังไง: หาดด้วย ``gt.shape[0]`` ให้ threshold เป็นต่อภาพจริงๆ พิมพ์ค่า cap ที่ตั้งไว้คู่กับค่าที่วัดได้ และเพิ่มค่าเตือน (ไม่ทำให้พัง) เมื่อค่าที่วัดต่ำกว่า cap เกิน 25% ตรวจ ast.parse ผ่านสะอาด และจำลองทั้งสองกรณีแล้ว

กฎที่ได้: ตัวกันที่เขียนไว้สำหรับข้อมูลรูปแบบหนึ่งจะเสื่อมสภาพเรื่อยๆ เมื่อรูปแบบข้อมูลเปลี่ยน ต้องกลับมาพิสูจน์ตัวกันเป็นระยะ ด้วยการจำลองความผิดพลาดที่มันควรจะจับได้ ถ้าทำให้ assert ทำงานไม่ได้ แปลว่ามันไม่ได้ป้องกันอะไรอยู่เลย

t01 เทรนที่ค่า default ของไลบรารี 512px / ~266 token ต่อภาพโดยไม่มีใครรู้ตัว ทำความแม่นยำราว

เกิดอะไรขึ้น: อ้างอิงในเชสชันนี้เป็นหายนะมาตรฐานของโปรเจกต์ t01 ไม่เคยตั้งค่า MAX_PIXELS ไว้ชัดเจน processor เลยถอยไปใช้ค่า default 512px (~266 visual token/ภาพ) แทนที่จะเป็นบัพที่ตั้งใจไว้ ความแม่นยำเลยร่วงลงยับ บันทึกไว้เป็น rule_of_tune ข้อ 12

ต้นเหตุจริง: hyperparameter ที่มีผลต่อความแม่นยำโดยตรงถูกปล่อยให้ค่า default ของไลบรารี แทนที่จะตั้งค่าและ assert ไว้ชัดเจน และไม่มีการพิมพ์ค่าจริงที่ใช้ออกมาตอนเริ่มรันเลย

เสียหายอะไร: การเทรนทั้งรอบ (ทั้งค่าเช่า GPU และเวลา) ได้ผลลัพธ์ที่แย่ง ต้องมาไล่วินิจฉัยย้อนหลังทีหลัง เป็นบั๊กที่แพงที่สุดในประวัติศาสตร์โปรเจกต์

แก้ยังไง: ตอนนั้นตั้งค่า MAX_PIXELS ไว้ชัดเจนใน train_t03.py และมี assert คมไว้ (ซึ่งตัว assert เองก็ต้องแก้ในเชสชันนี้ด้วย — ดูรายการเรื่อง guard หลวม 4 เท่า)

กฎที่ได้: ตั้งค่า hyperparameter ทุกตัวที่มีผลต่อความแม่นยำให้ชัดเจนเสมอ พิมพ์ค่าจริงที่ใช้ออกมาตอนเริ่มรัน แล้ว assert ไว้ ค่า default ของไลบรารีที่เราไม่ได้เลือกเองคือบั๊กเชิง config ที่รอวันระเบิดในรอบรันที่แพงที่สุด

pass 0 ระบุประเภทหน้าโครงหลังคามืด ใน 8 จาก 11 หลัง ทำให้คานหลังคาหายจาก BOQ ทั้งหมดโดยไม่มีใครรู้

เกิดอะไรขึ้น: ขั้นตอน pass 0 ที่จำแนกประเภทหน้ากระดาน ระบุ label ผิดสำหรับแผ่นโครงสร้างหลังคาใน 8 จาก 11 หลัง เพราะ pass 1 ใช้คำตอบของ pass 0 เพื่อจัดภาพเข้าโฟลเดอร์ พอผิด หน้าพวกนั้นก็หลุดออกจากสายพานทั้งหมด ทำให้คานหลังคาทุกตัวหายไปจาก BOQ ที่ได้

ต้นเหตุจริง: pass 0 คือจุดเดียวที่ล้มเหลวแล้วพังหมด (single point of failure) ที่ไม่มีการตรวจสอบข้ามขั้นตอนถัดไปเลย หน้าการจัดประเภทผิดจะไม่ถูกติง มันแค่หายไปเงียบๆ ไม่มีอะไรมาเทียบ 'จำนวนหน้าที่เข้ามา' กับ 'จำนวนหน้าที่ถูกใช้จริง' หรือเช็คว่า element type ที่ควรมีปรากฏอยู่ในผลลัพธ์หรือไม่

เสียหายอะไร: คานหลังคาหายไปจาก bill of quantities ทั้งหมดใน 8 หลัง เป็นการขาดปริมาณงานแบบเงียบๆ ในผลลัพธ์ที่ส่งออกไปแล้ว ไม่ใช่ error ที่มองเห็นได้

แก้ยังง: บันทึกไว้ใน pass0/README.md ว่าเป็นขั้นตอนที่อันตรายที่สุดของสายพาน เปลี่ยนเป้าหมายการจำแนกให้ตอบเป็นปลายทาง subtask โดยตรง (plan_beam, section, ...) แทนที่จะตอบเป็น pattern เพื่อให้ pass 1 รู้แน่ชัดว่าแต่ละภาพควรไปไหน

กฎที่ได้: ขั้นตอนที่ทำ routing/จำแนกประเภทที่อาจทำให้ข้อมูลหายได้ ต้องมีการตรวจสอบยืนยันในขั้นถัดไปเสมอ: นับจำนวนหน้าที่เข้ามาเทียบกับที่ถูกใช้จริง และ assert ว่าประเภท element ที่ควรมีไม่ว่างเปล่า การระบุผิดประเภทต้องส่งเสียงเตือน ไม่ใช่แค่ทำให้ผลลัพธ์เล็กลงเรื่อยๆ

ตัวนำเข้าไฟล์บนเว็บรู้จักแค่ชื่อ pattern ตามสเปก (`plan`/`gridline``) ซึ่งไม่มีอยู่ในไฟล์จริงสักไฟล์ — กดนำเข้าแล้วได้ 0 element แบบเงียบๆ

เกิดอะไรขึ้น: ลองจำลองว่าผู้ใช้จริงอัปโหลดบ้าน 01 ผ่านหน้าเว็บ แล้วพบว่า `raw-extraction-adapter.js` รู้จักแค่ชื่อ pattern `plan`` กับ `gridline`` ที่เขียนไว้ในเอกสารสเปก แต่ไฟล์จริงทั้ง 50 หลังใช้ชื่อ `beam_plan`` (85 ไฟล์), `footing_plan`` (60), `etc_plan`` (199), `roof_frame_plan`` (46) รวม 390 ไฟล์ บวก `grid_master`` อีก 42 ไฟล์ ส่วนไฟล์ชื่อ `plan`/`gridline`` มี 0 ไฟล์ พอกดปุ่มนำเข้ากับข้อมูลจริงจึงได้ 0 element ไม่มีกริดเลย และไม่มีอะไรฟ้อง

ต้นเหตุจริง: adapter ถูกเขียนตามเอกสารสเปก ไม่ได้เขียนตามข้อมูลจริง แล้วข้อมูลตัวอย่างสำหรับเทสในโปรเจกต์ก็ใช้ชื่อที่สเปกคิดขึ้นเองเหมือนกัน เทสจึงเขียวหมดทั้งที่ทางเดินจริงตายสนิท ข้อมูลตัวอย่างที่ตรงกับสเปกแต่ไม่ตรงกับข้อมูลจริง จับความต่างระหว่างสเปกกับความจริงไม่ได้เลย

เสียหายอะไร: ปุ่มนำเข้า JSON ด้วยมือพังกับไฟล์จริงมาตลอดโดยไม่มีใครรู้ และทางเดินใหม่ของ Purson worker ก็จะสืบทอดค่าศูนย์เงียบๆ แบบเดียวกัน

แก้ยังง: แก้ที่ต้นเหตุ: เพิ่ม `canonicalPattern()` + `PATTERN_ALIASES` ใน adapter แล้วให้จุดที่เทียบ `f.pattern`` ตรงๆ ทั้ง 5 จุดใน 2 ไฟล์ รังผ่านตัวนี้หมด ปุ่มนำเข้าด้วยมือที่มีอยู่เดิมก็ได้ผลทันที เพิ่ม `tests/raw-extraction-pattern-aliases.test.js`` (4 เทส) หลังแก้: เรียกโมเดล 7 ครั้ง → 6 ไฟล์ → 24 element / 10 spec / 14 เส้นกริด npm test ผ่าน 1153/1153

กฎที่ได้: สร้างข้อมูลตัวอย่างสำหรับเทสจากข้อมูลจริงที่ใช้งานจริง ไม่ใช่จากเอกสารสเปก ถ้า fixture ใช้ค่าที่ไม่เคยปรากฏในข้อมูลจริงเลยสักครั้ง เทสนั้นไม่ได้พิสูจน์อะไรทั้งนั้น

ฐานรากที่ไม่มีขนาดเหล็กระบุ ออกมาเป็น '(DBNull)' และค่าเหล็ก \$0 โดยไม่มีธงเตือนเลย — อ่านเหมือนกับว่าฐานรากนั้นไม่มีเหล็กเสริมจริงๆ

เกิดอะไรขึ้น: ฐานราก F3/F4/F6 ไม่มี 'main_bar_dia_mm' ทำให้ 'computeBOQ' สร้างคำอธิบายที่มี '(DBNull)' และรายการเหล็กราคา \$0 โดยไม่มีธง 'needs_review' เลย บน BOQ ก่อสร้างจริง จุดนี้จะอ่านเหมือนกับว่า 'ฐานรากนี้ไม่ต้องมีเหล็กเสริม'

ต้นเหตุจริง: ข้อมูลที่ขาดหายไหลผ่านเส้นทางคำนวณราคาโดยไม่มีธงเตือนเลย 'computeBOQ' ไม่มีตัวกันสำหรับกรณีที่ไม่มีขนาดเหล็กระบุ ทำให้การขาดหายถูกแปลงเป็นค่าศูนย์ที่ดูเหมือนถูกต้อง แทนที่จะเป็นค่าที่ไม่ทราบ

เสียหายอะไร: อันตรายนมากสำหรับ BOQ ก่อสร้างจริง — ฐานรากสามตัวดูเหมือนไม่ต้องการเหล็กเสริมเลย เป็นตัวเลขผิดพลาดประเภทที่แย่ที่สุดที่จะหลุดออกไปใช้งานจริง

แก้ยังง: เพิ่มธง 'needs_review' / 'main_bar_missing' และเปลี่ยนข้อความเป็น '(ไม่ทราบขนาดเหล็ก — ตรวจสอบแบบ)' ยืนยันแล้วว่าตรงกับ F4/F6/F3 พอดี

กฎที่ได้: ข้อมูลที่ขาดหายต้องไม่แสดงผลเป็นราคาศูนย์หรือปริมาณศูนย์เด็ดขาด เส้นทางคำนวณราคาทุกเส้นทางต้องแยกแยะระหว่าง 'วัดได้ว่าเป็นศูนย์' กับ 'ไม่ทราบค่า' และต้องตั้งธงเตือนกรณีหลังให้เห็นชัดเจน — รายการโครงสร้างที่เจียบเป็นศูนย์คือปัญหาความปลอดภัย ไม่ใช่แค่ปัญหาการแสดงผล

npm test รันแค่ไฟล์เดียว ทำให้ชุด test ทั้งชุดไม่เคยถูกรันจริง และพังอยู่แบบเงิบๆ

เกิดอะไรขึ้น: ในโปรเจกต์ Constant 'npm test' ถูกตั้งค่าให้รันแค่ไฟล์เดียว แทนที่จะรันทั้งโฟลเดอร์ tests/ พอขยายให้รันทั้งโฟลเดอร์ พบว่า test ของ bbs-shape-allowance ไม่เคยถูกรันจริงเลย ต้องแก้ DOM stub และ float assertion ก่อนถึงจะผ่าน

ต้นเหตุจริง: ขอบเขตของ test runner ถูกจำกัดแคบลงตอนไหนสักตอน (อาจเพื่อความเร็ว หรือระหว่างรีแฟกเตอร์) แล้วไม่เคยถูกคืนกลับมาเลย ทำให้ไฟล์ test ใหม่ๆ ที่เพิ่มเข้ามาเนาคาไว้โดยไม่มีใครสังเกตว่าไม่ได้ถูกรันเลย ไม่มีอะไรรายงานจำนวน test ที่รันจริงเทียบกับจำนวนไฟล์ test ที่มีอยู่

เสียหายอะไร: ไม่รู้ว่าครอบคลุมค่าเผื่อรูปร่างเหล็ก (rebar shape allowance) เป็นระยะเวลานานแค่ไหนที่เป็นศูนย์ — การคำนวณนี้ป้อนข้อมูลให้ทั้งความยาวตัดและน้ำหนักเหล็ก test ที่พึ่งต้องถูกแก้ก่อนจะผ่านได้ แปลว่ามันพังมานานเท่ากับที่มันถูกมองข้ามมา

แก้ยังง: เปลี่ยน npm test ให้รันทั้งโฟลเดอร์ tests/ แก้ DOM stub และ float assertion ใน test bbs-shape-allowance และรวมโฟลเดอร์ test/ ที่หลงเหลืออยู่เข้ากับ tests/ ผลลัพธ์: ผ่าน 48/48

กฎที่ได้: คำสั่ง test ต้องรันทั้งโฟลเดอร์ชุด test เสมอ ห้ามระบุไฟล์แบบเจาะจงด้วยมือเด็ดขาด test ที่ไม่ถูกรันแยกว่าไม่มี test เลยเสียอีก — เพราะมันโฆษณาความครอบคลุมที่ไม่มีอยู่จริง ให้ยืนยันว่าจำนวนไฟล์กับจำนวนที่ถูกรันจริงต้องตรงกัน

สูง 2026-07-28

บ้าน07 S-03: มีสมาชิกโครงสร้าง 3 ตัวหายไปทั้งหมดในโซนบันได แกรมพื้น S1 ถูกนับเกินเป็น 4 ห้องที่จริงมี 3

เกิดอะไรขึ้น: พอเคลียร์โซนบันไดใหม่ พบสมาชิก 3 ตัวที่ไม่เคย extract เลย — B2 ตรงกลางช่วง 3-4, แผง SP ตัวที่ 5, และ B6 ที่ขอบ ST2 บวกกับพื้น S1 ที่นับผิดเป็น 4 ห้องที่จริงมี 3

ต้นเหตุจริง: โซนที่รกในแบบ (บันไดที่มีสัญลักษณ์หลายชนิดซ้อนกัน) ถูกอ่านผ่านไป zoom ปกติแทนที่จะเคลียร์ที่ละเอียดจนหมดของที่หายไปไม่มีสัญญาณเตือนอะไรเลย ต่างจากของที่ผิด

เสียหายอะไร: สมาชิกจริง 3 ตัวและพื้น 1 รายการที่นับเกิน หลุดจากข้อมูลปริมาณงาน — การขาดหายทำให้ BOQ ต่ำกว่าความจริง และไม่มีอะไรตรวจจับ element ที่ไม่เคยถูกบันทึกได้เลย

แก้ยังง: เคลียร์โซนใหม่ทั้งหมดด้วยการเปิดรูปจริงแล้วซูมดูทุกจุด แก้ไขพร้อมบันทึกการกลับค่าไว้

กฎที่ได้: โซนที่รกต้องเคลียร์ให้หมดจดที่ zoom เต็ม ที่ละสมาชิก element ที่หายไปมองไม่เห็นในทุกขั้นตอนถัดไป จึงต้องตั้งใจไล่หามันโดยเฉพาะ

สูง 2026-07-28

บั๊กที่ 3: max_seq_length ปลอยไว้ที่ค่า default 2048 ตัดขาดกลาง token ของภาพ โดยไม่มีตัวกันบน path ของ vision เลย

เกิดอะไรขึ้น: max_seq_length ถูกปลอยไว้ที่ค่า default 2048 ทั้งที่ภาพเดียวใช้ประมาณ 5,100 token ผลคือจุดตัดไปอยู่กลางบล็อก token ของภาพพอดี ฟัง audio ของโค้ดเดียวกันมีตัวกันเรื่องนี้อยู่แล้ว แต่ฟัง vision ไม่มี

ต้นเหตุจริง: มีขีดจำกัดความยาวตัวที่สองซ้อนอยู่นอกเหนือจาก max_length และมีคนดูแลแค่ตัวเดียว ไลบรารีป้องกัน modality หนึ่งแต่ไม่ป้องกันอีกตัว ความไม่สมมาตรนี้จึงมองไม่เห็นถ้าใครคิดว่าสองฝั่งเท่ากัน

เสียหายอะไร: ถ้าไม่จับได้ โมเดลจะได้รับ sequence ของ image token ที่ถูกตัดกลางภาพ ซึ่งแย่กว่าการตัดข้อความธรรมดาเพราะบล็อกภาพที่ขาดกลางไม่มีความหมายอะไรเลย

แก้ยังง: แก้ไปแล้วพร้อมกับบั๊กอีก 4 ตัวในรอบตรวจก่อนเข้าเครื่อง

กฎที่ได้: ต้องไล่ลิสต์ทุกขีดจำกัดเรื่องความยาว/ขนาดในสแตกให้ครบ ไม่ใช่แค่ตัวที่รู้จักอยู่แล้ว และถ้าไลบรารีกันบั๊กบน path หนึ่งแต่ไม่กันอีก path ให้สันนิษฐานไว้เลยว่า path ที่ไม่มีตัวกันคือ path ที่เรากำลังใช้อยู่

สูง 2026-08-24

บ้าน 08 หน้า 20 โมเดลพ่น boilerplate ที่ทองจามา โดย source_image ซื้ไปบ้านคนละหลัง

เกิดอะไรขึ้น: t03 บนหน้า plan_beam เขียนแค่ metadata แล้วปิด JSON โดยไม่มี elements[] เลย แกรม metadata เองยังอ้างถึงบ้าน 22 — โมเดลแปะ boilerplate จากตัวอย่างเทรนที่หน้าตาคล้ายกัน แทนที่จะอ่านแบบจริง

ต้นเหตุจริง: การทำ SFT บน JSON envelope ที่เกือบเหมือนกันทุกไฟล์ สอนให้โมเดลจำ envelope ได้ พอส่วนยาก (elements[]) ทำไม่ได้ โมเดลก็ยังมี header ที่ความน่าจะเป็นสูงให้พ่นออกมาอยู่ดี และไม่มีอะไรในรูปแบบ output ที่บังคับว่า header ต้องตรงกับรูปที่ป้อนเข้าไป

เสียหายอะไร: ได้ผลลัพธ์ที่ JSON ถูกต้อง ผ่าน grammar แต่ผิด 100% รวมถึงฟิลด์ที่บอกที่มาของตัวเอง เป็นโหมดลั้มเหลวที่แย่ที่สุดสำหรับคนใช้ปลายทาง เพราะมันดูเหมือนผลอ่านจริงของอาคารผิดหลัง

แก้ยังง: ยังไม่ได้แก้ — บันทึกไว้เป็นหลักฐานว่า plan_beam ที่ได้ 0% คือการทองจำ ไม่ใช่การอ่านได้บางส่วน

กฎที่ได้: เพิ่มตัวตรวจเชิงกลไกว่า metadata ของผลลัพธ์ (source_image/house) ต้องตรงกับไฟล์ที่ใช้สร้างมันจริง assert บรรทัดเดียวนับ output แบบทองจำ boilerplate ได้ ซึ่งทั้ง JSON validity และ grammar ปลอยผ่านไปหมด

สูง 2026-08-25

ADAPTED_PATTERNS ใน raw-extraction-adapter.js ยังไม่รับ roof_plan — บั๊กกลับมาได้แบบเสียบๆ

เกิดอะไรขึ้น: บั๊กคันหลังคาหายถูกปิดด้วยการแก้ฝั่งข้อมูล (เปลี่ยนแท็กไฟล์เป็น plan) แต่โค้ดฝั่งผู้บริโภคใน Constantant ยังอ่านแค่ pattern:"plan" และ ADAPTED_PATTERNS ไม่เคยถูกขยายให้รับ roof_plan เจอระหว่างรอบกวาด แต่จงใจไม่แก้

ต้นเหตุจริง: แก้แค่ชั้นข้อมูลอย่างเดียว ช่องโหว่ฝั่งโค้ดที่ทำให้การติดแท็กผิดพลาด (คือทั้ง pattern ที่ไม่รู้จักเสียบๆ แทนที่จะเตือน) ยังอยู่เหมือนเดิม โหมดลึ้มเหลวเลยแค่ลบบ่อย ไม่ได้ถูกกำจัด

เสียหายอะไร: บ้านที่สกดใหม่หลังจากนี้ ถ้าติดแท็ก roof_plan โครงหลังคาจะหายไปจาก BOQ อีกแบบเสียบๆ ไม่มีค่าเตือน — คือบั๊กเดิมเป๊ะ ที่ถูกงัดไถปิ่นไว้ใหม่

แก้ง่าย: ยังไม่ได้แก้ — บั๊กที่ไว้ในไดอารี่ว่าเป็นความเสี่ยงที่รู้ๆ อยู่และยังเปิดค้าง

กฎที่ได้: เวลาปิดบั๊กด้วยการล้างข้อมูล ให้ถามต่อว่าถ้าเจอ input เสียบแบบเดิมอีก โค้ดจะยังพึงเสียบไหม เส้นทางที่ทิ้งข้อมูลเสียบๆ ยังเป็นบั๊กอยู่ถึงข้อมูลตอนนี้จะสะอาดแล้ว ทำให้มันเตือนแทนที่จะหายไป

สูง 2026-08-25

ค่ากำลังคอนกรีต/เหล็ก (fc_ksc, fy_*) ไม่เคยไปถึงผู้ใช้ปลายทางเลยสักบ้าน

เกิดอะไรขึ้น: fc_ksc, fy_main_ksc, fy_stirrup_ksc มีอยู่ใน sections[] ต้นทางของไฟล์ JSON แต่ไม่เคยถูก parse เข้า notes() ซึ่งเป็นที่ที่ผู้บริโภคอ่านจริง ค่า 62 ค่าจาก 26 ไฟล์เพ่งไปถึงผู้บริโภคเป็นครั้งแรกตอนรอบกวาดของเชสชันนี้

ต้นเหตุจริง: ชันสกดเขียนค่าลง section array แบบอิสระ แต่ไม่เคยมีชัน parse ไหนยกค่าขึ้นไปใส่ฟิลด์ที่มีโครงสร้างซึ่งผู้บริโภคอ่านจริง ช่องว่างนี้เขียนไว้ใน spec (S4a) แล้ว แต่ไม่มีใครลงมือ

เสียหายอะไร: ทุกการคำนวณปลายทางที่ต้องใช้ชันคุณภาพคอนกรีต/เหล็ก รันโดยไม่มีค่าจริงจากแบบมาตลอดทั้งประวัติโปรเจกต์จนถึงวันนี้

แก้ง่าย: parse ค่า 62 ค่าจาก sections[] ของไฟล์เอง เข้าไปใน notes() โดยเก็บข้อความที่พิมพ์อยู่บนแบบไว้ตรงๆ ใน notes.parsed_as

กฎที่ได้: การสกดค่าออกมาได้ ไม่เท่ากับการส่งค่าไปถึงคนใช้ ทุกฟิลด์ที่ spec กำหนด ให้พิสูจน์ว่ามีผู้บริโภคน้อยหนึ่งรายอ่านมันจริงตั้งแต่ต้นจนจบ โหนดที่เขียนว่า 'ไม่เคยไปถึงผู้บริโภค' คือบั๊กที่เปิดค้าง ไม่ใช่เชิงอรรถ

สูง 2026-08-25

รูปใน Markdown หายไปจาก PDF ที่ generate แบบเสียบๆ

เกิดอะไรขึ้น: tools/md_to_book_pdf.py เขียน HTML ตัวกลางลง temp directory ทำให้ path รูปแบบ relative ทุกอันชี้ไปที่ว่างเปล่า Chromium ชำมรูปที่หาไม่เจอโดยไม่ error PDF เลยออกมาโดยมีรูป 0 จาก 4 รูป และไม่มีค่าเตือนอะไรเลย

ต้นเหตุจริง: ทางเลี้ยงที่ตั้งใจทำ (ใช้ temp dir เพื่อหนีปัญหา Chromium จัดการ path ภาษาไทย/มีช่องว่างบน Windows ไม่ถูก) ไปทำลายข้อสันนิษฐานเรื่อง relative path ของแท็กรูป และโหมดลึ้มเหลวของตัว render เมื่อหารูปไม่เจอคือเสียบ ไม่ใช่ error

เสียหายอะไร: PDF สอนงานที่ผลิตจากเครื่องมือกลางตัวนี้ไม่มีรูปเลย และข้อบกพร่องนี้มองไม่เห็นจนกว่าจะมีคนเปิด PDF ดู ไม่รู้ด้วยว่าเอกสารก่อนหน้านี้ออกไปโดยไม่มีรูปकिनับ

แก้ง่าย: เพิ่ม absolutize_images() — เขียน src แบบ relative ใหม่ให้เป็น file:// URI เติมด้วย Path.as_uri() (ซึ่ง percent-encode ชื่อไฟล์เตอร์ภาษาไทยให้ด้วย) ปลอย http/data/file/absolute ไว้เหมือนเดิม และพิมพ์ค่าเตือนเมื่อไฟล์หายจริง ตรวจแล้วได้รูปฝังเข้าไป 0 → 4 รูป

กฎที่ได้: พอย้ายไฟล์ไป temp dir ทุก reference แบบ relative ในไฟล์นั้นทั้งหมด และห้ามยอมรับ renderer ที่กลืน asset ที่หายไปเสียบๆ ให้ใส่ค่าเตือนชัดๆ เพื่อให้ไฟล์ที่หายพังเสียงดัง ไม่ใช่ปลอ่ยรูปหว่าออกไป

สูง

confidence_flags, material_amount, labor_amount หายไปเสียบบๆ ตอนย่อสคีมามาจาก v2 เป็น v3

เกิดอะไรขึ้น: ตอนย่อ field/config จากสคีมามา v2 เป็น v3 มีสามฟิลด์ — confidence_flags, material_amount, labor_amount — หายไปเสียบบๆ จนผู้ใช้งานต้องเป็นคนจับตัวเอง

ต้นเหตุจริง: ไม่ได้ตรวจสอบรายการฟิลด์ทั้งหมดก่อน-หลังตอนย่อสคีมามา ฟิลด์จึงหายไปโดยไม่มีใครตัดสินใจตัดออกจริงๆ

เสียหายอะไร: ฟิลด์ข้อมูลจริงที่จำเป็น (การติดตามความมั่นใจ, ยอดต้นทุน) หายไปจากสคีมาย่อยแล้ว จนกว่าผู้ใช้จะจับได้

แก้ยังไม: ตั้งกฎการ: ทุกครั้งที่ย่อ field/config ต้องตรวจสอบรายการฟิลด์ทั้งหมดก่อน-หลังให้ครบ ไม่ให้ฟิลด์ไหนหายไปเสียบบๆ

กฎที่ได้: ตอนย่อ field/config ต้องตรวจสอบรายการฟิลด์ทั้งหมดก่อน-หลัง — อย่าปล่อยให้ฟิลด์หายไปเสียบบๆ

เรื่องที่เหลือในหมวดนี้ (7)

ฟิลด์ใหม่ spec_source กับ spec_confidence_score ไม่มีช่องให้แก้ คนตรวจเลยแก้ค่าไม่ได้

กลาง 2026-07-06

การเพิ่มฟิลด์ลง schema ยังไม่ถือว่าเสร็จ จนกว่าฟิลด์นั้นจะแก้ไขได้ในหน้าตรวจ ถึงรวมแบบอ่านอย่างเดียวมีไว้กันข้อมูลหาย ไม่ใช่ที่อยู่ของฟิลด์ที่ต้องให้คนแก้

สเปคจากแผ่น section กับแบบคานขัดแย้งกันเรื่องจำนวนเหล็กเสริมพิเศษของ B4 ยังไม่ได้แก้

กลาง 2026-07-15

การติดธงเมื่อพบข้อมูลขัดแย้งจากสองแหล่งแทนการเดาถือว่าถูกแล้ว แต่ธงนั้นต้องมีเจ้าของและทางแก้ ไม่งั้นมันจะกลายเป็นความคลุมเครือถาวรในชุดข้อมูลฝึกโมเดล

Dummy grid B' บันทึกลงเป็น 10.2 ทั้งที่ค่าจริงคือ 10.1 ทำให้ span ของคานที่คำนวณต่อผิดตามไปด้วย

กลาง 2026-07-20

ตำแหน่ง dummy grid ทุกจุดต้องคำนวณจากสายวัดระยะ (dimension chain) อิสระอย่างน้อย 2 สาย แล้วต้องให้ผลตรงกันก่อนถึงจะบันทึก — สายวัดสายเดียวไม่มีทางตรวจจับความผิดพลาดได้เอง

ค่าที่พิมพ์ผิบบนแบบ ('HITASHI') ถูกแก้เป็น 'HITACHI' เสียบบๆ ตอนสกัดข้อมูล

กลาง 2026-07-28

คัดลอกจากเอกสารต้นฉบับตามตัวอักษรเป๊ะๆ รวมถึงจุดที่ผิดชัดเจนด้วย ถ้าจะแก้ ให้บันทึกไว้เป็นอีกช่องหนึ่งพร้อมหมายเหตุ ห้ามแก้ไขให้ดีขึ้นเองแบบเสียบบๆ

pass 2.5 ที่เก็บข้อมูลเองกู้คืนชนิดชิ้นส่วนที่คลั่งเหมเพลตกลางไม่มีเลยไม่ได้

กลาง 2026-08-29

บันทึกจุดบอดเชิงโครงสร้างของกลไกกู้คืนทุกตัวไว้ให้ชัด และเพิ่มการเฝ้าดูกรณีจุดบอดโดยเฉพาะ (หน้าที่ CV ตรวจไม่เจออะไรเลย) เพื่อส่งหน้าพวกนั้นให้คนตรวจ แทนที่จะปล่อยผ่านไปเสียบบๆ

หน้าจอ loading ที่สร้างขึ้นใหม่ทั้งหมด ไม่เคยถูกเปิดดูในเบราร์เซอร์จริงเลย

กลาง 2026-09-01

ห้ามให้การตรวจสอบ UI ต้องรอฟังก์ชันจริง ให้เพิ่ม dev flag หรือจำลองสถานะความคืบหน้าปลอมไว้ เพื่อให้ตรวจดู UI ที่เกี่ยวกับการโหลด/รอได้ตามต้องการ ไม่งั้นมันจะถูกปล่อยออกไปโดยไม่เคยมีใครเห็นเลย

รูปขนาด 1400px จะล้นคอสม์นข้อความ A4 เพราะไม่มีกฎ CSS สำหรับ img เลย

ต่ำ 2026-08-25

การแก้ path โค้ดที่ตายเสียบบมานาน จะเปิดโปงบั๊กที่ไม่เคยถูกทดสอบซ่อนอยู่หลังมันทั้งหมด หลังเปิดใช้ path นั้นครั้งแรก ให้รีวทั้ง path ไม่ใช่แค่บรรทัดที่แก้

วิธีวัดผล และการประเมิน

ตัววัดที่วัดผิดสิ่ง เทียบกันแบบไม่แฟร์ หรือออกแบบการวัดที่ตอบคำถามไม่ได้

ทั้งหมด 94 เรื่อง · วิกฤต 23 · สูง 33 · กลาง 31 · ต่ำ 7

เรื่องที่ต้องอ่านให้ครบ (56)

วิกฤต 2026-07-28

บักยอรูปลือ 512px แบบเงียบๆ ถูกแก้แค่ในสคริปต์ของ t02 ทำให้ t01 เทรนด้วยรูปที่เบลอกว่าประมาณ 14 เท่า

เกิดอะไรขึ้น: ระหว่าง Phase 0.4 เจอบักว่าถ้าไม่ระบุ resize="max" ไว้ ตัว collator จะยอรูปลือ 512px เงียบๆ แล้วแก้เฉพาะในสคริปต์เทรนของ t02 เท่านั้น ไม่มีใครย้อนกลับไปดูสคริปต์ของ t01 (ที่เขียนไว้ก่อนหน้าเป็นเดือน) ซึ่งมีข้อบกพร่องเดียวกัน พอวัดจริงยืนยันว่า t01 เทรนที่ประมาณ 266 visual token ต่อรูป ขณะที่ t02 อยู่ที่ประมาณ 3,796-5,100

ต้นเหตุจริง: แก้บักเฉพาะจุดที่เห็นมันตอนนั้น ไม่ได้แก้ทุกจุดที่ใช้เส้นทางโค้ดเดียวกัน ข้อบกพร่องนี้มองไม่เห็นเพราะมันมาจากค่าเริ่มต้นที่ไม่ได้ตั้ง เลยไม่มีบรรทัดโค้ดให้อามาเทียบ

เสียหายอะไร: การเปรียบเทียบ t01 กับ t02 ทั้งหมด — ซึ่งเป็นเหตุผลทั้งหมดที่เข้าเครื่องมาทำ t02 — กลายเป็นการเทียบที่ไม่ยุติธรรม ข้อสรุปที่ว่า "Qwen3.6 ชนะ Qwen3-VL" ต้องลดระดับลงเป็น "Qwen3.6 ที่ได้รูปเบลอกว่ามาก ก็ยังชนะ Qwen3-VL" ซึ่งไม่ได้ตอบคำถามที่ตั้งไว้แต่แรก

แก้งัง: ตรวจสอบด้วยการโหลดโมเดลจริงแล้วรัน collator ของ t01 กับรูปจริง จับ log ของไลบรารีเองได้ว่า "Model does not have a default image size - using 512" มะขามเลือกทางเลือก A คือยอมรับผลลัพธ์พร้อมใส่หมายเหตุกำกับไว้ทุกที่ แทนที่จะจ่ายเงินเทรน t01 ใหม่

กฎที่ได้: พอเจอบัก ให้ grep หาทุกจุดที่เรียกใช้แบบเดียวกันก่อนปิดงาน — การแก้ที่ขี้เกียจที่สุดคือแก้ในฟังก์ชันที่ใช้ร่วมกัน ไม่ใช่แก้ไขในไฟล์ที่บังเอิญกำลังอ่านอยู่ และให้พิสูจน์การวินิจฉัยด้วยการวัดจริง ไม่ใช่ด้วยการอ่านโค้ด

การทดลอง A/B ทั้งหมดใช้ไม่ได้: t01 เทรนด้วยภาพที่เบลอกว่าราว 14 เท่า เพราะ collator ถูกเรียกด้วยค่า default ล้วน และตารางเทียบตัวแปรไม่มีบรรทัดของมันเลย

เกิดอะไรขึ้น: หลังผลทั้งวันออกมาแล้ว การตรวจสอบด้วย 43 agent พบว่า train_qwen36.py บรรทัด 164 (ฝั่ง t01) เรียก collator โดยไม่ใส่ resize= เลย — ซึ่งเป็นบั๊กตัวเดียวกับที่เคยเจอและแก้ไปแล้วแต่แก้เฉพาะฝั่ง t02 วัดจริงบนเครื่องเช่า: t01 เทรนที่ราว 266 visual token ต่อภาพ ขณะที่ t02 อยู่ที่ราว 3,796 — ต่างกันประมาณ 14 เท่า ส่วนตารางเทียบตัวแปร 16 บรรทัดที่สร้างขึ้นมาเพื่อคุมตัวแปรโดยเฉพาะ กลับไม่มีบรรทัดของ collator เลย

ต้นเหตุจริง: ตารางเทียบตัวแปรเปรียบเทียบเฉพาะค่าที่ตั้งใจกำหนดเอง ค่าที่ไม่เคยพิมพ์ลงไปเลยจึงไม่มีบรรทัดให้เทียบ ทำให้ค่า default ที่ต่างกันมองไม่เห็นโดยโครงสร้างของการตรวจแบบนั้น — และบั๊กตัวที่ 2 ก็ถูกแก้เดินหน้าไปในสคริปต์ใหม่ โดยไม่มีใครย้อนกลับไปดูว่าสคริปต์ตัวเก่ามีบั๊กแบบเดียวกันหรือเปล่า

เสียหายอะไร: ข้อสรุปพาดหัวของวันนั้น ('Qwen3.6 ชนะ Qwen3-VL') ต้องถอนออก แล้วลดเหลือแค่ 'Qwen3.6 ถึงจะเห็นภาพเบลอกว่ามาก ก็ยังทำได้ดีกว่า' คำถาม A/B ที่เป็นเหตุผลของการทำรอบนี้ทั้งรอบยังไม่ได้คำตอบ การเทรนที่จ่ายเงินไปบวกกับการวิเคราะห์เพิ่มเติมวัน ได้ผลเปรียบเทียบที่ตีความไม่ได้

แก้ยังง: พิสูจน์ด้วยการวัดจริง ไม่ใช่การอ่านโค้ด (โหลด Qwen3.6 แล้วสร้าง collator แบบเดียวกับ t01 ระบบพิมพ์ออกมาว่า 'Model does not have a default image size - using 512') ใส่กล่องเตือนตัวใหญ่ไว้ใน t02_workflow.md ที่เฟส 7 และ \$0.0 ก่อนคำว่า 'ชนะขาด' ออก และเพิ่ม Lesson 12 ลง rule_of_tune.md มะขามเลือกทางเลือก A (ยอมรับผลพร้อมค่าเตือน) แทนทางเลือก B (เทรน t01 ใหม่ด้วย resize='max' ราว \$0.60-1.80)

กฎที่ได้: ตารางคุมตัวแปรต้องใส่ทุก argument ที่ส่งเข้าไปในฟังก์ชันหรือคลาสที่ใช้ร่วมกัน โดยอ่านจากซอร์สจริง ไม่ใช่แค่ค่าที่เราเลือกจะเปลี่ยน ค่า default ที่ไม่ได้ตั้งก็เป็นตัวแปรเหมือนกัน และมันคือตัวที่พลาตาง่ายที่สุดเพราะไม่มีบรรทัดให้เทียบ อีกข้อ: เจอบั๊กในสคริปต์ตัวหนึ่งเมื่อไร ให้ไปไล่เช็คสคริปต์พี่น้องทุกตัวว่ามีรูปแบบเดียวกันไหมทันที

ตัวเลขพาดหัว 90% / 28.2% วัดมาจากของที่ไม่มีใครเอาไปรันจริง

เกิดอะไรขึ้น: ตัวเลขพาดหัวของทิม (JSON ถูกต้อง 90%, element recall 28.2%) มาจาก eval_fields.py ซึ่งโหลด bf16 + adapter ตอนรันด้วย load_in_4bit=False คือไม่มีขั้น merge และไม่มีการ quantize เลย แต่การใช้งานจริงรันด้วย GGUF ที่ merge แล้ว + Q4_K_M ซึ่งไม่เคยมีใครวัดความแม่นยำเลยสักครั้ง

ต้นเหตุจริง: การประเมินผลทำบนเส้นทางที่สะดวก (full precision, adapter ที่ยังไม่ merge) แทนที่จะทำบนเส้นทางที่ใช้งานจริง และสองเส้นทางนี้ต่างกันตรงสองขั้นตอนที่มีโอกาสทำคุณภาพตกมากที่สุดพอดี คือ merge กับ quantize

เสียหายอะไร: ตัวเลขคุณภาพที่ทีมยกมาอ้างบ่อยที่สุด อธิบายการตั้งค่าที่ไม่เคยถูกส่งมอบจริง การวางแผน ความมั่นใจ และการตัดสินใจไปต่อ/ไม่ไปต่อ ล้วนตั้งอยู่บนตัวเลขที่ใช้กับระบบที่รันอยู่จริงไม่ได้

แก้ยังง: แก้ไปบางส่วน — บันทึกความไม่ตรงกันนี้ไว้แล้วและระบุขอบเขตตัวเลขใหม่ชัดๆ ว่า 'ใช้ได้กับ adapter เท่านั้น' ส่วน GGUF ยังไม่มีตัวเลขความแม่นยำที่วัดมาเลย

กฎที่ได้: ตัวเลขความแม่นยำทุกตัวต้องแนบไว้เสมอว่า 'วัดจากอะไร' และต้องทำให้ชุดประเมินผลโหลดของขั้นเดียวกับที่ส่งมอบจริง ถ้า eval กับ production โหลดคนละไฟล์ ตัวเลข eval นั้นก็เป็นตัวเลขของโมเดลคนละตัว

ผ่านการเทรนเต็มรอบ export GGUF และใช้งานจริงแล้ว แต่การสกัดข้อมูลหลักเสริม — ซึ่งคือเป้าหมายจริง — ไม่เคยถูกทดสอบเลยสักครั้ง

เกิดอะไรขึ้น: ยังไม่เคยมีแบบโครงสร้าง (แผ่น S-*) แผ่นไหนที่ถูกสกัดข้อมูลจนครบเลย หน้า 07-11 ทำแค่จำแนกประเภทเท่านั้น กฎที่ใหญ่ที่สุดข้อเดียวใน prompt ('ต้องแยกหลักบน/หลักกลางเสมอ') ไม่มีหลักฐานทั้งฝั่งสนับสนุนและฝั่งค้าน

ต้นเหตุจริง: หน้าที่ใช้ประเมินผลและหน้าที่ใช้งานจริงถูกเลือกตามลำดับหน้า (01, 02, 03...) แทนที่จะเลือกตามความสำคัญต่อเป้าหมาย หน้าที่มีค่ามากที่สุดจึงไปอยู่ท้ายคิวเสมอและไม่เคยถูกทำถึง

เสียหายอะไร: ทุกคำกล่าวอ้างเกี่ยวกับความมีประโยชน์ของโมเดลต่อจุดประสงค์หลักของโปรเจกต์ ตั้งอยู่บนการวัดผลศูนย์ครั้ง งานหลายสัปดาห์ยังไม่ได้ตอบคำถามหลักเลย

แก้ยังไง: ยังไม่แก้ บันทึกไว้เป็นคำถามค้างที่ยังเปิดอยู่

กฎที่ได้: ทดสอบเคสที่มีค่ามากที่สุดก่อน ไม่ใช่เคสแรกตามลำดับ ถ้าจบครบหนึ่งรอบแล้วความสามารถหลักยังไม่ถูกทดสอบเลย แปลว่ารอบนั้นวัดผิดเรื่อง

t02 เห็นหน้า beam plan ตอนเทรนแค่ ~13 หน้า แล้วได้ element recall 11% บนหน้าพวกนั้น

เกิดอะไรขึ้น: พอับข้อมูลเทรนจริงของ t02 พบว่ามันเห็นหน้า beam plan แค่ ~13 หน้า เทียบกับ material_list ~158 หน้า และตารางประตู-หน้าต่าง ~24 หน้า ผลที่วัดได้ก็วิ่งตามจำนวนตัวอย่างจริงๆ เลย: ตารางประตู-หน้าต่าง 86% ส่วน beam plan 11% แกม beam plan ยังเป็น pattern ที่ยากที่สุดด้วย (~25 คานต่อหน้า แต่ละตัวต้องได้ grid ref ทั้งจุดเริ่มและจุดจบถูกต้อง)

ต้นเหตุจริง: สัดส่วนของชุดข้อมูลเทรนสะท้อนจำนวนหน้าดิบในแบบก่อสร้าง แทนที่จะถูกจัดสมดุลให้เอนไปทาง pattern ที่ยากที่สุด และมีค่าที่สุด pattern ที่ยากที่สุด + ตัวอย่างน้อยที่สุด = คู่ที่แย่ที่สุดเท่าที่จะเป็นไปได้ แล้วก็ไม่มีใครนับจำนวนตัวอย่างแยกราย pattern ก่อนเทรนเลย

เสียหายอะไร: beam plan ซึ่งเป็น pattern เดียวที่ BOQ พังพามากที่สุด ทำ recall ได้แค่ 11% การเดโมวงจรการสกัดข้อมูลทุกอย่างที่อยู่ปลายทางจึงติดคอขวดตรงนี้ grammar-constrained decoding แก่เรื่องรูปแบบได้ (JSON ที่ถูกต้อง 57.9%→96.8%) แต่ความแม่นยำไม่ขยับเลย

แก้ยังไง: ยังไม่ได้แก้ด้วยโค้ด เขียน dataset_sizing.md พร้อมตัวเลขจริง ตั้งเป้าไว้ที่ 25-30 หลัง (~99 หน้า beam plan) และสร้าง op04 ขึ้นมาให้ทำ annotation เฉพาะ pattern ของ Pass 2 เพื่อให้แรงงานไปลงที่ beam plan แทนที่จะไปลงกับ material list

กฎที่ได้: นับจำนวนตัวอย่างเทรนแยกตาม pattern ของหน้า ก่อนเทรน แล้วจัดสมดุลใหม่ให้เอนไปทาง pattern ที่ยากที่สุดและมีค่าที่สุด — ความสัมพันธ์ระหว่างจำนวนตัวอย่างกับความแม่นยำตรงและวัดได้จริง (13→11%, 24→86%) การสำรวจจำนวนแยกราย pattern จึงเป็นสิ่งที่ต้องทำก่อน fine-tune ไม่ใช่เครื่องมือไว้ชั้นสุดสทฟทีหลัง

บ้าน 08 ที่ใช้เป็น benchmark ดันอยู่ในชุดเทรนของ t03 — ถ้าไม่จับได้ การเทียบข้ามรอบจะไม่มีความหมายเลย

เกิดอะไรขึ้น: เลือกบ้าน 08 มาเป็น benchmark เทียบข้ามรอบเพราะมีผลของ t02 กับ base-Qwen อยู่แล้ว แต่บ้าน 08 อยู่ในชุด *เทรน* ของ t03 ในขณะที่ t02 ไม่เคยเทรนบนมันเลย ตัวเลขของ t03 จึงจะพุ่งขึ้นเพราะการท่องจำ และเทียบกันไม่ได้เลย

ต้นเหตุจริง: เลือกบ้านที่ใช้ทำ benchmark เพราะสะดวก มีผลเก่าอยู่แล้ว โดยไม่เช็คว่ารอบปัจจุบันมันตกอยู่ฝั่งไหนของการแบ่งชุดข้อมูล ไม่เคยมีใครเอาการเป็นสมาชิกของชุด train/val ไปตรวจไขว้กับแผนการเปรียบเทียบ

เสียหายอะไร: จับได้ก่อนที่จะเผยแพร่ผล — แต่ถ้าพลาดไป การเทียบ t02 กับ t03 ทั้งชุดจะใช้ไม่ได้เลย แล้วตัวเลข t03 ที่พุ่งขึ้นจะถูกนำเสนอเป็นความก้าวหน้าจริง

แก้ยังง: ย้ายบ้าน 08 ไปอยู่ฝั่ง val แล้วเทรนใหม่: train 887→854 (42 หลัง), val 229→262 (6 หลัง), อัตราส่วน 3.26:1 ตอนนี้บ้าน 08 ไม่เคยถูกเห็นโดยทั้งสามรอบจริงๆ แล้ว

กฎที่ได้: บ้านที่ใช้เป็น benchmark เทียบข้ามรอบ ต้องยืนยันว่ากันออกจากชุดเทรนใน *ทุกรอบ* ที่เอามาเทียบ ไม่ใช่แค่รอบปัจจุบัน เช็คก่อนเลือก benchmark ไม่ใช่เช็คทีหลัง

section 0% ส่วนใหญ่มาจากตัวชี้วัดที่พัง ไม่ใช่โมเดลที่พัง — AI ที่สมบูรณ์แบบก็ทำได้สูงสุดแค่ 4%

เกิดอะไรขึ้น: การประเมิน section เทียบสตริง element_id แบบตรงตัว แต่ ID ใน ground truth ของ section เป็นป้ายที่คนคิดขึ้นเอง (gate_front, door_lock_detail, SP_at_bathroom, purlin_rafter_connection, ห้องนอน) ซึ่งไม่ปรากฏบนแบบเลยสักที่ วัดเพดานได้: section 37% ในภาพรวม และแค่ 1 จาก 23 = 4% บนบ้าน 08 ซึ่งเป็นหลังที่ใช้ประเมินจริง ส่วน notes ยิ่งแย่กว่าที่ 2%

ต้นเหตุจริง: ID ใน ground truth ถูกสร้างขึ้นเพื่อความสะดวกในการจดบันทึกของคน แล้วถูกเอามาใช้เป็นกุญแจในการประเมินจริงๆ โดยไม่เคยเช็คเลยว่าข้อมูลที่ต้องใช้สร้าง ID พวกนั้นมีอยู่บนภาพต้นทางไหม

เสียหายอะไร: ตัวเลขพาดหัว 0% ถูกอ่านว่า 'โมเดลอ่าน section ไม่ได้' และกำลังจะพาไปสู่การไปแก้โมเดล/prompt ต่อ ซึ่งอาจกลายเป็นการเทรนเสียเงินอีกทั้งรอบเพื่อสิ่งที่ไม่มีโมเดลไหนทำได้ยุดี

แก้ยังง: เขียน measure_id_ceiling.py ขึ้นมาแยก ID ออกเป็นกลุ่มที่อ่านได้จากแบบ กับกลุ่มที่คนคิดขึ้นเอง แล้วคำนวณเพดานที่เป็นไปได้แยกตาม subtask ผลคือจัดประเภท section 0% ใหม่ว่าเป็นความล้มเหลวของการวัด ส่วน plan_beam 0% (เพดาน 88%) เป็นความล้มเหลวของโมเดลจริงๆ

กฎที่ได้: ก่อนจะไป optimize ตามตัวชี้วัดใดๆ ให้คำนวณเพดานที่เป็นไปได้ของมันเป็นชุดประเมินชุดนั้นๆ เสียก่อน ตัวชี้วัดที่มีเพดานอยู่ที่ 4% ไม่ได้กำลังวัดคุณภาพของโมเดล แต่กำลังวัดธรรมเนียมการทำ annotation ต่างหาก

เอกสารบอกว่าไม่เคยย่อภาพเลย แต่วัดจริงพบว่าย่อทุกภาพ 100%

เกิดอะไรขึ้น: `t03\_workflow.md` แถวที่ 7 บันทึกไว้ว่า "ภาพจริง 2339×1654 ต่ำกว่า cap 5,120 อยู่แล้ว = "ไม่ย่อเลย" แต่พอวัดตรงๆ พบว่าภาพทั้ง 973/973 รูป (100%) เกิน cap หมด ภาพเล็กสุดใน train.jsonl คือ 3307×2339 (7,554 token) และภาพใหญ่สุด 4631×3473 (15,707 token เสียรายละเอียดไป 67%) ไม่มีภาพขนาดที่บันทึกไว้เดิมอยู่ใน train set ปัจจุบันเลย

ต้นเหตุจริง: บันทึกเดิมได้มาจากการดูตัวอย่างเดียว (`train\_ds[0]`) แล้วสรุปเหมารวมทั้งชุดข้อมูล ซึ่งเป็นความผิดพลาดแบบเดียวกับที่เคยบันทึกไว้แล้วใน rule_of_tune ข้อ 12 และเกิดซ้ำอีกครั้ง

เสียหายอะไร: ทีมเชื่อว่าความละเอียดภาพไม่ใช่คอขวด แล้วตัดสินใจเรื่องต่อไปบนความเชื่อนั้น ทั้งที่จริงภาพ training ทุกภาพถูกย่อ บางภาพเสียรายละเอียดไปถึง 67%

แก้ยังไ: สร้าง `measure\_capacity.py` (read-only ไม่ใช้ GPU) คำนวณ visual token ของทุกภาพในชุดข้อมูลจากขนาดไฟล์จริง แล้วบันทึกข้อขัดแย้งนี้ไว้ในเอกสาร workflow แทนที่จะเขียนทับบรรทัดเดิมเสียๆ

กฎที่ได้: ห้ามสรุปลักษณะของชุดข้อมูลจากตัวอย่างเดียว ข้อความอ้างว่า "ภาพขนาด X" ต้องมาจากสคริปต์ที่วัดทุกไฟล์จริงและรายงาน min/max/จำนวนที่เกิน cap

การให้คะแนน id เทียบสตรงตรงเป๊ะ เว้นวรรคเกินหรือตัวพิมพ์ต่างกันก็ได้ 0 ทั้งที่ตอบถูก

เกิดอะไรขึ้น: `infer\_house\_t03.py:218` ให้คะแนนด้วย `set(gt\_ids) & set(pred\_ids)` เทียบสตรงตรงเป๊ะ ไม่มี normalize เลย โมเดลที่ตอบ "B1 " เทียบกับคำตอบจริง "B1" จะได้คะแนน 0 ตรวจสอบด้วยตัวเองยืนยันว่าโมเดลที่ตอบถูกทุกอย่างแต่มีเว้นวรรคเกิน/ตัวพิมพ์ต่างจะได้ 0% ภายใต้ตัวให้คะแนนแบบเก่า

ต้นเหตุจริง: ตัวให้คะแนนถูกเขียนแบบ set intersection บรรทัดเดียว ไม่มีขั้น normalize เลย ราวกับว่าเอาต์พุตของโมเดลเป็นข้อมูลมาตรฐานอยู่แล้ว ทั้งที่จริงมันคือข้อความอิสระ

เสียหายอะไร: ตัวเลข recall ที่รายงานไว้ตลอดประวัติรอบ t02/t03 ต่ำกว่าความสามารถจริงของโมเดลโดยไม่รู้ตัวต่างกันเท่าไร การตัดสินใจปรับจูนต่างๆ ถูกทาบบน metric ที่หลงโทษความรกของ format ราวกับเป็นความผิดพลาดในการอ่านแบบ

แก้ยังไ: ศึกษามาตรฐานก่อน (แนวทาง KIE evaluation, KIEval arXiv:2503.05488 ที่รายงานทั้ง exact และ relaxed matching แยกคอลัมน์) แล้วเพิ่ม `norm\_id()` (ตัดช่องว่าง / ยุบช่องว่างซ้ำ / casefold) ก่อนเทียบ ค่าคะแนนแบบเก่ายังเก็บไว้เป็นคอลัมน์เพื่อเทียบกับรอบก่อนๆ ได้

กฎที่ได้: ตัว metric ที่เทียบข้อความจากผลลัพธ์โมเดลต้องมีขั้นตอน normalize ชัดเจน และควรรายงานทั้งค่า normalize แล้วกับค่าดิบคู่กันเสมอ ก่อนจะประดิษฐ์ตัวให้คะแนนขึ้นมาเอง ให้เช็คแนวทางมาตรฐานของงานประเภทนั้นก่อน

ground truth element_id ปณมารค์จริงบนแบบกับชื่อที่ผู้ตรวจแท็กเดาขึ้นเอง ทำให้โมเดลถูกสอนให้เดาส่งที่เดาไม่ได้

เกิดอะไรขึ้น: GT `element\_id` มีทั้งมาร์คที่พิมพ์จริงบนแบบ (`B1`, `B3X`, `C1`, `F1A`, `RB1`) และชื่อบรรยายที่ผู้ตรวจแท็กคิดขึ้นเองเพื่อแยกรายละเอียดย่อย (`gate\_front`, `SP\_at\_bathroom`, `purlin\_rafter\_connection`, `door\_lock\_detail`, `ห้องนอน`, `ระดับพื้นอาคาร`) ไม่มีโมเดลไหนผลิตค่าพวกนี้ได้ เพราะไม่มีอะไรบนกระดาษที่เขียนไว้แบบนั้น วัตถุประสงค์ 300/455 ตัวอย่าง training ของ subtask section (66%) สอนให้โมเดลเดาชื่อที่ถูกแต่งขึ้น ครึ่งหนึ่งของนั้นแต่งขึ้นทั้ง 100%

ต้นเหตุจริง: §0.2 ของ schema สั่งให้ผู้ตรวจแท็กแต่งชื่อบรรยายขึ้นเองจริงๆ และห้ามเว้นคำว่า ธรรมเนียมที่ทำให้ field นี้เรียนรู้ไม่ได้ถูกเขียนไว้ในสเปกเลย

เสียหายอะไร: id-recall ของ subtask section ติดที่ 0% โดยเพดานจริงอยู่ที่แค่ 37% ทั้งชุดข้อมูล และ 4% ในบ้านทดสอบ ตัวเลขนี้แทบไม่มีความหมายอะไรเลย ที่แย่กว่าคือ 66% ของตัวอย่าง training กำลังสอนให้โมเดลหลงเชื่อขึ้นมาเองอยู่

แก้ยังไง: หลังได้รับอนุมัติชัดเจน แก่ไฟล์ GT ที่เดียว 343 ไฟล์ ย้าย 1,534 element ให้ `element\_id: null` และเก็บชื่อเดิมไว้ใน field ใหม่ `element\_name\_assigned` (คง 902 มาร์คที่พิมพ์จริงไว้ตามเดิม ตรงกับที่วัดไว้ก่อนหน้านี้) เพิ่ม `strip\_assigned()` ใน build_dataset_t03.py เพื่อไม่ให้ชื่อที่แต่งขึ้นหลุดเข้าไปเป็น training target แก่ `pass2\_used/section.md` และ §0.2 ใหม่เป็น "มีมาร์ค → copy มา, ไม่มีมาร์ค → null, ห้ามแต่งขึ้นเอง"

กฎที่ได้: field ของ ground truth ต้องมีแต่สิ่งที่ปรากฏจริงในข้อมูลนำเข้าเท่านั้น ถ้าผู้ตรวจแท็กต้องแต่งค่าขึ้นเองเพื่อกรอก field นั้น แปลว่า field นั้นกำลังสอนให้โมเดลหลง ให้ใช้ null แล้วเก็บชื่อที่มนุษย์อ่านเข้าใจไว้ใน field แยกที่ไม่ใช่ target

รอบเก็บเกี่ยวรอบ 2 รายงาน 44/44 สมบูรณ์แบบ ทั้งที่เป็นตัวเลขปลอมจาก threshold หลวมและเช็คทางเดียว

เกิดอะไรขึ้น: รอบเก็บเกี่ยวรอบสองรายงานมาว่า "44/44  " แต่เป็นตัวเลขปลอม เพราะ threshold ถูกลดลงมาที่ 0.55 และเกณฑ์ผ่านเช็คแค่ทิศทางเดียว (ตรวจเจอ $\geq 70\%$ ของ GT) ทำให้การตรวจเจอเกินจริงมหาศาลก็ยังนับว่าผ่าน — บ้าน 27 มี GT 20 จุด แต่เครื่องมือตรวจเจอ 293 จุด แล้วก็ยังผ่านอยู่ดี

ต้นเหตุจริง: เกณฑ์ผ่านมีแค่ขอบล่างด้านเดียว เมื่อรวมกับ threshold ที่หลวม มันให้รางวัลกับการตรวจเจอเกินจริง เพราะแค่ตรวจเจอทุกอย่างก็รับประกันได้ว่าเจออกรอบอย่างน้อย 70% ของความจริงอยู่แล้ว

เสียหายอะไร: ตัวเลขความครอบคลุมที่ดูสมบูรณ์แบบแต่ไร้ความหมาย ในไดอารี่ระบุชัดว่าเป็นบั๊กคลาสเดียวกับ metric ของ section ที่เพิ่งพบในเย็นวันเดียวกัน — ทีมสร้าง metric ที่พึ่งสองตัวในวันเดียว

แก้ยังไง: ตั้ง HARVEST_THRESH = FOOTING_THRESH (0.72) และวัดทั้งสองทิศทาง ( 70-160% /  ต่ำกว่า /  เกิน) ผลจริง: ถูกต้อง 23 / ต่ำกว่า 19 / เกิน 2

กฎที่ได้: เกณฑ์ผ่านต้องมีขอบทั้งสองด้าน เช็คแบบทางเดียว "อย่างน้อย X%" จะให้รางวัลการตรวจเจอเกินจริง และจะรายงานความครอบคลุมสมบูรณ์แบบให้กับตัวตรวจจับที่ยิงใส่ทุกอย่างไปหมด

วิกฤต 2026-08-25

schema เองสั่งให้ผู้ตรวจแก้แต่งชื่อบรรยายขึ้นเองและห้ามเว้น field ว่าง

เกิดอะไรขึ้น: `primary\_rawjson\_schema.md` §0.2 เขียนตรงข้ามกับสิ่งที่ควรเป็น: "ตั้งชื่อบรรยาย ห้ามเว้น" — คือสั่งให้แต่งชื่อบรรยายขึ้นเอง ห้ามเว้นว่างเด็ดขาด นี่คือคำสั่งที่เป็นต้นเหตุของปัญหา 66% ของชื่อที่แต่งขึ้นเองใน subtask section

ต้นเหตุจริง: กฎของ schema ถูกเขียนขึ้นเพื่อรับประกันว่า field จะไม่ว่างเปล่า (ดีต่อความครบถ้วนของข้อมูล) โดยไม่คิดว่า field นี้คือ target ที่ใช้เทรนโมเดล ความครบถ้วนเลยแลกมาด้วยการสอนให้โมเดลหลง

เสียหายอะไร: เป็นต้นเหตุแท้จริงของคอขวด section ที่ 0% ทั้งหมด: 300/455 ตัวอย่าง training, 1,534 element ครอบคลุม 343 ไฟล์ ต้องแก้ GT ทั้งหมดเป็นการใหญ่เพื่อย้อนกลับ

แก้ยังไง: เขียน §0.2 ใหม่ตามธรรมเนียมเดิมของ repo ("ไม่มีบนกระดาษ = null ไม่ใช่ชื่อที่แต่งขึ้น") บันทึกการเปลี่ยนแปลงไว้ใน `primary\_rawjson\_schema\_edit\_log.md` ตามกฎข้อ 7 และอัปเดต `pass2\_used/section.md` ให้สอนโมเดลตามกฎเดียวกัน

กฎที่ได้: กฎ schema ที่ห้าม null บน field ที่โมเดลต้องทำนาย คือคำสั่งให้หลง ก่อนเขียนกฎ field ที่บังคับต้องมีค่า ให้เช็คก่อนว่า "คำนี้อ่านได้จากข้อมูลนำเข้าเสมอไหม"

วิกฤต 2026-08-29

build_dataset ถูกแก้ไขเอาบ้านฝั่ง val (01-05) ไปใส่ใน train ด้วย — val เลยไม่ใช่ชุดที่กันไว้แยกอีกต่อไป

เกิดอะไรขึ้น: มีคอมเมนต์ในโค้ดบันทึกไว้ว่า "2026-08-29 มะขามสั่ง: val houses (01-05) go in train too" ตอนนี้ train.jsonl มี 956 แถว (727+229), val 229, test 33 ค่า val ไม่ใช่ held-out set ที่สะอาดอีกต่อไป — โมเดลเห็นข้อสอบตอนเทรนไปแล้ว ตัวเลข val จะพุ่งเกินจริงจากการจำ ไม่ใช่การวัด generalize

ต้นเหตุจริง: การเปลี่ยน split ถูกทำตรงในโค้ดพร้อมแค่คอมเมนต์บอก และถูกค้นพบโดยบังเอิญจาก session ที่หลัง ไม่ได้ประกาศไว้ ผลกระทบต่อการตีความ metric ไม่ได้ถูกบันทึกไว้พร้อมกับการเปลี่ยนแปลงนี้เลย

เสียหายอะไร: ตัวเลข val ทุกตัวตั้งแต่จุดนี้เป็นต้นไปสูงเกินความสามารถจริง และเทียบกับตัวเลข val ของรอบก่อนๆ ไม่ได้เลย เหลือแค่ test (บ้าน 08 ที่กันไว้จากทั้งสองชุด) ที่ยังเป็นตัวเทียบข้ามรอบที่ใช้ได้ — และนั่นก็ถูกยกเลิกไปทีหลังด้วยเช่นกัน (ดูรายการเรื่อง TEST_HOUSES)

แก้ยังไง: ตีธงไว้ในไดอารี่พร้อมระบุชัดว่า val ตอนนี่วัดแค่ความพอดี (fit) เท่านั้น และ test ยังใช้ได้อยู่

กฎที่ได้: การเปลี่ยน train/val split คือการเปลี่ยนวิธีวัดผล ต้องประกาศพร้อมผลกระทบที่มีต่อทุก metric ไม่ใช่แค่ทิ้งไว้เป็นคอมเมนต์ในโค้ด ห้ามเอาตัวเลข val ที่ปนเปื้อนไปเทียบกับตัวเลขในอดีตที่สะอาดกว่า

ดิงบ้าน 08 เข้าไปเทรนด้วย — ตัวเทียบข้ามรอบที่กันไว้เมื่อ 2026-08-24 หายไปแล้ว

เกิดอะไรขึ้น: ตั้ง `TEST\_HOUSES = set()` แบบถาวร และย้ายบ้าน 08 เข้า train (987→1020 ตัวอย่าง) เป็นการกลับคำตัดสินใจเมื่อ 2026-08-24 ที่เคยยอมเทรนใหม่ทั้งรอบโดยเฉพาะเพื่อกันบ้าน 08 ไว้เป็นตัวเทียบระหว่าง t02 กับ t03

ต้นเหตุจริง: ให้ความสำคัญกับปริมาณข้อมูลมากกว่าความน่าเชื่อถือของการวัดผล การตัดสินใจครั้งก่อนต้องเสียค่าเทรนใหม่ทั้งรอบ (~\$0.35 บวกเริ่มใหม่) เพื่อกันตัวเทียบนี้ไว้โดยเฉพาะ แต่ค่านั้นไม่เคยถูกนำมาชั่งน้ำหนักกับตัวอย่าง 33 ตัวที่ได้เพิ่มมา

เสียหายอะไร: ไม่มีบ้านที่กันไว้แยกเหลืออยู่ให้ทุกรอบใช้เทียบกันร่วมกันอีกแล้ว จะไม่มีโมเดลรุ่นไหนในอนาคตเทียบกับ t02 หรือ t03 บนฐานที่สะอาดได้เลย รวมกับ val ที่ปนเปื้อนแล้ว (ด่าบน) ตอนนี้ไม่มี evaluation set ที่สะอาดเหลืออยู่เลยแม้แต่ชุดเดียว

แก้ยังง: ไม่ได้แก้ — บันทึกไว้ว่าเป็นการกลับคำตัดสินใจโดยเจตนา และยอมรับการเสียตัวเทียบไปแล้ว

กฎที่ได้: ก่อนจะยกเลิกตัวเทียบที่กันไว้แยก (held-out benchmark) ให้เช็คว่าตอนสร้างมันเสียอะไรไปบ้าง และการเปรียบเทียบอะไรที่จะเป็นไปได้ถ้าไม่มีมัน ตัวอย่าง training เพิ่ม 33 ตัวไม่คุ้มกับการเสียความสามารถในการวัดความก้าวหน้าไปเลย

ชุด validation ปนเปื้อน 100% — บ้านทั้ง 5 หลังที่ใช้วัดผลอยู่ในชุด train ด้วย ตัวเลข recall ทั้งหมดจึงวัดแค่การท่องจำ ไม่ใช่ความสามารถจริง

เกิดอะไรขึ้น: เขียนสคริปต์ดึงชื่อบ้านจาก path รูปใน train.jsonl เทียบกับ val.jsonl แล้วพบว่าบ้านทั้ง 5 หลังในชุด val (บ้าน_เล็ก_1ชั้น_01/_02, บ้าน_เล็ก_2ชั้น_01/_02/_03) อยู่ในชุด train ด้วยทั้งหมด เรื่องนี้เป็นการตัดสินใจที่ตั้งใจไว้ตั้งแต่ยุค t03 (val = บ้านที่คนตรวจแล้ว คุณภาพ GT สูงสุด เอาไว้ทดสอบว่าโมเดลตอบคำตอบที่รู้อยู่แล้วได้ไหม) แต่กลับถูกอ่านเป็นตัวเลขวัดการ generalize

ต้นเหตุจริง: ชุดแบ่งข้อมูลที่สร้างมาเพื่อจุดประสงค์หนึ่ง (เช็คว่าโมเดลตอบคำตอบที่รู้อยู่แล้วได้ไหม) ถูกเอาไปใช้ซ้ำเพื่ออีกจุดประสงค์หนึ่ง (วัดว่า generalize ได้แค่ไหน) ข้ามรุ่นโมเดลไปเรื่อยๆ โดยไม่มีใครกลับไปตรวจซ้ำว่าบ้านซ้ำกันใหม่ตอนที่จุดประสงค์เปลี่ยน

เสียหายอะไร: ตัวเลข recall ทุกตัวที่รายงานจาก val ก่อนหน้านี้สูงเกินจริงหมด และไม่ได้บอกอะไรเลยเกี่ยวกับแบบที่โมเดลไม่เคยเห็น แถมยังทำให้คำแนะนำเดิมที่ Claude เคยให้ไว้ ('ใช้ split เดียวพอ จะได้เก็บงบไว้ทำ eval') เสียน้ำหนัก จนต้องวางแผนใหม่กลางทางไปเป็น k-fold

แก้ยังง: เปลี่ยนไปใช้ k-fold จริงจาก manifest k=5 ที่มีอยู่แล้ว และตรวจสอบให้ครบก่อนเสียเงิน: fold0 = 815/205, fold1 = 816/204 ตรงกับ fold_manifest เป๊ะ, ไม่มีบ้านซ้ำกันเลยทั้งสอง fold, ไม่มีรูปหาย, จำนวน token `<image>` เท่ากับ len(images) ทุกแถว, สัดส่วน train:val = 4.00:1 พอดี

กฎที่ได้: ทุกครั้งที่เอาชุดแบ่งข้อมูลเดิมมาใช้ซ้ำ ต้องเขียนสคริปต์ตรวจว่าซ้ำกันในระดับ 'บ้าน' (ระดับตัวตนจริง ไม่ใช่ระดับแถว) และห้ามพูดตัวเลข recall ออกไปเลยถ้ายังไม่ได้พิสูจน์ว่าชุดวัดผลไม่ทับกับชุดเทรน

prompt ที่ใช้ทดสอบก่อนตัดสินใจ มีเฉลยปนอยู่ในตัวคำถาม โมเดลที่ตาบอดสนิทเลยผ่าน 3/3

เกิดอะไรขึ้น: prompt เวอร์ชัน v1 ที่ใช้เป็นด่านตรวจ vision มีตัวอย่างเขียนไว้ว่า 'เช่น F1, B2, C1, พ2' ซึ่งบังเอิญ F1 กับ พ2 อยู่ในเฉลยจริง โมเดลก็แค่ลอกตัวอย่างกลับมาทั้งต้น ตอบสตริงเดียวกันเป๊ะทั้งกับแบบฐานรากและแบบ section แล้วสคริปต์ให้คะแนนก็นับว่าถูก อีกหน้าหนึ่ง 'ผ่าน' เพราะไล่จับ B1..B34 ไปเรื่อยๆ แล้วบังเอิญตัวแรกๆ ตรงกับมาร์คที่ถูก

ต้นเหตุจริง: ออกแบบข้อสอบโดยปล่อยเฉลยหลุดเข้าไปในคำถาม และตัวให้คะแนนใช้แค่การเช็คว่ามีสตริงนั้นอยู่ไหม โดยไม่เช็คเลยว่าเป็นพยางค์ที่ต่างกันต้องให้คำตอบที่ต่างกัน

เสียหายอะไร: โมเดลที่มองไม่เห็นอะไรเลยได้คะแนนเชียว 3/3 ถ้าเชื่อผลนี้จะพาทั้งคืนไปกับโมเดลที่อ่านอะไรไม่ออก ซึ่งจะเป็นการซ้ำรอย t04 แบบเป๊ะๆ

แก้ยังง: เขียน prompt ใหม่เป็น v2 (ตัดตัวอย่างออก, หน้าที่แตกต่างกันต้องให้คำตอบต่างกัน, ปฏิเสธคำตอบที่ไล่จับเรียงลำดับ) แล้วต่อด้วย v3 ที่ถามสามคำถามต่อหน้า และไม่เปิดช่องให้ตอบว่า 'อ่านไม่ออก' บันทึกไว้เป็น rule_of_tune บทเรียนที่ 16 ข้อ 1 และ 2

กฎที่ได้: ห้ามใส่คำหรือคำใดๆ ที่ปรากฏในเฉลยลงไป prompt ที่ใช้ทดสอบเด็ดขาด และต้องเขียนตรวจสอบว่าอินพุตต่างกันแล้วคำตอบต้องต่างกัน — คำตอบที่เหมือนกันเป๊ะข้ามอินพุต คือลายเซ็นของโมเดลที่ไม่เคยมองรูปเลย

เอา fold ของ t04 มาใช้ซ้ำ ทำให้ชุด validation ไม่มีแถว pass0 เลย จนวัดสมมติฐานหลักของรอบนั้นไม่ได้

เกิดอะไรขึ้น: หลังตัดสินใจใช้ fold ของ t04 เป็นแหล่งความจริงเดียว ปรากฏว่าชุด validation แทบไม่มีแถวของ pass0/pass2.4/pass3 เลย โดย fold1 ได้ 0/0/0 เพราะ t04 แบ่งบ้าน 40 หลังโดยไม่รู้ว่ามีแค่ 10 หลังที่มีข้อมูล pass0/2.4/3 (เฉพาะ pass0 มีแค่ 5 หลัง) บ้านพิเศษเหล่านั้นจึงตกลไปอยู่ฝั่ง train หมด

ต้นเหตุจริง: เอาการแบ่งข้อมูลที่ออกแบบมาสำหรับชุดข้อมูลที่กระจายเท่าๆ กัน ไปใช้กับชุดข้อมูลที่มีบางชนิดหายากมาก โดยไม่มีการ stratify ตามความมีอยู่ของข้อมูลแต่ละชนิด

เสียหายอะไร: GPU ทั้ง 4 ตัวจะเรนไปแล้วได้ผลที่อ่านไม่ได้ เพราะไม่มีทางวัดได้เลยว่าโมเดลเรียนรู้ pass0/pass3 หรือเปล่า ซึ่งเป็นสมมติฐานทั้งหมดของรอบนี้ จับได้เพราะมะขาม ไม่ใช่เพราะหิม

แก้ยังง: เขียน build_house_folds() ที่กระจายบ้านแบบ deterministic ในระดับบ้าน (บ้าน pass0 5 หลัง กระจาย fold ละหลัง, บ้านข้อมูลเยอะที่ไม่ใช่ pass0 5 หลัง กระจาย fold ละหลัง, บ้านธรรมดา 30 หลัง fold ละ 6 หลัง) ทำให้ทุก fold มีชุด val ที่มีครบทั้ง 4 pass

กฎที่ได้: แบ่ง fold โดยกระจายตามการมีอยู่ของข้อมูลชนิดหายากที่เรากำลังจะวัดจริงๆ และก่อนกดรันทุกครั้ง ให้พิมพ์จำนวนข้อมูลแต่ละชนิดในชุด val ของแต่ละ fold ออกมาดู — ถ้าช่องไหนเป็นศูนย์ แปลว่ารอบนั้นตอบคำถามของตัวเองไม่ได้

การ probe หา VRAM กรณีแย่มากที่สุด ประเมินต่ำกว่าจุดสูงสุดจริงตอนเทรน เพราะไม่ได้สะสม fragmentation

เกิดอะไรขึ้น: probe แบบ SORT_LONGEST วัดจุดสูงสุดได้ 91.4 GB แล้วถูกใช้เป็นหลักฐานว่าความเสี่ยง OOM หหมดไปแล้ว พอเทรนจริง gpu1 (fold0) และ gpu4 (fold3) OOM ทั้งคู่ ส่วนอีกสองตัวที่รอดก็ค้างอยู่ที่ 93.7 และ 94.66 GB จากเพดาน 94.97 GB

ต้นเหตุจริง: probe รันแค่ 2 ตัวอย่างแบบแยกเดี่ยวบน process ที่เพิ่งเริ่มใหม่ แต่ตัวจัดสรรหน่วยความจำแบบแคชของ PyTorch สะสม fragmentation ไปเรื่อยๆ ทุก step และไม่เคยคืนที่ จุดสูงสุดจริงจึงเกินค่าที่ probe สั้นๆ วัดได้เสมอ

เสียหายอะไร: 2 ใน 4 fold ล้ม เสียเวลาประมาณ 1 ชั่วโมงไปกับการ probe ใหม่และรีสตาร์ททั้งสี่ตัว และต้องทิ้ง checkpoint ที่เทรนไปได้บางส่วน

แก้ยังไง: ลด MAX_PIXELS จาก 7680 เหลือ 6912 เท่ากันทุก fold แล้ว probe ใหม่ด้วย SORT_LONGEST TEST_STEPS=3 บน gpu1 ยืนยันว่าจุดสูงสุดลดจาก 91.4 เหลือ 80.6 GB ก่อนจะรีสตาร์ท

กฎที่ได้: probe สั้นๆ วัดได้แค่ 'พื้น ' ไม่ใช่จุดสูงสุดจริง ต้องเผื่อ margin ให้ fragmentation อย่างน้อย 10% เหนือค่าที่ probe ได้ และต้องยืนยันด้วยตัวเลขจริงก่อนจะเชื่อว่าแก้ปัญหาหน่วยความจำได้แล้ว

คะแนนรูปแบบ JSON 96–100% กลบข้อเท็จจริงที่ว่าโมเดลกำลังตอบเรื่องบ้านคนละหลัง

เกิดอะไรขึ้น: การวัดผลของ t04 รายงานว่า JSON ถูกรูปแบบ 96–100% ทุกครั้ง ซึ่งดูเหมือนโมเดลสุภาพดี จนกระทั่งมีคนเปิดไฟล์คำตอบจริงมาอ่านด้วยตา ถึงพบว่าป้อนแบบฐานรากเข้าไป แต่มันตอบเป็นแปลนหลังคาของบ้านคนละหลังไปเลย

ต้นเหตุจริง: ตัวชี้วัดที่ใช้วัดแค่ 'รูปทรง' ของผลลัพธ์ (JSON ถูกต้อง มีฟิลด์ครบ) ไม่ได้วัดว่าเนื้อหาจริงหรือเปล่า โมเดลที่ท่องจำรูปแบบคำตอบแล้วฟันคำตอบจากชุดเทรนกลับมา จึงได้คะแนนเกือบเต็มทั้งที่ไม่เข้าใจอะไรเลย

เสียหายอะไร: ความล้มเหลวถูกปิดบังไว้ทั้งรอบการเทรน ทีมเชื่อว่าโมเดลใช้ได้ ทั้งที่มันตาบอดสนิท และยังเสียเวลากับเงินต่อไปภายใต้สัญญาเช่าเซิร์ฟเวอร์

แก้ยังไง: เปลี่ยนมาใช้วิธีเปิดไฟล์คำตอบจริงมาอ่านด้วยตา แทนที่จะเชื่อคะแนนรวม บันทึกไว้เป็นบทเรียนข้อ 2 และ 3 ในสรุป

กฎที่ได้: ห้ามตัดสินโมเดลจากตัวชี้วัดเรื่องรูปแบบ/โครงสร้างอย่างเดียว ทุกรอบการวัดผลต้องมีคนเปิดผลลัพธ์จริงมาอ่านด้วยตา — คะแนนรูปแบบเป็น 100% ได้ทั้งที่ความถูกต้องของเนื้อหาเป็น 0%

ด้านตัดสินใจราคา \$2 ตอนแรกขึ้นเขียวปลอม เพราะ prompt ที่ใช้ทดสอบมีเฉลยหลุดอยู่

เกิดอะไรขึ้น: การรันด้านตรวจก่อนเทรดครั้งแรกผ่าน พอไปสืบดูพบว่า prompt ที่ Claude เขียนใส่ตัวอย่างไว้ว่า 'เช่น F1, B2, C1, D2' แล้วบังเอิญ F1 กับ D2 เป็นคำตอบจริงในเฉลย โมเดลที่ตาบอดจึงแค่สะท้อนตัวอย่างกลับมา แล้วสคริปต์ให้คะแนนก็นับว่าถูก ส่วนอีกหน้าหนึ่งโมเดลไล่นับ B1, B2, B3 ... ไปจนถึง B34 แล้วสามตัวแรกบังเอิญตรงกับเฉลยพอดี

ต้นเหตุจริง: มีข้อผิดพลาดสองอย่างซ้อนกัน: (1) คนเขียน prompt ใส่ตัวอย่างที่เป็นค่าจริงลงใน prompt ที่มีไว้ทดสอบว่าโมเดลอ่านออกไหม ทำให้ข้อสอบปนเปื้อน (2) สคริปต์ให้คะแนนนับทุก token ที่ตรง โดยไม่เช็คว่าเป็นการไล่นับหรือสะท้อนคำตอบกลับมา การเดาเรียงลำดับแบบสุ่มมั่วจึงได้คะแนนบางส่วนไปด้วย

เสียหายอะไร: เกือบเผาค่าเช่า GPU ทั้งคืน (ประมาณ \$60-70) ไปกับโมเดลที่มองไม่เห็นอะไรเลย ซึ่งจะเป็นการช้ำรอย t04 เป๊าะๆ จับได้เพราะมีคนสงสัยผลที่ขึ้นเขียวเท่านั้น

แก้ยังไง: เขียน prompt ของด้านตรวจใหม่โดยเอาตัวอย่างออก แล้วรันซ้ำ คราวนี้โมเดลได้ 0/3 จึงหยุดสาขานี้ไป พร้อมตั้งกฎใหม่ว่า ห้ามใส่ค่าที่ปรากฏในเฉลยลงใน prompt ที่ใช้ทดสอบ

กฎที่ได้: prompt ที่ใช้ทดสอบต้องไม่มีตัวอย่างที่หยิบมาจากเฉลยเด็ดขาด และตัวให้คะแนนต้องปฏิเสธคำตอบที่แค่สะท้อนกลับหรือไล่เรียงลำดับ ถ้าด้านตรวจราคาถูกขึ้นเขียวแบบไม่คาดคิด ให้ถือว่าน่าสงสัยไว้ก่อน จนกว่าจะอธิบายได้ว่ามันผ่านเพราะอะไร

pass3 บังคับให้มี grid_ref แค่ตัวเดียวต่อ element ทำให้ระเบียนที่ถูกจัดกลุ่ม (count มากกว่า 1) ได้จัดอ้างอิงเป็นศูนย์

เกิดอะไรขึ้น: ground truth รวม item ที่เหมือนกันเป็นระเบียนเดียวพร้อมลิสต์ของ ref (เช่น 'F1 · count 11 · grid_refs [11 จุด]') แต่โค้ดบังคับว่า len(refs)==1 แล้วข้ามทุกอย่างที่ไม่ตรงเงื่อนไขนี้ทิ้ง — ทำให้ได้จัดอ้างอิงเป็นศูนย์ ทั้งที่ CV เจอจุดที่ตรงกันจริง 12 จุด

ต้นเหตุจริง: โค้ดถูกเขียนขึ้นตามรูปแบบของการจำลองครั้งแรก ซึ่ง Claude เป็นคนเขียนคำตอบของโมเดลเองแบบมือ ให้ ref แค่หนึ่งอันต่อ element การจำลองแบบนั้นเป็นรูปแบบที่แต่งขึ้นเองไม่เคยเจอรูปแบบกลุ่มแบบที่ GT จริงใช้เลย

เสียหายอะไร: pass3 หาจุดอ้างอิงไม่เจอเลยในหน้าที่มีข้อมูลติดอยู่แล้วจริงๆ และที่สำคัญกว่านั้นคือบักนี้มองไม่เห็นเลยในการจำลองครั้งแรก

แก้ยังไง: แก้ให้รองรับระเบียนที่มีหลาย ref บักนี้ถูกเปิดเผยออกมาได้ก็เพราะมะขามสั่งให้การจำลองรอบที่สองใช้คำตอบ GT จริงแทนที่จะใช้คำตอบที่ Claude เขียนเอง

กฎที่ได้: อย่ายสร้างการจำลองที่ input มาจากที่เราเขียนขึ้นเอง — fixture ที่เขียนด้วยมือจะเข้ารหัสข้อสันนิษฐานของเราเอง และบดบังบักที่การจำลองนั้นมีไว้เพื่อค้นหาพอดี ให้ป้อนข้อมูล ground-truth จริงเข้าไปในการจำลองเสมอ

การตรวจสอบผ่านเขี้ยวเพราะ prompt เองมีคำตอบที่ถูกต้องฝังอยู่ในตัว

เกิดอะไรขึ้น: การตรวจสอบรอบแรกขึ้นเขี้ยวหมด — แต่เขี้ยวเพราะ prompt มีคำตอบเขียนฝังอยู่ในตัวเองอยู่แล้ว การประเมินครั้งนั้นวัด prompt ไม่ได้วัดโมเดลเลย

ต้นเหตุจริง: ground truth รัวเข้าไปอยู่ใน prompt ที่ใช้ประเมิน ไม่มีการแยกระหว่างข้อมูลที่ใช้สร้าง prompt กับข้อมูลที่ใช้ให้คะแนนคำตอบเลย

เสียหายอะไร: ให้สัญญาณเท็จว่า 'โมเดลใช้ได้' ซึ่งถ้าปล่อยไว้จะทำให้โมเดลที่พังผ่านไปได้ เสียเวลาไปกับการวัดผลที่ไม่มีความหมายอะไรเลย

แก้ยังไง: ค้นพบและบันทึกไว้เป็นกัปดักในเอกสารสอน

กฎที่ได้: ก่อนจะเชื่อผลประเมินที่ขึ้นเขี้ยว ให้เทียบ prompt กับคำตอบที่คาดหวังก่อน — ถ้าข้อความคำตอบปรากฏอยู่ใน prompt คะแนนนั้นไม่มีความหมายอะไรเลย ให้สร้าง prompt สำหรับประเมินจากคนละ code path กับข้อมูลที่ใช้ให้คะแนน

ผลลัพธ์เด่น '191/191, 100%' ของ pass3 วัดจากหน้าที่คนตรวจสอบมาแล้ว พอลอง CV อัตโนมัตกับบ้านที่ไม่เคยตรวจ กลับล้มเหลว 5/5

เกิดอะไรขึ้น: pass3 ถูกยืนยันความแม่นยำแบบ leave-one-out ที่ 191/191 — แต่วัดจาก 18 หน้าที่คนได้ตรวจสอบและยืนยันแล้วว่ามั่วจืดอ้างอิงที่ตืออยู่แล้ว พอลองรันกับ house02 ด้วย CV อัตโนมัตี่ส่วนๆ บนบ้านที่ไม่เคยตรวจสอบมาก่อนเลย หน้าโครงสร้างทั้ง 5 หน้าล้มเหลวหมด: 2 หน้าฐานรากถูกด้านคุณภาพปฏิเสธ, 3 หน้าคานหาจุดอ้างอิงไม่เจอเลย

ต้นเหตุจริง: ชุดข้อมูลที่ใช้ตรวจสอบถูกเลือกจากคุณภาพ ซึ่งเปลี่ยนคำถามอย่างเสียวๆ จาก 'pass3 ใช้ได้กับแบบจริงไหม' เป็น 'สูตรการ fit ใช้ได้กับจุดอ้างอิงที่ตืออยู่แล้วไหม' ขั้นตอนตรวจจับจุดอ้างอิงต้นทาง — ส่วนที่ล้มเหลวจริงๆ — ถูกกันออกจากการวัดผลไปเลย

เสียหายอะไร: ตัวเลขความแม่นยำที่โดดเด่นเกินจริงไปมากเมื่อเทียบกับความสามารถในโลกจริง ถ้าปล่อยไว้จะทำให้เอา pass3 ไปใช้งานจริงโดยเชื่อผิดๆ

แก้ยังไง: วัดใหม่อย่างตรงไปตรงมาด้วย output CV ที่ไม่ผ่านการกรองบนบ้านใหม่ที่ยังไม่เคยตรวจ แล้วบันทึกผล 0/5 ไว้ตรงๆ คู่กับ 191/191

กฎที่ได้: อย่ายืนยัน pipeline ขั้นตอนไหนด้วยข้อมูลที่ถูกรวบรวมมาแล้วว่าสะอาด — เพราะจะวัดผิดขั้นตอน ให้รายงานตัวเลข end-to-end บนข้อมูล input ที่ไม่กรองเสมอ คู่กับตัวเลขที่มาจากชุดข้อมูลคัดสรรทุกครั้ง

ทำการสกัดข้อมูลสองรอบเต็มๆ แต่ไม่เคยเทียบกันอย่างเป็นระบบ ความสม่ำเสมอของการอ่านซ้ำจึงไม่เคยถูกวัดจริง

เกิดอะไรขึ้น: รอบ t2 ถูกสั่งให้อ่านภาพสดและอิสระ พร้อมรายงานว่าต่างจาก t1 ตรงไหน โดยเฉพาะเพื่อดูว่าการอ่านซ้ำมีความสม่ำเสมอแค่ไหน แต่การเทียบไฟล์ t1 กับ t2 ที่ละไฟล์อย่างเป็นระบบไม่เคยถูกทำเลย — มีแต่รายงานสรุปของเอเจนต์เองเท่านั้น

ต้นเหตุจริง: การเปรียบเทียบคือการวัดผลที่เป็นเหตุผลว่าทำไมต้องทำรอบที่สอง แต่ถูกวางแผนไว้ให้ทำหลังรอบเสร็จแทนที่จะสร้างเข้าไปเป็นส่วนหนึ่งของรอบเลย และเวลาที่เหลือของวันก็ไปลงกับเครื่องมือ Label Studio แทน

เสียหายอะไร: ต้นทุนของการทำรอบสกัดเต็มรอบที่สองถูกจ่ายไปแล้ว แต่ไม่ได้ผลลัพธ์หลักที่ต้องการ — ตัวเลขความสอดคล้องของการอ่านซ้ำที่แท้จริง โดยไม่มีมัน ความน่าเชื่อถือรายฟิลดจึงยังไม่รู้ และความขัดแย้งระหว่าง t1/t2 (กริด MEP, element กับ elements) ถูกเจอด้วยความบังเอิญ ไม่ใช่จากการ diff อย่างเป็นระบบ

แก้ยังง: ยังไม่ได้แก้ — จดไว้ว่า 'ควรทำสรุปเทียบ t1 กับ t2 ให้ห้ห้ชามเห็นภาพรวมคุณภาพ' และ 'ยังไม่ได้ทำ'

กฎที่ได้: ถ้ารอบงานมีไว้เพื่อวัดผลอะไรสักอย่าง ให้สร้างการวัดผลนั้นเข้าไปในนิยาม 'เสร็จ' ของรอบนั้นเลย การ diff ระหว่างการสกัดข้อมูลสองรอบควรเป็นสคริปต์อัตโนมัติที่รันตอนปิดรอบ ไม่ใช่งานสรุปที่ต้องมาแย่งเวลากับงานฟิเจอร์อื่น

รอบที่สอง (t2) ไม่ใช่การอ่านซ้ำแบบอิสระจริง: 6 หน้าโครงสร้างถูกก๊อปปามาจาก t1 ตรงๆ และเอเจนต์ถูกให้เห็น t1 เพื่อเอาไป diff

เกิดอะไรขึ้น: t2 ถูกตั้งขึ้นเป็นการสกัดข้อมูลสตรอบที่สอง โดยมีจุดประสงค์ส่วนหนึ่งคือวัดความสม่ำเสมอของการอ่านซ้ำ แต่หน้า 18, 21, 22, 23, 24 และ 25 ถูกก๊อปปามาจาก t1/ ใส่ t2/ ตรงๆ โดยไม่อ่านใหม่เลย ส่วนเอเจนต์ที่ทำหน้า 1-17 และ 26-37 ก็ถูกสั่งให้รายงานว่าอ่านต่างจาก t1 ตรงไหน — คือมีสิทธิ์เห็น t1 ระหว่างทำรอบ t2 เลย

ต้นเหตุจริง: ให้ความสำคัญกับความเร็วมากกว่าความสะอาดของการทดลอง โดยให้เหตุผลว่า pattern type เหล่านั้น 'ไม่ได้รับผลกระทบจากการอัปเดต schema รอบนี้' — ซึ่งจริงสำหรับ schema แต่การวัดความสม่ำเสมอคือเรื่องความสอดคล้องของการอ่านซ้ำ ซึ่งการก๊อปปไฟล์จะให้คะแนน 100% ปЛОมๆ แทนที่ 'ไม่มีใครแยกระหว่าง 'หน้าที่ต้องทำใหม่ตาม schema' กับ 'หน้าที่ต้องเป็นกลุ่มควบคุมแบบ blind'

เสียหายอะไร: ตัวเลขความสอดคล้อง t1-กับ-t2 ที่คำนวณทีหลังจะพองเกินจริง หกหน้าจะโชว์ความสอดคล้อง 100% โดยธรรมชาติของการก๊อป และหน้าที่เหลือก็ถูกวัดเทียบกับรอบที่ไม่ blind ความน่าเชื่อถือของการอ่านซ้ำ — ตัวเลขที่จะพิสูจน์ได้ว่าไว้ใจ pipeline การสกัดได้แค่ไหน — ไม่มีทางคำนวณได้จากข้อมูลชุดนี้เลย

แก้ยังง: ยังไม่ได้แก้ — หน้าที่ก๊อปปามาไม่เคยถูกอ่านใหม่แบบอิสระเลย และการเทียบ t1 กับ t2 อย่างเป็นระบบยังถูกจัดเป็นรายการค้าง ณ สิ้นวัน

กฎที่ได้: ถ้ารอบงานมีไว้วัดความสม่ำเสมอ ทุกหน้าในรอบนั้นต้องถูกอ่านใหม่แบบ blind ห้ามก๊อปปไฟล์เข้าชุดเปรียบเทียบเด็ดขาด และห้ามให้คนที่ทำรอบที่สองเห็นรอบแรก — แยกรอบ 'ทำใหม่ตาม schema' กับรอบ 'ควบคุมแบบ blind' ให้เป็นคนละชุดข้อมูล

Gen 1 บันทึกกำลังคอนกรีต 240 ksc ทั้งที่แบบจริงระบุ 210 ksc

เกิดอะไรขึ้น: มีความขัดแย้งค้างมาตั้งแต่ 5 ก.ค. ว่าหน้า 18 (หมายเหตุ S-01) ระบุ $F_c' = 210$ หรือ 240 กก./ตร.ซม. กันแน่ พออ่านซ้ำแบบอิสระในรอบ t3 ได้ผลชัดเจนว่า 210 ksc ตรงกับที่มะขามอ่านเองครั้งแรกเมื่อ 5 ก.ค. สรุปว่าค่า 240 ของ Gen 1 ผิด

ต้นเหตุจริง: รอบ Gen 1 บันทึกค่าคงที่โครงสร้างสำคัญจากการอ่านครั้งเดียวโดยไม่มีแหล่งที่สองมายืนยัน และไม่ติดธง confidence ไว้เลย ทำให้การอ่านผิดกลายเป็น 'ข้อเท็จจริง' ในชุดข้อมูลไปเนี่ย

เสียหายอะไร: ค่ากำลังคอนกรีตที่ผิดค้างอยู่ในข้อมูล ground truth สำหรับ fine-tune นาน 2 วัน ส่งผลต่อ BOQ หรือข้อสันนิษฐานเรื่องความแข็งแรงใดๆ ที่คำนวณต่อจากค่านี้

แก้ยังไง: อ่านหน้า 18 ใหม่อีกครั้ง มีสองคนอ่านอิสระได้ตรงกันว่า 210 ksc จึงลบล้างค่า 240 ของ Gen 1

กฎที่ได้: ค่าคงที่โครงสร้างตัวไหนที่มีผลกับทั้งหลัง (กำลังคอนกรีต ระบุหุ้ม ระบุทาบ) ต้องอ่านยืนยันจากสองคนอิสระก่อนถือเป็น ground truth เสมอ — การอ่านครั้งเดียวคือแค่สมมติฐาน ไม่ใช่ข้อมูลจริง

รายการ BOQ ของบ้านหลัง 2 หลายรายการถูกบันทึกที่ confidence 0.4-0.6 เพราะตัวหนังสือเล็กและหมุน 90 องศา

เกิดอะไรขึ้น: รายการในหมวดตกแต่ง สี และสุขภัณฑ์ ถูกสกัดออกมาที่ confidence 0.4-0.6 เพราะตัวหนังสือต้นฉบับเล็กมากและพิมพ์หมุน 90 องศา ตอนนี้อยู่ในชุดข้อมูลแล้วและติดธงไว้ว่าต้องมีคนไปชุมตรวจสอบก่อนใช้เป็น ground truth จริงจัง

ต้นเหตุจริง: การสกัดข้อมูลอ่านหน้ากระดาษตามที่เรนเดอร์มาเลย ไม่มีขั้นตอนเตรียมภาพ (หมุน ครอบ ขยาย) สำหรับบริเวณที่ต้นฉบับพิมพ์ตะแคงและตัวเล็ก

เสียหายอะไร: มีข้อมูล ground truth ก้อนหนึ่งที่มีโอกาสผิดมากกว่าถูก นอนอยู่ในชุดข้อมูลที่จะเอาไป fine-tune โดยที่การตรวจสอบยังไม่มีคิว

แก้ยังไง: ยังไม่ได้แก้ — บันทึกเป็นงานตรวจค้างข้อ 2: ชุมเข้าไปตรวจรายการที่ confidence ต่ำ

กฎที่ได้: ถ้า confidence ตกเพราะวิธีเรนเดอร์หน้ากระดาษ ให้ไปแก้ที่การเรนเดอร์ (หมุน/ครอบ/ขยาย แล้วอ่านใหม่) แทนที่จะส่งแถวที่ติดธง confidence ต่ำออกไป และห้ามให้แถวที่ติดธงเข้าไปอยู่ในข้อมูลเทรนโดยไม่มีคิวตรวจ

สูง 2026-07-24

dry run ในเครื่องใช้โมเดลตัวเล็ก 10.8GB แทน พิสูจน์ได้แค่ว่า 'รันได้' แต่ไม่เคยพิสูจน์ว่า 'ใช้งานได้จริง'

เกิดอะไรขึ้น: Phase 0.3 วันที่ 24 ทดสอบ inference ในเครื่องโดยดาวน์โหลดโมเดลทดสอบเล็กๆ ราว 10.8GB รันกับหน้าแบบจริง หน้าด้วย llama.cpp แล้วประกาศว่า 'ผ่าน' แต่ตัวจริงมีขนาด 21.2GB — ใหญ่กว่าเกือบสองเท่า พอเอมารันบนโน้ตบุ๊กที่มี VRAM 8GB ตัวจริง ได้ความเร็วแค่ 3.46 token/s ใช้เวลา 13-21 นาทีต่อหน้า และ llama.cpp ก็ log ว่า 'failed to fit params to free device memory ... abort'

ต้นเหตุจริง: dry run เลือกขนาดตามความสะดวก ไม่ได้เลือกให้ตรงกับของจริงที่มันควรจะแทน และเกณฑ์ผ่านเป็นแบบ 'ใช่/ไม่ใช่' (รันได้ไหม) ไม่มีเกณฑ์ความเร็วขั้นต่ำเลย โมเดล 10.8GB ใส่ลง VRAM ได้พอดีกว่ามาก คอขวดจาก CPU offload จึงไม่มีทางโผล่ให้เห็น

เสียหายอะไร: ทีมเข้าสู่การรันจริงวันที่ 29 โดยเข้าใจว่าการรันในเครื่องพิสูจน์แล้วว่าใช้ได้ แล้วเพิ่งมาเจอทีหลังว่าบ้านหนึ่งหลังที่ ~18 นาที/หน้า ใช้งานจริงไม่ได้เลย ต้องรีบบอกแบบใหม่กลางทางเป็น 2 เฟส (classify ก่อนแล้วค่อย extract) แคมป์ตั้ง timeout ผิดจนไปฆ่างานที่รันอยู่ปกติทั้ง

แก้ยังง: ไม่ได้แก้เป็นกฎกระบวนการ แต่แก้เฉพาะหน้าวันที่ 29 ด้วยการแยกเป็น pass classify แบบเร็วก่อน แล้วค่อย extract แบบเลือกจุด (extract_house01_local.py)

กฎที่ได้: dry run ต้องใช้ไฟล์ที่มีขนาดจริงบนฮาร์ดแวร์จริง และต้องยืนยันความเร็วที่ใช้งานได้จริง (วินาที/หน้า) ไม่ใช่แค่เช็คว่ามันไม่ crash

สูง 2026-07-24

ข้าม **baseline eval** ของ **Phase 4** ทำให้ตัวเลข 'ดีขึ้น 3 เท่า' ที่เป็นพาดหัว เทียบกับ **baseline** คนละรอบ

เกิดอะไรขึ้น: Phase 4 (การวัดผลตอนยังไม่ tune) ถูกข้ามไปเพื่อประหยัดเวลาตามคำแนะนำใน workflow doc ผลของ Phase 7 จึงถูกรายงานว่า element recall 28.2% '↑ ↑ ↑' จาก 9.4% และ view exact match '70% ↑' จาก 60% — ซึ่งตัวเลขเทียบมาจากรอบ training ก่อนหน้าที่รันบนฮาร์ดแวร์คนละตัวและขนาด dataset คนละขนาด (226 train เทียบกับ 315 train)

ต้นเหตุจริง: การวัด baseline ถูกมองเป็นแค่ต้นทุนที่ต้องประหยัด ไม่ใช่ตัวหารของทุกค่ากล่าวอ้างในรอบนั้น การข้าม baseline ทำให้วัดความดีขึ้นภายในรอบนี้ไม่ได้เลย จึงต้องเอาตัวเลขของรอบก่อนมาแทนแบบเสียบๆ

เสียหายอะไร: ค่ากล่าวอ้างเรื่องคุณภาพหลักของ tuning cycle ทั้งรอบ ตั้งอยู่บนการเทียบข้ามรอบที่ baseline ถูกวัดภายใต้เงื่อนไขคนละแบบ — ความดีขึ้นจริงอาจมากกว่าหรือน้อยกว่าที่รายงานก็ได้ และไม่มีอะไรในรอบนั้นบอกได้เลย

แก้ยังง: ยังไม่ได้แก้ — Phase 4 ยังคงถูกข้ามอยู่ และตัวเลขเปรียบเทียบก็ยังคงยืนยันตามที่รายงานไว้

กฎที่ได้: ห้ามข้าม baseline ในรอบที่จะเอาผลความดีขึ้นไปอ้างอิงเด็ดขาด baseline จากรอบอื่นคือตัวหารคนละตัว การเอามาปนกันทำให้เปอร์เซ็นต์ทุกตัวในรายงานพิสูจน์ผิดถูกไม่ได้เลย

ตาราง parity เทียบ A/B ตรวจแค่ค่าที่ตั้งไว้ชัดเจน จึงหลุดตรวจค่า default ที่ไม่ได้ตั้ง

เกิดอะไรขึ้น: มีการทำตาราง parity ละเอียดเพื่อยืนยันว่า t01 กับ t02 ต่างกันแค่ที่ตัวโมเดล โดยลิสต์ learning rate, batch size, epochs ฯลฯ ไว้ แต่มันจับความต่างเรื่อง resize ไม่ได้เลย เพราะ t02 ตั้งค่า resize ไว้ชัดเจน ในขณะที่ t01 ไม่ได้ตั้งอะไรเลย — เลยไม่มีบรรทัดให้เทียบกัน

ต้นเหตุจริง: การเทียบถูกสร้างจากสิ่งที่จำได้/ตั้งใจเปลี่ยน ไม่ใช่จาก argument ทั้งหมดของทุกฟังก์ชัน/คลาสที่ใช้ร่วมกัน argument ที่ไม่ได้ตั้งค่าก็ยังถือเป็นตัวแปรอยู่ดี และเป็นสิ่งที่มองข้ามได้ง่ายที่สุดเพราะมันมองไม่เห็น

เสียหายอะไร: ทำให้เข้าใจผิดว่าการทดลองถูกควบคุมตัวแปรครบแล้ว จนสรุปผลผิดๆ และปล่อยให้มันยืนอยู่จนกระทั่งมีการตรวจสอบเชิงลึกแยกต่างหากมาเจอ

แก้ยังไม: บันทึกไว้เป็น Lesson 12 ใน rule_of_tune.md: ตาราง parity ของ A/B ต้องลิสต์ argument ทุกตัวที่ส่งเข้าทุกฟังก์ชัน/คลาสที่ใช้ร่วมกัน อ่านจาก source code เท่านั้น ไม่ใช่จากความจำว่าเปลี่ยนอะไรไปบ้าง

กฎที่ได้: สร้างตาราง parity ของ A/B ด้วยการ diff argument จริงของการเรียกที่ใช้ร่วมกัน (รวมค่า default ด้วย) อ่านจาก source code ตรงๆ ห้ามใช้แค่รายการสิ่งที่ตั้งใจจะเปลี่ยน

โมเดล t01 ถูกทดสอบกับบ้าน01 ซึ่งอยู่ใน training set ของมันเอง (62 ตัวอย่าง) — ผลลัพธ์เลยดูดีกว่าความจริง

เกิดอะไรขึ้น: โมเดล t01 ที่ tune แล้วถูกรันทดสอบกับหน้าแบบของบ้าน01 บ้าน01 มีส่วนร่วม 62 ตัวอย่างใน training set โมเดลจึงจำเนื้อหานี้ได้อยู่แล้ว ผลลัพธ์ที่ได้จึงไม่ใช่การวัด generalization เลย

ต้นเหตุจริง: เอาบ้านที่สะดวกที่สุดตรงมือมาใช้ทดลองคุณภาพโดยไม่เช็คว่ามันถูกกันไว้เป็น held-out หรือเปล่า ไม่มีการบังคับแยก train/test ตอนรัน evaluation เลย

เสียหายอะไร: จับได้และถูกทำเครื่องหมายไว้ในไดอารี่ก่อนที่ใครจะเอาตัวเลขไปอ้างอิง — แต่ถ้าไม่มีการเตือนไว้ ผลลัพธ์แบบ in-sample นี้จะถูกเอาไปเผยแพร่เป็นหลักฐานความสามารถของโมเดลได้เลย

แก้ยังไม: ระบุไว้ชัดเจนว่าใช้ได้แค่เป็น smoke test เท่านั้น ห้ามเอาไปเป็นตัววัด generalization

กฎที่ได้: ก่อนจะอ้างผลลัพธ์ของโมเดลเป็นผลวัดคุณภาพ ต้องเช็คว่าข้อมูลอินพุตอยู่ใน training set หรือไม่ แล้วทำเครื่องหมายให้ชัด ผลแบบ in-sample เป็นได้แค่ smoke test เท่านั้น มีแค่ข้อมูล held-out เท่านั้นที่วัดความสามารถจริงได้

สูง 2026-07-28

กราฟ loss ลดลงสวยงาม (1.795 → 0.18-0.21) แต่โมเดลได้คะแนน element recall แค่ 20.6%

เกิดอะไรขึ้น: loss ตอนเทรนของ t02 ลดลงเรียบร้อยจาก 1.795 เหลือประมาณ 0.18-0.21 และ eval_loss ได้ 0.2076 ดูเหมือนทุกอย่างบ่งชี้ว่าเรียนรู้ได้ดี แต่พอวัดผลระดับ field จริงใน Phase 7 กลับพบว่า JSON valid แค่ 65%, element recall แค่ 20.6% แพ้ t01 ทุกตัวชี้วัด

ต้นเหตุจริง: loss วัดแค่ความใกล้เคียงทางคณิตศาสตร์กับ token sequence เป้าหมาย ไม่ได้วัดว่าตัวเลขหลักเสริมหรือข้อขึ้นส่วนถูกอ่านถูกหรือเปล่า การเอา loss มาเป็นสัญญาณคุณภาพคือการเอาความก้าวหน้าของการ optimize มาปนกับผลงานจริง

เสียหายอะไร: ได้โมเดลที่ดูเหมือนเทรนสำเร็จจากทุกตัวชี้วัดตอนเทรน แต่สอบตกเกณฑ์การรับจริง — เป็นกับดักที่ทำให้อยากรีบสรุปว่า 'สำเร็จแล้ว' ทั้งที่ยังไม่ใช่

แก้ยังง: ใช้การวัดผลระดับ field ใน Phase 7 เป็นเกณฑ์ตัดสินจริง ไม่ใช่ loss — บันทึกไว้เป็นบทเรียนข้อ 4 ของ CLAUDE.md เดิม และย้ำอีกครั้งใน rule_of_tune.md

กฎที่ได้: ห้ามสรุปคุณภาพโมเดลจากกราฟ loss เด็ดขาด ต้องวัดผลระดับ field เทียบกับข้อมูลที่กันไว้ (held-out) เท่านั้นถึงจะตัดสินได้ว่ารอบนั้นสำเร็จจริงหรือไม่

สูง 2026-07-28

ตัวเลขสรุปผลของ t01 มีสองชุดที่ขัดแย้งกันอยู่ในเอกสารเดียวกัน แกรมการวัดผลใช้ข้อมูลแค่ 20 จาก 88 ตัวอย่างใน validation set

เกิดอะไรขึ้น: หัวข้อที่ 14 (บันทึกวันที่ 21 ก.ค.) บอกว่า t01 ได้ JSON valid 90% / view 60% / element recall 9.4% ส่วนตารางเปรียบเทียบ t02 (บันทึกวันที่ 28 ก.ค.) บอกว่าโมเดลตัวเดียวกันนี้ได้ JSON valid 90% / view 70% / element recall 28.2% เอกสารไม่เคยอธิบายเลยว่าทำไม 60 กับ 70, 9.4 กับ 28.2 ถึงไม่ตรงกัน แต่ตัวเลข 28.2% กลับถูกอ้างซ้ำๆ ทั่วเอกสารว่าเป็น 'ตัวเลขที่เราเชื่อ' ผลลัพธ์ eval ที่แสดงก็เขียนไว้ '0/20' และ '0/170' ทั้งที่ val set มีทั้งหมด 88 บรรทัด

ต้นเหตุจริง: การวัดผลถูกรันคนละเวลากันด้วยค่า --limit ที่ต่างกันและอาจจะบนเครื่องคนละเครื่อง แต่ครั้งที่รันก็เขียนตัวเลขสรุปลงบันทึกโดยไม่ระบุขนาดตัวอย่างและไม่ได้บอกด้วยว่าตัวเลขนี้แทนที่ตัวเลขเก่าไปแล้ว

เสียหายอะไร: คนอ่าน (รวมถึงทีมเองด้วย) แยกไม่ออกว่าตัวเลข t01 ตัวไหนเป็นตัวที่ถูกต้อง ตัวเลข recall ที่เป็นตัวชี้ขาดการตัดสินใจว่า 'Qwen3.6 ดีกว่า Qwen3-VL' นั้นต่างกันถึง 3 เท่าในเอกสารเดียวกัน และอาจอิงจากแค่ราวๆ 23% ของ validation set เท่านั้น

แก้ยังง: ยังไม่ได้แก้ — ตัวเลขทั้งสองชุดยังอยู่คู่กันในเอกสารรวมที่รวมคำสอนไว้ โดยไม่มีคำอธิบายว่าทำไมถึงต่างกัน

กฎที่ได้: ทุกตัวเลขที่บันทึกไว้ต้องระบุขนาดตัวอย่างและระบุว่าเป็นการรันครั้งไหน และถ้าวัดผลซ้ำต้องระบุชัดว่าตัวเลขใหม่แทนที่ตัวเลขเก่าไปแล้ว ห้ามปล่อยให้อยู่คู่กันเฉยๆ โดยไม่บอกอะไรเลย

สูง 2026-07-29

ความสามารถหลักที่โปรเจกต์นี้มีไว้ทำ — การอ่านแบบหลักเสริม — ไม่เคยถูกทดสอบเลยสักครั้ง

เกิดอะไรขึ้น: ยังไม่เคยมีหน้าแบบโครงสร้าง (S-*) หน้าไหนถูก extract แบบเต็มรูปแบบเลย หน้า 07-11 ถูกแค่จำแนกประเภทเท่านั้น กฎที่สำคัญที่สุดของ prompt ('แยก rebar บนกับล่างเสมอ') จึงไม่มีหลักฐานสนับสนุนหรือคัดค้านเลย แม้จะผ่านการ tune มาแล้วหลายรอบ

ต้นเหตุจริง: การทดสอบเรียงตามลำดับหน้าและความสะดวก แทนที่จะเริ่มจากความสามารถที่มีมูลค่าสูงสุดก่อน ทำให้ข้อกำหนดหลักของงานถูกเลื่อนไปเรื่อยๆ ให้อยู่หลังหน้าที่ทดสอบง่ายกว่า

เสียหายอะไร: ผ่านการ tune มาแล้ว 3 รอบและประเมินผลมาหลายวัน แต่ทีมยังตอบไม่ได้เลยว่าโมเดลทำงานหลักที่ตั้งใจไว้ได้จริงหรือไม่

แก้ยังไม: ยังไม่ได้แก้ — ยังไม่เคย extract หน้า S-* เลยสักหน้า

กฎที่ได้: ต้องทดสอบความสามารถที่มีมูลค่าสูงสุดก่อนตั้งแต่วันแรก แม้จะใช้เครื่องมือทดสอบแบบหยาบๆ ก็ตาม โปรแกรมคุณภาพที่ไม่เคยทดสอบ use case หลักของตัวเอง คือไม่ได้วัดอะไรที่สำคัญเลยจริงๆ

สูง 2026-07-29

สาเหตุที่สงสัยไว้ 3 ข้อยังพัวพันอยู่ ไม่มีข้อไหนถูกทดสอบแยกทีละตัวแปร

เกิดอะไรขึ้น: merge ผิดพลาด, resolution ไม่ตรงกัน, และความเสียหายจากการ quantize ล้วนเป็นสาเหตุที่เป็นไปได้ของคุณภาพที่พังลง แต่บันทึกระบุว่ายังไม่มีข้อไหนถูกทดสอบแยกกันเพื่อดูว่าตัวไหนมีผลมากที่สุด

ต้นเหตุจริง: ทั้งสามอย่างเปลี่ยนไปพร้อมกันระหว่างชุดตั้งค่าที่วัดผลแล้วว่าดี (bf16 + adapter ที่ยังไม่ merge) กับชุดตั้งค่าที่ใช้งานจริง (merge แล้ว + Q4_K_M + resolution ภาพต่างกัน) — เปลี่ยนหลายตัวแปรพร้อมกัน จึงไม่มีทางวัดผลแล้วชี้ตัวการได้ตัวเดียว

เสียหายอะไร: ไม่รู้ว่าควรแก้อะไรก่อน และประเมินไม่ได้ว่าแต่ละจุดจะได้คุณภาพคืนมาเท่าไร เสี่ยงเสียเงินเทรนใหม่ทั้งที่จริงๆ แค่ว่า quantization หรือ flag เดียวก็อาจแก้ปัญหาได้ฟรีๆ

แก้ยังไม: ยังไม่แก้ — ระบุไว้ชัดเจนว่ายังต้องทำ ทดสอบทีละตัวแปร

กฎที่ได้: ห้ามเปลี่ยนตัวแปรมากกว่าหนึ่งอย่างพร้อมกันระหว่างชุดตั้งค่าที่วัดผลแล้วดีกับชุดที่ใช้งานจริง ถ้าเปลี่ยนไปหลายตัวแล้ว ให้ลงทุนทดลองแบบถูกๆ เพื่อแยกตัวแปรก่อน ค่อยเสียเงินแก้จริง — หาต้นเหตุก่อน แก้ทีหลัง

สูง 2026-07-29

ตัวเลข 90% / 28.2% ที่ทุกคนเชื่อถือ วัดมาจากพารามิเตอร์ที่ไม่ใช่ artifact ที่ใช้งานจริง

เกิดอะไรขึ้น: eval_fields.py รันด้วย bf16 เต็ม (load_in_4bit=False) พร้อมติด LoRA adapter ตอน runtime โดยไม่ merge แต่ artifact ที่ใช้งานจริงผ่านขั้นตอน merge (จุดที่น่าสงสัย) และผ่านการบีบอัด Q4_K_M ทั้งสองพารามิเตอร์ไม่เคยถูกเทียบกันเลย แต่ตัวเลข eval กลับถูกอ้างว่าเป็นความแม่นยำของผลิตภัณฑ์จริง

ต้นเหตุจริง: ชุดเครื่องมือ evaluation ถูกสร้างขึ้นรอบๆ artifact ที่ใช้เทรน โดยไม่มีใครถามว่าสิ่งที่วัดผลกับสิ่งที่ deploy จริงเป็นตัวเลขเดียวกันหรือเปล่า

เสียหายอะไร: การตัดสินใจหลายเดือนตั้งอยู่บนตัวเลขที่บรรยาย artifact ที่ไม่มีใครรันจริง GGUF ที่ shipped ออกไปยังไม่เคยมีการวัดความแม่นยำที่แท้จริงเลย

แก้ยังไม: ชี้แจงในบันทึกแล้วว่า 90%/28.2% ยังใช้ได้แค่กับตัว adapter เท่านั้น ส่วน GGUF ยังไม่ได้ตรวจสอบ งานเทรนเองยังใช้ได้ — adapter ปลอดภัยอยู่บน HuggingFace

กฎที่ได้: ตัวเลขความแม่นยำทุกตัวต้องระบุให้ชัดว่าวัดจาก artifact ตัวไหนและพารามิเตอร์ไหน และ artifact ที่จะใช้งานจริงต้องถูกวัดผลบนพารามิเตอร์ของตัวเองก่อนจะเอาตัวเลขไปอ้างอิงใดๆ

สูง 2026-07-29

คำตัดสิน 'อย่าเปลี่ยนไปใช้ Qwen3-VL' อาจผิดตรงเส้นทางที่จะเอาไปใช้งานจริง — ความละเอียดตอนเทรนของ t02 ตรงกับตอนรันจริงของ llama.cpp พอดี

เกิดอะไรขึ้น: เมื่อ 2026-07-28 การทดสอบเทียบ A/B สรุปว่า Qwen3.6 (t01) ชนะ Qwen3-VL (t02) ทุกตัวชี้วัด และบันทึกไว้ว่า 'อย่าเปลี่ยนไปใช้ Qwen3-VL' แต่คืนถัดมาพบว่า t02 ถูกเทรนที่ ~3,796 visual tokens ต่อภาพ ซึ่งตรงกับที่ llama.cpp ใช้ตอนรันจริงพอดี ในขณะที่ t01 ถูกเทรนที่ ~266 เท่านั้น — ดังนั้นบนเส้นทางที่จะใช้งานจริงในเครื่อง (local deployment) t02 อาจเป็นโมเดลที่ดีกว่าจริงๆ

ต้นเหตุจริง: การเปรียบเทียบรันบนเส้นทาง bf16 + runtime-adapter (เส้นทางที่ใช้วัดผล) ไม่ใช่เส้นทาง merge/quantize เป็น GGUF (เส้นทางที่ใช้งานจริง) และไม่เคยเอาความสอดคล้องของ token ระหว่างตอนเทรนกับตอนใช้จริงมาเป็นเกณฑ์เปรียบเทียบเลย

เสียหายอะไร: คำตัดสินเรื่องเลือกโมเดลที่เคยดูเหมือนจบแล้วกลายเป็นเชื่อไม่ได้ — และตัวที่ถูกปิดตกก็ใช้งานไม่ได้ยอดเยี่ยมในตอนนี้ เพราะบีก merge ของ MoE ตัวเดียวกันบล็อกการ export เป็น GGUF ของมันด้วย บีกเดียวกันจึงบล็อกทั้งสองรอบ ไม่ใช่แค่ t02

แก้ยังไม: ยังไม่ได้แก้ — ตั้งธงไว้ว่า 'ยังไม่ได้พิสูจน์' ในหัวข้อคำถามค้างคา

กฎที่ได้: ตัดสินโมเดลผู้ทำซึ่งบนเส้นทางที่จะเอาไปใช้งานจริง ไม่ใช่เส้นทางที่ใช้แค่วัดผล และถือว่าความสอดคล้องของ input ระหว่างตอนเทรนกับตอน inference เป็นตัวแปรเปรียบเทียบหลัก ไม่ใช่เรื่องคิดทีหลัง

สูง 2026-08-04

eval_fields.py ไม่เคยบันทึกว่า element ตัวไหนที่พลาด — บันทึกแค่จำนวน

เกิดอะไรขึ้น: สคริปต์วัดผลบันทึกผลระดับหน้าอย่างเดียว (เช่น 'element 9/14') ไม่เคยเขียนออกมาเลยว่า element id ตัวไหนที่พลาด ทั้งที่ตอนจับข้อมูลนั้นอยู่ในหน่วยความจำอยู่แล้ว ('pmap'/'ge'/'pe')

ต้นเหตุจริง: ตัวประเมินถูกสร้างมาเพื่อให้ตัวเลขคะแนนพาดหัว ไม่ได้สร้างมาเพื่อช่วยวินิจฉัย รายละเอียดรายตัวถูกคำนวณแล้วโยนทิ้งไปเลย

เสียหายอะไร: วัด recall ได้แต่วิเคราะห์ไม่ได้ — ไม่มีทางรู้ว่าที่พลาดไปกระจุกอยู่ที่ไหน ที่มาร์คจางๆ หรือที่ pattern ใด pattern หนึ่ง ทุกครั้งที่พยายามปรับปรุงคือการเดา และต้องรันใหม่ทั้งรอบแค่เพื่อขอรายชื่อ

แก้ง่าย: ยังไม่แก้ — ระบุว่าต้องเพิ่มบรรทัด log แค่บรรทัดเดียว (ไม่ใช่แก้ตรรกะ เพราะกฎของ t02 ห้ามแตะตรรกะการวัดผล) แล้วรันใหม่

กฎที่ได้: ตัวประเมินต้องพ่นรายชื่อสิ่งที่พลาดออกมา ไม่ใช่แค่คะแนน ตัวเลขที่ตกย่อยไม่ได้คือตัวเลขที่ปรับปรุงไม่ได้ — ให้ log สิ่งที่พลาดตอนจับคู่เลย เพราะตอนนั้นข้อมูลอยู่ในมืออยู่แล้ว

สูง 2026-08-20

แผนวัดผลรอบก่อนเลือกบ้าน 01/03/04 — ซึ่งอยู่ในชุด train/val หมดแล้ว เท่ากับวัดการท่องจำ

เกิดอะไรขึ้น: ร่างแผนทดลอง (ทดลองถอดแบบ_01_03_04_workflow.md) เลือกบ้าน 01/03/04 มาทดสอบความสามารถในการ generalize ของ t02 พอ grep เทียบกับ train.jsonl/val.jsonl พบว่าทั้งสามหลังอยู่ในข้อมูลเทรน เลยเปลี่ยนไปใช้บ้าน 08/09/10/11 หลังยืนยันว่าไม่อยู่ในชุดเทรน

ต้นเหตุจริง: เลือกบ้านสำหรับวัดผลตามความสะดวก (บ้านที่เข้าใจดีและมีเฉลยดี) โดยไม่ได้เช็กกับ manifest ของชุดเทรนแบบอัตโนมัติ และไม่มีตัวกันอัตโนมัติที่ห้ามเอาบ้านในชุดเทรนมาใช้วัดผล

เสียหายอะไร: จับได้ก่อนเสียเงินค่า GPU ถ้าปล่อยให้รันไป มันจะให้ตัวเลข recall ที่ดูดีแต่จริง ๆ วัดการท่องจำ แล้วตัวเลขนั้นจะถูกเขียนลง SUMMARY.md และมีคนเชื่อ

แก้ง่าย: ยืนยันด้วย grep ว่าไม่อยู่ใน train.jsonl/val.jsonl แล้วเปลี่ยนไปใช้บ้าน 08/09/10/11 พร้อมเขียน ทดลองถอดแบบ_08_09_10_11_workflow.md เป็นเอกสารเตรียมก่อนรันตาม rule_of_tune §4 ส่วนร่างเดิมปล่อยทิ้งไว้

กฎที่ได้: ทุกครั้งที่จะรันวัดผล ต้อง grep บ้านที่จะใช้เทียบกับ train.jsonl/val.jsonl ก่อนเข้า GPU และเอกสารเตรียมงานต้องบันทึกการเช็กนี้ไว้ด้วย — การวัดผลบนข้อมูลเทรนให้ตัวเลขที่แย่กว่าการไม่มีตัวเลขเลย

สูง 2026-08-24

แบ่ง val ครั้งแรกได้ 10:1 — เล็กเกินกว่าจะวัดอะไรได้ โดยเฉพาะ plan_beam

เกิดอะไรขึ้น: dataset ถูกสร้างครั้งแรกโดยให้ val เป็นบ้านหลัง 03 อย่างเดียว ได้อัตรา train:val = 10:1 และมีตัวอย่าง plan_beam ใน val แค่ 4 ตัว มะขามสั่งเปลี่ยน (4/1 ดีกว่า)

ต้นเหตุจริง: กำหนดขนาด val ตามความสะดวกของจำนวนบ้าน ไม่ได้ดูว่า subtask ที่เป็นคอขวดจะได้ตัวอย่างกี่ตัว ทั้งที่รอบนี้ทำเพื่อ plan_beam โดยเฉพาะ ตัวอย่าง val 4 ตัวแยกสัญญาณจาก noise ไม่ได้เลย

เสียหายอะไร: ตัวเลขวัดผลของ plan_beam ที่มาจาก 4 ตัวอย่างจะไม่มีคามหมายทางสถิติเลย รอบนี้จะได้ตัวเลขที่พิสุจน์ผิดไม่ได้

แก้ง่าย: สร้างใหม่โดยให้ val = บ้าน 01-05 (ทุกหลังที่คนรื้อแล้ว) → train 887 / val 229 = 3.87:1 และ plan_beam ใน val จาก 4 เป็น 18 ได้ของแถมคือ val กลายเป็น GT ที่คนรื้อแล้ว 100% ขณะที่ train เป็น op04 ล้วน กลายเป็นไม้บรรทัดที่สะอาด

กฎที่ได้: กำหนดขนาด val จากจำนวนตัวอย่างของ subtask ที่เราตั้งใจจะปรับปรุงจริงๆ ไม่ใช่จากจำนวนบ้าน และควรเอา GT ที่คนรื้อแล้วไปไว้ใน val เพื่อให้ไม้บรรทัดสะอาด

สูง 2026-08-24

ยกเลิกการเทียบกับ base model กลางคัน — ไม่มีหลักฐานว่า fine-tune คือสาเหตุที่ทำให้ดีขึ้น

เกิดอะไรขึ้น: เริ่มรัน Qwen3.6 ตัว base (ไม่ทูน) บนชุดงานเดียวกันเพื่อคุมตัวแปร แต่ยกเลิกไปก่อนที่งานแรกจะเสร็จด้วยซ้ำ เพราะมะขามมีเรียนตอนเช้าและเวลาหมด

ต้นเหตุจริง: การรัน baseline แบบคุมตัวแปรถูกจัดไว้เป็นลำดับสุดท้าย ต่อจากการเรนและการรันตัวที่ทูนแล้ว ในเซสชันข้ามคืนที่มีเวลาจำกัด การวัดที่สำคัญที่สุดทางวิทยาศาสตร์กลับได้เวลาสำรองน้อยที่สุด

เสียหายอะไร: ผลที่ t03 ดีขึ้นกว่า t02 จะเคลมว่ามาจาก fine-tune ไม่ได้ ตารางเปรียบเทียบต้องส่งออกโดยคอลัมน์ base เขียนว่า n/a จริงๆ ไม่ใช่ยังไม่วัด ยังค้างอยู่จนตอนนี้

แก้งัง: ยังไม่ได้แก้ ภาพเปรียบเทียบ (เปรียบเทียบผลทูน_t03_2026-08-25.png) ระบุชัดว่าข้อสรุปเรื่องสาเหตุยังพิสูจน์ไม่ได้ ยกไปรอบหน้า

กฎที่ได้: จัดตาราง baseline แบบคุมตัวแปรไว้ก่อน หรือรันคู่ขนานไปเลย อย่าไว้ท้ายสุด กราฟผลลัพธ์ที่ไม่มีคอลัมน์กลุ่มควบคุมคือข้ออ้างที่เราปกป้องไม่ได้

สูง 2026-08-24

เทียบผลโมเดล tuned กับ untuned เหลือแค่ 6 หน้าจาก 34 หน้า หลังตัด scope ไปสามรอบ

เกิดอะไรขึ้น: ขอบเขตหน้าถูกตัดจาก 34 → 22 → 12 ระหว่างทาง แล้วมะขามตัดทุกหน้าที่ t01 ทำให้จบออกอีก เหลือ 6 หน้าที่ยังสองตัวทำได้ครบ พาดหัวว่า 'JSON ถูกต้อง 100%' และผลเทียบ tuned กับ base ทั้งหมดยืนอยู่บน 6 หน้า

ต้นเหตุจริง: เราตัด scope แบบตอบสนองไปเรื่อยๆ ตามหน้าที่ติดขัด ผลคือตัวอย่างที่รอดมาคือชุดหน้าที่โมเดลทำได้ง่ายพอดี เป็น selection bias ไม่ใช่ตัวอย่างสุ่ม

เสียหายอะไร: ข้อสรุป tuned เทียบ base มาจากชุดง่ายที่คิดตัวเองมา ทั้งประโยค 'base มักเร็วกว่า' และการอ่านคุณภาพอยู่บน n=6 โดยหน้ายากถูกเอาออกไปแล้วตั้งแต่ต้น

แก้งัง: ยังไม่ได้แก้ — รายงานผลไปตามจริงพร้อมระบุจำนวนหน้ากำกับไว้

กฎที่ได้: การตัดรอบที่ล้นออกไม่ใช่การย่อ eval ให้สั้นลง แต่เป็นการเปลี่ยนสิ่งที่ eval กำลังวัด ให้บันทึกความล้มเหลวเป็นความล้มเหลวในตารางเปรียบเทียบ อย่าลบหน้านั้นออกจากกลุ่มตัวอย่าง

สูง 2026-08-25

จดเป็นข้อเท็จจริงว่า 'รูปไม่ได้ถูกย่อขนาด' ทั้งที่วัดจากตัวอย่างเดียว

เกิดอะไรขึ้น: ทีมเคยบันทึกไว้เป็นข้อเท็จจริงว่ารูปไม่ได้ถูกย่อขนาด ข้ออ้างนั้นมาจากการวัดรูปเดียวแล้วหารรวมทั้งชุด ซึ่งผิดกับข้อมูล 100% ของชุด

ต้นเหตุจริง: วัดตัวอย่างเดียวแล้วถือเป็นการวัดทั้งประชากร แคมจดลงบันทึกเป็นข้อเท็จจริงที่ตั้งไว้แล้ว เลยไม่มีใครกลับมาตรวจซ้ำเป็นบทเรียนเดียวกับข้อ 12 ใน rule_of_tune.md ที่ทีมเคยเจอมาแล้วตอนรอบ t01

เสียหายอะไร: บันทึกว่า 'อันนี้ปกติดี' ที่เป็นเท็จ ทำให้ไม่มีใครไปตรวจคอขวดตัวจริง กินเวลาหลายรอบเรนและค่า GPU หลายร้อยดอลลาร์

แก้งัง: เปลี่ยนจากข้ออ้างที่มาจากตัวอย่างเดียว มาเป็นสคริปต์วัดทั้ง dataset ที่ไลขนาดรูปทุกใบพร้อมต้นทุน token

กฎที่ได้: ห้ามจดข้ออ้างที่ครอบคลุมทั้ง dataset จากตัวอย่างเดียว ให้วัดครบทุกใบ หรือติดป้ายให้ชัดว่า 'ตัวอย่าง 1 ใบ ยังไม่ได้พิสูจน์'

สูง 2026-08-25

plan_beam 0% กับ section 0% หน้าตาเหมือนกันบนกระดานคะแนน แต่สาเหตุตรงข้ามกัน

เกิดอะไรขึ้น: สอง subtask รายงาน recall 0% เท่ากัน แต่การวิเคราะห์เผยพบว่า plan_beam มีเพดาน 88% (โมเดลล้มจริง) ส่วน section มีเพดานแค่ 4% บนบ้านที่ไขว่คว้า (ตัวชี้วัดเป็นไปไม่ได้ตั้งแต่ต้น) ถ้าเอามาเฉลี่ยหรือสรุปรวมกันจะได้ข้อสรุปว่า 'โมเดลอ่านแบบไม่เก่ง'

ต้นเหตุจริง: ตัวเลขที่หน้าตาเหมือนกันแต่มาจากคนละสาเหตุ ถูกรายงานเรียงข้างกันโดยไม่มีคอลัมน์เพดาน กระดานคะแนนจึงแยกไม่ออกว่าอันไหนโมเดลพัง อันไหนวิธีวัดพัง

เสียหายอะไร: เสี่ยงสรุปผิดในระดับโปรเจกต์และทุ่มเงินผิดที่ แคมหลังจากนี้ ต่อให้ทำอะไรไป คะแนนของ section ก็ยังบอกไม่ได้ว่าการแก้ันได้ผลหรือเปล่า

แก้ยังไง: เพิ่มคอลัมน์เพดานต่อ subtask ในรายงาน และแยกประเภทของทุก 0% ตามสาเหตุ (โมเดลพัง vs. วิธีวัดพัง) ก่อนตัดสินใจว่าจะลงแรงตรงไหน

กฎที่ได้: รายงานตัวเลขคู่กับเพดานที่ทำให้จริงเสมอ เลขศูนย์สองตัวที่เพดานต่างกันคือโรคคนละโรค ห้ามรวมเป็นข้อสรุปเดี่ยวเด็ดขาด

สูง 2026-08-25

ปัญหาเพดาน ID ไม่ได้มีแค่ section — notes ทำได้สูงสุด 2% และคะแนนทุก subtask ไม่เคยถูกปรับตามเพดานเลย

เกิดอะไรขึ้น: ผลจาก measure_id_ceiling.py บอกว่าทุก subtask มีเพดานต่ำกว่า 100%: notes อ่าน id ได้ 5 จาก 272 = 2%, section 37%, schedule 61%, plan_footing 75%, plan_slab 92%, plan_beam 88% แต่รอบ t01–t03 ทั้งหมดวัดและเทียบกันบนสเกลดิบ 100% ดังนั้น 'plan_footing 33%' จริงๆ คือ 33 จากเพดาน 75 และ notes ก็แทบให้คะแนนไม่ได้ตั้งแต่แรก

ต้นเหตุจริง: ระบบ evaluation ถูกสร้างมาเทียบสตริง element_id โดยไม่มีใครตรวจสอบว่า id เหล่านั้นมีอยู่จริงบนแบบสั๊กกี้เปอร์เซ็นต์ 0% ของ section ดังพออนุญาตให้เกิดการตรวจ ส่วน subtask ที่เจียบกว่าไม่เคยดังพอ

เสียหายอะไร: การเทียบข้ามรอบทุกครั้งในประวัติโปรเจกต์ ใช้ตัวหารที่ขยับได้และไม่เคยปรับ ตัวเลข schedule '0% → 21.9%' และ plan_footing 'ค่าที่ 33.3%' ความหมายเปลี่ยนทันทีเมื่อคิดเพดาน และ notes ถูกลากมาเป็น subtask ที่ไม่มีทางผ่านได้เลย

แก้ยังไง: ยังไม่ได้แก้ — เสนอให้เพิ่มคอลัมน์รายงานที่ปรับตามเพดาน (ปลอดภัย แค่สคริปต์) แต่ยังไม่ทำได้ทำ

กฎที่ได้: พอเจอว่าตัวชี้วัดหนึ่งวัดสิ่งที่เป็นไปไม่ได้ ให้รันเช็คเพดานกับตัวชี้วัดพี่น้องทุกตัวในรอบเดียวกันเลย ตัวที่พังไม่ค่อยพังอยู่ตัวเดียว

สูง 2026-08-25

คะแนน **subtask** หุ้กรอบมาจากบ้านที่ใช้วัดหลังเดียว (บ้าน 08) ซึ่งหน้า **section** เป็นรั้กับงานประปา

เกิดอะไรขึ้น: เพดานของ section คำนวน 'เฉพาะบ้าน 08 ที่ใช้วัดผลจริง' ซึ่งเผยว่าตัวเลขที่ขึ้นาการเรนสามรอบมาจากการวัดบ้านหลังเดียว บ้าน 08 เป็นบ้านชั้นเดียว และหน้า section แทบทั้งหมดเป็นรายละเอียดรั้/ประตู/ท่อน้ำ ไม่ใช่หน้าตัดคานหรือเสา เพดานเลยตกไปเหลือ 4% ทั้งที่เพดานทั้งค้ล้งข้อมูลอยู่ที่ 37%

ต้นเหตุจริง: เลือกบ้านหลังเดียวเป็นชุดวัดผล น่าจะเพื่อความเร็ว แล้วไม่เคยตรวจว่าบ้านหลังนั้นเป็นตัวแทนของทั้งค้ล้งข้อมูลได้จริงไหม ก่อนจะทำตัวเลขมาใช้เลือกว่าจะแก้ะไร

เสียหายอะไร: ผลต่างระหว่างรอบ (schedule 0→21.9%, plan_slab 0→20%) เป็นการอ่านจากตัวอย่างเดียว ซึ่งอาจขยับได้แค่เพราะเปลี่ยนบ้าน และการตัดสินใจที่แพงที่สุดของสัปดาห์ — 'เป้าหมายคือ plan_beam' — ก็ยืนอยู่บนบ้านหลังนี้หลังเดียว

แก้ยังง: ยังไม่ได้แก้ — ยังไม่ได้สร้างชุดวัดผลแบบหลายบ้าน

กฎที่ได้: ก่อนจะให้ตัวเลขไหนมาชี้ว่ารอบเรน \$5 รอบหน้าจะเรนอะไร ให้เช้ก่อนว่ามันมาจากบ้านก้หลัง กระดานคะแนนจากบ้านเดียววัดตัวบ้านพอๆ กับที่วัดโมเดล

สูง 2026-08-25

การแก้ **ground truth** เป็นชุดใหญ่ทำให้ **element_id** บางตัวที่พิมพ์อยู่บนแบบจริงถูกลบทิ้งไปด้วย

เกิดอะไรขึ้น: การแก้ไขชุดใหญ่ที่ได้รับอนุมัติแล้วเปลี่ยน element id ของ section ไป 1,534 รายการเป็น null โดยอิงจาก regex ของ PRINTED_ID id ที่ยาวแต่เป็นข้อความที่พิมพ์อยู่บนแบบจริง — เช่น `เหล็ก WF 150x150x7x10` — ถูกทำให้เป็น null ไปด้วยความเสียหายพลอยได้ และการสร้างชุดข้อมูลรอบถัดไปจะเรนให้เป็น null ตามนั้น

ต้นเหตุจริง: ใช้ heuristic แบบ regex ในการขับเคลื่อนการแก้ไข ground truth เป็นชุดใหญ่ ซึ่งมันแยกไม่ออกระหว่างข้อความสเปกที่พิมพ์อยู่บนแบบจริงยาวๆ กับชื่อที่ถูกแต่งขึ้นเอง

เสียหายอะไร: label ที่อ่านได้จริงบางส่วนจะถูกเรนเป็น null สอนให้โมเดลข้ามค่าที่จริงๆ อ่านได้ ความเสี่ยงนี้ได้แจ้งและยอมรับไว้ล่วงหน้าก่อนลงมือแล้ว

แก้ยังง: แจ้งความเสี่ยงเป็นประโยคเดียวให้มะขามรับทราบก่อนรัน และได้รับการยอมรับแล้ว (att1235) การกู้คืนทำได้เป็นรายการเพราะชื่อเดิมยังถูกเก็บไว้ใน `element\_name\_assigned`, git สะอาดก่อนรัน (ใช้เป็นตัวสำรอง) และสคริปต์เป็นแบบ idempotent (รันซ้ำ = ไม่มีอะไรเปลี่ยน)

กฎที่ได้: เมื่อใช้ heuristic ขับเคลื่อนการแก้ไข ground truth เป็นชุดใหญ่ ให้บอกทิศทางของความผิดพลาดที่อาจเกิดก่อนรันเสมอ, เก็บค่าเดิมไว้ในฟิลด์แยกเพื่อให้ย้อนกลับได้เป็นรายการรายการ, ทำให้สคริปต์เป็น idempotent และเริ่มจาก git ที่สะอาดเพื่อใช้เป็นตัวสำรอง

สูง 2026-08-25

แก้ไฟล์ **ground truth** ของ **section** ไป 343 ไฟล์ แต่ตั้งใจปล่อย **train/val/test.jsonl** ไว้เหมือนเดิม — ชุดข้อมูลตอนนี้ไม่ตรงกับ **ground truth** แล้ว

เกิดอะไรขึ้น: การเปลี่ยนกติกา **element_id** ย้าย **element 1,534** รายการไปที่ **element_name_assigned** ครอบคลุม 343 ไฟล์ **section** ใน **json** แก้ไขแล้ว/ แต่ **build_dataset_t03.py** ไม่ได้ถูกรันใหม่ เพราะการรันใหม่จะทับ **train/val/test.jsonl** ปัจจุบันที่ผลวัดของ **t03** ตอนนี้อ้างอิงอยู่ ชุดข้อมูลที่มีอยู่กับ **ground truth** ที่มันมาจากตอนนี้จึงพูดถึงคนละเรื่องกัน

ต้นเหตุจริง: ประวัติการวัดผลถูกผูกติดกับไฟล์ชุดข้อมูลไฟล์เดียว แทนที่จะผูกกับเวอร์ชัน/hash ของชุดข้อมูล ทำให้การรักษาความสามารถเทียบผลได้กับการรักษาความสอดคล้องกับ **ground truth** กลายเป็นสองอย่างที่เราเลือกได้แค่อย่างเดียว

เสียหายอะไร: การแทนหรือประเมินผลใดๆ ที่เริ่มก่อนสร้างชุดข้อมูลใหม่จะใช้ **target** ชื่อที่ตั้งขึ้นเองตัวเก่าไปเสียๆ ซึ่งเป็นปัญหาเดียวกันเป๊ะกับการแก้ **ground truth** พยายามกำจัดไป และความไม่สอดคล้องนี้ก็มองไม่เห็นเลยจากไฟล์ **.jsonl** เพียงอย่างเดียว

แก้ยังไง: จงใจไม่แก้ตอนนี้ — เลื่อนการสร้างชุดข้อมูลใหม่ไปทำในขั้นตอน **t03b** และบันทึกไว้ว่าเป็นขั้นตอนแรกที่ต้องทำของรอบนั้น

กฎที่ได้: ตั้งเวอร์ชันชุดข้อมูลด้วย **hash/ชื่อ** เพื่อไม่ให้เกิดการแก้ **ground truth** บังคับให้ต้องเลือกกระหว่างข้อมูลเก่าที่ค้างกับความสามารถเทียบผลที่เสียไป และต้องติดป้ายเตือนชุดข้อมูลที่ค้างให้เห็นชัด ไม่ใช่แค่เขียนไว้ในบันทึกประจำวันบรรทัดเดียว

สูง 2026-08-30

`infer\_house\_t04.py --all-val` ยังอ่านไฟล์ **`val.jsonl`** เดิมที่ปนเปื้อนอยู่ แทนที่จะอ่านไฟล์ **val** ของแต่ละ **fold**

เกิดอะไรขึ้น: **eval script** ถูกเขียนขึ้นก่อนที่ **k-fold** จะมีอยู่ ดังนั้น **`--all-val`** จึงโหลด **`val.jsonl`** (5 หลังที่ปนเปื้อนข้อมูล) แทนที่จะเป็น **`val\_fold{k}.jsonl`**

ต้นเหตุจริง: ตัวที่ใช้ข้อมูลปลายทางไม่ได้ถูกอัปเดตตามเมื่อโครงสร้างข้อมูลต้นทางเปลี่ยนจากแบบแบ่งครั้งเดียวเป็นแบบ **k-fold** script มีชื่อไฟล์ที่ **hardcode** ไว้โดยไม่รู้จัก **fold** เลย

เสียหายอะไร: ถ้าไม่จับได้ จะได้ตัวเลข **eval** จาก **split** ที่รู้อยู่แล้วว่าปนเปื้อน ซึ่งจะทำให้ปัญหา **recall** สูงเกินจริงกลับมาอีกครั้ง ทั้งที่งาน **k-fold** ทำขึ้นมาเพื่อกำจัดปัญหานี้โดยเฉพาะ

แก้ยังไง: แก้แบบชั่วคราวด้วยการอัปเดตโหลด **`val\_fold0.jsonl`** ทับชื่อ **`val.jsonl`** บนแต่ละเครื่องเช่า หลังจากตรวจสอบว่า **schema** ของ **key** ตรงกันแล้ว — เครื่อง **fold0** ไม่เคยมีไฟล์ **val** เดิมอยู่แล้ว จึงไม่มีความเสี่ยงที่จะทับข้อมูลจริง

กฎที่ได้: เมื่อโครงสร้างข้อมูลเปลี่ยน (จาก **split** เดียวเป็น **k-fold**) ต้อง **grep** หาชื่อไฟล์เดิมที่ **hardcode** ไว้ในทุกตัวที่ใช้ข้อมูลนั้นก่อนรัน **eval** ครั้งใดๆ — **eval script** ที่แอบอ่านไฟล์ผิดจะให้ตัวเลขที่ดูมั่นใจแต่ผิดทั้งหมด

สูง 2026-08-30

train_loss ที่ลดลงเรื่อยๆ กับ **valid JSON 96-100%** ถูกอ่านว่า 'เทรนแข็งแรง **converge** สม่าเสมอ' ทั้งที่โมเดลไม่ได้เรียนรู้อะไรนอกจาก **schema** เลย

เกิดอะไรขึ้น: fold0 จบด้วย train_loss 0.5503 พร้อมข้อความ 'ลดต่อเนื่องตลอด ไม่มีสัญญาณ overfit' ส่วน fold2 จบที่ 0.5590 ถูกบันทึกว่า 'สัญญาณดีว่าทั้ง 4 folds converge สอดคล้องกัน' ทั้งสองข้อความถูกใช้เป็นหลักฐานว่า run นี้ปกติดี การไล่หาสาเหตุภายหลังพบว่า loss ลดลงเพราะโมเดลกำลังจำ schema JSON และคำตอบของบ้านที่เคยเห็นแล้ว มันอ่านแบบก่อสร้างจริงไม่ออกเลย (ทุกภาพถูกยุบเหลือ 256 token เดียว) การอ่านผิดแบบเดียวกันเกิดซ้ำอีกตอนที่ค่าเตือน truncation $17773 > 8192$ ถูกมองข้ามครึ่งหนึ่งด้วยเหตุผล 'loss ที่ลดปกติดีเป็นสัญญาณบวก'

ต้นเหตุจริง: ทั้ง train_loss และความถูกต้องของ format ผลลัพธ์ มองไม่เห็นบั๊กใน preprocessing เลย — โมเดลที่จำ schema กับคำตอบที่เคยเห็นก็ทำให้ loss ลดลงเหมือนกับโมเดลที่เรียนรู้จริง ไม่มี metric ตัวไหนในลูปวัดว่าข้อมูลภาพที่ป้อนเข้าไปมีข้อมูลจริงอยู่หรือเปล่า

เสียหายอะไร: ปล่อยให้เทรนเต็มรอบไป 4 ครั้ง และอัปโหลดขึ้น HF พร้อม sha256 verify ไปแล้ว 2 ครั้ง ด้วยความเชื่อมั่นจาก metric ที่ตรวจจับข้อบกพร่องนี้ไม่ได้ตั้งแต่แรก กว่าจจะรู้ตัวก็ตอน eval กับบ้านที่ไม่เคยเห็นแล้ว

แก้ยังไง: ไม่ได้แก้ที่ metric แต่แทนที่ด้วยกฎของ t05 ที่ต้องตรวจ token ต่อภาพก่อนเทรน การอ่านผลผิดพลาดนี้ถูกบันทึกไว้ในส่วนต้นเหตุของรายงานวันเดียวกัน

กฎที่ได้: loss ที่ลดลงเรื่อยๆ ไม่ใช่หลักฐานว่าโมเดล 'มองเห็น' ข้อมูลจริง สำหรับงานด้านภาพ ต้องมีการเช็คฝั่ง input ควบคู่ไปด้วยอย่างน้อย 1 อย่าง (เช่น จำนวน token ต่อภาพ หรือดูตัวอย่างที่ resolution จริงที่โมเดลได้รับ) ก่อนจะสรุปว่า run นั้นแข็งแรง

สูง 2026-08-30

ติดตามแค่ตัวเลข **recall** รวม **ไม่มีใครเปิดไฟล์ output** ดิบเลยจนมะขามสั่งให้หาสาเหตุ — จึงเจอว่าโมเดลตอบคนละบ้านคนละแผ่นไปเลย

เกิดอะไรขึ้น: ผล eval รายงานว่า plan_beam 6/6=0%, plan_footing 1/1=0% มะขามต้องสั่ง 'หาสาเหตุก่อน มันไม่ควรแยขนาดนั้น' ก่อนที่จะมีใครเปิดไฟล์ output จริงรู้สึกไฟล์ พอเปิดดูก็เจOTHANที่ว่าโมเดลไม่ได้ตอบ element id ที่ใกล้เคียงเลย แต่ตอบด้วยแบบของบ้านคนละหลังและแผ่นคนละแผ่นไปเลย (ป้อนแผนฐานราก ได้คำตอบเป็นแผนหลังคาของบ้านอื่น) ข้อเท็จจริงเชิงคุณภาพแบบนี้มองไม่เห็นในตัวเลข recall แต่มันคือตัวชี้ว่าเป็นปัญหาการรับรู้ ไม่ใช่ปัญหาความแม่นยำ

ต้นเหตุจริง: eval harness รายงานแค่ตัวเลข (recall) ตัวเดียว และทีมงานเชื่อว่ามันเป็นภาพรวมทั้งหมด คะแนน 0% มีสาเหตุได้หลายแบบ (เกือบถูกแต่พลาดนิดเดียว, format ผิด, มองไม่เห็นเลย) ตัวเลขเดียวแยกแยะไม่ได้ ทำให้การวินิจฉัยติดอยู่ที่ 'คะแนนแยจนกว่าจะมีคนบังคับให้เปิดดูจริง'

เสียหายอะไร: ต้นเหตุแท้จริงของความล้มเหลวทั้งรอบนอนรออยู่หลังตัวเลขที่มีให้ดูมาหลายชั่วโมงแล้ว การวินิจฉัยหลังจากนั้นใช้แค่ 3 ขั้นตอนก็เจอ เมื่อได้อ่าน output จริงแล้ว

แก้ยังไง: หาต้นเหตุเจอในเชสชันเดียวกันโดยเปิดดู output จริง, dump image token จาก training log, อ่านซอร์สโค้ด mm_plugin.py ของ llamafactory, และย่อแผนคานจริงลงเหลือ 448×448 เพื่อดูภาพจริง

กฎที่ได้: เมื่อ metric ออกมา 0% (หรือ 100%) ต้องอ่าน output ดิบก่อนตั้งสมมติฐานใดๆ ตัวเลขบอกแค่ว่ามีอะไรผิด แต่มีแค่ผลลัพธ์จริงเท่านั้นที่บอกว่ามันผิดแบบไหน

สูง 2026-08-31

รอบ t04 เปลี่ยนทั้งโมเดลและความละเอียดตัวเลข (precision) พร้อมกัน ทำให้ผลลัพธ์แปลความหมายไม่ได้เลย

เกิดอะไรขึ้น: รอบ t04 เปลี่ยนทั้งตัวโมเดลและ precision พร้อมกันในครั้งเดียว พอมันล้มเหลวก็ไม่มีทางรู้ได้เลยว่าเกิดจากตัวแปรไหน

ต้นเหตุจริง: ไม่มีวินัยเชิงการทดลอง คือควรตรึงตัวแปรอื่นทั้งหมดให้คงที่และเปลี่ยนแค่ตัวเดียวระหว่างแต่ละรอบ

เสียหายอะไร: รอบที่ล้มเหลวไม่ได้ให้สัญญาณที่ใช้งานได้เลยว่าอะไรคือสาเหตุของความล้มเหลว

แก้ยังง: รอบนี้ตั้งใจทำให้เนื้อหาดาต้าเซตเหมือนกันทุกไบต์ในทั้งสองแขน เพื่อให้ความแตกต่างที่เกิดขึ้นมาจากตัวโมเดลเท่านั้น

กฎที่ได้: เปลี่ยนตัวแปรแค่ตัวเดียวต่อการรันที่มีต้นทุนสูง และตรึงทุกอย่างที่เหลือให้คงที่ — ข้อมูลที่เหมือนกันทุกไบต์ในทั้งสองแขนคือสิ่งที่ทำให้การเปรียบเทียบมีความหมายจริง

สูง 2026-08-31

ตัวเลข strict recall 11.8% ที่เป็นหัวข้อ ประเมินความสามารถในการอ่านแบบต่ำเกินจริงมาก เพราะรูปแบบการตั้งชื่อไม่ตรงกัน

เกิดอะไรขึ้น: ตัวชี้วัด strict recall ใช้การเทียบ string แบบเป๊ะๆ โดยไม่มีการ normalize เลย โมเดลอ่านหน้าฐานรากถูกต้อง 100% ตามตำแหน่งแต่ได้คะแนนแค่ 1/6 เพราะเขียน mark id ตามที่พิมพ์บนแบบ ("F4,C1") ในขณะที่ GT แยกเป็น "F4" กับ "C1" คนละตัว เส้น dummy grid ก็ตั้งชื่อต่างกัน (1.5B เทียบ 1') และหน้าฐานรากไม่มี grid_ref_start/end เลยด้วยซ้ำ

ต้นเหตุจริง: ใช้ตัวชี้วัดแบบ strict ตัวเดียวที่ออกแบบมาเพื่อเทียบผลระหว่างรอบ มาอ่านเป็นตัวชี้วัดความสามารถโดยตรง

เสียหายอะไร: ถ้าอ่านแค่ '11.8%' อย่างเดียว จะสรุปว่าโมเดลอ่านแบบแทบไม่ได้เลย ทั้งที่จริงมันอ่านทั้งหน้าฐานรากถูกต้องสมบูรณ์ — อาจนำไปสู่การตัดสินใจเชิงกลยุทธ์ที่ผิดพลาดร้ายแรง

แก้ยังง: เพิ่มตัวชี้วัด mark_base (88.2%) และ position+mark_base (43.5%) พร้อมเขียนสรุปการตีความชัดเจนเพื่อไม่ให้ใครอ่านแค่ตัวเลข strict อย่างเดียว

กฎที่ได้: ห้ามรายงานตัวชี้วัดแบบ strict ตัวเดียวว่าเป็นความสามารถจริง — ต้องคู่กับตัวชี้วัดที่ผ่อนปรนเรื่อง convention และ breakdown รายชั้น พร้อมระบุให้ชัดว่าตัวไหนสะท้อนความสามารถจริง

สูง 2026-08-31

ชุดเฉลย (ground truth) ของบ้านที่ใช้เบนซ์มาร์กมีผิด 2 จุด จับได้ตอนคนตัดสินขั้นสุดท้ายเท่านั้น

เกิดอะไรขึ้น: ทีมทำ ground truth ของ op04 สำหรับบ้านไทยพอเพียง3 โดยให้คนอ่าน 2 คนอ่านแยกกันทุกหน้าด้วยวิธีคนละแบบ ตอนคนตัดสิน (adjudicator) ชุมดูภาพจริง เจอว่าจุดที่สองคนอ่านไม่ตรงกัน 2 จุดนั้น จริงๆ คือ error ในชุดเฉลยเอง — เส้นกริดหลอก (dummy gridline) ที่ควรชื่อ 3' ดันเขียนเป็น 4' และทิศทางแกน y ของคาน B0 ผิด

ต้นเหตุจริง: การอ่านแบบก่อสร้างด้วยตาเปล่ามีโอกาสผิดได้เสมอ ต่อให้อ่านอย่างระมัดระวังแค่ไหน ถ้าใช้คนอ่านคนเดียวทำเฉลย error สองจุดนี้จะหลุดรอดไปแบบไม่มีใครรู้

เสียหายอะไร: ถ้าไม่จับได้ จะมี 2 จาก 34 รายการในเฉลยที่ผิด ทำให้ผลวัด recall จริงครั้งแรกของโมเดล destrier พังนไปทันที — และตัวเลขนี้คือตัวชี้วัดหลักของทั้งรอบ

แก้ยังง: ใช้กระบวนการอ่าน 2 คนอิสระต่อกัน บวกคนตัดสินอีก 1 คน คนตัดสินชมดูแล้วแก้ทั้งสองจุด จากนั้นเช็คเฉลยทั้งชุดซ้ำอีก รอบเทียบกับภาพจริง

กฎที่ได้: ชุดเฉลยที่ทำมือต้องมีคนอ่านอิสระอย่างน้อย 2 คน บวกคนตัดสิน 1 คนเสมอ เพราะเฉลยคือเครื่องมือวัดผล — ถ้าเฉลยผิด ก็แยกไม่ออกว่าโมเดลผิดจริงหรือเฉลยผิดกันแน่

สูง 2026-09-01

เกือบเอา fold ข้อมูลของ t04 ไปใช้กับโมเดลอื่น — มะขามจับได้เอง ไม่ใช่กระบวนการจับได้

เกิดอะไรขึ้น: มีแผนจะเอา fold ของ t04 ไปใช้ฝึกโมเดลอื่น มะขามจับได้เองว่าผิด พร้อมตารางตัวเลขยืนยัน

ต้นเหตุจริง: ไม่มีการติดตามที่มา (provenance) ของ fold/split ว่าเป็นของโมเดลไหน ทำให้การเอา fold ของโมเดลอื่นมาใช้ดูเหมือนเป็นทางลัดที่สมเหตุสมผล หยัดได้เพราะคนสังเกตเห็นเท่านั้น ไม่มีตัวกันอัตโนมัติเลย

เสียหายอะไร: ถ้าไม่จับได้ จะทำให้การแยก train/val ปนเบื้อนกัน และได้ตัวเลขประเมินผลที่ทำให้เข้าใจผิด จับได้ก่อนใช้จริง

แก้ยังง: หลีกเสี่ยงได้ทัน บันทึกไว้เป็นหัวข้อที่ 5 ของเอกสารสอนงาน พร้อมตารางตัวเลขยืนยัน

กฎที่ได้: fold เป็นข้อมูลเฉพาะของแต่ละโมเดล — ต้องบันทึกว่า fold แต่ละชุดเป็นของโมเดลไหน และห้ามเอาไปใช้ซ้ำข้ามรอบฝึก การที่คนจับได้คือโชคดี ต้องเพิ่มฟิลด์ provenance ไว้ป้องกันจริงๆ

สูง 2026-09-01

สคริปต์ตรวจให้ผ่านด่าน \$2 ดันนัการไล่พิมพ์ชื่อมาร์คมั่วๆ (B1...B34) เป็นคำตอบที่ถูกต้อง

เกิดอะไรขึ้น: นอกจากปัญหาตัวอย่างหลุดเข้าไปในชุดทดสอบแล้ว โมเดล t44 แบบตาบอด (blind) ผ่านหนึ่งหน้าด่านทดสอบได้แค่จากการพิมพ์ชื่อมาร์คไล่ลำดับ B1, B2, B3 ... ไปจนถึง B34 แล้วบังเอิญสามตัวแรกตรงกับเฉลยจริง สคริปต์ตรวจให้ับว่านั่นคือคำตอบที่ถูก ทั้งที่โมเดลไม่ได้เห็นแบบจริงเลยแม้แต่นิดเดียว

ต้นเหตุจริง: ตัวชี้วัดของด่านตรวจแบบนี้วัดแค่ recall — เช็คแค่ว่าเฉลยจริงโผล่อยู่ในคำตอบไหม ไม่มีการหักคะแนนจากความแม่นยำ (precision) และไม่มีเพดานว่าโมเดลตอบได้กี่ตัว โมเดลไหนก็ตามที่ไล่พิมพ์ชื่อในรูปแบบที่คาดเดาได้ (B1..Bn, C1..Cn) จะได้คะแนนไม่เป็นศูนย์โดยอัตโนมัติ

เสียหายอะไร: รวมกับบั๊กตัวอย่างหลุดเข้าไปในเฉลย ทำให้ด่านตรวจ go/no-go ขึ้นเขี้ยวปลอม ถ้าเชื่อผลนี้แล้วเดินหน้าเทรนจริงจะแพงเต็มจำนวน \$60-70 ไปกับโมเดลที่มองไม่เห็นอะไรเลย ชำร่อย t04 เป๊ะ

แก้ยังง: บันทึกไว้ในไดอารีเรื่องกฎห้ามใส่ตัวอย่างที่อยู่ในเฉลยลงไป prompt ที่ใช้ทดสอบ แต่ไม่มีบันทึกว่าเพิ่มการหักคะแนนจากการตอบเกินหรือเพดานคำตอบ — ช่องโหว่การไล่พิมพ์มั่วยังไม่ได้ปิด ยังไม่ได้แก้

กฎที่ได้: ตัวชี้วัดที่วัดแค่ recall จะได้ง่ายๆ ด้วยการเดาทุกอย่าง ด่านตรวจต้องมึบทลงโทษการตอบเกิน (precision หรือเพดานจำนวนที่ตอบได้) ไม่งั้นโมเดลที่พนชื่อทั้งหมดในรูปแบบที่คาดเดาได้จะผ่านด่านเสมอ

เรื่องที่เหลือในหมวดนี้ (38)

ตัวเลข baseline อาจถูกวัดภายใต้ resolution ของภาพที่ผิด

กลาง 2026-07-21

การวัด baseline กับการวัดหลัง training ต้องรันภายใต้ config เดียวกันเป๊ะและมีการยืนยันชัดเจน ต้อง log และเทียบ setting จริงของทั้งสองรอบ ไม่ใช่แค่เทียบคะแนน

ตัวเลขคุณภาพหลักที่รายงาน (0%→90%) วัดจากแค่ 20 ตัวอย่างจาก 88 ตัวอย่างในชุด validation ทั้งหมด

กลาง 2026-07-21

debug ด้วย `--limit` ได้ แต่ตัวเลขที่จะเขียนลงตารางผลสรุปต้องมาจากการรัน validation เต็มชุดเท่านั้น และต้องระบุขนาดตัวอย่างกำกับไว้ข้างเปอร์เซ็นต์ที่รายงานทุกครั้ง

ชุด validation ทั้งหมดดึงมาจากบ้านหลังเดียว (house03) — ไม่เคยวัดความสามารถของโมเดลข้ามบ้านเลย

กลาง 2026-07-21

เมื่อมีเอกสารต้นทางน้อย ควรหมุนเวียนบ้านที่ใช้ hold-out (cross-validation) หรือ hold-out หน้าจากหลายบ้าน ไม่งั้นจุดอ่อนเฉพาะบ้านนั้นๆ จะแยกไม่ออกจากจุดอ่อนจริงของโมเดล

รายงาน 'element_id ตรงกัน 100%' ว่าเป็นผลลัพธ์คุณภาพ ทั้งที่มันเป็นการวนซ้ำ (circular) เพราะจับคู่ element ด้วย element_id เอง

กลาง 2026-07-21

ห้ามรายงานว่าตรงกัน 100% กับฟิลต์ที่ใช้เป็น key ในการจับคู่เอง และต้องระบุกฎการจับคู่กับจำนวน n กำกับไว้ทุกครั้งที่ยานเมทริกซ์แบบมีเงื่อนไข (ในกรณีนี้ $n=16$)

ตัวเลขหลักของ t01 กับ t02 วัดมาจากคนละเครื่องเข้า และอาจใช้ library คนละเวอร์ชัน

กลาง 2026-07-28

เก็บรายละเอียดสภาพแวดล้อมไว้คู่กับทุกตัวเลขที่วัดได้ และถ้าจะเปรียบเทียบกัน ต้องวัดทุกฝั่งบนเครื่องเดียวกันในรอบเดียวกัน ตัวเลขที่ไม่รู้ว่ามีวัดมาในเงื่อนไขไหน เอาไปเทียบกับอะไรไม่ได้เลย

เอาโมเดลไปวัดผลกับหน้าประเภทที่ pipeline คัดทิ้งอยู่แล้ว

กลาง 2026-07-29

วัดผลบนข้อมูลชุดที่ระบบใช้งานจริง ถ้าจะรันวินิจฉัย ให้เรียงตามความเกี่ยวข้องกับ pipeline จริง ไม่ใช่เรียงตามลำดับไฟล์หรือเลขหน้า ไม่งั้นจะเสี่ยงวัดผลไปกับข้อมูลที่สูญหายก็ถูกกรองทิ้ง

เกือบเชื่อคำตัดสิน 'ผลลัพธ์แย่มาก' ตรงๆ ทั้งที่ต้นเหตุจริงเจอเพราะไปใส่ดูว่าอะไรบ้างที่ทำได้ถูก

กลาง 2026-07-29

ห้ามวินิจฉัยจากคะแนนรวม ให้แยกผลวัดออกเป็นส่วนที่โมเดลทำถูกกับส่วนที่ทำผิด แล้วดูรูปแบบของการแบ่งนั้น ถ้าเจอรูปแบบที่ผิดธรรมชาติ เช่น เรื่องง่ายผิดแต่เรื่องยากถูก นั่นคือสัญญาณบอกสาเหตุที่ตัวเลขเปอร์เซ็นต์ตัวเดียวกลบมันหายไปหมด

ทุกหน้าที่เคยเอาโมเดลไปรันจริง ล้วนเป็นข้อมูลที่ใช้เทรนมันเอง ไม่เคยทดสอบกับหน้าที่กันไว้เลย

กลาง 2026-07-29

กันบ้านไว้อย่างน้อยหนึ่งหลัง (หรือชุดหน้าตายตัวชุดหนึ่ง) ไม่ให้เข้าชุดเทรนเด็ดขาด แล้วใช้ชุดนั้นตรวจคุณภาพก่อนใช้งานจริงทุกครั้ง ตัวเลขคุณภาพที่วัดจากข้อมูลเทรน ตอบคนละคำถามกับที่เรากำลังถามอยู่ และรอบนี้มันบังเอิญกลายเป็นตัววินิจฉัยสำคัญ เพราะปัญหาอยู่ที่รูปแบบ ไม่ใช่เนื้อหา

เอาโมเดลไปวัดผลกับหน้า (index/notes/material_list/site_plan) ที่ KEEP PATTERNS คัดทิ้งอยู่แล้ว

กลาง 2026-07-29

ชุดวัดผลต้องดึงมาจากตัวกรองขอบเขตที่ใช้ตอนทำงานจริง ถ้าเอาโมเดลไปตัดสินกับข้อมูลที่ pipeline ทิ้ง จะได้คำตัดสินที่เอาไปทำอะไรต่อไม่ได้เลย

ตัดสินโมเดลว่าผิดเพราะไม่ออกช่อง confidence ทั้งที่ตั้งใจตัดช่องนั้นออกจาก prompt และชุดเทรนไปเอง

กลาง 2026-07-29

ก่อนให้คะแนนเทียบกับเฉลย ให้หักช่องที่เราตั้งใจตัดออกจาก prompt หรือข้อมูลเทรนออกไปก่อน ไม่งั้นเราจะกำลังตรวจการตัดสินใจเรื่องขอบเขตของตัวเอง แล้วนับเป็นความผิดของโมเดล

การทดสอบความสามารถในการทั่วไปของ t02 ไม่เคยทำจนจบ บ้านหลัง 10/11 ไม่ได้เริ่มเลย และหลัง 08 ก็ไม่ถึงเพดาน

กลาง 2026-08-21

ถ้าตัดขอบเขตงานลงจนการทดลองตอบคำถามของตัวเองไม่ได้แล้ว ให้ติดป้ายว่าตัวเลขที่ได้เป็นค่าชั่วคราวทุกที่ที่เอาไปอ้าง ตัวเลขพาดหัวที่มาจากบ้าน 1.5 หลังจากท้วงแผนไว้ 4 หลัง ต้องบอกขนาดตัวอย่างติดไปด้วยทุกครั้ง

ตัวเลขที่คนทำเองก็ยังวางลงบนแบบได้ไม่ครบ (45/48 กับ 52/53)

กลาง 2026-08-21

ตรวจสอบตัวเลขก่อนจะเอาไปใช้เป็นไม้บรรทัด ให้รายงานอัตราที่ตัวเลขเองระบุตำแหน่งได้ควบคู่กับ recall ของโมเดล ไม่งั้นเรา กำลังให้คะแนนโมเดลด้วยเฉลยที่ไม่ครบ โดยไม่มีใครรู้ว่าขาดไปแค่ไหน

ตัวประเมินผลเทียบ element ID แบบตรงตัวเป๊ะ ช่องว่างกับตัวพิมพ์เล็กใหญ่เลยทำให้นับว่าพลาดทั้งที่ตรง

กลาง 2026-08-25

ตัวให้คะแนนที่เทียบตรงต้อง normalize ก่อนเสมอ (ตัดช่องว่างหัวท้าย แปลงเป็นตัวพิมพ์เดียวกัน ยุบช่องว่างซ้ำใน) และควร log คู่ที่เกือบตรงไว้ด้วย จะได้เห็นว่าพลาดเพราะรูปแบบข้อความ ไม่ใช่ถูกให้ศูนย์ไปเสียๆ

HoughCircles ตรวจป้ายชื่อ section ได้ 5/5 เต็มในบ้านหลังเดียว แต่พอไปหลังอื่นเหลือ 3/20 กับ 0/18

กลาง 2026-08-25

เทคนิค computer vision ที่ได้คะแนนเต็มบนเอกสารหลังเดียว ถือว่ายังไม่ได้พิสูจน์อะไรเลย ต้องทดสอบกับอย่างน้อย 3 แหล่งที่มา จากคนละต้นทางก่อนจะบอกว่ามันใช้ได้ และให้จัดวิธีที่ล้มเหลวไว้ด้วย จะได้ไม่มีใครมาลองซ้ำ

หาโมเดลที่ไม่ใช่ Qwen แล้วได้ InternVL3 มา ซึ่งข้างในก็ใช้ backbone ของ Qwen อยู่ดี

กลาง 2026-08-25

ถ้าโจทย์คือการสถาปัตยกรรมที่ต่างกันจริงๆ ให้ไปดู backbone ใน model card หรือ config ชื่อแบรนด์มักบ่งไว้ว่าข้างในใช้ฐานเดียวกัน

งานสำรวจ VLM ทางเลือก 10 ตัว จัดอันดับจากสเปกบนกระดาษ มีแค่ 1 ใน 10 ที่มีคะแนน benchmark จริง

กลาง 2026-08-25

เอาเกณฑ์ที่ปลอมยากที่สุดขึ้นก่อนในการคัดกรอง licence กับจำนวนพารามิเตอร์อ่านได้ฟรี ซึ่งนั่นแหละคือเหตุผลว่าทำไมมันไม่ควรเป็นตัวที่ใช้เรียงอันดับ

การนับจุดเล็กเสริมได้ 1/5 แต่การทดสอบวัดคุณภาพภาพที่ crop มือ ไม่ได้วัดตัวโอเดีย

กลาง 2026-08-25

ถ้า input ของการทดลองทำด้วยมือแล้วไม่ได้ตรวจสอบ ผลที่ได้คือการวัดฝีมือมือคุณ ไม่ใช่วัดสมมติฐาน ให้มาร์กผลแบบนั้นว่ายังสรุปไม่ได้ การบันทึกการทดสอบที่ห่วยเป็นข้อพิสูจน์ว่าโอเดียใช้ไม่ได้ จะฆ่าโอเดียดีๆ ไปถาวร

สูตรประมาณ token เป็น chars/2.2 ที่ใช้กำหนด MAX_LENGTH ไม่เคยถูกตรวจสอบกับ tokenizer จริงเลย

กลาง 2026-08-25

heuristic ตัวไหนที่กลายเป็น input ของขีดจำกัดตายตัว ต้องเอาไปตรวจกับข้อมูลจริงสักครั้ง เอา tokenizer ตัวจริงรันกับตัวอย่างที่ยาวที่สุดสองสามตัว แล้วจดอัตราส่วนจริงไว้ ก่อนจะไว้ใจ margin ที่ต่ำกว่า 10%

การเก็บ template อัตโนมัติได้ของขยะเป็นส่วนใหญ่ 45 จาก 57 แล้วก็ 71 จาก 79

กลาง 2026-08-25

ถ้าตัวเก็บอัตโนมัติมี precision ต่ำกว่า 15% ต้องมีคนตรวจเป็นด่านบังคับ และห้ามเลื่อนชั้นให้อัตโนมัติเด็ดขาด จะจัดอันดับเพื่อให้คนตรวจเร็วขึ้นได้ แต่การจัดอันดับที่ตัดสินใจแทนคือทางที่ template พิษเล็ดลอดเข้ามา

เปิด xgrammar ให้ทุก pass ทำให้เทียบผล JSON ถูกต้องกับรอบก่อนหน้าไม่ได้เลย

กลาง 2026-08-29

ทุกครั้งที่เปลี่ยนค่าตอน inference ให้บันทึกไว้คู่กับตัวเลขที่มันกระโดด และเก็บ flag ไว้ปิดมันได้ เพื่อให้วัดผลกระโดดได้จริง ไม่ใช่มาบังเสียกัน

การแบ่ง k-fold กระดาษบ้าน 5 หลังที่คนตรวจแล้วไม่เท่ากัน — fold 0 กับ 3 ไม่ได้เลยสักหลัง

กลาง 2026-08-29

แบ่ง fold โดยถ่วงตามคุณภาพของ ground truth ด้วย ไม่ใช่ดูแค่จำนวนตัวอย่าง fold ที่วัดกับ label ที่ยังไม่มีคนตรวจ วัดคนละอย่างกับ fold ที่วัดกับ label ที่ตรวจแล้ว

สมมติฐานที่ว่า plan_beam พังเพราะหน้ามีผังสองแบบปนกัน ถูกพิสูจน์ว่าผิด — t03 อ่านผังคนไม่ได้จริงๆ

กลาง 2026-08-30

ทดสอบสมมติฐานของความล้มเหลวด้วยการแยกตัวแปรที่มันโทษออกมาตัวเดียว (เคสนี้คือครอบหน้าให้เหลืออันเดียว) ก่อนจะสร้างวิธีแก้ไขใดๆ ทุบลงไป

ช่องว่างข้อมูลจริงสามจุดขวางการสร้าง dataset ของ t05 — ไม่มี GT ของ pass0 เลย, ไฟล์ CV sidecar มีแค่ 10 หน้า และไม่มีตัวจับคู่ cv_mark กับ GT

กลาง 2026-08-30

ก่อนวางแผนรอบเทรนที่พึ่งสัญญาณ input ใหม่ ให้ตรวจก่อนว่า ground truth และไฟล์ sidecar ที่ต้องใช้มีอยู่จริงทั้งชุดข้อมูล — นับให้เห็นตัวเลข อย่าเชื่อเอกสารแผน

เกือบสรุปผิดไปแล้วว่า InternVL อ่าน pixel ได้น้อยกว่า Qwen ถึง 3.3 เท่า เพราะข้อจำกัดของสถาปัตยกรรมเอง

กลาง 2026-08-31

ก่อนเทียบ A/B ต้องคำนวณเพดาน input สูงสุดตามสถาปัตยกรรมของแต่ละตัวเลือกไว้ล่วงหน้า และบันทึกเป็นข้อควรระวังในการตีความ เพื่อไม่ให้เอา ceiling effect ไปเข้าใจผิดว่าเป็นคุณภาพของโมเดล

ตัวตรวจจับด้วย CV เจอกองแค่บางส่วนในราว 20 หน้า ทำให้ pass3 auto-matching ทำงานต่อไม่ได้

กลาง 2026-08-31

เมื่อขั้นตอนถัดไปเจอปัญหาซ้ำๆ กับข้อมูลชุดย่อยหนึ่ง ให้เช็ค recall ของตัวตรวจจับเองในชุดนั้นก่อน อย่าเพิ่งไปเขียน matching logic เพิ่ม เพราะตัว matcher ไม่มีทางถูกส่งที่ไม่เคยถูกตรวจจับได้เลย

บันทึกตัวเลข final loss ไว้สองค่าที่ต่างกันในวันเดียวกัน แล้วสรุปว่า overfit จากคู่ที่ค่าต่ำกว่า

กลาง 2026-08-31

ก่อนใช้ตัวเลข loss เป็นหลักฐานประกอบการตัดสินใจ ต้องระบุให้ชัดว่ามันคือ metric ตัวไหน แล้วอ่านจาก log ใหม่อีกครั้ง — ถ้ามีตัวเลขสองค่าสำหรับ metric เดียว แปลว่ายังใช้อ้างอิงไม่ได้ทั้งคู่

โมเดล soup ที่ merge แล้ว ไม่เคยถูกวัดผลจริงเลยตลอดกาล เพราะบ้านทั้ง 40 หลังถูกใช้เป็นข้อมูลเทรนหมด

กลาง 2026-09-01

ถ้าจะ merge โมเดลแบบ k-fold ต้องกันชุด holdout กลางไว้ตั้งแต่แรกที่ไม่มี fold ไหนได้เทรนบนมันเลย ถ้าไม่ได้กันไว้ ก็ต้องระบุตรงๆ ว่าโมเดลที่ merge แล้วยังไม่ผ่านการวัดผล ห้ามยืมคะแนนของ fold ใด fold หนึ่งมาอ้างแทน

ค่า recall ~43% ของ destrier วัดด้วยการจับคู่ชื่อแบบฟอนปรน ทำให้ตัวเลขนี้เทียบกับคะแนนแบบเข้มงวดไม่ได้

กลาง 2026-09-01

ต้องรายงานคะแนนแบบเข้มงวดควบคู่กับคะแนนแบบฟอนปรน/normalized เสมอ และห้ามยกตัวเลขแบบฟอนปรนมาอ้างเป็น 'ความแม่นยำ' เดี่ยวๆ ทุกครั้งที่บันทึกค่า recall ต้องระบุกฎการจับคู่ที่ใช้ไปด้วย

การจำลองบ้านหลังแรก ให้ Claude อ่านแบบเอง ซึ่งเปลืองโทเค็นและได้ผลลัพธ์แม่นยำน้อยกว่า

กลาง 2026-09-01

เวลาจำลองพฤติกรรมโมเดล ให้ใช้ output ที่รู้แล้วว่าถูกต้อง (ground truth) แทนคำตอบที่โมเดลอีกตัวดันสด — ถูกกว่าและตรงต่อความจริงกว่าอย่างชัดเจน

ออกแบบ k-fold ไว้ 5 กลุ่ม แต่รันจริงแค่ 4 fold ทำให้บ้าน 8 หลังอยู่ในชุดเทรนทุกรอบ และไม่เคยถูกใช้ประเมินผลเลย

กลาง 2026-09-01

ถ้าประกาศว่ามี k กลุ่ม ต้องรันให้ครบ k fold จริง หรือระบุตรงๆ ว่ากลุ่มไหนถูกตัดออกและเพราะอะไร ชุด holdout ที่ไม่เคยถูกกันไว้จริง ก็คือข้อมูลเทรนที่แค่แปะป้าย eval ไว้เท่านั้น

คำเตือนตอนที่มีส่วนหายไป ไปโทษว่าเป็นความผิดของโมเดล ('generation หลุด') ทั้งที่จริงๆ ตัว sanitizer ของเราเองเป็นคนลบข้อมูลนั้นทิ้ง

กลาง 2026-09-01

ข้อความ error ต้องระบุเฉพาะสิ่งที่โค้ดที่โยน error นั้นพิสูจน์ได้จริงเท่านั้น สาเหตุที่เดาเอาไว้ใน log แยกว่าไม่มีสาเหตุเลยเสียอีก เพราะมันจะพาคนไล่ debug คนถัดไปหลงไปผิดไฟล์

ผลลัพธ์การสกัดข้อมูลขึ้นส่วนเขียนตัวหารสองแบบต่างกันในเซลล์เดียวกัน และไม่เคยรายงานค่า precision เลย

ต่ำ 2026-07-21

งานสกัดข้อมูลทุกงานต้องรายงานทั้ง precision และ recall คู่กัน (หรือ F1) — การรายงานแค่ recall เพียงอย่างเดียวจะซ่อนผลลัพธ์ที่มันโน้มน้าตัวเอง (hallucinate) และทำให้รอบถัดไปแก้ผิดทิศทาง

เคยบันทึกว่า ERNIE-4.5-VL ต้องใช้ ERNIEKit เท่านั้น — จริงๆ มี Unsloth notebook ให้ใช้ได้อยู่แล้ว

ต่ำ 2026-08-25

ทุกข้อสรุปเรื่อง tooling รongรับอะไรบ้าง ให้ระบุวันที่และแหล่งที่เช็คไว้ด้วย แล้วเช็คซ้ำก่อนใช้มันตัดตัวเลือกใดๆ ทั้ง — วงการนี้เปลี่ยนทุกสัปดาห์

จำนวนเสาที่ตรวจนับได้ 19 คอสัมพันธ์บ้าน 17 หน้า 15 ทุกรายงานไปโดยไม่เคยตรวจสอบกับแบบจริงเลย

ต่ำ 2026-08-25

ต้องตรวจสอบตัวเลขก่อนใส่ลงตารางผลลัพธ์ ไม่งั้นอย่าใส่เลย — ตัวเลขที่ไม่ได้ตรวจสอบพอมันอยู่บนกับตัวเลขที่ตรวจแล้ว จะถูกอ้างอิงต่อราวกับว่าตรวจสอบแล้วเหมือนกัน

การทดลอง easyocr ล้มเหลว — sheet_code ถูกต้อง 0/4 แม้จะปรับจนไปแล้วสองรอบ

ต่ำ 2026-08-29

ให้ตั้งเงื่อนไขหยุดไว้ก่อนเริ่มเทียบเครื่องมือ ("ถ้ารอบ 1 ต่ำกว่า X ให้หยุด") ผลลัพธ์เชิงลบก็มีค่าและควรบันทึกไว้ — แต่ให้จบแค่การจูนหนึ่งรอบ ไม่ใช่สองรอบ ก่อนจะยอมรับผล

ข้อสรุป 'ไม่มี truncation warning' อ้างอิงจากตัวอย่างแค่ 40/1020 แกวเท่านั้น ไม่ใช่ครอบคลุมทั้งหมด

ต่ำ 2026-08-30

การรันแบบ dry-run ตัวอย่างสั้นๆ พิสูจน์คุณสมบัติของทั้งชุดข้อมูลไม่ได้ สำหรับคำถามเรื่องความยาว/truncation ให้รันแบบ CPU ล้วนได้ครบ 100% ของแถวแทนการอนุมานจากตัวอย่างแค่ 4%

ถ้าใช้วิธี sampling แบบ 'เอา N แถวแรก' แบบตรงไปตรงมา จะได้ตัวอย่างที่อยู่ในบ้านเดียวทั้งหมด เพราะไฟล์ jsonl เรียงตามบ้าน

ต่ำ 2026-08-30

อย่า sample ชุดข้อมูลที่เรียงลำดับอยู่แบบตามลำดับจริงๆ ให้ stratify ตามคีย์การจัดกลุ่ม (บ้าน/entity) และยืนยันว่าตัวอย่างครอบคลุมทุกกลุ่มก่อนเชื่อค่าเมตริกที่ได้

การ fit แบบ least-squares ด้วย anchor แค่ 2 จุด จะได้ residual เป็น 0 เสมอ ทำให้ตัวตรวจคุณภาพที่อิง residual จะผ่านทุกการ fit แม้จะผิดก็ตาม

ต่ำ 2026-09-01

ก่อนจะเชื่อ metric ตัวไหนสำหรับตรวจสอบความถูกต้อง ให้ถามก่อนว่ามันมีสถานการณ์ไหนบ้างที่มันจะ 'ผ่าน' ได้โดยไม่มีข้อมูลจริงรองรับเลย — metric ที่ไม่มีทางล้มเหลวไม่ใช่ตัวกัน มันคือของประดับ

คุณภาพข้อมูล ความถูกต้อง และ ข้อมูลหาย

ข้อมูลผิด ขาด ซ้ำ ปนเปื้อน หรือหายถาวร รวมถึงข้อมูลที่กู้คืนไม่ได้

ทั้งหมด 143 เรื่อง · วิกฤต 26 · สูง 45 · กลาง 60 · ต่ำ 12

เรื่องที่ต้องอ่านให้ครบ (71)

วิกฤต 2026-07-02

โมเดลอ่าน main_bar_type สลับ DB↔RB ทั้งหน้า ทั้งที่สัญลักษณ์เดียวกันนี้อ่านถูกตอนเป็นเหล็กปลอมในหน้าเดียวกัน

เกิดอะไรขึ้น: ตอนลองเทียบ Claude กับ Qwen ครั้งแรก (บ้าน 01 จำนวน 4 หน้า) เจอว่า main_bar_type ถูกอ่านสลับ DB↔RB อย่างเป็นระบบในหน้าหนึ่ง ทั้งที่สัญลักษณ์ตัวเดียวกันในหน้าเดียวกันนั้นกลับอ่านถูกเมื่อเป็นเหล็กปลอม เรื่องนี้กลายเป็นบทเรียนข้อ 6 ใน No_touch_box/CLAUDE.md และยังถูกอ้างถึงจนถึงทุกวันนี้

ต้นเหตุจริง: โมเดลเดาชนิดเหล็กจากขนาดเส้นผ่านศูนย์กลางหรือจากบริบทรอบๆ แทนที่จะดูจากสัญลักษณ์ Ø (วงกลม) ตรงๆ สัญลักษณ์เดียวกันเลยออกมาคนละคำขึ้นกับว่ากำลังเติมช่องไหน ตอนนั้นสเปกยังไม่มีการเขียนไว้ว่า Ø แปลว่า RB เสมอ การเดาจึงไม่มีอะไรกัน

เสียหายอะไร: ชนิดเหล็กแกนหลักผิด ลามไปที่น้ำหนักเหล็ก รายการตัดเหล็ก (BBS) และค่าเหล็กใน BOQ ของทุก element ที่โดน และเพราะมันผิดอย่างเป็นระบบ (ไม่ใช่สุ่ม) มันจึงไม่หักล้างกันเอง ภายหลังต้องมารอบแก้เฉพาะเมื่อ 2026-07-06 ไล่เขียน main_bar.type ของทุก element ในหน้า 21 view1 จาก DB เป็น RB ใหม่ทั้งหมด

แก้ยังไง: บันทึกเป็นบทเรียนข้อ 6 ใน No_touch_box/CLAUDE.md แล้ววันที่ 2026-07-06 เขียนเป็นกฎตายตัวในสเปก ('สัญลักษณ์ Ø (วงกลม) = RB เสมอ ห้ามเดา DB จากขนาดเส้นผ่านศูนย์กลางเด็ดขาด') พร้อมแก้ไฟล์ที่ได้รับผลกระทบ

กฎที่ได้: อย่าปล่อยให้โมเดลเดาค่าที่เป็นประเภท (ชนิดเหล็ก, เกรด) จากค่าที่เป็นตัวเลขต่อเนื่อง (ขนาดเส้นผ่านศูนย์กลาง) ให้เขียนการแมปสัญลักษณ์→ค่าลงในสเปกเป็นกฎเด็ดขาด และถ้าสัญลักษณ์เดียวกันในหน้าเดียวกันอ่านถูกในช่องหนึ่งแต่ผิดในอีกช่องหนึ่ง นั่นคือ prompt/สเปกบกพร่อง ไม่ใช่ข้อจำกัดของการมองภาพ — ให้ไปแก้กฎ ไม่ใช่แก้ตัวอย่าง

วิกฤต 2026-07-05

กำลังอัปเดตคอนกรีตขัดกันเอง 210 กับ 240 ksc ระหว่างการอ่านรอบใหม่กับข้อมูล Gen 1 เก่า และไม่เคียดดสินว่าอันไหนถูก

เกิดอะไรขึ้น: อ่านหน้า 18 (โน้ต S-01) รอบใหม่ได้ concrete_strength_ksc = 210 ($F_c' = 210 \text{ kg/cm}^2$ เขียนชัดในภาพ) แต่ไฟล์ Gen 1 เก่า qwen-output/บ้าน_เล็ก_1ชั้น_01_หน้า18.json บันทึกไว้ 240 ทั้งสองค่าไม่มีใครตรวจสอบยืนยัน เรื่องนี้ยังค้างอยู่จนจบวันที่ 2026-07-06 และถูกยกไปเป็นงานค้างข้ามวันทั้งสัปดาห์

ต้นเหตุจริง: การสกัดข้อมูลสองรุ่นเขียนข้อเท็จจริงเดียวกันลงคนละไฟล์ โดยไม่มีขั้นตอนมาเทียบกัน และไม่มีแหล่งความจริงเดียวสำหรับค่าคงที่ระดับบ้าน ไม่มีอะไรใน pipeline ที่เอาค่าที่อ่านใหม่ไปเทียบกับค่าของรุ่นก่อน หรือหยุดงานเมื่อค่าไม่ตรงกัน ความขัดแย้งจึงนอนอยู่ในชุดข้อมูลได้เสียๆ ไม่มีกำหนด

เสียหายอะไร: ค่าคงที่ทางโครงสร้างหลักของทั้งบ้านคลุมเครือ การคิดราคาคอนกรีตใน BOQ/BBS หรือการคำนวณอะไรก็ตามที่ขึ้นกับกำลังอัด จะผิด 50% เพราะเรื่องนี้ concrete_grade จึงถูกปล่อยเป็น null ในทุก element ของคานที่ join ไว้ใน Gen 3.3 ข้อมูลสเปกที่ join แล้วจึงไม่ครบไปด้วย

แก้ยังไม: ยังไม่ได้แก้ ค้างเป็นงานค้างข้อ 3 ในรายการทำวัน 2026-07-06 ('ความขัดแย้ง concrete_grade 210 กับ 240 ksc') และเขียน concrete_grade เป็น null ทุกที่แทนที่จะเดา

กฎที่ได้: ค่าคงที่ระดับบ้าน (เกรดคอนกรีต, เกรดเหล็ก, ระยะหุ้ม) ต้องอยู่ในไฟล์แม่ไฟล์เดียวต่อบ้าน และต้องเทียบกับทุกรุ่นก่อนหน้าก่อนจะสกัดข้อมูลต่อ เจอความขัดแย้งเมื่อไหร่ให้ล็อกชองนั้นไว้ — เขียน null พร้อมติดธง ห้ามเดา — แล้วตั้งคนรับผิดชอบกับกำหนดเวลาให้เสร็จในวันเดียวกัน ไม่ใช่ปล่อยเป็นบรรทัดค้างที่ยกไปวันแล้ววันเล่า

วิกฤต 2026-07-09

คำสั่งให้เปลี่ยนชื่อโฟลเดอร์ raw/ เป็น 'ข้อมูลเก่าไม่ได้ใช้แล้ว' เกือบทำ pipeline ที่ใช้งานจริงพังใน 5 สคริปต์

เกิดอะไรขึ้น: มีการเข้าใจว่าโฟลเดอร์ raw/ เลิกใช้แล้ว เลยสั่งให้เปลี่ยนชื่อ พอ grep ก่อนลงมือ พบว่ามันมีขนาด 533 MB และถูกเขียน path ตายตัวไว้เป็นเส้นทางที่ยังใช้งานอยู่ใน run_pipeline.py, analyze_folder.py, build_document_map.py, pdf-processor.py และ upload-to-supabase-storage.js (รวม 7 จุด) สุดท้ายเปลี่ยนชื่อไปจริงพร้อมแก้ path ในโค้ด แล้วก็ถูกยกเลิกต้องย้อนกลับทั้งหมด

ต้นเหตุจริง: สถานะของโฟลเดอร์ถูกเดาจากชื่อและจากความจำ ไม่ใช่จากสิ่งที่โค้ดอ้างถึงจริง และชื่อกลางๆ อย่าง raw/ ก็ไม่ได้บอกอะไรเลยว่ายังใช้งานอยู่หรือเปล่า

เสียหายอะไร: ถ้าทำไปเลยจะทำให้ pipeline สกัดข้อมูลหลักพังทันทีบนไดเรกทอรีข้อมูล 533 MB แต่ที่เกิดขึ้นจริงคือเสียเวลาไปกับการเปลี่ยนชื่อ-แล้ว-ย้อนกลับ ทั้ง 5 สคริปต์ ไฟล์ .gitignore และเจอเหตุโฟลเดอร์ถูกล็อกอีกหนึ่งเรื่อง

แก้ยังไม: grep หา path ที่เขียนตายตัวทุกจุดก่อนลงมือ ถามก่อนทำต่อ แล้วย้อนชื่อโฟลเดอร์ สคริปต์ทั้ง 5 ตัว และ .gitignore กลับสู่สภาพเดิม จบด้วย git status ที่สะอาด

กฎที่ได้: ห้ามเปลี่ยนชื่อหรือย้ายโฟลเดอร์ข้อมูลโดยดูจากชื่อมันอย่างเดียว — ให้ grep หา path นั้นทั้ง repo ก่อน และให้ถือว่า 'ผู้ใช้ออกว่ามันเก่าแล้ว' เป็นสมมติฐานที่ต้องพิสูจน์ ไม่ใช่คำสั่งที่ทำตามได้เลย

ถ้ารันตัวสร้าง task ให้บ้านหลังที่ 2 จะไปทับไฟล์ผลลัพธ์ของบ้านหลังที่ 1 แบบเจียบๆ

เกิดอะไรขึ้น: ไฟล์ label-studio-tasks-makham.js เขียนผลลัพธ์ลงชื่อไฟล์ตายตัว (label-studio-tasks-makham-elements.json ฯลฯ) โดยไม่มีชื่อบ้านอยู่ในนั้นเลย ถ้ารันให้บ้านหลังที่ 2 มันจะเขียนทับไฟล์ task ของบ้านหลังที่ 1 ที่สร้างไปแล้ว (และอัปโหลดไปแล้ว) โดยไม่มีค่าเตือนใดๆ

ต้นเหตุจริง: สคริปต์ถูกเขียนขึ้นตอนที่มีบ้านหลังเดียว และไม่มีใครกลับมาทบทวนสัญญาเรื่องไฟล์ผลลัพธ์ตอนที่ workflow เปลี่ยนไปใช้โครงสร้างโฟลเดอร์แบบหลายบ้านที่มีเลขกำกับ

เสียหายอะไร: จับได้ตอนตรวจความพร้อมก่อนเริ่มบ้านหลังที่ 2 พอดี เลยไม่มีข้อมูลหาย แต่ก็เป็นตัวขวางจริงที่ทำให้บ้านหลังที่ 2 หยุดไปหนึ่ง session

แก้ยังไง: เปลี่ยนชื่อไฟล์ผลลัพธ์ให้มีชื่อบ้านอยู่ด้วย รันบ้านหลังที่ 1 ซ้ำเพื่อยืนยันว่าได้จำนวนเท่าเดิม (34/39/26/99) แล้วลบไฟล์ชื่อตายตัวที่ค้างอยู่ทิ้ง

กฎที่ได้: สคริปต์ใดก็ตามที่ผลลัพธ์ผูกกับ entity ที่รับเข้ามา ต้องใส่ชื่อ entity นั้นในชื่อไฟล์ตั้งแต่วันแรก — path ผลลัพธ์ตายตัว บวกกับมิติข้อมูลใหม่ เท่ากับการเขียนทับแบบเจียบๆ

ตารางราคาบ้าน 10 แบบถูกอ่านผิดหลายแถวในสำเนาของทั้งห้าบ้าน ไม่ใช่แค่แถวเดียวที่เป็นขนวนให้ไปกวาด

เกิดอะไรขึ้น: การกวาดด้วยการซูม 8 เท่าเทียบตัวเลขอ้างอิง ซึ่งเป็นรอบที่ย้อน 'การแก้' ของบ้าน 10 นั้น ยังเจอความผิดพลาดในตารางเดียวกันอีกชุด: #1 แก้เป็น 1,020,000, #2 เป็น 1,460,000 (ราคาบ้าน 2 ของโปรเจกต์นี้เอง), #4 ราคาปูพื้นเป็น 1,824,000, ขนาดแบบบ้านหนึ่งเป็น $3/8=156$ และในสำเนาชุดที่ห้าของบ้าน 1 แก้ #8 ราคาปูพื้น 1,662,000→1,622,000 กับ #10 ราคาปูพื้น 2,235,000→2,035,000

ต้นเหตุจริง: เลขไทยถูกตัดสินที่ละตัวแบบโดดๆ ที่ระดับซูมสูง ซึ่ง ๑ กับ ๙ สับสนกันง่ายมาก แทนที่จะเทียบกับตัวเลขอ้างอิงในทีอื่นบนแผ่นเดียวกัน ความผิดพลาดนี้เป็นระบบ มันเลยเกิดซ้ำในทุกครั้งที่มีคนอ่านตารางเดียวกันนี้อีก

เสียหายอะไร: ตัวเลขราคาผิดหกจุดขึ้นไป กระจายอยู่ในไฟล์ของทั้งห้าบ้าน รวมถึงราคาบ้านของโปรเจกต์ตัวเองด้วย เจอได้ก็เพราะบังเอิญมี 'การแก้' ของแถวหนึ่งถูกตรวจซ้ำอีกรอบเท่านั้น

แก้ยังไง: แก้ครบทั้งห้าสำเนาเมื่อ 2026-07-19 โดยเทียบตัวเลขทุกหลักกับตัวเลขอ้างอิงบนแผ่นเดียวกัน ตอนนีตารางตรงกันครบทั้งห้าบ้าน

กฎที่ได้: การอ่านตัวเลขต้องมีตัวเลขอ้างอิงในแผ่นเดียวกันเป็นตัวเทียบ เหมือนกับการวัดจากฟิกเชลต้องมีมาตราส่วนที่ปรับเทียบแล้ว อย่าตัดสินเลขไทยแบบโดดๆ ตอนซูมเด็ดขาด

วิกฤต 2026-07-21

ทำลาย instance ทั้งก่อนที่จะดาวน์โหลดไฟล์ผลลัพธ์ตัวไหนเลย — adapter, merged model และ GGUF หายหมด

เกิดอะไรขึ้น: มะขามสั่ง destroy instance บน Vast.ai ตอนที่ LoRA adapter ขนาด 7.5GB, merged model 66GB และ GGUF Q4_K_M 21.2GB ยังอยู่แค่บนเครื่องนั้นเครื่องเดียว การ destroy ของ Vast.ai เป็นการลบถาวร (ต่างจาก stop) ทั้งสามไฟล์จึงหายไปโดยกู้คืนไม่ได้

ต้นเหตุจริง: ใน workflow ไม่มีด่านกั้นเรื่องการสำรองข้อมูลเลย ผลลัพธ์ถูกนับว่า 'เซฟแล้ว' เพียงเพราะมันอยู่บนดิสก์ ส่วนการอัปโหลดขึ้น HuggingFace ที่ตั้งใจจะทำ ถูกเลื่อนออกไป (เพราะติดเรื่องยังไม่มี token) แทนที่จะถือว่ามันคือเงื่อนไขที่เปลื้องงานนั้นเสร็จ

เสียหายอะไร: งาน train + merge + quantize ทั้งสิ้นสูญไปหมด — ทั้งค่าเช่า GPU ทั้งการอดนอนทั้งคืน ทั้งการขาดเรียน แลกกับผลลัพธ์ที่ตอนนี้ไม่มีอยู่แล้ว ต้อง train ใหม่ทั้งรอบตั้งแต่ต้น

แก้ยังไ: แก้ไม่ได้ — ไฟล์กู้คืนไม่ได้ สิ่งที่เหลือรอดมีแค่สคริปต์ที่แก้บั๊กแล้ว ชุดข้อมูล และบทเรียนที่จดไว้

กฎที่ได้: ไฟล์ที่อยู่บนที่เก็บของเครื่องเช่า ไม่นับว่าเป็นการสำรองข้อมูล ก่อนจะ stop หรือ destroy instance ใดๆ ต้องไปไล่ดูรายการไฟล์ที่ปลายทางนอกเครื่อง (HuggingFace/เครื่องตัวเอง) ให้เห็นกับตาว่าผลลัพธ์ทุกตัวไปถึงจริงแล้ว

วิกฤต 2026-07-21

ปิดเครื่องเช่าทั้งก่อนที่จะได้ก๊อป LoRA adapter, โมเดลที่ merge แล้ว และ GGUF ออกมา

เกิดอะไรขึ้น: เครื่อง Vast.ai ถูกปิดไปก่อนที่ผลงานของวันนั้น (LoRA adapter, โมเดลที่ merge แล้ว, GGUF) จะถูกดาวน์โหลดหรืออัปโหลดไปที่ไหนเลย พื้นที่เก็บข้อมูลบนเครื่องเช่าเป็นแบบชั่วคราว ทุกอย่างเลยหายไปแบบกู้คืนไม่ได้

ต้นเหตุจริง: ไม่มีด่านกั้นที่แข่งพอร์ระหว่าง "งานเสร็จ" กับ "ปิดเครื่อง" คำเตือนให้สำรองข้อมูลถูกพูดแบบเบาๆ ไม่ใช่เป็นขั้นตอนบังคับที่ต้องผ่านก่อน แถมยังไม่ได้เตรียมปลายทางสำหรับสำรองข้อมูล (HuggingFace) ไว้ล่วงหน้าด้วย

เสียหายอะไร: ค่าเช่า GPU การเทรน และการแปลง GGUF ของทั้งวัน ต้องทำใหม่หมดเพื่อให้ได้โมเดลที่ใช้ได้ สิ่งที่รอดมามีแค่สคริปต์ที่แก้บั๊กแล้วกับบทเรียนที่จดไว้

แก้ยังไ: กำหนดให้การอัปโหลดผลลัพธ์ขึ้น HuggingFace เป็นขั้นตอนหนึ่งของ pipeline อย่างชัดเจน และรอบถัดไปมีการทดสอบ HF write token ล่วงหน้าไว้แล้ว (Phase 0.1)

กฎที่ได้: การสำรองข้อมูลออกจากเครื่องเป็นขั้นตอนบังคับที่ต้องผ่านก่อน ไม่ใช่แค่คำเตือน — เครื่องชั่วคราวจะปิดได้ก็ต่อเมื่อพิสูจน์แล้วว่ามีส่วนอยู่ในที่ถาวรจริง และปลายทางนั้นต้องทดสอบให้ผ่านก่อนเริ่มงานด้วย

วิกฤต 2026-07-24

แบบผังคานสองแผ่นของบ้านหลัง 5 ขาดคานไป 31 ตัว ซึ่งคนเป็นคนหามาให้ทั้งหมด ไม่ใช่ระบบสกัดหรือระบบตรวจ

เกิดอะไรขึ้น: ตอนจัดรูปแบบหน้า 32/33 ของบ้านหลัง 5 ใหม่ (ผังคานชั้น 1 และชั้น 2) ต้องเพิ่มคานเข้าไป 31 ตัว (16 + 15) ที่มขามส่งมาให้ตรงๆ บวกกับลบเส้นที่บันทึกไว้แต่ไม่มีอยู่จริงออกไป 3 เส้น และเพิ่มกริดสมมติใหม่ B'=12.2 เพื่อให้คานที่เพิ่มเข้ามาตัวหนึ่งมีที่ลง

ต้นเหตุจริง: การสกัดข้อมูลจากแบบสองแผ่นนี้ครั้งแรกจับคานได้แค่ส่วนน้อยของที่มีอยู่จริง และการตรวจทานรอบก่อนๆ ของบ้านหลังนี้ก็ไม่ได้ตรวจความครบถ้วนเทียบกับตัวแบบ

เสียหายอะไร: คานราวครึ่งหนึ่งบนแบบสองแผ่นของบ้านที่อยู่ในชุดเทรนหายไป จนกระทั่งคนมานั่งไล่นับเองด้วยมือ — ทั้งระบบสกัดและระบบตรวจอัตโนมัติได้ศูนย์คะแนนบนแบบคู่นี้

แก้ยังไ: เพิ่มคาน 31 ตัวและลบเส้นที่ไม่มีอยู่จริง 3 เส้น เมื่อ 2026-07-24 แล้วตรวจสอบไขว้กับ page00 ไม่มี grid_ref ค้างเลย

กฎที่ได้: ถ้าคนต้องมานั่งไล่นับของที่หายไป 31 ตัวเอง แสดงว่าการสกัดข้อมูลจากแบบชนิดนั้นใช้ไม่ได้ ให้บันทึกว่าแบบชนิดนี้เป็นจุดอ่อนที่รู้ตัว แล้ววัดค่า recall ของมัน ดีกว่าไล่แก้ทีละบ้าน

วิกฤต 2026-07-28

เสนอข้ามเฟส 8 เพราะ t02 แพ้การเปรียบเทียบ — โดยไม่ทันสังเกตว่าการสำรองข้อมูล adapter ก็ถูกมัดรวมอยู่ในเฟสเดียวกันนั้น

เกิดอะไรขึ้น: หลังจาก t02 แพ้ทุกตัวชี้วัด ผู้ช่วยเสนอให้ข้ามเฟส export ไปเลยเพราะดูไม่มีประโยชน์แล้ว แต่การสำรองข้อมูล adapter ถูกมัดรวมอยู่ในเฟส 8 ด้วย ถ้าทำตามข้อเสนอั้นจริงๆ ทั้งรอบนี้จะไม่เหลือข้อมูลสำรองอะไรเลยสักอย่าง — ขำรอยความล้มเหลวครั้ง DAY OF SHAME เป๊ะๆ

ต้นเหตุจริง: คิดเรื่องเฟสหนึ่งจากหัวข้อของมัน ('export GGUF') โดยไม่ใส่ใจว่ามันทำอะไรอย่างอื่นอีกบ้าง การเอาขั้นตอนที่สำคัญต่อความปลอดภัย (การสำรองข้อมูล) ไปมัดรวมไว้ในขั้นตอนที่หน้าตาเหมือนของที่ข้ามได้ ทำให้ขั้นตอนความปลอดภัยนั้นถูกข้ามไปโดยไม่ตั้งใจ

เสียหายอะไร: จับได้ทันก่อนลงมือ ถ้าทำตามไปแล้ว การเทรนที่จ่ายเงินไป 85 นาฬิกาก็จะไม่เหลืออะไรกู้คืนได้เลย

แก้ยังไ: รับสำรอง LoRA adapter (5.14GB) ด้วยมือขึ้นไปที่ Sicilian44/qwen3vl-30b-thai-rc นอกกระบวนการปกติของเฟส 8 แล้วตรวจสอบขนาดไฟล์ผ่าน HF API

กฎที่ได้: ห้ามเอาการสำรองข้อมูลที่สำคัญต่อความปลอดภัยไปมัดรวมไว้ในเฟสที่ข้ามได้ด้วยเหตุผลทางธุรกิจเด็ดขาด การสำรองข้อมูลต้องเป็นขั้นตอนของตัวเองที่ทำเสมอไม่มีเงื่อนไข และก่อนจะข้ามเฟสไหนก็ตาม ต้องไล่ดูให้ครบว่ามันทำอะไรบ้าง ไม่ใช่ดูแค่ชื่อ

ไฟล์ section ของบ้าน 10/11 ไม่มี elements[] เลย — แต่ละไฟล์ตั้งชื่อ array ขึ้นมาเอง

เกิดอะไรขึ้น: ไฟล์ `pattern=section` ในบ้าน 10 และ 11 ไม่มี `elements[]` เลย แต่ละไฟล์กลับตั้งชื่อ array ของตัวเองขึ้นมา (`beams`, `columns`, `slabs`, `footing\_types`, `details`, `sections`) โดยข้างในมีรูปทรงของ element ต่างกันไปอีก แค่เปลี่ยนชื่อ array เนี่ย จึงไม่พอ

ต้นเหตุจริง: spec อธิบายวิธีเขียนแต่ละ *ไฟล์* แต่ไม่เคยล็อก *รูปทรงของไฟล์* ไว้ ไม่มีกฎที่บอกว่า 'ตัวห่อต้องเป็น elements[] เสมอ' การสกัดข้อมูลจึงจัดระเบียบไฟล์ยังไงก็ได้ตามที่รู้สึกว่าเป็นธรรมชาติ โดยไม่ผิดกฎที่เขียนไว้เลย

เสียหายอะไร: ไฟล์ 30 ไฟล์เข้ากันไม่ได้เชิงโครงสร้างกับทุกระบบปลายทาง ต้องรื้อโครงสร้างใหม่ (ไม่ใช่แค่เปลี่ยนชื่อ) และต้องหยุดเพื่อขออนุมัติชัดเจนก่อน เพราะมันเปลี่ยนโครงสร้าง schema จริง

แก้ยังไง: ยูน array ที่ตั้งชื่อกันเองทั้งหมดเข้าไปใน `elements[]` พร้อม `element\_id`/`element\_type` แปลงหน่วยและรูปทรง เหล็กเสริม และเก็บค่าดั้งเดิมทุกค่าไว้ในฟิลด์ `\_printed\_as` ภายหลังล็อกด้วย \$0 FORMAT LOCK ตัวใหม่

กฎที่ได้: ต้องกำหนดรูปทรงของตัวห่อ (container) ไม่ใช่แค่กฎของฟิลด์ ถ้า spec ไม่เคยบอกว่า 'ไฟล์หน้าตาแบบนี้' การสกัดข้อมูลแต่ละครั้งจะแต่งโครงสร้างที่ดูสมเหตุสมผลขึ้นมาเอง และไม่มีอันไหนผิดตามกฎที่เขียนไว้เลย

แบบผังคานทุกแผ่นที่ตรวจในบ้าน 01-05 มีคานหายไปหมด (20→27, 22→36, 30→38, 5→21) — ส่วนบ้าน 06-11 ยังไม่เคยตรวจเลย

เกิดอะไรขึ้น: ตอนเอาผังคานของบ้าน 01-05 มาตรวจกับแบบจริง พบว่าทุกแผ่นมีคานจริงหายไปหมด กรณีหนึ่งสกัดได้ 5 ตัวทั้งหมดที่มีอยู่จริง 21 ตัว ส่วนบ้าน 06 ถึง 11 ยังไม่เคยถูกตรวจเลย และต้องสันนิษฐานว่ามีข้อบกพร่องเดียวกัน

ต้นเหตุจริง: การสกัดคานตรวจไม่เจออย่างเป็นระบบ — คานที่ลากต่อข้ามหน้า คานที่วาดจางๆ หรือคานที่ป้ายชื่อไว้ด้านข้าง จะถูกข้ามไป และไม่มีอะไรมานับจำนวนคานที่สกัดได้เทียบกับการตรวจด้วยสายตา

เสียหายอะไร: เป็นข้อบกพร่องของข้อมูลที่เสียหายหนักที่สุดในชุดข้อมูลทั้งหมด การเทรนน label ที่คานหายไปคือการสอนให้โมเดลทำนายต่ำกว่าจริง ซึ่งเป็นตัวชี้วัดที่อ่อนที่สุดอยู่แล้ว และมีบ้านอีก 6 หลังที่แบกปัญหานี้อยู่ในปริมาณที่ยังไม่มีใครวัด

แก้ยังไง: บ้าน 01-05 แก้แล้วโดยการตรวจเทียบกับแบบ ส่วนบ้าน 06-11 ยังไม่ตรวจ ถูกตั้งธงซ้ำๆ ว่าเป็นงานค้างที่มีค่าสูงสุดมาแล้ว 4 เซสชัน

กฎที่ได้: ข้อบกพร่องประเภทตรวจไม่เจออย่างเป็นระบบที่พบในทุกบ้านที่สุ่มมาตรวจ ต้องสันนิษฐานว่ามีอยู่ในทุกบ้านที่ยังไม่ได้สุ่มด้วย และห้ามสร้างชุดข้อมูลสำหรับเทรนนผังคานที่ยังไม่ผ่านการตรวจสอบ — ตรวจให้เสร็จก่อน แล้วค่อยเทรน

วิกฤต 2026-08-02

คำสั่ง 'ตัด underscore ออกจาก element_id' จะทำให้ชิ้นส่วนที่ต่างกันจริง 17 ชิ้นยุบรวมเป็นชิ้นเดียว

เกิดอะไรขึ้น: ถ้าทำตามคำสั่งตรงๆ บันได_ST1 กับ `steps\_ST1` จะเหลือ `ST1` เหมือนกัน `B1\_خانพัก` (คานขานพัก) จะยุบรวมกับ `B1` ธรรมดา วัสดุขายคาสามชนิดและจันทันสามตัวแยกกันก็จะรวมกันหมด ตรวจพบการชนกัน 17 กรณี และย้อนคืน id ไป 37 ตัวใน 5 บ้าน

ต้นเหตุจริง: สั่งแปลงข้อความแบบเหมารวมโดยไม่ตรวจสอบว่าส่วนต่อท้ายนั้นมีความหมายที่แตกต่างกันจริงหรือเปล่า ซึ่งบางตัวมีจริงและเป็นการเข้ารหัสชิ้นส่วนโครงสร้างที่ต่างกันคนละตัว

เสียหายอะไร: เกือบทำข้อมูลหายแบบเสียๆ ไปแล้ว: ชิ้นส่วนโครงสร้างคนละตัวถูกรวมเป็น id เดียว คู่คืนไม่ได้นอกจากย้อนกลับไปด้วยแบบใหม่ จับได้ด้วยการตรวจการชนกันหลังการแปลง

แก้ยังไ: เก็บส่วนต่อท้ายไว้ใน 17 กรณีที่ชนกัน พร้อมธงอธิบายเหตุผล ตรวจรอบสุดท้ายยืนยันว่าชนกัน 0 กรณี

กฎที่ได้: ต้องรันการตรวจการชนกันทั้งก่อนและหลังการแปลง id ให้เป็นมาตรฐานเสมอ ถ้าของสองอย่างที่แตกต่างกันแปลงไปเป็น id เดียวกัน แปลว่าการแปลงนั้นผิดสำหรับกรณีนั้น — คำสั่งให้ทำให้เป็นมาตรฐาน ไม่ใช่คำสั่งให้ทำข้อมูลหาย

วิกฤต 2026-08-02

element_type บานไปถึง 359 ค่าที่ไม่ซ้ำกัน จาก element ทั้งหมด 4,507 ชิ้น

เกิดอะไรขึ้น: การตรวจสอบพบว่า `element\_type` ซึ่งควรจะเป็นชุดค่าศัพท์ปิดเล็กๆ ได้สะสมค่าที่ไม่ซ้ำกันไปถึง 359 ค่า จาก element ทั้งหมด 4,507 ชิ้น

ต้นเหตุจริง: เป็น enum แบบข้อความอิสระที่ไม่มีการบังคับใดๆ ตลอด 11 บ้านที่สกัดมา ทุกครั้งที่เจอชิ้นส่วนที่ไม่คุ้น การสกัดก็แต่งชื่อประเภทที่ดูสมเหตุสมผลขึ้นมาเอง

เสียหายอะไร: การจัดกลุ่ม คิตราค่า หรือกรองข้อมูลด้วย element_type ไม่มีความหมายเลยทั้งชุดข้อมูล และโมเดลที่เทรนบนข้อมูลนี้จะเรียนรู้ label ที่ความหมายใกล้เคียงกัน 359 ตัว

แก้ยังไ: เพิ่มรายการ element_type มาตรฐานใน §0.4 และให้ตัวตรวจบังคับใช้

กฎที่ได้: enum ไหนที่ไม่มีเครื่องหมายบังคับ จะบานไปเป็นหลายร้อยค่าแน่นอน ต้องจำกัดมันไว้ในตัวตรวจตั้งแต่วันแรก และค่าที่ไม่รู้จักต้องถือเป็นความล้มเหลวขั้นเด็ดขาด ไม่ใช่ปล่อยผ่านเสียๆ

วิกฤต 2026-08-02

element ราว 500 ชิ้นไม่มี element_id เลย — และอยู่ในบ้าน 01-07 ไม่ใช่บ้านใหม่ๆ

เกิดอะไรขึ้น: การตรวจสอบอิสระ (เทียบชนิดข้อมูล JSON ของทุก key ในไฟล์ทั้ง 2,366 ไฟล์ โดยจงใจไม่ใช่ตัวตรวจที่มีอยู่) พบ element ราว 500 ชิ้นที่ไม่มี `element\_id` ในบ้าน 01-07 ส่วนบ้าน 08-11 ครบถ้วนอยู่แล้ว ซึ่งตรงกันข้ามกับข้อสันนิษฐานที่ว่า 'ให้ยึดตามบ้าน 01-05'

ต้นเหตุจริง: มีสองสาเหตุ: หนึ่ง ไม่เคยมีการตรวจว่ามี `element\_id` อยู่หรือเปล่า และสอง การตรวจสอบที่เจอเรื่องนี้ทำได้เพราะมัน *ไม่* พังตัวตรวจที่มีอยู่ — ซึ่งตรวจได้แค่กฎที่มันรู้จักอยู่แล้วเท่านั้น

เสียหายอะไร: element ราว 500 ชิ้นเชื่อมโยงกับ spec หรือตารางรายการใดๆ ไม่ได้เลย และข้อสันนิษฐานเรื่องบ้านต้นแบบก็ถูกพิสูจน์ว่าผิดเป็นครั้งที่สองในวันเดียวกัน

แก้ยังไ: เดิม id ย้อนหลังจากมาร์คที่พิมพ์บนแบบ/item_no/รูปแบบเชิงบรรยาย พร้อมธงต่อรายการที่บันทึกว่าแต่ละตัวได้มาอย่างไร

กฎที่ได้: ต้องตรวจสอบเป็นระยะด้วยวิธีที่ *ไม่* ใช้ตัวตรวจของตัวเอง เพราะตัวตรวจหาเจอได้แค่ข้อบกพร่องที่มีคนคิดไว้แล้ว การทำ type-profiling ทุก key ทั่วทั้งชุดข้อมูลจะเจอข้อบกพร่องที่ไม่มีใครคาดคิด

เนื้อหาถอยหลัง: ตารางราคาของบ้าน 05 ยังเป็นตัวเลขผิดชุดเดิม ทั้งที่ log บอกว่า 'แก้ครบทั้ง 5 ฉบับแล้ว'

เกิดอะไรขึ้น: รายการที่ 37 บันทึกว่าตัวเลขราคาที่ผิด (1,060,000/1,560,000) ถูกแก้ครบทั้ง 5 ฉบับแล้ว ตรวจสอบด้วยการซูม 8 เท่า แต่ฉบับของบ้าน 05 หลุดไป — มันยังเป็นตัวเลขผิดชุดเดิมอยู่ ตรวจสอบบ้าน 03/04 แล้วถูกต้อง มีแต่ฉบับของบ้าน 05 ที่ผิด

ต้นเหตุจริง: การแก้ถูกตรวจสอบราย *ประเด็น* ('ปัญหาเรื่องราคาแก้แล้ว') แทนที่จะตรวจราย *ฉบับ* ('ทั้ง 5 ไฟล์แสดงตัวเลขที่ถูกต้องแล้ว') log บันทึกว่าการตัดสินใจเสร็จแล้ว ไม่ใช่ว่าการลงมือทำเสร็จแล้ว

เสียหายอะไร: ตัวเลขเงินที่ผิด — คลาดเคลื่อนหลายแสนบาท — ค้างอยู่ในชุดข้อมูลเป็นเวลานาน ขณะที่ log อ้างว่าแก้แล้ว และนี่คือครั้งที่สองที่ความล้มเหลวแบบ 'log บอกว่าแก้แล้ว' กัดโปะเจตน์นี้

แก้ยังไง: ค้นหาตัวเลขตามข้อสรุปของรายการที่ 37 (เป็นการลงมือทำตามการตัดสินใจที่ตรวจสอบแล้ว ไม่ใช่การตัดสินใจใหม่) ส่วนตัวเลขเก่าเก็บไว้ใน *_printed_as* พร้อมธง RE-CORRECTED

กฎที่ได้: คำว่า 'log บอกว่าแก้แล้ว' ต้องพิสูจน์รายฉบับ ไม่ใช่รายประเด็น เมื่อการแก้ไขกระทบไฟล์ N ฉบับ การตรวจว่าเสร็จต้องนับให้ครบ N ฉบับที่ตรวจแล้ว — ถ้าน้อยกว่านั้นคือคำกล่าวอ้าง ไม่ใช่การแก้

roof_frame_plan ถูกแยกเป็นสอง pattern — คานหลังคาของ 8 ใน 11 หลัง ไม่เคยไปถึง BOQ เลย

เกิดอะไรขึ้น: ไฟล์โครงหลังคาทั้ง 12 ไฟล์จาก 11 หลัง ระบุ discipline: "structural" เหมือนกันหมด แต่ 4 ไฟล์เป็น pattern: "plan" ส่วนอีก 8 ไฟล์เป็น pattern: "roof_plan" ในขณะที่ buildElements() ของ Constant อ่านแค่ pattern == 'plan' คานหลังคาของ 8 ใน 11 หลังจึงหายไปจาก BOQ แบบเงียบๆ เจอตอนแยก subtask ของผังตามชนิดชิ้นส่วนโครงสร้าง

ต้นเหตุจริง: ชื่อ pattern ว่า 'roof_plan' อ่านแล้วดูเป็นที่ยู่ที่ถูกต้องของแผ่นโครงหลังคาอย่างชัดเจน แต่ในระบบจัดหมวดหมู่ roof_plan ตั้งใจให้ใช้กับแผ่นหลังคาฝั่งสถาปัตย์ ส่วนโครงหลังคาเชิงโครงสร้างต้องเป็น plan คนทำ annotation เลยใส่ตามสัญชาตญาณ แล้วก็ไม่มีใครตรวจว่า pattern กับ discipline ไปด้วยกันได้ไหม แถมผังที่เอาไปใช้ก็โครงทั้งเงียบๆ แทนที่จะเตือนว่าเจอ pattern ที่ไม่รู้จัก

เสียหายอะไร: เป็นบั๊กข้อมูลหายของจริง: ชิ้นส่วนโครงสร้างทั้งคานหายไปจาก BOQ ของบ้าน 8 ใน 11 หลังที่ทำ annotation ไว้ ความแม่นยำของ t03 ที่วัดเทียบกับ ground truth ชุดนี้ ก็เท่ากับวัดเทียบกับข้อมูลที่ไมครบ

แก้ยังไง: ยังไม่ได้แก้ในตัวข้อมูลจนจบ session — บันทึกไว้ว่าเป็นสิ่งที่ต้องแก้ บรรเทาไว้ก่อนด้วยการเขียน §1 pattern #12 roof_plan ใหม่ เพิ่มบล็อกเตือน '⚠️ roof_plan vs plan' ที่ตอกให้ชัดว่าโครงหลังคาเชิงโครงสร้างต้องเป็น plan และเขียนกับดักนี้ลงในสเกล op04 ด้วย

กฎที่ได้: อะไรก็ตามที่กรองข้อมูลตามค่าในระบบจัดหมวดหมู่ ต้องเตือนเมื่อเจอค่าที่ไม่รู้จัก ไม่ใช่ทั้งเงียบๆ และทุกคู่ discipline+pattern ควรถูกตรวจโดย check_format.py — ป้ายที่ดูสมเหตุสมผลแต่ผิด แล้วคนทำ annotation ใส่ซ้ำๆ กันทุกคน จะมองไม่เห็นเลยถ้าไม่มีกฎที่ตรวจข้ามฟิลด์

ป้าย `roof\_plan` ผิดบนโครงหลังคาเชิงโครงสร้างของ 8 หลัง ทำให้ BOQ ใน Constistant ข้อมูลหายจริงบนระบบใช้งานจริง

เกิดอะไรขึ้น: บ้าน 8 หลังมีแผนโครงหลังคาเชิงโครงสร้างถูกติดป้ายเป็น `roof\_plan` แทนที่จะเป็น `plan` ในขณะที่ pipeline ของ BOQ ใน Constistant ไม่ได้กิน `roof\_plan` ขึ้นส่วนพวกนั้นจึงถูกทิ้งจากการถอดปริมาณแบบเงียบๆ เป็นบั๊กข้อมูลหายบนระบบใช้งานจริง

ต้นเหตุจริง: มีค่าในชุดคำศัพท์ประเภทหน้าอยู่ใน ground truth ที่ไม่มีใครปลายทางรองรับ และไม่มีอะไรตรวจว่าทุกค่าที่ผู้ผลิตข้อมูลพ่นออกมามีคนรับหรือเปล่า ป้ายที่ผิดจึงกลายเป็นการทิ้งข้อมูลเงียบๆ แทนที่จะเป็น error

เสียหายอะไร: ยอด BOQ ในแอป Constistant ที่ใช้งานจริงต่ำกว่าความเป็นจริงในส่วนโครงหลังคาของบ้าน 8 หลัง โดยไม่มี error ให้ใครเห็นเลย

แก้ยังไง: เปลี่ยนป้ายจาก `roof\_plan` เป็น `plan` ทั้ง 8 หลัง โดยทำไปพร้อมกับรอบปรับ schema ให้เป็นมาตรฐาน

กฎที่ได้: ทุกค่าใน enum ที่ฝั่งผลิตข้อมูลพ่นออกมาได้ ต้องมีฝั่งรับที่ชัดเจน หรือไม่ก็ต้องมีการปฏิเสธแบบดั่งๆ ค่าที่ปลายทางเมินเงียบๆ คือบั๊กข้อมูลหายที่รอวันปะทุ

beam plan ไม่เคยถูกตรวจว่าคานหายไหมเลยสักหลัง — คุณภาพ ground truth ที่ใช้เทรนยังไม่รู้เลย

เกิดอะไรขึ้น: การรีวิวเนื้อหาของบ้าน 06-11 เทียบกับแบบต้นฉบับยังค้างอยู่ และบันทึกไว้ว่า beam plan ไม่เคยถูกตรวจว่ามีคานหายไปไหม เลยสักหลังเดียว

ต้นเหตุจริง: แรงที่ใช้รีวิว ground truth พุ่งไปที่ความถูกต้องของ schema/รูปแบบ (ซึ่งเขียนสคริปต์ตรวจได้) แทนที่จะไปที่ความครบถ้วนของเนื้อหา (ซึ่งต้องเปิดแบบดูจริง) ground truth ของ subtask ที่เป็นคอขวดจึงไม่เคยถูกตรวจสอบเลย

เสียหายอะไร: เรากำลังเทรนโมเดลให้วิ่งเข้าหา ground truth ของ beam plan ที่ตัวมันเองอาจจะนับคานขาดอยู่ — แปลว่า recall 0% อาจจะบางส่วนเป็นการวัดเทียบกับ ground truth ที่ไม่ครบ แยกไม่ออกว่าโมเดลพังหรือ ground truth พัง

แก้ยังไง: ยังไม่ได้แก้ — ยกยอดไปเป็นงานค้าง

กฎที่ได้: ตรวจสอบ ground truth ของ subtask ที่กำลังจะไป optimize *ก่อน* ที่จะเสียเงินค่า GPU ไปกับการ optimize มัน ตัวเลข recall น่าเชื่อถือได้แค่เท่าที่ ground truth ของมันน่าเชื่อถือเท่านั้น

แก้ schema 43 ครั้งใน 5 สัปดาห์ โดยไม่เคยย้อนไปอัปเดตข้อมูลบ้านที่สเก็ตไปแล้วเลย

เกิดอะไรขึ้น: มะขามถามว่าทำไมไฟล์ gridmaster ของบ้าน 20 กับบ้าน 28 หน้าตาต่างกัน ไล่ย้อนกลับไปพบว่า schema หลักถูกแก้ไข 43 ครั้งในช่วง 5 สัปดาห์ และบ้านเก่าไม่เคยถูกย้ายมาตาม schema ใหม่เลย ทำให้ข้อมูลในชุดเดียวกันมีไฟล์คนละรุ่นที่เข้ากันไม่ได้ปะปนอยู่

ต้นเหตุจริง: ไม่มีขั้นตอน migration ผูกอยู่กับกระบวนการแก้ schema ทุกครั้งที่แก้สเปกจะถือว่าเป็นกฎสำหรับข้อมูลที่จะสเก็ตใหม่เท่านั้น ไม่มีข้อผูกมัดให้กวาดไฟล์เก่ากลับมาแก้ตาม การเบี่ยงเบนเลยสะสมไปทีละรอบโดยไม่มีใครรู้

เสียหายอะไร: ข้อมูล training 49 หลังไม่สอดคล้องกันเองภายใน การกวาดล้างรอบก่อนหน้าในวันที่ 08-24 ครอบคลุมแค่ 11 หลัง เหลืออีก 38 หลังที่ยังค้างจนถึง session นี้ โมเดลถูกเทรนด้วยข้อมูลที่ผสมกันหลาย generation ของ schema

แก้ยังไง: ปรับทั้งสองต้นไม้ของไฟล์ (`json\_แก้ไขแล้ว/` และ `rawjson\_ยังไม่ได้แก้ไขโดนคน/`) ให้ตรงกับสเปกปัจจุบันในรอบเดียว รวม 485 จุดแก้ / 423 ไฟล์ / ครบทั้ง 49 หลัง

กฎที่ได้: ทุกครั้งที่แก้ schema ต้องมีสคริปต์ retrofit รันย้อนไปทั่วข้อมูลเก่า หรือไม่ก็ต้องมีการตัดสินใจเขียนไว้ชัดๆ ว่าไฟล์เก่าจะคงอยู่ที่ generation เดิม อย่ายปล่อยให้ "ค่อยไปแก้ทีหลัง" เป็นแค่ความคิดในหัว เพราะการเลื่อนแบบไม่มีบันทึก 43 ครั้งคือหน้าตาของชุดข้อมูลที่แตกสาขาไปเสียๆ

ไฟล์บันทึกค่าเก่า→ค่าใหม่ยาว 1,220 บรรทัดถูกวางไว้ในโฟลเดอร์ training data กลายเป็นช่องทางปนเปื้อนข้ามบ้าน

เกิดอะไรขึ้น: `สิ่งที่ต้องแก้.md` ไฟล์บันทึกค่าเก่าและค่าที่แก้แล้วของแต่ละบ้านยาว 1,220 บรรทัด ถูกเก็บไว้ในโฟลเดอร์ training data เดียวกับข้อมูลจริง โมเดลที่กำลังสเก็ตบ้าน 30 อาจอ่านตัวเลขของบ้าน 01 จากไฟล์นี้แล้วหยิบค่าผิดบ้านไปใช้ได้

ต้นเหตุจริง: เอกสารทำงานของคนถูกวางไว้ในไดเรกทอรีเดียวกับข้อมูลที่โมเดลใช้อ่าน โดยไม่มีกฎแยก "บันทึกเกี่ยวกับข้อมูล" ออกจาก "ตัวข้อมูลจริง"

เสียหายอะไร: เสี่ยงข้อมูลปนเปื้อนข้ามบ้านทั่วทั้งชุด training โมเดลอาจถูกป้อนตัวเลขของบ้านอื่นราวกับเป็นค่าของบ้านที่กำลังทำอยู่

แก้ยังไง: ย้ายไปไว้ที่ `wait\_for\_thing/stale\_value\_logs/` พร้อม README อธิบายเหตุผล

กฎที่ได้: ห้ามวางบันทึกแก้ไขค่ารายชิ้นของมนุษย์ไว้ในโฟลเดอร์ที่โมเดลอ่าน แยกเอกสารทำงานไว้อีกต้นไม้อีกหนึ่งเสมอ ถ้าไฟล์ไหนมีค่าจริงของหลายเรื่องปนกันอยู่ ให้ถือว่ามันคือช่องทางปนเปื้อนโดยธรรมชาติของมันเอง

ชุด validation ที่รับต่อมา เป็นสับเซตของข้อมูลเทรน 100%

เกิดอะไรขึ้น: ตอนรวมชุดข้อมูล 4 pass เข้าด้วยกัน พบว่าไฟล์ validation ของ t04 ที่มี 229 แถว ซ้ำกับชุดเทรนทั้งหมด (id ซ้ำ 229/229) และบ้านสองหลังที่ t05 เลือกมาเป็น validation ก็อยู่ในชุดเทรนของ t04 ทั้งคู่

ต้นเหตุจริง: ชุดแบ่งข้อมูลถูกส่งต่อข้ามรอบโดยไม่มีใครตรวจ id ซ้ำอีกครั้ง แกรมรอบก่อนหน้าแบ่งข้อมูลทีละแถวแทนที่จะแบ่งทีละบ้าน บ้านหลังเดียวกันจึงไปโผล่ทั้งสองฝั่ง

เสียหายอะไร: ตัวเลข validation ทุกตัวที่ออกมาจาก split ชุดนั้นไม่มีความหมายเลย เพราะเป็นการวัดโมเดลด้วยข้อมูลที่มันท่องจำไว้แล้ว

แก้ยังง: หักชุด val เดิม แล้วแบ่งใหม่ทีละบ้านด้วยกฎเดียวกันทุก pass พร้อมเพิ่มการตรวจการปนเปื้อนของ val ลงใน smoke_4pass.py

กฎที่ได้: ทุกครั้งที่รับชุดข้อมูลต่อมาหรือรวมชุดข้อมูลเข้าด้วยกัน ต้องตรวจความถูกต้องของการแบ่งใหม่ทั้งระดับ id และระดับบ้าน — ห้ามเชื่อว่าไฟล์ val ของรอบก่อนสะอาดอยู่แล้วเด็ดขาด

ใส่ label แบบ batch ให้หน้า pass0 จำนวน 89 หน้า โดยตั้ง sheet_code เป็น null ทั้งที่เฉลยอ่านออกชัดเจน

เกิดอะไรขึ้น: ตอนใส่ label แบบ batch ให้หน้า material_list มีการตั้ง sheet_code กับ sheet_name เป็น null ทั้ง 89 หน้า ทั้งที่เฉลยแสดงค่าไว้ชัดๆ อย่าง 'ปร.4 (แผ่นที่ 1/23)' และ prompt ของ pass0 เองก็เขียนไว้ว่านี่คือโอกาสเดียวที่จะเก็บค่าพวกนี้ เพราะ pass ถัดๆ ไปจะถูกครอบห้วกระดาษออกไปแล้ว

ต้นเหตุจริง: สคริปต์ batch เขียนขึ้นมาเพื่อตอบคำถามเรื่อง layout (ครึ่งไหนเป็นแผ่นไหน) แล้วไม่เคยกลับไปดูฟิลต์อื่นที่จำเป็นเลย และไม่มีใครอ่านคำเตือน 'โอกาสเดียว' ใน prompt ซ้ำก่อนจะสร้าง label 89 อัน

เสียหายอะไร: จะสอนโมเดลให้ข้ามข้อความที่มันอ่านออกจริงๆ และทำให้เลขแผ่นของหน้าเหล่านั้นหายไปถาวร เพราะไม่มี pass ไหนหลังจากนี้เห็นหัวกระดาษอีกแล้ว จับได้เพราะมะขามถามว่า pass0 ทำ OCR ด้วยหรือเปล่า

แก้ยังง: เขียนใหม่ให้ฟิลต์ระดับบนสุดเก็บแผ่นที่เลขน้อยกว่า ส่วนแผ่นที่สองเก็บไว้ใน warnings[] เป็นข้อความที่อ่านได้ และทำให้สคริปต์รันซ้ำได้โดยไม่พัง ด้วยการหาหน้าเป้าหมายจากเฉลยโดยตรง ผลคือหน้า pass0 ที่มี sheet_code จริงเพิ่มจาก 63.2% เป็น 80.1%

กฎที่ได้: ตอนสร้าง label แบบ batch ต้องกลับไปอ่านข้อกำหนดใน prompt ให้ครบทุกฟิลต์ที่จำเป็น ไม่ใช่แค่ฟิลต์ที่กำลังทำอยู่ และฟิลต์ไหนที่ prompt บอกว่าเป็น 'โอกาสเดียวที่จะเก็บ' ห้ามตั้งเป็น null เด็ดขาดถ้ายังอ่านออก

วิกฤต 2026-08-31

รายชื่อ VAL_HOUSES ที่อาร์ดโค้ดไว้ ชนกับการแบ่ง fold ของ t04 ทำให้บ้านหลังหนึ่งรั่วข้าม pass

เกิดอะไรขึ้น: บ้าน_ใหญ่_1ชั้น_01 อยู่ฝั่ง train ตามรายชื่อ val ที่อาร์ดโค้ดไว้ แต่อยู่ฝั่ง val ใน fold0 ของ t04 ถ้าใช้ VAL_HOUSES แบบตายตัว บ้านหลังนี้จะอยู่ใน train ของ pass0 และอยู่ใน val ของ pass1 ภายในโมเดลตัวเดียวกัน

ต้นเหตุจริง: มีผู้มีอำนาจตัดสินใจการแบ่งข้อมูลสองเจ้าพร้อมกัน (รายชื่อบ้านที่อาร์ดโค้ด กับไฟล์ fold ของ t04) แล้วเอามาใช้กับโมเดลตัวเดียวกัน

เสียหายอะไร: ข้อมูลรั่วข้าม pass ซึ่งจะทำให้ตัวเลข validation สูงเกินจริงโดยไม่มีใครมองเห็น

แก้ยังไง: เลิกใช้รายชื่อ VAL_HOUSES แบบตายตัว แล้วให้การจับคู่บ้านกับฝั่ง train/val ของ t04 เป็นแหล่งความจริงเพียงแหล่งเดียวของแต่ละ fold ใช้เหมือนกันทุก pass

กฎที่ได้: ในหนึ่งโมเดล ต้องมีผู้กำหนดว่าบ้านหลังไหนอยู่ฝั่งไหนแค่เจ้าเดียว และใช้แบบเดียวกันกับทุก pass ของข้อมูล — การมีรายชื่ออาร์ดโค้ดเจ้าที่สองคือรอยร้าวที่รอวันเกิด

วิกฤต 2026-09-01

พื้นทุกแผ่นราคาออกมา \$0 เพราะ floor_area_sqm ไม่มีขั้นตอนไหนในสาย raw-extraction ผลิตค่านี้ให้เลย

เกิดอะไรขึ้น: ในทุกการรันจำลองบ้าน รายการพื้น (พื้น) ออกมาเป็น \$0 ทุกครั้ง สาเหตุนี้ถูกบันทึกไว้ใน CLAUDE.md อยู่แล้วว่า `floor\_area\_sqm` ไม่มีขั้นตอนไหนใน pipeline การถอดข้อมูล raw ผลิตค่านี้ขึ้นมาเลย

ต้นเหตุจริง: pipeline การถอดข้อมูลไม่เคยผลิตฟิลต์เดียวที่สูตร BOQ ของพื้นต้องใช้เลย ยิ่งซ้ำร้ายเข้าไปอีกเพราะ `plan\_slab` มีตัวอย่างข้อมูลฝึก 0 รายการในทั้งระบบ ทำให้โมเดลไม่เคยถูกสอนให้ตอบแบบแปลนพื้นเลยแม้แต่นิดเดียว

เสียหายอะไร: คอนกรีต เหล็กเสริม และแบบหล่อของพื้นทั้งหมดถูกตั้งราคา \$0 อย่างเงิบๆ ในทุก BOQ ที่ผลิตผ่านสาย extraction จริง — ทำให้ยอดรวมต่ำกว่าความจริงอย่างเป็นระบบ ในงานถอดปริมาณก่อสร้างจริง

แก้ยังไง: ยังไม่ได้แก้ — บันทึกไว้ตรงๆ ว่ายังพังอยู่

กฎที่ได้: เมื่อหมวด BOQ ไหนอ่านค่าได้ \$0 ตลอด ให้มองว่าเป็นบักขาของผู้ผลิตข้อมูล ไม่ใช่หมวดที่ว่างเปล่าจริงๆ ให้ไล่ทุกฟิลต์ที่จำเป็นย้อนกลับไปหาขั้นตอนที่ผลิตมัน และถ้าไม่มีขั้นตอนไหนผลิตเลย ฟิลเจอรันนั้นถือว่ายังไม่ได้ทำ ไม่ว่า pipeline จะดูเขียวแค่ไหนก็ตาม

วิกฤต 2026-09-01

หน้าแบบมุมมองหลายมุม (multi-view) ที่รอคิดติดยาด้วยมือ ถูกนับและพิมพ์ออกมาแต่ไม่เคยเข้าไปไฟล์เรนจริงเลย

เกิดอะไรขึ้น: `build\_pass0()` ใน `build\_t05\_night.py` อ่านแค่ `pass0\_labels.jsonl` เท่านั้น คือที่รอติดยาด้วยมือ — ซึ่งเก็บหน้ามุมมองหลายมุมทั้งหมด ส่วนใหญ่เป็นคู่ฐานราก+คาน — ถูกอ่านมาแค่ฉบับจำนวนเท่านั้น ไม่เคยเข้าไปในข้อมูลเรนจริงเลย และยังไม่มีใครสะสางมันเลย (เฉลี่ยประมาณ 40 หน้าต่อบ้าน)

ต้นเหตุจริง: คิวรอด้วยมือถูกสร้างขึ้นเป็นที่พักข้อมูลชั่วคราว แต่ไม่มีใครต่อสายให้เป็นผู้บริโภคมูลนั้นเลย การอ่านมันแค่เพื่อนับจำนวนทำให้ดูเหมือนว่ามันถูกรวมเข้า pipeline แล้วทั้งที่ไม่จริง

เสียหายอะไร: หมวดหน้าที่ยากที่สุดและมีค่าที่สุด (หน้ามุมมองหลายมุม — พอดีเป็นหน้าที่ล้มเหลวตอน generate อยู่เรื่อยๆ) ไม่มีสัญญาณการเรนเลยแม้แต่นิดเดียว ประมาณ 40 หน้าต่อบ้านที่ยังไม่ได้ติดยา

แก้ยังไง: ยังไม่ได้แก้ — บันทึกไว้ว่ายังค้างอยู่

กฎที่ได้: คิวที่ถูกนับแต่ไม่เคยถูกบริโภคจริง คือช่องทางที่ข้อมูลหายไปแบบแอบอ้างว่าเป็นความคืบหน้า คือทุกคิวต้องมีผู้บริโภคที่ระบุชื่อชัดเจน และต้องมีการยืนยันว่า item ในคิวจะออกจากคิวไปในที่สุด

plan_slab มีตัวอย่างข้อมูลเทรน 0 รายการในทั้งระบบ

เกิดอะไรขึ้น: ตรวจสอบ `pass0\_train+val.jsonl` จริง (171 แถว) พบว่า: plan_beam 14, plan_footing 7, also_gridline 32 และ `**plan_slab = 0` ในทั้งระบบ**

ต้นเหตุจริง: ไม่เคยมีการตรวจสอบความครอบคลุมของข้อมูลเทรนต่อ subtask ก่อนเทรนเลย มี subtask อยู่ใน taxonomy โดยไม่มีตัวอย่างข้อมูลรองรับเลย และไม่มีอะไรตั้งธงแจ้งเตือน class ที่ว่างเปล่านี้เลย

เสียหายอะไร: โมเดลไม่มีวันตอบแบบแปลนพื้นได้เลย — ซึ่งเป็นสาเหตุที่สองที่แยกจากกัน ของปัญหา BOQ พื้น \$0 ผลลัพธ์เกี่ยวกับพื้นจากโมเดลตัวไหนก็ตามจึงเชื่อถือไม่ได้เลยโดยธรรมชาติ

แก้ยังไง: ยังไม่ได้แก้ — บันทึกไว้ว่าต้องเพิ่มข้อมูลเทรน

กฎที่ได้: ก่อนการเทรนใดๆ ให้พิมพ์จำนวนตัวอย่างต่อ class ออกมา และหยุดการ build ทันทีถ้ามี class ที่มีตัวอย่างเป็นศูนย์ (หรือใกล้ศูนย์) subtask ที่ไม่มีข้อมูลเทรนเลยคือค่าโกหกใน taxonomy

deleteProject() ไม่ได้ลบคีย์ localStorage ของ Site/Foundation/manpower ทำให้ข้อมูลโปรเจกต์กลายเป็นข้อมูลเก่าพว้า

เกิดอะไรขึ้น: พบและแก้บั๊กใน project-store.js: `deleteProject()` ไม่ได้ลบคีย์ localStorage ของ Site Investigate, Foundation Design และ manpower-config ทั้ง ทำให้ข้อมูลเก่าพว้าค้างอยู่หลังโปรเจกต์ถูกลบไปแล้ว

ต้นเหตุจริง: รายชื่อคีย์ storage ที่ผูกกับโปรเจกต์ถูกลดด้วยมือฝังอยู่ใน deleteProject() ในขณะที่แต่ละ feature module ต่างประกาศชื่อคีย์ของตัวเองแยกกันเอง ทุกครั้งที่มีคีย์ใหม่ที่ผูกกับโปรเจกต์ ต้องมีคนจำไว้ให้ไปเพิ่มในอีกที่หนึ่งด้วย ซึ่งไม่มีใครทำ

เสียหายอะไร: โปรเจกต์ที่ถูกลบยังทิ้งเศษข้อมูลไว้ใน localStorage นอกจากพื้นที่ที่เสียไปแล้ว ข้อมูล Site/Foundation ที่เก่าพว้ายังอาจถูกดึงไปใช้โดยโปรเจกต์ใหม่ที่บังเอิญใช้ id ซ้ำ และยังขัดแย้งกับความเข้าใจของผู้ใช้ที่คิดว่าการลบคือลบหมดแล้วแบบเสียบๆ

แก้ยังไง: แก้การลบให้ครบ และแก้เชิงโครงสร้าง: สร้าง js/shared/storage-keys.js เป็นทะเบียนกลางของทุกคีย์ localStorage โดย PROJECT_SCOPED_KEYS ถูก derive มาจากทะเบียนนี้อัตโนมัติ แทนที่จะดูแลด้วยมือ และแก้ไข 14 feature module ที่เคยประกาศชื่อคีย์ของตัวเองแยกกัน

กฎที่ได้: รายชื่อใดๆ ที่ต้องซิงก์กับชุดของสิ่งที่ต้องจัดการ (คีย์ที่ต้องลบ, คีย์ที่ต้องซิงก์, คีย์ที่ต้องย้าย) ต้อง derive มาจากทะเบียนกลางเสมอ ห้ามพิมพ์ด้วยมือแยกกัน รายการคู่ขนานที่ดูแลด้วยมือมีแต่จะเพี้ยนไปจากกันในวันที่มีคนรีบเพิ่มรายการใหม่โดยไม่ทันนึกถึงมัน

รอบแรกจัดกริดไลน์โครงสร้างจริงบนหน้า MEP ผิดว่าเป็นแค่ label ข้อความ

เกิดอะไรขึ้น: หน้า MEP 29, 35, 36, 37 มีวงกลมกริด 1/2/3/A/B/C/D ปรากฏอยู่ รอบ t1 ตีความว่าเป็น label ทั่วไปแล้วติดแท็ก `grid\_ref\_is\_free\_text\_not\_grid\_notation: true` แต่เเจจนตรอบสองที่ทำงานอิสระอ่านตัวเลขระยะที่พิมพ์จริงบนหน้าแล้วพบว่า ตรงกับกริดหลักของบ้านเป๊ะ รวมถึง dummy grid A'/A" ด้วย — แปลว่านี่คือกริดโครงสร้างจริงที่ใช้ร่วมกันข้ามสาขางาน และ t1 อ่านผิด

ต้นเหตุจริง: t1 ตัดสินสัญลักษณ์จากรูปลักษณ์และสมมติฐานตามสาขางาน ('แบบ MEP ไม่น่ามีกริดโครงสร้าง') แทนที่จะตรวจสอบกับหลักฐานที่วัดได้จริงบนหน้ากระดาษ — คือตัวเลขระยะที่พิมพ์ไว้ ตอนนั้นยังไม่มีไฟล์ grid master กลางให้เทียบกับ จึงตรวจสอบไม่ได้ตั้งแต่แรก

เสียหายอะไร: ข้อมูล 4 หน้าใน dataset มีการตีความโครงสร้างที่ผิด และการจัดแนวกริดข้ามสาขางานก็หายไป ณ สิ้นวันที่ 2026-07-06 ความขัดแย้งนี้ยังไม่ถูกชี้ขาด ('ยังต้องตัดสินใจว่าจะยึด schema ไหนเป็นมาตรฐาน') dataset จึงถือการอ่านที่ขัดแย้งกันสองแบบไว้พร้อมกัน

แก้ยัง: ยังแก้ไม่สมบูรณ์ t2 ใช้ `grid\_source` อ้างอิงไฟล์ grid master กลางแทน ส่วนความขัดแย้ง t1-กับ-t2 ถูกทิ้งไว้เป็นรายการค้าง #4 พร้อมโน้ตว่า t2 น่าจะถูกกว่า แต่ควรตรวจภาพซ้ำก่อนสรุปจริง

กฎที่ได้: จำแนกสัญลักษณ์จากหลักฐานที่วัดได้ (ตัวเลขระยะที่พิมพ์จริง เทียบกับ grid master ของบ้าน) เท่านั้น อย่าตัดสินจากว่ามันอยู่บนแบบสาขางานไหน ต้องมีไฟล์ grid master ก่อนเริ่มสักรอบแรก ไม่ใช่มาสร้างทีหลังหลังจากรอบแรกเดาไปแล้ว

สัญลักษณ์มาร์คพื้น SO/SI/SX/ST ถูกจัดเป็น unknown_symbol ทั้งที่จริงคือพื้น (slab)

เกิดอะไรขึ้น: สัญลักษณ์มาร์คพื้น SO/SI/SX/ST เคยถูกสักรัดเป็น 'unknown_symbol' พอเทียบกับ claude_output_03 ยืนยันว่า element_type แบบ 'slab' ครอบคลุมมันจริง t2 จึงเปลี่ยน element_type เป็น 'slab'

ต้นเหตุจริง: การสักรัดข้อมูลถอยไปใช้ถักรวม 'unknown' แทนที่จะไปเช็คคำอธิบายสัญลักษณ์ที่แบบระบุไว้จริง การมีถึง unknown ที่หลวมเกินไปทำให้ไม่มีแรงกดดันให้แก้สัญลักษณ์ให้ชัดเจน

เสียหายอะไร: element ประเภทพื้นมองไม่เห็นเลยสำหรับ consumer ปลายทางที่กรองด้วย element_type — ไม่มีพื้นที่พื้น ไม่มีคอนกรีตพื้น ไม่มีเหล็กพื้นสำหรับมาร์คเหล่านี้ ที่สำคัญคือการแก้นี้ทำภายใต้คำสั่ง 'ปรับตามที่เห็นสมควร' และไม่เคยถูกยืนยันอย่างเป็นทางการโดยมะขาม — การแก้นี้จึงยังไม่ได้รับการรับรอง

แก้ยัง: Generation 3.1 ข้อ 3: ยืนยันว่า 'slab' ครอบคลุม SO/SI/SX/ST แล้ว เปลี่ยน element_type ในรอบ t2 การยืนยันอย่างเป็นทางการยังคงอยู่

กฎที่ได้: การจัดเป็น 'unknown' คือรายงานบั๊ก ไม่ใช่คำตอบ — ทุก unknown ต้องถูกไล่ตรวจกับ symbol legend ของแบบก่อนปิดรอบ และการแก้ schema ภายใต้คำสั่งกว้างๆ แบบ 'ปรับตามที่เห็นสมควร' ก็ยังต้องได้รับการรับรองชัดเจน ไม่งั้น dataset จะถือคำตัดสินที่ไม่มีใครเซ็นรับรองไว้

schema ไม่มีทางระบุกริดไลน์ที่ไม่มีชื่อ ทำให้คาน B1 ผิดทั้งในการสกัดใหม่และใน Gen 2

เกิดอะไรขึ้น: มะขามเองสังเกตเห็นบนหน้า 06 ว่ามีเส้นผนัง/คานยาวเต็มความสูงอาคาร อยู่ห่างจากกริด '1' ไป 2.55 ม. โดยไม่มีวงกลมมาร์คกริดในแบบต้นฉบับเลย ตรงกับคาน B1 ที่มีปัญหาทั้งในการสกัดสดของหน้า 19 และใน claud_output_03 (Gen 2) — ข้อบกพร่องเดียวกันเกิดซ้ำอิสระในสองเจนเนอเรชัน

ต้นเหตุจริง: schema สมมติว่าทุกเส้นโครงสร้างอยู่บนกริดที่มีชื่อ แต่แบบจริงมีกริดไลน์ที่ไม่มีชื่ออยู่ การสกัดจึงไม่มีที่ให้ใส่ตำแหน่งของ B1 และให้ผลความมั่นใจต่ำหรือผิดทุกครั้ง — เป็นช่องโหว่ของ schema ที่แสดงตัวออกมาเป็นปัญหาคุณภาพการสกัด

เสียหายอะไร: ตำแหน่งของคาน B1 ไม่น่าเชื่อถือตลอดสองเจนเนอเรชันการสกัดที่ทำแยกกัน และต้องอาศัยมนุษย์มองแบบด้วยตาถึงจะสังเกตเห็น ไม่ใช่จากการตรวจสอบอัตโนมัติใดๆ พอเจอปัญหาแล้วก็ต้องออกแบบข้อตกลงใหม่ ไฟล์ master ใหม่ และสกัดหน้าที่เกี่ยวข้องใหม่ทั้งหมด

แก้ยังไง: ออกแบบข้อตกลง Dummy Grid (เติมเครื่องหมาย prime ต่อท้ายกริดหลักที่ใกล้ที่สุด: 1', A', 3', เก็บไว้เป็น type: 'dummy' แยกจาก type: 'named' เสมอ) และสร้างไฟล์ grid master กลาง บ้าน_เล็ก_1ชั้น_01_หน้า00_gridline.json บันทึกเป็น Generation 3.2

กฎที่ได้: เมื่อ element เดียวกันอ่านออกมาไม่ติงสองรอบที่ทำแยกกันอิสระ อย่าโทษคนอ่าน ให้ตรวจสอบก่อนว่า schema สามารถแทนความจริงได้หรือไม่ ความมั่นใจต่ำที่เกิดซ้ำๆ ให้ถือว่าเป็นช่องโหว่ schema ไว้ก่อน จนกว่าจะพิสูจน์เป็นอย่างอื่น

element_id อ่านเป็น 'S1' ทั้งที่แบบเขียนว่า 'SI' และ sheet_name ในไฟล์เดียวกันก็ผิดด้วย

เกิดอะไรขึ้น: ในไฟล์ mk_test หน้า 23 element_id ถูกสกัดเป็น 'S1' ทั้งที่ค่าที่ถูกต้องคือ 'SI' และ sheet_name ในไฟล์เดียวกันก็ผิดด้วย ทั้งสองจุดถูกแก้แล้ว พร้อมบันทึก confidence_flags ระบุที่มาของการแก้ไข

ต้นเหตุจริง: ความสับสนทางสายตาระหว่างเลข 1 กับตัวพิมพ์ใหญ่ I ที่ความละเอียดของแบบ โดยไม่มีเครื่องมือซูมให้ใช้ และไม่มีการตรวจสอบไขว้กับชุดมาร์คพื้นที่ถูกต้องของบ้านหลังนี้ (SO/SI/ST/SX) ซึ่งถ้ามีจะปฏิเสธ 'S1' ได้ทันที

เสียหายอะไร: element id ที่ผิดทำให้การ join สเปกด้วย element_id พังไป สเปกของพื้นแผ่นนั้นจะไปติดกับ element ที่ผิดหรือไม่ติดกับอะไรเลยแบบเงิบๆ และเนื่องจากไฟล์เดียวกันมี sheet_name ผิดด้วย ความผิดพลาดจึงไม่ได้อยู่แค่จุดเดียว

แก้ยังไง: แก้ element_id จาก S1 → SI และแก้ sheet_name ในไฟล์ mk_test หน้า 23 แล้วเพิ่ม confidence_flags อธิบายที่มาของการแก้ไข

กฎที่ได้: สำหรับตัวอักษรที่หน้าตาคล้ายกันที่ความละเอียดของแบบ (1/I, 0/O, 5/S, 8/B) ต้องตรวจสอบทุก id ที่สกัดได้กับชุดมาร์คที่รู้จักของบ้านหลังนั้นเสมอ และปฏิเสธอะไรที่อยู่นอกชุดนั้น อายายอมรับ id แบบพิมพ์อิสระถ้ามีชุดค่าปิด (closed set) อยู่แล้ว

สูง 2026-07-10

ชุดข้อมูล 101 ไฟล์ชุดเดียวกันมีอยู่สองโพลเดอร์ โดยไม่มีใครตัดสินว่าอันไหนคือตัวจริง

เกิดอะไรขึ้น: mk_test/t4/ ถูก copy (ไม่ใช่ move) เข้าไปที่ rawjson_.../01House_Small_1Story_01/ ทำให้ชุดข้อมูลเดียวกันอยู่สองที่ และไม่มีใครตัดสินอย่างเป็นทางการว่าจะแก้ที่อันไหนต่อไป ตอนลองไล่ขั้นตอนการแก้เป็นตัวอย่างก็ต้องเดาคำตอบเอาเอง

ต้นเหตุจริง: การย้ายจากโพลเดอร์ทดสอบขึ้นไปเป็นของจริง ทำด้วยการ copy โดยไม่ปลดระวางต้นทาง แล้วเลื่อนการตัดสินใจเรื่องเจ้าของข้อมูลออกไป

เสียหายอะไร: การแก้ครึ่งไหนก็ตามในอนาคตเสี่ยงจะได้ข้อมูล ground truth ชุดเดียวกันสองเวอร์ชันที่ขัดกัน ความเสี่ยงนี้ยังค้างอยู่ตอนจบวันและถูกยกยอไปอีกหลายวัน

แก้ยังง: ยังไม่ได้แก้ — แศขยาย rule_of_tune.md ให้คุ้มครองโพลเดอร์ใหม่ด้วย และบันทึกความคลุมเครือนี้เป็นการตัดสินใจที่ยังค้างอยู่

กฎที่ได้: การเลื่อนชั้นจากทดสอบไปเป็นของจริง ต้องปลดระวางหรือแข่งขันต้นทางในการกระทำเดียวกัน — ข้อมูล ground truth ที่มีสองชุดใช้งานพร้อมกันคือบักข้อมูลหรือแค่การแก้ครั้งแรกเท่านั้น

สูง 2026-07-12

ส่วนต่างราคาฐานราก 45% ที่ผิดปกติ ถูกตีความว่าเป็นข้อมูลต้นฉบับจริง แทนที่จะทำให้กลับไปอ่านใหม่

เกิดอะไรขึ้น: ตอนอ่านตารางราคา 10 หลังของบ้านหลัง 4 มีคนสังเกตความผิดปกติว่า บ้านหลังที่ 10 มีส่วนต่างราคาระหว่างฐานรากแผ่กับฐานรากเสาเข็มถึง ~45% ขณะที่บ้านหลังอื่นต่างกันแค่ 5-10% แต่ถูกบันทึกเป็นแค่ข้อสังเกตเกี่ยวกับเอกสารต้นฉบับแล้วปล่อยไว้ อีกหนึ่งสัปดาห์ต่อมา การกวาดเทียบตัวอักษรที่ 8 เทาพบว่าตัวเลขของบ้านหลัง 10 อ่านผิดจริงๆ (2,945,000/2,034,000 ทั้งที่ของจริงคือ 2,145,000/2,035,000) — ซึ่งคือต้นเหตุของส่วนต่าง 45% ปลอมนั้นพอดี

ต้นเหตุจริง: ทีมเคยสรุปไว้แล้ว (จากความผิดปกติเรื่องฐานรากแผ่กับเสาเข็มของบ้านหลัง 6) ว่าตัวเลขแปลกๆ ในตารางนี้ "เป็นข้อมูลจริงจากต้นฉบับ" ความผิดปกติอันที่สองซึ่งเป็นคนละเรื่องเลยรับข้อสรุปนั้นมาใช้ต่อ แทนที่จะถูกตรวจด้วยตัวมันเอง — ค่าที่หลุดออกจากกลุ่มถูกมองเป็นแค่หมายเหตุ ไม่ใช่สัญญาณว่าอ่านผิด

เสียหายอะไร: ตัวเลขที่อ่านผิดนอนอยู่ในชุดข้อมูลของบ้านหลายหลังนานประมาณหนึ่งสัปดาห์ และสัญญาณเตือนล่วงหน้าที่ได้มาฟรีๆ ถูกทิ้งไป กว่าที่จะกู้คืนได้ก็ต้องรอการกวาดเทียบตัวอักษรทั้งหลายบ้านในวันที่ 19 ก.ค.

แก้ยังง: แก้ทางอ้อมเมื่อ 2026-07-19 ด้วยการกวาดเทียบตัวอักษรอ้างอิงที่ 8 เทาครบทั้ง 5 ชุด ซึ่งแก้หลังที่ 10 เป็น 2,145,000/2,035,000 (ส่วนต่างกลับมาที่ ~5%)

กฎที่ได้: ค่าที่หลุดจากรูปแบบของเพื่อนบ้านตัวเอง คือสัญญาณให้กลับไปอ่านใหม่ ไม่ใช่แค่จุดโน้ต และอย่าปล่อยให้ข้อสรุปเก่าที่ว่า "ต้นฉบับมันแปลกจริงตรงนี้" กลายเป็นใบอนุญาตให้ข้อผิดพลาดอันที่สองซึ่งไม่เกี่ยวกันเลยผ่านไป

สูง 2026-07-14

ค่ากริด dummy ที่ได้มาจากการบอกปากเปล่า ถูกใส่ confidence_score 1 ทั้งที่ยังไม่ได้ตรวจสอบภาระ

เกิดอะไรขึ้น: กริด dummy 1'=3.45 ถูกเพิ่มเข้ามาตามคำสั่งโดยตรงและได้ confidence_score: 1 ตามธรรมเนียม ทั้งที่ในบันทึกเองก็เขียนว่ายังชี้จุดเส้นภาระที่แน่นอนไม่ได้ 100% แล้วธรรมเนียมเดียวกันนี้ก็ถูกเอาไปใช้กับกริดเป็นชุดๆ ในบ้านหลัง 2, 3, 4 และ 5 ("ตามค่าที่มะขามให้มาโดยตรง confidence 1 ทั้งหมด")

ต้นเหตุจริง: ธรรมเนียมนี้เอา "มีคนบอกอย่างนี้" มาปนกับ "ตรวจสอบกับภาพแล้วอย่างอิสระ" ทั้งที่ confidence_score คือ field ที่คนใช้ข้อมูลปลายทางใช้ตัดสินใจว่าจะเชื่ออะไร ตอนนั้นมันบอกแค่ที่มาของข้อมูล ไม่ได้บอกว่าตรวจแล้ว

เสียหายอะไร: ค่าที่ตรวจสอบไม่ได้ ปนอยู่กับค่าที่วัดจริงจนแยกไม่ออกในชุดข้อมูล B'=12.5 และ B'=10.2 ที่ใส่ confidence 1 ทั้งคู่ ที่หลังพบว่าผิดทั้งคู่ (ที่ถูกคือ 12.2 กับ 10.1) — พิสูจน์ว่าคะแนนนี้ไม่ได้หมายความว่าความแม่นยำที่มันบอก

แก้ยังไง: ยังไม่ได้แก้ — ธรรมเนียมนี้ยังใช้อยู่ ส่วนค่าที่ผิดถูกจับได้ที่ละตัวและแก้ที่หลังโดยเทียบกับแผนต้นฉบับ

กฎที่ได้: confidence_score ต้องบอกว่าหลักฐานเชิงแค่ไหน ไม่ใช่บอกว่าใครให้ตัวเลขมา ค่าที่คนบอกมาต้องมีธงบอกที่มาของตัวเอง เพื่อให้การตรวจย้อนหลังหาเจอทุกตัว

สูง 2026-07-19

ความผิดพลาดแบบ B2X โผล่ซ้ำในบ้านหลัง 5 — ช่วงคานถูกบันทึกเป็น 'รอยเว้า F-E' แทนที่จะเป็นส่วนต่อขยาย F→F'

เกิดอะไรขึ้น: ตอนกำลังหาค่า dummy grid F'(14.40) ของบ้านหลัง 5 เจอการอ่านผิดในหน้า 32/33: สิ่งที่บันทึกไว้ว่าเป็น 'รอยเว้า F-E' จริงๆ แล้วคือช่วงคาน B2X ที่ยื่นจาก F ออกไปถึง F' — ซึ่งเป็นการตีความผิดแบบเดียวกันเป๊ะกับที่เคยเจอและแก้ไปแล้วในบ้านหลัง 2

ต้นเหตุจริง: คานสั้นๆ ที่ยื่นเลยเส้นกริดหลักออกไป หน้าตาเหมือนรอยเว้าที่เว้าเข้าข้างในเป๊ะ ถ้ายังไม่มี dummy grid ที่มันยื่นไปถึงอยู่ใน grid master ตัว dummy grid กับการตีความคานมันขึ้นต่อกันไปมา การอ่านผิดเลยอยู่นิ่งๆ ได้จนกว่าจะมีฝั่งใดฝั่งหนึ่งถูกแก้

เสียหายอะไร: ความผิดพลาดประเภทเดียวกันหลุดเข้าไปในบ้านหลังที่สอง ถ้าไม่ได้ทำงานเรื่อง F' ก็คงไม่มีใครเจอ

แก้ยังไง: แก้แล้วเมื่อ 2026-07-19 ตอนเพิ่ม F'=14.40 พร้อมกับ 4'(9.6) สำหรับคานสั้น BX1 ที่ยื่นเลยกริด 4 ออกไป 0.60

กฎที่ได้: พอแก้บั๊กเรื่องการตีความในบ้านหลังหนึ่งได้แล้ว ให้ไล่กวาดแบบชนิดเดียวกันในบ้านหลังอื่นทันที — ความผิดพลาดแบบ B2X โผล่มาแล้วทั้งในบ้านหลัง 2 และ 5 และเจอห่างกันเป็นเดือนโดยบังเอิญทั้งคู่

สูง 2026-07-19

คาน B3X มีรายการ additional_bars ที่ไม่มีอยู่จริง — บันทึกเหล็กเสริมที่ไม่มีในแบบ

เกิดอะไรขึ้น: พอชุมเข้าไปที่ S-08 (หน้า 36) ของบ้านหลัง 5 แล้วนับจุดในรูปตัดจริง พบว่า additional_bars ที่บันทึกไว้ของ B3X ไม่มีอยู่ในแบบเลยแม้แต่จุดเดียว จึงลบรายการนั้นทิ้ง ไม่ใช่แก้ไข

ต้นเหตุจริง: additional_bars ถูกคัดลอกมาจาก label ที่พิมพ์ไว้ในแบบ โดยไม่เอาไปตรวจกับจุดเหล็กเสริมจริงในรูปตัด — ต้นเหตุเดียวกับบั๊กเรื่องระบุผิดหน้าคาน แต่ผลลัพธ์คือข้อมูลที่ถูกสร้างขึ้นมามั่วๆ ไม่ใช่ข้อมูลที่วางผิดที่

เสียหายอะไร: มีเหล็กเสริมปลอมอยู่ในชุดข้อมูล ซึ่งแยกว่าเหล็กที่หายไป เพราะมันทำให้ปริมาณเหล็กสูงเกินจริง และสอนให้โมเดลเรียนรู้ที่จะสร้างเหล็กขึ้นมาเอง

แก้ยังไง: ลบทิ้งหลังจากชุมเข้าไปนับจุดในหน้า 36 แล้ว

กฎที่ได้: บั๊ก additional_bars ผิดหน้าคาน มีฝาแฝดคือเหล็กที่ไม่มีอยู่จริงเลย ทุกรายการ additional_bars ต้องยืนยันด้วยการนับจุด ไม่ใช่แค่เชื่อว่าอยู่หน้าไหน

สูง 2026-07-20

ฐานรากซ้ำที่จุดกริดเดียวกัน ทำให้นับซ้ำในสองแผ่นแบบ

เกิดอะไรขึ้น: หน้า 29 (S-02 ฐานรากแผ่) ของบ้านหลัง 3 มี F0.90x0.90 ที่จุด B3 ซ้ำอีกตัวนอกเหนือจากรายการหลัก และหน้า 30 (S-03 ฐานรากเสาเข็ม) ก็มีปักแบบเดียวกันเป๊ะ คือมี F5 ที่ B3 ซ้ำ

ต้นเหตุจริง: ไม่มีการตรวจว่า (mark, grid_ref) ต้องไม่ซ้ำกันภายในผังฐานรากเดียวกัน ฐานรากตัวเดียวกันที่ถูกบันทึกสองครั้งเลย รอดออกมาจากขั้นตอนสกัดข้อมูล

เสียหายอะไร: ฐานรากถูกนับซ้ำ ปริมาณและค่าใช้จ่ายบวมเกินจริงตรงๆ ในสองแผ่นแบบ และการที่มันโผล่แบบเดียวกันเป๊ะทั้งสองแบบ แปลว่ามันเป็นปัญหาเชิงระบบ ไม่ใช่เรื่องบังเอิญครั้งเดียว

แก้ยังไง: ลบตัวซ้ำออกทั้งสองที่ และตั้ง confidence_score=1 ให้หน้า 29

กฎที่ได้: ตั้งกฎตรวจสอบว่า (mark ของ element, จุดกริด) ห้ามซ้ำกันภายในแผ่นแบบเดียวกัน — ของซ้ำที่จุดกริดเดียวกันคือ บักเสมอ ไม่เคยเป็นข้อมูลจริง

สูง 2026-07-20

หน้า 31 (S-04) ของบ้านหลัง 3 มี mark และจุดปลายคานผิด 7 จุด รวมถึงวางคานบนเส้นกริดที่ไม่มีคานอยู่จริง

เกิดอะไรขึ้น: การตรวจหน้า 31 ใหม่ทั้งแผ่นด้วยสเกลที่คาลิเบรตแล้ว แก่ข้อผิดพลาดไป 7 จุด: D1-C1 จริงๆ คือ D1'-C1' (ไม่มีคานแนวตั้งที่กริด 1 ในแถบ D-C ทั้งสองชั้น), คาน 2 ตัวที่ระบุไว้ B3X จริงๆ เป็น B3 และมี B3X แยกต่างหากที่หายไปข้างๆ แต่ละตัว, C2-C3 เป็น B3 ไม่ใช่ B2, คานแนวตั้ง B1 ช่วง B2-A2 หายไปทั้งตัว และธง 'อาจเป็นช่องเปิด' 2 อันที่ผิดต้องปลดออก

ต้นเหตุจริง: การสกัดข้อมูลรอบแรกกำหนด mark จากความใกล้เคียง และสมมติว่าที่ไหนมีเส้นกริดตัดกันต้องมีคาน โดยไม่วัดเทียบกับสเกลที่คาลิเบรตแล้ว และไม่เช็คว่าเสาอยู่บนเส้นกริดพอดีหรือเยื้องออกไป

เสียหายอะไร: มี element โครงสร้างผิดหรือหายไป 7 จุดในแผ่นแบบเดียวของบ้านที่อยู่ในชุดเทรน mark ผิดแปลว่าสเปกปลายทาง ผิดด้วย เพราะสเปกอ้างอิงจาก mark

แก้ยังไง: แก้ครบทั้ง 7 จุดเมื่อ 2026-07-20 โดยใช้สเกลที่คาลิเบรตไว้ที่ 78.1 พิกเซล/เมตร ของแผ่นนั้น

กฎที่ได้: mark ในผังคานกำหนดจากความใกล้เคียงไม่ได้ ต้องมีสเกลที่คาลิเบรตแล้ว บวกกับการเช็คตรงๆ ว่า 'เส้นนี้มีคานอยู่จริงหรือเปล่า' ทีละแถบ ทีละชั้น

สูง 2026-07-20

แผ่นแบบคานคอดินของบ้านหลัง 3 ขาดไป 6 element และมีการระบุระยะเยื้องผิด ซึ่งชุม 10 เท่าแล้วพิสูจน์ว่าไม่จริง

เกิดอะไรขึ้น: หน้า 33 (S-06 คานคอดิน) เพิ่มจาก 17 เป็น 23 element หลังตรวจจากภาพเต็ม: คานแนวตั้ง D2-C2 กับ B2-A2 หายไปทั้งตัว, คาน B'2-B'3 และ B3-B'3 ก็หายไป และ D1'-C1' เป็นการระบุผิด — ชุม 10 เท่าแล้วยืนยันว่าเสาอยู่บนกริด 1 พอดี ไม่มีระยะเยื้อง คานคอดินจึงเป็นวงต่อเนื่อง ไม่ได้ขาดตรงช่องบันได

ต้นเหตุจริง: เอารูปแบบระยะเยื้องที่เห็นใน S-04/S-05 มาสมมติว่าใช้กับ S-06 ได้เลยโดยไม่ตรวจกับแผ่นนั้นจริง แกรม 26% ของ element ในแผ่นนั้นไม่เคยถูกสกัดออกมาเลย

เสียหายอะไร: คานคอดินในแผ่นนั้นหายไปจากชุดข้อมูลถึงหนึ่งในสี่ และรูปแบบการต่อวงถูกบันทึกผิด

แก้ยังไง: ตรวจใหม่ทั้งหมดเทียบกับภาพต้นฉบับเมื่อ 2026-07-20 เพิ่ม element ครบทั้ง 6 ตัวและแก้ที่ระบุผิด

กฎที่ได้: รูปแบบระยะเยื้องที่เห็นในแผ่นหนึ่ง ต้องตรวจยืนยันใหม่ในทุกแผ่นที่จะเอาไปใช้ — แผ่นแบบของบ้านหลังเดียวกันต่างกันได้จริง

ไฟล์ต้นทางของบ้านหลังที่ 05 จำนวน 7 ไฟล์ถูกแก้ไขหลังจากสร้าง dataset ไปแล้ว — รวมถึง gridmaster ด้วย ทำให้ทั้ง t01 และ t02 เทรนบนข้อมูลที่รู้อยู่แล้วว่าผิด

เกิดอะไรขึ้น: dataset ของ t02 ถูกก๊อปปี้มาแบบเหมือนเดิมทุกไบต์แทนที่จะสร้างใหม่ เพราะไฟล์ 7 ไฟล์ในบ้านหลังที่ 05 (รวมถึง หน้า00_gridline.json) ถูกแก้ไขหลังจากสร้าง dataset ครั้งแรกไปแล้ว ถ้าสร้างใหม่จะได้ dataset คนละชุด ทำให้เทียบผลกันไม่ได้ จึงตัดสินใจไม่เอาการแก้ไขเข้ามา นั่นแปลว่าข้อมูลเทรนยังเป็นเวอร์ชันก่อนแก้ไข รวมถึงตัวอย่างประเภท gridmaster ที่สคริปต์ระบุว่า เป็นหัวข้อที่โมเดลพลาดมากที่สุดด้วย

ต้นเหตุจริง: ไม่มีการเชื่อมโยงระหว่าง raw JSON ที่แก้ไขแล้วกับ dataset ที่สร้างไปแล้ว การแก้ไขไฟล์ต้นทางจึงไม่ถูกส่งต่อไปที่ dataset โดยอัตโนมัติ และไม่มีตัวบ่งชี้เลยว่าตัวอย่างไหนใน dataset เป็นข้อมูลเก่าที่ล้าสมัยแล้ว

เสียหายอะไร: ทั้ง t01 (recall 28.2%) และ t02 (recall 20.6%) ถูกเทรนบนข้อมูลที่รู้อยู่แล้วว่ามีข้อผิดพลาดที่เคยแก้ไขไปแล้วปอนอยู่ และตอนนี้ dataset ถูกแก้ไขไว้แบบนี้ ไม่สามารถสร้างใหม่ได้อีก เพราะจะทำให้ตัวเลข A/B ที่วัดไว้ทั้งหมดเป็นโมฆะ

แก้ยังไง: ยังไม่ได้แก้ — จงใจแก้ไข dataset ไว้ตามเดิม บันทึกไว้ที่ tune_ai/t02/data_before_tune/DATASET_PROVENANCE.md พร้อมหลักฐาน checksum และเหตุผลว่าทำไมห้ามสร้าง dataset ใหม่

กฎที่ได้: ต้องบันทึกว่าตอนสร้าง dataset ใช้ไฟล์ต้นทางไหนบ้างและค่า hash ของแต่ละไฟล์ ณ ตอนสร้าง เมื่อไฟล์ต้นทางถูกแก้ไขทีหลัง dataset นั้นถือว่าล้าสมัยทันที ต้องเลือกอย่างใดอย่างหนึ่ง: สร้างใหม่ก่อนเริ่มเปรียบเทียบ หรือยอมรับว่าทั้งการทดลองจะสืบทอดข้อผิดพลาดเก่าไปตลอด

ขั้นตอนจำแนกหน้าแบบเองก็ผิด — หน้ารูปด้านถูกอ่านเป็นหน้าตัด อ้างว่ามี 5 มุมมองทั้งที่มีแค่ 2

เกิดอะไรขึ้น: หน้า 07/08/09 ถูกจำแนกเป็น section/section/title/gridline/notes ทั้งที่ของจริง (gold) คือ side_profile ทั้งคู่ โมเดลยังอ้างว่าแผ่นนั้นมี 5 มุมมอง ทั้งที่จริงมีแค่ 2

ต้นเหตุจริง: จุดอ่อนของโมเดล (น่าจะจาก GGUF ที่ไม่ได้ tune ให้ดีพอ บวกกับ resolution ไม่ตรงกับตอนเทรน) ไปกระทบขั้นตอน classify ด้วย และไม่มีใครตั้ง gate วัดความแม่นยำของขั้น classify ก่อนจะปล่อยให้มันเป็นตัวคัดกรองว่าหน้าไหนควรส่งเข้า extraction ที่แพงกว่า

เสียหายอะไร: กระบวนการ 2-phase ตรงๆ — ถ้า classify ผิด หน้าที่ต้องอ่านจริงจะถูกข้าม ส่วนหน้าที่ไม่มีประโยชน์กลับถูกเอาไป extract เต็มๆ หน้าละ 13-21 นาที ความผิดพลาดตรงนี้เสียบบากและกินเวลา/ทรัพยากรฟรี

แก้ยังไง: ยังไม่แก้ — บันทึกไว้เป็นข้อควรระวัง

กฎที่ได้: เมื่อขั้นตอนที่ถูกจะกลายเป็นตัวกำหนดว่าอะไรจะไปเข้าขั้นตอนที่แพง ต้องวัดความแม่นยำของขั้นตอนถูกก่อนเสมอ ถ้าตัว router ผิด 30% การ 'ประหยัด' จะกลายเป็น 'ขาดทุน' ทันที

สูง 2026-07-29

เลขอ้างอิงแผ่นแบบ (A-05, A-07, A-09) ถูกโมเดลหลอนเป็นชื่อมาร์คของชิ้นส่วน

เกิดอะไรขึ้น: หน้า 06 โมเดลตอบว่ามีชิ้นส่วน 59 ชิ้น ทั้งที่ของจริงมีแค่ 17 — สร้างเสา 12 ต้น ประตู 8 บาน หน้าต่าง 4 บานที่ไม่มีอยู่จริงขึ้นมาเอง แล้วตั้งชื่อว่า A-05/A-07/A-09 ซึ่งจริงๆ คือเลขแผ่นแบบ (ในแบบมีข้อความ 'ดูรายละเอียดที่แผ่น A-05' โมเดลอ่านแล้วเข้าใจว่าเป็นชื่อชิ้นส่วน)

ต้นเหตุจริง: โมเดลอ่านตัวหนังสือถูก แต่ไม่เข้าใจว่าข้อความนั้นทำหน้าที่อะไร — ข้อความอ้างอิงข้ามแผ่นกับชื่อมาร์คของชิ้นส่วนหน้าตาคล้ายกันในเชิงตัวอักษร และไม่มีอะไรในระบบคอยแยกสองอย่างนี้ออกจากกัน

เสียหายอะไร: นับชิ้นส่วนเกินจริง 3.5 เท่าในหน้าเดียว ถ้าข้อมูลนี้หลุดไปถึง BOQ จะสั่งเสาที่ไม่มีอยู่จริงเป็นสิบต้น

แก้ยังไง: ยังไม่แก้

กฎที่ได้: เพิ่มกฎตรวจสอบให้ปฏิเสธชื่อมาร์คที่มีรูปแบบเหมือนเลขแผ่น (ตัวอักษร-ขีด-ตัวเลข) แล้วเช็คจำนวนชิ้นส่วนย้อนกับตารางสรุปในแบบเอง อย่าปล่อยให้การจับคู่ตัวอักษรที่คล้ายกันกลายเป็นข้อสรุปเชิงความหมายโดยไม่มีการตรวจสอบโครงสร้างมายืนยัน

สูง 2026-07-29

อักษรกริดสลับด้าน แกรมสร้างตาราง grid_table ที่ไม่มีจริงขึ้นมาเอง — ทำให้ช่วงระยะทุกจุดผิดพลาด

เกิดอะไรขึ้น: กริดแนวนอนของจริงคือ D=0, C=4.0, B=6.0, A=9.5 แต่โมเดลตอบ A=0, B=1.5, C=5.0, D=9.5 — อักษรสลับด้าน ระยะภายในผิดพลาด เหลือแค่ค่ารวม 9.5 ที่ถูก ในหน้า 06 โมเดลยังสร้างตาราง grid_table ที่ใส่ค่าให้กับ 'จุดตัดกริด' แต่ละจุดซึ่งไม่ใช่ปริมาณที่มีอยู่จริงในโลกจริง โดยเอาตัวเลขจริงจากแบบ (7.0, 9.5, 4.0, 3.5) มาสลับใส่ในช่องต่างๆ เอง

ต้นเหตุจริง: โมเดลได้ตัวเลขชุดที่ถูกจัดโครงสร้างผิด และเพราะ prompt สั่งให้คำนวณ span จากตาราง grid ตารางที่ดูสมเหตุสมผลแต่แต่งขึ้นเองนี้จึงกลายเป็นฐานของทุกการคำนวณ span ต่อไปโดยไม่มีใครรู้

เสียหายอะไร: span ทุกตัวที่คำนวณต่อจากกริดนี้ผิดพลาด และเป็นตัวเลขที่ดูสมเหตุสมผลจนตรวจด้วยตาเปล่าไม่มีทางจับได้

แก้ยังไง: ยังไม่แก้

กฎที่ได้: ต้องตรวจสอบโครงสร้าง ไม่ใช่แค่ว่ามีค่าใส่มาหรือเปล่า — เช็คตัวอักษรกริดเรียงตามทิศทางจริงของแบบ และเช็คค่าค่าที่ใส่ในตารางตรงกับปริมาณที่มีอยู่จริงในภาพ ตัวเลขที่แต่งขึ้นแต่ดูสมเหตุสมผลคืออันตรายที่สุด เพราะผ่านทุกการตรวจแบบ 'มีค่าใหม่' หมด

สูง 2026-07-29

ผนังก่ออิฐมอญถูกอ่านเป็นฝ้าเพดานบุมวลเบา — BOQ ผิดพร้อมกันสองชั้น

เกิดอะไรขึ้น: หน้า 04 ของจริงระบุว่าเป็นผนัง (ผนัง) ก่อด้วยอิฐมอญ แต่โมเดลตอบว่าเป็นฝ้าเพดาน สร้างด้วยบล็อกมวลเบา (AAC) — ผิดทั้งประเภทงานและวัสดุ

ต้นเหตุจริง: ไม่มีการตรวจสอบความสอดคล้องข้ามฟิลด์ระหว่างประเภทงานกับวัสดุ ทำให้คู่ที่ผิดทั้งสองฝั่งแยกกันโดยอิสระ กลายเป็นรายการที่ดูเข้ากันเองแต่เท็จทั้งหมด

เสียหายอะไร: ถ้าเอาไปตั้งราคาจะผิดสองชั้นพร้อมกัน คือผิดทั้งประเภทงานและผิดทั้งราคาต่อหน่วย เป็นความผิดพลาดประเภทที่ทำให้ยอด BOQ ผิดโดยไม่มีใครสังเกต

แก้ยังไง: ยังไม่แก้

กฎที่ได้: ตั้ง whitelist ว่าวัสดุแต่ละประเภทใช้กับงานประเภทไหนได้บ้างตั้งแต่ตอน extract (บล็อกมวลเบาไม่ใช่วัสดุฝ้าเพดาน) เพื่อให้คู่ที่ไม่สมเหตุสมผลถูกปฏิเสธก่อนไปตั้งราคา ฟิลด์คูไหนที่กำหนดเงินรวมกันต้องมีการตรวจสอบร่วมกัน ไม่ใช่ตรวจแยกกันคนละที

สูง 2026-07-29

confidence_score / confidence_flags หายไปทั้งหมดในไฟล์ผลลัพธ์ทั้ง 6 ไฟล์

เกิดอะไรขึ้น: schema ต้นฉบับกำหนดให้ทุกชิ้นส่วนมี confidence_score และ confidence_flags (เช่น 'ตัวหนังสือเล็กเกิน ต้อง zoom ตรวจ') แต่โมเดลไม่ได้ผลิตฟิลด์เหล่านี้เลยแม้แต่ตัวเดียวในทั้ง 6 ไฟล์

ต้นเหตุจริง: เป็นความล้มเหลวเดียวกันที่ไม่ทำตาม schema ที่เรนไว้ — โมเดลไม่ได้ใส่ schema ที่ผ่านการเรนออกมาเลย เพราะขั้นตอนประเมินความมั่นใจมีอยู่แค่ในข้อมูลเรน (training targets) เท่านั้น

เสียหายอะไร: ไม่มีทางรู้เลยว่าส่วนไหนของผลลัพธ์เชื่อถือได้บ้าง กลไกทั้งหมดของ pipeline ที่ควรส่งจุดที่ไม่แน่ใจไปให้คนตรวจถูกปิดไปเสียๆ

แก้ยังง: ยังไม่แก้

กฎที่ได้: ถือว่าการมีอยู่ของขั้นตอนประเมินความมั่นใจตัวเองเป็นข้อบังคับของผลลัพธ์ — ถ้าไฟล์ extraction ไม่มีฟิลด์ confidence เลยแม้แต่ตัวเดียว ต้องปฏิเสธไม่รับ ไม่ใช่ถือว่าเสร็จสมบูรณ์ ผลลัพธ์ที่ไม่มีตัวบอกความไม่แน่นอนไม่ใช่ผลลัพธ์ที่มั่นใจสูง มันคือผลลัพธ์ที่ไม่มีป้ายกำกับ

สูง 2026-07-29

หน้า 06 นับได้ 59 ชิ้นส่วน ทั้งที่ของจริงมีแค่ 17 — และรายการเดิมยังถูกนับซ้ำในมุมมองหลังคาอีกรอบ

เกิดอะไรขึ้น: ในหน้าผังพื้น โมเดลตอบว่ามีชิ้นส่วน 59 ชิ้น ทั้งที่ของจริงมี 17 ส่วนเกินคือเสา 12 ต้น + ประตู 8 บาน + หน้าต่าง 4 บานที่ไม่มีในผังนี้เลย และบันทึกระบุชัดที่ผังชิ้นนี้ 'ถูกนับซ้ำอีกรอบในมุมมองหลังคา' คือรายการเดียวกันที่ไม่มีจริงถูกนับสองครั้งข้ามสองมุมมองของหน้าเดียวกัน

ต้นเหตุจริง: โมเดลไม่มีความเข้าใจว่า 'มุมมองนี้เคยนับรายการนี้ไปแล้ว' มันแจกแจงรายการใหม่ทุกมุมมองและแต่งสมาชิกขึ้นมาเติมช่องว่างของ schema แทนที่จะปล่อยว่างไว้ ไม่มีขั้นตอนไหนคอย de-duplicate ชิ้นส่วนข้ามมุมมองของผังเดียวกัน และไม่มีการเช็คจำนวนเทียบกับ gold ก่อนเขียนไฟล์

เสียหายอะไร: นับชิ้นส่วนเกินจริง 3.5 เท่าในหน้าเดียว ถ้าเอาไปทำ BOQ จะสั่งเสา/ประตู/หน้าต่างของชั้นนั้นเกินเกือบ 3 เท่า และเพราะนับซ้ำข้ามมุมมองด้วย ต่อให้แก้การอ่านต่อมุมมองแล้ว ปริมาณรวมก็ยังเป็นสองเท่าอยู่ดี

แก้ยังง: ยังไม่แก้ — วัตถุเทียบกับ gold เท่านั้น การ extract ถูกหยุดรอผลทดสอบ LoRA-in-GGUF

กฎที่ได้: จำนวนชิ้นส่วนคือสัญญาณตรวจสอบที่ถูกที่สุดที่มี — เทียบจำนวนต่อหน้ากับ gold (หรือกับขอบเขตที่สมเหตุสมผล) ก่อนเขียนผลลัพธ์เสมอ และถือว่าการนับซ้ำข้ามมุมมองเป็นความผิดพลาดคนละแบบที่ต้องแก้แยก — ตัว extractor ที่ทำงานที่ละมุมมองต้องมีขั้นตอน de-duplicate ชัดเจน ไม่จ้นผังที่มีหลายมุมมองจะถูกนับปริมาณเกินทุกครั้ง

สูง 2026-07-29

สกัดข้อมูล 4 หน้าที่ย้ายแล้วเขียนลงโพลเดอร์ผลลัพธ์จริงไปแล้ว ก่อนจะรู้ว่า pipeline พัง

เกิดอะไรขึ้น: หน้า 03-06 ถูกสกัดด้วย GGUF ที่ต้องสงสัย แล้วเซฟลงโพลเดอร์ผล/ ที่เป็นของจริง ก่อนที่จะรู้ว่าไฟล์นั้นอาจไม่มีการ fine-tune อยู่เลย ถ้าทำต่อไปเรื่อยๆ จะยัดข้อมูลผิดเข้าเต็มโพลเดอร์บ้านทั้งหลัง

ต้นเหตุจริง: เริ่มสกัดข้อมูลจริงก่อนที่ artifact จะผ่านการตรวจสอบความถูกต้องของเนื้อหา (ดูเรื่องตัวตรวจ sanity ที่อ่อนและการทดสอบ Phase 10 ที่ตื่นเกินไป) และไม่มีพื้นที่ staging แยกผลทดลองออกจากผลที่ยอมรับแล้ว

เสียหายอะไร: ไฟล์ผลลัพธ์ 4 หน้าที่เป็นต้นตอต้องถูกทิ้งหรือรันใหม่ เกือบพลาดจนยัดข้อมูลผิดเต็มโพลเดอร์บ้านทั้งหลัง

แก้ยังง: หยุดการสกัดข้อมูลไว้ก่อน รอผลทดสอบว่ามี LoRA จริงหรือไม่

กฎที่ได้: เขียนผลลัพธ์ของโมเดลลงพื้นที่ staging ไว้ก่อน จนกว่า artifact จะผ่านการตรวจสอบความถูกต้องของเนื้อหา ห้ามให้ pipeline ที่ยังไม่ผ่านการตรวจสอบเขียนลง dataset จริงตรงๆ เด็ดขาด

สูง 2026-07-29

โมเดล `grid_table` ให้ค่าความยาวหนึ่งค่าต่อจุดตัดกริดหนึ่งจุด — ปริมาณที่ไม่มีอยู่จริง — ทำให้ทุกช่วงที่คำนวณต่อจากนั้นพังหมด

เกิดอะไรขึ้น: ในหน้า 06 โมเดลสร้างโครงสร้าง `grid_table` ที่มีค่าตัวเลขหนึ่งค่าต่อจุดตัดกริดหนึ่งจุด แต่จุดตัดกริดคือจุด ไม่มีความยาว โมเดลเอาตัวเลขมิติจริงบนแบบ (7.0, 9.5, 4.0, 3.5) ไปกระจายใส่ผิวดช่อง เพราะ `PROMPT_SHORT` ระบุว่า 'ความยาวช่วงต้องมาจากตาราง `grid` ไม่ใช่การประเมินด้วยสายตาเอง' ทุกช่วงที่คำนวณจากหน้านี้จึงผิดทั้งหมด

ต้นเหตุจริง: โมเดลสร้างโครงสร้างที่ดูสมเหตุสมผลในเชิงโครงสร้างแต่ไม่มีความหมายทางกายภาพจริง และไม่มีอะไรใน pipeline ตรวจสอบว่าเซลล์ในตาราง `grid` ต้องแทนช่วงระหว่างเส้นที่อยู่ติดกัน ไม่ใช่แทนจุด

เสียหายอะไร: ไม่ใช่แค่ตัวเลขผิดที่ละจุด — เพราะ prompt ทำให้ทั้งสายการคำนวณช่วงขึ้นอยู่กับตารางนี้ โครงสร้างที่ก่อกำเนิดขึ้นจึงทำให้ทุกปริมาณที่คำนวณต่อในหน้านั้นผิดหมด แล้วยังไหลต่อเข้า BOQ อีก

แก้ยังง: ยังไม่ได้แก้ — บันทึกไว้เป็นข้อบกพร่องเมื่อ 2026-07-29 ยังไม่มีการตรวจสอบ schema ของ `grid_table` ในเส้นทางการสกัดข้อมูลเลย

กฎที่ได้: ต้องตรวจสอบว่าโครงสร้างที่โมเดลสร้างออกมาแทนปริมาณทางกายภาพจริง ไม่ใช่แค่เป็น JSON ที่ถูกต้อง — ตารางที่ index ด้วยจุดแทนที่จะเป็นช่วง จะผ่านการตรวจสอบไวยากรณ์ทุกอย่างได้สบาย แต่ทำให้ทุกอย่างที่คำนวณต่อจากมันเสียหาย

สูง 2026-08-01

เพิ่ม `dummy grid 2'=5.2` เข้าไฟล์ `grid master` ของบ้าน 04 แล้วแต่ไม่ได้ใส่แก้ในไฟล์แปลนที่เกี่ยวข้อง

เกิดอะไรขึ้น: แถบ x ตัวใหม่ `dummy grid '2'(5.2)` ถูกใส่เข้าไฟล์เส้น `grid` ของบ้าน 04 ระหว่างกริด 2 กับ 3 ด้วยความมั่นใจระดับ 1 — แต่ไม่ได้ถูกใส่ในไฟล์แปลนที่ `pattern=plan` (แปลนฐานราก/คาน หน้า 31–34 และอื่นๆ) ที่อ้างอิงกริดนั้นเลย

ต้นเหตุจริง: ไฟล์ `grid master` กับไฟล์แปลนที่ใช้มันถูกแยกกันเป็นคนละงาน โดยไม่มีขั้นตอน cascade หรือตัวตรวจสอบใดๆ บังคับว่าการแก้ `grid master` ต้องไปถึงไฟล์ที่พึ่งพามันด้วย

เสียหายอะไร: `grid master` กับไฟล์แปลนของบ้าน 04 ขัดแย้งกันเอง ชิ้นส่วนใดก็ตามที่วางตำแหน่งอ้างอิงกริด 2 หรือ 3 บนแปลนเหล่านั้น ตอนนี้อ้างอิงกริดที่เก่าไม่ตรงกับความจริงแล้ว

แก้ยังง: ยังไม่ได้แก้ — ตั้งใจปล่อยค้างไว้เป็นงาน cascade + ตรวจสอบซ้ำกับภาพต้นฉบับ

กฎที่ได้: การแก้ `grid master` จะยังไม่ถือว่าเสร็จ จนกว่าไฟล์แปลนทุกไฟล์ที่พึ่งพามันจะถูกอัปเดตและตรวจสอบซ้ำแล้ว เพิ่มกฎให้ตัวตรวจสอบ: ทุกกริดที่อ้างอิงใน master ต้องแก้ในแปลนได้จริง และย้อนกลับก็ต้องจริงเหมือนกัน

สูง 2026-08-02

บ้าน 10/11 เก็บค่าเหล็กเสริม 65 รายการเป็นสตริงธรรมดา แทนที่จะเป็น object ตาม schema §6

เกิดอะไรขึ้น: ค่า ``main_bar`/`stirrup`` 65 รายการในบ้าน 10 และ 11 เป็นสตริงธรรมดา (พิมพ์ตามที่เห็นบนแบบ) แทนที่จะเป็น object ตาม §6 ที่ schema กำหนด ต้องแปลงย้อนหลังทั้งหมด

ต้นเหตุจริง: ไม่มีการบังคับ 'รูปแบบ' ของค่าในฟิลด์ — schema บอกว่าต้องเป็น object แต่ตัวตรวจสอบไม่ได้เช็คจริง จึงสกัดข้อความดิบมาใส่ได้ตรงๆ แล้วผ่านการตรวจ

เสียหายอะไร: สเปกเหล็กเสริม 65 รายการไม่สามารถถูกอ่านโดยอะไรก็ตามที่คาดหวังจำนวน/ขนาดที่เป็นโครงสร้าง ซึ่งเป็นฟิลด์ที่การคำนวณ BBS และ BOQ ในขั้นถัดไปพึ่งพาโดยตรง

แก้ยังง: แปลงทั้ง 65 รายการเป็น object ตาม §6 พร้อมเก็บสตริงต้นฉบับไว้ใน ``printed_as`` ของทุกรายการ ไม่ให้ข้อมูลเดิมหายไป

กฎที่ได้: ต้องบังคับ 'รูปแบบ' ของฟิลด์ ไม่ใช่แค่บังคับว่ามีฟิลด์นั้น และเวลาแปลงค่าใดๆ ต้องเก็บค่าต้นฉบับไว้ใน sibling ชื่อ ``printed_as`` เสมอ — ถือว่าการแปลงเป็นการสูญเสียข้อมูลจนกว่าจะพิสูจน์ได้ว่าไม่ใช่

สูง 2026-08-02

บ้าน 05 เต็มคำอธิบายต่อท้าย element_id (F1.30x1.30_E1) ทำให้จับคู่ข้ามหน้าไม่ได้

เกิดอะไรขึ้น: บ้าน 05 เขียน id ของ element โดยเติมคำต่อท้ายเข้าไปหลังมาร์คที่พิมพ์อยู่บนแบบ พอ id ไม่ตรงกับมาร์คที่ใช้ในหน้าอื่น การจับคู่ข้ามหน้าเลยพัง

ต้นเหตุจริง: ไม่เคยมีกฎเขียนไว้ว่า `element\_id` ต้องเป็นมาร์คที่พิมพ์บนแบบเป๊ะๆ ห้ามมีอะไรเกิน พอ id แบบมีคำอธิบายดูให้ข้อมูลมากกว่า ก็เลยเขียนแบบนี้ไปเลย

เสียหายอะไร: element ของบ้าน 05 จับคู่ระหว่างหน้าผังกับหน้า schedule/section ไม่ได้ — ซึ่งการจับคู่นี้คือกลไกทั้งหมดที่ทำให้สเปกไปถึงตัว element

แก้ยังง: ลด element_id 82 ตัวให้เหลือแค่มาร์คที่พิมพ์บนแบบ แล้วเก็บของเดิมไว้ในช่อง `element\_id\_printed\_as` ที่หลังล็อกเป็นกฎ §0.3

กฎที่ได้: หน้าที่เดียวของ id คือเอาไว้จับคู่ ข้อมูลที่เป็นคำอธิบายให้แยกไปอยู่ช่องของมันเอง ห้ามยัดใส่ id เด็ดขาด — id ที่ "ช่วยให้เข้าใจง่ายขึ้น" ก็คือ id ที่จับคู่ไม่ได้

สูง 2026-08-02

array ชื่อ `column\_sections` แต่ข้างในเป็น section ของคานล้วน — cut ref ทั้ง 17 ตัวเป็นมาร์ค B

เกิดอะไรขึ้น: เจอ array ชื่อ `column\_sections` ที่ข้างในไม่มีเสาเลยสักตัว — ค่า `cut\_ref` ทั้ง 17 ตัวเป็นมาร์คคานหมด (B1, B2A, B3, R1, RB0, B4, B5, B6X)

ต้นเหตุจริง: ชื่อ array ตั้งตามหัวข้อบนหน้าแบบหรือตั้งตามที่เดาเอา ไม่ได้ตั้งตามสิ่งที่ข้างในมันอ้างถึงจริงๆ และไม่มีใครเช็คชื่อ array ตรงกับมาร์คข้างในหรือเปล่า

เสียหายอะไร: section ของคาน 17 ตัวถูกจัดประเภทเป็นเสา ถ้าไหลต่อไปข้างล่างมันจะถูกคิดราคา วางแผน และเสริมเหล็กแบบเสา แถมยังสอนโมเดลให้ติดป้ายคานผิดอีกด้วย

แก้ยังง: แปลงทั้ง 17 ตัวเป็น `element\_type: "beam"` โดยเก็บป้าย cut เดิมไว้ใน `section\_ref`

กฎที่ได้: อย่าเชื่อชื่อกล่องว่ามันบอกสิ่งที่อยู่ข้างในจริง ต้องตรวจ element_type เทียบกับ prefix ของมาร์คเสมอ — เจอมาร์ค B อยู่ใน array ของเสา ให้ถือว่าไม่ผ่านทันที

สูง 2026-08-07

แนวนคานหน้า 18/19 ของบ้าน 14 ยังไล่ไม่ครบ — ปลอยไว้ที่ confidence ≤ 0.6

เกิดอะไรขึ้น: แนวนคานบนหน้า 18 และหน้า 19 ของบ้าน 14 ยังไล่ไม่ครบ รายการถูกทิ้งไว้ที่ confidence ≤ 0.6 พร้อมติดธง `needs\_zoom\_review` แต่ตัวบ้านกลับถูกรายงานว่าเสร็จแล้ว

ต้นเหตุจริง: การไล่แนวนคานบนผังเป็นส่วนที่ยากที่สุดและทำอัตโนมัติได้น้อยที่สุดของการสกัดข้อมูล และ workflow ก็ยอมให้ปิดบ้านได้ทั้งที่ยังมีรายการคานความมั่นใจต่ำค้างอยู่ — เป็นปัญหาชนิดเดียวกับที่บ้าน 01–05 คานหาย

เสียหายอะไร: มีบ้านอีกหลังเข้า corpus ทั้งที่รู้ว่าข้อมูลคานไม่ครบ ไปเพิ่มหนี้เรื่องคานหาย ซึ่งเป็นจุดบกพร่องที่ใหญ่ที่สุดของชุดข้อมูลอยู่แล้ว

แก้ยังง: ยังไม่แก้ — ติดธง `needs\_zoom\_review` แล้วแบกเป็นงานค้างไว้

กฎที่ได้: คานคือจุดอ่อนที่รู้กันอยู่แล้วของทั้ง pipeline นี้ ถ้ารายการคานตัวไหน confidence ต่ำกว่าเกณฑ์ ให้ถือว่าบ้านหลังนั้นยังปิดไม่ได้ ไม่ใช่แค่ติดธงแล้วปลอยผ่านไป

สูง 2026-08-10

แบบโครงสร้างกับแบบสถาปัตยกรรมของบ้าน 19 ติดป้ายแกนกริดกลับด้านกัน ทั้งที่เป็นอาคารเดียวกัน

เกิดอะไรขึ้น: ในบ้าน 19 แบบโครงสร้างกับแบบสถาปัตยกรรมของอาคารเดียวกันติดป้ายแกนกลับด้านกัน — เป็นการสลับป้ายข้ามสาขาวิชาชีพจริงๆ ภายในชุดแบบชุดเดียว

ต้นเหตุจริง: แต่ละสาขาออกแบบทำแบบของตัวเองด้วยธรรมเนียมการติดป้ายกริดของตัวเอง แล้วชุดแบบถูกออกมาโดยไม่มีใครมาปรับให้ตรงกัน

เสียหายอะไร: element ไหนก็ตามในบ้านนี้ที่ต้องจับคู่ข้ามระหว่างแบบโครงสร้างกับแบบสถาปัตย์ จะถูกวางผิดแกนทันที ถ้าไม่มีใครตรวจเจอความขัดแย้งนี้แล้วแก้

แก้ยังไง: แก่ด้วยกฎลำดับความสำคัญว่า 'แบบโครงสร้างชนะ' และพอนตัวเช็คให้รับได้ทั้งสองแบบการวางแกน

กฎที่ได้: ธรรมเนียมกริดต่างกันได้*ภายใน*ชุดแบบชุดเดียว ไม่ใช่แค่ต่างกันระหว่างบ้าน ต้องตรวจหาความขัดแย้งของการติดป้ายแกนแยกตามสาขา แล้วใช้กฎลำดับความสำคัญที่เขียนไว้ชัดเจน แทนที่จะเหมาว่าอาคารหนึ่งหลังมีนิยามกริดชุดเดียว

สูง 2026-08-21

notes คือ pattern ที่เพี้ยนหนักที่สุด — 55 ไฟล์เก็บเนื้อหาเดียวกันใต้ชื่อ key ต่างกันถึง 6 แบบ

เกิดอะไรขึ้น: ไฟล์ notes 55 ไฟล์เก็บเนื้อหาแบบเดียวกัน แต่ใช้ชื่อ container key ต่างกัน 6 แบบ — notes (22), notes_sections (9), sections (8), spec_notes (3), notes_text (1), raw_text (1) และยังมีชื่อแปลกๆ อีกประปราย ทั้งที่สเปกกำหนดไว้ว่าต้องเป็น sections[] แต่มีแค่ 8 จาก 55 ไฟล์ที่ใช้ตามนั้นจริง

ต้นเหตุจริง: check_format.py ไม่เคยบังคับชื่อ container key ของ pattern นี้เลย บวกกับ notes เป็น pattern ที่มีโครงสร้างหลวมที่สุดในบรรดา 16 แบบ พอไม่มีตัวตรวจ แต่ละรอบที่นั่งสกดข้อมูลก็คิดชื่อ key ที่ดูสมเหตุสมผลขึ้นมาเอง เป็นความผิดพลาดคลาสเดียวกับ roof_frame_plan และเรื่องสะกดคำ discipline ไม่ตรงกัน แต่รอบนี้ใหญ่กว่ามาก

เสียหายอะไร: ไฟล์ notes 85% อ่านไม่ได้เลยสำหรับ consumer ตัวไหนก็ตามที่ยึดชื่อ key ตามสเปก ทางเดินข้อมูลใหม่ที่เพ่งนิยามไว้ใน §4a จึงว่างเปล่าในทางปฏิบัติ จนกว่าจะเขียนไฟล์พวกนี้ใหม่

แก้ยังไง: ยังไม่ได้แก้ บันทึกรไว้ว่าต้องย้ายทั้ง 55 ไฟล์มาเป็น sections[] + notes{} เป็นงานค้าง

กฎที่ได้: ชื่อ container/key ต้องให้ตัวตรวจ format บังคับจริง ไม่ใช่แค่เขียนไว้ในสเปก — pattern ไหนที่ตัวตรวจเจียบ ให้เตรียมใจว่าจะได้ชื่อ key ใหม่ทุกรอบที่ทำงาน และควรไล่ใบการกระจายของชื่อ key ทั้ง dataset เป็นระยะ

สูง 2026-08-21

ออกแบบ **plan_column** เป็น **subtask** ของตัวเอง แต่ทั้ง 11 หลังมีไฟล์อยู่แค่ 2 ไฟล์

เกิดอะไรขึ้น: ตอนออกแบบ pass เราแตก plan ออกเป็น plan_footing / plan_column / plan_beam / plan_slab แต่พอเน้นข้อมูลจริงกลับพบว่า plan_column มีแค่ 2 ไฟล์จาก 11 หลัง เพราะเราแทบไม่เคยมีแผนแปลนของตัวเอง มันไปโผล่เป็นมาร์กเกอร์บนแปลนคาน/ฐานราก และในตารางเสาแท่น ถ้าปล่อยให้เป็น subtask เดียวก็จะมีข้อมูลเทรนเลย

ต้นเหตุจริง: เราแตก subtask ตามหมวดหมู่ชิ้นส่วนโครงสร้างในตำรา ไม่ได้แตกจากการนับว่าจริง ๆ แล้วในชุดข้อมูลมีแผนแบบแต่ละแบบอยู่กี่แผ่น คือสร้างหมวดหมู่ก่อนนับของ

เสียหายอะไร: subtask ที่มีตัวอย่างเทรนแค่ 2 ตัวจะถูกเทรนและวัดผลราวกับว่าเป็นของจริง ได้โมเดลสาขาหนึ่งที่ใช้งานไม่ได้ กับตัวเลขวัดผลที่ไม่มีใครตีความได้ โชคดีที่จับได้เพราะนับก่อนเทรน

แก้ยังง: ยังไม่ตัดสินใจ บันทึกไว้เป็นเรื่องที่ต้องเคาะก่อนไปต่อ — จะยุบ plan_column เข้า plan_beam หรือย้ายไปอยู่ฝั่ง schedule และเขียนไว้ใน skill op04 ด้วยว่าเป็นกับดัก (plan_column ปกติไม่มีเป็นแผนแบบอยู่แล้ว การที่มันหายไปคือเรื่องปกติ ไม่ใช่การอ่านตก)

กฎที่ได้: นับไฟล์จริงของแต่ละหมวดก่อนตั้งมันเป็น subtask สำหรับเทรน — หมวดที่มีตัวอย่างต่ำกว่าราวๆ 20 ตัวคือเครื่องผลิตตัวเลขวัดผล ไม่ใช่โมเดล ผลลัพธ์ 13 ตัวอย่าง/recall 11% ก็พิสูจน์เพดานนี้ไปแล้ว

สูง 2026-08-21

annotate ไปแล้ว 11 หลัง แต่ t02 ถูกเทรนด้วยแค่ 4 หลัง — แรงงาน **annotate** ส่วนใหญ่ไม่เคยไปถึงโมเดล

เกิดอะไรขึ้น: พอไปนับ dataset จริงถึงรู้ว่า จาก 11 หลังที่ annotate ครบใน json_แก้ไขแล้ว/ (1,180 ไฟล์ / 4,625 element) รอบเทรนของ t02 ใช้แค่ 4 หลัง นี่คือเหตุผลที่มีแผนแปลนคาน 35 แผ่นอยู่บนดิสก์ แต่โมเดลเห็นแค่ราวๆ 13 แผ่น และเป็นเหตุผลตรงๆ ที่ recall ของแปลนคานออกมาที่ 11%

ต้นเหตุจริง: ชุดเทรนถูกประกอบจากบ้านที่ annotate ไว้เพียงบางส่วน และไม่มีใครกลับไปเช็ค coverage รายชนิดหน้าต่างก่อนและหลังรัน กว่าจะรู้ว่าขาดก็ตอนนับเพื่อประเมินขนาด t03 หลายสัปดาห์ให้หลัง

เสียหายอะไร: แรงงาน annotate ของคนราวๆ 7 หลัง ซึ่งเป็นต้นทุนที่แพงที่สุดของโปรเจกต์นี้ ไม่ได้สร้างมูลค่าให้การเทรนเลยสักนิด และโมเดลที่ออกมาก็มี recall แค่ 11% บนหน้าที่สำคัญที่สุด

แก้ยังง: ยังไม่ได้แก้ บันทึกไว้ใน tune_ai/t03/dataset_sizing.md ตั้งเป้าที่ 25-30 หลัง และรอบเทรนถัดไปต้องใช้ทุกอย่างที่ annotate ไว้

กฎที่ได้: ก่อนเทรน ให้นับจำนวนตัวอย่างรายชนิดหน้าต่างที่เข้าไปใน train.jsonl จริงๆ ไม่ใช่จำนวนบ้านที่ annotate ไว้ — annotation ที่ไม่ได้ถูกใช้ ไม่ต่างอะไรกับไม่เคย annotate เลย

สูง 2026-08-24

schema ผิดสะสม 267 จุดใน 246 ไฟล์ GT โดยไม่มีใครรู้

เกิดอะไรขึ้น: การไล่ normalize schema ทั้ง 11 หลังใน json_แก้ไขแล้ว พบว่าต้องแก้ 267 จุดใน 246 ไฟล์ — discipline architecture→architectural (157 จุด), คำนวณคลังคำ 6 จุด, รูปร่าง container ผิด (element→elements และ steel_members/void_bays/stair_details/reinforcement/sheet_index ทั้งหมดไปอยู่นอก elements), stirrup_or_tie→stirrup, rebar_cover_m→cover_mm

ต้นเหตุจริง: ไฟล์ GT ถูกผลิตโดย agent และคนรีวิวหลายคนข้ามเวลา โดยไม่มี validator ของ schema คอยเป็นด่านอยู่เลย พอ schema เปลี่ยน ไฟล์เก่าก็ไม่เคยถูกเอามาตรวจซ้ำ ความเพี้ยนจึงสะสมเจียบๆ

เสียหายอะไร: ทุกตัวที่รับข้อมูลต่อ (ตัวสร้าง dataset สำหรับเทรน, BOQ ของ Constistant) อ่านชื่อฟิลด์และรูปร่าง container ที่ไม่ตรงกัน และชุดเทรนก็พาความเพี้ยนนี้เข้าไปในโมเดลด้วย

แก้ยังไง: normalize ทั้ง 246 ไฟล์ แล้วตรวจซ้ำทั้ง 1,180 ไฟล์ด้วย json.loads บันทึกไว้ใน json_แก้ไขแล้ว/README.md หัวข้อ 2026-08-24

กฎที่ได้: ให้รัน check_format หรือการตรวจ schema เป็นด่านของทุก commit ของ GT ไม่ใช่ทำเป็นแคมเปญเก็บกวาดเป็นระยะ และเวลา schema เปลี่ยน ต้องกระตุ้นให้ตรวจซ้ำทั้ง repo ในวันนั้นเลย

สูง 2026-08-25

ไฟล์ JSON ที่ขาดกลางคัน พังพร้อมกันทั้งสองต้นไม้ข้อมูล

เกิดอะไรขึ้น: ไฟล์ 31.../หน้า25_plan_footing.json ขาดกลางไฟล์และ parse ไม่ได้ ทั้งใน json_แก้ไขแล้ว/ และ rawjson_ยังไม่ได้แก้ไขโดนคน/

ต้นเหตุจริง: การเขียนไฟล์ถูกขัดจังหวะหรือบันทึกออกมาไม่ครบ แล้วความเสียหายก็ลามไปทั้งสองต้นไม้ เพราะต้นไม้ที่แก้แล้วถูกสร้างต่อจากต้นไม้ดิบโดยไม่มีการเช็ค parse ตรงจุดคัดลอก

เสียหายอะไร: แปลนฐานรากหนึ่งหน้าใช้ไม่ได้ทั้งสองต้นไม้ และข้อมูล element F10 ในนั้นหายถาวร (ไม่มีต้นทางให้กู้ค่าที่ขาดกลับมา)

แก้ยังไง: ปิดไฟล์ให้ parse ผ่าน เก็บเรคคอร์ด F5 ที่สมบูรณ์ไว้ ทั้งเศษ F10 ที่ไม่ครบแทนที่จะเอาค่าที่ขาด แล้วแนบค่าเตือนไว้

กฎที่ได้: เช็ค parse ไฟล์ JSON ทุกไฟล์ตอนเขียนเสร็จ และเช็คซ้ำทุกจุดที่คัดลอกข้ามต้นไม้ พอเจอเรคคอร์ดที่ขาดและกู้ไม่ได้ ให้ทั้งพร้อมค่าเตือน อย่าเติมค่าที่หายด้วยการเดา

สูง 2026-08-26

ฐานราก footing-spread ของบ้าน 07 มีจำนวนเหล็กและความลึกผิดในไฟล์ท้องถิ่น — อะไรที่คำนวณต่อจากตัวเลขนี้ต้องเช็คใหม่

เกิดอะไรขึ้น: การ merge ไฟล์ footing-spread ของบ้าน 07 เผยว่าไฟล์ท้องถิ่นมีข้อมูลแบบจริงผิด: F0.80 นับจำนวน main_bar เป็น 8 ทั้งที่ควรเป็น 5, F1.00 กับ F1.50 มี `depth\_m` เป็น 1.5 ทั้งที่ควรเป็น 1.2 และ F1.00 ขาด block `main\_bar` ไปหิ้งก่อนตัวเลขของเพื่อนร่วมทีมถูกต้อง

ต้นเหตุจริง: สองคนอ่านแบบเดียวกันแต่ได้ตัวเลขต่างกัน โดยไม่มีขั้นตอนกระหนาบเลย ไม่มีอะไรแจ้งเตือนความไม่ตรงกันนี้จนกระทั่ง git บังคับให้เจอ

เสียหายอะไร: จำนวนเหล็กและความลึกฐานรากที่ผิดอยู่ใน ground truth มาโดยตลอด การคำนวณ BOQ/BBS ใดๆ ที่ใช้ตัวเลขท้องถิ่นเก่าจะผิดและต้องรันใหม่

แก้ยังไง: merge ตามกฎของมะขาม — ตัวเลขแบบจริงของเพื่อนร่วมทีมชนะ ส่วนโครงสร้าง/schema ของ JSON ของมะขามชนะ และแจ้งไว้ว่าผลการคำนวณข้างล่างต้องตรวจสอบใหม่

กฎที่ได้: เมื่อสองคนถอดแบบเดียวกัน ให้ตกลงกฎล่วงหน้าว่าตัวเลขของเจ้าของข้อมูลชนะ และ schema ของเจ้าของรูปแบบชนะ และเมื่อ ground truth เปลี่ยน ต้องระบุให้ชัดว่าผลลัพธ์ข้างล่างตัวไหนค้างเก่าแล้วบ้าง

สูง 2026-08-28

grid master 16 จาก 40 ไฟล์ไม่มีข้อมูลระดับแกน z เลย

เกิดอะไรขึ้น: มีแค่ 24 จาก 40 grid master ที่มีข้อมูลระดับแกน z การเติมข้อมูลที่ขาดต้องใช้ 3 วิธีกู้คืน คือ 6 หลังสร้างใหม่จาก element ที่มีอยู่ 2 หลังขุดจากข้อความบรรยาย และ 8 หลังต้องเปิดภาพแบบต้นฉบับมาอ่านด้วยมือ พร้อมตรวจสอบว่าหวั่งโซ่ z ปิดครบใน 9 หลัง

ต้นเหตุจริง: ฟิลด์ z_levels ถูกเพิ่มเข้าสเปกทีหลัง หลังจากที่หลายบ้านถูกสกัดข้อมูลไปแล้ว และไม่มีการรัน sweep ย้อนหลังเพื่อเติมข้อมูลให้ครบตอนที่สเปกเปลี่ยน ข้อมูลเดิมกระจัดกระจายอยู่ใน 4 รูปแบบการเก็บที่ทำกันเองแบบเฉพาะกิจ (คีย์ป้ายไทย, ข้อความบรรยาย "ระดับ... +X.XX", level_labels_as_printed, dict levels_m)

เสียหายอะไร: 40% ของบ้านไม่มีข้อมูลมิติแนวตั้งให้ใครใช้เลย การกู้คืนต้องอ่านภาพด้วยมือถึง 8 หลัง

แก้ยังไง: ตอนนี้ grid master ครบ 40/40 หลังแล้วมี z_levels โดยเก็บเกี่ยวจากข้อมูลรายหน้าที่ตรวจทานแล้วของแต่ละบ้านเอง และตรวจสอบหวั่งโซ่ z ว่าปิดครบใน 9 หลัง

กฎที่ได้: เมื่อเพิ่มฟิลด์ใหม่เข้าสเปก ต้องรันรายงานความครอบคลุมกับข้อมูลเดิมทั้งหมดทันทีและวางแผนเติมข้อมูลย้อนหลัง ฟิลด์ที่มีในสเปกแต่หายไป 40% ของข้อมูลจริง แยกว่าไม่มีฟิลด์นั้นเลยด้วยซ้ำ

โยนหน้าเข้าถึง `etc\_plan` มั่วซ้ำ 382 หน้า — 183 หน้าเป็นแบบโครงสร้างจริงที่จัดหมวดได้

เกิดอะไรขึ้น: `etc\_plan` เก็บไว้ 382 รายการ ใน 183 รายการนั้นเป็นแบบโครงสร้างที่จัดหมวดได้จริง (คาน 82, ฐานราก 57, โครงหลังคา 44) หลังจากแยกออกแล้ว ไม่มีหน้าแบบโครงสร้างเหลือค้างในถังนี้เลย

ต้นเหตุจริง: มีถังรวมที่รับได้ทุกอย่างโดยไม่มีการตรวจสอบเป็นประจำ agent ที่สกัดข้อมูลโยนอะไรก็ตามที่ไม่ชัดเจนเข้าถังนี้ พอไม่มีอะไรพัง ถึงกับวุ่นขึ้นเรื่อยๆ โดยไม่มีใครสังเกต

เสียหายอะไร: หน้าแบบโครงสร้าง 183 หน้าถูกซ่อนจากทุกคนที่กรองตามประเภทแบบเฉพาะ — รวมถึงตัวสร้างชุดข้อมูลเทรนที่เลือกตาม subtask ด้วย

แก้ยังง: แยก `etc\_plan` จาก 382 เหลือ 199 รายการ โดยดันขึ้นเป็นคาน/ฐานราก/โครงหลังคา 183 รายการ ตอนนี้ไม่มีหน้าโครงสร้างตกค้างเลย

กฎที่ได้: ต้องตรวจถังรวมมั่วเป็นประจำตามตารางเวลา และมองว่าถ้าถังโตขึ้นเรื่อยๆ นั่นคือสัญญาณว่าระบบจัดหมวดพัง ไม่ใช่แค่หมวดหนึ่งที่มีอยู่ปกติ ถึงชื่อ "etc" ที่มีหลายร้อยรายการ คือความล้มเหลวในการจัดหมวด

การพึ่งพากฎใน prompt อย่างเดียวเพื่อกันโมเดลลบ element ถูกปฏิเสธเพราะไม่น่าเชื่อถือพอ

เกิดอะไรขึ้น: prompt ของ pass 3 ระบุว่าโมเดลห้ามลบ element ออกจากรายการที่ยืนยันแล้วเด็ดขาด ทีมไม่เชื่อว่า prompt จะบังคับใช้กฎนี้ได้จริง จึงเขียน tools/merge_guard.py เพื่อใส่ element ที่หายไปกลับเข้ามาเป็น stub ที่ติดธงไว้

ต้นเหตุจริง: คำสั่งใน prompt เป็นการร้องขอเชิงความน่าจะเป็น ไม่ใช่ข้อบังคับ กฎที่บังคับด้วยภาษารธรรมชาติอย่างเดียวจะถูกละเมิดในอัตราหนึ่งเสมอ และมักละเมิดแบบเสียบๆ

เสียหายอะไร: ป้องกันได้ — แต่ถ้าไม่มี merge_guard.py element จะหายไปจากรายการโดยไม่มีร่องรอยเลย เหมือนกับกรณีคานหลังคาหายไป pass 0 ก่อนหน้านี้

แก้ยังง: tools/merge_guard.py บังคับใช้กฎความสมบูรณ์ของข้อมูลในโค้ด และติดธง stub ทุกตัวที่ถูกใส่กลับเข้ามาให้คนตรวจสอบ

กฎที่ได้: ห้ามพึ่งคำสั่งใน prompt อย่างเดียวเพื่อรักษาความสมบูรณ์ของข้อมูลเด็ดขาด ต้องบังคับใช้ในโค้ดหลังจากโมเดลตอบกลับมา และติดธงทุกครั้งที่มีการละเมิด ไม่ใช่แอบซ่อมเสียบๆ

สูง 2026-08-29

กฎ `scaled_from_grid` ใหม่ไม่มีตัวอย่างบวกเลยแม้แต่ตัวเดียวใน GT — โมเดลถูกสอนด้วยข้อความ `prompt` ล้วนๆ

เกิดอะไรขึ้น: การนับ GT เพื่อตัดสินใจว่าจะสอนกฎการวัด `span` ใหม่ได้อย่างไร พบว่ามีเคส `unresolved` 113 เคสจาก 20 ไฟล์ และมี `pos_m` `null` แค่ 2 เคส — หมายความว่าไม่มีตัวอย่างค่า `'scaled_from_grid'` ที่วัดถูกต้องเลยแม้แต่เคสเดียวให้เทรน

ต้นเหตุจริง: กฎนี้ถูกคิดขึ้นหลังจาก GT ถูกเขียนไปแล้ว ไม่มีการย้อนกลับไปทำ `pass` แก่ไขเพื่อสร้างตัวอย่างของมัน และคำตอบ (unresolved) ก็มียูเอเออยู่แล้วในข้อมูล

เสียหายอะไร: การ `fine-tune` ไม่สามารถสอนพฤติกรรมที่ไม่เคยเห็นตัวอย่างเลยได้ โมเดลถูกขอให้เรียนรู้ขั้นตอนการวัดจากคำสั่งเท่านั้น ในระบบ `supervised-imitation` ที่ให้รางวัลแค่การเลียนแบบรูปแบบผลลัพธ์ — ซึ่งเป็นจุดอ่อนที่งาน `prompt` เองก็ระบุไว้แล้ว

แก้ยังไง: เลือกใช้การสอนผ่าน `prompt` เพียงอย่างเดียว บวกกับข้อบังคับใส่ตัวเลขพิกเซล P/U ในแฟล็ก และบันทึกทางสำรองไว้ (ให้โค้ด CV วัดแทน หรือให้ intern เพิ่มข้อมูล 113 เคสลง GT)

กฎที่ได้: ก่อนปล่อยค่าผลลัพธ์ใหม่ ต้องนับก่อนว่ามีตัวอย่างจริงใน GT กี่ตัว ถ้าเป็นศูนย์ แปลว่ากำลังพึ่งพาการทำตามคำสั่งของโมเดลที่ถูกเทรนมาให้เลียนแบบเท่านั้น และรอบ GPU ต้องถูกมองว่าเป็นการทดสอบการพนันครั้งนี้ ไม่ใช่การทดสอบว่ากฎถูกหรือผิด

สูง 2026-08-30

Supabase service key รั่วเข้าไปในบทสนทนาผ่านการ `diff` ไฟล์อัตโนมัติ

เกิดอะไรขึ้น: ระหว่างแก้ไฟล์ `'worker_config.json'` ค่า Supabase service key โผล่ขึ้นมาในแชท เพราะระบบส่ง `file diff` เข้าไปให้ assistant โดยอัตโนมัติ

ต้นเหตุจริง: `secret` ที่ยังใช้งานอยู่จริงถูกเก็บไว้ในไฟล์ที่เครื่องมือ `diff` และแสดงเข้าไปในบทสนทนาโดยอัตโนมัติ — เลือกที่เก็บ `secret` โดยไม่ได้คำนึงถึงพฤติกรรม `diff` อัตโนมัติของเครื่องมือ

เสียหายอะไร: ตอนนี้มี service key ที่มีสิทธิ์เต็มปรากฏอยู่ในบทสนทนาแล้ว ยังใช้ทดสอบได้อยู่ แต่ต้อง `revoke` หักให้เร็วที่สุด

แก้ยังไง: ยังไม่ได้แก้ — ตั้งธงไว้ว่าต้อง `revoke` แล้วออก key ใหม่หลังงานนี้เสร็จ

กฎที่ได้: อย่าเก็บ `secret` ที่ใช้งานจริงไว้ในไฟล์ที่ agent tooling จะอ่านหรือ `diff` ได้เด็ดขาด — เก็บไว้ใน `env var` หรือไฟล์ที่ไม่ถูก `track` และสั่งไม่ให้ agent เปิดอ่าน หาก key หลุดเข้าบทสนทนาแม้แต่ครั้งเดียวต้อง `rotate` ทันที

สูง 2026-08-30

วัดจริงพบว่า CV template ครอบคลุมแค่ 23 จาก 44 หลังพอดี — 19 หลังตรวจจับไม่ครบ 2 หลังเจอ false positive และคานไม่มี template เลยสักตัว

เกิดอะไรขึ้น: การสำรวจ library จริงโดยตรง (วัดจริง ไม่ได้อ่านจากรายงานเก่า) พบว่ามี 3,149 หน้า แต่ template bank มีแค่ 17 template (footing 14, column 3) และ harvest_report ครอบคลุม 'พอดี 23 / ขาด 19 / เกิน(FP) 2 หลัง' คานไม่มี template bank เลยสักตัว — การตรวจจับคานใช้วิธีสังเคราะห์เป็นคู่เส้นจาก BEAM_GAP แทน ดังนั้นชั้น CV ที่แผน t05 ตั้งใจจะป้อนให้โมเดลเป็น input ระดับ production เองก็ผิดในเกือบครึ่งหนึ่งของบ้านทั้งหมด และอ่อนที่สุดตรง element เดียวกับที่โมเดลได้คะแนน 0% อยู่แล้วพอดี (plan_beam)

ต้นเหตุจริง: template bank ถูกสร้างขึ้นแบบฉวยโอกาสจากอะไรที่ harvest ได้ง่าย ๆ แทนที่จะขับเคลื่อนให้ครอบคลุมทุกหลังจริงๆ คานถูกจัดการด้วย heuristic แทน template เพราะรูปทรงของมันไม่เข้ากับวิธี icon-template ความครอบคลุมไม่เคยถูกใช้เป็นเงื่อนไขปล่อยงานเลย

เสียหายอะไร: ดาต้าเซต pass2.5/pass3 ที่สร้างจาก bank นี้จะสืบทอด 19 หลังที่ตรวจจับไม่ครบและ 2 หลังที่ false positive มาด้วย ถ้าเทรนจากข้อมูลนี้ก็เท่ากับสอนความผิดพลาดพวกนี้เข้าไปในโมเดลเลย

แก้งัง: ยังไม่ได้แก้ สร้าง `tools/review\_staging.py` ขึ้นมาเพื่อเร่งการรื้อด้วยคนสำหรับ candidate 146 ตัวที่ค้างอยู่ ซึ่งเป็นเส้นทางไปสู่ความครอบคลุมที่ดีขึ้น แต่ตัวเลขความครอบคลุมเองยังไม่ดีขึ้น และช่องโหว่เรื่องคานยังไม่ได้แก้

กฎที่ได้: ก่อนจะยกระดับชั้น CV ขึ้นเป็น 'input ระดับ production' สำหรับข้อมูล training ต้องวัดความครอบคลุมต่อหลังก่อน และถือว่าการตรวจจับไม่ครบกับ false positive เป็นตัวล็อกงาน — ตัวตรวจจับที่ผิดในครึ่งหนึ่งของชุดข้อมูลจะสอนความผิดนั้นเข้าไปในโมเดลที่เทรนจากมันแน่นอน

สูง 2026-08-31

path รูปภาพในดาต้าเซตชี้ไปที่โฟลเดอร์เบนที่มีอยู่แคบนเครื่องเทรนเก่าเท่านั้น

เกิดอะไรขึ้น: แถวข้อมูลของ pass1 กับ pass2.4 อ้างอิง images/<file>.png ซึ่งเป็นโฟลเดอร์แบบ flatten ที่มีอยู่แคบนเครื่องเช่าของ t04 ไม่มีอยู่ในตัว repo เลย

ต้นเหตุจริง: path ถูก bake ไว้ตอนสร้างดาต้าเซตโดยอ้างอิงตาม layout ของเครื่องชั่วคราว แทนที่จะอ้างอิงตาม repo root

เสียหายอะไร: ถ้าไม่จับได้จะหารูปไม่เจอเลยสักไฟล์บนเครื่องเช่าใหม่ — จับได้ทันบน CPU ฟรีก่อนจะเช่าเครื่องจริง

แก้งัง: เขียน path ใหม่เป็น image/<house>/<file>.png อ้างอิงตาม repo root

กฎที่ได้: path ของดาต้าเซตทุกตัวต้องอ้างอิงตาม repo root ที่มั่นคง ห้ามอ้างอิงตาม layout ของเครื่องเช่าเด็ดขาด — ต้องตรวจสอบด้วยการเปิดไฟล์ทุกไฟล์ที่ถูกอ้างถึงจริงๆ ก่อนจะเช่าฮาร์ดแวร์

สูง 2026-08-31

โมเดลพ่นค่าตัวเลขติดลบและ key ของ JSON ที่เพี้ยน ไหลเข้าไปถึงผลลัพธ์จริง

เกิดอะไรขึ้น: ผลลัพธ์จริงมีค่า height_mm:-500, spacing_mm:-200 และ key ที่เพี้ยนอย่าง "height_mm:-500,": ":null," กับ width-mm รวมถึงคานที่ถูกติดป้ายเป็นเสา

ต้นเหตุจริง: การ generate JSON แบบอิสระโดยไม่มีการบังคับ grammar ใดๆ ทั้งกับชื่อ field และช่วงค่าที่ยอมรับได้

เสียหายอะไร: ค่าที่เป็นไปไม่ได้ทางกายภาพอาจไหลเข้าไปในขั้นตอนถอดปริมาณงานและการตั้งราคาต่อได้

แก้งัง: เพิ่มฟังก์ชัน sanitize_elements() ใน worker.py ให้ทั้ง element ที่มีค่าตัวเลขติดลบ หรือ key ที่มี ':' หรือ ';' ปนอยู่โดยไม่พยายามซ่อมหรือเดาเลย และบันทึกจำนวนที่ถูกทิ้งไว้ใน warnings เสมอ ทดสอบกับไฟล์จริงแล้วทิ้งถูก 2 จาก 4 รายการ

กฎที่ได้: ต้องตรวจสอบความเป็นไปได้ทางกายภาพของผลลัพธ์โมเดล ณ จุดที่จะนำไปใช้จริง แล้วทิ้งทันทีแทนที่จะพยายามซ่อมพร้อมบันทึกทุกรายการที่ถูกทิ้งเพื่อให้ข้อมูลหายไปแบบเงียบๆ

สูง 2026-09-01

guessItemMaterialKey รู้จักแค่ 3 จาก 7 ประเภทแบบหล่อ ทำให้บรรทัดราคาแบบหล่อจับคู่ไม่ได้เลย

เกิดอะไรขึ้น: `guessItemMaterialKey` รู้จักแค่ 3 จาก 7 ประเภทแบบหล่อที่มีอยู่จริงในตารางราคา 'แบบหล่อ ฐานราก' (footing formwork) จับคู่ไม่ได้เลย ทั้งที่ `formwork.footing` มีราคาจริงรออยู่ที่ \$250/m²

ต้นเหตุจริง: มีสองรายการที่ต้องตรงกัน — คำศัพท์ที่ตัวเดาคีย์รู้จัก กับคีย์ในตารางราคา — แต่ดูแลแยกกันจนค่อยๆ เพี้ยนไม่ตรงกัน

เสียหายอะไร: ค่าแบบหล่อหายไปเสียๆ จาก BOQ ทั้งที่มีราคาจริงรออยู่แล้วในตาราง

แก้ยังไง: รวมให้ใช้ตารางเดียวกัน พร้อมเพิ่มเทสต์ที่ยืนยันว่าทุกคีย์ที่ตัวเดาคืนกลับมาได้ ต้องมีราคาจริงอยู่ในตาราง กันไม่ให้สองรายการนี้แยกทางกันได้อีก

กฎที่ได้: เมื่อมีสองรายการที่ต้องตรงกันเสมอ ให้สร้างรายการหนึ่งจากอีกรายการ หรือใส่เทสต์ที่ตรึงว่าทั้งคู่ต้องตรงกัน อย่าดูแลคำศัพท์อ้างอิง (lookup vocabulary) แยกจากตารางข้อมูลด้วยมือ

สูง 2026-09-01

มีแค่ ~10 จาก 40 หลังเท่านั้นที่มี annotation pass0/2.4/3 — pass0 มีแค่ 5 หลัง จนวัดผล pass ใหม่ๆ ไม่ได้เลยไม่ว่าจะแบ่งชุดข้อมูลยังไง

เกิดอะไรขึ้น: ตอนตรวจสอบ fold ของ t04 พบว่าปัญหาไม่ได้มีแค่การแบ่งชุดข้อมูลแบบตาบอด แต่ชุดข้อมูลเองก็ขาดด้วย — มีแค่ ~10 จาก 40 หลังที่มีข้อมูล pass0/2.4/3 และ pass0 มีแค่ 5 หลังเท่านั้น ทุก fold มีไฟล์ eval ของ pass1 อยู่ 204 ไฟล์ แต่ไฟล์ pass0 มี 0 ไฟล์

ต้นเหตุจริง: ทีมโฟกัสทำ annotation pass1 (ซึ่งพร้อมกว่า) ในขณะที่ pass0/2.4/3 ยังอยู่ระหว่างออกแบบ ไม่มีใครนับจำนวนหลังต่อ pass ก่อนจะสัญญาว่าจะประเมินผลระดับ fold ของ pass เหล่านั้น 5 หลังไม่สามารถแบ่งเป็น 4–5 fold แล้วยังเหลือหลังสำหรับ eval ได้เลย

เสียหายอะไร: ต่อให้แบ่งชุดข้อมูลได้สมบูรณ์แบบแค่ไหน ก็วัดไม่ได้ว่าโมเดลเรียนรู้ pass0 หรือ pass3 จริงหรือไม่ — ความสามารถที่กำลังสร้างอยู่วัดผลไม่ได้เลยด้วยข้อมูลตอนนี้ เทรนไปเท่าไรก็จบด้วย 'ยังไม่รู้ยุดีว่าใช้ได้ไหม'

แก้ยังไง: ยังไม่ได้แก้ — บันทึกจำนวนและข้อสรุปไว้ ('ถ้าใช้ fold เดิมต่อ = เทรนไปแล้วก็ยังไม่รู้ยุดี') แต่ไม่มีบันทึกว่าเติม annotation เพิ่ม

กฎที่ได้: ก่อนออกแบบวิธีแบ่งชุดข้อมูล ต้องนับจำนวนตัวอย่างต่อความสามารถที่จะวัด ไม่ใช่นับต่อหลัง ความสามารถที่มีตัวอย่างแค่ 5 ตัว ไม่มีทางมีชุด eval ที่ใช้ได้เลยไม่ว่า k เท่าไหร่ — ทางแก้คือเพิ่ม annotation ไม่ใช่ทำตัวแบ่งชุดข้อมูลให้ดีขึ้น

pass0 จำแนกอัตโนมัติได้แค่ 31 จาก 72 หน้าจริง — สัญญาณเทรนของ subtask ผังพื้นมีแค่หยิบมือเดียว

เกิดอะไรขึ้น: การจำลองบ้านหลังที่สองแบบตรงไปตรงมา (ขับด้วย pass0_labels.jsonl จริงแทนที่จะเป็น mapping ที่แปลงมา) แสดงว่า pass0 จำแนกหน้าได้อัตโนมัติแค่ 31/72 หน้า บ้อนเข้าไปเป็นงาน pass2 จริงแค่ 24 งาน พอเช็คไฟล์เทรนจริงก็เข้าใจสาเหตุทันที — จาก 171 แยกทั้งระบบมี plan_beam 14, plan_footing 7, also_gridline 32

ต้นเหตุจริง: ชุดข้อมูลเทรนของ pass0 ไม่เคยถูกจัดสมดุลงต่อ subtask เลย — คลาสหน้าแปลนที่สำคัญที่สุดสำหรับ BOQ มีตัวอย่างแค่หลักเดียวถึงหลักสิบต้นๆ ทำให้กว่าครึ่งของบ้านจริงหลังหนึ่งต้องตกไปอยู่ในมือให้คนทำเอง

เสียหายอะไร: มากกว่าครึ่งของทุกหลังต้องให้คนติดป้ายกำกับก่อนที่จะเริ่มถอดข้อมูลได้เลยด้วยซ้ำ — อัตราการทำงานอัตโนมัติที่เคยรายงานจากการจำลองก่อนหน้านี้สูงเกินจริง เพราะการจำลองนั้นเอา label ของ pass0 มาจากชื่อไฟล์ ไม่ได้มาจากความสามารถจริงของโมเดล

แก้ยังไง: ยังไม่ได้แก้ — วัดผลและบันทึกไว้ ต้องการข้อมูลเทรน pass0 เพิ่มต่อแต่ละ subtask

กฎที่ได้: ต้องวัด coverage ของ classifier บนบ้านทั้งหลังที่ไม่เคยเห็นมาก่อน ไม่ใช่วัดบนหน้าที่มันเคยเทรนมาแล้ว และต้องนับจำนวนตัวอย่างเทรนต่อคลาสก่อนจะประกาศอัตราการทำงานอัตโนมัติ

เรื่องที่เหลือในหมวดนี้ (72)

เอกสารเปรียบเทียบผลนารองเขียนไว้ชัดเจนว่าเป็นกราฟที่ยังไม่ตรวจ แต่ไปวางปนกับผลจริงจนคนเข้าใจผิดว่าเป็นข้อมูลอ้างอิงได้

กลาง 2026-07-02

สถานะ 'กราฟต์/ยังไม่ตรวจ' ต้องอยู่ในชื่อไฟล์หรือตำแหน่งโพลเดอร์ ไม่ใช่แค่เขียนไว้ในหัวเอกสาร อะไรที่ยังไม่ผ่านการตรวจสอบแยกไต่แรกหรือออกจากผลงานที่งานถัดไปอนุญาตให้เชื่อถือได้

จำนวนประตู/หน้าต่างในหน้า 06 เอามาจากตาราง schedule แทนที่จะนับจากผังจริง

กลาง 2026-07-05

ถ้าค่าที่ได้มาจากแหล่งอื่นที่ไม่ตรงกับนิยามของฟิลด์นั้น ให้บันทึกแหล่งที่มาไว้ในฟิลด์ provenance (แบบเดียวกับที่ span_source ทำกับช่วงคำ) ไม่ใช่แค่เขียนบรรยายไว้เฉยๆ เพราะการสลับแหล่งข้อมูลแบบเสียบๆ จะไม่ต่างจากการอ่านที่ถูกต้องเลย

การสกัด Gen 3 รอบใหม่ทำผิดเรื่อง DB→RB ซ้ำอีก และผิดทุก element ในหน้า 21

กลาง 2026-07-06

จุดที่โมเดลพลาดซ้ำๆ ไม่ใช่ปัญหาของโมเดลอย่างเดียว ให้เขียนกฎที่ต้องลงใน spec การสกัดข้อมูลไปเลย เพราะคนหรือ agent ที่เข้ามาอ่านสตร ก็จะมีมานับแบบเดียวกัน ให้เขียนการ map สัญลักษณ์→ชนิดหลักเป็นกฎ และห้ามให้ขนาดเส้นผ่านศูนย์กลางมาบอกชนิดหลักเด็ดขาด

ไฟล์ stub ขนาด 0 ไบตันนอนอยู่ในรีโป Training ทั้งวัน โดยทำทำเป็นเอกสาร schema

กลาง 2026-07-07

หลังย้ายไฟล์ข้ามรีโปทุกครั้ง ให้เช็คไฟล์ปลายทางมีขนาดไม่เป็นศูนย์และตรงกับต้นทาง ก่อนจะลบต้นฉบับทิ้ง

แบบ BOQ ต้นฉบับมีเลขที่ผิดจริงๆ และการอ่านแยกกันสองรอบก็ให้ผลผิดเหมือนกันทั้งคู่

กลาง 2026-07-08

บันทึกข้อผิดพลาดของเอกสารต้นฉบับไว้เป็น flag ในแถวนั้นให้ชัด เพื่อไม่ให้คนอ่านคนถัดไปมาไล่สืบซ้ำ สิ่งผิดปกติที่ยืนยันได้สองรอบคือข้อมูลเกี่ยวกับตัวแบบ ไม่ใช่บั๊กของคนอ่าน

แบบของบ้านหลังที่ 2 มีข้อผิดพลาดจริงหลายจุด ซึ่ง agent เลือกที่จะติดธงไว้แทนการแอบแก้ไข

กลาง 2026-07-10

การติดธงเมื่อเจอข้อขัดแย้งถูกต้องเสมอ และการเสียๆ ปรับให้มันเข้ากันเองผิดเสมอ แต่ธงพวกนี้ต้องถูกรวบมาไว้ในคิวตรวจเดียว ไม่งั้นมันก็จมอยู่ในไฟล์ JSON เป็นร้อยไฟล์

แถว raw_json_data_log ของบ้านหลังที่ 2 ถูกลิ้ม แล้วต้องมาเติมย้อนหลัง

กลาง 2026-07-11

บันทึกที่มาของข้อมูลตอนสร้างข้อมูล ไม่ใช่ตอนทำเสร็จ ชุดข้อมูลที่ยังไม่สมบูรณ์ยังต้องมีแถว log มากกว่าชุดที่เสร็จแล้วด้วยซ้ำ แค่ทำเครื่องหมายไว้ว่ายังไม่สมบูรณ์

แบบของบ้านหลังที่ 3 มีบันไดติดป้ายผิด ชั้นติดป้ายผิด ลูกตั้งไม่ตรงกัน และสเปกแอร์หายไป

กลาง 2026-07-11

ตรวจไขว้ป้ายหรือขนาดกับแบบอีกแผ่นก่อนจะติดธง เพื่อให้ธงนั้นแยกออกว่า 'แบบผิด' หรือ 'เราอ่านผิด' และให้บันทึกสเปกที่ขาดหายไปเป็นช่องว่างที่ชัดเจน อย่าเดาค่าใส่ลงไปเด็ดขาด

ระดับชั้นที่ติดป้ายผิดในผังทอสุขภาพิบาลแนวตั้ง เกิดซ้ำทุกบ้าน และยังได้แค่ติดธงไว้ ไม่เคยแก้ไขจบ

กลาง 2026-07-12

ข้อบกพร่องที่โผล่ในเอกสารต้นทางของบ้านสามหลังติดกัน ต้องมีคำตัดสินระดับโปรเจกต์บันทึกไว้ใน spec หนึ่งครั้ง ไม่ใช่ติดธงไล่ทีละบ้าน

กริดหลอก A' ของบ้านหลังที่ 5 มีค่าระยะยื่นขัดกันสามค่าจากแบบโครงสร้างสามแผ่น และค้างอยู่ที่ค่าความมั่นใจ 0.65

กลาง 2026-07-13

เวลาแบบโครงสร้างขัดกันเองเรื่องระยะ คำตอบมักอยู่บนแบบสถาปัตย์หรือแบบงานระบบ ให้ขยายการค้นออกไปนอกสาขาก่อนจะติดธงว่าหาข้อสรุปไม่ได้

แบบคานและคานคอดินของบ้านหลังที่ 2 ขาดคานบางตัวไป ซึ่งเพิ่งโผล่มาตอนไล่ dummy-grid cascade

กลาง 2026-07-19

แบบคานต้องมีขั้นตอนตรวจสอบความครบถ้วน (เช็กทุกจุดตัดของกริดว่ามีคานครบไหม) ไม่ใช่แค่อ่าน label เฉยๆ การไล่ dummy-grid cascade บังเอิญหาหน้าทีนั้นได้ — ควรทำให้เป็นขั้นตอนที่ตั้งใจทำจริง

คำเตือนถูกทิ้งค้างไว้ในไฟล์หลังจากปัญหาที่มันเตือนถูกแก้ไปแล้ว แกรมมีคำเตือนหนึ่งที่ตัวมันเองผิดข้อเท็จจริงด้วยซ้ำ

กลาง 2026-07-19

คำเตือนคือข้อมูลที่เสื่อมอายุได้ ทุกครั้งที่แก้ปัญหามันต้องปิดคำเตือนที่เกี่ยวข้องด้วย และคำเตือนที่ยืนยันสิ่งที่ 'ไม่เจอ' (negative claim) ต้องมีหลักฐานรองรับเชิงมวดเท่ากับการยืนยันว่าเจออะไรบ้าง

คานตัวหนึ่งถูกบันทึกระยะโดยเดาจากขอบกล่องในภาพ แทนที่จะใช้ dimension ที่พิมพ์ไว้จริง

กลาง 2026-07-20

ฟิลด์ dimension ต้องใส่เฉพาะค่าที่พิมพ์ไว้บนแบบเท่านั้น ค่าที่วัดจากภาพเองต้องมี flag บอกที่มาอย่างชัดเจน ไม่งั้นห้ามบันทึกเลย

มีการบันทึกคานที่ไม่มีอยู่จริง ต้องลบตามคำสั่ง — เกิดขึ้นทั้งในบ้านหลังที่ 3 และหลังที่ 5

กลาง 2026-07-20

การทำนายเกินจริง (over-prediction) เป็นความผิดพลาดที่เกิดขึ้นได้จริงแม้ในข้อมูลที่คนแก้แล้วก็ตาม ไม่ใช่แค่ในโมเดลที่ fine-tune เส้นที่ไม่มีมาร์คและไม่มี dimension พิมพ์กำกับ ไม่ควรถือเป็นคานโดยปริยาย

แบบคานของบ้านหลังที่ 2 มีสเปคที่ขัดแย้งกับแผ่น section ที่มันควรจะอ้างอิงมา

กลาง 2026-07-20

การแยกสเปคที่ซ้ำกันออกมาเป็น specs{} แก้ปัญหาความซ้ำภายในไฟล์เดียวได้ แต่กลับสร้างความซ้ำข้ามไฟล์แทน แผ่น section ต้องเป็นแหล่งสเปคเดียวเท่านั้น แผ่นแบบคานควรอ้างอิงกลับไป ไม่ใช่ก๊อปปี้มาเอง

ห้องสองห้องได้จัดมุมชดเดี๋ยกันเป๊ะ แต่ชื่อคนละชื่อ

กลาง 2026-07-29

ใส่ตัวตรวจง่าย ๆ หลังสกดข้อมูล ว่าห้ามมีสองชั้นที่มีชุดพิกัดเหมือนกันเป๊ะ ข้อมูลที่ขัดกันเองเป็นความผิดพลาดที่ตรวจด้วยเครื่องง่ายที่สุด แต่ใช้ตาอ่านหายากที่สุด

ชื่อห้องภาษาไทยของจริงไปกองอยู่ในช่อง notes ส่วน element_id กลับตัวเองเป็น F1-F11

กลาง 2026-07-29

เวลาผลลัพธ์ออกมาผิด ให้เช็คก่อนว่าค่าที่ถูกไปโผล่ในช่องผิดหรือเปล่า ก่อนจะสรุปว่าโมเดลอ่านไม่ออก ใส่ผิดช่องกับอ่านผิด ต้องแก้คนละแบบเลย

'1 of 15' ในกรอบชื่อแบบ ถูกอ่านเป็นเลขหน้า 15 และเกิดขึ้นถึงสองหน้า

กลาง 2026-07-29

ตัวเลขแบบ X of Y ต้องแยกอ่านให้ชัด อย่าคว้าวตัวเลขที่อยู่ใกล้ๆ มาใช้ และให้ตรวจว่าเลขหน้าห้ามซ้ำและต้องอยู่ในช่วง [1, จำนวนหน้าทั้งหมด] เลขหน้าซ้ำคือสัญญาณผิดพลาดที่ตรวจได้ฟรีและแน่นอน

สกดฝั่งบริเวณออกมาโดยไม่มีระยะสกดตัวเดียว ได้ output ที่ไม่มีเนื้อหา แต่ยังดูเหมือนใช้ได้

กลาง 2026-07-29

กำหนดช่องที่ต้องมีสำหรับแต่ละ pattern (ฝั่งบริเวณต้องมีระยะ ตารางต้องมีแถว) แล้วให้หน้าไหนที่ขาดถือว่าไม่ผ่าน 'JSON' ถูกรูปแบบ' กับ 'มีคำตอบอยู่ข้างใน' เป็นเกณฑ์คนละเรื่องกัน

คำศัพท์งานก่อสร้างภาษาไทยถูกสะกดเพี้ยนจนกลายเป็นคำที่ไม่มีอยู่จริง

กลาง 2026-07-29

ตรวจคำศัพท์เฉพาะทางที่สกดมาเทียบกับรายการคำที่กำหนดไว้ แล้วติดธงคำที่ใกล้เคียงแต่ไม่ตรง คำเฉพาะทางที่สะกดเพี้ยนเป็นสัญญาณเตือนราคาสูงกว่าภาพที่ป้อนเข้าไปมีปัญหา ให้ตามไปหาสาเหตุ อย่าคิดว่าเป็นแค่เรื่องความสวยงาม

รายการหน้าใน manifest เดาชวงหน้าฝั่งบริเวณผิด (บอก 3 หน้า จริงๆ มีหน้าเดียว)

กลาง 2026-07-30

อย่าเชื่อชวงหน้าที่ manifest บอก ให้เปิดดูทุกหน้าแล้วจำแนกจากเนื้อหาบนหน้านั้นเอง ก่อนสกด ขอบเขตของหมวดพิสูจน์จากสิ่งที่พิมพ์อยู่บนหน้า ไม่ใช่จากการที่หน้ามันติดกัน

ตาราง BOQ ต้องอ่านด้วยมือที่ความมั่นใจ 0.35-0.65 เพราะตารางหมุน 90 องศาและตัวหนังสือเล็กมาก

กลาง 2026-07-30

เช็คทิศทางการกับขนาดตัวอักษรก่อนสกด ถ้าจำเป็นให้หมุนหรือขยายก่อน หรือส่งไปให้ตัวสกดที่ทำมาเพื่อการนี้โดยเฉพาะ และค่าความมั่นใจต่ำต้องบันทึกเป็นรายแถว ห้ามเอามาเฉลี่ยจนหายไป

ตารางราคาก่อสร้าง 10 บอกว่าฐานรากเสาเข็มถูกกว่าฐานรากแผ่ ซึ่งกลับด้านกับบ้านทุกหลัง

กลาง 2026-07-30

ถ้าค่าไหนขัดกับรูปแบบที่เห็นชัดจากบ้านหลังอื่นๆ ให้ติดธงแล้วส่งต่อให้คนตัดสินใจ ห้าม 'แก้' ให้ตรงกับเสียงส่วนใหญ่เสียๆ การเขียนทับข้อมูลผิดปกติที่เป็นของจริง แยกว่าปล่อยให้มันติดธงไว้

กฎที่ว่า 'รายการ BOQ ข้อ 2-5 เหมือนกันทุกระบบฐานราก' มีข้อยกเว้นจริง และจับได้เพราะยังอ่านแผ่นอยู่

กลาง 2026-07-30

'แผ่นพวกนี้เหมือนกันหมด' เป็นข้อสันนิษฐานที่ต้องพิสูจน์ทีละชุด ไม่ใช่ใบอนุญาตให้ข้ามการอ่าน ให้ diff โฟล์ด้วยเนื้อหาจริง อย่าสรุปว่าเหมือนกันจากรูปแบบที่เห็นในบ้านหลังสองหลัง

ตาราง BOQ ปร.4 ของบ้าน 11 อ่านได้ด้วยความมั่นใจ 0.45-0.55 แยกว่าตารางที่หมุนอยู่ซึ่งมันมาแทนเสียอีก

กลาง 2026-07-31

ตัดสินใจว่าหน้าอ่านง่ายไหมจากค่าความมั่นใจที่วัดจากตัวอย่างจริง ไม่ใช่จากทิศทางการหรือรูปแบบตาราง ให้ลองสกด 2-3 แผ่นแล้วดูความมั่นใจก่อน ค่อยตัดสินใจลุยทั้งชุด 29 แผ่น

กริดหลอก A'/D' ที่จับคู่กับฐานราก C0 ไม่มีโครงสร้างรองรับอยู่ที่ไหนเลยในชุดแบบ

กลาง 2026-07-31

กริดหรือมาร์คที่อธิบายไม่ได้ ให้ติดธงไว้ ห้ามลบ การลบทิ้งเท่ากับทำลายหลักฐานว่าต้นฉบับไม่ครบ ส่วนการเดาโครงสร้างให้มันเท่ากับปั่นข้อมูลขึ้นมาเอง

บ้าน 07 ใช้ค่า pattern 3 ตัวที่ไม่มีอยู่ใน schema (detail/elevation/diagram)

กลาง 2026-08-02

รายการค่าที่กำหนดไว้ตายตัว ต้องบังคับด้วยเครื่องตอนเขียนข้อมูล กฎที่มีอยู่แค่ในตัวแทนหนังสือ เดียวก็ถูกละเมิดด้วยค่าที่ฟังดูสมเหตุสมผลตัวแรกที่ผ่านมา

ตารางเสาหลายชั้นถูกเก็บสองแบบที่เข้ากันไม่ได้ — แบบ levels[] ซ้อนกัน กับแบบแถวแบนๆ

กลาง 2026-08-02

ตรงไหนที่ spec ระบุว่าขอบรูปแบบไหนมากกว่าระหว่างสองแบบที่ดูใช้ได้ทั้งคู่ ตัวตรวจต้องปฏิเสธแบบที่ไม่ได้เลือก ความชอบที่ไม่มี การบังคับ สุดท้ายจะได้มาทั้งสองแบบ

ตารางราคาชุดเดียวกันเป๊ะ ถูกเก็บไว้ 5 แบบใน 5 บ้าน

กลาง 2026-08-02

เนื้อหาที่ซ้ำกันข้ามบ้าน ต้องตัดสินใจครั้งเดียวว่ารูปแบบมาตรฐานคืออะไร ตรวจสอบหาหน้าเหมือนกันด้วย hash แล้วเอาผลสกัดเติมมาใช้ซ้ำ แทนที่จะมาจัดรูปแบบใหม่

บล็อก specs{} ใน schema มีข้อมูลแค่ 24 จาก 1,183 ไฟล์ และมีสองบ้านที่ไม่มีเลย

กลาง 2026-08-02

ช่องที่มีผลต่อการ join ปลายทาง ต้องมีตัวเลขความครอบคลุมโชว์ใน output ของตัวตรวจ ไม่งั้นความครอบคลุม 2% กับ 100% จะดูเหมือนกันเป๊ะจากบรรทัดผ่าน/ไม่ผ่าน

เหล็กเสริมที่พิมพ์กำกับว่า 'หยุดที่ L/8' หรือกลางช่วง ในบ้าน 10-11 ยังไม่ถูกรวมเข้าหน้าเหล็ก

กลาง 2026-08-02

รายการที่ต้องเปิดแบบดูถึงจะทำได้ ให้เข้าคิวไปอยู่ในรอกงานที่เปิดแบบจริงพร้อมระบุคนรับผิดชอบ ไม่ใช่เอามาใส่รายการค้างท้ายรอบจัดรูปแบบซ้ำๆ ซึ่งเป็นรอบที่ยังไงก็ทำไม่ได้

ไฟล์ BOQ 10 ไฟล์ของบ้าน 04 ถูกเก็บเป็น pattern=unknown พร้อม elements[] แทนที่จะเป็นรายการวัสดุ

กลาง 2026-08-02

'unknown' ที่ผ่านการตรวจ pattern ไปได้ คือช่องโหว่ ไม่ใช่การผ่าน ตัวตรวจควรติดธงไฟล์ที่เนื้อหาดูเหมือน pattern ที่รู้จัก แต่ถูก label ว่า unknown อย่างน้อยก็เป็นระดับ NOTE

ช่องที่ควรเป็นตัวเลขถูกเก็บเป็นสตริงบ้าง อาร์เรย์บ้างทั่วทั้งคลัง เฉพาะ level_m อย่างเดียวก็ ~170 ค่าแล้ว

กลาง 2026-08-02

ล็อกชนิดข้อมูลของทุกช่อง ไม่ใช่แค่ชื่อกับความหมาย spec ที่บอกว่าช่องนี้หมายถึงอะไร แต่ไม่บอกว่าต้องเก็บเป็นรูปแบบไหน จะเก็บค่าเดียวกันไว้ห้าแบบ

เพิ่ม pattern ใหม่เข้า schema สองตัว โดยไม่มีตัวอย่างจริงอยู่ในคลังเลยสักอัน

กลาง 2026-08-04

รายการใน schema ที่เพิ่มเข้ามาโดยไม่มีตัวอย่างจริง ให้มาร์คว่าเป็นฉบับร่าง แล้วปล่อยให้ตัวอย่างจริงตัวแรกเป็นตัวกำหนดรูปร่างอีกอย่างคือ การแก้ spec กับการแก้ตัวตรวจ ต้องอยู่ใน commit เดียวกัน ไม่งั้นตัวตรวจจะปฏิเสธ spec ใหม่

บ้านหลัง 14 ไม่มีรหัสแผนแบบและไม่มีหน้าสารบัญเลย sheet_code เป็น null ทั้งหลัง

กลาง 2026-08-07

กฎที่เป็นจริงมาตลอด N หลังจากผู้จัดทำเจ้าเดียว คือธรรมเนียมของเจ้านั้น ไม่ใช่กฎสากล พอเจอชุดแบบจากแหล่งใหม่ ต้องกลับไปตรวจสอบข้อสันนิษฐานเชิงโครงสร้างใหม่ทุกข้อ และถ้าของนั้นไม่มีจริงๆ ให้บันทึกเป็น null พร้อมคำเตือน ห้ามแต่งค่าขึ้นมาเอง

บ้านหลัง 14-18 ใช้ธรรมเนียม png/doc_page ของ grid master สลับกันกับหลัง 01-13

กลาง 2026-08-09

ธรรมเนียมที่ทำตามๆ กันมาโดยไม่มีการเขียนไว้ สุดท้ายจะกลับหัวกลับหางแน่นอน ให้เขียนลง spec แล้วเพิ่มกฎในตัวตรวจทันทีที่เห็นบ้านสองหลังทำไม่เหมือนกัน

มีสามฟิลด์ที่ใช้งานมาตั้งแต่บ้านหลัง 06/11 แต่ไม่เคยอยู่ใน spec เลย

กลาง 2026-08-09

เพิ่มรายงาน key ที่ไม่รู้ลักเข้าไปในตัวตรวจ ฟิลด์ที่โผล่ในข้อมูลจริงแต่ไม่มีใน spec คือ spec ตกหล่นหรือไม่ก็พิมพ์ผิด ทั้งสองแบบต้องเห็นทันที ไม่ใช่มารู้เอาตอนผ่านไปสามหลังแล้ว

หน้า26.json เขียน spec.rebar เป็น array ของ string ซึ่งผิด schema 50.6

กลาง 2026-08-18

ถ้าเนื้อหาในแบบไม่พอดีกับฟิลด์ที่มีโครงสร้าง ให้ย้ายไปใส่ฟิลด์ข้อความอิสระที่เตรียมไว้ อย่าไปคลาซชนิดข้อมูลของฟิลด์โครงสร้างให้หลวมลง และต้องรัน check_format.py กับทุกหลังก่อนบันทึกว่าเสร็จ

แปดหน้า BOQ ส่งออกไปเป็นแค่ stub อ้างอิง แทนที่จะเป็นการถอดข้อมูลจริง

กลาง 2026-08-18

การลดขั้นตอนจะยอมรับได้ก็ต่อเมื่อมันประกาศตัวเองในรูปแบบที่เครื่องอ่านได้ ให้เขียนเหตุผลลง warnings[] ในตัวไฟล์เลย ห้ามเขียนแค่ในไดอารี่ที่สคริปต์เทรนไม่มีวันอ่าน

มาร์คพื้น (SO/SI/ST) ไม่มี grid_ref เลยสักตัว พื้นจึงระบุตำแหน่งหรือวาดทับบนแบบไม่ได้เลย

กลาง 2026-08-20

'คาดไว้แล้ว' กับ 'ยอมรับได้' คนละเรื่องกัน ชนิดชั้นส่วนที่ไม่มีกลไกไหนหาตำแหน่งให้ได้เลย ควรถูกบันทึกเป็นช่องโหว่ ไม่ใช่ปล่อยผ่าน เพราะทุกคนที่ใช้ข้อมูลต่อจะรับช่องโหว่นี้ไปด้วยทั้งหมด

ฟิลด์ discipline มีค่าเดียวกันเขียนสองแบบ: architectural (182 ไฟล์) กับ architecture (157 ไฟล์)

กลาง 2026-08-21

ฟิลด์ที่เป็น enum ทุกตัวต้องมีชุดค่าที่ปิดตายไว้ใน spec และมี assert ใน check_format.py ว่าค่าต้องอยู่ในชุดนั้น ตั้งแต่วันแรก ฟิลด์หมวดหมู่ที่ปล่อยให้พิมพ์อิสระจะแตกออกเป็นหลายแบบเสมอ และมันจะแตกแบบเงิบๆ เพราะยังไม่มีใครเอาไปใช้จนจะมาบ่น

สีไฟล์บันทึก rebar_cover_m เป็นหน่วยเมตร ซึ่งผิดกฎ 50.5 ที่บอกให้ใช้ mm สำหรับขนาดชิ้นส่วน

กลาง 2026-08-21

บังคับเรื่องหน่วยใน check_format.py ด้วยช่วงค่าที่สมเหตุสมผลรายฟิลด์ ไม่ใช่เขียนไว้แค่ในเอกสาร เพราะความผิดพลาดเรื่องหน่วยเงิบโดยธรรมชาติ ทั้งสองค่าเป็นตัวเลขที่ถูกต้องทั้งคู่ และระยะหุ้มที่ผิดไป 1000 เท่าคือตัวเลขความปลอดภัยของโครงสร้าง

ชื่อไฟล์เพี้ยนกันไปหมด (beam_plan / beam_plan_floor1 / beam_floor_plan / beam_plan_สุขา) เลยใช้ชื่อไฟล์ตัดสินใจส่งงานไม่ได้อีกต่อไป

กลาง 2026-08-21

ห้ามให้ความหมายอยู่แค่ในชื่อไฟล์ ถ้าค่าไหนสำคัญ ให้มันไปอยู่ในฟิลด์ พอชื่อไฟล์เพี้ยนไปทั้ง dataset แล้ว เครื่องมือที่อาศัยชื่อไฟล์ก็ตายถาวร ต้องเปลี่ยนไปจำแนกจากเนื้อหาแทน

หน้าแปลน 8 หน้าให้ตัวอย่างเทรน 0 ชิ้น เพราะไม่มีชิ้นส่วนไหนในนั้นตรงกับ subtask ของตัวเองเลย

กลาง 2026-08-24

บรรทัดที่เขียนว่า 'ข้ามและรายงานแล้ว' คือสิ่งที่ต้องทำต่อ ไม่ใช่เรื่องที่จบแล้ว ให้ส่งชื่อหน้าที่ถูกข้ามเข้าคิวรอตรวจ GT ไม่งั้นทุกครั้งที rebuild มันจะทิ้งหน้าเดิมๆ เงิบๆ ตลอดไป

gridmaster ทั้ง 12 ไฟล์ยังไม่มี dimension_chains เลย ไม่เคยไล่ดูจากรูป และการคำนวณย้อนกลับก็ถูกปฏิเสธเพราะเป็นการปลอมที่มาของข้อมูล

กลาง 2026-08-24

การไม่ยอมแต่งที่มาของข้อมูลขึ้นมาเองนั้นถูกแล้ว แต่ช่องโหว่ที่บันทึกไว้ใน 12 ไฟล์ก็ยังเป็นช่องโหว่อยู่อุต ให้ตั้งงานไล่ดูจากรูปด้วยมือเป็นงานจริงในตาราง ไม่งั้นขั้นตอนถัดๆ ไปจะเข้าใจเองเงิบๆ ว่าฟิลด์นี้มีข้อมูลครบแล้ว

ไฟล์ MEP/สถาปัตย์/สารบัญ/สัญลักษณ์ รว 132 ไฟล์ยังมีโครงสร้าง container เพี้ยนอยู่ หลังจากรอบปรับ schema ให้เป็นมาตรฐาน

กลาง 2026-08-24

ถ้าการปรับให้เป็นมาตรฐานทำได้แค่บางส่วน ให้บอกว่าครอบคลุมก็เปอร์เซ็นต์ของ repo ไว้ข้างๆ บรรทัดผ่าน/ไม่ผ่าน เพราะ 'ตรวจผ่านทั้งหมด' วางคู่กับ '132 ไฟล์ยังไม่ได้แตะ' คือข้อเท็จจริงสองอย่างที่คนจะอ่านรวมเป็นอันเดียว

วันที่ 08-24 บันทึกไว้ว่า มยผ. 1103-64 เป็นภาพสแกนอ่านไม่ได้ แต่ PDF ตัวจริงมี text layer

กลาง 2026-08-25

"PDF เป็นสแกน" เป็นคุณสมบัติของสำเนาที่โฮสต์อยู่ที่นั่นไฟล์เดียว ไม่ใช่คุณสมบัติของมาตรฐานนั้น ให้ลองหาแหล่งโฮสต์อื่นก่อนจะบันทึกว่าอ่านไม่ได้แล้วไปเอามาตรฐานต่างประเทศมาใช้แทน

ไฟล์ gridmaster 25 ไฟล์ยังไม่มี dimension_chains[] กับ unassigned_dimensions[]

กลาง 2026-08-25

อย่าเติมค่าว่างลงในฟิลด์บังคับเพียงเพื่อให้ผ่าน validator array ว่าคือการยืนยันว่า "ตรวจแล้วไม่มี" ให้บันทึกช่องโหว่นั้นจริงๆ แล้วเข้าคิวไว้ทำงานจริง

ไฟล์ notes รว 76 ไฟล์ไม่มี sections[] เลย เลย parse fc_ksc/fy_* จากมันไม่ได้ตลอดกาล

กลาง 2026-08-25

การแก้แบบ parse จากไฟล์ตัวเอง จะรักษาได้แค่ไฟล์ที่อยู่ในรูปแบบใหม่แล้ว ต้องนับและบันทึกส่วนที่เหลือไว้ให้ชัด ไม่จั่นการแก้จะดูเหมือนเสร็จสมบูรณ์ทั้งที่ครอบคลุมแค่ครึ่งเดียว

22 ไฟล์มีรูปร่าง container ของ notes ผิด และอีก 20 ไฟล์มี key เฉพาะกิจโผล่นอก notes{}

กลาง 2026-08-25

schema ที่ไม่มีชุดคำศัพท์ key แบบปิด จะเชื่อเชิญให้คนคิด key เอาเอง ตัวตรวจต้องปฏิเสธ key ที่ไม่รู้จักด้วย ไม่ใช่แค่ตรวจ key ที่รู้จักว่าถูกไหม

บ้านหลังเดียวที่ไม่ได้อยู่ในชุด BMA ไม่มี GT ของ plan_footing/plan_beam เลย การตรวจสอบข้ามชุดแบบเลยติดตรงข้อมูลที่ขาด

กลาง 2026-08-25

ติดตามความครอบคลุมของ GT แยกตามชุดแบบ ไม่ใช่แค่นับจำนวนบ้าน ชุดข้อมูล 44 หลังจากออฟฟิศเดียวไม่ได้พิสูจน์อะไรเลยกับออฟฟิศที่สอง

validation ล้มแบบร้ายแรง 4 จุด: BOM ใน 22 ไฟล์ กริดหลอก และข้อความบรรยายไฟล์ใน grid_ref

กลาง 2026-08-28

ตรวจรูปแบบของฟิลด์ ไม่ใช่แค่ตรวจว่ามีค่าอยู่ ฟิลด์ grid_ref ต้องปฏิเสธอะไรก็ตามที่ไม่ใช่รหัสกริด และให้เพิ่มการตรวจ BOM เข้าไปในตัวตรวจรูปแบบด้วย บั๊กเรื่อง encoding มองไม่เห็นในทุก editor ที่เชื่อมั่นไว้

แบบต้นฉบับ 2 ใบขัดแย้งกันเอง บันทึกไว้แต่ยังไม่ได้ตัดสิน

กลาง 2026-08-28

เมื่อข้อมูลต้นทางขัดแย้งกันเอง ให้บันทึกทั้งสองค่าที่อ่านได้แล้วปักธงไว้ อย่าเลือกเองเสียๆ การเดาเพื่อให้มันลงตัว พอเข้าไปอยู่ใน GT แล้วจะแยกไม่ออกจากข้อมูลจริง

จับกับดักการจำแนกแบบก่อสร้างที่เจอซ้ำๆ 4 ข้อ หลังจากทำผิดซ้ำแล้วซ้ำอีก

กลาง 2026-08-28

ทุกครั้งที่เราแนกผิดเป็นครั้งที่สอง ให้เขียนมันลงใน spec เป็นกับดักที่มีชื่อ รายการกับดักมีต้นทุนถูกกว่าการมานั่งไล่หาความสับสนเดิมใหม่ทุกหลัง

บ้านหลัง 08 ถูกปักธงว่า CV มั่ว แต่จริงๆ GT น่าจะเป็นฝั่งที่ผิดมากกว่า

กลาง 2026-08-28

เมื่อ detector ให้ผลไม่ตรงกับ GT ให้ไปดูแบบก่อสร้างจริงก่อนจะสรุปว่า detector ผิด และให้ log ทิศทางของทุกความไม่ตรงกันไว้ ข้อผิดพลาดของ GT จะได้โผล่ขึ้นมา แทนที่จะถูกกลืนไปรวมกับอัตราความผิดพลาดของ detector

บ้านหลัง 51/52 (เข้าเป็น 39/40) เข้าชุด GT ทั้งที่ยังมีจุดกำกวมในแบบ 3 จุดที่ยังไม่เคลียร์

กลาง 2026-08-29

จุดกำกวมที่ติดธงแล้วยังหลุดเข้าชุด train คือการเดาแบบเสียบๆ ให้เลือกอย่างใดอย่างหนึ่ง: ติดธงนั้นเป็นฟิลด์บนตัว element เพื่อให้ปลายทางถ่วงน้ำหนักได้ หรือกันหน้าที่ติดธงออกจาก train จนกว่าจะมีคนชมดูแบบต้นฉบับ

เนื้อหาข้อความฝั่ง assistant มีสองรูปแบบปนกัน (list กับ string) ในชุดข้อมูลที่รวมแล้ว

กลาง 2026-08-31

จัดรูปแบบทุกฟิลด์ให้เป็นมาตรฐานเดียวกันตรงจุดที่รวมข้อมูล และใส่ assert เช็ค schema ไว้ใน smoke test แทนที่จะเชื่อว่าตัวสร้างข้อมูลต้นทางทุกตัวตกลงกันแล้ว

ใช้กฎที่ตรวจด้วยตาเพียง 2 หน้า ไปติด label อัตโนมัติให้ทั้งหมด 89 หน้า

กลาง 2026-08-31

เมื่อกฎเดียวจะถูกใช้ label ข้อมูลทั้งชุด ต้องตรวจสอบกฎนั้นด้วยขนาดตัวอย่างที่เหมาะสมกับขนาดชุดข้อมูลจริง ไม่ใช่เหมาะกับเวลาที่มี และต้องตรวจซ้ำทั้งชุดใหม่ทุกครั้งที่เจอบั๊กในกฎนั้น

RB1/RB2 ปรากฏซ้ำสองครั้งด้วยขนาดต่างกัน แต่ BOQ เสียบๆ ใช้แค่ค่าแรกที่เจอเท่านั้น

กลาง 2026-09-02

ถ้ามาร์คไม่การันตีว่าไม่ซ้ำ การใช้ตัวที่เจอก่อนก็เท่ากับโยนเหรียญ ต้องตรวจจับมาร์คซ้ำที่มีสเปกต่างกันแล้วติดธงให้ตรวจสอบ แทนที่จะเลือกใช้ตัวใดตัวหนึ่งไปเสียบๆ

ข้อมูลทดสอบ/dry-run ค้างอยู่ใน annotated/ และ manifest.json โดยไม่ได้เคลียร์ทิ้ง

กลาง

อย่าปล่อยข้อมูลทดสอบ/dry-run ค้างอยู่ใน annotated/ หรือ manifest.json — เคลียร์และคืนค่า manifest.json ทันทีหลังทดสอบเสร็จ โดยไม่ต้องรอให้สั่ง

รายละเอียดขยายคาหน้า 14 อ้างถึงตัวอักษร 'E' ที่ไม่มีอยู่จริงบนกริด ปล่อยค้างไว้ไม่ตัดสินใจ

ต่ำ 2026-07-06

แยกฟังก์ชันโครงสร้างออกจากป้ายชื่อ detail-callout ในสคีมาให้ชัด เพื่อไม่ให้ตัวอักษรที่กำกวมกลายเป็นเส้นกริดโดยไม่ตั้งใจ และควรให้รางวัลกับ agent ที่หยุดแล้วตั้งธงแทนการเดาต่อไป — ความยับยั้งชั่งใจแบบนี้ช่วยปกป้อง grid master ไว้ได้จริง

ตารางราคาของบ้าน 6 แสดงว่าฐานรากแบบแผ่แพร่ (spread footing) แพงกว่าฐานรากเสาเข็ม ในการอ่านทุกรอบที่ทำแยกกันอิสระ

ต่ำ 2026-07-11

เมื่อการอ่านสองรอบที่ทำแยกกันอย่างอิสระได้ค่าที่ไม่สมเหตุสมผลตรงกัน ให้หยุดอ่านซ้ำแล้วส่งให้ผู้เชี่ยวชาญด้านนี้ตัดสินใจ — การที่ผลตรงกันซ้ำๆ แปลว่าปัญหามันย้ายจากเรื่อง 'อ่านผิด' ไปเป็นเรื่อง 'เอกสารต้นทางมีปัญหา' แล้ว

ปั้มน้ำ 200W ถูกจัดเข้าหมวด BOQ ผิดในตารางต้นทุน

ต่ำ 2026-07-12

การกำหนดหมวดหมู่ที่อ่านจากแผ่น BOQ ต้องมีการเช็คความสมเหตุสมผล (สิ่งนี้ควรอยู่หมวดนี้จริงไหม) ไม่ใช่แค่คัดลอกตามが見ตรงๆ

ความขัดแย้งของค่า CN เดิมโผล่ซ้ำในบ้าน 09 — มันเป็นข้อบกพร่องจริงของเอกสารต้นทาง ไม่ใช่การอ่านผิด

ต่ำ 2026-07-30

ทำ index ความผิดปกติข้ามบ้านไว้ เมื่อความผิดปกติเดิมเกิดซ้ำในบ้านอิสระหลายหลัง ให้จัดว่าเป็นข้อบกพร่องของเอกสารต้นทาง แล้วหยุดไล่สืบสวนซ้ำในทุกบ้าน

แผ่น BOQ 2 แผ่นของบ้าน 10 มีตำแหน่งหมวดงานที่ไม่ชัดเจน ปล่อยทิ้งไว้ให้ตรวจชมภายหลัง

ต่ำ 2026-07-30

คำว่า 'เสร็จ 100%' ควรหมายถึงไม่มีง needs_zoom_review เหลือค้างเลย หรือไม่ก็ควรเปลี่ยนไปใช้ตัวเลขจำนวนแทน บ้านที่ปิดงานทั้งที่ยังมีงค้างอยู่ จะถูกอ่านว่าเสร็จสมบูรณ์แล้วโดยคนที่ทำงานต่อจากมัน

หน้าดัชนีแผ่น S-01 ของบ้าน 11 มีรายชื่อไม่ตรงกับแผ่นจริง

ต้า 2026-07-31

title block ของแผ่นงานแต่ละแผ่นมีความสำคัญเหนือหน้าดัชนีเสมอ ห้ามจัดหมวดหมู่แผ่นงานตามข้อมูล metadata ในดัชนีให้อ่านจาก title block ของแผ่นนั้นโดยตรง แล้วบันทึกความไม่ตรงกันกับดัชนีเป็นข้อบกพร่องของเอกสารต้นทาง

ตารางราคาชุดของบ้าน 01 ยังเป็นข้อความดิบไม่มีโครงสร้าง ถูกติง PENDING แทนที่จะแปลงให้เรียบร้อย

ต้า 2026-08-02

การติงแทนการเดาเป็นเรื่องถูกต้องแล้ว แต่ตง PENDING ต้องมีที่ที่มันจะถูกหยิบไปทำต่อจริงๆ ผ่านไปสามเซสชันแล้วมันยังเป็นแค่บรรทัดหนึ่งในไดอารี่เท่านั้น

หน้าเครดิตหน้าเดียวกันปะ ถูกจัดหมวดเป็น pattern คนละแบบถึง 4 หมวดในบ้านต่างกัน

ต้า 2026-08-02

หน้าที่เหมือนกันแบบ byte ต่อ byte ซ้ำบ้านคือการตรวจสอบความสม่ำเสมอฟรีๆ — หน้าที่เป็นไฟล์เดียวกันปะไม่ควรได้ label มากกว่าหนึ่งแบบ และควรบังคับตรวจสอบนี้ไว้ใน checker เลย

ปริมาณทางกายภาพตัวเดียวกันถูกเรียกชื่อสามแบบ — height_mm, height_m และ depth_m

ต้า 2026-08-02

หนึ่งปริมาณ หนึ่งชื่อฟิลด์ หนึ่งหน่วย บังคับด้วย checker เท่านั้น — ชื่อที่ฝังหน่วยเข้าไปด้วยจะชวนให้เกิดชื่อที่สองสำหรับของสิ่งเดียวกันได้ง่ายๆ

แผ่นผนังสำเร็จรูป (precast plank) 18 แผ่นถูกแท็ก pattern:section ทั้งที่ชื่อไฟล์เป็น plank_layout — น่าจะถูกแล้ว แต่เสียงโดน 'แก้' ผิดงามมาก

ต้า 2026-08-21

เวลาป้ายที่ถูกอยู่แล้วดูเหมือนบั๊ก ให้เขียนเหตุผลไว้ในตัวไฟล์เอง (เช่นใน warnings[] หรือฟิลด์ note) ไม่ใช่แค่จดในไดอารี่ — เพราะไดอารี่จะหยุดคนที่มาไล่เคลียร์ขุดอีกหกลบดาให้ช่างหน้าไม่ได้

มีสามไฟล์ที่ค่าฟิลด์ grid_ref เป็นแค่ข้อความ 'grid'/'dummy' แทนที่จะเป็นการอ้างอิงกริดจริง

ต้า 2026-08-25

ค่าตัวยึดพื้นที่ (placeholder) แยกว่าค่าว่างเสียอีก เพราะมันผ่านการเช็คค่า 'มีค่า' แต่ไม่ผ่านการเช็คค่า 'มีความหมาย' ต้อง validate รูปแบบของทุกฟิลด์ที่เป็นการอ้างอิงด้วย

รูป 151 รูปถูกลดขนาดลงทั้งที่เกินเพดานพิกเซลแค่ 0.13%

ต้า 2026-08-31

ตั้งเพดานความละเอียดให้มีช่วงผ่อนปรนเล็กน้อย และก่อนจะเชื่อว่าอะไรคือต้นเหตุจริง ให้วัดพิกเซลก่อนลงมือแก้ — ทฤษฎีเรื่อง DPI ไม่สม่ำเสมอในเคสนี้ผิด

สเปกข้อมูล และ ข้อตกลงที่ค่อยๆ เปลี่ยน

สองที่ที่ต้องตรงกัน (โครงสร้าง ชื่อ หน่วย) แล้วค่อยๆ ไม่ตรงกันโดยไม่มีใครรู้

ทั้งหมด 40 เรื่อง · วิกฤต 4 · สูง 10 · กลาง 22 · ต่ำ 4

เรื่องที่ต้องอ่านให้ครบ (14)

วิกฤต 2026-07-08

ตัวสร้าง task ไม่เคยถูกอัปเดตตาม schema Gen 4 ทำให้ทุกคานโซว์ grid_ref วางและไม่มีข้อมูลหลักเลยในหน้าจอ labeling

เกิดอะไรขึ้น: ไฟล์ label-studio-tasks-makham.js ยังอ่าน el.grid_ref กับ mainBar.count อยู่ ทั้งที่ Gen 4 แยกออกเป็น grid_ref_start/grid_ref_end/span_length_m และเปลี่ยน main_bar เป็นโครงสร้างซ้อน {top,bottom} ค่าจริงเลยตกไปอยู่ใน other_fields_json ที่อ่านได้อย่างเดียว ทำให้ grid_ref ของทุกคาน และหลักแกนหลักของทุกคาน/เสา ขึ้นวางเปล่าใน Label Studio บั๊กนี้กระทบ t3 อย่างเจียบๆ มาตั้งแต่ต้น

ต้นเหตุจริง: มีการแก้ schema ในสเปกและในการสกัดข้อมูล โดยไม่ได้ไล่ตรวจว่าใครเป็นผู้ใช้ปลายทางบ้าง และตัวสร้าง task ก็ล้มแบบเจียบ — ช่องที่หาไม่เจอถูกเรนเดอร์เป็นกล่องว่าง แทนที่จะโยน error

เสียหายอะไร: ไฟล์ task JSON ของ t3 ที่สร้างไปแล้วทั้งหมด (และอาจ import เข้า Label Studio Cloud ไปแล้ว) แสดงข้อมูลหลักและ grid เป็นช่องว่างให้คนที่มา label เห็น ซึ่งเป็นสองช่องที่สำคัญที่สุดในชุดข้อมูล ต้องสร้างใหม่และ import ใหม่ทั้งหมด

แก้ยังไง: สอนตัวสร้างให้อ่านได้ทั้งรูปแบบใหม่และเก่า, ใส่ช่อง grid_ref_start/end/span_length_m และแยกหลักบน/ล่างลงใน XML ของ elements (จาก 3 ช่องเป็น 6 ช่อง) แล้วสร้าง task JSON ใหม่ พร้อมสุ่มตรวจกับคานปกติ (B1) และคานที่ไม่สมมาตร (B5, บน=2/ล่าง=3)

กฎที่ได้: การแก้ schema ยังไม่ถือว่าเสร็จจนกว่าผู้ใช้ข้อมูลปลายทางทุกตัวจะถูกอัปเดตในการแก้ครั้งเดียวกัน และตัวสร้างข้อมูลต้องล้มดังๆ เมื่อหาช่องที่ต้องการไม่เจอ ห้ามปล่อยกล่องว่างออกมาเด็ดขาด

standards-config.js ใช้ค่าระยะทาบและระยะหุ้มผิด พอแก้ตาม มยพ. คานที่ตัดจริงเปลี่ยนความยาวทันที

เกิดอะไรขึ้น: อ่าน มยพ. 1101-64 (31 หน้า) และ 1103-64 แล้วพบว่าค่าในทะเบียนผิด: ระยะทาบเหล็กข้ออ้อยเดิมใช้ค่าเดียว 48d แทนที่จะแยกตามเกรด 30/36/45d, เหล็กกลม 36d แทนที่จะเป็น 40d, และระยะหุ้ม 40/40/25 แทนที่จะเป็น คาน 30 / เสาค 35 / พื้น 20; ขององ 180° ไม่มีค่าต่ำสุด 60มม.เลย ตัวอย่างคานหนึ่งตัว ความยาวเหล็กแกนที่ตัดเปลี่ยนจาก 4998→4749 มม. และปลอกจาก 952→1032 มม.

ต้นเหตุจริง: ทะเบียนค่าเดิมถูกรอกมาจากมาตรฐานคนละตัว (วสท.) และไม่เคยถูกนำมาเทียบกับเอกสาร มยพ. ที่มันอ้างว่ายึดตามเลย และไม่มีอ้างอิงข้อมาตรฐานต่อค่า เลยไม่มีทางเห็นความไม่ตรงกันนี้

เสียหายอะไร: ความยาวเหล็กตัดและปริมาณเหล็กทุกตัวที่คำนวณก่อนแก้ไขนี้ผิดหมด ต้องอัปเดต regression pin test ทั้ง 7 ไฟล์ โดยคำนวณมือใหม่ทุกตัว

แก้ยังง: ย้ายทั้งทะเบียนไปใช้ค่าตาม มยพ. พร้อมอ้างข้อมาตรฐานต่อค่า ค่าที่มาตรฐานไม่ได้กำหนดไว้ (top-bar 1.3x, Ld ของ SDM) ยังคงไว้พร้อมระบุช่องว่างไว้ในฟิลด์ clause npm test 1249/1249 ผ่านหมด โดย pin ทุกตัวคำนวณมือใหม่

กฎที่ได้: ค่าคงที่ทางวิศวกรรมทุกตัวต้องระบุมาตรฐานและข้อที่มาชัดเจน เพื่อให้เห็นความไม่ตรงกันได้ทันทีตอนตรวจสอบ แทนที่จะไปเจอเอาปีถัดไป และเมื่อค่าคงที่เปลี่ยน ต้องคำนวณ regression pin ทุกตัวด้วยมือใหม่ ห้าม bulk-update pin ให้เท่ากับผลลัพธ์ของโค้ดใหม่เฉยๆ

worker.py เขียนไฟล์ผลลัพธ์โดยไม่ใส่ฟิลด์ `pattern` ทำให้ adapter อ่าน grid master ไม่ได้ และคานทุกตัวหาปริมาณไม่ได้

เกิดอะไรขึ้น: ไฟล์ผลลัพธ์ของ worker ไม่มีฟิลด์ `pattern` เลย ถ้าไม่มีฟิลด์นี้ adapter จะจำหน้า grid-master ไม่ได้ ผลคือคานทุกตัวจะได้ `span\_source: 'unresolved'` — หมายความว่าปริมาณคานทั้งบ้านใน BOQ เป็นศูนย์หมด ไฟล์ GT ของทีม training มีฟิลด์ `pattern: "grid\_master"` อยู่แล้ว แต่ฝั่ง worker สิมใส่

ต้นเหตุจริง: worker ฟังพาให้โมเดลส่ง metadata ที่ worker เองรู้อยู่แล้ว — worker เป็นคนสั่ง dispatch subtask นั้นๆ เองจึงรู้ pattern อยู่แล้วก่อนจะเรียกใช้ด้วยซ้ำ การไปเชื่อผลลัพธ์ของโมเดลสำหรับฟิลด์ที่เรามีอยู่แล้วเป็นการฟังพาที่ไม่จำเป็น

เสียหายอะไร: ถ้าปล่อยไว้จะทำให้ปริมาณคานหายไปหมด (เป็นหมวดใหญ่ที่สุดใน BOQ — \$106,821 จาก \$144,704 ของบ้าน house01) ตั้งแต่การรันจริงครั้งแรกเลย โชคดีที่จับได้เพราะมีการจำลองบ้านทั้งหลังแบบ end-to-end โดยไม่ใช่ GPU จริง

แก้ยังง: เพิ่มตาราง `SUBTASK\_PATTERN` แล้วประทับค่า pattern จากฝั่ง worker เอง ไม่ใช่จากคำตอบของโมเดล

กฎที่ได้: อย่าไปถามโมเดลหา metadata ที่เรามีอยู่แล้ว ฟิลด์ไหนที่ถูกกำหนดจาก subtask ที่เราสั่ง dispatch เอง ต้องให้ตัว dispatcher เป็นคนประทับค่าอย่างแนนอนตายตัว

normalizeMainBar() เข้าใจแค่รูปแบบ **top/middle/bottom** ทำให้เหลือเสาทุกต้นออกมาเป็น **null ('DBNull')**

เกิดอะไรขึ้น: `normalizeMainBar()` ใน `js/shared/schema.js` อ่านได้แค่โครงสร้างหลักแบบ `top/middle/bottom` ที่ซ้อนกันเป็นชั้น แต่เสาโดยธรรมชาติตอบมาแบบแบนๆ — บนแบบเขียนแค่ `'4-DB12'` ไม่มีการแยก `top/bottom` เลย ทำให้รูปแบบ `{count, dia_mm}` ที่เป็นข้อมูลแบนๆ ได้ผลลัพธ์เป็น `null` และเหลือเสาทุกต้นหายไปจากบ้านทั้งหลัง ปรากฏออกมาเป็น `'DBNull'` ในขั้นตอนถัดไป

ต้นเหตุจริง: ตัว `normalizer` ถูกเขียนขึ้นตามรูปแบบข้อมูลของคาน แล้วสมมติว่าเป็นรูปแบบสากลใช้ได้กับทุกอย่าง แต่เสาบนแบบก่อสร้างไทยมีรูปแบบธรรมชาติที่ต่างออกไปและง่ายกว่าจริงๆ และไม่มี `fallback` รองรับรูปแบบนี้เลย

เสียหายอะไร: เหล็กเสริมของเสาทั้งหมดหายไปจาก `BOQ/BBS` ในทุกบ้าน และค่าที่ผิดยังโผล่ออกมาเป็นข้อความ `'DBNull'` ที่มองเห็นได้ชัด แทนที่จะเป็น `error`

แก้ยังง: ขยาย `normalizeMainBar()` ให้รองรับรูปแบบแบน `{count, dia_mm}` ด้วย

กฎที่ได้: ตัว `normalizer` ต้องรองรับทุกรูปแบบที่ `domain` ต้นทางผลิตออกมาจริง ไม่ใช่แค่รูปแบบแรกที่เราสร้างมันขึ้นมารองรับ เมื่อฟิลต์ไหนแสดงผลเป็นข้อความ `'null'` ตรงๆ ในขั้นตอนถัดไป นั่นคือช่องโหว่ของ `normalizer` ไม่ใช่บั๊กการแสดงผล

ฟิลต์ span_source เคยถูกตัดออกจากสเปก ต้องเอากลับมาใหม่หลังทำให้การสกัดข้อมูลความมั่นใจต่ำ

เกิดอะไรขึ้น: ฟิลต์ `'span_source'` (`'grid_table' / 'local_dimension' / 'unresolved'`) สำหรับ `element` ประเภทคานใน `pattern` แบบ `'plan'` ไม่มีอยู่ใน `Gen 3 spec` พอสกัดหน้า 19 ใหม่ ก็เจอปัญหาความมั่นใจต่ำเพราะขาดฟิลต์นี้พอดี เลยต้องเพิ่มกลับมากใน `Generation 3.1` โดยพิสูจน์แล้วว่า 'จำเป็นจริง'

ต้นเหตุจริง: `Gen 3 schema` ถูกออกแบบโดยลดความซับซ้อนจาก `Gen 2` โดยไม่ได้ตรวจทีละฟิลต์ว่าฟิลต์ที่ถูกตัดออกมีไว้ทำไม ฟิลต์ที่บันทึก `provenance` (ที่มา) ของตัวเลขถูกมองเป็นสิ่งรบกวนกระทำการขาดหายไปทำให้ตัวเลขนั้นตรวจสอบไม่ได้

เสียหายอะไร: หน้า 19 `view2` (แปลนคาน) ได้ `confidence` แคราว 0.4 และต้องสกัดใหม่ทั้งหมดในรอบ `t2` ด้วยการแบ่งเป็น `atomic segment` พร้อม `span_source` คานทั้งหน้าต้องทำใหม่

แก้ยังง: เพิ่ม `'span_source'` กลับเข้าสเปกสำหรับ `element` ประเภทคานใน `pattern 'plan'` (`Generation 3.1` ข้อ 1) และสกัดหน้า 19 `view2` ใหม่ใน `mk_test/t2/`

กฎที่ได้: เวลาลดความซับซ้อนของ `schema` ห้ามตัดฟิลต์ `provenance` ออกโดยไม่เขียนก่อนว่าทำไมมันถึงมีอยู่ และจะพึงตรงไหนถ้าไม่มี ฟิลต์ `provenance` (ตัวเลขนี้มาจากไหน) คือประกันที่ถูกที่สุดที่กันไม่ให้ข้อมูลตรวจสอบไม่ได้ — เก็บไว้เป็นค่าเริ่มต้นเสมอ

สูง 2026-07-06

grid_ref ของคานถูกโมเดลเป็นตัวเลขนับรวมคร่าวๆ แทนที่จะเป็นช่วงแต่ละอันจริง ทำให้ข้อมูลรายช่วงหายไป

เกิดอะไรขึ้น: กติกาเดิมของ Gen 3 เก็บ **grid_ref** ของคานเป็นค่ารวมคร่าวๆ พร้อม `count` ตัวเดียว พอเทียบกับ `claude_output_03` พบว่าผิด จึงเปลี่ยนกติกาให้ 1 entry = 1 ช่วงกริดจริงพอดี จัดกลุ่มด้วย (`element_id`, `span_length_m`, `span_source`)

ต้นเหตุจริง: schema โมเดลมาร์คของคานเป็นวัตถุก่อนเดียวเพราะมันปรากฏแบบนั้นในตารางกำหนดขนาด (schedule) ไม่ใช่ในฐานะชิ้นส่วนกายภาพจริงที่แยกกันบนแปลน การรวมค่าก่อนบันทึกทำลายความยาวของแต่ละช่วงที่ต้องใช้ในขั้นถัดไป

เสียหายอะไร: แปลนคานทุกอันที่สเก็ตภายในได้กติกาเก่าต้องเขียนใหม่หมด หน้า 19 view2 และหน้า 20 view1 ถูกทำใหม่ทั้งคู่ด้วย atomic segment หน้า 20 ความมั่นใจขยับจากราว 0.5 เป็น 0.87 และได้ atomic segment ครบ 17 ช่วง ซึ่งรูปแบบตัวเลขรวมไม่มีทางให้ข้อมูลแบบนี้ได้เลย

แก้ยังง: Generation 3.1 ข้อ 2: เปลี่ยน **grid_ref** ของคานให้เป็น atomic segment หนึ่งรายการต่อหนึ่งช่วงกริด สเก็ตหน้า 19 กับ 20 ใหม่ใน `mk_test/t2/`

กฎที่ได้: บันทึกข้อมูลในระดับที่มันมีอยู่จริงทางกายภาพ แล้วปล่อยให้การรวมค่าเกิดขึ้นในขั้นตอนถัดไปแทน การรวมค่าตอนสเก็ตคือการทำลายข้อมูลแบบย้อนกลับไม่ได้ — ตัวเลขรวมคำนวณจากช่วงย่อยได้เสมอ แต่ช่วงย่อยไม่มีทางกู้คืนจากตัวเลขรวมได้

สูง 2026-07-06

เอเจนต์สองตัวสะกด **schema key** เดียวกันต่างกัน (**element** กับ **elements**) และวิธีแก้คือยอมรับความเพี้ยนแทนที่จะกำจัดมันทิ้ง

เกิดอะไรขึ้น: ไฟล์ `mk_test/t2/บ้าน_เล็ก_1ชั้น_01_หน้า29_floor_plan.json` ที่ทำโดยเอเจนต์ MEP ใช้คีย์ `elements` (พหูพจน์) ในขณะที่สเปกกับไฟล์อื่นทั้งหมดใช้ `element` (เอกพจน์) สคริปต์สร้างงานของ Label Studio ถูกเขียนให้เช็คทั้งสองชื่อเป็น fallback แทนที่จะแก้ไฟล์ข้อมูลให้ถูก

ต้นเหตุจริง: มีการส่งเอเจนต์หลายตัวทำงานพร้อมกันจากสเปกที่เป็นข้อความ (prose) ไม่มี schema ที่เครื่องตรวจสอบได้ และไม่มีขั้นตอนตรวจสอบผลลัพธ์ของเอเจนต์เลย ไม่มีอะไรปฏิเสธไฟล์ที่คีย์ top-level ไม่ตรงสเปก ความเพี้ยนจึงเข้า dataset แบบเงียบๆ

เสียหายอะไร: dataset ตอนนี้มีการสะกดคีย์เดียวกันสองแบบ และทุก consumer ในอนาคตต้องรู้จักรองรับทั้งสองแบบ ไดอารี่เองก็ตั้งข้อสังเกตว่านี่คือสัญญาณว่าเอเจนต์ที่ทำงานพร้อมกันจะยังคงสร้างความแปรปรวนแบบนี้ต่อไปเรื่อยๆ

แก้ยังง: แก้ไว้แค่บางส่วน — สคริปต์สร้าง task ยอมรับคีย์ทั้งสองแบบ ส่วนไฟล์ข้อมูลต้นทางจงใจไม่แก้ และยังไม่ได้ตัดสินใจว่าจะบังคับชื่อฟิลด์ให้ตรงเป๊ะ หรือจะยอมรับความหลากหลายต่อไป

กฎที่ได้: ต้องตรวจสอบผลลัพธ์ของทุกเอเจนต์กับ schema ที่เครื่องอ่านได้ ก่อนจะรับเข้า dataset การทำให้ตัวอ่านทนต่อความเพี้ยนคือทางเดินที่เดียว — การสะกดแบบที่สองจะไม่มันถูกลบออก และเครื่องมือปลายทางทุกตัวก็ต้องรับภาระสาขานี้ไปตลอดกาล

สูง 2026-07-06

Gen 2 กับการสกัดข้อมูลด้วยตัวเองสลับความหมายของแกนกริดกันโดยไม่รู้ตัว

เกิดอะไรขึ้น: การเทียบ Gen 2 (claude_output_03) กับการสกัดข้อมูลแบบสดพบว่าทั้งสองสลับความหมายของแกนกริดกัน — ข้อมูลจริงตรงกันหมด แค่ label สลับกัน มีการเสนอข้อตกลงที่ชัดเจน: x_lines = แนวนอน/ตัวเลข, y_lines = แนวตั้ง/ตัวอักษร

ต้นเหตุจริง: ข้อตกลงการตั้งชื่อแกนไม่เคยถูกเขียนไว้เป็นลายลักษณ์อักษร แต่ละเจเนอเรชันจึงเลือกตีความเองอิสระ เพราะข้อมูลจริงตรงกันเป๊ะ จึงไม่มีอะไรพังดังพอที่จะเผยให้เห็นความขัดแย้งนี้

เสียหายอะไร: dataset สองชุดดูเหมือนสอดคล้องกันแต่จริงๆ อธิบายแกนตรงข้ามกัน consumer ตัวไหนที่เอามาต่อกันด้วยชื่อแกนจะได้อาคารที่สลับแกนผิด เจอปัญหานี้ได้ก็เพราะมีคนตั้งใจเทียบด้วยมือ ไม่ใช่จากการตรวจสอบอัตโนมัติ

แก้ยังง: เสนอข้อตกลงที่ชัดเจน (x_lines=แนวนอน/ตัวเลข, y_lines=แนวตั้ง/ตัวอักษร) เป็น Generation 3.1 ข้อ 9

กฎที่ได้: เขียนความหมายของทุกแกน ทิศทาง และข้อตกลงการวางแผน ลงในสเปกพร้อมตัวอย่างประกอบ ก่อนเริ่มสกัดข้อมูลรอบแรก ความขัดแย้งระดับ label ที่ข้อมูลจริงตรงกันเป๊ะ คือประเภทที่ตรวจจับยากที่สุด เพราะไม่มีอะไรพัง ตัวเลขแค่มีความหมายกลับตาลปัตร

สูง 2026-07-07

มาร์คัวร์สวน C1/F/GB ชนกับมาร์คัวร์โครงสร้างจริงของบ้าน

เกิดอะไรขึ้น: หน้า 16-17 (รื้อสวน) ใช้มาร์คัวร์ C1, F และ GB ซึ่งบนแบบโครงสร้างหมายถึงเสาหลักกับฐานราก ไม่มีอะไรในข้อมูลที่แยกแยะว่า C1 ของรื้อกับ C1 ของโครงสร้างเป็นคนละตัวกัน

ต้นเหตุจริง: ตัวตนของ element ในสคีมายังอิงแค่ชื่อมาร์คัวร์อย่างเดียว ไม่มีตัวระบุขอบเขต/ประเภทแบบกำกับ ทั้งที่ชุดแบบใช้มาร์คัวร์ซ้ำข้ามโครงสร้างย่อยที่ไม่เกี่ยวกันเลย และสคีมามีทางบอกความต่างนี้ได้

เสียหายอะไร: ถ้าวัดข้อมูลเข้าด้วยกันจะทำให้เสาหลักของรื้อสวนกับเสาโครงสร้างหลักปนกันโดยไม่รู้ตัว ยังไม่ได้แก้ ณ ตอนบันทึก

แก้ยังง: ยังไม่ได้แก้ — ตัดตรงไว้ว่าต้องตัดสินใจก่อนว่าจะแยกทั้งสองอย่างไรก่อนจะรวมข้อมูล

กฎที่ได้: id ของ element ต้องกำหนดขอบเขตตามแบบ/ประเภทงานด้วย ไม่ใช่ดูแค่ชื่อมาร์คัวร์อย่างเดียว — ให้สันนิษฐานไว้เลย ว่าชื่อมาร์คัวร์จะถูกใช้ซ้ำแน่นอนในชุดแบบที่มีมากกว่า 60 หน้า

สูง 2026-07-07

schema แบบเก่า (ก่อน Gen4) เก็บเหล็กบน/เหล็กล่างแยกกันไม่ได้ — คาน B5 ที่มีบน 2 เส้น ล่าง 3 เส้น ถูกยุบเหลือตัวเลขเดียว

เกิดอะไรขึ้น: พอลองหน้า 21 (S-04) ซ้ำอีกรอบ เห็นชัดจากภาพเลยว่าคาน B5/B5X มีเหล็กบน 2-Ø16mm และเหล็กล่าง 3-Ø16mm แยกกัน แต่ schema ตั้งแต่ Gen 3 ลงมาเก็บ main_bar เป็นก้อนแบนๆ {count, dia_mm, type} ไม่มีช่องแยกบน/ล่าง คานตัวนี้เลยบันทึกได้แค่ตัวเลขรวมตัวเดียว

ต้นเหตุจริง: ตอนออกแบบ schema ใหม่ๆ คานตัวอย่างที่หยิบมาดูบังเอิญเป็นแบบสมมาตร (บนล่างเท่ากัน) เลยเก็บเหล็กหลักเป็นจำนวนเดียว — คือหน้าตาของ field ถูกออกแบบตามข้อสันนิษฐานที่มีต่อข้อมูล ไม่ได้ออกแบบตามโครงสร้างจริงในแบบ

เสียหายอะไร: คานทุกตัวที่สกัดด้วยรูปแบบแบนๆ นี้ เสียความต่างทางวิศวกรรมระหว่างเหล็กบนกับเหล็กล่างไปหมด — สองอย่างนี้ทำหน้าที่ต่างกัน (รับโมเมนต์ลบ vs โมเมนต์บวก) และกู้คืนจากตัวเลขรวมไม่ได้เลย พอต้องแก้ก็เลยลากไปเปลี่ยน schema ซึ่งไปทำ Label Studio generator พังต่อ (บั๊ก main_bar ว่าง) และสร้างความเสี่ยงนับซ้ำที่รื้อหน้า 16/17 อีก

แก้ยังง: Gen 4 บังคับให้ทุก element ใช้โครงสร้างซ้อน main_bar: {top:{...}, bottom:{...}} โดยอ้างหน้า 21 เป็นหลักฐานประกอบ

กฎที่ได้: ออกแบบหน้าตาของ field จากแบบที่ยากที่สุดเท่าที่ทำได้ ไม่ใช่จากตัวแรกที่ยากที่สุด field ที่ยุบข้อมูลจนเสียความต่างไปแล้ว จะแตกกลับคืนจากข้อมูลที่เก็บไว้ไม่ได้เลย ต้องกลับไปอ่านรูปใหม่ทุกใบ

สูง 2026-07-08

การบังคับให้แยกहेลิกบน/ล่าง ทำให้เกิดการนับซ้ำเมื่อแบบพิมพ์มาแค่บรรทัดเดียว

เกิดอะไรขึ้น: หน้า 16/17 (งานรู้) พิมพ์สเปกเหล็กของ C1 กับคานทับหลังมาบรรทัดเดียว ไม่ได้แยกบน/ล่าง แต่ schema บังคับให้กรอกทั้งสองช่อง ใครที่มามวก top.count + bottom.count ที่หลังจะนับเหล็กชุดเดียวกันสองรอบ

ต้นเหตุจริง: กฎใน schema ถูกเขียนจากกรณีทั่วไป (คานที่ไม่สมมาตรมีเหล็กบนกับล่างแยกกันจริงๆ) แล้วตั้งเป็นช่องบังคับ โดยไม่เปิดทางให้บอกได้ว่า "ชั้นนี้มีเหล็กชุดเดียว ไม่ได้แยก"

เสียหายอะไร: ทุก element ที่มีสเปกบรรทัดเดียวมีความเสี่ยงนับเหล็กเกิน 2 เท่าแบบซ่อนอยู่ ปัญหานี้ยังไม่ถูกปิดที่ระดับ schema และยังคงยกเป็นความเสี่ยงอยู่แม้ผ่านไปอีกรวัน

แก้ยังง: มี agent เพิ่ม field `total\_main\_bar\_count` เข้ามาเป็นตัวกันไว้ก่อน แต่ช่องโหว่ของ schema ตัวจริงยังไม่ได้ปิด

กฎที่ได้: ช่องที่บังคับกรอก ต้องมีวิธีบอกว่า "ไม่มีข้อมูลนี้/ไม่เกี่ยวข้อง" ได้อย่างถูกกฎ ถ้าไปบังคับให้แยกในสิ่งที่ต้นฉบับไม่ได้แยก จะได้ข้อมูลที่หน้าตาดูจริงแต่บวกออกมาผิด

สูง 2026-07-10

สคริปต์นำ annotation กลับเข้าระบบไม่เคยถูกปรับให้เข้ากับ schema Gen4 และถูกทิ้งไว้หลายวัน

เกิดอะไรขึ้น: label-studio-import-annotations.js กับ -import-repeater-annotations.js ยังพูดภาษา schema ก่อน Gen4 อยู่ ถูกยกเป็นช่องว่างตั้งแต่ 7 ก.ค. ย้ำอีกรวันที่ 8 ก.ค. และวันที่ 10-11 ก.ค. ก็ยังบอกชัดว่ายังไม่ได้แตะ แปลว่างานที่คน annotate ไว้ใน Label Studio ไม่มีทางกลับเข้าชุดข้อมูลได้เลย

ต้นเหตุจริง: การย้าย schema ทำครบแค่ขาไป (การสกัดข้อมูลและการสร้าง task) แต่ไม่ทำขากลับ และขาที่ขาดไปนั้นไม่เคยถูกจัดคิวให้ทำ

เสียหายอะไร: ข้อมูลบ้านสามหลังถูกส่งเข้า Label Studio ให้คน annotate โดยไม่มีทางเอาสิ่งที่แก้แล้วกลับมาได้ — งาน annotation ทั้งหมดไม่มีทางกลับบ้าน

แก้ยังง: ยังไม่ได้แก้ — ค้างเป็นงานคาอยู่อย่างน้อยสี่วัน

กฎที่ได้: ถ้าย้าย schema ให้ย้ายทั้งขาไปขากลับในการเปลี่ยนครั้งเดียวกัน — ทางส่งออกที่ไม่มีทางนำเข้ากลับ แปลว่างาน annotation ไม่มีที่ลง

สูง 2026-09-01

เหล็กในฐานรากหายไปเพราะแค่ **SPEC_FIELDS** ไม่มี `dowel_bar` — จริงๆ ตัว `join` ทำงานถูกอยู่แล้วและข้อมูลก็มีอยู่จริง

เกิดอะไรขึ้น: ต้นเหตุจริงของเหล็กฐานรากที่หายไปไม่ใช่ `join` ที่ฟังก์ `buildSpecs()` ใน `raw-extraction-adapter.js` เป็น generic `join` ผ่าน `element_id` อยู่แล้ว และหน้ารายละเอียด pile cap (S-11/S-12) ก็มี `element_id` ตรงกับหน้าแปลนเป๊ะ (F4/F6/F3) พร้อม `width_mm/height_mm/dowel_bar` ครบ ปัญหาเดียวคือฟิลด์นี้ชื่อ `dowel_bar` ไม่ใช่ `main_bar` และ `SPEC_FIELDS` ไม่มีชื่อนี้อยู่ในรายการ

ต้นเหตุจริง: schema ไม่มีชุดฟิลด์ทางการสำหรับฐานราก (มีบันทึกไว้ใน `schema_note` ของไฟล์เองด้วย) ทำให้ข้อมูลเทรนใช้ชื่อฟิลด์ที่ถูกต้องตามหลักวิศวกรรมแต่ไม่อยู่ในรายการที่รู้จัก ตัว `allowlist` ของ `adapter` จึงเมินฟิลด์นี้ไปเสียๆ ตอนแรกวินิจฉัยผิดว่าเป็นปัญหาการ join ทั้งที่จริงๆ แค่ชื่อฟิลด์ขาดไปตัวเดียว

เสียหายอะไร: เหล็กเสริมในฐานรากหายทั้งชุดข้อมูล ทั้งที่คอนกรีตกับแบบหล่อถูกต้องมาตลอด ทำให้ดูเหมือนเป็นปัญหา `join` ที่ลึกมาก จนวินิจฉัยผิดไปรอบแรก

แก้ยังใ้: `buildSpecs()` ตอนนี้อยู่ไปใช้ `dowel_bar` เมื่อไม่มี `main_bar` (ถูกต้องตามหลักวิศวกรรม — เหล็ก dowel จากเสาต่อมอลฐานรากคือเหล็กหลักของฐานราก รูปทรงเดียวกัน ผ่าน `normalizeMainBar()` ที่มีอยู่แล้ว) ตอนนี้ F4/F6/F3 ได้ผลลัพธ์ 39.34/199.33/19.67 กก. สถานะ ok พร้อมทดสอบใหม่ 2 ตัว

กฎที่ได้: เวลาข้อมูล 'ไม่มา' ให้เช็ค `allowlist` ของฟิลด์ก่อนไปออกแบบ `join` ใหม่ และถ้า schema ไม่มีชุดฟิลด์ทางการของหมวดไหน ชื่อฟิลด์ของหมวดนั้นมักจะเพี้ยนไปเรื่อยๆ — ต้อง `grep` ไฟล์ข้อมูลจริงหาชื่อที่ใช้จริง อย่าคิดเอาเองว่า schema ครอบคลุมแล้ว

สูง 2026-09-01

ฐานรากแผ่ (spread footing) ใช้ `element_id: null` ไม่มี key ร่วมกับหน้าแปลนเลย

เกิดอะไรขึ้น: ฐานรากแผ่ (ตามธรรมเนียมวาดในหน้า S-09) ใช้ `element_id: null` แล้วจับคู่ด้วยขนาดแทน แต่หน้าแปลนก็ไม่มี `width_mm/height_mm` ให้เทียบด้วยเช่นกัน — จึงไม่มี key ร่วมกันเลยระหว่างสองหน้า

ต้นเหตุจริง: ธรรมเนียมการวาดฐานรากแผ่ไม่มีตัวระบุที่ `join` ต้องใช้ และ schema ก็ไม่มี key อื่นมาแทนได้ นี่คือช่องว่างของโมเดลข้อมูลจริงๆ ไม่ใช่บั๊กของโค้ด

เสียหายอะไร: ฐานรากแผ่ไม่มีทาง `join` เข้ากับหน้ารายละเอียดได้เลย เหล็กเสริมของมันจึงถูกคืนไม่ได้ด้วย schema ปัจจุบัน

แก้ยังใ้: ยังไม่ได้แก้ — ประกาศว่าอยู่นอกขอบเขต ต้องออกแบบ schema ใหม่ ไม่ใช่แค่เพิ่มฟิลด์อีกตัว

กฎที่ได้: ก่อนจะ `join` สองข้อมูล ต้องตรวจสอบก่อนว่ามี key ร่วมกันจริงในข้อมูลจริง ถ้าไม่มี นั่นคืองานออกแบบ schema ใหม่ อย่าไปแปะฟิลด์ที่ละตัวเพื่อกลบปัญหา

เรื่องที่เหลือในหมวดนี้ (26)

คิด `element_type` ขึ้นมาใหม่ 9 ตัวกลางคันตอนสกัดข้อมูล แล้วเขียนลงชุดข้อมูลเลยโดยไม่ผ่านการอนุมัติ spec

กลาง 2026-07-05

ถ้าจะขยายขอบเขต ต้องขยายคำศัพท์ที่ควบคุมไว้ (controlled vocabulary) ในการตัดสินใจครั้งเดียวกัน ตอนที่การสกัดข้อมูลต้องคิดคำใหม่ขึ้นมาเพื่อไปต่อได้ นั่นคือการตัดสินใจระดับ spec ที่ถูกทำโดยคนที่บังเอิญอยู่ตรงนั้น ให้หยุดแล้วขอให้เคาะให้ชัดก่อน ไม่งั้นมันจะถูกเขียนกระจายไปหลายสิบไฟล์

สร้าง **grid master** กลางขึ้นมาแล้ว แต่ไม่เคยย้อนไปแก้หน้าเดิมให้อ้างอิงมัน

กลาง 2026-07-06

การสร้างแหล่งความจริงเดียวเป็นแค่ครึ่งงาน อีกครึ่งคือย้ายข้อมูลเดิมออกจากสำเนาเก่าให้เสร็จในรอบเดียวกัน การมีธรรมเนียมสองแบบใช้งานอยู่พร้อมกัน แยกว่ามีแบบเดียวที่ไม่ดี เพราะการแก้ไขจะไปถึงข้อมูลแค่บางส่วน

การ **join** สเปกทำให้แค่คาน ทำให้ **marker** ของพื้นเหลือแค่ **flag** ชี้ทาง ไม่มีข้อมูลที่ **join** แล้ว

กลาง 2026-07-06

เวลาเปลี่ยนวิธีแทนความสัมพันธ์ระหว่างข้อมูล ต้องทำให้ครบทุกประเภท element ในรอบเดียวกัน หรือไม่ก็เขียนช้อยกเว้นไว้ใน spec ให้ชัด การเปลี่ยนแบบทำครึ่งๆ กลายเป็นเงื่อนไขแตกสาขาถาวรในทุกที่ใช้ข้อมูลนั้น

เอกสาร **schema** รวมรุ่นที่ 4 ตกรุ่นภายใน 48 ชั่วโมงหลังเขียนเสร็จ

กลาง 2026-07-06

อย่าปล่อยให้ schema สองสายพัฒนาคู่ขนานกัน ให้ปรับให้ตรงกันอย่างต่อเนื่อง (รายวัน) หรือไม่ก็เลือกให้สายหนึ่งเป็นตัวหลักไปเลย การเอา schema ที่แตกกันมารวมทีเดียวก่อนใหญ่ จะได้เอกสารที่ล้าสมัยก่อนที่ใครจะได้เอาไปใช้

pattern type ชื่อ '**site_profile**' จริงๆ หมายถึงรูปด้านของอาคาร ไม่ใช่ข้อมูลไซตงาน

กลาง 2026-07-06

ทุกค่าใน enum ต้องมีคำนิยามเขียนไว้ตั้งแต่ตอนที่สร้างมันขึ้นมา ชนิดข้อมูลที่ความหมายอยู่แค่ในชื่อ จะถูกใช้ผิดแน่นอน แล้วสุดท้ายก็ต้องมานั่ง migrate ที่หลัง

เพิ่ม **pattern type** ใหม่ 3 ตัวลง **schema** โดยยังไม่มีชุดฟิลด์ที่ตรวจสอบแล้ว

กลาง 2026-07-08

ถ้าจำเป็นต้องปล่อยชนิดข้อมูลใหม่ออกไปก่อนที่ฟิลด์จะนิ่ง ให้ทำเครื่องหมายทุกเรคอร์ดที่เขียนภายใต้ชนิดนั้น เพื่อให้การ migrate ที่หลังหามั่นใจ อย่าไปหวังพึ่งรายการค้างให้ช่วยจำ

เขียน **grid_ref** แบบ **range** โดยเอาแกนนอนขึ้นก่อน สานทางกับ **convention** เดิมของโปรเจกต์เอง

กลาง 2026-07-14

เวลาคิด format ฟิลด์ใหม่ ต้องอิงจาก convention ที่ใกล้เคียงที่สุดที่มีอยู่แล้ว แล้วเขียนลง schema ไปพร้อมกันในการแก้ครั้งนั้นเลย — convention ที่ไม่เคยเขียนไว้เป็นลายลักษณ์อักษรจะถูกละเมิดทันทีที่มีรูปแบบข้อมูลใหม่โผล่มา

grid_ref แบบจุด (**point**) ถูกตัดชิด (**dash**) ออกในข้อมูลจริง ทั้งที่ **schema** ยังเขียนไว้ตรงข้าม

กลาง 2026-07-14

ห้ามปล่อยให้ข้อมูลนาหน้า spec เด็ดขาด ถ้าจะเปลี่ยน convention แล้วเอาไปใช้กับไฟล์จริง ต้องตัดสินใจเรื่อง spec ให้จบก่อน — spec ที่ขัดกับข้อมูลจริงแยกว่าไม่มี spec เลยด้วยซ้ำ

แบบฐานรากมีทั้ง **grid_source** และก๊อปปี้ **grid** เต็มแทรกอยู่ในตัว ขัดกับกฎห้ามซ้ำของ **spec** เอง

กลาง 2026-07-14

กฎของ spec ด้านโครงสร้างต้องมี validator script คอยเช็ค ไม่ใช่แค่เขียนไว้เป็นข้อความ กฎแบบ 'ห้ามทำ X' ทุกข้อควรแปลงเป็นสคริปต์ตรวจทุกไฟล์ก่อนจะประกาศว่าบ้านหลังนั้นเสร็จ

สเปคคานเหมือนกันเป๊ะถูกพิมพ์ซ้ำทุกครั้งที่มาาร์คเดียวกันปรากฏ

กลาง 2026-07-14

ข้อมูลที่เป็นระดับต่อมาร์คต้องเก็บเป็น object แยกต่อมาร์ค ห้ามพิมพ์ซ้ำทุกครั้งที่พบ ค่าที่ซ้ำกันเป๊ะแบบนี้คือระเบิดเวลาที่รอวันแก้ไม่ครบ

กฎเรียงลำดับ **element** ถูกเขียนกลับด้าน ต้องทำใหม่ทั้งหมดในวันถัดมา

กลาง 2026-07-15

ก่อนจะเอากฎเรียงลำดับหรือการเรียงไปใช้กับไฟล์จริง ต้องเช็กับตัวอย่างที่คำนวณมือจริงก่อนเสมอ — กฎที่มีทิศทางแต่ทำผิดทางจะทำให้ต้องเขียนใหม่ทั้งหมดสองรอบ

หน้าโปรโมทของทั้งซีรีส์ถูกจัดหมวดเป็น title/unknown ในทุกบ้านจนกว่าจะคิดรูปแบบที่ 14 ขึ้นมาใหม่

กลาง 2026-07-15

taxonomy แบบปิดต้องมีช่องหนี้ที่แยกออกจาก default ได้ชัดเจน การยัดหน้าที่ไม่เข้าพวกลงในหมวดที่ใกล้เคียงที่สุดจะบังตาไม่ให้เห็นว่า taxonomy ยังไม่ครบ

จุดเริ่มต้นแกน y ของกริดบ้านหลังที่ 5 ต่างจากทุกบ้านอื่น ต้องมากำหนดจุดเริ่มใหม่

กลาง 2026-07-19

ต้องกำหนด convention จุดเริ่มต้นของ grid ให้เป็นมาตรฐานเดียวกันทั้งโปรเจกต์ก่อนสัปดาห์แรก จุดเริ่มต้นที่ไม่ตรงกันจะทำให้ตำแหน่งที่คำนวณต่อมาดูผิดในแบบที่เข้าใจผิดว่าเป็นความผิดพลาดจากการวัดได้ง่าย

ข้อความอ้างอิงค่า '1'(2.25)' ตกค้างอยู่ในไฟล์ ทั้งที่ค่า master เปลี่ยนเป็น 2.35 ไปแล้ว

กลาง 2026-07-20

การไล่แก้ค่าที่เปลี่ยนต้องครอบคลุมข้อความบรรยาย โน้ต และคำเตือนด้วย ต้อง grep หาตัวเลขเดิมทั่วทั้งบ้าน ไม่ใช่แค่ grep หาชื่อไฟล์

คานตัวหนึ่งอยู่คนละตำแหน่งระหว่างชั้น 1 กับชั้น 2 เกินกว่าค่าความคลาดเคลื่อนที่ยอมรับได้ ยังไม่ได้แก้

กลาง 2026-07-20

เมื่อการวัดพบว่าค่าต่างกันระหว่างชั้นเกินค่าที่ยอมรับได้ นั่นคือช่องว่างของโครงสร้างข้อมูล ไม่ใช่ความผิดพลาดจากการวัด — schema ต้องรองรับค่า grid แยกตามชั้น ไม่ใช่แค่ตัดตรงแล้วเลื่อนปัญหาออกไป

resolve_grid.py ของ King-try ขึ้น KeyError กับไฟล์ grid ปัจจุบันทุกไฟล์ หลังจาก schema ย้ายไฟล์

กลาง 2026-08-20

เครื่องมือทุกตัวที่อ่าน schema ของ rawjson ต้องเอามารันกับไฟล์ปัจจุบันใหม่ทุกครั้งที่ schema ย้ายไฟล์ และควร assert รูปร่างข้อมูลที่มีขนาดหวังพร้อม error ที่บอกชื่อชัดๆ แทนที่จะปล่อยให้ KeyError โผล่จากการไล่ key ซ้อนกัน

การแก้ลูเข้าครั้งแรกทำได้แค่ลดโอกาสพัง และตัวกัน n-gram ก็ถูกเลี่ยงได้ง่ายๆ

กลาง 2026-08-21

การรับประกันเชิงโครงสร้าง (วงเล็บครอบคู่ รูปร่างตรง schema) ต้องใช้ข้อจำกัดที่ทำให้ผลลัพธ์แยก เป็นไปไม่ได้ ไม่ใช่ค่าปรับที่แค่ทำให้มันเกิดยากขึ้น และอย่าคิดว่าตัวกัน n-gram แบบตรงเป๊ะจะครอบคลุมลูที่เปลี่ยนตัวกันไปเรื่อยๆ ได้

ชื่อไฟล์ candidate อ่านแล้วเหมือนบอกว่า "นี่คือฐานราก" ทั้งที่มันแค่หมายความว่า "เก็บมาจากหน้าฐานราก"

กลาง 2026-08-25

อย่าให้ชื่อไฟล์หรือชื่อไฟล์สื่อถึงการจำแนกที่โศกไม่ได้ทำจริง ถ้ารู้แค่ที่มา ก็ตั้งชื่อตามที่มา

ไฟล์ที่พิมพ์ทั้ง SD-40 และ SD-50 อยู่บนหน้าเดียวกัน ปลอ่ยวางไว้แทนที่จะเดา

กลาง 2026-08-25

เมื่อข้อมูลต้นทางกำกวมจริงๆ ให้ปลอ่ยไฟล์ว่างแล้วบันทึกเหตุผลไว้ อย่าให้สคริปต์เลือกให้ และถ้าเจอความกำกวมแบบเดียวกันซ้ำๆ นั่นเป็นสัญญาณว่ารูปร่างของ schema ผิด ไม่ใช่ข้อมูลผิด

ปลอ่ยช่องว่างของ schema ไว้: หน้าที่มีแผนแบบจริงสองแผ่นควรมี sheet_code แยกต่อมุ่มมอง

กลาง 2026-08-31

เมื่อข้อสันนิษฐานของ schema ถูกข้อมูลจริงพิสูจน์ว่าผิด ให้บันทึกช่องว่างนั้นไว้พร้อมผลกระทบที่จะตามมา เพื่อให้มีคนตัดสินใจ ไม่ใช่ลืมไปเฉยๆ

ยังไม่ได้ตัดสินใจว่าโมเดลหรือตัว adapter ควรเป็นคนแยกมาร์คที่รวมกันแบบ 'F4,C1' ออกจากกัน

กลาง 2026-08-31

ต้องตัดสินใจและกำหนดรูปแบบการเขียนรหัสขึ้นส่วนให้เป็นมาตรฐานเดียวกันทั้งฝั่งโมเดล ground truth และ adapter ที่รับข้อมูล และต้องเช็คว่าโค้ดฝั่งที่รับข้อมูลรองรับ output จริงของโมเดลก่อนที่จะไปตัดคะแนนว่าโมเดลตอบผิด

ไฟล์ตัวเลข/array เสี่ยงกลายเป็น string เสียบบๆ ตอนเขียนไฟล์ใช้งานจริง

กลาง

ไฟล์ตัวเลข/array ต้องตรงกับชนิดข้อมูลต้นฉบับเป๊ะก่อนเขียนลงไฟล์ที่ใช้งานจริง — อย่าปลอ่ยให้ตัวเลขกลายเป็น string หรือ array กลายเป็น string โดยไม่ตั้งใจ

โค้ด SQL ก้อนหนึ่งอยู่ในรูปคอมเมนต์ในไฟล์ schema.js แทนที่จะอยู่ในโพลเดอร์ database

ต้า 2026-07-05

ชิ้นงานต้องอยู่ในโพลเดอร์ที่เป็นเจ้าของประเภทของมัน ไม่ใช่อยู่ข้างๆ โค้ดที่เป็นแรงบันดาลใจให้เขียนมัน คอมเมนต์ไม่ใช่ที่เก็บ source code ของภาษาอื่น

taxonomy ของ pattern ตอนออกแบบมีแค่ 9 ชนิดและไม่มีช่อง catch-all เลย ต้องมาผูก `unknown` เพิ่มทีหลังตอนสกัดข้อมูลจริง

ต้า 2026-07-08

การจัดกลุ่มแบบปิด (closed classification) ที่ใช้กับเอกสารจริงต้องมีค่า 'ไม่เข้าพวกไหนเลย' ตั้งแต่วันแรก ไม่งั้นรายการแรกที่ไม่ตรงกลุ่มไหนจะถูกจัดผิดประเภทไปเสียๆ

บ้าน 2 หน้า 07/10 ลำดับแถวของกริดถูกเขียนสลับกลับด้านในการสกัดข้อมูลต้นฉบับ

ต้า 2026-07-15

ข้อตกลงเรื่องลำดับ (ordering convention) ต้องอยู่ในสคีมาก่อนสกัดข้อมูลบ้านหลังแรก ทุกข้อตกลงที่มาตั้งทีหลังต้องเสียเวลาไปไล่แก้ย้อนหลังทุกบ้านที่ทำไปแล้ว

ไฟล์ sidecar ของ CV/pass3 เกือบถูกป้อนเข้า raw-extraction adapter ในฐานะ pattern ที่ไม่รู้จัก

ต้า 2026-09-01

เวลามี stage ใหม่เขียนไฟล์ลงใน results directory ที่ใช้ร่วมกัน ให้ฝั่งผู้ผลิตตัดสินใจตั้งแต่ต้นว่าไฟล์แต่ละแบบเป็น output ที่ใช้งานได้จริงหรือเป็นแค่ sidecar สำหรับ diagnostic แล้วให้ฝั่งผู้บริโภคมี allowlist ชัดเจน แทนที่จะปล่อยให้มันเตือนทุกอย่างที่มันไม่รู้จักร

กัปดักการตั้งค่า สภาพแวดล้อม และ ไลบรารี

ค่าตั้งต้น เวอร์ชัน หรือสภาพแวดล้อมที่ต่างกันนิดเดียว แล้วทำให้ผลลัพธ์เปลี่ยนไปเสียๆ

ทั้งหมด 87 เรื่อง · วิกฤต 18 · สูง 26 · กลาง 32 · ต่ำ 11

เรื่องที่ต้องอ่านให้ครบ (44)

วิกฤต 2026-07-21

รัน `pip install` ของ llama.cpp คู่ขนาน ทำให้ transformers กับ torch ถูกลดเวอร์ชันไต้หวัน training ที่กำลังรันอยู่

เกิดอะไรขึ้น: ขณะทำงาน training 3 epoch กำลังรันอยู่ มี tmux อีกหน้าต่างหนึ่งรัน `pip install -r requirements.txt` ของ llama.cpp ในสภาพแวดล้อม Python ตัวเดียวกัน มันลดเวอร์ชัน transformers จาก 5.5.0 เป็น 4.57.6 และ torch จาก 2.10.0+cu128 เป็น 2.11.0+cpu สคริปต์ training เลยพังตอนขั้นสุดท้ายที่สร้างตัวอย่างผลลัพธ์ ด้วย FileNotFoundError ที่ `modeling\_qwen3\_5\_moe.py`

ต้นเหตุจริง: ไม่มีการแยก virtual environment — งานสองงานใช้ interpreter เดียวกัน และปล่อยให้ requirements.txt ของโปรเจกต์อื่นไปหาเวอร์ชันแพ็คเกจที่ไม่ได้ pin ไว้ ซึ่งเป็นแพ็คเกจที่งานที่กำลังรันอยู่พึ่งพา

เสียหายอะไร: งานที่กำลังรันพังกลางคัน (โชคดีที่ adapter เพิ่งบันทึกเสร็จพอดี) torch เวอร์ชัน CUDA ถูกแทนที่ด้วยเวอร์ชัน CPU ล้วน ต้องซ่อมสามขั้นตอนแล้วรัน verify_env.py ใหม่ กว่าจะวัดคุณภาพผลลัพธ์ได้

แก้ยังไง: รัน pip install transformers==5.5.0 (เวอร์ชันเป๊ะๆ จาก log ของรอบที่สำเร็จ) แล้ว pip install torch==2.10.0 --index-url .../cu128 เพื่อกู้เวอร์ชัน CUDA กลับมา จากนั้น verify_env.py ก็ผ่านอีกครั้ง

กฎที่ได้: ห้าม pip install อะไรบนเครื่องที่มีงานสำคัญรันอยู่เด็ดขาด — งานเสริมต้องใช้ venv หรือ container แยกเสมอ และให้จดเวอร์ชันแพ็คเกจที่ใช้ได้จริงจาก log ของรอบนั้นไว้ด้วย เวลาที่จะได้เป็นการ pin ไม่ใช่การเดา

วิกฤต 2026-07-21

CUDA OOM บนการ์ด 95GB เพราะตั้ง LORA_R=64 แล้วโดนคูณด้วย 256 experts ของ MoE

เกิดอะไรขึ้น: เทรนแล้วพังกลางทางด้วย CUDA out of memory — ใช้ VRAM ไป 94.87GB จาก 95GB เหลือว่างแค่ประมาณ 100MB ตอนนั้นตั้ง LORA_R ไว้ที่ 64

ต้นเหตุจริง: ประเมินหน่วยความจำที่ LoRA ใช้เหมือนกับว่าโมเดลเป็นแบบ dense ธรรมดา แต่ Qwen3.6-35B-A3B เป็น MoE ที่มี 256 experts ทำให้ LoRA adapter ถูกสร้างขึ้นมา 256 ชุด — ค่า rank ที่เลือกโดนคูณด้วยจำนวน experts ไม่ใช่แค่บวกเพิ่ม

เสียหายอะไร: การเทรนหยุดกลางคันบนเครื่องที่เสียเงินเช่า ต้องแก้ config แล้วเริ่มรันใหม่

แก้ยังไง: ลด LORA_R จาก 64 เหลือ 32 (งานวิจัยก็บอกด้วยว่า rank เล็กกว่าเหมาะกับชุดข้อมูล 226 ตัวอย่างมากกว่า)

กฎที่ได้: ถ้าเป็นโมเดล MoE ให้คิดขนาด LoRA rank เทียบกับจำนวน experts ไม่ใช่จำนวนพารามิเตอร์ — คำนวณ VRAM เป็น rank × experts ตั้งแต่ก่อนรันรอบแรก อย่ารอให้ OOM ก่อนแล้วค่อยคิด

วิกฤต 2026-07-21

pip install ในอีกหน้าต่างไปหับ transformers จนสคริปต์เทรนที่ยังรันอยู่พังคาที่

เกิดอะไรขึ้น: ระหว่างที่เทรน 3 epoch อยู่ ไปเปิด SSH อีกหน้าต่างแล้วรัน pip install เพื่อเตรียม llama.cpp ล่วงหน้า การติดตั้งครั้งนั้นไปอัปเดต/แทนที่ `transformers` ทำให้ไฟล์ที่ Unsloth ใส่มาให้รองรับ Qwen3.5-MoE หายไป สคริปต์เทรน — ซึ่งเซฟ LoRA adapter ไว้แล้วแต่ยังไม่จบ — เลยพังตอนขั้นสุดท้ายที่สร้างตัวอย่างผลลัพธ์ ด้วย error หาไฟล์ไม่เจอ

ต้นเหตุจริง: ทั้งสองหน้าต่างใช้ Python environment เดียวกันทั้งเครื่อง ไม่มีอะไรกันงานข้างเคียงออกจากงานหลักที่กำลังรันอยู่ และไม่มีใครคิดถึงความเสี่ยงเพราะรู้สึกว่ามันอยู่ "คนละหน้าต่าง"

เสียหายอะไร: ต้องซ่อม environment ก่อนถึงจะรัน eval ได้เลย ส่วน LoRA adapter รอดมาได้เพราะจังหวะพอดีล้วนๆ — ถ้าเกิดเร็วกว่านั้นแค่ไม่กี่นาที ต้องเทรนใหม่ทั้งรอบ (ทั้งเวลาและเงิน)

แก้ยังไง: ซ่อม environment ที่หลังแล้วรัน eval_fields.py ตั้งกฎใหม่ว่า ห้ามแตะ package ที่ใช้ร่วมกันขณะที่งานสำคัญกำลังรัน ถ้าจะทำงานข้างเคียงให้ใช้ virtual environment

กฎที่ได้: ให้ถือว่างานเทรนที่กำลังรันอยู่ล็อก Python environment ทั้งเครื่องไว้ งานติดตั้งอะไรก็ตามที่เตรียมล่วงหน้า ต้องแยกไปอยู่ใน venv ของตัวเอง หรือไม่ก็รอให้งานหลักจบสนิทก่อน

วิกฤต 2026-07-24

MAX_LENGTH=9216 ตัดคำตอบของตัวอย่างเทรน grid-master ทั้ง 5 ตัวทิ้งไปเสียบๆ

เกิดอะไรขึ้น: ค่า MAX_LENGTH ถูกตั้งมาจากตัวอย่างที่มีรูปเดียว แต่ในชุดข้อมูลมีตัวอย่างพิเศษแบบ "grid master" อยู่ 5 ตัวที่ใช้รูป 2-4 รูปพร้อมกัน ต้องการประมาณ 22,000 token — เกินขีดจำกัดไป 2.4 เท่า ส่วนที่เกินจะถูกตัดจากท้าย และท้ายนั้นก็คือคำตอบที่กำลังจะสอนโมเดล

ต้นเหตุจริง: คำนวณงบประมาณยาวลำดับจากตัวอย่างทั่วๆ ไป แทนที่จะดูจากเคสที่หนักที่สุด และการตัดข้อความก็เียบสนิทโดยการออกแบบ — ไม่มี error ไม่มีค่าเตือน บั๊กนี้มีมาตั้งแต่ก่อนเพิ่มบ้านหลัง 05 และเพิ่งเจอตอนอ่านสคริปต์เทียบกับจำนวน token จริงในชุดข้อมูล

เสียหายอะไร: ถ้าไม่เจอ โมเดลจะถูกเทรนด้วยคำตอบที่ถูกตัดขาด เฉพาะกับตัวอย่างชุดเดียวที่สอนเรื่องเส้นกริดทั้งอาคาร — ซึ่งเป็นหัวข้อที่สคริปต์เองบอกว่าฝึกบอยที่สุด จับได้ก่อนรันรอบที่เสียเงิน

แก้ยังไง: เพิ่ม MAX_LENGTH จาก 9,216 เป็น 24,576 (มะขามเลือกเอง เพื่อรักษารายละเอียดของรูปไว้เต็มที่) ยอมรับความเสี่ยง OOM ที่สูงขึ้นในตัวอย่าง 5 ตัวนั้น

กฎที่ได้: ตั้งค่าความยาวลำดับจากตัวอย่างที่ยาวที่สุดจริงๆ ในชุดข้อมูล แล้วใส่ assert หรือ log ให้ร้องทุกครั้งที่มีตัวอย่างโดนตัด — การตัดแบบเสียบๆ ทำลาย label ทั้งโดยไม่แจ้งว่าพัง

วิกฤต 2026-07-24

MAX_LENGTH 9,216 มาจากตัวอย่างที่มีรูปเดียว เลย์ตัดคำตอบของตัวอย่าง **gridmaster** หลายรูปทั้ง 5 ตัวทิ้งเฉยๆ

เกิดอะไรขึ้น: MAX_LENGTH ใน train_qwen36.py ถูกตั้งไว้ที่ 9,216 token ซึ่งเป็นตัวเลขที่คำนวณจากตัวอย่างที่มีรูปเดียว แต่ในชุดข้อมูลมีตัวอย่าง "grid master" อยู่ 5 ตัวที่ใช้รูป 2-4 รูปพร้อมกัน ต้องการถึงประมาณ 22,000 token — เกินไป 2.4 เท่าอะไรที่เกินจะถูกตัดจากท้าย และท้ายนั้นคือคำตอบ JSON ที่กำลังจะสอน

ต้นเหตุจริง: ตั้งค่าคงที่ตัวหนึ่งโดยอิงจากตัวอย่างทั่วไป แทนที่จะอิงจากค่าสูงสุดที่วัดได้จริงในชุดข้อมูล และตอนเทรนก็ไม่มีอะไรรายงานว่ามีการตัดข้อความเลย — มันตัดเฉยๆ

เสียหายอะไร: ถ้าไม่แก้ ตัวอย่าง 5 ตัวเดียวที่สอนเรื่องเส้นกริดทั้งอาคาร (หัวข้อที่สคริปต์เองบอกว่าพลาดบอยที่สุด) จะถูกเทรนโดยคำตอบถูกตัดหายไป บั๊กนี้ไม่มีมาก่อนบ้านหลัง 05 และไม่เคยมีใครจับได้

แก้ยังไง: เพิ่ม MAX_LENGTH เป็น 24,576 (มะขามเลือกเพื่อรักษารายละเอียดรูปไว้เต็ม) ภายหลังพิสูจน์ด้วยการวัดจริง — แกวที่ยาวที่สุดคือ 22,515 token ไม่มีแกวไหนเกินเลย เหลือที่ว่าง 2,061 token

กฎที่ได้: ตั้งขีดจำกัดความยาว/ขนาดทุกตัวโดยอิงค่าสูงสุดที่วัดได้จริงในชุดข้อมูล ไม่ใช่จากตัวอย่างตัวไหน และทำให้การตัดข้อความโยน error ดังๆ ห้ามให้มันตัดเฉยๆ

วิกฤต 2026-07-28

บั๊กที่ 1: ค่า **max_length 10240/13312** ของ **t02** จะตัดตัวอย่าง **gridmaster** เหลือแค่ **42%/39%** ตัดคำตอบ JSON ที่อยู่ท้ายทิ้ง

เกิดอะไรขึ้น: ค่า preset ของ t02 ยังใช้ max_length ที่ 10,240 กับ 13,312 อยู่ — ซึ่งเป็นค่าเก่าจากก่อนการตัดสินใจเมื่อ 2026-07-24 ตัวอย่าง gridmaster ต้องการประมาณ 22,500 token เลยจะถูกตัดเหลือ 42% กับ 39% และส่วนท้ายที่ถูกตัดก็คือคำตอบ JSON นั้นเอง

ต้นเหตุจริง: การตัดสินใจที่ทำกับสคริปต์หนึ่ง (train_qwen36.py เมื่อ 2026-07-24) ไม่เคยถูกส่งต่อไปยังสคริปต์พี่น้องของมัน สองไฟล์มีการตั้งค่าตัวเดียวกันโดยไม่มีกลไกอะไรคอยทำให้ตรงกัน

เสียหายอะไร: จะเป็นการทำบั๊กตัวเดียวกับที่เพิ่งแก้ไปสัปดาห์ก่อนซ้ำอีกครั้ง บนรอบที่แพงกว่าเดิม และเฉยๆ ด้วย

แก้ยังไง: เปลี่ยนเป็น 24576 ในทุก preset พร้อมบันทึกชัดเจนว่าค่าเหล่านั้นเป็นค่าแปลกล้อม และถ้าเจอ OOM ให้ไล่แก้ตามลำดับ max_pixels → lora_r → ขยับไปการ์ดใหญ่ขึ้น ห้ามย้อนกลับมาลด max_length เด็ดขาด

กฎที่ได้: เวลาตัดสินใจเรื่องการตั้งค่าอะไรสักอย่าง ให้ grep หาไฟล์อื่นทุกไฟล์ที่มีการตั้งค่าตัวเดียวกันแล้วแก้ไขใน commit เดียวกันเลย การแก้ฝาแฝดแค่ตัวเดียว คือบั๊กที่รอวันที่ฝาแฝดอีกตัวจะออกโรง

เจอบั๊กอีกสามตัว จากการเอาค่า hyperparameter ของ t02 มาเทียบกับ t01 ทีละตัว — การตรวจที่ได้อารมณ์ว่าควรทำตั้งแต่แรก

เกิดอะไรขึ้น: พอเอาค่าตั้งของ t02 มาเทียบกับ t01 ทีละบรรทัดในช่วงท้ายๆ ก็เจอบั๊กเพิ่มอีกสามตัว: lora_dropout=0.05 (MoE ParamWrapper ไม่รองรับ dropout ที่ไม่ใช่ 0 — ยืนยันจาก error จริงของ t01), optim='adamw_8bit' (ค่าตรงเป๊ะที่ทำให้ t01 เจอ OOM ต้องใช้ paged_adamw_8bit) และไม่มี bf16=True เลย (โมเดลตระกูล MoE นี้ต้องตั้งค่าแบบ bf16 ตามที่ t01 พิสูจน์มา) ทั้งสามค่านี้เป็นค่าที่ t01 เคยพามาแล้วและเลิกใช้ไปแล้ว

ต้นเหตุจริง: รอบใหม่เริ่มจากสคริปต์ต้นฉบับที่เก่าแล้ว แทนที่จะเริ่มจากรอบที่ผ่านการแก้บั๊กมาแล้ว และไม่มีการเอามา diff กับค่าที่รู้ว่าจะใช้ได้อย่างเป็นระบบ จนกระทั่งช่วงท้ายๆ ความรู้จากความล้มเหลวที่ได้มายากๆ อยู่แต่ในไฟล์ของ t01 เท่านั้น

เสียหายอะไร: ค่าที่รู้อยู่แล้วว่าพังสามตัวยังคงคาอยู่ตอนใกล้จะเข้าเครื่อง และทั้งสามตัวเคยทำให้เกิดความล้มเหลวจริงมาแล้วในรอบ t01

แก้ยังไง: แก้ทั้งสามตัว และทำตารางเทียบ 16 บรรทัดไว้ใน §0.4 ยืนยันว่าตัวแปรเดียวที่ยังต่างกันคือตระกูลของโมเดล

กฎที่ได้: เริ่มรอบใหม่ด้วยการเอา config ไป diff กับรอบล่าสุดที่รู้ว่าใช้ได้ ทีละ argument ก่อนทำอย่างอื่นทั้งหมด ค่าไหนที่รอบก่อนเลิกใช้ไปแล้ว คือค่าที่เคยทำให้พังมาแล้ว

transformers 5.5.0 ถอด ip.max_pixels/min_pixels ออกไปเลย ทำให้ Phase 4 พังบนเครื่องที่เสียเงินเช่า

เกิดอะไรขึ้น: การรันจริงครั้งแรกบนเครื่องเช่าพังทันทีด้วย AttributeError: property of 'Qwen2VLImageProcessor' object has no setter เพราะเวอร์ชันของ transformers ที่ pip ดึงมา (5.5.0) ถอด attribute max_pixels/min_pixels ออกไปแล้ว — ไม่ใช่ทำให้อ่านได้อย่างเดียว แต่ถอดออกเลย ทั้ง train_qwen3vl.py และ eval_fields.py ใช้เส้นทางที่ถูกถอดออกไปนั้น

ต้นเหตุจริง: เวอร์ชันไลบรารีเคลื่อนไปเรื่อยๆ ขณะที่โค้ดไม่เคยถูกรันเลยตั้งแต่วันที่เขียน ไม่ได้ pin เวอร์ชัน dependency ไว้ pip เลยไปเอาเวอร์ชันที่ใหม่กว่าทุกเวอร์ชันที่โค้ดเคยถูกทดสอบด้วย และ API ที่โค้ดพึ่งพาก็หายไปแล้ว

เสียหายอะไร: พังยับที่ Phase 4 บนเวลา GPU ที่เสียเงิน ทั้งที่การตรวจ Phase 0 ผ่านหมดทุกข้อ — พิสูจน์ว่า Phase 0 จับความล้มเหลวประเภทนี้ไม่ได้ เพราะมันรันบนเครื่องที่มีเวอร์ชันไลบรารีคนละชุด

แก้ยังไง: ไปอ่านซอร์สจริงของไลบรารีแทนที่จะเดา พบว่า ip.size['longest_edge'] / ['shortest_edge'] ยังใช้ได้อยู่ แล้วลบสองบรรทัดที่พึ่งออกจากทั้งสองไฟล์

กฎที่ได้: pin เวอร์ชันของ dependency ทุกครั้งที่จะรันงานที่เสียเงิน และเวลา API พัง ให้ไปอ่านซอร์สจริงของไลบรารี อย่าเดาว่าตัวแทนคืออะไร โค้ดที่นอนนิ่งไม่ได้รันมานาน เสี่ยงกว่าที่ตาเห็น — โค้ดไม่ขยับ แต่โลกข้างนอกขยับไปแล้ว

GGUF ของ t01 อาจไม่มีการ tune อยู่ในนั้นเลย — LoRA merge ล้มแบบเจียบๆ บน MoE ParamWrapper

เกิดอะไรขึ้น: t01 (tune ช่วง 21-24 ก.ค. แปลงเป็น GGUF แล้วใช้ในเครื่องมาตลอด) ให้ผลการสกัดข้อมูลที่ใช้ไม่ได้ พอไปดูพบว่า format/schema ผิดทั้งหมด (0 จาก 6 หน้า) ขณะที่การอ่านเนื้อหากลับดี — ซึ่งเป็นลายเซ็นของโมเดลฐานที่ไม่ได้ tune แล้วทำตาม prompt เท่านั้น ข้อสงสัย: merge_and_unload() ไม่เคย merge LoRA เข้าไปใน weight จริงก่อนแปลงเป็น GGUF

ต้นเหตุจริง: merge_and_unload() ของ peft/Unsloth merge LoRA แบบ ParamWrapper บนโมเดล MoE ผิด — พิสูจน์แล้วเมื่อวันที่ 28 ด้วยการวัด tensor ขนาด 201 ล้านค่าแล้วพบว่ามีเพียง 0.67% ที่เปลี่ยน โครงสร้างผ่านหมดและไม่มี error ขึ้นมาเลย ความล้มเหลวจึงเจียบสนิท และ t01 ก็ใช้ฟังก์ชันตัวเดียวกันบนโมเดลตระกูลเดียวกัน

เสียหายอะไร: ทุกข้อสรุปที่ได้จาก GGUF ของ t01 ตั้งแต่ 24 ก.ค. อาจเป็นเรื่องของโมเดลฐาน ไม่ใช่โมเดลที่ tune แล้ว ผลการสกัดข้อมูลบ้าน 01 จำนวน 6 หน้าปนเปื้อน งาน inference ในเครื่องทั้งหมดตั้งแต่วันที่ 24 มีค่าที่ไม่รู้ และเรื่องนี้นับล็อกห้รอบ t01 และ t02

แก้ยังง: ยังไม่แก้ — ออกแบบการทดสอบไว้แล้วแต่ยังไม่ได้รัน: เอาหน้า 03 ไปรันกับ GGUF ของ Qwen3.6 ตัวฐานที่ไม่ได้ tune ด้วย prompt เดียวกัน แล้วเทียบกับ output ของ t01 (ราว 20 นาที ไม่มีค่าใช้จ่าย)

กฎที่ได้: ขั้นตอน merge/convert ที่ไม่ขึ้น error ยังไม่ได้พิสูจน์ว่ามันทำงานสำเร็จ หลังทุกการ merge weight ให้เอา tensor ที่ merge แล้วมาเทียบเชิงตัวเลขกับตัวฐาน (ดูว่าเปอร์เซ็นต์ของค่าที่เปลี่ยนไป) ก่อนจะแปลงหรือส่งมอบ — โครงสร้างผ่านไม่ได้แปลว่าพฤติกรรมถูก

t02 ยังไม่มี GGUF เลยสักตัว — บั๊ก MoE merge ตอนนับล็อกการ tune ทั้งสองรอบ

เกิดอะไรขึ้น: การแปลง GGUF ของ t02 ถูกจับได้ว่าล้มเมื่อวันที่ 28 ด้วยตัวกันที่เข้มกว่า และยังไม่แปลงไม่ได้จนถึงตอนนี้ พอถึงวันที่ 29 ก็เข้าใจแล้วว่าบั๊ก MoE merge ของ peft/Unsloth ตัวเดียวกันนับล็อก t01 ด้วย เท่ากับไม่มีรอบไหนเลยที่ได้ของที่ deploy ได้อย่างน่าเชื่อถือ

ต้นเหตุจริง: ข้อบกพร่องต้นทางเพียงจุดเดียวที่ยังแก้ไม่ได้ใน merge_and_unload() สำหรับ LoRA แบบ ParamWrapper บนโมเดล MoE และยังหาทางเลี่ยงไม่ได้

เสียหายอะไร: การ tune เติมรูปแบบสองรอบ ไม่ได้ของที่ส่งมอบให้รันในเครื่องได้เลย ที่น่าสังเกตคือ t02 เทรนที่ 3,796 token ต่อภาพ ซึ่งตรงกับที่ llama.cpp ป้อนให้พอดี มันจึงอาจเป็นโมเดลที่ดีกว่าสำหรับรันในเครื่อง และบั๊ก merge นี้แหละที่ทำให้ไม่มีทางรู้ได้

แก้ยังง: ยังไม่แก้ — มีทางเลือกที่ระบุไว้แล้ว: หาเส้นทาง merge อื่น หรือทิ้ง GGUF ไปเลยแล้วเสิร์ฟ bf16+adapter บนเครื่องเช่าแทน (เส้นทางนั้นรู้อยู่แล้วว่าได้ 90%)

กฎที่ได้: เมื่อข้อบกพร่องต้นทางเพียงจุดเดียวบล็อกเส้นทาง deploy ของทุกสิ่งที่เราผลิต ให้หยุดผลิตของแล้วไปแก้เส้นทาง deploy ก่อน อีกข้อ: ต้องมีโหมดเสิร์ฟสำรองที่รู้ว่าใช้ได้แน่ (bf16 + adapter ตอนรัน) เผื่อไว้เพื่อไม่ให้เกิดการแปลงที่พึ่งบล็อกการส่งมอบทั้งหมด

ภาพเทรน 973 จาก 973 ภาพโดนยอลงหมด — คือ 100% ของ dataset และไม่มีใครรู้มาหลายรอบ

เกิดอะไรขึ้น: พอดีด้วยเครื่องมือที่ทำขึ้นมาเฉพาะ (measure_capacity.py) พบว่าทุกภาพในชุดเทรนเกินเพดาน MAX_PIXELS ที่ตั้งไว้ 5,120 visual token ขนาดที่พบบ่อยที่สุด (3309×2339 จำนวน 615 ภาพ) ต้องการ 7,558 token คือหายไป 32% ส่วนภาพที่ใหญ่ที่สุด (4631×3473) หายไป 67% แม้แต่ภาพที่เล็กที่สุดในชุดก็ยังเกินเพดาน

ต้นเหตุจริง: ตั้ง MAX_PIXELS ไว้ที่ 5,120 โดยไม่เคยคำนวณเลยว่าภาพจริงใน dataset ต้องการเท่าไร ต้นทุน token ของภาพหนึ่งภาพไม่เคยถูกคำนวณจากขนาดไฟล์จริงเลย จนกระทั่งวันนี้

เสียหายอะไร: การเทรนทุกรอบที่ผ่านมามีภาพที่ถูกทำให้เสียคุณภาพ เส้นคานและสัญลักษณ์เล็กๆ ถูกทำลายไปก่อนที่จะโมเดลจะได้เห็นด้วยซ้ำ ซึ่งเป็นเหตุที่เป็นไปได้โดยตรงของ plan_beam 0% เท่ากับหลายรอบที่จ่ายเงินไป รันอยู่บน input ที่เสียหาย

แก้ยังไง: เขียน measure_capacity.py ขึ้นมาคำนวณ token จากขนาดภาพจริง (patch 16 × merge 2 → 1 token = 32×32 px) ขยับ MAX_PIXELS เป็น 7,680 และ MAX_LENGTH เป็น 47,104 ทำให้สัดส่วนภาพที่ถูกยอลดจาก 100% เหลือ 30% โดยไม่ต้องเพิ่มฮาร์ดแวร์เลย (VRAM สูงสุด 87.5 → 90.9 GB บนการ์ด 96GB ตัวเดิม)

กฎที่ได้: คำนวณต้นทุนทรัพยากรจริงของ dataset จริงที่เรามี ก่อนจะตั้งเพดานอะไรก็ตาม เพดานที่ตั้งโดยไม่วัดข้อมูลก่อน คือค่าตั้งค่าที่ทำลายข้อมูลแบบเงียบๆ

ลืมใส่ `crop\_to\_patches: true` ทำให้รูปฝึกทุกรูปถูกยอลือแค่ tile เดียว 256 token โมเดล 78B เลยไม่เคยเรียนรู้จะอ่านแบบเลย

เกิดอะไรขึ้น: หลังเทรน InternVL3-78B แบบ QLoRA ครบ 4 fold ผลประเมินออกมาว่า plan_beam 6/6 = 0% และ plan_footing 1/1 = 0% พอเปิดดู output ดิบ (ไม่ใช่แค่ตัวเลข recall) พบว่าโมเดลตอบเป็นคนละบ้านคนละแผ่นเลย — ป้อนแบบฐานราก แต่มันตอบเป็นแบบหลังคาของอีกบ้านหนึ่ง สืบสวนแล้วเจอหลักฐาน 3 ทาง: (1) นับจำนวน image token ในล็อกเทรน = 768 token หาร 3 รูป = ได้พอดี 256/รูป (2) อ่านซอร์สโค้ด llamafactory/data/mm_plugin.py พบว่าการ tiling ต้องเปิดผ่าน flag `crop\_to\_patches: true` ซึ่ง yaml ไม่เคยตั้งไว้เลย (3) ลองย่อแบบคานจริงลงเหลือ 448×448 (ขนาดที่โมเดลเห็นจริงๆ) พบว่ามาร์คคาน ตัวเลขระยะ และโน้ตหลักเสริมทุกตัวอ่านไม่ออกเลยด้วยตาตนเอง

ต้นเหตุจริง: yaml config การเทรนถูกเขียนโดยก๊อปปีจาก example ตัวอย่างที่เป็นทางการ โดยไม่ได้ใส่ดูซอร์สโค้ดของ framework ว่าจริงๆ แล้วผลิต token ต่อรูปกี่ตัว การ tiling แบบไดนามิกของ InternVL3 ต้องเปิดเองผ่าน flag ตัวเดียว ถ้าไม่เปิดรูปทุกรูปไม่ว่าความละเอียดต้นฉบับจะสูงแค่ไหน จะถูกบีบเหลือ tile เดียว 256 token เสมอ ไม่มีใครตรวจสอบจำนวน token ต่อรูปจริงก่อนจะเสียเงินรันเทรนเต็มรอบ

เสียหายอะไร: เสียเงินไปประมาณ 63 ดอลลาร์ และเวลาประมาณ 15-16 ชั่วโมง × GPU 4 ตัว ไปกับการเทรนที่ได้ adapter ที่เรียนรู้แค่ schema ของ JSON (ถูกต้อง 96-100%) และจำคำตอบของบ้านที่เคยเห็นตอนเทรนได้เท่านั้น — อ่านแบบจริงได้ 0% ทำให้รอบ k-fold 4 fold ทั้งรอบ บ้านทดสอบใหม่ และแผนทำ soup ที่วางไว้เสียเปล่าหมด เป็นครั้งที่ 3 ที่เจอบั๊กประเภทเดียวกัน (t01 Unsloth fallback ไป 512px → t02 ไม่ได้เทียบขนาดที่ data_collator resize → t04 ไม่รู้ค่า crop_to_patches)

แก้ยังไง: บันทึกต้นเหตุไว้เป็นกฎข้อ 15 ใน `rule\_of\_tune.md`: ก่อนจะเทรนจริงบน framework/โมเดลคู่ใหม่ ต้องใส่ซอร์สโค้ดหาจำนวน token ต่อรูปจริงและวัดด้วยสคริปต์ probe แบบ CPU-only (probe_img_tokens.py) ก่อนจะจ่ายเงินรันเต็มรอบ ส่วนตัวแก้จริง (เพิ่ม `crop\_to\_patches: true` แล้วเทรนใหม่) ยังไม่ได้ทำ — รอระดมความคิดเห็นทิศทางต่อ

กฎที่ได้: ก่อนจะเสียเงิน fine-tune เต็มรอบบน framework+โมเดลคู่ใหม่ ให้รันสคริปต์ probe แบบ CPU-only ที่พิมพ์จำนวน token ต่อรูปจริง และขนาดพิกเซลจริงที่โมเดลเห็น แล้วเปิดดูรูปตัวอย่างที่ถูกยอลแล้วด้วยตาตัวเองก่อน ถ้าเราอ่านไม่ออกโมเดลก็อ่านไม่ออกเหมือนกัน

t04 เทรนด้วยรูปแค่ 1 tile เพราะ LLaMA-Factory กับ transformers ตั้งค่า crop_to_patches สวนทางกัน

เกิดอะไรขึ้น: การเรน t04 (InternVL3-78B) ทั้งรอบให้ JSON ที่ถูกต้องแต่ recall 0% วันนี้ไล่เจอตันเหตุแล้ว: preprocessor_config ของโมเดลตั้ง crop_to_patches:false มาให้ ซึ่ง LLaMA-Factory ทำตามตอนเรน แต่ `InternVLProcessorKwargs\_defaults` ของ transformers ฮาร์ดโค้ดเป็น True ตอน inference ผลคือรูปที่ใช้เรนทุกใบยุบเหลือ tile เดียว 256 token ไม่ว่าต้นฉบับจะมีกี่ล้านพิกเซลก็ตาม

ต้นเหตุจริง: โลบารีสองตัวใน stack เดียวกันตั้งค่าเริ่มต้นของฟิลด์เดียวกันสวนทางกัน และไม่มีใครตรวจว่าค่าการแบ่ง tile ที่มีผลจริงเป็นเท่าไรบน batch ที่ใช้เรนจริง — config เขียนตามเอกสาร ไม่ได้เขียนจากจำนวน token ที่วัดได้จริง

เสียหายอะไร: เสียการเรนทั้งรอบที่ใช้ 4 GPU ไปเปล่าๆ (ทั้งเงินและเวลาเต็มวัน) โมเดลที่ได้ต้องทิ้ง และอาการที่เห็นก็ดูเหมือนปัญหาข้อมูล/pipeline อยู่หลายวัน

แก้ยังง: ตั้ง crop_to_patches: true ใน yaml ของ LF (LF เวอร์ชัน $\geq 0.9.3$, PR #7817) แล้วคุมจำนวน tile ทางอ้อมผ่าน image_max_pixels เพราะ max_patches=12 ถูกฮาร์ดโค้ดไว้ใน plugin ภายหลังตรวจสอบบนเครื่องเรนที่ mm_plugin.py:843 ยืนยันว่าค่านี้ทำให้แบ่งเป็น 13 tile / 3,482 token จริง

กฎที่ได้: อย่าไว้ใจค่าเริ่มต้นของขั้นตอนเตรียมข้อมูลที่โลบารีสองตัวอาจตั้งไม่ตรงกัน ก่อนจะจ่ายเงินเรนทุกครั้ง ให้พิมพ์จำนวน tile และจำนวน visual token ต่อบatch จริงออกมาดู แล้วเขียน assert ว่าตรงกับตัวเลขที่ตั้งใจไว้

การทำ quantization 4-bit ของ LLaMA-Factory ไปโดนส่วน vision ด้วย ทำให้ InternViT-6B ตาบอด

เกิดอะไรขึ้น: เจอปัญหาตัวที่สองซ่อนอยู่ใต้บั๊กเรื่อง tile: เอกสารของ InternVL เองเตือนไว้ว่า BNB 4-bit บน InternViT-6B จะให้ผลลัพธ์เพี้ยนและอ่านรูปไม่ออก ส่วน `quantization\_bit:4` ของ LF ก็ quantize ทั้งโมเดลโดยไม่มีการยกเว้น ViT ผ่าน modules_to_not_convert เลย

ต้นเหตุจริง: ตั้งค่า quantization ครอบทั้งโมเดลโดยไม่ได้ตรวจว่าตัว vision encoder หน 4-bit ได้ไหม ทั้งที่ผู้พัฒนาโมเดลเตือนไว้เองในเอกสาร แต่ไม่มีใครอ่านก่อนลงมือทำ arm นี้

เสียหายอะไร: arm ของ Voldemort (InternVL3-78B) ทั้งเส้นใช้ไม่ได้เลย ต้องตัดทิ้งหลังพิสูจน์ว่าโมเดลตาบอดสนิท ทั้งที่แก้ crop_to_patches ถูกต้องแล้ว

แก้ยังง: ไล่ดู source ของ quantization.py ยืนยันว่าไม่มีการยกเว้น ViT จริง แล้วตัด arm นี้ทิ้งหลังผ่านด่านตัดสินใจราคาประมาณ \$2 แล้วย้าย GPU 2 ตัวไปทำ k-fold ของ Courser แทน ไม่ต้องเผาทเรนทั้งรอบ

กฎที่ได้: ก่อนจะ quantize โมเดล vision-language ให้ไปอ่านข้อจำกัดเรื่อง quantization ของฝั่ง vision tower ที่ผู้พัฒนาระบุไว้ก่อน และต้องพิสูจน์ว่าโมเดลตั้งต้นยังอ่านหน้าจริงออกที่ความละเอียด bit นั้น ก่อนจะจ่ายเงินเรน

การถอดรหัสแบบ greedy ทำให้ recall เป็น 0% และพ่นข้อความหลอนหลายภาษายาว 16,363 ตัวอักษร

เกิดอะไรขึ้น: การวัด recall ครั้งแรกของ destrier ได้ 0/34 หน้า 26 ตอบ elements: [] พร้อม confidence 1.0 และติดธงว่าไม่มีฐานรากในแบบ ทั้งที่มีอยู่ชัดๆ 6 ตัว ส่วนหน้า 27 พ่นออกมา 16,363 ตัวอักษรใน 18 นาที: มีความคิดภายในหลุดออกมาทั้งที่ปิด thinking ไว้แล้ว ตามด้วยคำพ้องภาษาอังกฤษยาวเป็นพริต แล้วสลับไปภาษาจีนตรงวลี 'losing getting lost' → 迷路 ต่อด้วยสำนวนจีนเป็นชุด บทความให้กำลังใจ คำปฏิเสธความรับผิดชอบที่เขียนเอง แล้วพยายามกลับมาเขียน JSON ต่อก่อนจะ token หมด

ต้นเหตุจริง: ใช้ do_sample=False มาตลอด ทั้งที่ Qwen เขียนไว้ชัดๆ ว่า 'ห้ามใช้ greedy decoding' เพราะจะทำให้พ่นข้อความซ้ำไม่จบ แล้วยังไปซ้ำเติมด้วย repetition_penalty=1.15 กับ no_repeat_ngram_size=8 ซึ่งไปลงโทษเนื้อหาที่ซ้ำกัน โดยธรรมชาติของแบบก่อสร้าง โมเดลจึงถูกบีบให้หนีไปหาคำศัพท์ที่ไม่เกี่ยวข้องเลย

เสียหายอะไร: การวัดผลทั้งรอบได้ 0% และทำให้ดูเหมือนโมเดลอ่านอะไรไม่ออกเลย เสียเวลา GPU ที่จ่ายเงินไป 18 นาทีต่อหน้า

แก้ยังไง: แก้บรรทัดเดียวใน serve_purson.py: do_sample=True, temperature=0.7, top_p=0.8, top_k=20 แล้วเอา repetition_penalty กับ no_repeat_ngram_size ออก recall ชยับจาก 0% เป็น 11.8% แบบเข้มงวด / 43.5% แบบวัดตำแหน่ง+mark_base และข้อความภาษาจีนหายไปหมด

กฎที่ได้: อ่านและใช้คำการ generate ที่ผู้พัฒนาโมเดลประกาศไว้ก่อนจะรันวัดผลใดๆ และห้ามใส่ค่าลงโทษการซ้ำกับเนื้อหาเฉพาะทางที่ซ้ำกันเป็นเรื่องปกติอยู่แล้ว

t04 (Purson) เทรนด้วยรูปที่ถูกย่อเหลือ 448×448 แบบเสียบๆ เพราะไม่เคยตั้ง crop_to_patches

เกิดอะไรขึ้น: การ fine-tune t04 เสร็จแล้วได้ความแม่นยำในการอ่าน 0% ทั้ง plan_beam และ plan_footing ไฟล์ config ขาดบรรทัด 'crop_to_patches: true' ตัวโหลดข้อมูลจึงไม่ได้ซอยแบบเป็น patch แต่ย่อกระดาษ A1 ทั้งแผ่นเหลือรูปเดียวขนาด 448×448 วัดจาก log การเรนได้ 768 token สำหรับรูป 3 ใบ = 256 token ต่อรูปพอดี ทั้งที่แบบก่อสร้างจริงต้องใช้เป็นพันๆ token

ต้นเหตุจริง: การซอยรูปเป็น tile ต้องเปิดเองผ่าน flag ใน config ที่ทีมไม่รู้ว่ามีอยู่ และไลบรารีก็ไม่ส่งค่าเตือนหรือ error ใดๆ ตอนที่มันปิดอยู่ แล้วยังมีอะไรใน pipeline ที่ assert ว่า token ต่อรูปต้องไม่ต่ำกว่าเท่าไร ค่าเริ่มต้นที่มองไม่เห็นจึงทำลายตัวอย่างเรนทุกใบแบบเสียบๆ

เสียหายอะไร: เสียการเรนไปทั้งรอบ คิดเป็นค่า GPU หลายพันบาท บวกรอบวัดผลที่ตามมาอีก plan_beam กับ plan_footing recall เป็น 0% ทั้งคู่ โมเดลตายตั้งแต่ออกจากโรงงาน และเพิ่งมารู้ตอนจ่ายเงินไปแล้ว

แก้ยังไง: ไล่ต้นเหตุด้วยสามวิธีที่แยกจากกัน (นับ token จาก log, อ่าน source ของ mm_plugin.py, และลองย่อ beam plan จริงเป็น 448×448 แล้วยืนยันว่าอ่านไม่ออกจริง) แล้วตั้งกฎการใหม่เป็น rule_of_tune.md ข้อ 15: ก่อนเรนจริงทุกครั้ง ต้องวัด token ต่อรูปก่อน ซึ่งรันบน CPU ได้ ไม่เสียเงิน ไม่ต้องใช้ GPU

กฎที่ได้: ก่อนจ่ายเงิน fine-tune ทุกครั้ง ให้วัดและ assert จำนวน token ต่อรูปที่ตัวโหลดข้อมูลผลิตออกมาจริงบนตัวอย่างจริง อย่าไว้ใจการจัดการรูปแบบค่าเริ่มต้นของ pipeline ฝั่ง vision เพราะรูปที่ถูกย่อแบบเสียบๆ ไม่มี error ไม่มีเบาะแสอะไรเลย มีแต่ผลลัพธ์ขยะหลังจากที่จ่ายเงินไปแล้ว

t44 (Voldemort) ทำ quantization 4-bit โดนส่วน vision encoder ไปด้วย ทั้งที่ผู้พัฒนาเตือนไว้ชัดๆ

เกิดอะไรขึ้น: เพื่ออัด InternVL3-78B ให้ลง GPU ได้ เลยทำ quantization 4-bit ทั้งโมเดล รวมถึง vision tower ที่เป็น InternViT-6B ด้วย ทั้งที่เอกสารทางการของ InternVL เขียนไว้ตรงๆ ว่าการ quantize vision encoder จะ 'ให้ผลลัพธ์ที่ไม่มีคุณภาพและอ่านรูปไม่ออก' ผลคือโมเดลบรรยายแบบฐานรากภาษาไทยว่าเป็นเอกสารกมๆ พื้นหลังสีชมพู มีโลโก้ Facebook/Apple/Barbie เขียนว่า 'Happy Birthday' และบรรยายแปลนคานว่าเป็นโลโก้ BB-8 จาก Star Wars

ต้นเหตุจริง: ทำ quantization ทั้งโมเดลในฐานวิธีอัดให้พอดีหน่วยความจำ โดยไม่อ่านเอกสารของโมเดลเองว่าส่วนไหนต้องคงความละเอียดเต็ม มอง vision tower เป็นแค่ก้อนน้ำหนักรอกก่อน ทั้งที่มันคือส่วนที่งานทั้งหมดพึ่งพาความคมชัดของมัน

เสียหายอะไร: โมเดลตามยอดสนธิ จับได้ที่ด้านราคา \$2 ก่อนจะเทรนเต็ม ความเสียหายตรงๆ จึงน้อย แต่ก็เสียแรงไปทั้งสาขา (ทั้งการเลือกโมเดล ติดตั้ง และเช่าเครื่อง)

แก้ยังไง: หยุดทันทีหลังด้านตรวจได้ 0/3 แล้วเลิกสาขาโมเดลนี้ไป

กฎที่ได้: อ่านคำแนะนำเรื่อง quantization ของผู้พัฒนาโมเดลเองก่อนจะ quantize อะไรก็ตาม และห้าม quantize vision encoder ในงานที่ต้องอ่านรูปเด็ดขาด เวลายัดให้พอดีหน่วยความจำ ให้เลือก quantize เป็นส่วนๆ และจดไว้ว่าส่วนไหนคงความละเอียดเต็มไว้

server มี timeout การ generate เป็นของตัวเอง 25 นาที ที่ไม่เคยถูกอัปเดตตอนฝั่ง client ขยับเวลาเพิ่ม ทำให้ JSON ถูกตัดกลางคำตอบ

เกิดอะไรขึ้น: งาน f63bc71e เสร็จ 9/9 แต่ 3 หน้า (page_03_notes, page_12_plan_footing, page_13_notes) ได้ JSON ที่พัง สาเหตุจริงคือ 'serve_purson.py' มี 'PAGE_TIMEOUT_S = 25*60' เป็นของตัวเอง แยกต่างหากจาก timeout ฝั่ง client ใน 'worker.py' ที่เพิ่งถูกปรับขึ้นเป็น 2700 วินาทีเมื่อวันก่อนหน้า ตัว timeout ฝั่ง server ทำงานผ่าน 'StoppingCriteria' ที่ตัดการ generate กลางทางโดยไม่สนใจเลยว่า JSON จะสมบูรณ์หรือยัง

ต้นเหตุจริง: มีนิยาม timeout สำหรับ operation เดียวกันอยู่สองที่แยกกัน ในไฟล์คนละไฟล์ และมีแค่ตัวเดียวที่ถูกอัปเดต ที่แยกว่านั่นคือกลไกฝั่ง server ตัด output แบบเสียบๆ แทนที่จะโยน error ทำให้คำตอบที่ถูกตัดดูเหมือนเป็นความล้มเหลวของการจัด format ของโมเดล แทนที่จะเป็นเรื่อง timeout

เสียหายอะไร: 3 หน้าจากการรัน GPU ที่เสียเงินไปแล้วเสียหาย จากนั้นต้องรันบ้านทั้งหลังใหม่ทั้งหมด (job 43323920) เสียรอบ GPU เต็มรอบไปเปล่าๆ บวกกับเวลาไล่ตืบแก้ไขโดยเข้าใจผิดว่าเป็นปัญหาของโมเดล

แก้ยังไง: ปรับ timeout ฝั่ง server ขึ้นเป็น '40*60' (2400 วินาที ตั้งใจให้ต่ำกว่าฝั่ง client ที่ 2700 วินาที เพื่อให้ฝั่ง client ไม่มีวันเป็นตัวที่ยอมแพ้ก่อน) รีستาร์ท server แล้วส่งงานบ้านนั้นใหม่โดยใช้ path รูปที่เก็บไว้แล้ว ไม่ต้องอัปโหลดใหม่

กฎที่ได้: ต้องนิยาม timeout แค่นี้เดียว ถ้าทั้ง client และ server ต่างก็ต้องการ timeout ให้ดึงมาจากแหล่งเดียวกันและยืนยันว่า server < client เสมอ และ timeout ตัวไหนที่ตัด output แทนที่จะโยน error ต้องประทับเครื่องหมาย 'ถูกตัดแล้ว' ไว้ ไม่ให้ความล้มเหลวถูกเข้าใจผิดว่าเป็นความผิดของโมเดล

สูง 2026-07-02

เหมเพลตเช่า GPU บันทึกลงไว้ผิดทั้งประเภทเครื่อง โมเดล และขั้นตอนอนุมัติ ไม่ตรงกับสิ่งที่เกิดขึ้นจริงเลยสักอย่าง

เกิดอะไรขึ้น: No_touch_box/vast-template/ (README.md + onstart.sh) ระบุเครื่อง RTX 3090 24GB เป็นค่าเริ่มต้น พร้อมโมเดล Qwen3-VL-8B-Instruct โดยหลักการตัดสินใจเลือกโมเดลจริงไปให้ 'King เซ็นอนุมัติ' แต่การ fine-tune จริง (t01/t02) ถูกชะมกับ Claude รันตรงบนเครื่องเช่า RTX PRO 6000 96GB และจบลงที่โมเดล Qwen3.6-35B-A3B

ต้นเหตุจริง: โครงสร้างพื้นฐานและขั้นตอนถูกเขียนล่วงหน้าก่อนจะลงมือทำงานจริง แล้วงานจริงกลับทำต่างออกไป และเหมเพลตก็ไม่เคยถูกอัปเดตหรือแก้ไขเลย มันบรรยายอนาคตที่ตั้งใจไว้ ไม่ใช่สิ่งที่สังเกตเห็นจริง และไม่มีอะไรผูกมันเข้ากับการรันจริงเลย

เสียหายอะไร: เหมเพลตที่ดูรันได้จริงแต่เข้าไปที่ GPU ผิดขั้นและโมเดลผิดตัว นอนอยู่ในโปรเจกต์เป็นเวลาเดือนหนึ่ง ใครก็ตามที่ provision จากเหมเพลตนี้จะเช่าการ์ด 24GB สำหรับโมเดลที่ต้องการมากกว่านั้นมาก เสียเงินค่าเช่าไปกับเครื่องที่รันงานนี้ไม่ได้ ขั้นตอน 'King เซ็นอนุมัติ' ก็บรรยายห่วงโซ่อนุมัติที่ไม่มีใครทำตามอยู่แล้วด้วย

แก้งัง: ย้ายเก็บถาวรเมื่อ 2026-08-04 ไปที่ wait_for_thing/No_touch_box/vast-template/ และตั้งธงไว้ใน ARCHIVE.md ว่าลบทั้งจริงได้เลย เพราะไม่มีอะไรในโปรเจกต์ทำตามขั้นตอนนั้นอีกแล้ว

กฎที่ได้: อย่าปล่อยเหมเพลต provision ไว้ในโปรเจกต์ถ้ายังไม่เคยถูกใช้รันงานจริง ให้อัปเดตตามงานที่เกิดขึ้นจริง หรือลบทิ้งไปเลย — เหมเพลต infra ที่ล้ำสมัยคือค่าเชิญให้เช่าเครื่องผิดแล้วต้องจ่ายเงินสำหรับมัน

สูง 2026-07-05

Schema กระจายอยู่ 5 ที่มี factory ที่ตายแล้ว และ pricing enum ชัดกันอยู่ 3 แบบ

เกิดอะไรขึ้น: จากการรื้อ Data Schema / Data Flow พบว่า schema ถูกนิยามกระจายอยู่ 5 ที่ต่างกัน มี v2 factories ใน schema.js ที่เป็นโค้ดตายแล้ว โมดูล site/foundation/BBS-manual ก็ประกาศ shape ของตัวเองแยกออกไปนอก schema.js และมีชุด enum สำหรับ pricing ที่ชัดกันอยู่ถึง 3 แบบ

ต้นเหตุจริง: schema.js ถูกตั้งใจให้เป็น single source of truth แต่ไม่มีอะไรบังคับจริง แต่ละโมดูลจึงสร้าง entity object เองได้ตามใจ ไม่มีกฎที่ต้องผ่าน create* factory เท่านั้น พอเวลาผ่านไปแต่ละโมดูลก็ป็น definition ของตัวเองขึ้นมาซ้ำแนวคิดเดิม

เสียหายอะไร: การมี vocabulary สามแบบสำหรับแนวคิด price-source เดียวกัน ทำให้ใครก็ตามที่เอาไปเทียบกันได้คำตอบผิด, dead v2 factories ทำให้คนอ่าน schema.js เข้าใจผิดว่า shape ไหนคือของจริง และ shape ที่ประกาศนอก schema.js ก็ไม่มีทางถูกตรวจสอบจากศูนย์กลางได้เลย

แก้งัง: บันทึกลงไว้ใน docs/DATA_SCHEMA_REVIEW.md เป็นแผน 4 เฟส วันนี้ทำเสร็จแค่ Phase A ส่วน factory และ enum มาตรฐานถูกทิ้งไว้ให้ Phase B-D

กฎที่ได้: 'single source of truth' ที่เป็นแค่ข้อตกลงปากเปล่า ไม่ใช่ของจริง ต้องบังคับใช้จริง: ห้ามสร้าง entity object มือเปล่าเด็ดขาด ต้องผ่าน create* factory เท่านั้น แล้วเขียนเทสต์ที่จะ fail ถ้ามีใครประกาศ shape นอกโมดูล schema

สูง 2026-07-05

Site, Foundation กับ Planner คอยกันด้วยการอ่าน localStorage key ของกันและกันตรงๆ ไม่ผ่านสัญญาอะไรเลย

เกิดอะไรขึ้น: จากการรีวิว schema พบว่า Site → Foundation → Planner มี data flow แบบไม่เป็นทางการ คือโมดูลอ่าน localStorage key ของโมดูลอื่นตรงๆ ซ้ำทั้ง schema.js และกลไก PIPELINE_EVENT ที่ส่วนอื่นของแอปใช้กัน

ต้นเหตุจริง: ไม่เคยมีสัญญาข้ามโมดูลที่ชัดเจนสำหรับสายนี้ ทางที่เร็วที่สุดในการเข้าถึงข้อมูล — อ่าน storage key ของเพื่อนบ้านตรงๆ — เลยกลายเป็นกลไกเชื่อมต่อจริงไปโดยปริยาย ส่วนทางที่เป็นทางการ (factory + event) ครอบคลุมแค่สาย pipeline เท่านั้น

เสียหายอะไร: ถ้ามีการเปลี่ยนชื่อ storage key หรือเปลี่ยน shape ใน Site เมื่อไหร่ Foundation กับ Planner จะพังเสียบๆ หันที่ไม่มี compile-time หรือ test-time signal เตือนเลย แล้วก็ไม่มี event ยิงออกมา consumer เลยไม่มีทางรู้ว่าข้อมูลเปลี่ยนไปแล้ว

แก้ยังง: บันทึกลงไว้แล้ว สำคัญสัญญา Site → Foundation → Planner ที่ชัดเจน (factory createSiteInvestigation() / createFoundationDesign() พร้อม getter กับ event) ฤกว่าเป็นงานแรกของ Phase B แต่ยังไม่ได้ทำในวันนี้

กฎที่ได้: ข้อมูลข้ามโมดูลต้องวิ่งผ่าน getter/setter ที่มีชื่อชัดเจนกับ event เท่านั้น ห้ามให้โมดูลหนึ่งอ่าน storage key ของอีกโมดูลตรงๆ เด็ดขาด ถ้ามีทางลัดแบบนี้อยู่แล้ว ให้เขียนสัญญาให้มันก่อนที่มันจะมี consumer ตัวที่สาม

สูง 2026-07-06

ระบบตรวจสอบ Label Studio เดิมทั้งหมดผูกติดกับ Gen 1 schema แน่น จนอ่านข้อมูล Gen 3 ไม่ได้เลย

เกิดอะไรขึ้น: Label Studio ทั้งสองรูปแบบเดิม (แบบ JSON ทั้งไฟล์ต่อบ้าน 'Constant01' และแบบ Repeater ต่อหน้าของคิง 'Structural Review'/'BOQ Review') ผูกกับ Gen 1 schema — plan[]/section[]/schedule[] แบบแบน หรือ categories[].items[] — และอ่านจาก qwen-output/ หั้คงไม่รองรับ nested main_bar{}stirrup{} ของ Gen 3, pattern type ใหม่ๆ หรือไฟล์ที่ถูกแยกเป็นหนึ่งไฟล์ต่อหนึ่ง pattern/view แล้ว

ต้นเหตุจริง: เครื่องมือรีวิวถูกสร้างจาก shape ของ schema แบบเจาะจง โดยฝัง shape ไว้ทั้งในตัวสร้าง task และใน XML config เลย และไม่เคยถูกทบทวนตลอดสามเจเนอเรชันของ schema — schema พัฒนาไปทางหนึ่ง แต่เครื่องมือไม่ได้ตามไปด้วยเลย

เสียหายอะไร: ต้องเขียนเครื่องมือรีวิวใหม่ทั้งหมดกลางช่วงเร่งสกัดข้อมูล: ตัวสร้าง task ใหม่ พร้อม XML config ใหม่สามตัว แล้วยังต้องแก้อีกห้ารอบในวันเดียวกัน ข้อมูล Gen 3 ทั้งหมดตรวจสอบไม่ได้เลยจนกว่างานนี้จะเสร็จ

แก้ยังง: เขียน label-studio-tasks-makham.js พร้อม label-studio-makham-elements.xml, -material_list.xml และ -single.xml จัดกลุ่มไฟล์ตาม structure จริงแทนที่จะจัดตามชื่อ pattern

กฎที่ได้: เครื่องมือรีวิว/QA คือส่วนหนึ่งของ schema ไม่ใช่สิ่งที่ตามหลัง schema เวลาออกแบบ schema เจเนอเรชันใหม่ ต้องตั้งเวลาสำหรับอัปเดตเครื่องมือไว้ในแผนเดียวกันเลย ไม่งั้นข้อมูลจะผลิตเร็วกว่าที่จะตรวจสอบได้ทัน แล้วขั้นตอนรีวิวก็จะกลายเป็นคอขวดในจังหวะที่แย่มากที่สุด

สูง 2026-07-21

เลือก LORA_R=64 โดยไม่คิดเรื่อง MoE expert 256 ตัว (และไม่คิดเรื่องชุดข้อมูลแค่ 226 ตัวอย่าง)

เกิดอะไรขึ้น: ตั้ง LoRA rank 64 / alpha 128 ไว้ตั้งแต่ต้น พอเป็นโมเดล MoE ตัว buffer ของ LoRA แต่ละ expert คุณ 256 เข้าไป VRAM 95GB เลยไม่พอ ลดเหลือ 32/64 แล้ว OOM หาย — และงานวิจัยที่เคยหาไว้ก่อนหน้านี้ก็บอกอยู่แล้วว่า rank ต่ำเหมาะกับชุดข้อมูล 226 ตัวอย่างมากกว่า

ต้นเหตุจริง: เลือก rank จากค่า default ทั่วไปโดยไม่คุณต้นทุนหน่วยความจำด้วยจำนวน expert ของโมเดล และไม่ได้เอาคำแนะนำเรื่องขนาดชุดข้อมูลที่ทีมจดไว้เองเมื่อ 2026-07-20 มาใช้

เสียหายอะไร: ช้อม TEST_STEPS ไป 4 รอบและ OOM พังหลายรอบบนเครื่อง \$1.056/ชม. กว่าจะเจอวิธีแก้จริง และค่าเดิมที่ตั้งไว้ยังน่าจะแย่กว่าในแง่คุณภาพด้วย (เสี่ยง overfitting และลืมความรู้เดิม)

แก้ยังไง: ลด LORA_R 64→32, LORA_ALPHA 128→64 (คงอัตราส่วนเดิม) มะขามเป็นคนตัดสินใจหลังจากได้ดูตัวเลือก 3 แบบ ตามกฎข้อ 2 ของ rule_of_tune.md

กฎที่ได้: สำหรับโมเดล MoE ให้คำนวณขนาด LoRA rank โดยคิดหน่วยความจำแบบ จำนวน expert × rank และตรวจ rank เทียบกับขนาดชุดข้อมูลก่อนรันจริงครั้งแรกที่เสียเงิน — งานวิจัยที่ตัวเองจดไว้แล้วนับเป็นข้อจำกัด ไม่ใช่แค่เชิงอรรถ

สูง 2026-07-24

ดิสก์เริ่มต้น 150GB ของ Vast.ai จะทำให้เจอปัญหาดิสก์เต็มซ้ำแบบรอบก่อนหน้าถ้าไม่ระวัง

เกิดอะไรขึ้น: screenshot ของหน้าเช่าเครื่อง Vast.ai เผยว่าตัวเลื่อน Container Size / Disk ตั้งค่าเริ่มต้นไว้ที่ 150GB ทั้งที่ข้อกำหนดที่รู้อยู่แล้วจากปัญหาดิสก์เต็มซ้ำๆ ในรอบ 2026-07-21 คือต้องมีอย่างน้อย 300GB ถ้ากด Rent ด้วยค่าเริ่มต้นนี้จะเจอปัญหาเดิมทันที

ต้นเหตุจริง: ค่าเริ่มต้นของผู้ให้บริการขัดแย้งกับข้อกำหนดที่ทีมเรียนรู้มาแบบเจ็บตัวแล้ว แต่ข้อกำหนดนั้นถูกบันทึกไว้แค่ในบันทึกประจำวันเก่า ไม่ได้ใช้ในการเช็ก่อนเช่าหรือขั้นตอนเช่าเครื่องเลย

เสียหายอะไร: จับได้ก่อนเช่า จึงไม่เสียเงินในรอบนี้ — แต่ค่าเริ่มต้นเดียวกันนี้เคยทำให้เกิดปัญหาดิสก์เต็มหลายครั้งมาแล้วในรอบ 2026-07-21

แก้ยังไง: เพิ่มเป็นรายการชัดเจนใน Phase 0.2 และภายหลังทำให้เข้มข้นใน verify_env.py ด้วยการเช็ที่จะ fail ถ้าพื้นที่ดิสก์รวมต่ำกว่า 280GB (กันกรณี 'ลืมลากตัวเลื่อน')

กฎที่ได้: ค่าเริ่มต้นของผู้ให้บริการที่เคยทำให้เจ็บตัวมาแล้ว ต้องกลายเป็นการตรวจสอบอัตโนมัติ ไม่ใช่แค่โน้ตให้จำ ให้เขียนการเช็คนั้นเข้าไปในตัว verify environment ไม่ใช่แค่ในเอกสาร

สูง 2026-07-28

Training พังทันทีเพราะ library เปลี่ยน: ip.max_pixels เซตค่าไม่ได้แล้วใน transformers 5.5.0

เกิดอะไรขึ้น: รอบรัน t02 จริงครั้งแรก script ตายทันทีด้วย AttributeError: property of 'Qwen2VLImageProcessor' object has no setter โค้ดตั้งค่าความละเอียดรูปผ่าน ip.max_pixels ซึ่ง library เวอร์ชันใหม่ทำให้เขียนไม่ได้แล้ว

ต้นเหตุจริง: train_qwen3vl.py ถูกเขียนไว้หลายเดือนก่อนและไม่เคยถูกรันเลย พอ pip ติดตั้ง transformers เวอร์ชันใหม่บนเครื่องเช่าใหม่ API ของ library ก็เปลี่ยนไปตอนที่โค้ดถูกทิ้งไว้เฉยๆ ไม่มีการ pin เวอร์ชันไว้เลย

เสียหายอะไร: เกิดความล้มเหลวหลังมีเตอร์เริ่มเดินแล้ว ตรงก้าวแรกของการรันที่ต้องจ่ายเงิน

แก้ยังไง: อ่าน source code จริงของ library (ไม่ใช่เดา) แล้วเปลี่ยนไปใช้ ip.size["longest_edge"] แทน ลบ path เก่าทิ้งทั้งหมด

กฎที่ได้: โค้ดที่ไม่เคยถูกรัน คือโค้ดที่ไม่เคยถูกทดสอบ ไม่ว่าจะเก่าแค่ไหนก็ตาม ต้อง pin เวอร์ชัน library ใน environment spec หรือไม่ก็ smoke-run ทุก script ให้จบตั้งแต่ต้นจนจบในเครื่องตัวเองก่อนเช่าฮาร์ดแวร์

onstart.sh ติดตั้งไลบรารีทุกตัวแบบไม่ล็อกเวอร์ชัน — ต้นเหตุจริงของปัญหาเวอร์ชันดริฟท์ไม่เคยถูกแก้ไข

เกิดอะไรขึ้น: สคริปต์ on-start ของ Vast.ai ติดตั้ง triton, unsloth, trl, peft, accelerate, bitsandbytes และ pillow ด้วย pip ธรรมดาโดยไม่ล็อกเวอร์ชันเลย นี่คือกลไกเดียวกันเป๊ะที่ทำให้วันที่ 28 ก.ค. ได้ transformers เวอร์ชัน 5.5.0 มาติดตั้งจนทำให้การเทรนพังตั้งแต่เริ่ม การแก้ไขที่หาไปคือแก้ไขโค้ดฝั่งที่เรียกใช้ (ใช้ `ip.size['longest_edge']`) ไม่ใช่แก้ไขขั้นตอนการติดตั้ง

ต้นเหตุจริง: การเช่าเครื่องแต่ละครั้งจะได้ environment ใหม่ที่ไม่ได้ล็อกเวอร์ชันไว้ เครื่องใหม่แต่ละเครื่องจึงอาจต่างจากเครื่องที่เคยทดสอบสคริปต์ไว้ได้เสียๆ และไม่มีใครเพิ่ม lockfile หรือ pin เวอร์ชันหลังจากเจอบั๊กดริฟท์นี้เลย

เสียหายอะไร: รับประกันได้เลยว่าปัญหาแบบเดียวกันจะเกิดซ้ำอีกในการเช่าครั้งถัดไป และมันจะเกิดในจังหวะที่แย่มากที่สุดด้วย คือทันทีที่เริ่มเสียเงินไปแล้ว นอกจากนี้ยังทำให้การเช่าแต่ละครั้งเทียบกันไม่ได้เป๊ะๆ ซึ่งเป็นความไม่แน่ใจแบบเดียวกับที่บังคับให้ต้องทำ Phase 7.5 วัตถุประสงค์แบบไม่ได้วางแผนไว้

แก้ยังไง: ยังไม่ได้แก้ — เอกสาร Appendix A ยังระบุอยู่ว่า onstart.sh ติดตั้งแพ็คเกจแบบไม่ล็อกเวอร์ชัน

กฎที่ได้: ต้องล็อกเวอร์ชันไลบรารีทุกตัว (หรือมี lockfile) ในสคริปต์ provisioning มิฉะนั้น environment ที่ไม่ล็อกเวอร์ชันบนเครื่องเช่าจะเปลี่ยนงานตั้งค่าที่ควรฟรี ทำซ้ำได้ ให้กลายเป็นการเสี่ยงดวงที่ต้องจ่ายเงินทุกครั้ง

ตัวแปรของการเทรนสองตัวเพี้ยนหายไปเสียๆ ระหว่าง t01/t02 กับ t03

เกิดอะไรขึ้น: ตารางเทียบค่าที่ทำก่อนเช่าเครื่อง (rule_of_tune ข้อ 12) พบว่า `PYTORCH_ALLOC_CONF=expandable_segments` หายไปจาก t03 (t01 มี ใส่ไว้หลังเจอ OOM ที่ขาดอีกแค่ 100MB) และ `FINETUNE_VISION` ถูก hardcode ใน t03 ทั้งที่ t01/t02 ทำเป็น env var

ต้นเหตุจริง: `train_t03.py` ถูกเขียนขึ้นใหม่แทนที่จะ copy มา ของแก้ปัญหาสภาพแวดล้อมที่ได้มามากจากรอบก่อนเลยไม่ถูกพามาด้วย และไม่มีสัญญาณที่เครื่องตรวจได้ระหว่างรอบ มีแค่ตารางเทียบที่ทำด้วยมือ

เสียหายอะไร: ถ้าไม่เจอก็จะไปเจอ OOM ตัวเดิมที่ `expandable_segments` ถูกใส่มาเพื่อแก้พอดิ และทำให้ตัวแปร A/B ปรับตอนรันไม่ได้ โชคดีที่จับได้ก่อนเช่าเครื่อง ต้นทุนเลยเป็นศูนย์

แก้ยังไง: ใส่กลับทั้งสองตัว โดยตั้งค่าเริ่มต้นให้ตรงกับรอบก่อนหน้าทุกไบต์

กฎที่ได้: ตารางเทียบค่าก่อนเช่าเครื่องไม่ใช่ของทางเลือก ทุกค่าในสคริปต์เทรนที่เขียนใหม่ต้องเอาไป diff ทีละจุดเรียกใช้กับรอบล่าสุดที่ใช้ได้ โดยเฉพาะ environment variable ที่เคยใส่ไว้เพื่อแก้ปัญหาในอดีต

สูง 2026-08-25

ย้อนหลัง: t01 ตั้ง MAX_PIXELS ไว้ที่ 512px ทำให้รูปเบลอทุกใบ และผลทั้งรอบใช้ไม่ได้

เกิดอะไรขึ้น: ในรอบเทรน t01 ตั้ง MAX_PIXELS ผิด ทำให้รูปถูกย่อเหลือ 512px เบลอมาก ตัวเลข evaluation ทั้งรอบจึงวัดจาก input ที่เป็นขยะ

ต้นเหตุจริง: ตั้งค่าความละเอียดโดยไม่ได้ตรวจสอบว่าโมเดลได้รับอะไรจริงๆ และไม่ได้เปิดดูตัวอย่างเทรนที่ผ่าน preprocess ด้วยตา ก่อนรัน

เสียหายอะไร: ผลของรอบเทรนทั้งรอบใช้ไม่ได้ ต้องทิ้ง บันทึกไว้เป็นบทเรียนข้อ 12 ใน rule_of_tune.md — แล้วความผิดพลาดประเภทเดียวกันก็กลับมาอีกในวันที่ 2026-08-25

แก้ยังไง: จัดไว้เป็นบทเรียนข้อ 12 ใน rule_of_tune.md ส่วนการวัดรอบวันที่ 2026-08-25 คือการแก้ไขระบบที่ควรกันไม่ให้เกิดครั้งที่สาม

กฎที่ได้: ก่อนรันเทรนทุกครั้ง ให้ dump รูปที่ผ่าน preprocess ออกมาหนึ่งใบ ให้เหมือนที่โมเดลจะได้รับปะ แล้วเปิดดูด้วยตาตัวเอง ตัวอย่างเบลอใบเดียวทำให้ตัวเลขทั้งรอบใช้ไม่ได้

สูง 2026-08-25

t03 OOM สองครั้งเพราะดันตัวเลขขึ้นโดยไม่ประเมินขนาดก่อน จนต้องลด LoRA rank

เกิดอะไรขึ้น: รอบก่อนดัน MAX_LENGTH ขึ้นเป็น 32,768 โดยไม่ได้ประเมิน VRAM ก่อน ผลคือ OOM ต้องลด LoRA rank จาก 32 เหลือ 16 แล้วรัน eval ก็ยัง OOM ซ้ำอีกจากการ upcast logit เป็น fp32 ที่ความยาวเดิมนั้น รอบล่าสุด OOM ตอนเหลือ VRAM แค่ 470 MiB

ต้นเหตุจริง: ปรับค่า config ขึ้นด้วยสัญชาตญาณ แทนที่จะแตกออกมาเทียบกับ VRAM ที่วัดได้จริง แถมยังไม่ได้คิด fp32 logit upcast ของเส้นทาง eval เข้าไปในการประเมินเลย คิดแค่ขั้น training อย่างเดียว

เสียหายอะไร: รัน GPU ล่มสองครั้ง (เผาเวลาเช้าหึง) โดนบังคับให้ลดความจุ LoRA ($r\ 32 \rightarrow 16$) ซึ่งกดเพดานของรอบนั้นลง และเสีย eval ไปหนึ่งรอบ

แก้ยังไง: เซสชันนี้แตก VRAM สูงสุดออกจากค่าที่วัดได้จริงค่าเดียวที่มี (87.5 GB ที่ seq 32,768, $r=16$): weights bf16 74.0 + LoRA/grad/optim 5.7 + activation 7.8 โดยมีแค่ก่อน activation ที่โตตามความยาว sequence แล้วดัน MAX_PIXELS 5120 $\rightarrow$ 7680 / MAX_LENGTH 32768 $\rightarrow$ 47104 บนแบบจำลองนี้ ไม่ใช่ดันมั่ว

กฎที่ได้: ห้ามดันค่าคงที่ที่ผูกกับหน่วยความจำ โดยไม่แตกก่อนว่าหน่วยความจำไปอยู่ตรงไหนและก่อนไหนโตตามค่านั้น คำนวณบนกระดาษก่อน แล้วยืนยันด้วยการรัน TEST_STEPS สั้นๆ ก่อนจะผูกมัดกับการเข้าเต็มรอบ

สูง 2026-08-25

MAX_PIXELS ผัง training ปรับเป็น 7680 แล้ว แต่สคริปต์ inference ยังค้างที่ 5120 — ความไม่ตรงกันระหว่าง train กับ eval ที่รอวันเกิดปัญหา

เกิดอะไรขึ้น: train_t03.py ถูกปรับเป็น MAX_PIXELS=7680 แต่ infer_house_t03.py` จงใจปล่อยไว้ที่ 5120 เพื่อให้ตรงกับ adapter t03 ตัวเดิม ต้องย้ายเป็น 7680 ก่อนประเมิน t03b ไม่งั้นรอบถัดไปจะถูกประเมินที่ความละเอียดคนละค่ากับตอนที่เทรน

ต้นเหตุจริง: ค่าคงที่ความละเอียดถูกก๊อปปี้ซ้ำไว้ทั้งในสคริปต์ training และสคริปต์ inference โดยไม่มีแหล่งเดียวที่ใช้ร่วมกัน การปรับค่าหนึ่งจึงทำให้อีกค่าหนึ่งไม่ตรงกันไปเสียๆ

เสียหายอะไร: ถ้าพลาดจุดนี้ t03b จะถูกประเมินที่ความละเอียดผิดจากตอนที่เทรน ได้ตัวเลขที่วัดความไม่ตรงกันแทนที่จะวัดตัวโมเดลจริง และความไม่ตรงกันนี้จะดูเหมือนโมเดลถดถอยลง

แก้ยังไง: ยังไม่ได้แก้ — เพิ่มคอมเมนต์เตือนชัดเจนไว้ที่บรรทัด MAX_PIXELS ของสคริปต์ inference และใส่ไว้ในรายการด้านบังคับตอนปิดวันสำหรับรอบถัดไป

กฎที่ได้: ค่าคงที่ที่ต้องตรงกันระหว่างสคริปต์ training กับ inference ควรอยู่ใน config กลางที่ใช้ร่วมกัน ไม่ใช่ก๊อปปี้แยกไว้สองที่ คอมเมนต์เป็นแค่ตัวเตือนความจำ ไม่ใช่การรับประกัน ทางแก้ที่ยั่งยืนคือมีแหล่งเดียวให้ทั้งสองสคริปต์ import มาใช้

สูง 2026-08-25

label training ของภาพเส้นกริดถูกตัดกลาง JSON ไปเสียๆ ที่ MAX_LENGTH=24,576 ตลอดรอบ t01/t02

เกิดอะไรขึ้น: MAX_LENGTH ตั้งไว้ที่ 24,576 ในรอบ t01/t02 ตัวอย่างเส้นกริดใช้ภาพ 4 รูปและวัดโทเคนได้ประมาณ 26,791 โทเคน คำตอบ JSON ของมันจึงถูกตัดกลาง object ระหว่างเทรน เพิ่งมาเจอและปรับขึ้นเป็น 32,768 ในรอบ t03 — รอบก่อนหน้านี้เทรนด้วย label ที่ถูกตัดไปแล้วโดยไม่รู้ตัว บั๊กคลาสเดียวกันนี้ถูกพูดถึงซ้ำอีกครั้งในวันเดียวกันว่าเป็นเหตุผลที่ห้ามลด MAX_LENGTH เดี่ยวๆ โดยไม่ลด MAX_PIXELS ตามไปด้วย ("บั๊กคลาส t01 \$0.4")

ต้นเหตุจริง: ตั้งความยาวลำดับ (sequence length) เป็นเลขกลมๆ แทนที่จะวัดเทียบกับตัวอย่างที่ยาวที่สุดจริงในชุดข้อมูล ไม่มีการตรวจสอบว่า label จะพอดีกับบิตที่ตั้งไว้หรือไม่ ทำให้การตัดข้อความออกไปดูเหมือนการร่นปกติแทนที่จะขึ้น error

เสียหายอะไร: การ tuning ทั้งสองรอบ (t01, t02) เทรน subtask เส้นกริดด้วย label ที่ JSON ถูกตัดกลาง ทำให้ความแม่นยำของเส้นกริดในรอบเหล่านั้นเสียหายไม่ทราบขนาด และ metric ของเส้นกริดจากสองรอบนั้นเชื่อถือไม่ได้เลย

แก้ยังไง: ปรับ MAX_LENGTH จาก 24,576 → 32,768 ในรอบ t03 แล้วปรับต่อเป็น 47,104 ในวันเดียวกัน และเขียนลำดับแผนสำรองไว้ใน train_t03.py ห้ามลด MAX_LENGTH เดี่ยวๆ อย่างชัดเจน

กฎที่ได้: วัดจำนวนโทเคนของตัวอย่างจริงที่ยาวที่สุดก่อนตั้งเพดานความยาวลำดับ และต้องมีการตรวจสอบยืนยันด้วย label ที่ถูกตัดไม่มีทางขึ้น error ให้เห็น มันแค่สอนขยะให้โมเดลไปเสียๆ เท่านั้น

สูง 2026-08-29

MAX_PIXELS ตอนเทรนกับตอน infer ไม่ตรงกัน (7680 vs 5120)

เกิดอะไรขึ้น: ตอน infer ใช้ MAX_PIXELS 5120 ในขณะที่ตอนเทรนใช้ 7680 — ทั้งสองค่าถูกแก้ไขให้ตรงกันเมื่อ 2026-08-29 เท่านั้น แล้วพอเปลี่ยนโมเดลอีก การแก้นี้ก็กลายเป็นไม่มีผลสำหรับ t04

ต้นเหตุจริง: สคริปต์เทรนกับสคริปต์ infer ต่างมีสำเนาค่าคงที่ประมวลผลภาพของตัวเอง ไม่มีแหล่งร่วมกัน จึงพี้ยนออกจากกันทุกครั้งที่มีการปรับค่าใดค่าหนึ่ง

เสียหายอะไร: การ infer ป้อนภาพเข้าโมเดลที่ความละเอียดจริงต่างจากตอนเทรน เป็นการลดคุณภาพแบบเงิบๆ ที่มองไม่เห็นในตัวอย่างใดๆ ที่ไม่ได้แยกเจาะจงตรวจเรื่องนี้

แก้ยังไง: ตั้งค่า infer MAX_PIXELS เป็น 7680 ให้ตรงกับตอนเทรน พร้อมเปลี่ยนชื่อ OUT_DIR/out-root จาก outputs_t03 เป็น t04

กฎที่ได้: ค่าคงที่ในการประมวลผลภาพต้องอยู่ในโมดูลรวมเดียวที่ทั้งสคริปต์เทรนและ infer เรียกใช้ ค่าใดก็ตามที่ต้องตรงกันข้ามสคริปต์ ห้ามมีอยู่สองที่เด็ดขาด

สูง 2026-08-30

ตั้งค่า decoder ที่สืบทอดมาจาก t03/Qwen ห้ามการตอบซ้ำที่ ground truth จริงๆ ต้องการ

เกิดอะไรขึ้น: decoder ตอน eval ยังมี `no\_repeat\_ngram\_size=8` และ `repetition\_penalty=1.15` สืบทอดมาจากยุค t03/Qwen ทั้งที่ ground truth จริงของแบบก่อสร้างมีการซ้ำ mark เยอะมากเป็นปกติ (B2×10, RB1×24) ตั้งค่าแบบนี้จึงไปกดผลลัพธ์ที่ถูกต้องทั้ง พิสูจน์แยกด้วย fixtest ว่าลบสองค่านี้ออกทำให้จำนวน element ที่สกัดได้เพิ่มจาก 0 เป็น 10 (แม้ recall จะยังเป็น 0% เพราะมีต้นเหตุแยกอีกตัวคือ crop_to_patches)

ต้นเหตุจริง: hyperparameter ของการ generate ถูกส่งต่อข้ามรุ่นโมเดล/งานโดยไม่มีการถามใหม่ว่าการกระจายผลลัพธ์เปลี่ยนไปหรือยัง ตัวกันการซ้ำ (anti-repetition) เป็น heuristic สำหรับ text generation ทั่วไป แต่ผิดโดยสิ้นเชิงสำหรับงานสกัดข้อมูลเชิงโครงสร้างที่ ground truth ต้องซ้ำกันจริงๆ

เสียหายอะไร: ซ้ำเติมปัญหา recall ที่แย่อยูแล้ว และถ้าไม่แก้จะบดบังหรือบิดเบือนการวัดผลในอนาคตทุกครั้ง เฉพาะปัญหานี้อย่างเดียวก็ทำให้การสกัด element เป็น 0 แล้ว

แก้ยังไง: นำไปแก้ต่อใน t05 แทนที่: ลบ `no\_repeat\_ngram\_size=8` ออก และลด `repetition\_penalty` จาก 1.15 เหลือ 1.0

กฎที่ได้: ห้ามสืบทอด hyperparameter ของ decoder/generation ข้ามรุ่นโมเดลหรือข้ามงานโดยไม่ตรวจสอบทีละตัวว่าเหมาะสมกับรูปแบบผลลัพธ์เป้าหมายหรือไม่ — ตัวกันการซ้ำต้องปิดสำหรับงานสกัดข้อมูลเชิงโครงสร้างที่ ground truth ซ้ำกันเอง

สูง 2026-08-30

AutoProcessor แบ่งภาพเป็น 13 tile ตอน inference แต่ตอน training ได้แค่ 1 tile — mismatch การเตรียมข้อมูลต่างกัน 13 เท่า

เกิดอะไรขึ้น: ระหว่างสืบหาสาเหตุปัญหา recall พบว่าตอน inference AutoProcessor แบ่งภาพแต่ละรูปเป็น 13 tile แต่ตอน training (เพราะขาด `crop\_to\_patches`) ได้แค่ 1 tile ต่อรูปเท่านั้น

ต้นเหตุจริง: ตอน training ใช้เส้นทาง mm_plugin ของ LLaMA-Factory ที่ปิดการแบ่ง tile แต่ตอน inference ใช้ HF AutoProcessor แบบ default ที่เปิดการแบ่ง tile — สอง preprocessing stack นี้ไม่เคยถูกเทียบกันเลยว่าทำงานเหมือนกันหรือไม่

เสียหายอะไร: ต่อให้แก้จุดอื่นครบแล้ว โมเดลก็ยังถูกป้อนข้อมูลในรูปแบบที่ไม่เคยเห็นตอนเทรนอยู่ดี รับประกันได้ว่าผลลัพธ์จะแย่ง

แก้ยังไ: บันทึกไว้เป็นป้จจัยซ้ำเติมในรายงานต้นเหตุ ส่วนการแก้จริง (ทำให้ preprocessing เหมือนกันทั้งสองฝั่ง) ถูกรวมเข้าไปในกฎการตรวจสอบของรอบ t05

กฎที่ได้: ต้องยืนยันว่า shape ของ tensor / จำนวน token ที่โมเดลได้รับตอน inference เท่ากับตอน training เป๊ะๆ — ปรึ้นท์และเทียบทั้งสองฝั่งจริง อย่าสมมติว่าสอง library เข้าใจตรงกัน

สูง 2026-08-31

hyperparameter 3 ตัวต่างกันแบบเสียบๆ ระหว่างสองแขน เพราะ yaml ไม่ได้ระบุค่าไว้ เลยไปใช้ค่า default ของ library

เกิดอะไรขึ้น: การตรวจสอบความเท่าเทียมพบว่า weight_decay (0.01 เทียบกับ default ของ HF ที่ 0.0), optim (adamw_8bit เทียบกับ default adamw_torch) และ lora_dropout ต่างกันระหว่างสองแขน เพราะ yaml ไม่ได้เขียนค่าพวกนี้ไว้เลย จึงไปรับค่า default แบบเสียบๆ ช่องโหว่ optim แบบเดียวกันนี้เกิดขึ้นใน t04 มาก่อนแล้วโดยไม่มีใครสังเกตเห็น

ต้นเหตุจริง: สมมติว่าค่าเท่ากันจากค่าที่เขียนไว้ในไฟล์ โดยไม่ได้ใส่ค่าที่ไม่ได้เขียนไว้และตรวจ default ของแต่ละ library เทียบกัน

เสียหายอะไร: สองแขนนี้จริงๆ แล้วเทียบกันไม่ได้เลย และช่องว่างเดียวกันนี้ได้ทำให้การเปรียบเทียบของ t04 เสียหายไปแล้วโดยไม่รู้ตัว

แก้ยังไ: เขียนทั้ง 3 ค่าลงไปตรงๆ ในทั้งสอง yaml ของ fold และยืนยันแล้วว่า parse ผ่าน

กฎที่ได้: เวลาเทียบ training stack สองตัว ต้องใส่ทุก hyperparameter ที่แต่ละ framework อาจใช้ค่า default แบบเสียบๆ ได้ แล้วเขียนค่านั้นลงไปตรงๆ ทั้งสองฝั่ง — การไม่ระบุค่าคือความแตกต่าง ไม่ใช่ความเหมือน

สูง 2026-08-31

Dry-run รอบแรก: transformers ที่ลงไว้ไม่รู้จักร architecture qwen3_5_moe

เกิดอะไรขึ้น: โหลดโมเดลไม่ผ่าน เพราะ transformers 4.57.6 ไม่รู้จัก architecture qwen3_5_moe

ต้นเหตุจริง: ไม่ได้ pin เวอร์ชัน framework ให้ตรงกับ architecture ของโมเดลตัวนั้นๆ

เสียหายอะไร: ถ้าไม่ได้ dry-run ก่อน จะพังตั้งแต่วันที่แรกของการรันจริงที่เสียเงิน

แก้ยังไ: Pin เป็น transformers==5.5.0 (ช่วงที่ใช้ได้คือ [5.2.0, 5.5.0] ขอบบนมาจากที่ unsloth เองประกาศไว้ว่า <=5.5.0)

กฎที่ได้: ต้อง pin เวอร์ชัน framework ให้ตรงกับ architecture ของโมเดลเป๊ะๆ และพิสูจน์ว่าโหลดได้จริงผ่าน dry-run ก่อนรันจริงทุกครั้ง

สูง 2026-08-31

unsloth ดึง torch build cu130 มาลงบน image ที่เป็น CUDA 12.8 ทำให้ JIT compile kernel พังกลางการเทรน

เกิดอะไรขึ้น: การเทรนพังตอน JIT compile kernel เพราะ nvrtc หา libnvrtc-builtins.so.13.0 ไม่เจอ ทั้งที่ไฟล์มีอยู่จริงแต่ไม่ได้อยู่ใน library path เพราะ unsloth ลง torch wheel แบบ cu130 บน image ที่เป็น CUDA 12.8

ต้นเหตุจริง: package manager เลือก CUDA build ผิดให้ไม่ตรงกับ base image และปัญหานี้จะโผล่มาตอน JIT compile ระหว่างเทรนเท่านั้น ไม่ใช่ตอน import

เสียหายอะไร: อาการพังโผล่มาหลังจากเริ่มรันไปแล้ว คือหลังจากเสียเงินไปแล้ว

แก้ยังไง: ชี้ LD_LIBRARY_PATH ไปที่ nvidia/cu13/lib แทนที่จะ downgrade torch

กฎที่ได้: หลังลง ML stack บนเครื่องเช่า ต้องเช็ค CUDA build ของ torch ตรงกับ CUDA ของ image จริง และบังคับให้มีขั้นตอนที่ต้อง JIT compile อย่างน้อย 1 ครั้งใน dry-run เพื่อให้บั๊กแบบนี้โผล่ก่อนรันจริง

สูง 2026-08-31

เช่าเครื่อง 2 เครื่องห่างกัน 2 ชั่วโมง ได้ peft คนละเวอร์ชัน ทำให้ layer ของ MoE สลับทิศทางการ

เกิดอะไรขึ้น: ตอน merge พบว่า LoRA factor ของ MoE layer ใน fold0 มีทิศทางการตรงข้ามกับ fold2/3 เพราะ gpu1 ได้ peft 0.18.1 ส่วนเครื่องที่เช่าทีหลังได้ peft 0.20.0 — เช่าห่างกันแค่ 2 ชั่วโมงและไม่ได้ pin เวอร์ชันไว้

ต้นเหตุจริง: ไม่ได้ pin เวอร์ชัน library ที่กำหนดรูปแบบไฟล์น้ำหนักที่บันทึกลงดิสก์ ทำให้เครื่องที่เช่าห่างกันแค่ไม่กี่นาทีก็ได้อะไรที่เวอร์ชันเลื่อนไปคนละแบบ

เสียหายอะไร: adapter เข้ากันไม่ได้แบบเสียบๆ ตอนจะเอามาเฉลี่ยารวมกัน ต้องเขียนตัวแปลง permutation ขึ้นมาใหม่ และพิสูจน์ว่า $\max|\Delta W_{old} - \Delta W_{new}^T| = 0.00e+00$

แก้ยังไง: เขียนตัวแปลง permutation ทำให้ merge สำเร็จ และบันทึกเป็นข้อ 18 ใน rule_of_tune ว่าต้อง pin ทุก library ที่กำหนดรูปแบบไฟล์

กฎที่ได้: ต้อง pin เวอร์ชันของ library ทุกตัวที่กำหนด layout ของไฟล์ผลลัพธ์ให้ตรงกันเป๊ะในทุกเครื่องที่รันขนานกัน

สูง 2026-08-31

โหลด adapter ที่ merge แล้วด้วย peft เวอร์ชันเก่าจะได้ผลลัพธ์ขยะแบบเสียบๆ

เกิดอะไรขึ้น: adapter destrier ที่ merge แล้วเขียนตามรูปแบบของ peft 0.20 ถ้าเครื่องมี peft 0.18.1 อยู่เดิม จะโหลดออกมาเป็นขยะแบบเสียบๆ โดยไม่มี error ใดๆ

ต้นเหตุจริง: รูปแบบการเขียนไฟล์ของ peft แต่ละเวอร์ชันต่างกัน แล้วถ้าเวอร์ชันไม่ตรงกันจะพังแบบเสียบๆ ไม่ error

เสียหายอะไร: การประเมินผลใดๆ ที่รันด้วย peft เวอร์ชันผิด จะได้ตัวเลขที่ดูเหมือนใช้ได้แต่ไม่มีความหมายเลย

แก้ยังไง: สร้าง venv แยกต่างหากที่ลง peft 0.20 ไว้เฉพาะสำหรับ smoke test

กฎที่ได้: ก่อนจะประเมินผลโมเดล ต้อง pin และตรวจสอบให้แน่ใจว่าใช้ library เวอร์ชันเดียวกับตอนที่สร้าง artifact นั้น — เวอร์ชันไม่ตรงแล้วโหลดผ่านแบบเสียบๆ อันตรายกว่าโหลดแล้ว error ทันที

สูง 2026-08-31

GPU-A รันไปโดยไม่ได้ผลลัพธ์อะไรเลย เสียเงินไปราว \$2.4 เพราะ worker ใช้ vanilla PeftModel ที่ไม่มีตัวกันการ generate พิเศษ

เกิดอะไรขึ้น: worker_page_raw_pratyad.py ของ GPU-A ใช้ AutoModelForImageTextToText + PeftModel แบบธรรมดา ซึ่งเข้ามากับ MoE LoRA แบบ 256-expert และไม่ได้ตั้ง repetition_penalty หรือ no_repeat_ngram_size ต่างจาก serve_purson.py หน้า 09 กับ 10 timeout ที่ 25 นาทีทั้งแบบปกติและแบบ xgrammar ไม่ได้ผลลัพธ์อะไรเลยแม้แต่คนเดียว

ต้นเหตุจริง: โปรเจกต์เดียวกันมีโค้ด inference สองเส้นทางที่แยกกันเดิน เส้นหนึ่งไม่ได้รับความรู้ในการตั้งค่าที่ส่งสมมาจากอีกเส้นเลย

เสียหายอะไร: เสียทั้งเงินราว \$2.4 และเวลาราว 2 ชั่วโมงของ GPU เข้า ได้ผลลัพธ์เป็นศูนย์

แก้ยังง: แทนที่จะเข้าเครื่องเพิ่ม เขียน pure_power_endpoint.py ยิง prompt แบบ raw-power ไปที่ endpoint ที่ GPU-B รันอยู่แล้ว ได้ผลลัพธ์ใกล้เคียงกันภายใน 133-364 วินาทีต่อหน้า แทนที่จะกิน 25 นาที

กฎที่ได้: ควรมีโค้ด inference เพียงเส้นทางเดียวในโปรเจกต์ ถ้ามีเส้นที่สอง ต้อง diff ค่า generation config เทียบกับตัวหลักก่อนจะเอาไปใช้เฟา GPU

สูง 2026-08-31

เมนูตัวรันงานเริ่มโมเดลผิดพลาด เพราะไปสืบทอดค่า default ของ adapter จากไฟล์อื่น

เกิดอะไรขึ้น: กดเลือกเมนูข้อ 1 แล้ว server เริ่มด้วย --adapter Sicilian44/t03 ซึ่งเป็นโมเดลเก่า เพราะ go.py เรียก `up` โดยไม่ใส่ argument เลย จึงไปหยิบค่า default ของ presentation.py มาใช้แทน มะขามเป็นคนจับได้

ต้นเหตุจริง: พังพาค่า default จากไฟล์อื่นแทนที่จะส่งพารามิเตอร์ให้ตรงๆ

เสียหายอะไร: เข้าการจอตเปิดใช้งานโมเดลผิดพลาด (instance 49410094) แต่จับได้ทัน เลยสลับให้ถูกได้ทันเวลา

แก้ยังง: เปลี่ยนชื่อ preset เป็น destrier ตามชื่อ adapter จริง ทำให้เมนูต้องถามเสมอว่าจะใช้โมเดลไหน บังคับส่ง --model ตรงๆ เสมอ ไม่พึ่งค่า default จากไฟล์อื่นอีก และแสดงชื่อโมเดลที่กำลังโหลดอยู่ตลอดเวลา

กฎที่ได้: ห้ามให้ตัวรันงานสืบทอดค่า default จากไฟล์อื่นสำหรับค่าที่กำหนดว่าจะรันอะไร ต้องส่งค่าให้ตรงๆ เสมอ และแสดงให้เห็นชัดๆ ในหน้าจอด้วย

สูง 2026-08-31

ตัวกันการพูดซ้ำ (repetition_penalty / no_repeat_ngram_size) ที่ใส่ไว้เพื่อความปลอดภัย กลับผิดสำหรับแบบก่อสร้าง

เกิดอะไรขึ้น: serve_purson.py รันด้วย repetition_penalty=1.15 และ no_repeat_ngram_size=8 แต่แบบก่อสร้างมีลักษณะซ้ำๆ โดยธรรมชาติ (เช่น กริดที่อ้างซ้ำ F7,C1 / F7,C1) โมเดลจึงถูกลงโทษตอนตอบถูกที่ต้องซ้ำ แล้วถูกดันให้แต่งค่าใหม่แทน กลายเป็นต้นเหตุของเอาต์พุตที่วิ่งไม่หยุด

ต้นเหตุจริง: ใส่ค่ากันการพูดซ้ำแบบทั่วไป (generic) มาโดยไม่เช็คว่าคำตอบที่ถูกต้องในโดเมนนี้เป็นแบบซ้ำๆ อยู่แล้ว ซึ่งเป็นบั๊กที่รู้จักกันดีของ Qwen3-VL (issue #1611)

เสียหายอะไร: รวมกับ greedy decoding แล้วทำให้ได้ recall 0% และเอาต์พุตวิ่งยาว 16,363 ตัวอักษรบนหลายภาษา หลังถอดทั้งสองตัวออก recall ขึ้นมาเป็น 11.8% (strict) / 43.5% (position)

แก้ยังง: ถอดทั้งสองค่าออก เปลี่ยนไปใช้ค่า sampling ที่ Qwen แนะนำ (temperature 0.7 / top_p 0.8 / top_k 20)

กฎที่ได้: ตัวกันการพูดซ้ำ (anti-degeneracy) ต้องปรับตามโดเมน ถ้าคำตอบที่ถูกต้องซ้ำกันโดยธรรมชาติ การลงโทษการซ้ำคือบั๊ก ไม่ใช่ตัวป้องกัน

สูง 2026-09-01

pass3 ยึด `cv\_mark` เป็นหลัก ทั้งที่โมเดลตอบฟิลต์นี้แค่ 9% ของเวลา

เกิดอะไรขึ้น: pass3 ต้องการ element ที่มี `cv\_mark` เพื่อสร้าง anchor แต่ Destrier เห็น `cv\_mark` แค่ 92 จาก 1,034 ตัวอย่างฝึก (9%) พอใช้งานจริง โมเดลแทบไม่เคยตอบฟิลต์นี้เลย ทำให้ทุกหน้าได้ anchor เป็น 0 และวัดค่าอะไรไม่ได้เลย

ต้นเหตุจริง: ออกแบบเลือกคีย์อ้างอิง (anchor) โดยไม่เช็คความถี่ของฟิลต์นั้นในข้อมูลฝึกจริงก่อน เลือกฟิลต์ที่ต้องพึ่งพาจากผลลัพธ์โมเดล จาก schema เนี่ยๆ ไม่ได้ดูพฤติกรรมโมเดลจริง

เสียหายอะไร: pass3 จะไม่ได้คำตอบอะไรเลยตอนใช้งานจริง ทั้งที่ตรรกะการ fit นั้นถูกต้องอยู่แล้ว

แก้ยังไง: เพิ่ม fallback ชื่อ `anchors\_by\_shape()` — จับคู่ 'จุดที่โมเดลให้ grid_ref' กับ 'จุดที่ CV เห็น' โดยดูรูปทรงการกระจายตัวในเชิงพื้นที่ (normalize จุดทั้งสองชุดให้อยู่ใน [0,1] แล้วหาจุดที่ใกล้กันแบบ mutual nearest neighbour) ปลอดภัยเพราะถ้าจับคู่ผิด ค่า residual จะพุ่งสูงและ `build\_transform` จะปฏิเสธเอง

กฎที่ได้: ก่อนพึ่งพาฟิลต์ใดก็ตามที่โมเดลเป็นคนตอบ ต้องวัดก่อนว่าฟิลต์นั้นปรากฏในข้อมูลฝึกจริงบ่อยแค่ไหน ฟิลต์ที่โมเดลตอบไม่แน่นอนต้องมีเส้นทางสำรอง (fallback) และ fallback นั้นต้องตรวจสอบตัวเองได้ (จับคู่ผิดต้องล้มเหลวชัดเจน ไม่ใช่ให้ตัวเลขที่ดูสมเหตุสมผลแต่ผิด)

สูง 2026-09-01

การ export BOQ ใช้ document.createElement เพื่อ escape HTML — รันนอกเบราว์เซอร์ไม่ได้ และไม่ escape เครื่องหมายคำพูดด้วย

เกิดอะไรขึ้น: `boq-export-variants.js` เขียน `escapeHtml` โดยใช้ `document.createElement` ทำให้ export variant ทั้ง 4 แบบรันนอกเบราว์เซอร์ไม่ได้เลย (ทดสอบอัตโนมัติไม่ได้ สร้างเอกสารฝั่งเซิร์ฟเวอร์ก็ไม่ได้) และไม่เคย escape `"` หรือ `"` เลยด้วย เพราะ `textContent` ไม่ทำแบบนั้นให้ — เป็นช่องโหว่แทรกโค้ด (injection) ถ้าเอาผลลัพธ์ไปวางในแอตทริบิวต์ HTML

ต้นเหตุจริง: การยืม DOM มาใช้เป็นเครื่องมือ escape ทำให้ตรรกะที่ควรเป็น string ล้วนๆ ไปผูกติดกับ runtime ของเบราว์เซอร์ และรับสืบทอดความหมายการ escape ของ `textContent` มาด้วย (ครอบคลุมแค่รับข้อความ ไม่ครอบคลุมบริบทแอตทริบิวต์) บั๊กกลุ่มเดียวกันนี้เคยถูกแก้ที่อื่นในโค้ดเบสมาแล้ว (`escAttr` ในโมดูล planner) แต่ไม่ได้เอามาใช้ที่นี่

เสียหายอะไร: บล็อกการทดสอบอัตโนมัติของ export ทั้งหมด และการสร้างเอกสารฝั่งเซิร์ฟเวอร์ทำไม่ได้ แล้วยังมีช่องโหว่ XSS/แอตทริบิวต์ตกซ่อนอยู่ในเอกสารที่ export ออกไป

แก้ยังไง: เปลี่ยนไปใช้การ escape แบบ string replacement ล้วนๆ และเพิ่ม `"` กับ `"` เข้าไปในชุดตัวอักษรที่ต้อง escape

กฎที่ได้: การ escape ต้องเป็นฟังก์ชัน string ล้วนๆ เสมอ ห้ามใช้ DOM — และต้องครอบคลุมบริบทแอตทริบิวต์ (`"` และ `"`) ไม่ใช่แค่รับข้อความ เมื่อแก้บั๊กกลุ่มนี้ที่จุดหนึ่งแล้ว ให้ grep หา helper ตัวเดียวกันที่ implement ซ้ำที่อื่นด้วย

เรื่องที่เหลือในหมวดนี้ (43)

เอกสารของ pipeline รุ่นที่ตายไปแล้ว ยังค้างอยู่ในรีโปกอีกเป็นเดือนหลังถูกแทนที่

กลาง 2026-07-01

เวลาเปลี่ยน pipeline ใหม่ ให้เก็บกวาดเอกสารของรุ่นเก่าไปพร้อมกันในการแก้ครั้งเดียวกัน โดยเฉพาะ quickstart เพราะคนใหม่เข้ามาจะอ่านอันนั้นเป็นอันแรกเลย เอกสารกู้คืนจาก git history ได้อยู่แล้ว ลบหรือย้ายไปเก็บได้เลยไม่ต้องลังเล ดีกว่าปล่อยคู่มือที่พาคนหลงทางทิ้งไว้

URL รูปใน task ของ Label Studio ที่ generate ออกมาชี้ไปที่ raw.githubusercontent รูปเลยโหลดไม่ได้จนกว่าจะ push รีโปกขึ้นไปก่อน

กลาง 2026-07-06

เวลาตัว generator ปลอ่ย URL ภายนอกออกมา ให้มันลองยิงเช็คสักอันว่าเข้าถึงได้จริงก่อนจะบอกว่าสำเร็จ การไปแอบพึ่งเงื่อนไข 'ต้องมีคน push รีโปก่อน' คนที่เจอปัญหาจะเป็นคนตรวจ ไม่ใช่คนเขียน

ตัวไดอารี่งานเองมี path เก่าค้างอยู่ 4 จุด และมีงานที่ทำเสร็จแล้วยังค้างอยู่ในรายการรอทำ

กลาง 2026-07-06

จัดโครงสร้างไดเรกทอรีใหม่เมื่อไหร่ ให้ grep หา path เก่าใน log และเอกสารทันทีทุกครั้ง และเคลียร์รายการค้างให้จบทุกท้ายเซสชัน รายการค้างที่ไม่ตรงความจริงจะพาคนเข้าใจผิด แยกว่าไม่มีรายการเลย

สร้าง Label Studio ไปได้แค่ครึ่งวงจร ไม่มีอะไรแปลงผลตรวจที่ export ออกมากลับเข้าเป็น annotated/*.json

กลาง 2026-07-06

สร้างเส้นทางบางที่สุดที่วิ่งครบวงจรก่อน (เข้าหนึ่ง task ออกหนึ่งการแก้ไข) แล้วค่อยไปขัดฝักการแสดงผล เครื่องมือตรวจงานที่ผลลัพธ์กลืนกลับเข้าระบบไม่ได้ ให้ข้อมูลที่แก้แล้วเป็นศูนย์ ต่อให้หน้าตาดีแค่ไหนก็ตาม

ไฟล์ label-studio-tasks.js ฝั่ง Constaint เป็นเวอร์ชันที่รู้อยู่แล้วว่าพัง มั่นอ่าน path qwen-output ที่แชร์กันอันเดียว แทนที่จะอ่านแยกตามบ้าน

กลาง 2026-07-07

ก่อนลบอะไรที่คิดว่าเป็น 'ไฟล์ซ้ำ' ให้ diff ทุกไฟล์ก่อน ไฟล์ที่เนื้อหาต่างกันไม่ใช่ไฟล์ซ้ำ มันคือการแก้ที่หายไป หรือบั๊กที่พื้นดินชีพ และต้องรู้ให้ได้ก่อนว่าเป็นอันไหนก่อนจะกดลบ

config ของ Label Studio ทั้ง 5 ชุดปล่อยออกไปโดยฟิลด์ส่วนหัวเป็นข้อความอ่านอย่างเดียว คนตรวจเลยแก้ house/page/sheet/pattern ไม่ได้

กลาง 2026-07-07

ฟิลด์ไหนที่คนตรวจอาจต้องแก้ ต้องแก้ไขได้ ไม่ใช่แค่แสดงผล และเวลาทำให้ฟิลด์แก้ไขได้ ให้เอา label ไว้นอกช่องกรอก เพราะ label ที่ฝังอยู่ในกล่องจะกลายเป็นส่วนหนึ่งของข้อมูลที่ถูกบันทึก

เครื่องมือใช้ทำงานไม่มี Python เลย ทั้งที่สคริปต์หลักของ pipeline เป็น Python

กลาง 2026-07-10

ถ้าสคริปต์หลักของรีโปเขียนด้วยภาษาที่เครื่องทำงานรันไม่ได้ ก็ให้ติดตั้งภาษานั้นซะ หรือไม่ก็เลิกบอกว่าสคริปต์พวกนั้น 'ใช้งานอยู่จริง' การแก้ไขที่รันไม่ได้ คือการแก้ที่ไม่มีใครตรวจสอบได้

'lora_dropout=0.05' ใช้ไม่ได้กับตัวห่อ MoE LoRA ของ Unsloth

กลาง 2026-07-21

เวลา fine-tune โมเดล MoE ต้องตรวจสอบทุก hyperparameter ของ LoRA กับ code path ของ MoE โดยเฉพาะ ค่า default ที่ใช้กับโมเดลแบบ dense ใช้ข้ามกันไม่ได้

ตั้งค่า MAX_LENGTH เป็น 10,240 จากการเดา ทั้งที่ stats.json บอกว่าค่าสูงสุดจริงคือ 8,890

กลาง 2026-07-21

ค่าคงที่เกี่ยวกับความจุ (sequence length, batch size, buffer size) ต้องมาจากสถิติของ dataset ที่วัดมาแล้วจริงเท่านั้น ห้ามใช้การประมาณเอาเองบวกด้วย 'ระยะเผื่อความปลอดภัย'

การเพิ่ม MAX_LENGTH เป็น 24576 ทั้งที่รู้อยู่แล้วจะทำให้เสีย OOM อีกครั้งบนการ์ดที่เหลือพื้นที่แค่ ~100MB

กลาง 2026-07-24

เมื่อการแก้ปัญหาหนึ่งจะเพิ่มแรงกดดันด้านทรัพยากร ต้องเขียนแผนสำรอง (fallback) เป็นลำดับขั้นไว้ล่วงหน้าก่อนรันงานที่เสียเงินจริง และต้องระบุชัดว่าห้ามใช้ fallback ตัวไหนที่จะย้อนกลับไปทำลายการแก้ไขนั้น

การแก้ MAX_LENGTH รู้อยู่แล้วดันงานเข้าใกล้ OOM โดยรอบก่อนเหลือพื้นที่ว่างแค่ ~100MB จาก 95GB

กลาง 2026-07-24

เมื่อยอมรับความเสี่ยงที่รู้อยู่แล้ว ต้องเขียนลำดับการแก้ไขล่วงหน้าและระบุชัดว่าห้ามทำอะไร เพื่อไม่ให้หยิบวิธีแก้ที่ผิดมาใช้ตอนกดดัน

llama-server ค้างด้วยข้อความ 'non-consecutive token position' ตั้งแต่รัน inference รอบแรก จนต้องสั่ง kill ทั้ง

กลาง 2026-07-28

จุดพลิกที่ต้องใช้ตอนสั่งรันของทุกตัว serving ไว้คู่กับโมเดล และถ้ามันค้างโดยไม่มี output ออกมาเลย ให้ถือว่าตั้งค่าผิดแล้วรีบไปหาสาเหตุทันที ไม่ใช่นั่งรอเพราะคิดว่ามันทำงานช้า

ตั้ง --ctx-size 8192 ทำให้ผลลัพธ์ขาดกลางประโยค เพราะ prompt + รูป + คำตอบ ต้องใช้ 8,960 token

กลาง 2026-07-28

คำนวณ context ที่ต้องใช้จาก prompt + token ของรูป + ความยาวคำตอบที่คาดไว้ ก่อนจะตั้งค่า และถ้าผลลัพธ์ขาดกลางประโยค ให้สงสัยเรื่อง context size ก่อนเสมอ อย่าเพิ่งไปโทษว่าโมเดลไม่ดี

Qwen3-VL ตั้งค่าเริ่มต้นไว้ที่ 16,384 visual token ต่อรูป เกินงบประมาณ 3.2 เท่า และรู้ตอนเปิดไฟล์ config จริงบนเครื่องเช่า

กลาง 2026-07-28

โมเดลตระกูลใหม่ทุกตัว ให้เปิดดู config ของ preprocessor/vision ตัวจริงบนเครื่องจริง แล้วเทียบค่าเริ่มต้นกับบงที่ตั้งไว้ ค่าเริ่มต้นของผู้ผลิตคือตัวแปรที่เราไม่ได้ตั้งเอง

llama.cpp พิมพ์แฟล็กที่ต้องใช้แก้ (--image-min-tokens 1024) ออกมาให้เลย แต่จนจบรอบงานก็ยังไม่ได้ใส่

กลาง 2026-07-29

วิธีแก้ที่ไม่มีต้นทุนและผู้ผลิตบอกชื่อมาให้ตรงๆ ให้ใส่ทันทีที่อ่านเจอ โดยเฉพาะก่อนจะรันเทียบผล ไม่งั้นทุกการวัดหลังจากนั้นจะยังมีตัวแปรกวนที่รู้อยู่แล้วติดไปด้วย และต้องวัดใหม่หมด

CLAUDE.md เขียนว่า Structure Planner เป็นแท็บตายที่ไม่มีใครเรียก หึ่งที่ต่อสายไปเรียบร้อยแล้ว

กลาง 2026-08-18

ห้ามพูดว่าโค้ดตรงไหนถูกเรียกหรือไม่ถูกเรียกโดยอ่านจากเอกสารอย่างเดียว ต้อง grep หาจุดเข้าจริงก่อนเสมอ และโน้ตแนวว่า 'X ตายแล้ว/ไม่มีใครเรียก' ที่เก๋เกินไม่ก็สับสนให้ถือว่ายังไม่ได้พิสูจน์จนกว่าจะเช็คใหม่

หัวข้อ op4 ใน README กับ op04 SKILL.md เป็นคู่ที่ต้อง sync กันด้วยมือ ซึ่งยังงี้ก็จะเพี้ยน

กลาง 2026-08-21

เนื้อหาที่รู้อยู่แล้วว่าซ้ำกันสองที่ ต้องมีทดสอบจับความเพี้ยน ไม่ใช่โน้ตขอให้คนช่วยจำ ให้เพิ่มตัวตรวจว่าหัวข้อที่แชร์กันของสองไฟล์เหมือนกันปะ แบบเดียวกับที่โปรเจกต์นี้ป้องกันกฎอื่นๆ อยู่แล้ว

หน้า 22 ค้างเกิน 45 นาทีบน GPU เช่า เพราะใช้ greedy decoding ล้วนๆ

กลาง 2026-08-24

เอาวิธีแก้ decoding ที่พิสูจน์แล้วติดไปกับ runner script ตัวใหม่ทุกตัวตั้งแต่ตอนสร้าง runner ตัวใหม่ห้ามส่งออกไปโดยไม่มีการ repetition_penalty และตัวกันค้างเด็ดขาด

คำแนะนำการจูนของทีปรึกษาชี้ไปที่ max_frames=768 ซึ่งเป็นค่าสำหรับวิดีโอที่โปรเจกต์นี้ไม่เคยใช้เลย

กลาง 2026-08-25

ก่อนแก้ค่า config ตัวไหน ให้ grep ในโค้ดเพื่อพิสูจน์ว่าเส้นทางโค้ดที่อ่านค่านั้นทำงานจริงในงานของเรา ปุ่มที่ไม่มีใครอ่านค่า ก็ไม่ใช่ปุ่ม

มีเลขพิเศษตัวที่สองที่ผูกกับความละเอียดภาพ (ความยาวคานขึ้นต่ำ 100px) ฮาร์ดโค้ดไว้โดยไม่มี CLI ให้ปรับ ต่างจากค่า gap 21px ที่อยู่ข้างๆ

กลาง 2026-08-25

พอเจอค่าคงที่ตัวหนึ่งที่ขึ้นกับการ scan ให้ grep หาพี่น้องของมันในไฟล์เดียวกันแล้วแก้ไปพร้อมกันใน commit เดียว ค่าคงที่ประเภทเดียวกันมักพึ่งพาร่วมกัน

รูปเปรียบเทียบ template โดนตัด F2 หายไป เพราะฮาร์ดโค้ดความสูง canvas ไว้

กลาง 2026-08-25

คำนวณขนาด layout จากเนื้อหาจริง อย่าฮาร์ดโค้ดจากตัวอย่างเดียว และให้เปิดไฟล์ผลลัพธ์ดูด้วยตาทุกครั้ง บั๊กตอน render มองไม่เห็นจากการตรวจระดับโค้ด

BEAM_GAP=21px ถูกคาลิเบรตจาก DPI และมาตราส่วนของ scan ชุดเดียว

กลาง 2026-08-25

ค่าคงที่ที่อยู่ในหน่วยพิกเซลต้องจดไว้ว่าวัดที่ DPI และมาตราส่วนเท่าไร และต้องเปิดเป็นปุ่มให้ปรับได้ การตรวจไม่เจออะไรเลยแบบเสียบๆ เป็นความพังที่แย่ที่สุด ให้ใส่ตัวตรวจที่เตือนเมื่อ detector ไม่เจออะไรเลยบนหน้าที่ควรจะมี

การทดสอบ A/B ของ FINETUNE_VISION ไม่เคยได้รัน เพราะติดเงื่อนไข GGUF export ที่เลิกใช้ไปแล้ว

กลาง 2026-08-25

พอเลิกใช้เงื่อนไขข้อไหน ให้ไล่ลิสต์กันทีว่เคยตัดสินใจอะไรไปบ้างเพราะเงื่อนไขนั้น ตัวขวางที่หมดอายุแล้วยังขวางต่อไปแบบเสียบๆ ได้

ใส่ glossary ภาษาไทยเข้าไปทุก subtask ทำให้ความยาว sequence เกิน MAX_LENGTH และจะบังคับให้ต้องเพิ่ม VRAM

กลาง 2026-08-26

วัดการกระจายความยาว sequence ทั้งก่อนและหลังทุกครั้งที่เปลี่ยน prompt และตัด subtask ที่การเปลี่ยนนั้นช่วยอะไรไม่ได้ออกไป ใส่ของเพิ่มใน prompt แบบเลือกเฉพาะที่ ไม่ใช่ใส่เหมือนกันหมด

prompt เรื่อง gridline โตขึ้นสะสม ~1.5k ตัวอักษรในวันเดียว เข้าใกล้เพดาน MAX_LENGTH

กลาง 2026-08-29

ตั้งบตัวอักษร/token ต่อ prompt ของแต่ละ subtask ไว้ตายตัว แล้วเช็คทุกครั้งที่แก้ การปรับปรุงมันสะสมขึ้นเสียบๆ แต่เพดานไม่ได้ขยับตาม

จำนวน epoch เปลี่ยนสามรอบ (3→5→4→3) ด้วยเหตุผลที่ไม่ใช่เชิงเทคนิค ทำให้ต้องเริ่มเทรนใหม่สองครั้งและลบผลลัพธ์ทั้งสองรอบ

กลาง 2026-08-30

ลือกค่า hyperparameter ที่กระທค่าใช้จ่ายให้หนึ่งก่อนกดรันงานที่ต้องจ่ายเงิน และตั้งค่าจากสัญญาณที่วัดได้ (loss curve / dry-run) ถ้าจำเป็นต้องเปลี่ยนหลังกดรัน ให้เปลี่ยนก่อนเผา GPU ไปมาก และบันทึกไว้ด้วยว่าไม่ใช่การตัดสินใจเชิงเทคนิค

cutoff_len / image_max_pixels / lora_rank ถูกปล่อยเป็นค่า default ที่ยกมาจากโมเดลคนละตระกูล โดยไม่ตรวจสอบ

กลาง 2026-08-30

ถ้ามีรายการที่ติดธงว่า 'ต้องตรวจสอบก่อนรันจริง' ให้ระบุให้ชัดว่าต้องวัดอะไรถึงจะถือว่าเคลียร์ — dry-run ที่เช็คแค่ VRAM กับ loss ไม่ได้ปลดค่าเตือนเรื่องพารามิเตอร์ความละเอียดภาพ

ความยาว sequence เกิน model_max_length ของ tokenizer (17773 > 8192) — ไม่รู้ว่า label ถูกตัดหึ่งตอนเทรนหรือเปล่า

กลาง 2026-08-30

วัดการกระจายความยาว token ของ dataset เทียบกับ model_max_length ของ tokenizer และค่า cutoff_len ที่ตั้งไว้ ก่อนเทรน แล้วใส่ assert ว่าไม่มีตัวอย่างไหนล้นแบบเสียบๆ

onstart.sh ของ Courser สิม TORCH_CUDA_ARCH_LIST ที่ของ t02 มีอยู่แล้ว

กลาง 2026-08-31

สคริปต์ bootstrap ตัวใหม่ ให้ diff กับเวอร์ชันล่าสุดที่รู้ว่าใช้ได้ อย่าเขียนใหม่จากความจำ

dry-run รอบ 2: peft เรียก symbol ของ torchao ที่ torchao ถอดออกไปแล้ว

กลาง 2026-08-31

เมื่อ dependency สองตัวตีสัน ให้เช็คก่อนว่าตัวที่มีปัญหาถูกใช้งานจริงในคอนฟิกนี้ไหม ก่อนจะไปไล่ pin เวอร์ชัน — บ่อยครั้งการถอดหึ่งเป็นวิธีที่เล็กกว่า

ตั้งชื่อ env var ตัวใหม่ PYTORCH_ALLOC_CONF ไว้อย่างเดียว แต่ error message ตอน OOM ของ torch เองยังเรียกชื่อตัวเก่าอยู่

กลาง 2026-08-31

เวลา library เปลี่ยนชื่อ environment variable ให้ตั้งทั้งชื่อเก่าและชื่อใหม่ควบกันไปก่อน จนกว่าชื่อเก่าจะหายไปจาก message ของ library เองจริงๆ

ฝั่ง client ส่งค่า temperature 0.7 แทนที่จะเป็น 0 ซึ่งไม่มีผลตอนนี้เพราะฝั่ง server ทับค่าไว้ก่อน แต่ถ้าวุ่นไหน server เปลี่ยนพฤติกรรมจะเป็นหายนะทันที

กลาง 2026-09-01

ห้ามปล่อยให้สองส่วนของระบบมีค่าพารามิเตอร์สำคัญไม่ตรงกัน โดยอาศัยว่าตอนนี้อีกฝั่งทับค่าให้อยู่แล้ว ต้องทำให้ทั้งสองฝั่งตรงกันจริงๆ เพื่อให้เปลี่ยนฝั่งไหนก็ได้อย่างปลอดภัย

วาง XML ของ Label Studio แล้วพัง เพราะถือปมาตั้งแต่บรรทัดที่ 2 ทำให้ root tag `<View>` หายไป

ต้า 2026-07-06

เวลาวาง config แบบ paste มือ ให้แจ้งจำนวนบรรทัดที่ควรจะมีไว้ด้วย แล้วเช็คหลังวางทุกครั้ง error แบบ 'extra content' ของ XML ส่วนใหญ่มักมาจาก root tag หายหรือซ้ำ ไม่ใช่ที่บรรทัดที่ parser บอก

หัวข้อ spec ยังเขียนวันที่ 2026-07-05 ทั้งที่ถูกแก้ไขต่อเนื่องมาถึง 2026-07-06

ต้า 2026-07-06

ทุก spec ที่ยังแก้ไขอยู่ต้องมีบรรทัด 'แก้ไขล่าสุด' และอัปเดตพร้อมๆ กับตอนแก้ไขเนื้อหาเสมอ ห้ามฝังวันที่ไว้ในชื่อไฟล์ของเอกสารที่ยังพัฒนาต่อ เพราะชื่อไฟล์จะกลายเป็นข้อมูลเท็จที่แก้ทีหลังไม่ได้แล้วถ้ามีใครอ้างอิงไปแล้ว

ไฟล์ material_list.xml ยังใช้ layout แบบเก่าก่อนยุคของมะขาม ไม่สอดคล้องกับ config ไฟล์อื่นๆ

ต้า 2026-07-09

เวลาต้อง config N ไฟล์ให้ทำงานเหมือนกัน ต้องแก้ไขพร้อมกันเป็นชุด แล้ว diff เทียบกันหลังแก้ไขเสร็จ — ไฟล์พี่น้องที่ดูแลด้วยมือจะค่อยๆ เพี้ยนออกจากกันโดยที่มองแยกไฟล์ไม่เห็น ต้องเทียบเคียงกันถึงจะเห็น

ฟิลด์ header ถูกวางไว้ในคอนสแตนท์ที่ปึกหมุดไว้ กินพื้นที่ที่ควรสงวนไว้สำหรับตัวแบบก่อสร้าง

ต้า 2026-07-09

ใน UI ติดป้ายแบบสองบานที่มีบานหนึ่งปึกหมุด บานที่ปึกหมุดควรมีแค่สิ่งเดียวคือชิ้นงานที่กำลังอ่าน อย่างอื่นทั้งหมดต้องไปอยู่ฝั่งที่เลื่อนได้

README ของ production ตัวใหม่ที่เพิ่งส่งมอบ มีคำเตือนที่กลายเป็นเท็จไปแล้วในเซสชันเดียวกันที่เขียนมันขึ้นมา

ต้า 2026-07-10

เมื่อการแก้ไขโค้ดในเซสชันเดียวกันทำให้เอกสารที่เขียนไว้ก่อนหน้ากลายเป็นเท็จ ต้องแก้เอกสารในรอบเดียวกันเลย อย่าหวังพึ่งการตรวจสอบรอบหลังที่ไม่เกี่ยวข้องมาจับให้

คำสั่งใน README ใช้ ``python`` แต่ Docker image ที่เขามีแค่ ``python3``

ต้า 2026-07-21

ทุกครั้งที่ล็อกอินเข้า image ที่เข้าใหม่ ให้ตรวจสอบพื้นฐานก่อน (``python`/`python3``, pip, CUDA, พื้นที่ดิสก์) แล้วอัปเดต runbook ก่อนรันอะไรที่มีค่าใช้จ่าย

ค่า default ของ `--adapter` ใน `eval_fields.py` ชี้ไปที่ `outputs/lora` ซึ่งไม่มีอยู่จริง

ต้า 2026-07-28

แก้ default ที่พังได้ แต่ห้ามแตะโค้ดวัดผลที่ทั้งสองฝั่งของการเปรียบเทียบใช้ร่วมกันเด็ดขาด การเปลี่ยนไม่บรรทัดกลางการทดลองจะทำลายความสามารถในการเปรียบเทียบผลลัพธ์

ตั้งค่า `n_gpu_layers` ไว้ตายตัวที่ 999 ทำให้ llama.cpp ยกเลิกการคำนวณจัดสรรหน่วยความจำของตัวเองแทนที่จะเลือกวิธี split เอง

ต้า 2026-07-29

อย่าบังคับค่า `n_gpu_layers` บนเครื่องที่หน่วยความจำไม่พอโหลดโมเดลได้ ปล่อยให้ runtime จัดสรรเอง และควรอ่านบรรทัด abort/warning ที่ loader พิมพ์ออกมาก่อนเริ่มงานที่ใช้เวลาหลายชั่วโมง — log ตัวนี้มีคำตอบทั้งเรื่องความเร็วและเรื่องวิธีแก้ปัญหานั้นแล้ว

ข้อความของโหวของสเปก t03 ถูกเขียนซ้ำไว้ใน README สองไฟล์พร้อมกัน ซึ่งจะเพี้ยนตามกันไม่ทัน — เพิ่งมาเจอตอนจัดโพลเดอร์ใหม่

ต้า 2026-08-21

เอกสารก็ควรใช้กติกาห้ามเขียนซ้ำแบบเดียวกับที่ใช้กับ prompt — เรื่องเดียวควรอยู่ไฟล์เดียว ถ้าสองไฟล์ต้องพูดเรื่องเดียวกัน อีกไฟล์ควรลิงก์ไปแทนที่จะก๊อปปี้มา

ช่องว่างระหว่าง **template** ของคาน 21 พิกเซล ถูกฝัง **hardcode** ไว้สำหรับความละเอียดสแกนและสเกลค่าเดียวเท่านั้น

ต้า 2026-08-25

เวลาค่าคงที่ถูกวัดเป็นพิกเซลจากภาพสแกนภาพเดียว ต้องให้มันคำนวณจากสเกลของแบบตอน runtime หรือไม่ก็ทำเป็นพารามิเตอร์ที่ต้องระบุและมีเอกสารกำกับ — อย่าปล่อยเป็นค่า default ที่มองไม่เห็นแล้วไปพึ่งที่อื่นแบบเงิบๆ

ค่า **default FOLD** ของสคริปต์เทรนเป็นค่าว่าง ชี้ไปที่ไฟล์ **train/val** แบบรวมที่เพิ่งถูกลบไป

ต้า 2026-08-31

เวลาลบไฟล์ ให้ grep หาทุก default ที่ยังชี้ไปที่ไฟล์นั้น — fallback ที่ชี้ไปยัง path ที่ถูกลบไปแล้วคือกับระเบิด ไม่ใช่ตาข่ายนิรภัย

เครื่อง การัดจ่อเช่า และ ค่าใช้จ่าย

เครื่องเช่า โควตา เน็ต และเงินที่เสียไปโดยไม่ได้อะไร

ทั้งหมด 124 เรื่อง · วิกฤต 7 · สูง 54 · กลาง 46 · ต่ำ 17

เรื่องที่ต้องอ่านให้ครบ (61)

วิกฤต 2026-07-15

additional_bars ถูกบันทึกผิดพลาดหน้าของคาน — บั๊กตัวเดียวกันนี้เจอครบทั้งห้าบ้าน

เกิดอะไรขึ้น: เหล็กเสริมถูกบันทึกผิดพลาดหน้าแล้วซ้ำเล่า: คาน B2 ของบ้าน 2 ที่ 'หยุดที่ L/8' บันทึกว่าอยู่ด้านบน แต่จริงๆ อยู่ด้านล่าง (2→4); การต่อทาบของ B2X บันทึกว่าอยู่กลางช่วง แต่จริงๆ อยู่ด้านบน (2→3); บ้าน 3 ที่ B3X และ B4; บ้าน 4 ที่ B4, B4A, B4X, B5; บ้าน 5 ที่ B2X, B3X, B4, B5 เจอได้ก็ต่อเมื่อซูมเข้าไปในรูปตัดแล้วนับจุดเหล็กจริง แทนที่จะเชื่อป้ายที่พิมพ์ไว้

ต้นเหตุจริง: การสกัดข้อมูลไปเชื่อข้อความบนป้ายและทิศทางที่เส้นขีดดูเหมือนจะชี้ไป แทนที่จะนับจุดเหล็กจริงในรูปตัด ตำแหน่งของป้ายบนแบบไม่ได้บอกอย่างน่าเชื่อถือว่าเหล็กอยู่หน้าไหน

เสียหายอะไร: การกระจายเหล็กบนคานหลักผิดในทุกบ้านที่รีวิว เป็นความผิดพลาดของปริมาณเชิงโครงสร้างโดยตรง บางกรณีจำนวนเหล็กของหน้าหนึ่งเกือบเป็นสองเท่าหรือเหลือครึ่งเดียว

แก้ยังไ: ซูมเข้าไปในภาพรูปตัดคานแล้วนับจุดเหล็ก จากนั้นรวม additional_bars เข้าหน้าที่ถูกต้องของ main_bar แล้วส่งต่อไปยัง `specs{}` ในผังคาน บันทึกเป็นกตีกถาถาวรว่า ให้ตรวจ `additional\_bars[].position` เทียบกับเส้นชี้จริงเสมอ ห้ามเชื่อป้ายที่พิมพ์ไว้

กฎที่ได้: การระบุว่าเหล็กอยู่หน้าไหน ให้นับจุดในภาพรูปตัดที่ซูมแล้ว — ห้ามเชื่อป้ายที่พิมพ์หรือทิศที่เส้นขีดดูเหมือนจะชี้ไป และเมื่อบั๊กโผล่ในสองบ้าน ให้กวาดทุกบ้านทันที ไม่ใช่รอไปเจอทีละบ้าน

วิกฤต 2026-07-19

การ 'แก้' ตารางราคาข้ามบ้าน กลายเป็นการอ่านผิดเสียเอง ทำให้ไฟล์เสียไป 4 ไฟล์ก่อนจะย้อนกลับ

เกิดอะไรขึ้น: วันที่ 2026-07-16 การตรวจข้ามบ้านสรุปว่าค่าของบ้าน 3 (2,945,000/2,034,000) ถูก แล้ว 'แก้' ไฟล์ของบ้าน 2 (2,145,000/2,035,000) ให้ตรงกัน พอถึง 2026-07-19 ซูม 8 เท่าแล้วเทียบกับตัวเลขอ้างอิงบนแผ่นเดียวกัน พบว่าทั้งสี่สำเนาตรงกันที่ 2,145,000/2,035,000 — บ้าน 2 ถูกมาตลอด ที่ผิดคืออ่าน ๑ เป็น ๙ ตอนซูมเข้าไป

ต้นเหตุจริง: ตัวเลขถูกตัดสินแบบโดดๆ ที่ระดับซูมสูง แทนที่จะเทียบกับตัวเลขตัวเดียวกันที่รู้ค่าอยู่แล้วในที่อื่นบนแผ่นเดียวกัน การซูมสูงทำให้รูปตัวอักษรเสียคุณภาพและตัดบริบทรอบข้างที่ช่วยแยกแยะออกไป

เสียหายอะไร: ค่าที่ถูกต้องถูกเขียนทับด้วยค่าที่ผิดข้ามไฟล์ของหลายบ้าน แล้วต้องย้อนกลับใน 5 ไฟล์ และมีสองวันที่เชื่อตัวเลขที่ผิดอยู่ การกวาดรอบเดียวกันนี้เจอค่าที่ผิดจริงๆ อีก (#1, #2, #4, #8 และขนาดแบบบ้าน 3/8) ที่รอบก่อนหน้ามองข้ามไป

แก้ยังไ: ย้อนการแก้ของวันที่ 16 ก.ค. แล้วแก้ #1=1,020,000, #2=1,460,000, #4 ราคาปูพื้น=1,824,000, #8 ราคาปูพื้น 1,662,000→1,622,000, #10 ราคาปูพื้น 2,235,000→2,035,000 และยืนยันว่าตอนนี้ตารางตรงกันครบทั้งห้าบ้าน

กฎที่ได้: อย่าตัดสินเลขไทยแบบโดดๆ — ให้เทียบกับตัวเลขตัวเดียวกันที่รู้ค่าอยู่แล้วบนแผ่นเดียวกันที่ระดับซูมเท่ากันเสมอ และเมื่อการตรวจข้ามแหล่งพบว่าไม่ตรงกัน ให้ตรวจสอบทั้งสองฝั่งใหม่ก่อนจะไปเขียนทับฝั่งไหน

HuggingFace write token ถูกวางลงในแชท และจนจบวันก็ยังไม่ได้เพิกถอน

เกิดอะไรขึ้น: token ของ constistant-tune_t02 บน HuggingFace ซึ่งมีสิทธิ์ repo.write ถูกวางลงในบทสนทนา มีการจดไว้ว่าจะเพิกถอนหลัง Phase 8 แต่พอถึงตอนปิดไดอารี่ ก็ยังขึ้นสถานะว่ายังไม่ได้ทำ

ต้นเหตุจริง: ไม่มีช่องทางส่งข้อมูลกลับที่ปลอดภัยสำหรับการทำงานกับเครื่องเช่า credential ที่มีสิทธิ์เขียนเลยต้องเดินทางผ่านบทสนทนาที่ถูกบันทึกไว้ และการเพิกถอนก็ถูกจัดเป็นรายการที่ต้องทำ แทนที่จะทำทันทีตอนที่เลิกใช้ token แล้ว

เสียหายอะไร: credential ที่เขียนข้อมูลได้ ซึ่งเข้าถึงบัญชีที่เก็บ backup ชุดเดียวของ adapter ทั้งสองรอบ ถูกเปิดเผยอยู่ในบทสนทนา และยังใช้งานได้อยู่ตอนจบเซสชัน

แก้ยังไม: ยังไม่ได้แก้ — ยังอยู่ในรายการค้าง: "เพิกถอน HF token ที่วางในแชทวันนี้"

กฎที่ได้: ห้ามวาง credential ที่มีสิทธิ์เขียนลงในบทสนทนา ให้ส่งความลับผ่าน environment ของเครื่องเองหรือผ่านที่เก็บความลับ ถ้าเผลอเปิดเผยไปแล้ว ให้เพิกถอนทันทีที่งานนั้นเสร็จ ไม่ใช่รอตอนจบวัน

เฟส 8 export ล้ม: การ merge LoRA adapter เข้าไปในโมเดลฐานแบบ MoE ได้โมเดลที่ output schema ผิดไปคนละเรื่อง

เกิดอะไรขึ้น: export_gguf.py โหลดโมเดลฐาน + adapter แล้ว merge โดยไม่ error เลย แต่ sanity check จับได้ว่าผลลัพธ์ที่ merge แล้วพ่นออกมาเป็น page_number/title/project แทนที่จะเป็น schema จริง

(png/doc_page/discipline/sheet_code/views) สคริปต์วินิจฉัย 4 ตัวพิสูจน์ว่าตัว adapter เองไม่มีปัญหา (schema ตรง 100% ก่อน merge) รูปทรงและทิศทางของ delta_weight ถูกต้อง scaling factor ถูกที่ 2.0 แต่ค่า tensor หลัง merge เปลี่ยนไปแค่ 0.67% ของขนาดจากทั้งหมด 201 ล้านค่า และ _merge_lora() ของ Unsloth เองก็ไม่รองรับ MoE ParamWrapper เลย

ต้นเหตุจริง: เป็นข้อบกพร่องจริงในเส้นทาง merge ของ peft/Unsloth สำหรับ LoRA แบบ target_parameters บน MoE expert — การ merge ใส่ delta ที่แทบเป็นศูนย์ลงไปเจียบๆ แทนที่จะล้มให้รู้เรื่อง ไม่ใช่ปัญหาจากการเทรนหรือชุดข้อมูล และซ้ำเติมด้วยการที่ไม่มีการตรวจตั้งแต่ต้นทางว่าตัว merge รองรับสถาปัตยกรรมที่ใช้อยู่หรือเปล่า ก่อนจะตัดสินใจใช้เส้นทาง export นี้

เสียหายอะไร: t02 ไม่มี GGUF เลย รอบนี้จึงไม่ได้โมเดลที่รันในเครื่องได้สักตัว และการวินิจฉัยก็กินเวลาเครื่องที่จ่ายเงินไปไม่น้อยกับสคริปต์วินิจฉัยถึง 4 ชุด

แก้ยังไม: sanity check ของสคริปต์เองหยุดมันไว้ได้ก่อนที่จะ quantize แล้วอัปโหลดโมเดลขยะขึ้นไป มะขามตัดสินใจ: หยุด หลังทดสอบสมมติฐานหลักไปแล้ว 3 ข้อ ส่วน LoRA adapter ถูกสำรองไว้แยกต่างหากและพิสูจน์แล้วว่าใช้ได้

กฎที่ได้: ต้องวาง sanity check เรื่อง schema/output ไว้ทันทีหลังทุกขั้นตอน merge, quantize หรือ convert เพื่อไม่ให้ของที่เสียแบบเจียบๆ หลุดขึ้นไปเผยแพร่ได้ และต้องตรวจให้แน่ก่อนว่า toolchain ของ merge/export รองรับสถาปัตยกรรมโมเดลของเรา ก่อนที่จะเลือกใช้มันเป็นเส้นทางส่งมอบ

"รอยแผลแห่งความอาย" 2026-07-21 — ปิดเครื่องตอนงาน "ดูเหมือนเสร็จ" เลยเสีย LoRA 7.5GB กับ GGUF 21GB ไปฟรีๆ

เกิดอะไรขึ้น: เคสนี้คือเหตุผลที่ทำให้ต้องมีด้านตรวจก่อนปิดเครื่องใน `op3` เลย — เครื่องดับไปตอนทำงาน *ดูเหมือน* จะเสร็จแล้ว แต่จริงๆ ยังไม่มีอะไรถูกเซฟลงไปสักอย่าง ผลคือ LoRA ขนาด 7.5 GB กับไฟล์ GGUF อีก 21 GB หายไปทั้งหมด

ต้นเหตุจริง: เอา "ดูเหมือนเสร็จ" มาใช้แทน "เสร็จจริง" — ระหว่างตอนทำงานจบกับตอนกดปิดเครื่อง ไม่มีจุดตรวจที่พิสูจน์ว่าเซฟ/commit แล้วจริงๆ เลย และไม่มีกฎที่ต้องพิมพ์สรุปให้ดูก่อนจะดับ

เสียหายอะไร: LoRA ที่เทรนเสร็จแล้ว 7.5GB กับไฟล์ GGUF ที่ export แล้ว 21GB หายเกลี้ยง เท่ากับเผาทั้งชั่วโมง GPU และการเทรนทั้งรอบทั้ง

แก้ยังง: ออกแบบ `op3` ใหม่ให้มีด้านตรวจ 6 เดือนไซก่อนปิดเครื่อง: ไฟล์ต้องเขียนลงแล้วและ parse ผ่าน → `check\_format.py` ต้อง ALL PASS → มีแถวในสมุดบันทึกข้อมูล → `git commit` แล้ว → พิมพ์สรุปให้ผู้ใช้เห็นก่อน → แล้วค่อย `shutdown /s /t 120` ห้ามใช้ `/t 0` เด็ดขาด และถ้าขึ้นไหนไม่ผ่าน = ไม่ปิดเครื่องเลย

กฎที่ได้: อย่าปิดเครื่องเพราะ "ดูเหมือนเสร็จ" เด็ดขาด ทำตอนจบให้เป็นอัตโนมัติแต่ต้องมีด้านแข็งๆ — ผลลัพธ์ต้องตรวจแล้วเช็คผ่านแล้ว บันทึกแล้ว commit แล้ว และรายงานแล้ว — แคมต้องเหลือเวลาหนึ่งให้ยกเลิกทันเสมอ ห้าม `/t 0`

Pass 2 เป็นไค้ตายที่ยังเสียเงินอยู่ — ยิง API N ครั้งต่อรอบ ผลลัพธ์ไม่มีใครใช้ error ก็ถูกกลืน

เกิดอะไรขึ้น: Pass 2 เก็บ `layoutIds` มาแล้วไม่เคยเอาไปใช้เลย (Pass 3 ไปวนลูปบน id ของ Pass 1 แทน) แคม error ก็ถูก `catch` เปลาๆ กลืนหายไป มันเสียค่ายิง API N ครั้งทุกรอบที่รันจริง แล้วไม่ได้อะไรกลับมาเลย

ต้นเหตุจริง: มีขั้นตอนใน pipeline ที่ถูกแทนที่ไปแล้วแต่ไม่เคยถูกลบออก และบล็อก `catch` เปลาๆ ก็ปิดบังความจริงว่ามันพังหรือว่างงานอยู่ ไม่มีใครวัดว่า output ของมันถูกเอาไปใช้จริงหรือเปล่า

เสียหายอะไร: เผาเงินทุกครั้งที่รันจริงโดยไม่ได้ประโยชน์อะไร แคมเสียด้วย ที่แยกหน้านั้นคือ Pass 2 อาจเป็นตัวช่วยเพิ่ม recall ได้จริงถ้าต่อสายให้ถูก (มันจะจับ mark ที่มีในผังแต่ไม่มีในแผน section ได้) เท่ากับจ่ายเงินไปแล้วแต่ไม่ได้ประโยชน์ที่ควรได้

แก้ยังง: ระบุตัวปัญหาและบันทึกไว้ ข้อเสนอคือให้เลือกอย่างใดอย่างหนึ่ง: ต่อสายให้มันทำงานจริง (อาจช่วยเรื่อง recall) หรือไม่ก็ลบทิ้งไปเลย

กฎที่ได้: อย่าปล่อยให้มี `catch` เปลาๆ เด็ดขาด และให้ไล่ตรวจทุกจุดที่ยิง API แบบเสียเงินว่าผลลัพธ์ถูกเอาไปใช้จริงไหม — ขั้นตอนใน pipeline ที่ไม่มีใครอ่าน output ของมัน ถ้าไม่ใช่บั๊กก็คือบิลค่าใช้จ่าย

requeue_stale() ใช้เวลาที่ **claim** งานเป็น **heartbeat** ทำให้งานจริงที่กำลังรันอยู่ถูกนับว่าตายทั้งที่ยังทำงานอยู่

เกิดอะไรขึ้น: `requeue_stale()` กรองงานโดยดูจากฟิลด์ `claimed_at` ซึ่งถูกเขียนแค่ครั้งเดียวตอน worker คว้างานไปทำ แล้วไม่เคยอัปเดตอีกเลย งานจริงใช้เวลารัน 75-80 นาที ในขณะที่ `STALE_PROCESSING_MIN = 45` ทำให้งานจริงทุกงานเข้าเงื่อนไข 'stale' กลางคันตลอด ถ้ามี worker ตัวที่สองเริ่มขึ้นมา (เช่น มะขามดับเบิลคลิก GO.bat ซ้ำ หรือเปิดหน้าต่างใหม่เพราะคิดว่างานค้าง) มันจะพลิกงานที่กำลังรันอยู่กลับไปเป็น 'pending' แล้วแย่งคว้างานไปทำใหม่

ต้นเหตุจริง: สืบสนระหว่าง 'เวลาที่งานเริ่ม' กับ 'เวลาที่งานยังมีชีวิตอยู่' ระบบไม่มีแนวคิด heartbeat เลย — ตัวเช็ควางานยังมีชีวิตอยู่ใหม่อ่าน timestamp ที่ตามนิยามแล้วไม่เคยอัปเดต ทำให้เกณฑ์ timeout ไม่มีทางที่งานที่ยังทำงานปกติจะผ่านได้เลย

เสียหายอะไร: เสียค่า GPU สองเท่าสำหรับบ้านเดียวกัน แล้ว worker สองตัวยังเขียนผลลัพธ์ทับกันเองอีก ที่ยังไม่เจอปัญหาจริงในโลกจริง เป็นเพราะยังไม่เคยมีการรัน worker สองตัวพร้อมกันเท่านั้น

แก้ยังไง: เปลี่ยนตัวกรองไปใช้ `updated_at` แทน ซึ่งถูก trigger `purson_jobs_touch_updated_at` อัปเดตทุกครั้งที่มีการเขียน `set_progress` — งานที่ยังรายงานความคืบหน้าอยู่จะไม่มียันถูกมองว่าตาย

กฎที่ได้: ตัวเช็ควางานยังมีชีวิตอยู่ใหม่ ต้องอ่านฟิลด์ที่กระบวนการที่ยังทำงานอยู่จริงๆ อัปเดตอย่างสม่ำเสมอ ก่อนจะปล่อย timeout ให้ออกไปใช้งาน ให้ถามตัวเองว่า 'อะไรเป็นตัวอัปเดต timestamp นี้นอนงานยังมีชีวิตอยู่' — ถ้าคำตอบคือ 'ไม่มีอะไรอัปเดตเลย' แปลว่าตัวเช็คนั้นผิดตั้งแต่ต้น

แก้ **config** หั้ววันแต่แก้แค่ในไฟล์บนเครื่อง โปรเจกต์ **Label Studio** ตัวจริงยังใช้ **config** เกาที่ฝังอยู่

เกิดอะไรขึ้น: ที่แก้ไปทั้งวัน (แก้ scroll, ย้าย header, แบ่ง 50/50, ออกแบบ hdrbar ใหม่) แก้ในไฟล์ XML บนดิสก์อย่างเดียว แต่โปรเจกต์ Elements และ Single (271482, 271493) ที่สร้างไว้แล้วบน Label Studio Cloud ไม่เคยถูกวาง config ใหม่ทับของจริงเลยยังมีบั๊กเนื้อหาหายอยู่

ต้นเหตุจริง: config ของ Label Studio ต้อง deploy ด้วยการ copy-paste เข้าหน้าเว็บด้วยมือ และในขั้นตอนการทำงานไม่มีอะไรผูก "แก้ไฟล์" เข้ากับ "deploy" เลย

เสียหายอะไร: บั๊ก scroll ที่เป็นต้นเรื่องของงานทั้งวัน ยังพังอยู่จริงสำหรับคนทำ annotation ทั้งที่ประกาศว่า "แก้แล้ว" และถูกยกยอดไปเป็นงานค้างของวันถัดไป

แก้ยังไง: ยังไม่ได้แก้ในรอบนี้ — บันทึกเป็นงานค้างข้อ 1: เอา config ล่าสุดไปวางทับในโปรเจกต์ที่สร้างไว้แล้ว

กฎที่ได้: config อะไรก็ตามที่ deploy ด้วยมือ ให้ถือว่า "แก้ไฟล์แล้ว" กับ "ระบบที่รันอยู่เปลี่ยนแล้ว" เป็นคนละเรื่องกัน และห้ามรายงานว่าแก้เสร็จจนกว่าจะไปตรวจสอบในเครื่องมือตัวจริงแล้ว

ข้อมูลหลักเสริมที่เสี่ยงสุดของบ้านหลัง 2 และ 3 ยังติดธง confidence ต่ำ และยังไม่มีการไปชมดู

เกิดอะไรขึ้น: บ้านหลัง 2 ส่วนที่อ่านแล้วไม่มั่นใจเต็มร้อยถูกติดธงไว้แทนที่จะเอา ได้แก่ ส่วนต่อขยายลงใต้เลยกริด C ในผังคาน/คานคอดิน ตำแหน่งของ RB2 ที่ไม่ได้อยู่บนกริดหลัก และแถวหลังคา/คานคอดินในตารางเสาที่ดูเหมือนช่องว่างหรือลูกศร บ้านหลัง 3 ก็เจอแบบเดียวกัน คือตำแหน่งพื้นที่ส่วนต่อขยายหน้าต่างโค้ง RB2 ที่ออกนอกกริดหลักอีกครั้ง และจำนวนเหล็ก dowel ที่มีเครื่องหมาย '+' โพลมาสองครั้ง ทุกอันได้ confidence_score ต่ำพร้อมธง needs_human_review_zoom_required

ต้นเหตุจริง: ความละเอียดของภาพและธรรมเนียมการเขียนแบบ (ลูกศรที่แปลว่า "ต่อจากชั้นล่าง" แถวตารางที่ดูเหมือนว่างเปล่า) เกินกว่าจะอ่านได้แน่นอนที่ระดับการซูมที่ใช้อยู่ และยังไม่มีการจัดคิวให้ไปชมตรวจรายการที่ติดธงเหล่านั้น

เสียหายอะไร: นี่คือส่วนโครงสร้าง — ส่วนที่บอกไว้เองว่าเสี่ยงสุดและต้องอ่านเองแทนที่จะให้ agent อ่าน ด้วยเหตุผลนี้พอดี ตำแหน่งและจำนวนเหล็กที่ติดธงแต่ยังไม่ได้ตรวจ นอนอยู่ในโพลเดอร์ของจริงของบ้านหลัง 2 และ 3 และเป็นตัวเลือกที่จะกลายเป็น ground truth สำหรับ fine-tune

แก้ง่าย: ยังไม่ได้แก้ — บันทึกธงไว้ ระบุว่าตรวจยังไม่เสร็จ แต่ในรายการงานค้างวันที่ 11 ก.ค. ระบุรายการ BOQ ของบ้านหลัง 2 ที่ confidence ต่ำ ไม่ได้ระบุรายการโครงสร้างเลย

กฎที่ได้: ติดธงแทนการเอาเป็นสิ่งที่ถูกแล้ว แต่ธงที่ไม่มีคิวตรวจก็คือการเอาที่แค่เลื่อนออกไป ธงในส่วนโครงสร้างต้องมีรอบตรวจที่ระบุชัดเจนจะเรียกบ้านหลังนั้นว่าเสร็จ และรายการงานค้างต้องเขียนมันลงไปจริงๆ

ลิมิต session ของบัญชี Claude ฆ่า agent ทำ BOQ ทั้งสองตัวกลางทางบนบ้านหลังที่ 3

เกิดอะไรขึ้น: ทั้ง agent ทำ BOQ ฐานรากแผ่ (หน้า 64-74) และ agent ทำ BOQ ฐานรากเสาเข็ม + ผังบริเวณ (หน้า 75-89) ถูกหยุดโดยลิมิต session ของบัญชี ซึ่งรีเซ็ตตอนตี 2 เวลาไทย ตัวแรกขาดไฟล์ page74_2 หนึ่งไฟล์ ตัวที่สองขาดส่วน B ทั้งหมด (หน้า 86-89)

ต้นเหตุจริง: ส่ง agent ห้าตัวออกไปพร้อมกันโดยใช้บ session ของบัญชีร่วมกัน โดยไม่ได้เช็คก่อนว่าเหลือบเท่าไร และไม่มีการ retry ต่อ agent

เสียหายอะไร: บ้านหลังที่สองติดกันที่ค้างไม่เสร็จเพราะ agent ถูกฆ่า ต้องมาซ่อมด้วยมือ และถ้าส่ง agent ใหม่ก็จะชนลิมิตเดิมอีก เพราะมันยังไม่รีเซ็ต

แก้ง่าย: ซ่อมด้วยการอ่านหน้าที่ขาดเองแทนที่จะส่ง agent ใหม่ และเขียนบทเรียนลงบันทึกไว้ว่า ให้เช็คก่อนว่า agent ไปได้ถึงไหนแล้วค่อยอ่านส่วนที่ขาดด้วยมือ

กฎที่ได้: ให้ถือว่าลิมิต session ของบัญชีเป็นทรัพยากรที่ใช้ร่วมกัน เช็คว่ามีเหลือเท่าไรก่อนกระจาย agent ขนานกัน และเวลา มีตัวไหนถูกฆ่า ให้วัดก่อนว่ามันผลิตอะไรออกมาได้บ้าง แล้วค่อยตัดสินใจว่าจะซ่อมมือหรือส่งใหม่

สูง 2026-07-12

ส่ง agent เบื้องหลัง 5 ตัวพร้อมกัน ขนลิมิต session ของบัญชีกลางทางซ้ำแล้วซ้ำอีก

เกิดอะไรขึ้น: ส่ง agent เบื้องหลัง 5 ตัวขนานกันเพื่อทำส่วนที่เหลือของเอกสาร สามในห้าตัว (สถาปัตยกรรม, BOQ ฐานรากแผ่, BOQ ฐานรากเสาเข็ม) ถูกลิมิต session ของบัญชีหยุดกลางทาง เรื่องเดียวกันนี้เกิดมาแล้วกับบ้านหลัง 2 และ 3

ต้นเหตุจริง: รูปแบบการกระจาย agent ขนกันเกิน session ที่ใช้ร่วมกันเร็วกว่าที่งานของบ้านหลังเดียวจะเสร็จ ขั้นตอนการทำงานถูกออกแบบมาโดยไม่มีการคิดงบเลยว่าบัญชีรับ agent ได้กี่ตัวจริงๆ

เสียหายอะไร: ต้องทำงานสกัดข้อมูลใหม่ด้วยมือประมาณ 55 หน้า กระจายอยู่ในสามส่วน และเสียหายแบบเดิมซ้ำทุกหลังตั้งแต่หลังที่ 2 เป็นต้นมา

แก้ยังไง: แทนที่จะส่ง agent ใหม่ (ซึ่งจะแพงไปอีก) เลย์อ่านหน้าที่ขาดเอง วิธีกู้คืนที่เชื่อได้คือ ไปดูไฟล์ที่ถูกเขียนลงดิสก์จริง ไม่ใช่ดูข้อความสุดท้ายที่ agent รายงาน

กฎที่ได้: ห้ามเชื่อข้อความสุดท้ายของ agent ที่ถูกตัดกลางทางว่าเป็นจุดเช็คพอยต์ — ให้ diff ไฟล์บนดิสก์กับรายการหน้าที่คาดว่าจะได้เสมอ และกำหนดจำนวน agent ที่กระจายขนานกันตามงบ session จริง ไม่ใช่ส่งเต็มพิกัดทุกครั้ง

สูง 2026-07-16

regex ที่ใช้ตัดชั้ดกลาง ไปทำ sheet_code พังหมดเจียบๆ

เกิดอะไรขึ้น: regex ที่ใช้ตัดชั้ดกลางออกจาก grid ref แบบจุด ดันไปแมตช์กับค่า `sheet\_code` ด้วย ทำให้ 'S-04' กลายเป็น 'S04' ในทุกไฟล์ที่มันรันผ่านในบ้านหลัง 3

ต้นเหตุจริง: regex แมตช์รูปแบบ ตัวอักษร-ขีด-ตัวเลข ที่ไหนก็ได้ในไฟล์ แทนที่จะจำกัดขอบเขตไว้เฉพาะฟิลด์ grid_ref ที่ตั้งใจจะแก้จริงๆ

เสียหายอะไร: ฟิลด์หนึ่งพังไปทั้งบ้านแบบลูกหลง ต้องไล่หาและย้อนคืนทีละไฟล์ ทีหลังพอรันกับบ้านหลัง 5 ก็ต้องเช็คการ normalize ด้วย perl เป็นพิเศษเพื่อยืนยันว่า "sheet_code รอด"

แก้ยังไง: ไล่เช็คและแก้ sheet_code กลับคืนในทุกไฟล์ที่โดน แล้วการ normalize แบบยกชุดครั้งต่อๆ มาก็คตรวจสอบความถูกต้องของ sheet_code หลังรันทุกครั้ง

กฎที่ได้: จำกัดขอบเขต regex ที่รันยกชุดให้อยู่แค่ฟิลด์ JSON ที่ต้องการจริงๆ ห้ามรันกับข้อความดิบทั้งไฟล์ — และต้อง diff ผลลัพธ์ดูตัวอย่างสักชุดเพื่อหาการแมตช์ลูกหลง ก่อนรันจริงทั้งบ้าน

สูง 2026-07-19

B2X ถูกอ่านเป็นคาน 4 ตัวแนวตั้ง ทั้งที่จริงเป็นช่วงหนึ่งของคานแนวนอน

เกิดอะไรขึ้น: ในไฟล์ `หน้า25\_beam\_plan.json` ของบ้านหลัง 2 B2X ถูกบันทึกเป็นคานแนวตั้ง 4 ตัว พอซูม 3 เท่าถึงเห็นว่ามันคือช่วงกลางของคานแนวนอนบนแนว E/D ในช่วงส่วนต่อขยายที่ยื่นออกไป 1.00 ม. ถึง `1'"/4'`

ต้นเหตุจริง: เดาแนววางคานจากตำแหน่งของ mark ในแบบ แทนที่จะไล่ตามแนวเส้นของตัวคานจริงๆ แกมคานแนวนอนบนแนว E/D รอบๆ ก็หายไปหมด เลยไม่เหลือบริบทที่จะให้เห็นแนววางที่ถูกต้อง

เสียหายอะไร: แนววางผิดไป 4 ตัว บวกกับคานแนวนอนที่หายไปทั้งต้นอีก 6 ตัว (E1-E2, E2-E3, E3-E4, D1-D2, D2-D3, D3-D4) และ label ที่ grid2/3 ผิด (B2 ควรเป็น B1)

แก้ยังไง: สกัดข้อมูลใหม่ที่ซูม 3 เท่า เพิ่มคานแนวนอนที่หายไปทั้งหมดและคานแนวตั้งที่ `1'"/3'"/4'"/F` แล้วแก้ label ของ mark ให้ถูก

กฎที่ได้: ไล่ตามแนวเส้นของคานทุกตัวก่อนบันทึกแนววาง และเช็คด้วยว่าคานในแต่ละแถวต่อเนื่องกันครบไหม — ช่องว่างในแถวคือสัญญาณว่ามีของหาย ไม่ใช่ช่องที่ว่างเปล่าจริง

สูง 2026-07-20

onstart.sh ทำให้เครื่องเช่าเครื่องแรกวูลัพพัง เพราะ **pip upgrade** ชนกับ **pip** ที่ลงมาจาก **apt**

เกิดอะไรขึ้น: คำสั่ง `pip install --upgrade pip` ใน `onstart.sh` ชนกับ `pip` ที่ติดตั้งมาจาก `apt` แล้ว error ตัว `set -euo pipefail` เลยฆ่าสคริปต์ทั้งตัว ทำให้เครื่องเช่าเครื่องแรกวูลัพพังไปเลย

ต้นเหตุจริง: เอาคำสั่งที่ไม่จำเป็น (แค่ทำเพื่อความสะดวก) ไปใส่ในสคริปต์โหมดเซมิงวอดที่ไม่ยอมให้อะไรพลาดได้เลย บนเครื่องที่ `base image` ติดตั้ง `pip` มาทาง `apt` อยู่แล้ว

เสียหายอะไร: เครื่อง GPU ที่เช่ามาถูกเผาทิ้งไปกับการแรกวูลัพพัง ต้องเช่าเครื่องที่สอง (id 45406175) มาใหม่

แก้ยังง: เติม `|| true` ต่อท้ายคำสั่ง `pip upgrade` แล้วเครื่องที่สองรันผ่านตลอด ยืนยันจาก log แล้ว

กฎที่ได้: ในสคริปต์ provisioning ที่ใช้ `set -euo pipefail` คำสั่งที่ไม่จำเป็นทุกตัวต้องมี `|| true` และบนเครื่องเช่า สคริปต์ตอนบูตพังเท่ากับเสียเงิน ให้ซ่อมรันบนเครื่องถูกๆ ก่อนขึ้นเครื่องแพง

สูง 2026-07-21

เช่า GPU แพงมาก่อน แล้วนั่งติ๊กสคริปต์สตาๆ บนเครื่องนั้น

เกิดอะไรขึ้น: เช่า RTX PRO 6000 96GB ราคา \$1.056/ชม. มาก่อน ทั้งที่ยังไม่เคยพิสูจน์เลยว่าสคริปต์เทรน/ประเมินผลรันได้จริงไหม แล้วก็นั่งไล่บั๊ก 4 ตัวกับ OOM อีก 4 รอบไปโดยที่มีเตอร์เดินอยู่

ต้นเหตุจริง: ในกระบวนการไม่มีขั้นตอนซ่อมบนเครื่องถูกๆ เลย สมมติเอาว่าสคริปต์ถูกอยู่แล้วโดยไม่ได้ตรวจสอบ ที่รันจริงครั้งแรกเลยกลายเป็นเครื่องที่แพงที่สุดไปด้วย

เสียหายอะไร: เสียเวลาหลายชั่วโมงที่ \$1.056/ชม. ไปกับการติ๊กแทนที่จะเทรน มะขามหักตรงๆ ว่ามันเปลืองเงินชัดๆ ("ขนาดเซตบอดยังรู้เลยว่าเปลืองเงิน") และต้องเติมเงินอีก \$10 (รวมประมาณ \$19)

แก้ยังง: บันทึกลงหน่วยความจำการเป็นกฎประจำ: เช่าเครื่องถูกๆ มาทดสอบสคริปต์ก่อน แล้วค่อยเช่าเครื่องแพง พร้อมเพิ่มสวิตช์ `TEST_STEPS=5` ไว้ซ่อมใน `train_qwen36.py`

กฎที่ได้: ห้ามติ๊กบน GPU ระดับพรีเมียม ให้พิสูจน์ทุกสคริปต์ตั้งแต่ต้นจนจบบนเครื่องที่ถูกที่สุด (หรือ dry-run เล็กๆ ในเครื่องตัวเอง) ก่อน การดแพมมีไว้สำหรับงานที่รู้แล้วว่ารันผ่านเท่านั้น

สูง 2026-07-21

ดิสก์เต็มซ้ำๆ ระหว่าง merge/GGUF และเต็ม 100% พอดีตอนแปลงไฟล์ค้างอยู่

เกิดอะไรขึ้น: ดิสก์ 150GB ของเครื่องเต็มซ้ำแล้วซ้ำอีกระหว่างขั้นตอน `merge`→`f16`→`quantize` ต้องลบ checkpoint เก่า 44GB, HF cache 67GB, pip cache 5.4GB และไฟล์ระหว่างทาง `q8_0/f16` ทั้งกลางคัน และมีจังหวะหนึ่งดิสก์เต็ม 100% พอดีตอนที่แปลง `f16` เสร็จ ทำให้ต้องลบ `merged/` แบบฉุกเฉิน

ต้นเหตุจริง: ไม่ได้คำนวณงบพื้นที่ดิสก์ก่อนเริ่ม ทั้งที่ไฟล์ระหว่างทาง (โมเดล 70GB + `f16` 66GB + quantized 21GB + cache) มันเกิน 150GB ชัดๆ อยู่แล้ว ส่วน `verify_env.py` ก็เช็คแค่ GPU/VRAM/ไลบรารี ไม่ได้เช็คพื้นที่ดิสก์ว่าง

เสียหายอะไร: งานสะดุดหลายรอบและต้องลบไฟล์แบบฉุกเฉินภายใต้ความกดดันเรื่องเวลา — และการลบ `merged/` แบบตกใจนี้แหละที่ทำให้ทีหลังสก็ด `mmproj` ไม่ได้

แก้ยังง: ลบ cache / checkpoint / ไฟล์ระหว่างทางไปเรื่อยๆ ตามที่พื้นที่หมด แล้วแปลงต่อจนจบ

กฎที่ได้: คำนวณพื้นที่ดิสก์สูงสุดที่ทั้ง pipeline ต้องใช้ก่อนเช่าเครื่อง เลือกเครื่องที่มีที่เหลือเผื่อ และเพิ่มการเช็คพื้นที่ว่างเข้าไปใน `verify_env.py` — และห้ามเรียกคืนพื้นที่ด้วยการลบไฟล์ที่ยังใช้ไม่เสร็จ

สูง 2026-07-21

เช่าเครื่องดิสก์ 150GB มาใช้ pipeline ที่ต้องการพื้นที่รวมเกิน 200GB

เกิดอะไรขึ้น: เครื่องที่เช่ามีดิสก์แค่ 150GB แต่ chain merge/GGUF ต้องใช้โมเดลต้นทาง 70GB + checkpoint 44GB + โมเดลที่ merge แล้ว 66GB + GGUF f16 ~70GB + ผลลัพธ์ Q4_K_M 21GB ซึ่งบางส่วนต้องอยู่พร้อมกันในเวลาเดียว การแปลงทั้งหมดเลยกลายเป็นการลบฉุกเฉินต่อเนื่อง: checkpoint เก่า (44GB), HF cache (67GB), pip cache (5.4GB), ไฟล์ q8_0/f16 ระหว่างทาง — และดิสก์ก็เต็ม 100% กลางการแปลงอยู่ดี

ต้นเหตุจริง: ไม่เคยคำนวณขนาดดิสก์เทียบกับ pipeline ที่จะใช้จริงตอนเลือกเครื่องเช่า — เช็คแค่ GPU/VRAM เท่านั้น (verify_env.py เช็ค VRAM แต่ไม่เชื่อดิสก์ว่าง) การวางแผน chain merge/quantize ทั้งหมดทำหลังจากเช่าเครื่องไปแล้ว

เสียหายอะไร: เสียเวลาหลายชั่วโมงดับไฟระหว่างการแปลง; บังคับให้ต้องลบโฟลเดอร์ `merged/` ฉุกเฉิน ซึ่งเป็นเหตุผลตรงๆ ที่ทำให้ฝั่งส่วน vision (mmpoj) ออกมาไม่ได้อีกเลย

แก้ยังไง: ลบไฟล์แบบตอบสนองเฉพาะหน้าไปเรื่อยๆ ทุกครั้งที่ล้นเหลว จนการแปลงผ่านไปได้แบบเบียดๆ

กฎที่ได้: เลือกขนาดดิสก์ของเครื่องเช่าจาก 'พื้นที่รวมสูงสุดที่ต้องใช้พร้อมกัน' ของทั้ง pipeline (ต้นฉบับ + merged + f16 + quantized) ไม่ใช่แค่ขนาดโมเดล และต้องตัดสินใจตอนเช่าเครื่อง เพราะขยายดิสก์ทีหลังไม่ได้ ควรเพิ่มการเช็ค disk ว่างเข้าไปใน verify_env.py ด้วย

สูง 2026-07-21

ดิสก์ของเครื่องเช่าเต็มซ้ำๆ ระหว่าง export GGUF (150GB เล็กเกินไป)

เกิดอะไรขึ้น: ระหว่าง merge และแปลง GGUF เครื่องดิสก์เต็มหลายครั้ง ต้องลบ checkpoint เก่า, cache ของโมเดลต้นทาง, และ artifact ระหว่างทางด้วยมือกลางการทำงาน

ต้นเหตุจริง: ดิสก์ถูกกำหนดขนาดไว้สำหรับเทรนอย่างเดียว ไม่มีใครรวมยอดพื้นที่สูงสุดที่ใช้พร้อมกันของโมเดลต้นทาง + โมเดล merge แล้ว + GGUF f16 + GGUF quantize ก่อนเช่าเครื่อง

เสียหายอะไร: งานสะดุดซ้ำๆ ต้องทำความสะอาดด้วยมือระหว่าง session ที่เสียเงิน มีความเสี่ยงลบของที่ยังต้องใช้อยู่

แก้ยังไง: ลบ artifact ที่ไม่ใช่แล้วไปเรื่อยๆ ตามที่งานดำเนินไป และในการเช่าครั้งถัดไปคำนวณดิสก์ที่ต้องใช้ล่วงหน้าไว้ที่ 300GB

กฎที่ได้: กำหนดขนาดดิสก์ของเครื่องเช่าจากพื้นที่รวมสูงสุดที่ทุก artifact ของทั้ง pipeline ต้องการพร้อมกัน (เทรน + merge + export + quantize) ไม่ใช่แค่ขั้นตอนเทรนอย่างเดียว

สูง 2026-07-24

แก้บั๊กบน GPU เช้าขณะมีเตอร์นับเวลาทำงานอยู่ แทนที่จะแก้บนเครื่องฟรีที่บ้านก่อน

เกิดอะไรขึ้น: ระหว่างรอบ 2026-07-21 พบปัญหาหลังเข้าเครื่องไปแล้ว แก่สดๆ บนเครื่องที่เสียเงินอยู่ ~30 บาท/ชั่วโมง แทนที่จะจับได้ล่วงหน้าฟรีๆ บนเครื่องท้องถิ่น

ต้นเหตุจริง: ไม่มีการแยกว่าอะไรต้องใช้ GPU เช้า กับอะไรไม่ต้อง — ทุกอย่างเช็คได้ฟรี (ความต้องการของ token, ยอดเงินในบัญชี, ค่าดิสก์เริ่มต้น, ความถูกต้องของสคริปต์, โมเดลรันบนเครื่องท้องถิ่นได้ไหม) ถูกเลื่อนไปทำหลังจากที่ทรัพยากรเสียเงินเริ่มนับแล้ว

เสียหายอะไร: ชั่วโมงเช้าถูกใช้ไปกับการวินิจฉัยปัญหาแทนการเทรนจริง ไม่ได้วัดเป็นตัวเลข แต่มากพอที่ทีมต้องสร้างระบบ Phase 0 ทั้งชุดขึ้นมาเพื่อหยุดปัญหานี้

แก้ยังไง: สร้าง Phase 0: ทดสอบ token, เช็คยอดเงิน, dry run บนเครื่องท้องถิ่น, อ่านสคริปต์ซ้ำ, เลือกเครื่อง — ทำให้เสร็จทั้งหมดก่อนกดปุ่ม Rent

กฎที่ได้: แยกทุกการเช็คก่อนเริ่มงานออกเป็น 'ต้องใช้ทรัพยากรเสียเงิน' กับ 'ไม่ต้อง' ทุกอย่างในกลุ่มหลังต้องผ่านให้หมดก่อนจะเสียเงินสตางค์แรก

สูง 2026-07-24

สคริปต์เทรนค้างอยู่ตอนจบโดยยึด VRAM ไว้ 83GB ทำให้การรันครั้งถัดไป OOM

เกิดอะไรขึ้น: หลังการทดสอบสั้นๆ (`TEST_STEPS=5`) เสร็จ โปรเซสไม่ยอมปิดตัว ค้างยึด VRAM ไว้ 83GB ต้อง kill เองด้วยมือ พอไม่ได้ kill รอบแรกของ Phase 6 ก็ตายด้วย error คล้าย OOM (meta-tensor) — เพราะไม่มี VRAM วางพอตอนเริ่มใหม่อากาศค้ำนี้เกิดซ้ำทุกครั้งตอน demo-generate ตอนจบหลัง save เสร็จ

ต้นเหตุจริง: ชัน demo-generate ตอนท้ายหลัง save ค้าง (ภายหลังพบว่ามาจากปัญหา template reasoning-mode เดียวกับที่ทำให้ evaluation พัง) และโปรเซสยึด CUDA context ไว้จนกว่าจะถูก kill

เสียหายอะไร: การเทรนรอบแรกเสียไปเปล่าเพราะ OOM; ทุกรอบถัดมาต้อง kill ด้วยมือ บนเวลา GPU ที่เสียเงินจ่ายอยู่

แก้ยังไง: kill โปรเซสด้วยมือหลังจาก adapter ถูก save แล้ว (ไม่กระทบผลการเทรน) ส่วนต้นเหตุจริงคือ mismatch ของ template `enable\_thinking` ซึ่งไปแก้ทีหลัง

กฎที่ได้: ต้องตรวจสอบให้แน่ใจว่า VRAM ถูกคืนจริงก่อนเริ่มรันครั้งถัดไปบน GPU เช้า — context ที่ยังถูกยึดอยู่จะทำให้ OOM ของรอบถัดไปดูเหมือนปัญหาขนาดโมเดล ทั้งที่จริงคือปัญหาการเคลียร์ทรัพยากร และให้ถือว่าสคริปต์ที่ไม่ยอมจบเป็นบั๊กที่ต้องหาต้นเหตุจริง ไม่ใช่แค่พฤติกรรมแปลกๆ ที่ workaround ไปได้

ลบ `vision_config` เพื่อข้ามการแปลง `mmapproj` แต่กลับทำให้การ `save` โมเดลหลักพังเป็นชื่อ `tensor` ซ้อนกันสามชั้น

เกิดอะไรขึ้น: Unsloth ตรวจจับว่าโมเดลนี้เป็น VLM อัตโนมัติและพยายามแปลง `mmapproj` ทั้งที่ตั้งใจข้ามไปแล้ว การแปลงล้มเหลวเพราะสถาปัตยกรรมใหม่เกินไปและทำให้ `export` ทั้งชุดพังไปด้วย ความพยายามแก้ครั้งแรก — ลบ ``model.config.vision_config`` ชั่วคราว — กลับทำให้เกิดปัญหาอื่นแทน ชื่อ `tensor` ซ้อนลึกลงไปสามชั้น (``language_model.language_model.language_model...``)

ต้นเหตุจริง: การแก้ไขไปแตะ `config field` ที่มีผลต่อวิธี `save` โมเดลหลักด้วย ไม่ใช่แค่การแปลง `mmapproj` อย่างเดียว ไม่ได้ไล่ตามผลกระทบของการเปลี่ยน `config` นี้ก่อนลงมือแก้

เสียหายอะไร: `export` พังต้องเข้าวงจรแก้บั๊กและรันซ้ำอีกรอบบนเวลา GPU ที่เสียเงิน หุกรอบที่รันซ้ำยังเพิ่มปัญหาดิสก์เต็มไปด้วย

แก้ยังไง: monkey-patch ``unsloth.save.convert_to_gguf`` ให้บังคับ ``supported_vision_archs=set()`` (บังคับ ไม่ใช่ตั้งค่าเริ่มต้น เพราะ `kwarg` นี้อยู่เสมอ) ทำให้ทริกเกอร์ fallback ในตัวของ Unsloth เองที่ ``is_vlm=False`` ซึ่งไม่แตะการ `save` โมเดลหลักเลย

กฎที่ได้: ก่อนแก้ object `config` ที่ใช้ร่วมกันเพื่อปิด code path หนึ่ง ให้ไล่ผู้ใช้งาน field นั้นทุกจุดก่อน เลือก patch ฟังก์ชันเฉพาะที่ต้องการเปลี่ยนพฤติกรรม ดีกว่าไปลบ state ที่มีหลาย path อ่านอยู่

export ล้มเหลวด้วย 'No space left on device' จาก artifact เก่าที่ค้างอยู่ 147GB

เกิดอะไรขึ้น: การ `export` เจอ ``No space left on device`` การทำความสะอาดกู้คืนได้ 147GB: checkpoint เก่าที่เข้ากับ LoRA adapter จริง (33+11GB), pip cache (3GB), และโฟลเดอร์ ``outputs_qwen36/gguf/`` เก่าที่ค้างอยู่ (66GB)

ต้นเหตุจริง: ทุกครั้งที่ `export` ล้มเหลวแล้วรันซ้ำ จะเขียน artifact ชุดใหม่เพิ่มขนาดโดยไม่ลบชุดก่อนหน้านี้ทิ้ง และไม่มีการเช็คพื้นที่ดิสก์ระหว่างการลองแต่ละครั้ง

เสียหายอะไร: `export` ล้มเหลวบนเวลา GPU ที่เสียเงิน บวกกับวงจรทำความสะอาดทั้งหมด ถ้าใช้ดิสก์เริ่มต้น 150GB จะเจอปัญหานี้เร็วกว่านี้มาก

แก้ยังไง: ลบ checkpoint เก่า, pip cache, และโฟลเดอร์ `gguf` เก่า กู้คืนพื้นที่ได้ 147GB

กฎที่ได้: สคริปต์ไหนที่เขียน artifact ขนาดหลาย GB ต้องลบผลลัพธ์รอบก่อนหน้านี้ก่อนเริ่มใหม่ และเช็คพื้นที่ว่างเทียบกับขนาดผลลัพธ์ที่คาดหวังไว้ก่อน บนเครื่องเช่า ดิสก์เต็มกลาง `export` คือเงินที่จ่ายไปแบบไม่ได้อะไรกลับมาเลย

สูง 2026-07-24

รัน llama-cli โดยไม่ใส่ --ctx-size ทำให้จอง KV cache ขนาด 262k token จนแรมเหลือแค่ 0.1GB

เกิดอะไรขึ้น: การทดสอบครั้งแรกในเครื่องท้องถิ่นใช้ `llama-cli --n-gpu-layers 999` โดยไม่ใส่ `--ctx-size` แรมลดลงเหลือ 0.1GB จาก 31.4GB ทั้งหมด — `llama-cli` จองหน่วยความจำเสมือนไปถึง 34.96GB (มากกว่าแรมที่ติดตั้งจริงทั้งหมด) เพราะโมเดลนี้รองรับ context สูงสุดถึง 262,144 token และจอง KV cache ไว้ตามนั้น ผู้ใช้เป็นคนสังเกตเห็นก่อน

ต้นเหตุจริง: `--ctx-size` มีค่าเริ่มต้นตาม context สูงสุดที่โมเดลประกาศไว้ ซึ่งโมเดลนี้มีค่าสูงมาก และ KV cache จะขยายตาม context ทำให้ค่าเริ่มต้นจองพื้นที่เกินความจำเป็นของงานหน้าเดียวนมาก

เสียหายอะไร: เครื่องทำงานเกือบค้างขณะกำลังใช้งานอยู่ ความผิดพลาดแบบเดียวกันเกิดซ้ำวันถัดไปที่ ctx-size 12288 (แรมเหลือ 0.45GB) ต้อง kill ดูกเงินอีกรอบ

แก้ยังง: kill โพรセス (แรมคืนกลับมา 21GB) แล้วรันใหม่ด้วย `--ctx-size 8192` (พอสำหรับหนึ่งหน้าบวกคำตอบ) ใช้แรมประมาณ 20GB และเสถียรดี

กฎที่ได้: ต้องใส่ `--ctx-size` ชัดเจนให้เหมาะกับงานจริงเสมอ อย่าปล่อยให้ default ไปที่ context สูงสุดของโมเดล บนเครื่องที่ใช้ร่วมกัน ให้เช็คแรมว่างระหว่างรันคำสั่ง inference ใหม่ทุกครั้งด้วย

สูง 2026-07-25

เครื่อง GPU ที่เช่าไว้อาจยังรันอยู่ เพราะไม่เคยยืนยันว่าถูกทำลายจริง

เกิดอะไรขึ้น: ยืนยันข้อมูลสำรองไว้แล้วสองแหล่งอิสระ และไฟเขียวให้ทำลายเครื่องที่เช่าไว้ แต่ไม่เคยยืนยันว่าการทำลายเกิดขึ้นจริง Phase 11 ยังไม่ถูกตี๊กและต้องถือว่าเครื่องยังรันอยู่ที่ราว \$0.111/ชม.

ต้นเหตุจริง: ขั้นตอนส่งมอบตอนปิดงานไม่มีการยืนยันเลย — ทุกคนยกเว้นในไดอารี่ถือว่า 'ให้ไฟเขียวแล้ว' กับ 'เครื่องถูกทำลายแล้ว' เป็นเหตุการณ์เดียวกัน

เสียหายอะไร: ค่าใช้จ่าย GPU ยังเดินต่อโดยไม่รู้ว่านานแค่ไหน รายการนี้ยังโยงไปถึงเหตุการณ์ 'วันแห่งความอัปยศ' ก่อนหน้าเมื่อวันที่ 21 ที่ไม่ได้สำรองข้อมูลก่อนทำลายเครื่อง

แก้ยังง: ยังไม่ได้แก้ — บันทึกไว้ตามจริงว่ายังไม่ยืนยัน และถือว่าเครื่องยังรันอยู่จนกว่าผู้ใช้จะยืนยัน

กฎที่ได้: การปิดงานยังไม่จบจนกว่า provider จะยืนยันจริง ต้องเช็ครายชื่อเครื่องที่กำลังรัน อย่ายอมรับ 'ให้ไฟเขียวแล้ว' ว่าเท่ากับทำเสร็จ เงินยังเผาไหม้ต่อในช่วงว่างระหว่างการอนุมัติกับการลงมือทำจริง

สูง 2026-07-28

บ้าน07 EE-03/04: สวิตช์ (จุดทึบ) ถูกนับปนกับโคมไฟอย่างเป็นระบบ กลุ่มหนึ่งหายไปทั้งกลุ่ม อีกแถวถูกใส่ผิดห้อง

เกิดอะไรขึ้น: ตลอดแบบไฟฟ้าทั้งชุด สัญลักษณ์สวิตช์จุดทึบถูกนับเป็นโคมไฟ — เป็นแบบระบบ ไม่ใช่แค่ครั้งเดียว พอนับใหม่ได้ ชั้น 1 = โคมไฟ 29 จุด (กลุ่ม b เคยหายไปทั้งกลุ่ม) และชั้น 2 = 19 จุด (แถว d เคยถูกใส่ผิดห้อง)

ต้นเหตุจริง: การจำแนกสัญลักษณ์ทำโดยดูรูปร่างคล้ายกันแทนที่จะเทียบกับ legend ของแบบเอง และมันเป็นความผิดพลาดแบบระบบเพราะใช้กฎผิดตัวเดียวกันทุกที่ กลุ่มที่หายและแถวที่ผิดห้องแสดงว่าไม่เคยมีการทวนยอดนับต่อห้องเทียบกับ legend เลย

เสียหายอะไร: จำนวนโคมไฟผิดทั้งสองชั้นในสองทิศทาง — เกินจริงจากสวิตช์ที่นับปน และขาดจริงจากกลุ่มที่หายไป กระทบปริมาณงานและต้นทุนงานไฟฟ้า

แก้ยังง: นับใหม่ทั้งหมดแยกตามชนิดสัญลักษณ์เทียบกับ legend บันทึกยอดที่ถูกต้องคือ 29 และ 19

กฎที่ได้: นับสัญลักษณ์แต่ละชนิดแยกกันเทียบกับ legend ของแบบเสมอ และทวนยอดต่อห้อง ห้ามรวมสัญลักษณ์ที่หน้าตาคล้ายกันเป็นกลุ่มเดียว ความผิดพลาดจากการจำแนกแบบระบบจะทำให้ทุกแผ่นที่มันแตะเสียหายไปด้วย จึงต้องแก้กฎแล้วนับใหม่ทั้งหมด ไม่ใช่แค่แก้จุดที่เจอ

สูง 2026-07-28

บ้าน07 M-02/03: อ่านความใกล้เคียงกันในแปลนเป็นการวางซ้อนจริง — คู่ CU, ห้องของ CEF-3, และหลังคาของ CU-2/01 วางผิดพลาด

เกิดอะไรขึ้น: ความผิดพลาดเรื่องตำแหน่งงานระบบเครื่องกล 3 จุด มาจากสาเหตุเดียวกัน: คู่ CU ถูกบันทึกว่าวางซ้อนกันทั้งที่จริงเรียงกันตามแนวนอน, CEF-3 ถูกวางในโถงบันไดแทนที่จะเป็นห้องน้ำ, และ CU-2/01 ถูกวางบนปีกที่ปีกกระเบื้องของ S-05 แทนที่จะเป็นหลังคาแผ่นเหล็กของโครงสร้าง S-04

ต้นเหตุจริง: แปลน 2 มิติฉายภาพการจัดวางแบบ 3 มิติลงบนพื้นผิวเดียว สัญลักษณ์สองตัวที่วาดใกล้เคียงกันถูกตีความว่าซ้อนกันจริงในเชิงกายภาพ และการกำหนดห้อง/หลังคาก็อนุมานจากความใกล้เคียงในแปลนแทนที่จะไล่ไปตามขอบเขตที่ล้อมรอบหรือโครงสร้างที่อยู่ด้านล่าง

เสียหายอะไร: อุปกรณ์ 3 ชั้นวางตำแหน่งผิด กระทั่งว่าโครงสร้างไหนต้องรับน้ำหนักมัน (หลังคาแผ่นเหล็กกับปีกกระเบื้องคือคนละ load case กันเลย) และเป็นของห้องไหน

แก้ยังไง: แก้โดยอ่านใหม่เทียบกับขอบเขตห้องจริงและผังโครงสร้าง S-04/S-05 บันทึกไว้เป็นหนึ่งใน 3 ประเภทข้อผิดพลาดที่เกิดขึ้น

กฎที่ได้: ห้ามอนุมานการจัดวางแบบ 3 มิติหรือการอยู่ในขอบเขตจากความใกล้เคียงกันในแปลน 2 มิติเด็ดขาด ต้องไล่อุปกรณ์ไปหาขอบเขตห้องที่ล้อมรอบ และไปหาโครงสร้างที่วาดอยู่ด้านล่างในแบบโครงสร้าง ก่อนจะบันทึกตำแหน่ง

สูง 2026-07-28

RAM ที่ Vast.ai แจงไว้ 76GB หลอกหลวง — เครื่องจริงมี 188GB — และก่อนหน้านี้ไม่เคยเอา RAM มาเป็นเกณฑ์เลือกเครื่องเลย

เกิดอะไรขึ้น: เครื่องที่ดูไว้แจ้ง RAM 76GB ต่ำกว่าเกณฑ์ใหม่ที่ตั้งไว้ ($\geq 80\text{GB}$) จนต้องออกแบบมาตรการป้องกัน 3 ชั้น (เตือนใน verify_env, แปลง adapter เป็น HF ก่อนทำ GGUF, แผนสำรอง swap 64GB) แต่พอเช่าจริงเครื่องกลับมี RAM 188GB แกมรอบ t01 ที่ผ่านมาก็ไม่เคยตั้งเกณฑ์ RAM เลยแม้แต่น้อย รอดมาได้เพราะเครื่องตอนนั้นบังเอิญมีพอ

ต้นเหตุจริง: เกิดปัญหาซ้อนกันสองอย่าง: ตัวเลข RAM ที่ผู้ขายโฆษณาไม่ตรงกับของจริงที่ได้รับ และทรัพยากรที่จริงๆ แล้วเป็นตัวฆ่างาน merge bf16 30B (RAM ของเครื่อง ราวๆ 60GB ถ้าไม่พอจะโดน Killed แบบเสียบๆ) ไม่เคยถูกนำมาเป็นเกณฑ์เลือกเครื่องมาก่อน เพราะไม่เคยเป็นคอขวดจริงมาก่อน

เสียหายอะไร: งานวางแผนและมาตรการป้องกันทั้งหมดทำขึ้นจากตัวเลขที่ผิด และยังเปิดช่องโหว่แฝงไว้ด้วย — รอบก่อนๆ อาจเคยเสี่ยงโดน RAM ฆ่างานแบบเสียบๆ โดยไม่มีเกณฑ์อะไรป้องกันเลย

แก้ยังไง: ตั้งเกณฑ์ RAM $\geq 80\text{GB}$, ขยาย verify_env.py ให้อ่านค่า RAM จริงจาก /proc/meminfo และวางแผนมาตรการป้องกัน 3 ชั้นรวมถึงแผนสำรอง swap ที่เขียนไว้เป็นลายลักษณ์อักษร ซึ่งช่วยแก้ปัญหาได้โดยไม่ต้อง train ใหม่

กฎที่ได้: ต้องตรวจสอบสเปกที่โฆษณาไว้กับเครื่องจริงที่ได้รับก่อนเชื่อ และตั้งเกณฑ์เลือกเครื่องจากทุกทรัพยากรที่มีโอกาสฆ่างานได้จริง รวมถึงตัวที่รอบก่อนรอดมาได้เพราะโชคช่วยด้วย ถ้าเจอ 'Killed' แบบเสียบๆ ให้สงสัย RAM ของเครื่อง ไม่ใช่ GPU

สูง 2026-07-28

เครื่อง GPU ที่เช่าไว้ถูกปล่อยให้รันต่อทั้งที่งานจบแล้ว ยังคิดเงินอยู่ที่ \$1.161/ชั่วโมง

เกิดอะไรขึ้น: ตอนปิดงานวันนั้น เครื่องยังไม่ได้ shutdown เพราะติดเงื่อนไข hard block ของ Phase 11 ที่ต้องรอคำสั่งชัดเจนจากมะขามก่อน จึงยังคิดเงินต่อไปเรื่อยๆ จนกว่าจะได้คำสั่งนั้น

ต้นเหตุจริง: เกทที่ต้องรอคนอนุมัติก่อน shutdown นั้นถูกต้องแล้ว (ป้องกันไม่ให้ทำลายงานที่ยังไม่ได้สำรองข้อมูล) แต่ไม่มีกลไกอะไรที่รับประกันว่าจะมีการขออนุมัตินั้นจริงๆ ก่อนจบ session กฎความปลอดภัยจึงกลายเป็นค่าใช้จ่ายที่ไม่มีขอบเขตแทน

เสียหายอะไร: เสียเงินต่อเนื่องที่ \$1.161/ชั่วโมงกับเครื่องที่ไม่ได้ทำงานอะไรเลย เป็นเวลาไม่จำกัดหลังจากงานที่มีประโยชน์ทั้งหมดเสร็จแล้วและข้อมูลถูกยืนยันว่าปลอดภัยแล้ว

แก้ยังไง: ยังไม่ได้แก้ — บันทึกไว้ในรายการ Pending พร้อมโน้ตว่าข้อมูลทั้งหมดปลอดภัยอยู่แล้วไม่ว่าจะ shutdown เมื่อไหร่ก็ตาม

กฎที่ได้: เกทที่ต้องรอคนอนุมัติก่อน shutdown ต้องมาพร้อมเส้นตายหรือ dead-man's switch เสมอ พอยืนยันว่าสำรองข้อมูลแล้ว เครื่องที่ยังรันเปล่าๆ คือเงินเสียเปล่าล้วนๆ ต้องขออนุมัติให้ชัดเจนก่อนจบ session อย่าปล่อยเป็นแค่ to-do ที่ค้างไว้

สูง 2026-07-28

ปฏิเสธตัวเลือก B (train t01 ใหม่ที่ resize='max') ทำให้คำถามหลักของทั้งรอบไม่มีคำตอบไปตลอด

เกิดอะไรขึ้น: เมื่อเจอว่าการเปรียบเทียบไม่ valid มีสองทางเลือก: ยอมรับผลพร้อมค่าเตือน หรือ train t01 ใหม่ที่ resolution เดียวกันด้วยค่าใช้จ่ายเพิ่มอีกแค่ \$0.60-1.80 สุดท้ายเลือกทางแรก คำถาม A/B ที่เป็นเป้าหมายของทั้งรอบจึงยังไม่มีคำตอบ และถ้าจะตอบทีหลังต้อง train t01 ใหม่อยู่ดี

ต้นเหตุจริง: การตัดสินใจเกิดขึ้นหลังจากเห็น sunk cost ของทั้งวันไปแล้ว ค่าใช้จ่ายเพิ่มเติมจึงดูเหมือนเป็นเรื่องแยกต่างหาก ทั้งที่จริงๆ ค่าเสียหายที่ใหญ่กว่ามากคือการที่ทั้งรอบไม่ได้คำตอบอะไรเลย

เสียหายอะไร: ทั้งรอบการ tune ที่เสียเงินไปพร้อมกับการวิเคราะห์อีกทั้งวัน ไม่ได้คำตอบของคำถามที่ตั้งไว้เลย งานที่ค้างอยู่ตอนนี้ใหญ่กว่าค่าใช้จ่าย \$0.60-1.80 ที่จะปิดมันได้ทันทีตอนนั้นเสียอีก

แก้ยังไง: ยังไม่ได้แก้ — บันทึกไว้ในรายการ Pending ว่าต้อง train t01 ใหม่ด้วย resize='max' ก่อน ถึงจะตอบคำถามว่า 'ตระกูลไหนดีกว่าจริง' ได้อย่างเป็นธรรม

กฎที่ได้: เมื่อพบว่าการทดลองไม่ valid ต้องชั่งน้ำหนักค่าใช้จ่ายในการแก้ไขกับค่าใช้จ่ายทั้งหมดที่ลงไปแล้ว ไม่ใช่เทียบกับศูนย์รอบที่ไม่ตอบคำถามอะไรเลย คือรอบที่เสียค่าใช้จ่ายมากกว่าค่าแก้ไขไปแล้ว

สูง 2026-07-29

โมเดล 21GB บนการ์ดจอ 8GB — CPU offload ทำให้ inference บนเครื่องท้องถิ่นใช้เวลาประมาณ 20 ชั่วโมง/บ้าน

เกิดอะไรขึ้น: เลือกใช้ Q4_K_M เพื่อให้โมเดลรันบนโน้ตบุ๊กได้ แต่ 21GB ก็ยังเกิน VRAM 8GB อยู่ดี llama.cpp เจอ error 'failed to fit params to free device memory ... abort' ประมาณ 2 ใน 3 ของโมเดลต้องไปรันบน CPU แทน ความเร็วในการ generate ตกเหลือ 3.46 t/s หนึ่งหน้าใช้เวลา 13-21 นาที รวมแล้วประมาณ 20 ชั่วโมง/บ้าน คุณ 5 บ้าน

ต้นเหตุจริง: เลือกระดับ quantization เพื่อลดขนาดไฟล์เฉยๆ ไม่ได้เลือกให้ตรงกับ VRAM ที่วัดไว้จริง ไม่มีใครคำนวณเลยว่า '21GB ใส่ใน 8GB ได้ไหม' ก่อนที่จะตัดสินใจใช้แผน inference บนเครื่องท้องถิ่น

เสียหายอะไร: แผน extraction บนเครื่องท้องถิ่นทั้งหมดใช้งานจริงไม่ได้เลยในเชิงเศรษฐศาสตร์ด้วยความเร็วขนาดนี้ และการยอมเสียคุณภาพจากการ quantize ก็ไม่ได้แตกต่างกับการ 'พอดี้' อย่างที่คิดไว้เลยด้วยซ้ำ

แก้ยังไง: ยังไม่ได้แก้ — เสนอให้เช่า GPU แต่ตัดสินใจอย่างถูกต้องแล้วว่าให้ออนกกว่าจะตัดสินใจเรื่อง LoRA ให้เรียบร้อยก่อน

กฎที่ได้: ก่อนเลือกระดับ quantization ให้เขียนตัวเลขจำนวนไว้ให้ชัด: ขนาดโมเดลเทียบกับ VRAM ที่มี พร้อมเผื่อพื้นที่สำหรับ context ถ้าไม่พอดี้ การจะ quantize ต่อหรือจะเช่าเครื่องเป็นการตัดสินใจที่ต้องทำล่วงหน้า ไม่ใช่มาเจอเอาจาก error message กลางการรันงาน

สูง 2026-07-29

เกือบเช่า GPU เพื่อแก้ปัญหา 'ช้า' ทั้งที่ปัญหาคือ 'ผิด'

เกิดอะไรขึ้น: มีข้อเสนอให้เช่า GPU ($\geq 24\text{GB}$, $\$0.2\text{--}0.5/\text{ชม.}$) เพื่อเร่งความเร็วการ extraction แต่จับได้ทันเวลาว่าการเช่าแก้ได้แค่เรื่องความเร็ว ถ้า LoRA หายไปจริงๆ จาก GGUF เครื่องที่เช่ามาก็จะให้ผลลัพธ์ที่ใช้ไม่ได้แบบเดิม เพียงแต่เร็วขึ้น 10 เท่าเท่านั้น

ต้นเหตุจริง: เป็นปฏิกิริยาแบบสัญชาตญาณที่จะแก้อาการที่มองเห็น (ความช้า) ก่อนที่การวินิจฉัยปัญหาที่มองไม่เห็น (คุณภาพ) จะเสร็จสมบูรณ์

เสียหายอะไร: ยังไม่เสียเงินไป เพราะจับได้ทันก่อนจะใช้จ่าย ถ้าไม่ทันจะเสียทั้งค่าเช่าเครื่องบวกกับผลลัพธ์ที่ใช้ไม่ได้อีกรอบหนึ่ง

แก้ยังไง: แก้แล้ว — บล็อกการเช่าเครื่องไว้ชัดเจนจนกว่าจะทำการทดสอบเปรียบเทียบกับ base model เสร็จก่อน

กฎที่ได้: ห้ามซื้อกำลังประมวลผลเพิ่มเพื่อแก้ปัญหาคุณภาพเด็ดขาด ต้องเรียงลำดับงานตาม 'อะไรที่เปลี่ยนการตัดสินใจได้' การวินิจฉัยฟรีๆ 20 นาทีต้องมาก่อนขั้นตอนที่เสียเงินเสมอ เพราะถ้าวินิจฉัยผิด จะยังทำให้ความล้มเหลวเกิดเร็วขึ้นและแพงขึ้นเท่านั้น

สูง 2026-07-29

เครื่อง Vast.ai ที่เช่าไว้ตั้งแต่วันที่ 28 ยังไม่ได้ shutdown

เกิดอะไรขึ้น: การ shutdown เครื่อง Vast.ai ที่เช่าไว้เป็น hard block ของวันที่ 28 (Phase 11) และจนถึงวันที่ 29 ก็ยังถูกบันทึกไว้ว่าค้างอยู่และยังไม่ได้ทำ

ต้นเหตุจริง: ขั้นตอน teardown อยู่ในเช็คลิสต์แบบข้อความท้ายรอบทำงาน จึงเป็นสิ่งที่ถูกลืมเมื่อ session จบกะทันหันหรือความสนใจย้ายไปที่ปัญหาถัดไป

เสียหายอะไร: เสียค่าเช่ารายชั่วโมงต่อเนื่องกับเครื่องที่ไม่ได้ทำงานอะไรเลย เป็นเวลากว่าหนึ่งวัน

แก้ยังไง: ยังไม่ได้แก้ — ยังบันทึกไว้ว่าเป็นงานค้างจากวันที่ 28

กฎที่ได้: การปิดเครื่องไม่ควรเป็นแค่รายการในเช็คลิสต์ ควรเป็นเส้นตายอัตโนมัติ: ตั้ง self-destruct/auto-stop ให้เครื่องตั้งแต่อนเช้า เพื่อให้ลืมแล้วไม่ต้องเสียเงิน ห้ามให้ 'การปิดเครื่อง' ขึ้นอยู่กับวามนุษย์จะจำได้หลังจากงานที่นำสนใจจบไปแล้วหรือไม่

สูง 2026-07-29

เกือบเช่า GPU เพื่อแก้ปัญหาคุณภาพผลลัพธ์แย่ ทั้งที่จะเสียเงินเปล่า

เกิดอะไรขึ้น: ตอนที่การสกัดข้อมูลเข้าและได้ผลแย่ การเช่า GPU ดูเป็นทางออกที่เป็นธรรมชาติ แต่ถูกสกัดกั้นตรงๆ ว่า: เช่า GPU แก้ได้แค่เรื่องความเร็ว ไม่ได้แก้ความถูกต้อง ถ้า LoRA หายไปจริงจากไฟล์ GGUF การรันบน GPU เช่าก็ได้ขยะแบบเดิม แค่เร็วขึ้น

ต้นเหตุจริง: อาการสองอย่างที่แยกกัน (ช้า, ผิด) ล่อให้แก้ด้วยวิธีแพรววิธีเดียว โดยไม่หาต้นเหตุของ 'ผิด' ก่อน เงินเลยจะถูกทุ่มไปที่อาการที่ประเมินราคาจ่ายที่สุด ไม่ใช่ต้นเหตุจริง

เสียหายอะไร: หลีกเสี่ยงได้หัน — สั่งให้เปรียบเทียบกับ base model แบบฟรี 20 นาทีก่อน ถ้าข้ามขั้นนี้ไป เงินค่าเช่า GPU จะให้ผลลัพธ์แย่แบบเดิมเป๊ะ

แก้ยังไง: ตั้งลำดับความสำคัญใหม่: (1) เปรียบเทียบกับ base model แบบฟรีก่อน เพื่อยืนยันว่ามี LoRA จริง (2) แก้วตรวจสอบตอน export แบบฟรี (3) ค่อยตัดสินใจเรื่องที่ต้องเสียเงิน และห้ามสกัดข้อมูลบ้าน 01 ต่อระหว่างนี้เด็ดขาด

กฎที่ได้: ห้ามใช้เงินแก้อาการที่ยังไม่หาต้นเหตุจริง รันทุกวิธีตรวจสอบที่ฟรีก่อนเสมอ และหยุดสร้างผลลัพธ์เข้า dataset ในขณะที่ pipeline ยังถูกสงสัยว่ามีปัญหา

สูง 2026-08-01

commit 5 ตัวรวม PNG แบบก่อสร้างหนัก ~2.8GB ค้าง push ไม่ขึ้นเป็นเวลานาน เพราะ GitHub จำกัด push ครั้งละไม่เกิน 2GB

เกิดอะไรขึ้น: repo Training มี commit เก่า 5 ตัวรวม PNG แบบบ้านหนัก ~2.8GB ที่ push ไม่ขึ้น สาเหตุคือ GitHub จำกัด push ต่อครั้งไม่เกิน 2GB ต้องแบ่งเป็นก้อนละ ~500MB แล้ว push ทีละก้อน

ต้นเหตุจริง: ไฟล์ binary ขนาดใหญ่ถูก commit ลง git ตรงๆ โดยไม่มีการคำนึงถึงขนาด และตอน push ล้มเหลวครั้งแรกก็ไม่ได้หาสาเหตุ บ่อยครั้งไว้เป็นสัปดาห์

เสียหายอะไร: งานหนัก ~2.8GB มีอยู่แค่นับถือปเครื่องเดียวเป็นเวลานานโดยไม่มีสำรองข้อมูลบน remote เลย แล้วต้องเสียเวลาทั้งเซสชันไปกับการแบ่งและ push ทีละก้อน

แก้ยังไง: แบ่งประวัติเป็นก้อนละ ~500MB แล้ว push เรียงลำดับจนครบ แล้วตรวจสอบว่า diff ตรงกับต้นฉบับ 100%

กฎที่ได้: หาสาเหตุของ push ที่ล้มเหลวทันทีที่มันเกิด — commit ที่ push ไม่ขึ้นก็คือ งานที่ไม่มีสำรองข้อมูล พยายาม push ให้แต่ละครั้งไม่เกิน 2GB และเก็บไฟล์ binary ก้อนใหญ่ไว้นอก git (หรือใช้ LFS) ตั้งแต่แรก

สูง 2026-08-01

ku_project_jop push ไม่ได้เลย — ไฟล์ zip 2.1GB ฝังอยู่ในประวัติ commit

เกิดอะไรขึ้น: repo `ku\_project\_jop` push ไม่ได้เลย เพราะมีไฟล์ zip 2.1GB ('ข้อมูลจากรุ่นพี่') ถูก commit ไว้ แล้วค่อยลบใน commit ถัดมา — แต่ blob ยังอยู่ในประวัติ push ทุกครั้งจึงยังต้องแบกไฟล์นั้นไปด้วย ต้องเขียนประวัติใหม่เพื่อลบมันออก

ต้นเหตุจริง: ไฟล์ archive ขนาดใหญ่ถูก commit โดยไม่ได้ตรวจสอบขนาดก่อน แล้ว 'วิธีแก้' คือ `rm` แล้ว commit ซึ่งไม่ได้ลบ blob ออกจากประวัติเลย เป็นความเข้าใจผิดที่พบบ่อยมากเกี่ยวกับวิธีที่ git เก็บข้อมูล

เสียหายอะไร: ทั้ง repo push ไม่ได้และไม่มีสำรองข้อมูลเลย วิธีแก้ (เขียนประวัติใหม่) เป็นการกระทำที่ทำลายล้าง (destructive) และยังรอกการยืนยันอยู่ ณ ตอนที่เขียนบันทึกนี้

แก้ยังไง: ยังไม่ได้แก้ — รอกการยืนยันเพื่อเขียนประวัติใหม่และตัด blob ออก

กฎที่ได้: การลบไฟล์ใหญ่ใน commit ถัดมา ไม่ได้ทำให้ repo เล็กลงเลย ห้าม commit archive ขนาดหลาย GB เด็ดขาด ถ้าเผลอ commit ไปแล้ว ให้เขียนประวัติใหม่ทันที (filter-repo/BFG) ก่อนที่จะมี commit อื่นทับซ้อนขึ้นไปอีก

สูง 2026-08-02

การสั่งปิดเครื่องของ op3 รู้จักแค่บ้านของตัวเอง — มันจะฆ่างานอื่นบนเครื่องไปด้วย

เกิดอะไรขึ้น: บันทึกลงไปเป็นข้อจำกัดตั้งแต่ตอนสร้าง op3 ว่าห้ามใช้ op3 ตอนที่มีอย่างอื่นรันอยู่บนเครื่อง (งานเทรน งานอัปโหลด หรือ agent ตัวอื่น) เพราะด้านตรวจของมันเป็นแค่บ้านของตัวเองเสร็จหรือยัง

ต้นเหตุจริง: ตัวตรวจว่างานเสร็จมองแค่งานชิ้นเดียว แต่สิ่งที่มันสั่งให้ทำ (ปิดเครื่อง) มีผลกับทั้งเครื่อง

เสียหายอะไร: เป็นอันตรายที่ซ่อนอยู่: ถ้ารัน op3 ตอนงานเทรนหรืองานอัปโหลดกำลังวิ่งอยู่ งานนั้นจะพังแบบเดียวกับเหตุการณ์ Mark of Shame ครั้งแรกเป๊ะ

แก้ยังไง: เขียนเป็นข้อห้ามการไว้ใ้ในนิยามของ op3 แทนที่จะบังคับด้วยโค้ด

กฎที่ได้: ถ้าตัวตรวจแบบมองเฉพาะจุดไปสั่งการกระทำที่ทำลายทั้งระบบ ต้องเลือกอย่างใดอย่างหนึ่ง: จำกัดขอบเขตของการกระทำ หรือไปตรวจสอบสถานะทั้งเครื่อง ข้อห้ามที่เขียนไว้เนี่ยๆ อ่อนกว่าโค้ดที่เช็คว่ามีงานอื่นรันอยู่ไหมก่อนปิดเครื่อง

สูง 2026-08-20

เช่าเครื่อง Vast.ai ที่มีดิสก์แค่ 78GB เพราะปล่อยแถบเลื่อนไว้ที่ค่าเริ่มต้น — โดนกับดักนี้เป็นครั้งที่ 3

เกิดอะไรขึ้น: เครื่องแรกที่เช่ามารัน inference ของ t02 มีดิสก์แค่ 78GB ซึ่งเล็กเกินไปสำหรับงานนี้ เพราะแถบเลื่อนขนาดดิสก์ของ Vast.ai ตั้งค่าเริ่มต้นไว้ต่ำและไม่ได้เลื่อนขึ้น บันทึกระบุว่าเป็นครั้งที่ 3 แล้วที่โดนกับดักเดิม

ต้นเหตุจริง: ฟอรัมเช่าเครื่องของ Vast.ai ตั้งค่าดิสก์เริ่มต้นไว้ต่ำกว่าที่งาน VLM ระดับ 30B ต้องการ และไม่มี checklist ก่อนเช่าที่บังคับให้เปลี่ยนค่านี้ ขั้นตอนที่ต้องทำด้วยมือขั้นเดิมเลยถูกลืมทุกครั้งตอนรับ

เสียหายอะไร: เสียค่าเช่าไปรอบหนึ่ง บวกเวลาที่ใช้ไปกับการหาว่าปัญหาคืออะไร ทำลายเครื่อง แล้วเช่าใหม่ รวมแล้วโดนแบบเดียวกัน 3 ครั้ง ทั้งที่เสี่ยงได้

แก้ยังไง: ทั้งเครื่องนั้นไป สุดท้ายเช่าเครื่องที่สาม (ดิสก์ 158GB) แล้วใช้งานได้

กฎที่ได้: เขียนตัวเลขขั้นต่ำของ disk/VRAM/RAM ลงในเอกสาร workflow เตรียมงานเป็น checklist ที่ต้องติ๊กให้ครบก่อนกดเช่า — กับดักที่โดนซ้ำ 3 ครั้งคือ checklist ที่ขาดไป ไม่ใช่ความสะเพร่า

สูง 2026-08-20

อ่านราคา GPU ผิดเป็น \$0.058/ชม. ทั้งที่จริง \$1.792/ชม. — ผิด 30 เท่า จับได้ตอนวางแผนรันไป ~11 ชม.แล้ว

เกิดอะไรขึ้น: ราคาของเครื่องที่ใช้งานได้ถูกบันทึกว่า \$0.058/ชม. แต่ราคาจริงคือ \$1.792/ชม. ความผิดพลาด 30 เท่านี้เพิ่งสังเกตเห็นตอนประเมินว่าจะใช้เวลาราว 11 ชั่วโมง / \$20 — คือหลังจากวางแผนทั้งหมดบนฐานต้นทุนที่ผิดไปแล้ว

ต้นเหตุจริง: อ่านตัวเลขราคาจากหน้ารายการของ Vast.ai โดยไม่เทียบกับอัตราคิดเงินจริงของเครื่องที่กำลังรันอยู่ และไม่ได้ตรวจสอบความสมเหตุสมผล ('\$0.058/ชม. สำหรับ RTX PRO 6000 96GB เป็นไปได้จริงหรือ') กับตัวเลขที่ถูกฉกฉวยผิดปกติ

เสียหายอะไร: การวางแผนต้นทุนทั้งเซสชันผิดไป 30 เท่า จับได้ก่อนที่จะบานปลาย แต่ทุกการตัดสินใจเรื่องขอบเขตงานก่อนหน้านี้ตั้งอยู่บนตัวเลขเศรษฐศาสตร์ที่ผิด

แก้ยังไง: แก้เป็น \$1.792/ชม. แล้วหลังจากนั้นต้องตัดขอบเขตงานลงเรื่อยๆ เมื่อเห็นต้นทุนและเวลาจริง

กฎที่ได้: อ่านอัตราค่าเช่าต่อชั่วโมงจากเครื่องที่กำลังรันอยู่ ไม่ใช่จากหน้ารายการ และตรวจสอบความสมเหตุสมผลด้วย — การ์ดเรือธง 96GB ราคาต่ำกว่า \$0.10/ชม. คือการอ่านผิด ไม่ใช่ของถูก แล้วเขียนอัตราที่ยืนยันแล้วลงในเอกสาร workflow ก่อนเริ่มงาน

SSH หลุดกลางคันตอนสั่ง nohup หลังรีสตาร์ท ทำให้ job เก่าถูกฆ่าแล้ว job ใหม่ไม่เคยเริ่มเลย

เกิดอะไรขึ้น: หลังจัดลำดับใหม่กลางการรัน process batch เก่าถูกฆ่าทิ้งแล้วสั่ง process ใหม่ผ่าน nohup — แต่ SSH หลุดระหว่างการสั่ง ทำให้ process ใหม่ไม่เคยเริ่มทำงานจริง ไม่มีอะไรกำลังรันอยู่เลย เจอปัญหานี้ได้จากการเช็ค pgrep หลังจากนั้นไม่ใช่เชื่อ exit code ของคำสั่งที่ส่งไป

ต้นเหตุจริง: nohup ที่รันผ่าน SSH session ที่ไม่มั่นคง ไม่ให้การยืนยันที่เชื่อถือได้ว่า process รอดจากการหลุดการเชื่อมต่อ exit code ของคำสั่งบอกแค่ shell fork สำเร็จ ไม่ได้บอกว่า job ยังทำงานต่อจริง การฆ่า job เก่าก่อนแล้วค่อยรันใหม่ที่หลัง ทำให้เกิดช่วงเวลาที่ไม่มีการรัน job อยู่นั่นเอง

เสียหายอะไร: เสียเวลาไปเสียๆ บนเครื่องที่เช่าอยู่ในอัตรา \$1.792/ชม. ท่ามกลางเดดไลน์ตีหนึ่งที่กำลังจะพลาด รอดมาได้เพราะมีการตรวจสอบจริง ไม่ใช่แค่สันนิษฐาน

แก้ยังไง: ตรวจสอบด้วย pgrep แล้วรัน job ใหม่อีกครั้ง ไม่มีการเปลี่ยนเครื่องมือใดๆ บันทึกลงไว้

กฎที่ได้: ห้ามเชื่อ exit code ของคำสั่งที่ส่งบนเครื่อง remote เด็ดขาด ต้องยืนยันด้วย pgrep/ps หลังสั่งรันเสมอ และควรใช้ tmux/screen แทน nohup เพื่อไม่ให้ SSH ที่หลุดทิ้ง orphan job ไว้ และควรเริ่ม job ใหม่ก่อนฆ่า job เก่าถ้าทำได้

ต้นเหตุจริงของความล้มเหลวหลังใส่ grammar: lm-format-enforcer ปลอຍค่อมมาท้ายออกมา ซึ่ง json ของ Python ไม่รับ

เกิดอะไรขึ้น: หลังใส่ grammar-constrained decoding เข้าไปแล้ว รอบแรกก็ยังได้ JSON ที่ parse ไม่ผ่าน ต้นเหตุจริง (เจอหลังไล่อ่านโค้ดทีละชั้น) คือ lm-format-enforcer มองว่า {"a":1,} — มีค่อมมาก่อนปิดปีกกา — เป็น JSON ที่ถูกต้อง แต่ตัว parser ของ Python ไม่รับ คือสองตัวนี้นิยามคำว่า JSON ที่ถูกต้อง ไม่ตรงกัน พิสูจน์ได้ในบรรทัดเดียว: ปลอຍค่อมมาเกินออกจากไฟล์ที่ พัง แล้วมัน parse ผ่านทันที

ต้นเหตุจริง: เอา grammar backend มาใช้โดยไม่ตรวจสอบว่านิยาม JSON ที่ถูกต้องของมันตรงกับของฝั่งที่รับไปใช้ (Python json.loads) หรือเปล่า grammar ทำงานถูกต้องมาตลอด แต่นิยามคำว่า ถูก ของสองฝั่งไม่ตรงกันเฉยๆ

เสียหายอะไร: เสียรอบไล่บั๊กไปหนึ่งรอบบนเวลา GPU ที่เข้ามา แถมยังเขียนสรุปผิดๆ ลง SUMMARY.md แบบมั่นใจว่าเป็นความผิดของ Unsloth และเกือบทำให้ทั้งแนวทาง grammar ถูกทิ้งไปเพราะคิดว่าใช้ไม่ได้

แก้ยังไง: เปลี่ยน backend ตั้งต้นไปใช้ xgrammar (ซึ่งออกค่อมมาท้ายไม่ได้ตั้งแต่ระดับ grammar) แล้วลด lmfe เป็นตัวสำรอง บันทึกลงไว้เป็น rule_of_tune บทเรียนข้อ 13

กฎที่ได้: เวลาชั้นที่คอมไพล์โครงสร้างกับ parser เห็นไม่ตรงกัน ให้เอนิยามคำว่า ถูกต้อง ของทั้งสองฝั่งมาเทียบกันก่อนจะโทษฝ่ายไหน และให้เลือก backend ที่ grammar ของมันทำให้ผลลัพธ์ผิดเกิดขึ้นไม่ได้ทางกายภาพ ไม่ใช่แค่เกิดยาก

สูง 2026-08-24

อ่านราคา Vast.ai ผิดไปราวๆ 20 เท่าเป็นครั้งที่สอง — หน้าเว็บโชว์ค่าพื้นที่เก็บข้อมูล ไม่ใช่ค่า GPU

เกิดอะไรขึ้น: หน้าเว็บ Vast.ai แสดง \$0.056/ชม. แต่คำนวณจาก DLPerf/\$ แล้วได้ประมาณ \$1.16/ชม. บิลจริงของเครื่องรอบเช้าคือ \$5.43 โดย GPU อยู่ที่ \$1.1067/ชม. ตัวเลข \$0.056 คือค่าพื้นที่เก็บข้อมูล เรื่องนี้ถูกระบุไว้ว่าเป็น คลาสเดียวกับที่อ่านผิด 30 เท่าเมื่อ 2026-08-20 คือพลาดแบบเดียวกันมาแล้วเมื่อสัปดาห์ก่อน

ต้นเหตุจริง: หน้าเว็บวางค่าพื้นที่เก็บข้อมูลไว้ในตำแหน่งที่อ่านแล้วเข้าใจว่าเป็นราคาเครื่อง และทีมก็ยังดูราคาจากหน้าเว็บแทนที่จะดูจาก API ครั้งที่เกิดเมื่อ 2026-08-20 ไม่ได้เขียนกฎอะไรไว้ มันเลยเกิดซ้ำ

เสียหายอะไร: การวางแผนงบผิดไปราวๆ 20 เท่าในการตัดสินใจเช่าเครื่องอย่างน้อยสองครั้ง และบิลไม่ถูกตรวจสอบตลอดเกือบทั้งวันในขณะที่ตัดสินใจอยู่บนราคาที่ผิด

แก้ยังไง: ไปเช็คบิลจริงผ่าน vastai CLI (GPU \$1.1067/ชม., storage 0.24, DL 0.48, รวม \$5.43) ตั้งกฎประจำใหม่: ดูราคาจาก vastai search offers (ฟิลด์ dph_total) เท่านั้น ห้ามดูจากหน้าเว็บ

กฎที่ได้: ห้ามเอาราคาค่าเช่ามาจากหน้าเว็บของผู้ให้บริการ ให้อ่านจากฟิลด์ใน API/CLI ที่มันใช้เก็บเงินจริง (dph_total) และถ้าเจอเรื่องราคาผิดพลาดครั้งหนึ่ง ให้เขียนกฎไว้วันนั้นเลย ไม่งั้นมันจะเกิดซ้ำ

สูง 2026-08-24

เรนรอตแต่ eval หาย epoch OOM — accelerate อับทั้ง logits tensor เป็น fp32

เกิดอะไรขึ้น: TEST_STEPS=5 ผ่านฉลุย (3,778 visual token ต่อภาพ, seq 21,445 < 32,768, ราวๆ 55 วินาทีต่อ step, loss 1.86) แล้วรอบเรนจริงก็ไปตายตอน eval บน GPU ที่เช่ามา

ต้นเหตุจริง: จังหวะ evaluate → convert_to_fp32: accelerate ห่อ forward แล้วอัปผลลัพธ์ทั้งก้อนเป็น fp32 ทำให้ logits ถูกกางออกมาที่ seq 21k × vocab 152k × 4 ไบต์ ประมาณ 13GB หับบน bf16 อีก 6.5GB รวมขอ 24.72GB ตอนเรนไม่เจอเพราะ Unsloth ใช้ fused loss ที่ไม่เคยกาง logits ออกมา ส่วน t01 รอตเพราะภาพ 512px seq สั้น และ t02 รอตเพราะโมเดล 30B ยังเหลือที่ t03 คือรอบแรกเอาโมเดล 35B มาคู่กับภาพความละเอียดเต็ม

เสียหายอะไร: รอบเรนเต็มตายนเวลา GPU ที่จ่ายเงิน หลังผ่านการตรวจก่อนรันมาทุกข้อ ต้องเริ่มใหม่

แก้ยังไง: ตั้ง eval_strategy="no" แต่คง save_strategy="epoch" ไว้ เพื่อให้ยังมี checkpoint สามตัวไว้ย้อนกลับมาวัดผลได้

กฎที่ได้: smoke test ที่ข้ามขั้น eval ไม่ใช่ smoke test พิสูจน์หน่วยความจำตอน eval สูงกว่าตอนเรนได้ 2-3 เท่า เพราะ eval กาง logits ออกมา ให้คำนวณ VRAM จากเส้นทาง eval ไม่ใช่เส้นทางเรน

สูง 2026-08-25

ราคา GPU \$0.67/ชม. ที่พูดถึงกันทั่วไปคืออ่านผิดคอลัมน์ min_bid แบบ interruptible — เจอกับดักเดิมซ้ำเป็นครั้งที่สอง

เกิดอะไรขึ้น: ตัวเลข "\$0.67/ชม. RTX PRO 6000" ที่พูดถึงกันถูกนำไปเช็กกับ API สตของ vast.ai แล้วพบว่าไม่จริง — ข้อเสนอที่ถูกที่สุดจริงๆ คือ \$1.01/ชม. ตัวเลข \$0.67 น่าจะมาจากการอ่านผิดคอลัมน์ `min\_bid` (แบบ interruptible) บันทึกระบุว่านี่คือกับดักเดียวกับที่เคยเจอมาก่อนตอนอ่านผิดราคาสตอเรจกับ GPU เป็น `\$0.056`

ต้นเหตุจริง: รายการในตลาดแสดงราคาหลายคอลัมน์ (on-demand, interruptible min_bid, storage) แล้วตัวเลขที่ถูกที่สุดที่เห็นก็ถูกหยิบไปพูดโดยไม่เช็กความมั่นคงของคอลัมน์ไหน กับดักเดียวกันนี้เคยเกิดมาแล้วครั้งหนึ่งและไม่มีการเขียนกฎกันไว้ไม่ให้เกิดซ้ำ

เสียหายอะไร: วางแผนงบประมาณด้วยราคาที่ต่ำกว่าความจริงถึง 40% การเจอกับดักเดิมซ้ำแปลว่าครั้งแรกไม่ได้สร้างบทเรียนที่คงอยู่จริง

แก้ยังไง: ดึงข้อมูลจาก `vastai search offers` โดยตรง (ไม่ใช่ scrape เว็บ ไม่ใช่ aggregator) และบันทึกราคา on-demand จริง: RTX PRO 6000 96GB \$1.01–1.11/ชม., ×2 \$2.00–2.19/ชม., H200 141GB \$3.29/ชม.

กฎที่ได้: อ่านราคา GPU จาก API สตของผู้ให้บริการเองเสมอ และระบุให้ชัดว่าตัวเลขนั้นมาจากคอลัมน์ไหน (on-demand vs interruptible vs storage) เมื่อเจอกับดักเดิมซ้ำ ให้เขียนลงในไฟล์กฎ ไม่ใช่แค่ในบันทึกประจำวัน

สูง 2026-08-25

เอกสารบอกว่าเฟรมเวิร์กรองรับ แต่ของจริงไม่ตรง — GLM-4.6V มีบั๊กเปิดค้างที่บล็อก LoRA, InternVL3-78B โหลดใน Unsloth ไม่ได้

เกิดอะไรขึ้น: การเช็คว่าเฟรมเวิร์กรองรับจริงหรือไม่ด้วยการดู GitHub issue จริงแทนคำกล่าวอ้างในเอกสาร พบว่า GLM-4.6V มีบั๊กที่บล็อก LoRA เปิดค้างและยังไม่แก้ (hiyouga/LLaMA-Factory#9635, `NotImplementedError: get\_input\_embeddings`), InternVL3-78B โหลดใน Unsloth ไม่ได้ (มี issue เปิดค้าง 2 อัน #3401, #2341) และ Pixtral-Large / Molmo-72B / NVLM-D-72B ไม่มีเฟรมเวิร์กหลักไหนยืนยันได้เลยว่าใช้งานได้จริง

ต้นเหตุจริง: เอกสารของเฟรมเวิร์กระบุว่าโมเดลรองรับตามความตั้งใจหรือรองรับบางส่วนเท่านั้น ไม่ได้ยืนยันจากการรัน fine-tune จริงที่สำเร็จ ถ้าเชื่อเอกสารก็จะได้รายชื่อโมเดลผู้ทำขิงที่ผิด

เสียหายอะไร: ถ้าไม่ตรวจสอบก่อน จะเสียเงินเช่า GPU ไปกับโมเดลที่ทำ LoRA fine-tune ไม่ได้เลย และโมเดลที่รอดมาถึงรอบสุดท้าย 2 ตัว ก็ยังต้องเปลี่ยนเฟรมเวิร์ก (จาก Unsloth เป็น LLaMA-Factory) หรือเปลี่ยนโหมด training (จาก bf16 LoRA เป็น 4-bit QLoRA) ซึ่งเป็นการเปลี่ยน pipeline จริง ไม่ใช่แค่สลับตัวโมเดลเฉยๆ

แก้ยังไง: ยืนยันการรองรับจาก GitHub issue ที่เปิดค้างอยู่ และยืนยันการรันจริงจากบุคคลที่สาม (เช่น Llama-4-Scout ที่มีบล็อกของ Fixstars รายงานว่ารันบน 8×H100 จริง) ก่อนจะเข้ารอบชิงชนะเลิศ

กฎที่ได้: อย่าเชื่อตารางความรองรับโมเดลของเฟรมเวิร์กเด็ดขาด ให้ค้นหา issue tracker ของมันเทียบกับโมเดลและโหมด training ที่จะใช้จริง และต้องมีหลักฐานว่ามีคนรันจริงจนสำเร็จก่อนจะควักเงินเช่า

สูง 2026-08-29

path --cv-dir ผิด ทำให้แขนทดลอง 2.4a เสื่อมกลายเป็นแขนควบคุมแบบเจียบๆ ทำให้การทดลองใช้ไม่ได้และเสีย GPU ฟรี

เกิดอะไรขึ้น: apply_arm() หาไฟล์ hint sidecar ถ้าหาไม่เจอจะคืนค่า prompt เดิมพร้อม hint_used: false แทนที่จะฟ้อง error การพิมพ์ --cv-dir ผิดจึงทำให้ได้รับทั้งชุดที่แขนทดลองเหมือนแขนควบคุมเป๊ะแบบเจียบๆ

ต้นเหตุจริง: เส้นทาง 'หา hint ไม่เจอ' ถูกออกแบบให้ไม่ทำให้พัง (ซึ่งถูกต้องสำหรับบ้านที่ไม่มีข้อมูล CV จริงๆ) แต่ไม่มีการแยกแยะระหว่าง 'ไม่มีจริงๆ' กับ 'path ผิด' ปัญหานี้โผล่มาแค่เป็นแฟล็กที่ฝังอยู่ในผลลัพธ์เท่านั้น

เสียหายอะไร: การรันทดลองทั้งชุดอาจให้ผลลัพธ์ที่ตอบคำถามที่ตั้งใจทดลองไม่ได้เลย ทั้งที่เสียค่าเช่า GPU ไปฟรีๆ

แก้ยังไม: บันทึกไว้เป็นการตรวจสอบก่อนรันที่บังคับทำ คือต้องยืนยันว่า hint_used เป็น true สำหรับหน้าที่มี sidecar ก่อนรันต่อ

กฎที่ได้: ก่อนรันจริงที่ต้องจ่ายเงิน ต้องยืนยันว่า treatment ของการทดลองถูกใช้จริง — เช็คแฟล็กจากตัวอย่างสักแถวก่อนการเสื่อมกลายเป็นควบคุมแบบเจียบๆ แยกว่าโปรแกรมพังไปเลย เพราะจ่ายเต็มราคาแต่ได้ผลลัพธ์ที่ไร้ความหมาย

สูง 2026-08-30

คำสั่ง `hf upload <repo> <folder> --exclude 'pattern'` อัปโหลดไฟล์ได้ 0 ไฟล์แบบเจียบๆ ทั้งที่ log ขึ้นเหมือนสำเร็จ

เกิดอะไรขึ้น: ตอนอัปโหลด adapter ของ fold2 ด้วย `--exclude 'checkpoint-\*/\*'` log ขึ้นว่า 'Start hashing 19 files / Processing Files (0/0) / New Data Upload 0.00B/0.00B' ทั้งที่ไฟล์มีอยู่จริง ไม่ใช่ปัญหา token หรือสิทธิ์การเข้าถึง (เช็ค `whoami` แล้วปกติ และอัปโหลดไฟล์เดี่ยวทดสอบก็สำเร็จ)

ต้นเหตุจริง: ยังหาสาเหตุแน่ชัดไม่ได้ คาดว่าเป็นปัญหาการทำงานร่วมกันระหว่าง `--exclude` กับ logic การ sync แบบ incremental ของโฟลเดอร์ใหญ่ใน HF CLI เครื่องมือของ vendor รายงานว่าสำเร็จทั้งที่ไม่ได้อะไรเลย

เสียหายอะไร: การสำรองข้อมูลที่ดูเหมือนเสร็จแล้วจริงๆ ไม่ได้เกิดขึ้น ถ้าไม่จับได้ อาจเข้าใจผิดว่า adapter ปลอดภัยอยู่บน HF แล้วไปทำลาย GPU ทั้ง ทำให้เสีย weights ไปเลย

แก้ยังไม: วิธีแก้ที่ได้ผล 100%: คัดลอกเฉพาะไฟล์ระดับบนสุด (ไม่รวม `checkpoint-\*/\*`) ไปไว้โฟลเดอร์ staging แยกต่างหาก แล้วอัปโหลดโฟลเดอร์นั้นโดยไม่ใช้ `--exclude` เลย อัปโหลดสำเร็จ 842MB บันทึกไว้กันพลาดซ้ำตอน fold3/fold4

กฎที่ได้: อย่าเชื่อ exit status ของเครื่องมืออัปโหลดเด็ดขาด — ต้องตรวจสอบจำนวนไบต์/รายการไฟล์ที่ปลายทางจริง (และเช็ค sha256 หลังดึงกลับมา) ก่อนจะลบหรือทำลายต้นฉบับทุกครั้ง

สูง 2026-08-30

อัปโหลด tarball รูปภาพได้แค่ครึ่งเดียว (514MB จาก 1005MB) แล้ว tar แตกไฟล์ล้มเหลวแบบเจียบๆ จนข้อมูล training เหลือรูป 0 ไฟล์

เกิดอะไรขึ้น: บนเครื่อง fold1 tarball รูปภาพโอนมาได้แค่ 514MB จาก 1005MB จากนั้น `tar` แตกไฟล์ล้มเหลวแบบเจียบๆ เหลือไฟล์รูป 0 ไฟล์ ทั้งที่ log ของขั้นตอนก่อนหน้าดูปกติดี

ต้นเหตุจริง: ไม่มีการตรวจสอบความสมบูรณ์ของไฟล์ระหว่างขั้นตอนอัปโหลดกับแตกไฟล์ — pipeline ถือว่าการโอนที่ไม่มี error คือโอนครบแล้ว และการที่ tar ล้มเหลวบน archive ที่ขาดหายก็ไม่ถูกแสดงให้เห็น

เสียหายอะไร: ถ้าไม่จับได้ทัน fold นี้จะเทรนด้วยดาต้าเซตที่ไม่มีรูปภาพเลย ต้องอัปโหลดใหม่และเริ่มตั้งค่า fold นั้นใหม่หมด

แก้ยังไม: อัปโหลดใหม่ และเพิ่มขั้นตอนบังคับให้เทียบขนาดไฟล์ด้วย `stat` ระหว่างเครื่อง local กับ remote ก่อนแตกไฟล์ทุกครั้ง — วิธีนี้ถูกใช้กับ fold2/fold3 ตั้งแต่ต้นและผ่านตั้งแต่ครั้งแรก

กฎที่ได้: เทียบขนาดไฟล์ (หรือ checksum) ระหว่าง local กับ remote ก่อนแตกไฟล์หรือใช้งาน archive ที่โอนมาทุกครั้ง — คำสั่งโอนที่ exit ด้วยรหัส 0 ไม่ใช่หลักฐานว่าข้อมูลมาครบ

สูง 2026-08-30

ขยายเป็น 4 GPU พร้อมกันด้วยเครดิตเหลือแค่ \$49.75 ทั้งทำงานต้องใช้ \$63 — เครื่องจะถูกหยุดกลางคันแน่ๆ

เกิดอะไรขึ้น: หลังจากขยายจาก 2 เป็น 4 fold และเข้าการ์ดเพิ่มอีก 2 ตัว ต้นทุนโดยประมาณรวมอยู่ที่ ~\$63 แต่บัญชีมีเครดิตแค่ \$49.75 เครื่องทั้ง 4 ตัวเผาเครดิต \$4.29/ชม. เหลือเวลาก่อนเครดิตหมดประมาณ 10 ชั่วโมง

ต้นเหตุจริง: ตัดสินใจขยายจำนวนเครื่องเข้าก่อนที่จะเช็คเครดิตพอสำหรับเวลาที่ต้องรันจริงหรือไม่ — คำนวณต้นทุนหลังจากตัดสินใจไปแล้ว แทนที่จะใช้เป็นเงื่อนไขก่อนตัดสินใจ

เสียหายอะไร: เสียทั้ง 4 การเรนจะถูกฆ่ากลางคันเสียเวลา GPU ไปสูงสุด ~14 ชั่วโมงรวม 4 การ์ด รอดมาได้เพราะมะขามเดิมเครดิตห้น (\$76.63)

แก้ยังไ: รายงานให้มะขามทราบทันทีพร้อมบอก deadline เครดิตถูกเติมด้วยมือ และเปิด auto top-up เป็นชั้นป้องกันที่สอง

กฎที่ได้: ก่อนเริ่มรัน GPU ระยะยาวแบบไม่มีคนเฝ้า ให้คำนวณ ชั่วโมงรวม × อัตราเผาเครดิตต่อชั่วโมง แล้วห้ามเริ่มถ้าเครดิตไม่พอครอบคลุมบวก buffer — ให้การเช็คยอดเครดิตเป็นงานบังคับ ไม่ใช่สิ่งที่ทำให้หลัง

สูง 2026-08-31

fold1 ยัง OOM ที่ step 68/252 แม้จะลด MAX_PIXELS ลงจนอีก 3 fold รอดแล้ว

เกิดอะไรขึ้น: หลังลด MAX_PIXELS ลงเท่ากันทุก fold เหลือ 6912 อีก 3 fold รันจบ แต่ fold1 ตายที่ step 68 จาก 252 เหลือแค่ checkpoint-50 บน HuggingFace

ต้นเหตุจริง: องค์ประกอบข้อมูลของแต่ละ fold ไม่เหมือนกัน เพดานหน่วยความจำที่ตั้งแบบเดียวกันทุก fold จึงไม่ปลอดภัยเท่ากันทุก fold — margin ที่เหลือยังบางเกินไปสำหรับ sequence ที่ยาวที่สุดของ fold1

เสียหายอะไร: เหลือใช้งานได้แค่ 3 ใน 4 fold ต้องเอา model soup มา merge ที่ k=3 แทนที่จะเป็น k=4

แก้ยังไ: ตัดสินใจไม่รันซ้ำ (เพราะตั้งค่าเดิมมีแนวโน้ม OOM อีก และถ้าลดเฉพาะ fold1 จะทำให้ parity ระหว่าง fold เสียไป) จึง merge ที่ k=3 ด้วยน้ำหนักเท่ากัน 1/3 โดยยังครอบคลุมบ้านทั้ง 40 หลังเหมือนเดิม

กฎที่ได้: ตั้งเพดานหน่วยความจำโดยอิงจาก fold ที่หนักที่สุด ไม่ใช่ค่าเฉลี่ย และถ้าจำเป็นต้องปรับ fold ใด fold หนึ่งให้ต่างยอมเสีย fold นั้นไปดีกว่าทำให้ parity ข้าม fold เสียหาย

สูง 2026-08-31

รัน inference หลายชั่วโมงบน gpu1 ที่ช้ากว่าที่ควรถึง ~20 เท่า เพราะ torch build ไม่ตรงกับ CUDA ของ image

เกิดอะไรขึ้น: smoke_destrier กับคิวงาน pure-power ทั้งหมดรันบน gpu1 ที่ความเร็วราว 20 นาทีต่อหน้า จนต้องตั้งกฎ timeout 25 นาที ทำระบบ retry แบบ xgrammar ขึ้นมา และต้องตัดชุดทดสอบจาก 65 หน้าเหลือ 8 หน้า พอมาเช่า GPU-B ที่หลังถึงรู้ว่าเครื่องที่ torch 2.11+cu128 ตรงกับ CUDA 12.8 รันเร็วกว่าราว 20 เท่า

ต้นเหตุจริง: gpu1 มี torch cu13.0 อยู่บน image ที่เป็น CUDA 12.8 (บั๊กแบบเดียวกับที่เคยบล็อก flash-linear-attention) ทำให้ทุกอย่างถอยไปใช้เส้นทาง torch แบบช้า และไม่มีใครเทียบความเร็วกับเครื่องที่รู้ว่าใช้ได้ดีกว่าจะยัดคิวงานยาวๆ ลงไป

เสียหายอะไร: เสียเวลาไปเกือบทั้งวันกับ inference ที่ช้ากว่าที่ควร 20 เท่า ต้องเขียนระบบ timeout/retry ทั้งชุดมาแก้ปัญหา และการทดสอบ pure-power ถูกลดจาก 65 หน้าเหลือแค่ 8 หน้า

แก้ยังไ: บันทึกค่าคอนฟิกที่ใช้งานได้ของ GPU-B (torch 2.11+cu128 บน CUDA 12.8) ไว้ที่ results/gpuB_working_config/ เป็นสูตรสำหรับการเช่าครั้งต่อไป

กฎที่ได้: ก่อนจะยัดงานหลายชั่วโมงลงในเครื่องเช่าใหม่ ต้องเบนซ์มาร์คสัก 1 หน้าเทียบกับเครื่องที่รู้ว่าใช้ได้ก่อนเสมอ — ความช้าลง 20 เท่า มักถูกอ่านผิดว่า 'โมเดลมันช้า' ทั้งที่จริงคือ 'เครื่องนี้ตั้งค่าผิด'

สูง 2026-09-01

เจอบั๊กจริง 8 ตัวในแอป Constistant ที่ซ่อนอยู่ จนกระทั่งจำลองรันทั้งบ้านโดยไม่ใช้ GPU

เกิดอะไรขึ้น: วันที่ 1 ก.ย. ทีมจำลองการรันทั้งบ้านผ่านแอป Constistant โดยไม่เช่า GPU เลย และเจอบั๊กจริง 8 ตัวทันที ซึ่งแก้ครบทั้งหมดในวันนั้น

ต้นเหตุจริง: ผังแอปพลิเคชันไม่เคยถูกทดสอบครบวงจรกับทั้งบ้านมาก่อน — การทดสอบที่ผ่านมาเป็นแบบทีละโมดูล ทำให้บั๊กระดับ integration สะสมอยู่แบบไม่มีใครเห็น จนกระทั่งรันจำลองทั้ง pipeline ครั้งแรก

เสียหายอะไร: ถ้าไม่จับได้ บั๊กแฝงทั้ง 8 ตัวนี้จะโผล่มาตอนรัน GPU จริงที่มีค่าใช้จ่ายแทน เสียเวลาเช่าเครื่องไปกับการดีบั๊กโค้ดแอปแทนที่จะเป็นงานโมเดล

แก้ยังไง: แก้ครบทั้ง 8 ตัวในวันเดียวกัน รายละเอียดบันทึกไว้ในไดอารี่วันนั้น

กฎที่ได้: ให้รัน pipeline ทั้งหมดในโหมดจำลองราคาถูก (ไม่ต้องใช้ GPU ไม่ต้องเช่าเครื่อง) ก่อนรันจริงทุกครั้งที่มีค่าใช้จ่ายสูง บั๊กระดับ integration มองไม่เห็นด้วยเทสต์ทีละโมดูล และหาบ่นแลปที่อุปถูกกว่าหาบ่นการดีบั๊กที่เข้ามา

สูง 2026-09-01

งานค้าง pending นานกว่า 7 นาทีโดย GPU ไม่ทำงานเลย เพราะ SSH tunnel ไม่เคยเปิดจริง

เกิดอะไรขึ้น: งานที่ส่งมาจากเว็บค้างสถานะ `pending` นานกว่า 7 นาที ไม่มีการทำงานของ GPU เลย และไม่มี error message ใดๆ ตรวจสอบพบว่า `presentation.py up` ไม่เคยเปิด SSH tunnel จริง ทำให้ `localhost:8000` เข้าไม่ถึง และ `worker.py` ไม่มีที่ส่ง request ไป

ต้นเหตุจริง: คำสั่ง `presentation.py up` รายงานว่าสำเร็จ ทั้งที่ไม่ได้ตรวจสอบว่า tunnel เปิดใช้งานได้จริง เป็นการเริ่มระบบแบบเสียครึ่งอย่างเสียบๆ — คำสั่ง orchestration คินค่า OK ทั้งที่ผลลัพธ์สำคัญที่สุด (tunnel) ยังไม่เกิดขึ้นจริง และไม่มีขั้นตอนถัดไป เช็คว่าเข้าถึงได้ก่อนรับงาน

เสียหายอะไร: เสียเวลาเช่า GPU ที่จ่ายเงินไปกว่า 7 นาทีกับงานที่ไม่ได้ทำอะไรเลย บวกเวลาดีบั๊กระหว่างเซสชันเดโมสด ต้องรีเซ็ตงานกลับเป็น `pending` ด้วยมือ และรีสตาร์ท `worker.py` ใหม่

แก้ยังไง: เปิด tunnel ด้วยมือ, รีเซ็ตงานที่ค้างกลับเป็น `pending`, รีสตาร์ท `worker.py` แล้วยืนยันว่า GPU ทำงานจริง ภายหลังเสริมความแข็งแรงด้วย `ServerAliveCountMax=6` และ `ExitOnForwardFailure=yes` บน tunnel

กฎที่ได้: คำสั่งเริ่มระบบต้องตรวจสอบผลลัพธ์สำคัญของตัวเองก่อนรายงานว่าสำเร็จ — `presentation.py up` ควรลองเช็ค `localhost:8000` และรายงาน error ทันทีถ้า tunnel ไม่เปิด อย่าปล่อยให้งานเข้าสถานะ `pending` ทั้งที่ยังไม่พิสูจน์ว่าเส้นทางส่งข้อมูลใช้งานได้

สูง 2026-09-01

ส่ง pass3 ออกไปพร้อมแค่ self-check สังเคราะห์ที่รู้คำตอบล่วงหน้าอยู่แล้ว

เกิดอะไรขึ้น: pass3 (`pass3_measure.py`) เขียน self-check โดยใช้กริดสมมติ 50 px/m ที่รู้คำตอบที่ถูกต้องล่วงหน้าอยู่แล้ว ไม่เคยรันกับแบบจริงเลยสักหน้า ค่า px/m และค่าความคลาดเคลื่อน (residual) จริงจึงไม่เคยถูกเห็นเลย ตัวรายการเองก็ยังติดธงเรื่องนี้ไว้เป็นสิ่งที่ค้างอยู่ลำดับต้นๆ

ต้นเหตุจริง: self-check ตรวจสอบเลขคณิตเทียบกับข้อมูลที่ทดสอบสร้างขึ้นเอง จึงพิสูจน์ได้แค่ว่า 'ฟังก์ชัน fit เข้ากับข้อมูลที่ fit สมบูรณ์แบบ' แต่ตรวจไม่ได้เลยว่าแบบจริงจะให้จุดอ้างอิง (anchor) ที่ใช้ได้พอหรือเปล่า — ซึ่งกลายเป็นจุดพังจริงในภายหลัง (ดูบั๊ก cv_mark และ count > 1)

เสียหายอะไร: เชื่อว่าขั้นตอนการวัดทั้งขั้นใช้งานได้ ทั้งที่จุดพังหลักที่เจอในโลกจริง (0 anchor, สเกลผิด, element ที่มีหลาย reference) ยังไม่เคยถูกทดสอบเลย และภายหลังพบบั๊กจริงในอีก 3 ตัวแยกกัน

แก้ยังไง: ปิดจบในวันเดียวกันด้วย `validate_pass3_real.py` ที่ใช้ข้อมูลจริงที่คนตรวจสอบแล้ว และวิธี leave-one-out: ถูก 191/191, fit สำเร็จ 15/18 หน้า ค่า error 0.004–0.093 เมตร

กฎที่ได้: self-check ที่ทำจากข้อมูลสังเคราะห์เองพิสูจน์ได้แค่ว่าโค้ดรันได้ ไม่ได้พิสูจน์ว่าโค้ดใช้งานได้จริง ก่อนบอกว่าขั้นตอนการวัดเสร็จแล้ว ต้องรันกับข้อมูลจริงที่มีอยู่บนเครื่องอยู่แล้ว — ในกรณีนี้ข้อมูลจริงมีอยู่ในเครื่องแล้วและไม่ต้องใช้ GPU เลยด้วยซ้ำ

สูง 2026-09-01

worker.py ไม่มีการ retry เลยสักจุด ทำให้เน็ตสะดุดชั่วคราวก็ทำให้งานตายกลางทาง

เกิดอะไรขึ้น: `update_job`, `download_image` และ `call_purson` ไม่มี retry logic เลย เน็ตสะดุดแค่ชั่วคราวระหว่างงาน GPU ที่ใช้เวลา 75–80 นาที ก็ทำให้ operation พังไปเลย

ต้นเหตุจริง: โค้ดเขียนขึ้นโดยสมมติว่าเน็ตในเครื่องเสถียร แท้จริงมันรันข้าม GPU ที่เข้ามาจากระยะไกลผ่าน SSH tunnel ซึ่งเป็นเส้นทางที่หลุดง่ายโดยธรรมชาติ — ตามที่เห็นตอน HuggingFace ค้างในวันถัดมา

เสียหายอะไร: เน็ตสะดุดแค่ครั้งเดียวก็เสียเวลา GPU ที่เหลือไปเปล่าๆ กับงานที่รายงานผลหรือดึงข้อมูลต่อไม่ได้แล้ว

แก้ยังไง: เพิ่ม `_retry()` พร้อม backoff 5/15/30/60 วินาที ครอบ `update_job`, `download_image` และ `call_purson`

กฎที่ได้: ทุกการเรียกเครือข่ายที่ข้ามไปยังเครื่องเช่าต้องมี backoff retry ตั้งแต่เวอร์ชันแรก ต้นทุนของ call ที่หลุดไปวัดกันเป็นนาทีของ GPU ไม่ใช่ मिलินาที

สูง 2026-09-01

ถ้า retry แบบไม่คิดหน้าคิดหลังตอน ReadTimeout จะทำให้ GPU รันสองงานซ้อนกัน

เกิดอะไรขึ้น: ตอนเพิ่ม retry ให้ `call_purson` เกือบสร้างปัญหาใหม่: ถ้า retry ตอนเจอ `ReadTimeout` โมเดลอาจจะยังทำงานค้างอยู่ แล้วส่ง request ซ้ำอันเดิมไปอีกรอบ กลายเป็น GPU ต้องรันสองงานพร้อมกัน

ต้นเหตุจริง: 'พังแล้ว retry' ใช้ไม่ได้กับงานที่กินเวลานานและทำซ้ำไม่ได้ (non-idempotent) — read timeout บอกแค่ว่าฝั่ง client เลิกรอ ไม่ได้บอกว่าฝั่ง server ยังทำงานอยู่หรือเปล่า

เสียหายอะไร: ถ้าไม่จับได้ก่อน จะทำให้ throughput ของ GPU หายไปครึ่งหนึ่งและต้นทุนต่อหน้าเพิ่มเป็นสองเท่า แต่จับได้ตั้งแต่ตอนออกแบบ retry เลย ไม่ได้เกิดจริง

แก้ยังไง: ตั้งใจไม่ retry ตอนเจอ ReadTimeout เลย — retry เฉพาะตอน connection ล้มเหลวหรือ error 5xx เท่านั้น

กฎที่ได้: ก่อน retry ต้องแยกประเภท error ก่อนเสมอ: connection-refused กับ 5xx retry ได้ปลอดภัย แต่ read-timeout ตอนโมเดลกำลัง generate อยู่ retry ไม่ได้ อย่า retry แบบเหมารวมกับงานที่แพงและทำซ้ำไม่ได้

สูง 2026-09-01

tunnel ไม่มีระบบเชื่อมต่อใหม่อัตโนมัติ — ถ้า tunnel ตายสนิทต้องให้คนมาพิมพ์คำสั่งเองใหม่

เกิดอะไรขึ้น: ถ้า tunnel ตายสนิท ไม่มีอะไรรีสตาร์ทมันเลย ต้องให้คนมารัน `python presentation.py tunnel` เอง worker จะ retry แล้วรอไปเรื่อยๆ แต่งานจะค้างจนกว่าจะมีคนสังเกตเห็น

ต้นเหตุจริง: เพิ่ม retry ให้ฝั่ง client call แล้ว แต่ไม่ได้เพิ่มให้ตัว transport เอง ทำให้ชั้นการกู้คืนหยุดอยู่ก่อนถึงจุดที่ฟังจริงหนึ่งชั้น

เสียหายอะไร: ถ้ารันแบบไม่มีคนเฝ้าข้ามคืน อาจค้างอยู่ที่ tunnel ที่ตายไปเรื่อยๆ ไม่จบ ในขณะที่ GPU ยังคิดเงินตลอดเวลา

แก้ยังไง: ยังไม่ได้แก้ — บันทึกไว้เป็นเรื่องค้าง

กฎที่ได้: ระบบกู้คืนต้องครอบคลุมถึงชั้น transport ด้วย ไม่ใช่แค่ตัว call เเฉๆ ถ้างานต้องรันแบบไม่มีคนเฝ้า ทุกชั้นที่มันพึ่งพาต้องกู้คืนตัวเองได้ ไม่งั้น GPU จะคิดเงินไปเรื่อยๆ ทั้งที่ไม่ทำอะไรเกิดขึ้น

สูง 2026-09-01

ไปป์ไลน์ pass0→pass3 เติมรูปแบบไม่เคยรันจริงบน GPU จริงเลยสักครั้งเดียว

เกิดอะไรขึ้น: หลังจากทำงานจำลองมาเต็มวันจนพิสูจน์ได้ทุกขั้นตอน บันทึกบอกตรงๆ ว่าสายพานทั้งหมดไม่เคยรันกับ GPU จริงเลย — สิ่งที่พิสูจน์แล้วทั้งหมดคือ 'ทุกขั้นตอนยกเว้นตัวโมเดลเอง'

ต้นเหตุจริง: แรงกดดันเรื่องต้นทุน GPU ทำให้การตรวจสอบทั้งหมดถูกผลักไปทำแบบจำลองบน CPU ซึ่งมีประโยชน์ก็จริง แต่โดยธรรมชาติมันจับปัญหาที่เกิดจากการโต้ตอบกับโมเดลจริงไม่ได้เลย (timeout, ข้อความถูกตัด, JSON เพี้ยน) — ซึ่งเป็นสิ่งที่ฟังหั่นที่ในวันถัดไปพอดี

เสียหายอะไร: การรัน GPU จริงครั้งแรกในวันถัดมา เจอ 3 หน้าพังจาก timeout ฝั่ง server ทันที — เป็นความล้มเหลวประเภทที่ไม่มีการจำลองไหนจับได้เลย

แก้ยังไง: วันนั้นยังไม่ได้แก้ — รันจริงในวันที่ 2026-09-02 แล้วเจอบั๊กใหม่ทันที

กฎที่ได้: การจำลองพิสูจน์ได้แค่ว่าโค้ดเราถูก ไม่เคยพิสูจน์การโต้ตอบกับโมเดลจริงเลย ควรกันบไว้รัน end-to-end บน GPU จริงราคาถูกอย่างน้อยหนึ่งครั้งทุกครั้งที่แก้ pipeline ใหญ่ๆ — เพราะความล้มเหลวที่เกิดเฉพาะจุดต่อกับโมเดล (timeout, ข้อความถูกตัด, output เพี้ยน) ไม่มีทางเห็นได้จากการจำลองใดๆ เลย

ไม่มี checkpoint ทำให้แค่พังจุดเดียวก็ต้องเริ่มงาน GPU ที่กินเวลา 80 นาทีใหม่ตั้งแต่ต้น

เกิดอะไรขึ้น: ก่อนแก้ งานที่พังกลางทางต้องเริ่มรันใหม่ตั้งแต่ต้น — เรียกโมเดลซ้ำทุกครั้งที่เคยทำเสร็จไปแล้ว นี่คือสาเหตุที่ทำให้หน้า 12 พังแล้วต้องรันบ้านทั้งหลังใหม่หมด

ต้นเหตุจริง: ผลลัพธ์ถูกเขียนแค่ตอนงานเสร็จสมบูรณ์เท่านั้น ทำให้งานที่ทำไปครึ่งทางถูกทิ้งหมดเมื่อพัง สำหรับงานที่จ่ายเงินและกินเวลา 75–90 นาที นี่ทำให้แค่พังหน้าเดียวก็เสียหายหนักมาก

เสียหายอะไร: ต้องรันบ้านทั้งหลังใหม่บน GPU ที่จ่ายเงินแล้วเพราะแค่หน้าเดียวพัง (เป็นสาเหตุตรงๆ ที่ทำให้ต้องรันงาน 43323920 ใหม่)

แก้ยังไง: `save_checkpoint()` เขียน `result` หลังจบทุกครั้งของ `pass0/gridline/แต่ละหน้าของ pass2/pass2.5` `claim_next_job()` ก็ดึง `result` มาด้วยเพื่อให้ `run_house_extract` ซ้ำ `pass` ที่ checkpoint ไว้แล้ว — เป็น resume จริง ไม่ใช่แค่บันทึกผลอย่างเดียว ฝั่งเว็บจำ `job id` ล่าสุดต่อโปรเจกต์และมีปุ่ม '🔄 ทำต่อ' ที่ PATCH งานเดิมกลับเป็น pending แทนที่จะสร้างงานใหม่

กฎที่ได้: งานที่จ่ายเงินและรันนาน ต้อง checkpoint หลังทำงานหนักแต่ละหน่วยเสร็จ และตอน resume ต้องข้ามหน่วยที่ทำเสร็จแล้วจริงๆ — แค่ 'เราบันทึกผลอยู่' ไม่ใช่การ resume

เช่า GPU กว่าจะสำเร็จต้องลอง 7 ครั้ง — 2 ครั้งค้างตอนโหลด 3 ครั้งติดปัญหา SSH key ยังไม่กระจายทัน

เกิดอะไรขึ้น: กว่าจะได้ GPU เช่าที่ใช้งานได้จริงสำหรับบ้าน 09 ต้องลองถึง 7 ครั้ง — 2 instance ค้างตอนโหลด 3 instance ปฏิเสธ SSH key ด้วยข้อความ 'Permission denied (publickey)' ทั้งที่ key ถูกต้อง (สาเหตุคือ key ยังกระจายไม่ทัน)

ต้นเหตุจริง: vast.ai เปิด instance ให้ใช้ก่อนที่ SSH key จะกระจายเสร็จ ทำให้การเชื่อมต่อครั้งแรกพังบน host ที่ปกติดีทุกอย่าง สคริปต์ของทีมเชื่อมต่อทันทีและมองว่าความล้มเหลวนั้นจบสิ้น จึงวินิจฉัยผิดว่า host เสียทุกครั้งที่เกิด lag แล้วตอบด้วยการทำลายแล้วเช่าใหม่

เสียหายอะไร: เสียการเช่าไปเปล่าๆ 5 ครั้งและเวลาติดตั้งจำนวนมาก แต่รอบทำลาย/เช่าใหม่เสียทั้งเงินและเวลา — เป็นอาการซ้ำจากที่เจอมาแล้วเมื่อ 2026-09-01

แก้ยังไง: หาต้นเหตุจริงได้ว่าเป็น propagation lag แล้วแก้ใน `presentation.py` ด้วยการ retry 6 ครั้งภายใน 60 วินาที ก่อนที่จะ mkdir/scp ครั้งแรก แทนที่จะทำลายแล้วเช่าใหม่ทุกครั้ง

กฎที่ได้: ต้องแยกให้ออกระหว่าง 'host เสีย' กับ 'host ยังไม่พร้อม' ให้ instance ที่เพิ่งเปิดใหม่มีช่วง retry ที่จำกัดเวลา (60 วินาที) ก่อนจะทำลายทิ้ง — การทำลายแล้วเช่าใหม่เป็นการตอบสนองแรกต่อ publickey error จะเผาเงินไปกับ host ที่จริงๆ ดีอยู่แล้ว

สูง 2026-09-02

ดาวนโหลดจาก HuggingFace ค้าง timeout ซ้ำๆ 4 ครั้งตอนดึงโมเดล 70GB ลงเครื่องเช่าที่จ่าย \$2.05/ชม.

เกิดอะไรขึ้น: บน instance ที่เช่าราคา \$2.05/ชม. ที่เริ่มได้สำเร็จ เน็ตของ HuggingFace ค้างหนักมาก — timeout ซ้ำๆ 4 ครั้ง กับไฟล์คนละไฟล์ตอนดาวนโหลดโมเดล 70GB มะขามหยุดกลางทางแทนที่จะจ่ายเงินต่อไปกับเน็ตที่แย่ และขอเลือกเครื่องเอง

ต้นเหตุจริง: น้ำหนักโมเดลถูกดึงจาก HuggingFace ทุกครั้งที่เช่าเครื่องใหม่ ทำให้การติดตั้งทั้งหมดขึ้นอยู่กับความเร็วของ CDN บุคคลที่สามไปยัง datacenter ตัวนั้นๆ ไม่มี cache ในเครื่อง ไม่มี mirror ไม่มี image ที่เตรียมไว้ล่วงหน้า

เสียหายอะไร: เวลาที่จ่ายเงินไปหมดกับการดาวนโหลดที่ล้มเหลวโดยไม่ได้อะไรเลย เครื่องที่เช่าถูกทิ้งไปโดยไม่มีผลงานอะไร

แก้ังัง: ห้างเครื่องที่เช่าไป เหตุการณ์นี้เป็นที่มาของพีเจอร์ 'ต่อกลับเช่าเครื่องที่เช่าไว้เอง' โดยตรง (GO.bat เมนูข้อ 7 / ``presentation.py attach <id>``)

กฎที่ได้: การดาวนโหลด 70GB จากบุคคลที่สามทุกครั้งที่เราเช่าเครื่อง ทำให้ต้นทุนขึ้นอยู่กับความเร็ว CDN ของเขา ควรฝังน้ำหนักโมเดลไว้ใน snapshot/image หรือ mirror ไว้ที่ storage ในภูมิภาคเดียวกัน และควรเลิกเช่าเร็วๆ ทันทีที่เห็นว่าดาวนโหลดแย่ชัดเจน แทนที่จะจ่ายเงินไปเรื่อยๆ กับการ retry

สูง 2026-09-02

งานรายงานว่า 'สำเร็จ 9/9' ทั้งที่สามในเก้าหน้ามี JSON ที่พังใช้งานไม่ได้

เกิดอะไรขึ้น: งาน f63bc71e จบด้วยสถานะ จบสำเร็จ 9/9 แต่สามหน้า (page_03_notes, page_12_plan_footing, page_13_notes) มี JSON พัง/ถูกตัด ไม่มีอะไรเลยระหว่างขั้นตอน generate กับสถานะ 'เสร็จ' ที่เข้าไปตรวจว่าแต่ละหน้า parse ได้จริงหรือไม่

ต้นเหตุจริง: สถานะเสร็จของ worker นับแค่จำนวนหน้าที่ generate ออกมา ไม่ได้นับว่าหน้าไหน parse เป็น JSON ที่ใช้ได้จริง — timeout ฝั่ง server ตัดกลางคำ แต่ worker ก็ยังทำเครื่องหมายว่าหน้านั้นเสร็จอยู่ดี

เสียหายอะไร: สถานะเขียวที่บอก 9/9 ที่ผู้ใช้จะเชื่อสนิทใจ ความพังถูกพบแค่จากการเปิดข้อความดิบมาดูด้วยมือ และสามหน้าจากงาน GPU ที่จ่ายเงินไปประมาณ 87 นาทีกลายเป็นใช้ไม่ได้แบบเสียบๆ

แก้ังัง: ตรวจพบด้วยการดูด้วยมือ แล้วกู้คืนด้วย ``salvage_json.py`` ที่เขียนขึ้นเอง (ตัววิ่งไล่ตาม bracket-depth) — แต่ยังไม่ได้ใส่ด้านตรวจความถูกต้องเข้าไปในตัว worker เอง

กฎที่ได้: pipeline ที่ส่งออก JSON ต้อง `json.loads()` ทุกหน้าก่อนที่จะรายงานว่าสำเร็จ — 'โมเดลคืนข้อมูลมาแล้ว' ไม่เหมือนกับ 'หน้านั้นใช้งานได้จริง' และสถานะเขียวปลอมแยกว่าความล้มเหลวที่ตั้งชัดเจนเสียอีก

เรื่องที่เหลือในหมวดนี้ (63)

ไฟล์ `-all.json` ที่ไว้อ้างอิงอย่างเดียว กับไฟล์ `config.xml` ถูกอัปโหลดเข้าแท็บ Data Import ของ Label Studio

กลาง 2026-07-07

ไฟล์ที่ห้าม import เด็ดขาด ต้องไม่ใช่ชื่อขึ้นต้นเหมือนหรืออยู่โพลเดอร์เดียวกับไฟล์ที่ import ได้ ให้ตั้งชื่อแบบ `*-REFERENCE-DO-NOT-IMPORT` หรือแยกไปอยู่โพลเดอร์ของมันเอง

การรวม 3 โปรเจกต์ Label Studio เป็นอันเดียวด้วย `visibleWhen` จะพังเพราะบั๊กของตัวเครื่องมือที่รู้กันอยู่แล้ว

กลาง 2026-07-09

ค้น issue tracker ของผู้พัฒนาก่อนจะออกแบบระบบทับบนพีเจอร์แสดงผลแบบมีเงื่อนไข และให้เลือก config ตรงไปตรงมา 3 ชุด ดีกว่า config ฉลาดๆ ชุดเดียวที่แสดงผลผิดได้

ข้อความจั่วที่หมุน 90 องศาหน้า BOQ สกัดออกมาได้ค่าความมั่นใจต่ำ แล้วปล่อยผ่านไปโดยคิดว่าต้องมาตรวจ

กลาง 2026-07-12

หน้าไหนมีข้อความหมุนหรือตัวเล็กลง ให้หมุนและซูมเป็นรอบเฉพาะก่อนสกัดข้อมูล อย่ายอมรับการจับคู่ที่ค่าความมั่นใจต่ำแล้วเลื่อนไว้ก่อน เพราะของที่เลื่อนไว้จะกองสะสมและไม่มีใครมาเคลียร์

สคริปต์สร้าง task ของ Label Studio ทิ้งไฟล์ที่รูปทรง JSON ไม่รองรับไปเสียๆ

กลาง 2026-07-12

ตัวแปลงข้อมูลที่ข้ามอะไรไป ต้องบันทึกว่าข้ามอะไรและเพราะอะไร จำนวนที่ไม่ตรงกันแบบเสียๆ คือบั๊กข้อมูลหาย และให้ตัดสินตั้งแต่ครั้งแรกที่เจอว่าจะแก้ที่ schema หรือแก้ที่สคริปต์ อย่าลากช่องว่างนี้ไปที่ละบ้านๆ

Push ขึ้น Git ถูกปฏิเสธด้วย error 403 เพราะบัญชีไม่มีสิทธิ์ collaborator ใน repo ขององค์กร

กลาง 2026-07-20

ต้องเช็คสิทธิ์ push ตั้งแต่วันแรกที่ตั้ง repo ไม่ใช่รอไปเจอตอน push ครั้งแรก ช่องโหว่เรื่องสิทธิ์แบบนี้ทำให้งาน 'commit งาน' ที่ควรใช้เวลาแค่ 5 นาทีกลายเป็นงานที่ติดล๊อคไปเลย

เจอ error 'Jupyter connection refused' ซ้ำๆ ทั้งที่คำสั่งไล่ตามวิธีเชื่อมต่อที่ setup นี้ไม่ได้รองรับตั้งแต่แรก

กลาง 2026-07-20

ต้องตรวจสอบก่อนว่าช่องทางการเข้าถึงนั้นถูกจัดเตรียมไว้จริงก่อนจะมานั่ง debug ปัญหาการเชื่อมต่อ — และควรเลือกใช้ SSH/scp/tmux ที่ใช้ได้ทุกที่ แทนการตั้ง notebook server ที่ต้องมาลุ้นว่าใช้งานได้ไหม

เสียเวลาหลายวันสู้กับ Jupyter web UI ที่ขึ้น 'connection refused' ก่อนจะเปลี่ยนไปใช้ SSH

กลาง 2026-07-21

ก่อนเข้าเครื่อง ต้องอ่านเอกสารของผู้ให้บริการเองและใช้วิธีที่คนทำงานจริงจะใช้กัน (SSH/tmux สำหรับเข้า GPU) อย่าเสียเวลาพิสูจน์ว่า UI สะดวกใช้ไม่ได้

การรัน baseline eval เกิดรันตัวเองซ้ำโดยไม่มีใครรู้ ทำให้เสียเวลา GPU ไปเปล่าๆ

กลาง 2026-07-21

หลังจากงานที่ยาว/แพงรายงานว่าเสร็จแล้ว ต้องเช็ค `ps`/nvidia-smi`` ว่ามันไม่ได้เริ่มรันซ้ำเองก่อนที่จะเดินหนีไปทำอย่างอื่น

ฟอร์มเช่าเครื่องของ Vast.ai ตั้งค่าดิสก์ default ไว้ที่ 150GB ซึ่งเป็นค่าที่ทำให้เกิดปัญหาดิสก์เต็มพอดี

กลาง 2026-07-24

ค่า default ทุกตัวในฟอร์มเช่า/provision เครื่องคือการตัดสินใจที่เรากำลังทำโดยไม่รู้ตัว — ต้องเขียนค่าที่ไม่ใช่ default ที่จำเป็นไว้ใน checklist ก่อนเช่า และตรวจสอบหน้ายืนยันก่อนกดเช่าจริง

แถบเลื่อนดิสก์ของ Vast.ai default ไว้ที่ 150GB เสียๆ ต้องลากมือไปที่ 300 เอง

กลาง 2026-07-24

ต้องบันทึกค่า default ของ UI ผู้ให้บริการที่ต้องแก้ไขทุกครั้งไว้ใน checklist ก่อนเช่า ค่า default ที่ทำให้เครื่องเล็กเกินไปโดยไม่รู้ตัวจะทำให้ต้องเช่าใหม่ ไม่ใช่แค่ resize

การตรวจ export sanity check เพิ่มค่า token limit ไปแล้วถึง 3 รอบ ก่อนจะรู้ว่าตัวเช็คนั้นเองผิดตั้งแต่ต้น

กลาง 2026-07-24

พอเพิ่ม limit แล้วยังพังเป็นครั้งที่ 2 ให้หยุดเพิ่มแล้วกลับไปตั้งคำถามกับข้อสมมติฐานของตัวเช็คแทน ตัวเช็คที่ดีควรพิสูจน์แค่สิ่งที่จำเป็นน้อยที่สุด ไม่ใช่จำลอง output แบบเต็ม production

ขั้นตอน export รายงานว่า 'file not found' ทั้งที่ conversion และ quantization สำเร็จจริงๆ

กลาง 2026-07-24

ห้ามเดา path ของ output จาก library เอง ต้อง list directory หรือใช้ path ที่มันคืนค่ามาจริง และเมื่อขั้นตอนที่ใช้ทรัพยากรมากรายงานว่าล้มเหลว ให้เช็ก่อนว่า artifact นั้นมีอยู่จริงหรือเปล่าก่อนจะรันซ้ำ

การดรอที่เลือกไว้ตอน pre-flight เฟส 0.5 ไม่ใช่ตัวที่เช่าจริง แคมเครื่องแรกที่เช่าจริงก็ถูกเปลี่ยนไปอีก

กลาง 2026-07-24

pre-flight ควรกำหนดแค่ 'เกณฑ์' ของเครื่อง (VRAM, ดิสก์, ความเสถียร, ราคาต่อชั่วโมงสูงสุด) ไม่ใช่ล๊อค listing id เจะจบการเลือก id ล่วงหน้าหลายชั่วโมงรับประกันได้เลยว่ามันจะเกินไปแล้วและบังราคาที่ดีกว่าไว้

ความผิดพลาดเรื่อง RAM หมดแบบเดียวกับที่เกิดในเฟส 10 เกิดซ้ำอีกครั้งในวันถัดมาที่ ctx-size สูงขึ้น

กลาง 2026-07-25

เมื่อจะเพิ่มพารามิเตอร์ด้านทรัพยากรเพื่อแก้ปัญหาอื่น ต้องเช็คข้อจำกัดที่พารามิเตอร์นั้นเคยถูกปรับไว้แต่แรกอีกครั้ง จัดการปัญหา truncation ด้วยการรันเฉพาะหน้าที่ได้รับผลกระทบใหม่ที่ ctx สูงขึ้น ไม่ใช่ปรับค่าทั้งระบบ

บ้าน 07 แผ่น SN-04: นับบอพักได้ 8 บอ ทั้งที่จริงมี 11

กลาง 2026-07-28

การนับทุกครั้งต้องมาจากการใส่ขุมดูให้ครบทั้งแผ่น แล้วจดจำนวนแยกเป็นโซนๆ ไว้ ไม่ใช่กวาดตาดูทั้งแผ่นรอบเดียวแล้วนับ

รันโมเดลขนาด 21GB บนเครื่องตัวเอง จนเครื่องใช้ทำอะไรอย่างอื่นไม่ได้ ต้องหยุดตอนทำไปได้แค่ 2 จาก 61 หน้า

กลาง 2026-07-28

ประเมินเวลาและทรัพยากรที่จะใช้ก่อนเริ่มงาน GPU ยาวๆ และห้ามรันบนเครื่องที่มีคนต้องใช้ทำงานอยู่ อีกอย่างคือ งาน batch ยาวๆ ต้องทำให้รันต่อจากจุดที่ค้างไว้ได้ตั้งแต่แรก

บักที่ 5: สคริปต์เทรนค้างตอนช่วงท้าย ทั้งที่ยังจอง VRAM อยู่

กลาง 2026-07-28

สคริปต์ที่รันบนเครื่องที่คิดเงินตามเวลา ต้องมีสวิตช์ปิดขั้นตอนเสริมทุกอย่างได้ และต้องคืนทรัพยากรก่อนถึงจุดที่มันมีโอกาสค้าง

สั่งงาน GPU ช้อนลงบนการ์ดที่มีงานรันอยู่แล้ว — วันเดียวกับที่เจตเป็นบทเรียนการช้อ 11

กลาง 2026-07-28

สองเรื่อง หนึ่งคือห้ามสั่งงาน GPU ตัวที่สองลงบนการ์ดที่มีงานจริงรันอยู่ สองคือเวลาพลาดจนต้องตั้งเป็นกฎการ ให้จดเหตุการณ์ที่เกิดขึ้นด้วย ไม่ใช่จดแค่ตัวกฎ

GPU ชิ้น 100% แต่กินไฟแค่ 33.7W — การ์ดกำลังรออยู่ ไม่ได้ทำงาน

กลาง 2026-07-29

ห้ามสรุปว่าคอคขวดอยู่ตรงไหนจากเปอร์เซ็นต์การใช้งานอย่างเดียว ให้ดูกำลังไฟกับหน่วยความจำประกอบด้วย ถ้า 100% แต่วัตต์ต่ำ แปลว่าการ์ดดอตอยาก และจุดที่ต้องแก้จุดตันทางของมัน

token ของ HuggingFace ชื่อ constistant-tune_t02 ยังใช้งานได้อยู่ ทั้งที่งานที่ออกให้จบไปแล้ว

กลาง 2026-07-29

ออก credential แบบผูกกับงาน โดยตั้งวันหมดอายุให้เลยเวลางานไปสักไม่กี่ชั่วโมง อย่าออกแบบไม่มีวันหมดอายุแล้วตั้งใจว่าจะไปเพิกถอนทีหลัง ถ้าแพลตฟอร์มตั้งวันหมดอายุไม่ได้ ให้เอาการเพิกถอนไปใส่ในสคริปต์เดียวกับที่คืนเครื่อง

เครื่องเช่า Vast.ai เครื่องที่สอง SSH ต่อไม่ติดเลย

กลาง 2026-08-20

เพื่อให้รู้ว่าแต่ละ session จะเจอเครื่องเช่าเสียหนึ่งถึงสองเครื่อง และให้เช็ค SSH กับ nvidia-smi ภายในสองนาทีแรกของการเช่าทุกครั้ง ถ้าประกาศเดิมให้เครื่องเสียสองครั้งติด ให้เปลี่ยนประกาศ ไม่ใช่กดเช่าซ้ำที่เดิม

t02 ไม่มี GGUF ที่ใช้ได้เพราะ export ล้มเหลว ทำให้ต้องรัน inference ด้วย transformers/PEFT ดิบๆ บน GPU เช่า

กลาง 2026-08-20

การ export ที่ล้มเหลวไม่ใช่เรื่องเล็ก มันเปลี่ยนต้นทุนของการทดลองทุกครั้งต่อนั้น ให้แก้ export ให้ได้ ไม่งั้นก็ต้องเขียนในแผนตรงๆ ว่าการทดสอบทุกครั้งต่อนี้คือค่าเช่า GPU ที่ต้องจ่ายจริง

รอบก่อนหน้า OOM ทั้งที่ยังเหลือ VRAM วางอยู่ 470 MB

กลาง 2026-08-25

รัน smoke run 5 step แล้วอ่านค่า peak VRAM จริงก่อนเริ่มงานเทรนเต็มรูปแบบที่เสียเงินเสมอ โดยเฉพาะหลังจากแก้พารามิเตอร์อะไรก็ตามที่กระทบหน่วยความจำ

ต้องลดขนาด LoRA ลงครึ่งหนึ่งกลางแผน เพื่อให้ MAX_LENGTH 32,768 พอดีกับการ์ด 95GB และก็ยังพอดีแบบเนียนฉิว

กลาง 2026-08-25

คำนวณงบ VRAM แยกเป็นส่วนๆ (น้ำหนักของ vs สถานะที่เทรน vs activation) ก่อนเลือกความยาว context กับ rank ของ LoRA ไปพร้อมกัน ไม่ใช่ไปหั่นอะไรที่หั่นง่ายที่สุดตอนที่รันแล้วมันไม่ยอมเริ่ม

เหลือเครดิตแค่ \$8.91 ไม่พอจบแม้แต่รอบเดียวบนเส้นทางอัปเกรดไป H200

กลาง 2026-08-25

เช็คยอดเครดิตเทียบกับต้นทุนรวมที่คาดว่าจะใช้ทั้งรอบ ก่อนกดเริ่ม งานที่ตายตอน 60% เพราะเครดิตหมด เสียเงินเท่ากับงานที่ไม่เคยเริ่ม แต่ได้เงินคืนน้อยกว่า

ประเมิน Pixtral-Large อย่างละเอียดเรื่องความสามารถ แล้วมาตัดทิ้งเพราะ licence ในบรรทัดเดียว

กลาง 2026-08-25

คัด licence ของโมเดลก่อนจะเริ่มประเมินความสามารถ licence เป็นด่านผ่าน/ไม่ผ่าน ส่วนความสามารถเป็นการจัดอันดับ อย่าเอาตัวที่ยังไม่ผ่านด่านมาจัดอันดับ

MiniMax M3 มีกับดัก licence เรื่องใช้เซิร์ฟเวอร์แบบเดียวกับ Pixtral-Large แต่ปล่อยค้างไว้ไม่ได้เคลียร์

กลาง 2026-08-25

พอโดนกับดัก licence เข้าไปครั้งหนึ่งแล้ว ให้เคลียร์ตัวเลือกทุกตัวที่มีกับดักหน้าตาเดียวกันให้จบใน session เดียว การเก็บไว้ "ค่อยอ่านทีหลัง" คือวิธีเสียเวลาชั่วโมงเต็มซ้ำสองรอบ

ตัวจำแนก PRINTED_ID เป็นแค่ regex heuristic ที่จำแนกป้ายภาษาไทยที่พิมพ์จริงผิด

กลาง 2026-08-25

เวลา heuristic เป็นตัวซับซ้อนที่เผยแพร่ออกไปหรือซับซ้อนเกินไปให้บอกทิศทางความผิดพลาดของมันในประโยคเดียวกับตัวเลข และทำให้การแก้ย้อนกลับได้รายการการ ตัวที่ heuristic พลาดจะได้กู้คืนได้

ข้อเสนอ VRAM เยอะที่สุด (8x RTX 4090, 384GB ที่ \$2.99/ชม.) จะเป็นการซื้อที่แย่มากสำหรับโมเดล MoE เพราะไม่มี NVLink

กลาง 2026-08-25

สำหรับ MoE ให้ดู bandwidth ของการเชื่อมต่อระหว่างการ์ดก่อนราคาต่อชั่วโมงและ VRAM รวม ข้อเสนอ VRAM เยอะที่สุดมักเป็นตัวเลือกที่การ์ดคุยกันไม่ได้

ตั้ง auto top-up ของ Vast.ai ไว้ที่ < \$0.00 — มันเติมเงินหลังยอดหมดแล้ว ไม่ใช่ก่อน

กลาง 2026-08-30

ตั้ง threshold เติมเงินอัตโนมัติให้สูงกว่าศูนย์อย่างน้อยเท่ากับค่าใช้จ่ายหลายชั่วโมง (เช่น อัตราเหมาเงิน × 4 ชม.) และห้ามคิดว่า การเติมที่ threshold ศูนย์จะปกป้องงานที่รันทิ้งไว้ได้

เช่าเครื่องที่แสดงสัญญาณผิดปกติตั้งแต่ตอนสร้าง — 26 นาทีต่อมามันหลุดออฟไลน์

กลาง 2026-08-30

ถ้าเครื่องเช่ามีอะไรผิดปกติตั้งแต่ตอน provision (ไม่มี direct port, เน็ตช้า, คะแนนความน่าเชื่อถือต่ำ) ให้ทำลายแล้วเช่าใหม่ทันที ค่าเช่าใหม่ถูกกว่าการที่เครื่องพังตอนรันไปแล้ว 10 ชั่วโมงเยอะ

`vastai destroy instance` ถ้าม y/N แบบโต้ตอบ พอร์นในสคริปต์มันเลิกไปเงียบๆ — การ์ดอาจยังเปิดอยู่ทั้งที่คิดว่าคืนไปแล้ว

กลาง 2026-08-30

หลังสั่งทำลาย/รีบูตทุกครั้ง ให้ถามฝั่งผู้ให้บริการซ้ำว่าเหลือ instance กี่ตัว และต้องเป็นศูนย์เท่านั้น ห้ามใช้ 'คำสั่งรันแล้วไม่ error' เป็นหลักฐานว่าทรัพยากรถูกคืนแล้ว

การ evaluate เติมรูปแบบใช้เวลา 7-8 ชั่วโมงต่อ fold — พอๆ กับเวลาเทรน แต่ไม่ได้ตั้งบไว้

กลาง 2026-08-30

ตั้งบ GPU-hours ของขั้นตอน evaluate รวมไว้ในประมาณการเดียวกับการเทรน วัดเวลาต่อตัวอย่างตอน smoke test แล้วคุณออกมาก่อนตัดสินใจรัน eval เติม

vLLM รองรับ MoE LoRA จริง แต่ `is\_3d\_lora\_weight` ประกาศผิดได้ แล้วมันจะพ่นผลขยะออกมาเงียบๆ โดยไม่มี error

กลาง 2026-08-30

ทุกครั้งที่เปลี่ยน inference stack ให้รันหน้าทีรู้คำตอบอยู่แล้วเป็น regression แล้ว diff กับผลอ้างอิงก่อนเชื่อ — ในงาน ML serving 'ไม่มี error' ไม่ใช่หลักฐานว่าถูกต้อง

อิมเมจของ **vast.ai** ไม่มี `/workspace` (redirect log ตอน onstart เลยลัมเจียบๆ) และ **torch** อยู่ใน `/venv/main` ไม่ใช่ **python3** ของระบบ

กลาง 2026-08-30

ห้าม redirect log ตอน bootstrap ลงโพลเดอร์ที่ยังไม่ได้สร้างในสคริปต์เดียวกัน และให้ระบุ path ของ interpreter ให้ชัดเจน (which python, path ของ venv) แทนที่จะเดาว่า `python3` มี dependency ครบ

อิมเมจ **vastai/pytorch** สร้าง **authorized_keys** ด้วย **permission** ผิด ทำให้ **SSH** เข้าไม่ได้เลย

กลาง 2026-08-31

เลือกใช้อิมเมจตัวเดียวที่พิสูจน์แล้วว่า SSH เข้าได้ อ่าน log ของ vastai เพื่อดู error จริงของ sshd แทนการเดา และถ้าเครื่องใหม่เข้าไม่ได้ ให้ทำลายแล้วเข้าใหม่ อย่าเปลี่ยนเป็นโปรเจกต์ซ่อมเครื่อง

เทรน **fold1** ไป **27 step** ต้องทิ้งทั้งหมด เพราะเปลี่ยนค่า **MAX_PIXELS** กลางรอบเทรน

กลาง 2026-08-31

ต้องตรวจสอบค่า resolution และ memory ให้เรียบร้อยก่อน step แรกจริงๆ เพราะการเปลี่ยนพารามิเตอร์ที่กำหนดรูปแบบข้อมูลกลางทาง จะทำให้ทุกอย่างที่เทรนไปก่อนหน้านี้ใช้ไม่ได้ทันที

ลง **flash-linear-attention** ไม่ได้ เพราะ **CUDA toolkit 12.8** ชนกับ **torch cu13.0**

กลาง 2026-08-31

ตอนเข้าเครื่อง ต้องจับคู่ CUDA build ของ torch ให้ตรงกับ CUDA toolkit ของ image — คู่ที่ไม่ตรงกันจะทำให้ความเร็วช้าลงเป็นเท่าตัวโดยไม่มีใครรู้ตัว ไม่ใช่แค่เสียพีเออร์เสริม

watcher process หลุดการเชื่อมต่อ **SSH** กลางคัน ทำให้ **queue** เก่ายังรันงานที่ถูกยกเลิกไปแล้วต่อไปเรื่อยๆ

กลาง 2026-08-31

ห้ามรันขั้นตอน kill-and-relaunch หลายขั้นตอนบน SSH session เดียว ให้สั่งทีละคำสั่ง แล้วเช็คสถานะเครื่องจริง (ps) ก่อนเชื่อ log ทุกครั้ง

มีการเช่า **GPU** ทั้งทีมาบอกรายการให้หยุดไปแล้ว

กลาง 2026-08-31

เมื่อได้รับคำสั่งให้หยุด ต้องยกเลิกขั้นตอนการเช่าเครื่องที่อยู่ใน queue หรือคำสั่งทำอย่างชัดเจน แล้วเช็คให้แน่ใจว่าเหลือ instance เป็นศูนย์จริง ไม่ใช่แค่สันนิษฐานว่าจะไม่มีอะไรเริ่มต่อ

เครื่องที่เพิ่งเช่าจาก **vast.ai** ปฏิเสธ **SSH key** ที่ถูกต้องอยู่หลายนาที จนต้อง **destroy** แล้วเช่าใหม่

กลาง 2026-09-01

ถ้าเครื่องที่เพิ่งเช่ามีปัญหา และเช็ค config ฝั่งเราแล้วสะอาดดี ให้ destroy แล้วเช่าใหม่ทันที อย่าเสียเวลาไล่ debug ปัญหาของ provisioning ฝั่งอื่นเกินไม่จำเป็น แต่ต้องเช็ค fingerprint ก่อนว่าเป็นของฝั่งเราจริง ไม่ใช่ของเรา

การเช็คสถานะล้มเหลวแค่ครั้งเดียว ทำให้ **web flow** ทั้งหมดหยุดทำงาน ทั้งที่งานจริงยังรันอยู่ปกติ

กลาง 2026-09-01

การ poll สถานะล้มเหลว กับงานจริงล้มเหลว เป็นคนละเรื่องกัน การเช็คสถานะที่พลาดไปครั้งเดียว ต้องไม่ทำให้งานที่กำลังสังเกตอยู่ถูกยกเลิกไปด้วย

SSH tunnel ไม่มี **keepalive** หรือตัวตรวจจับตอน **forward** ล้มเหลว ทำให้มันตายไปเงียบๆ ได้

กลาง 2026-09-01

SSH tunnel ที่ pipeline ฟังพาวอยู่ ต้องรันพร้อม keepalive และตั้ง `ExitOnForwardFailure=yes` — tunnel ที่ตายเงียบๆ แย่กว่า tunnel ที่ไม่เคยเริ่มทำงานเลยเสียอีก

งานเรื่อง **network-resilience** ทั้งหมดถูกยืนยันด้วย **simulated test** เท่านั้น ไม่เคยเจอกับเน็ตหลุดจริงบน **GPU run** จริงเลย

กลาง 2026-09-01

โค้ดที่ทนต่อความล้มเหลวแต่พิสูจน์ด้วย mock อย่างเดียว ยังเป็นแค่สมมติฐาน ต้องกำหนดให้มีการจำลองความล้มเหลวจริงสักครั้ง (ตัดเน็ต 60 วินาทีกลางงาน) บนเครื่องเช่าราคาถูกก่อนจะเชื่อได้

ระบบ checkpoint/resume ถูกส่งมอบพร้อม unit test เท่านั้น ไม่เคยถูกทดสอบกับงาน GPU ที่ล้มเหลวจริงเลย

กลาง 2026-09-02

เส้นทางกู้คืนงานที่พิสูจน์ด้วย unit test อย่างเดียว ยังถือว่าไม่ได้พิสูจน์จริง ต้องลองทำงานกลางคันจริงๆ สักครั้งบนเครื่องเช่าราคาถูก แล้วเช็คว่าการ resume ข้ามขั้นตอนที่ทำเสร็จแล้วได้จริงหรือไม่

เลือกออฟเพอร์ vast.ai เองแล้วรับเช่า แต่ระบบกลับบอกว่าทรัพยากรไม่พร้อมใช้งาน แล้วเข้าคิวรอเปลี่ยนสถานะ

กลาง 2026-09-02

ให้ถือว่า 'state change queued' คือเครื่องที่ตายแล้ว ไม่ใช่แค่ล่าช้า — ยกเลิกทันที ออฟเพอร์ที่ถูกนำเสนอแล้วสตาร์ทไม่ได้เป็นปัญหาของผู้ให้บริการ การรอมันเสียเวลามากกว่าเช่าใหม่

มีคนขอให้ทำคอลัมน์ลาก-ปรับขนาดได้ แต่ภาษา config ของ Label Studio ไม่มี splitter ให้ใช้

ต่ำ 2026-07-09

ก่อนสัญญาว่า UI จะทำอะไรได้ในภาษา config ของเครื่องมือภายนอก ต้องยืนยันก่อนว่า tag นั้นมีอยู่จริงในภาษานั้น — การเห็น behavior แบบนั้นใน UI ของ vendor เองไม่ได้แปลว่า config ของเราจะทำแบบเดียวกันได้

งบ GPU ที่เช่าตั้งไว้ \$9.49 (ประมาณ 9 ชั่วโมง) ใช้หมดกลางเซสชัน ต้องเติมเพิ่ม \$10

ต่ำ 2026-07-21

ตั้งงบเช่า GPU ครั้งแรกไว้ที่ 2-3 เท่าของเวลาที่ประเมิน และตั้ง alert เตือนยอดเงินคงเหลือ — instance ที่เครดิตหมดกลางคันจะสูญเสียทุกอย่างที่อยู่บนนั้น เหมือนกับการ destroy เลย

พยายามรัน Jupyter บน GPU ที่เช่าแล้วแล้วช้า ก่อนจะเปลี่ยนมาใช้ SSH

ต่ำ 2026-07-21

ทุกครั้งที่เช่า GPU ให้เชื่อมต่อผ่าน SSH + tmux ตั้งแต่นาทีแรก มองว่า notebook UI เป็นแค่เครื่องมือช่วย debug เท่านั้น ห้ามใช้เป็นช่องทางควบคุมงานที่รันหลายชั่วโมง

เจอ error 404 จาก HuggingFace แล้วเข้าใจผิดว่าอัปโหลดล้มเหลว ทั้งที่จริงแค่ repo เป็นแบบ private

ต่ำ 2026-07-24

ต้องตรวจสอบไฟล์บนคลาวด์ผ่าน authenticated API เท่านั้น ไม่ใช่เปิดดูผ่านเบราว์เซอร์ที่ไม่ได้ล็อกอิน error 404 ของ repo ที่เป็น private ส่วนใหญ่แปลว่า 'ยังไม่ได้ล็อกอิน' มากกว่า 'ยังไม่ได้อัปโหลด'

บ้าน 07 หน้า A-30/31: วางหัวเตียงผิดผนัง และบันทึกละเมิดเป็นเตียงในห้องแต่งตัว

ต่ำ 2026-07-28

ต้องเทียบสัญลักษณ์กับ legend และห้องที่มีนอยู่ก่อนบันทึกทุกครั้ง และต้องระบุทิศทางเทียบกับจุดอ้างอิงที่มีชื่อ (เส้นกริดหรือทิศ) เสมอ ห้ามใช้คำแบบ 'ผนังที่มันดูเหมือนอยู่ด้วย'

ไดอารี่สองฉบับบันทึกค่าเช่าเครื่องของ t01 ไม่ตรงกัน — ต้องมีตัวใดตัวหนึ่งผิด

ต่ำ 2026-07-28

ต้องบันทึก machine ID, ค่าเช่าต่อชั่วโมง และยอดบิลจริงของเครื่องที่เช่าจริง ไม่ใช่เครื่องที่เลือกไว้ตอนวางแผน ไม่จำเป็นการเทียบต้นทุนข้ามรอบจะไม่มีคามหมายเลย

สมมติเอาเองว่าโมเดลขนาดใกล้เคียงกันจะใช้เวลาเทรนใกล้เคียงกัน แต่ t02 ใช้เวลานานกว่า t01 ถึง 2.9 เท่า ทั้งที่ใช้ hyperparameter เดียวกัน

ต่ำ 2026-07-28

ห้ามคำนวณงบเวลาเช่าเครื่องจากจำนวนพารามิเตอร์ ให้วัด throughput จริงจากการรันทดสอบสั้นๆ บนโมเดลจริงก่อนตัดสินใจรันเต็มรูปแบบ

เฟส 11 (การ destroy instance ที่เช่า) ยังไม่ถูกทำ — เป็นความเสี่ยงเรื่องค่าใช้จ่ายที่เปิดค้างอยู่

ต่ำ 2026-07-29

ต้องคู่กฎ 'ห้าม destroy โดยไม่ยืนยันก่อน' เข้ากับกฎ 'ยืนยันว่าสำรองข้อมูลแล้วและตัดสินใจภายใน N ชั่วโมง' — การถือครอง resource ที่คิดเงินตามเวลาไว้แบบไม่มีกำหนด จะเปลี่ยนกฎความปลอดภัยให้กลายเป็นช่องรั่วไหลของเงินแทน

ล็อกอินบนเว็บที่ deploy แล้วล้มเหลวด้วยข้อความ 'Invalid login credentials' — แยกไม่ออกว่าสาเหตุคืออะไร

ต้า 2026-08-20

เมื่อ error การล็อกอินถูกออกแบบให้กำกวมโดยตั้งใจ อย่าไปไล่ debug ที่ตัว deployment ให้ไปตรวจสอบว่าบัญชีมีอยู่จริงหรือไม่ที่ provider โดยตรงก่อน และควรจำไว้ว่าเส้นทางเดโมที่ไม่ต้องล็อกอินคือทางลัดที่เร็วกว่าในการปลดบล็อกเดโม

ฝั่งผู้ผลิตโมเดลตัวเอง: GLM-4.6V เทรนแบบ LoRA ไม่ได้ ส่วน NVLM-D-72B รันบน vLLM ไม่ได้เลยด้วยซ้ำ

ต้า 2026-08-25

ก่อนนับโมเดลไหนเป็นตัวเลือก ต้องยืนยันก่อนว่ามีทาง train จริงและทาง serve จริงที่ใช้ได้วันนี้ — และให้เช็ค issue tracker ของผู้ผลิตเองด้วย ไม่ใช่แค่ดูจาก model card

ใช้ `--max\_steps 5` เป็นตัว override ผ่าน CLI ของ LLaMA-Factory แต่ parser พัง — มันต้องการ `key=value` ไม่มีขีดกลาง

ต้า 2026-08-30

ต้องเช็ครูปแบบคำสั่งตัวอย่างของเฟรมเวิร์กเองก่อนรันบนเครื่องที่เก็บเงิน การรันพลาดหนึ่งครั้งบน GPU เซ้า่นั้นคือการเสียเงินฟรีๆ

แผนสันนิษฐานว่าต้องเช่าสองเครื่องแยกกัน (การ์ดถูกไว้ dry-run + การ์ดใหญ่ไว้เทรน) แต่วัดจริงพบว่าการดเดียวขนาด 96GB ครอบคลุมทั้งสองงานได้เลย

ต้า 2026-08-30

ให้วัด VRAM สูงสุดจริงจากการ dry-run 5 step ก่อนวางแผนใช้หลาย GPU หรือเช่าหลายรอบ — การประมาณความต้องการมักบวกเผื่อเกินความจำเป็นเสมอ

ถ้าเริ่ม server ก่อนที่ pip install ของ onstart จะรันเสร็จ จะทำให้ import ของ unsloth ล้มเหลว และ health check จะไม่มีวันเขียวเลย

ต้า 2026-08-30

ห้ามเริ่ม service ที่ต้องพึ่งสคริปต์ provisioning ก่อนจะยืนยันได้แน่ชัดว่า provisioning เสร็จแล้ว — ให้ poll เช็คว่า process ตัวติดตั้งจบแล้วจริง อย่างนอนรอเฉยๆ แล้วหวังว่ามันเสร็จ

HF token ที่ก๊อปปี้ไปไว้บนการ์ด fold3 ไม่เคยถูกยืนยันก่อนที่เครื่องนั้นจะออฟไลน์ไป — ต้องมาก๊อปปี้ใหม่ทั้งหมดบนเครื่องทดแทน

ต้า 2026-08-30

ให้ยืนยันทันทีในขั้นตอนเดียวกันเลย ไม่ใช่ปล่อยเป็นงานติดตามทีหลัง — และอย่าเสียเวลาดึงค่ากับเครื่องที่มีสัญญาณผิดปกติตั้งแต่ตอน provision จนกว่าจะพิสูจน์ได้ว่ามันเสถียร

SSH proxy ของเครื่องที่เข้ามา bind พอร์ต forwarding ไม่ได้

ต้า 2026-08-31

เข้าเครื่อง เชื่อม SSH แล้วตรวจให้ครบวงจรก่อนอัปโหลดอะไรทั้งนั้น — ปัญหาเน็ตเวิร์กฝั่งผู้ให้บริการเจอตอนเครื่องยังว่างเปล่าคือถูกที่สุดแล้ว

gpu3 สตาร์ทไม่ขึ้น พ้อง CUDA out of memory ทั้งที่เพิ่ง tmux kill-server และ GPU ก็ว่างอยู่

ต้า 2026-08-31

หลัง kill process บน GPU แล้ว ให้ poll nvidia-smi จนแน่ใจว่าหน่วยความจำถูกปล่อยจริง ค่อยรันตัวใหม่ อย่ารันทันที

ตั้งเวลา auto-shutdown 90 นาทีไว้ตั้งแต่ต้น session แล้วต้องมายกเลิกกลางคัน แคมยังมีตัวจับเวลาค้างจาก session ก่อนหน้าอยู่แล้ว

ต้า 2026-09-01

auto-shutdown ที่ตั้งไว้ต้องถูกปิดโดยตัวงานที่ทำเสร็จ ไม่ใช่รอให้ใครสักคนมาสังเกตนาฬิกาเอง และก่อนเริ่มงานที่ใช้เวลานาน session ควรตรวจก่อนว่าไม่มีตัวจับเวลาค้างอยู่

กับดักของเครื่องมือ และ พฤติกรรมของบริการภายนอก

เครื่องมือหรือบริการภายนอกที่ทำงานไม่เหมือนที่เอกสาร/ชื่อ/ผลลัพธ์บอกไว้

ทั้งหมด 58 เรื่อง · วิกฤต 2 · สูง 14 · กลาง 32 · ต่ำ 10

เรื่องที่ต้องอ่านให้ครบ (16)

วิกฤต 2026-07-24

แตกไฟล์ mmproj ไม่สำเร็จเพราะเครื่องมือยังไม่รองรับโมเดลนี้ แล้วเพิ่งมารู้ตอนงานเกือบเสร็จ

เกิดอะไรขึ้น: ตอน export GGUF วันที่ 2026-07-21 แตกไฟล์ mmproj (ตัวแปลงรูปเป็นตัวเลข) ออกมาไม่ได้ — โมเดลใหม่เกินกว่าที่เครื่องมือจะรองรับ ผลคือได้ GGUF ที่อ่านได้แต่ข้อความ และเรื่องนี้เพิ่งเจอตอนขั้นสุดท้ายของ pipeline

ต้นเหตุจริง: คิดเอาเองว่าเครื่องมือรองรับสถาปัตยกรรมโมเดลใหม่เอี่ยมนี้ โดยไม่ได้ตรวจสอบ แคมจัดตรวจยังไม่อยู่ปลายสุดของ pipeline ที่ยาวและแพง แทนที่จะอยู่ตอนต้นซึ่งตรวจฟรี

เสียหายอะไร: การจนท้วงรอบได้ผลลัพธ์ที่ใช้งานตามจุดประสงค์ไม่ได้เลย และปัญหานี้ค้างอยู่ตั้งแต่ 07-21 ถึง 07-24

แก้ยังไม: พบว่า finetune_vision_layers=False แปลว่าฝั่งที่อ่านรูปเหมือนกับโมเดลตั้งต้นทุกไบต์ เลยดาวน์โหลด mmproj (899MB) ที่ผู้ผลิตเผยแพร่เองมาจับคู่กับ GGUF ที่จนท้วงได้แล้ว ไม่ต้องแตกไฟล์เองเลยสักนิด

กฎที่ได้: ตรวจสอบก่อนเทรนว่าเครื่องมือรองรับสถาปัตยกรรมโมเดลใหม่จริงไหม อย่าไปตรวจตอน export และถ้าพิสูจน์ได้ว่าส่วนไหนไม่ได้ถูกแตะระหว่างเทรนเลย ก็เอาเวอร์ชันที่ผู้ผลิตเผยแพร่มาใช้ ดีกว่าสร้างใหม่เอง

วิกฤต 2026-08-02

ตัวตรวจเวอร์ชันแรกจะยุบกลุ่มฐานราก 4 กลุ่มที่มี confidence_flags ต่างกันทิ้งไปเลย

เกิดอะไรขึ้น: ตัวตรวจฐานรากเข้าเวอร์ชันแรกตัดสินใจว่าเข้าจาก 'confidence_score' กับ 'note' เท่านั้น มันจะยุบกลุ่ม 4 กลุ่มที่แต่ละรายการมีเหตุผลใน 'confidence_flags' *ต่างกัน* ทิ้งไป รวมถึงรายการที่ติดธง 'inferred_from_repeating_module' ด้วย

ต้นเหตุจริง: ญญแสำหรับตัดข้อมูลเข้าถูกสร้างจากฟิลด์ที่นี้กออก ไม่ใช่จากทุกฟิลด์ที่แยกแยะสองรายการออกจากกันได้จริง

เสียหายอะไร: จะทำลายบันทึกว่าตำแหน่งฐานรากไหนอ่านมาจากแบบจริง กับตำแหน่งไหนอนุมานเอา — เป็นการสูญเสียที่มาของข้อมูลแบบกู้คืนไม่ได้ แคมถูกนำเสนอในรูปของการ 'แก้ไข' ที่ดูสะอาดตา

แก้ยังไม: ตอนนี้ตัวตรวจนับ 'confidence_flags' ด้วยแล้ว ส่วนอีก 11 กลุ่มที่เหลือถูกบันทึกไว้ใน \$0.10 ว่าเป็นช้อยกเว้นที่อนุญาตเพื่อไม่ให้มีใครมา 'แก้' มันทีหลัง

กฎที่ได้: ญญแสำหรับตัดข้อมูลเข้าต้องรวมทุกฟิลด์ที่แยกแยะสองรายการออกจากกันได้จริง โดยเฉพาะฟิลด์ที่บอกที่มาของข้อมูล และต้องบันทึกช้อยกเว้นที่ตั้งใจไว้ใน spec ไม่งั้นการทำงานรอบหน้าจะมา 'เก็บกวาด' มันทิ้ง

สูง 2026-07-06

ไม่มีเครื่องมือชุม/ครอบภาพ ทำให้รอบสกัดข้อมูลใหม่แม่นยำน้อยกว่า Gen 2 เกาที่ชุมภาพจริงได้

เกิดอะไรขึ้น: หน้า 19 view2 (แปลนคาน) ในรอบ t2 ยังมีส่วนความมั่นใจต่ำอยู่ (B1, ท่อนบนของ B3, B3X/B5X ในโซนต่อขยายใต้แนว A) ไดอารี่ระบุว่านี่ 'ดีกว่า t1 (โครงสร้างชัดกว่า) แต่ก็ยังไม่แม่นยำเท่า Gen 2 ที่มีการชุมภาพจริง' เพราะไม่มีเครื่องมือชุม/ครอบภาพจริงให้ใช้

ต้นเหตุจริง: ความสามารถของเครื่องมือหายไประหว่างเจเนอเรชัน — เวอร์กโฟลว์ Gen 2 ชุมภาพได้ เวอร์กโฟลว์เอเจนต์/มือของ Gen 3 ทำไม่ได้ ไม่มีใครสร้างความสามารถชุม/ครอบกลับมาก่อนเริ่มสกัดข้อมูลเต็มสองรอบที่ต้องอ่านรายละเอียดเล็กๆ

เสียหายอะไร: ความแม่นยำถดถอยลงอย่างวัดได้เมื่อเทียบกับเจเนอเรชันก่อนหน้าตลอดสองรอบงาน และเกิดขึ้นตรงกับหน้า (แปลนคาน) ที่มีข้อมูลโครงสร้างมีค่าที่สุด มีหลายหน้าตลอดสองวันถูกจดว่าต้องมีคนชุมเข้าไปตรวจสอบซ้ำ

แก้งัง: ยังไม่ได้แก้ บันทึกไว้เป็นข้อจำกัดที่รู้อยู่แล้ว หน้า 19 view2 ความมั่นใจดีขึ้นแต่ยังต่ำกว่า Gen 2

กฎที่ได้: ก่อนเริ่มเจเนอเรชันการสกัดข้อมูลใหม่ ต้องทำ inventory ความสามารถที่เจเนอเรชันก่อนมี แล้วยืนยันว่าแต่ละอย่างยังอยู่ครบ การเขียนใหม่ที่ทำเครื่องมือหายไปเสียๆ จะได้ข้อมูลแยกกว่าเดิมทั้งที่ดูเหมือนความก้าวหน้า และขั้นตอนครอบ/ชุมคือสิ่งจำเป็นสำหรับการอ่านรายละเอียดในแบบ ไม่ใช่แค่ของเสริม

สูง 2026-07-10

agent เบื่องหลังถูกฆากกลางทางทั้งที่ยังไม่ได้เขียนไฟล์เลยสักไฟล์ ทำให้ BOQ 10 หน้าว่างอยู่สองวัน

เกิดอะไรขึ้น: Agent 4 (BOQ ฐานรากแผ่ หน้า 49-58 ของบ้านหลังที่ 2) ถูกหยุด — จะโดนสั่งหยุดเองหรือ session ปิดก็ไม่เคยยืนยันได้ — ตอนที่ยังตรวจรหัสรายการอยู่ และยังไม่ได้เขียนไฟล์ออกมาสักไฟล์ บ้านหลังที่ 2 เลยค้างอยู่ที่ 88 ไฟล์ ครอบคลุม 62 จาก 72 หน้า และค้างแบบนั้นจนต้องมาทำมือเสร็จวันที่ 11 ก.ค.

ต้นเหตุจริง: agent ถูกออกแบบให้เก็บงานทั้งหมดไว้ในหน่วยความจำแล้วค่อยเขียนตอนจบ แทนที่จะเขียนทีละหน้าเมื่อทำเสร็จ พอโดนฆากกลางทางงานเลยหายหมด และไม่มีใครคอยตรวจว่าเสร็จหรือยัง ช่องว่างเลยเจอตอมา validate

เสียหายอะไร: บ้านหลังหนึ่งขาดข้อมูลไป 10 หน้าเป็นเวลาสองวัน ทำให้งาน label-studio กับการบันทึก data log ของบ้านหลังนั้นติดขัด และต้องกลับมาทำใหม่ที่ละหน้าด้วยมือ

แก้งัง: อ่านหน้า 49-58 ใหม่เองทีละหน้าในวันที่ 11 ก.ค. จนบ้านหลังที่ 2 ครบ 107/107 ไฟล์ และ 72/72 หน้า

กฎที่ได้: agent ที่ทำงานสกัดข้อมูลยาวๆ ต้องเขียนลงดิสก์ทันทีที่ทำเสร็จเป็นชิ้นๆ ห้ามเก็บไว้เขียนตอนท้าย และต้อง diff ไฟล์ที่ได้จริงกับรายการหน้าที่คาดว่าจะได้เสมอ ก่อนจะประกาศว่าบ้านหลังนั้นเสร็จ

สูง 2026-07-28

ไม่เคยยืนยันเลยว่า Unsloth export GGUF ให้โมเดล MoE VL ตัวนี้ได้จริงหรือไม่ — ทั้งที่มี issue เรื่อง GGUF เสียหายอยู่แล้วในโปรเจกต์ต้นทาง

เกิดอะไรขึ้น: จนถึงตอนปิด session ยังไม่ยืนยันได้ว่า Unsloth export GGUF ให้โมเดล MoE VL ตัวนี้ได้จริงไหม ทั้งที่มี issue #3899 รายงานว่า Qwen3-VL-2B ได้ GGUF เสียหายไปแล้ว นอกจากนี้ VRAM ที่ต้องใช้จริงของ bf16 LoRA 30B-A3B ก็ยังไม่ยืนยันเช่นกัน เป็นแค่ค่าประมาณจาก t01 ที่ ~74GB ซึ่งประเมินจากรัน TEST_STEPS=5 เท่านั้น

ต้นเหตุจริง: ความเป็นไปได้ของแนวทางส่งมอบนี้ถูกสันนิษฐานเอาเองแทนที่จะไปเช็คกับ issue tracker ต้นทางก่อนที่จะหุ้มรอบงานที่เสียเงินลงไป และตัวเลขทรัพยากรสำคัญถูกประมาณจากการรันแค่ 5 step แทนที่จะวัดจากการรันเต็ม

เสียหายอะไร: ผลงานหลักของทั้งรอบ (ไฟล์ GGUF ที่รันได้จริง) ถูกบล็อกด้วยข้อจำกัดที่น่าจะเจอได้ตั้งแต่แรกถ้าค้น issue tracker ต้นทางก่อน

แก้ยังไม: ยังไม่ได้แก้ — บันทึกว่ายังไม่ยืนยัน พร้อมโน้ตว่าถ้าจะเดินหน้าต่อต้องรอ peft/unsloth เวอร์ชันใหม่ หรือไม่ก็ต้องเขียนตัว merge เองเลย

กฎที่ได้: ก่อนหุ้มค่าใช้จ่ายลงกับแนวทางส่งมอบใดๆ ให้ค้น issue tracker ของ toolchain นั้นดูตระกูลโมเดลของเราให้ชัดเจน และห้ามเอาตัวเลขที่ประมาณจากการรันแค่ 5 step มาถือเป็นความต้องการทรัพยากรที่ยืนยันแล้ว

สูง 2026-07-29

REQUEST_TIMEOUT_S=1200 ฆ่างานที่กำลังรันอยู่จริงๆ ที่หน้า 03

เกิดอะไรขึ้น: ค่า timeout ถูกตั้งจากหน้า 01-02 ซึ่งเป็นปกและสารบัญ หน้าที่เขาที่สุดในเล่ม หน้า 03 ต้องใช้เวลา 1,256 วินาที แต่ถูกฆ่าที่ 1,200 วินาที พอปรับเพิ่มเป็น 3,600 หน้านั้นก็ทำงานเสร็จปกติ

ต้นเหตุจริง: ตัวอย่างที่ใช้ตั้งค่า threshold คือกรณีที่เจอก่อน ไม่ใช่กรณีที่หนักที่สุด หน้า 01-02 เป็นเนื้อหาที่ถูกที่สุดในแบบก่อสร้างชุดไหนก็ตามอยู่แล้วโดยโครงสร้าง งบประมาณที่คำนวณจากหน้าพวกนี้จึงไม่ครอบคลุมเนื้อหาอื่นๆ

เสียหายอะไร: งานต้องรีสตาร์ทและเสียเวลาจริงไป ทั้งที่งานกำลังรันอยู่อัตรา ~13-21 นาที/หน้า และยังทำให้เข้าใจผิดชั่วขณะว่างานค้าง (hang)

แก้ยังไม: แก้แล้ว — ปรับ REQUEST_TIMEOUT_S จาก 1200 เป็น 3600 หน้า 03 ทำงานเสร็จภายใน 1,256 วินาที

กฎที่ได้: ต้องตั้งค่า timeout/threshold โดยอิงจากกรณีที่หนักที่สุดที่เป็นไปได้ ไม่ใช่จากอะไรก็ตามที่รันก่อน ถ้ามีตัวอย่างแค่ 2 ตัวและทั้งคู่เป็นรายการที่เขาที่สุดในชุด นั้นไม่ใช่ sample ที่เป็นตัวแทนเลย

สูง 2026-08-02

กฎ §0.1 ที่เพิ่งเขียน กว้างเกินไป — เดือน 106 ไฟล์ แต่ 104 ไฟล์ถูกอยู่แล้ว

เกิดอะไรขึ้น: ร่างแรกของกฎ §0.1 ห้าม array ชื่อ `sections` หรือ `columns` ตัวเช็คใหม่เลยขึ้นเดือน 106 ไฟล์ พอเปิดดูจริงพบว่า 104 ไฟล์ถูกต้องอยู่แล้ว — คำว่า `sections` ในไฟล์ `index` หมายถึงหัวข้อของเอกสาร ส่วน `columns` ใน BOQ หมายถึงหัวคอลัมน์ของตาราง มีแค่ 2 ไฟล์เท่านั้นที่ผิดจริง

ต้นเหตุจริง: กฎไปห้ามที่*ชื่อ* แทนที่จะห้ามตาม*บริบท* โดยลืมไปว่าคำเดียวกันมีความหมายต่างกันไฟล์คนละแบบ

เสียหายอะไร: ตัวตรวจที่เพิ่งสร้างมีอัตราเดือนผิด 98% ถ้าเชื่อมั่นโดยไม่ตรวจ ไฟล์ที่ต้อง 104 ไฟล์จะโดน "แก้" จนกลายเป็นผิด

แก้ยังไม: แก้ที่ตัวสเปกกับตัวเช็ค (ไม่ใช่แค่ข้อมูล) ให้ห้ามตามบริบทของ pattern แล้วแก้เฉพาะ 2 ไฟล์ที่ผิดจริง

กฎที่ได้: เวลาตัวตรวจใหม่ขึ้นเดือนเป็นพรวด ให้เปิดดูของจริงก่อนแก้อะไรทั้งนั้น — โอกาสที่ตัวตรวจผิดมีมากกว่าที่ไฟล์ 104 ไฟล์จะผิดพร้อมกัน และให้ห้ามตามบริบท ไม่ใช่ห้ามตามชื่อ

สูง 2026-08-10

check_format.py ฮาร์ดโค้ดว่า 'ตัวอักษร = แถว, ตัวเลข = คอลัมน์' — จริงกับ 18 บ้าน แต่ผิดกับบ้านที่ 19

เกิดอะไรขึ้น: ตัวตรวจ grid_ref ใน `tools/check\_format.py` เหมาเอาว่าตัวอักษรคือแถวและตัวเลขคือคอลัมน์เสมอ แต่แบบโครงสร้างของบ้าน 19 หักกลับกัน — ใช้ตัวอักษรกับแกนอนและตัวเลขกับแกนตั้ง ข้อมูลที่ถูกต้องเลยไม่ผ่านตัวเช็ค

ต้นเหตุจริง: เอาข้อตกผลงที่สังเกตว่าเป็นจริงกับ 18 บ้านติดกันมาฮาร์ดโค้ดเป็นกฎสากล ตัวอย่าง 18 หลังจากผู้เขียนแบบไม่ก็รายดูเหมือนเป็นข้อพิสูจน์

เสียหายอะไร: ด้านตรวจจะปฏิเสธข้อมูลที่ถูกต้องของบ้าน 19 แล้วจะมีแรงกดดันให้ไป "แก้" บ้านให้ตรงกับตัวเช็ค — เท่ากับทำลายข้อมูลที่สัปดาห์มาถูกต้องเพื่อสนองกฎที่ผิด

แก้ยังไง: แก่ตัวตรวจให้รับได้ทั้งสองแบบการวางแกน แล้วรันใหม่ทั้ง 19 บ้านโดยไม่มีอะไรถอยหลัง

กฎที่ได้: อย่าฮาร์ดโค้ดข้อตกผลงที่สังเกตมาเป็นกฎตรวจสอบ ไม่ว่าจะมิตัวอย่างรองรับก็ตัวก็ตาม เวลาด้านตรวจไม่ผ่านกับข้อมูลชุดใหม่ ให้ถามก่อนว่าด้านตรวจผิดหรือเปล่า — ตรงกัน 18 บ้านคือรูปแบบ ไม่ใช่กฎธรรมชาติ

สูง 2026-08-21

check_format.py ไม่มีตัวตรวจความครบของหน้า บ้านที่มีแค่ 38 หน้าจึงผ่านเหมือนบ้านที่เสร็จสมบูรณ์

เกิดอะไรขึ้น: ตอนออกแบบโหมดบ้านไม่ครบของ op04 ต้องไปอ่าน check_house() แล้วยืนยันว่า check_format.py ชำมีไฟล์ที่ขึ้นต้นด้วย _ และไม่มีตัวตรวจความครบของจำนวนหน้าเลย ดังนั้นโฟลเดอร์ที่มี 38 จาก 107 หน้าก็ผ่าน validation ได้อย่างถูกต้องแยกไม่ออกจากบ้านที่เสร็จจริง

ต้นเหตุจริง: ตัวตรวจ format ตรวจแค่รูปร่างของแต่ละไฟล์ ส่วนความครบเป็นคุณสมบัติของทั้งโฟลเดอร์ ซึ่งไม่มีใครดูแล แลมี log การสัปดาห์ข้อมูลก็บันทึกว่าบ้านเสร็จแล้วโดยไม่บันทึกขอบเขตงาน

เสียหายอะไร: บ้านที่ไม่ครบทุกหลังใน dataset จะถูกนับเป็นเสร็จเสียๆ ทำให้ขนาด dataset ดูใหญ่เกินจริง และทำให้คนที่ประเมินขนาดรอบเทรนเข้าใจผิด หน้า stub ของบ้านหลัง 24 จากวันที่ 08-18 ก็เป็นปัญหาเดียวกันในอีกรูปแบบ

แก้ยังไง: เพิ่มไฟล์ _scope.json ที่บังคับต้องมี เก็บสามรายการแยกกัน (subtasks_done / subtasks_absent_from_this_house / deliberately_skipped) — สองอันหลังต้องแยกกันเพราะถ้ายุบรวมจะทำลายสัญญาณเดียวที่บอกได้ว่าหน้านั้นหายไปหรือแค่ไม่ต้องการ และแถวใน log ของรอบ op04 ต้องระบุ op04 / pass2-only

กฎที่ได้: ตัวตรวจที่ดูรูปร่างรายชิ้น ต้องมาคู่กับตัวบอกความครบในระดับชุดข้อมูลเสมอ และห้ามยุบ ไม่มีในต้นฉบับ รวมทั้งตั้งใจข้าม เพราะความต่างนี้คือสิ่งเดียวที่บอกคนอ่านทีหลังได้ว่าข้อมูลหายไปหรือแค่ไม่ต้องการ

สูง 2026-08-26

GLM-4.6V ผ่านทุกเกณฑ์บนกระดาษ แต่เทรน LoRA ไม่ได้เพราะบั๊กต้นน้ำที่ยังไม่ถูกแก้

เกิดอะไรขึ้น: GLM-4.6V มีสัญญาณอนุญาต MIT ขนาดเหมาะสม และน่าจะเป็นตัวเลือกอันดับสอง แต่ LLaMA-Factory issue #9635 ขึ้น NotImplementedError และยังไม่มีใครแก้ ทำให้เทรน LoRA เป็นไปไม่ได้

ต้นเหตุจริง: เอกสารที่บอกว่าเฟรมเวิร์กรองรับ ไม่ได้แปลว่ารองรับจริง โมเดลอาจถูกระบุว่ารองรับทั้งที่โค้ดจริงพังกลางทาง

เสียหายอะไร: ถ้าไม่เจอตอนนี้ จะไปเจอตอนเช่า GPU แล้วลองรันจริงเท่านั้น

แก้ยังไง: จับได้ตอนวิจัยรวมโมเดล โดยใช้เกณฑ์ 'มีเครื่องมือเทรน LoRA ที่ยืนยันว่าใช้ได้จริง (ไม่ใช่แค่เอกสารอ้าง)' อย่างจริงจัง

กฎที่ได้: ยืนยันการรองรับของเฟรมเวิร์กด้วยการหาคนที่รันจริงมาแล้ว หรือเช็ค issue tracker หาปัญหาที่ยังเปิดค้าง อย่าอ่านจากตารางความรองรับเด็ดขาด คำกล่าวอ้างในเอกสารไม่ใช่หลักฐาน

สูง 2026-08-28

CV ตรวจจบบั๊กเพดเจอรูปร่างดี แต่เจอเสาและคานเก็บบเป็นศูนย์ ต้องตัดขอบเขตการทดลองทิ้ง

เกิดอะไรขึ้น: รัน pass 1.5+2.5 กับหน้าแบบจริง 25 หน้าจาก 5 บ้านชุด val พบว่าตรวจจบบั๊กฐานรากได้แม่นยำ (9-20 จุด/หน้า) แต่เสาและคานเจอเก็บบเป็นศูนย์ ทำให้การทดลองคั้นนั้นซึ่งวางแผนไว้ 3 แขน ต้องตัดเหลือแค่ `plan\_footing` อย่างเดียว

ต้นเหตุจริง: คลังเทมเพลตมีเทมเพลตเสาแค่ 1 แบบ และค่า BEAM_GAP ตายตัวเดียวคือ 21px เทมเพลตเดียวไม่มีทางครอบคลุมแบบแปลนหลายชุดได้ และค่าระยะห่างค่าเดียวก็มองไม่เห็นชุดที่ใช้ระยะห่างอื่น

เสียหายอะไร: การทดลอง CV-hint 3 แขน ไม่สามารถทดสอบ subtask ที่ตั้งใจสร้างมันขึ้นมา (plan_beam) ได้ ในคั้นที่วางแผนไว้

แก้ยังไง: คั้นเดียวกันนั้น ขยายคลังเทมเพลตเสาจาก 1 เป็น 7 แบบ (เลือกมือจากมอนทาว 78 ตัวเลือก) และเปลี่ยน BEAM_GAP 21 เป็น BEAM_GAPS (16,21) ที่วัดจาก 33 หลังจริง แล้วสแกน cv_val ใหม่ ทำให้ทั้ง plan_footing และ plan_beam รันได้

กฎที่ได้: วัดความครอบคลุมของตัวตรวจจบบั๊กกับข้อมูลทั้งชุดก่อนวางแผนการทดลองที่พึงพามัน ค่าคงที่เชิงเรขาคณิตค่าเดียวจะมองไม่เห็นข้อมูลครึ่งหนึ่งของชุดที่มีความหลากหลายเสมอ

สูง 2026-08-31

LLaMA-Factory แคเตือน (ไม่ error เลย) เมื่อไฟล์สื่อที่อ้างอิงผ่าน media_dir หายไป

เกิดอะไรขึ้น: อ่านซอร์สของ LF ยืนยันแล้วว่าโค้ดส่วน join ใน converter.py บรรทัด 58 แคปล่อย warning ไม่ error ถ้าไฟล์รูปหาย

ต้นเหตุจริง: library ของ vendor เลือกวิธีจัดการความล้มเหลวแบบพ่อนปรนสำหรับ media ที่หาย ทำให้ดาต้าเซตที่พึ่งสามารถเทรนจนจบได้แบบเฉยๆ ทั้งที่ input ไม่ครบ

เสียหายอะไร: run อาจจบแล้วดูปกติดี ทั้งที่จริงๆ เทรนด้วยรูปที่ขาดหายบางส่วน ความเสี่ยงถูกลดด้วยการตรวจแบบมือเท่านั้น

แก้ยังไง: ตรวจสอบแล้วว่า path ทั้ง 1,416 รายการ resolve ผ่าน media_dir ได้ครบ ไม่มีไฟล์หายเลยสักไฟล์ และทำให้การรัน smoke test เป็นขั้นตอนบังคับก่อนเริ่มเทรนทุกครั้ง

กฎที่ได้: ให้สมมติไว้ก่อนว่า library ที่โหลด media จะแคเตือนไม่ error ต้องรันขั้นตอน 'เปิดไฟล์ทุกไฟล์ที่ถูกอ้างถึงจริง' อย่างชัดเจนก่อนจะจ่ายเงินเทรนทุกครั้ง

สูง 2026-08-31

ตัวมอนิเตอร์เทรนได้ SSHFAIL ทุกบรรทัดนาน 7 ชั่วโมงโดยไม่แจ้งเตือนเลย

เกิดอะไรขึ้น: watch4.sh ตั้งให้เช็คทุก 10 นาที และปลุกเมื่อเจอ CRASHED หรือ ALL_4_DONE แต่มันได้ SSHFAIL ทุกบรรทัดตั้งแต่รอบแรกนาน 7 ชั่วโมง ไม่เคยได้ข้อมูลจริงเลยและไม่เคยจบการทำงาน เพราะ SSHFAIL ไม่ถูกจัดว่าเป็น CRASHED

ต้นเหตุจริง: ปลอมมอนิเตอร์ทำงานโดยไม่ตรวจผลลัพธ์รอบแรกก่อน และรูปแบบความล้มเหลวของมันเอง (เชื่อมต่อไม่ได้) ถูกปล่อยผ่านแทนที่จะถือเป็นสัญญาณเตือน

เสียหายอะไร: เทรนแบบเสียเงินโดยไม่มีการดูอยู่ 7 ชั่วโมง 2 fold ที่ OOM ถูกพบจากการเช็คด้วยมือ ไม่ใช่จากมอนิเตอร์

แก้ยังไง: บันทึกลงเป็นข้อ 17 ใน rule_of_tune: ก่อนจะปลอมมอนิเตอร์ทิ้งไว้ ต้องดูผลลัพธ์รอบแรกด้วยตาก่อนเสมอ และ 'เชื่อมต่อไม่ได้' ต้องถือเป็นเงื่อนไขแจ้งเตือน

กฎที่ได้: อย่าเชื่อมมอนิเตอร์ที่ยังไม่เคยเห็นผลลัพธ์จริงแม้แต่ครั้งเดียว และให้การเชื่อมต่อล้มเหลวเป็นสัญญาณแจ้งเตือน ไม่ใช่แค่ข้ามไปเฉยๆ

สูง 2026-08-31

สั่ง `kill -f` <ชื่อสคริปต์> ผ่าน SSH ไปฆ่า SSH session ของตัวเอง — เกิดขึ้น 2 ครั้งในวันเดียว

เกิดอะไรขึ้น: เพราะคำสั่ง SSH เองมีชื่อสคริปต์ที่กำลังค้นหาอยู่ในนั้นด้วย `kill -f serve_purson.py` เลยไปแมทช์ session ตัวเองแล้ว exit ด้วยโค้ด 255 ก่อนคำสั่งถัดไปจะได้รัน ครั้งแรก: คิว GPU-A ไม่ยอมตาย ต้องไปหา PID 3411/3412 มาฆ่าตรงๆ ครั้งที่สอง: สั่ง `kill; mv; nohup` ต่อกัน กลับไม่เหลือ server รันอยู่เลย ไม่มีใครรู้จนกว่าจะเช็ค `ps -C python` แล้วพบว่าว่างเปล่า

ต้นเหตุจริง: เครื่องมือฆ่าโปรเซสแบบ pattern-matching ค้นหาทั้งบรรทัดคำสั่ง ซึ่งรวมถึงคำสั่งที่กำลังเรียกมันเองอยู่ด้วย

เสียหายอะไร: มีช่วงเวลาที่ไม่มีการ inference server รันอยู่เลยโดยไม่มีใครรู้ — งานใดๆ ที่ยิงเข้าเครื่องช่วงนั้นจะพลัดเป้าไปเลย

แก้ยังไง: ตั้งกฎใหม่: ห้าม `kill` โดยใช้ชื่อสคริปต์ผ่าน SSH เต็มขาด ให้ใช้ PID ตรงๆ หรือแยกคำสั่งออกจากกัน และต้องตรวจสอบสถานะโปรเซสจริงหลังทำเสมอ

กฎที่ได้: ห้ามฆ่าโปรเซสด้วย pattern ที่มีโอกาสไปแมทช์คำสั่งของตัวเอง ให้ฆ่าด้วย PID และตรวจสอบสถานะโปรเซสจริงหลังทำทุกครั้ง อย่าเชื่อแค่คำสั่งรันจบแล้วจะแปลว่าได้ผลตามที่ตั้งใจ

สูง 2026-08-31

ลูป `wait-for-pip` แมทช์คำสั่งของตัวเอง ทำให้ inference server ไม่เคยเริ่มทำงานทั้งที่ GPU กำลังเสียเงินอยู่

เกิดอะไรขึ้น: ตัวรันทานใช้ `while pgrep -f 'pip install' > /dev/null; do sleep 10; done && nohup ... serve_purson.py` แต่คำสั่งของ bash process เองก็มีคำว่า 'pip install' อยู่ในนั้น เลยแมทช์ตัวเองตลอดเวลา เงื่อนไขเลยเป็นจริงถาวร บนเครื่อง 49411642 pip เสร็จไปตั้งแต่วันที่ 3 แต่ลูปย่วนต่ออีกราว 17 นาที ใช้ GPU แค่ 2 MiB ไม่มี log ของ purson.log เลย ดูเผินๆ เหมือนกำลังโหลดโมเดล 70GB อยู่

ต้นเหตุจริง: บั๊กประเภทเดียวกับ `kill` ข้างบน คือการค้นหา pattern จากคำสั่งทั้งบรรทัดซึ่งรวมตัวผู้ค้นหาเองด้วย

เสียหายอะไร: เสียเวลา GPU ที่ต้องจ่ายเงินไปฟรีๆ ราว 17 นาที โดยดูเหมือนกำลังสตาร์ทตามปกติ และเป็นบั๊กประเภทเดียวกันนี้ที่เกิดขึ้นครั้งที่สามในวันเดียว

แก้ยังไง: เปลี่ยน pattern เป็น `[p]ip install` ให้วงเล็บเหลี่ยมทำให้ regex ไม่แมทช์กับ string จริงในคำสั่งของตัวเอง และพิสูจน์ทั้งสอง pattern กับ regex จริงก่อนจะใช้งานจริง

กฎที่ได้: pattern ของ `pgrep/kill/ps-grep` ทุกตัวต้องเขียนให้ไม่แมทช์คำสั่งที่เรียกมันเอง (ใช้เทคนิควงเล็บเหลี่ยม) และลูป wait ต้องพิสูจน์ว่าจบได้จริงบนเครื่องจริงก่อนจะเชื่อใจมัน

สูง 2026-08-31

บั๊กประเภท 'คำสั่งค้นเจอบรรทัดคำสั่งของตัวเอง' เกิดขึ้น 3 ครั้งในวันเดียว

เกิดอะไรขึ้น: `kill -f` ฆ่า SSH session ของตัวเอง (2 ครั้ง) และ `pgrep -f` แมทช์ลูป `wait` ของตัวเอง (1 ครั้ง) เป็นบั๊กประเภทเดียวกัน เกิดขึ้น 3 ครั้งในวันเดียวกันในสคริปต์คนละตัว

ต้นเหตุจริง: ไม่เคยมีกฎมาตรฐานเรื่องการค้น pattern บนบรรทัดคำสั่งของโปรเซส ทำให้แต่ละครั้งถูกแก้แบบเฉพาะจุดโดยไม่ได้สรุปเป็นกฎรวม

เสียหายอะไร: มีช่วงที่ไม่มี server รันอยู่เลยโดยไม่มีใครรู้ เสีย GPU เปล่าไปราว 17 นาที บวกกับเวลาที่เสียไปกับการดีบั๊กซ้ำๆ

แก้ยังไง: บันทึกบั๊กประเภทนี้ไว้ชัดเจน พร้อมวิธีแก้ไขได้จริง 2 แบบ คือใช้ PID แทนการค้นหาด้วยชื่อ และใส่วงเล็บเหลี่ยมนำหน้าตัวอักษรแรกของ pattern

กฎที่ได้: ให้ถือว่า `pgrep/kill/ps-grep` ที่ค้นด้วยชื่อไม่ปลอดภัยโดยปริยาย ให้ใช้ PID หรือใส่วงเล็บกัน และเมื่อพบบั๊กประเภทนี้ครั้งแรก ต้องรีบสรุปเป็นกฎแล้วใส่แก้ทุกสคริปต์ทันที

เรื่องที่เหลือในหมวดนี้ (42)

IDE สร้างไฟล์ขนาด 0 ไบต์ขึ้นมาที่ path เดิมของเอกสารที่ย้ายไปแล้ว ทำให้ดูเหมือนงาน 24KB หายไป

กลาง 2026-07-06

หลังย้ายไฟล์ ให้เปิดแท็บ editor ทุกอันที่ยังชี้ path เดิม แล้วตรวจให้แน่ใจว่า path เดิมหายไปจริงก่อนลบเซสชัน 'ไฟล์ 0 ไบต์' ที่ path ที่เคยถูกต้อง แยกว่าไฟล์ที่หายไปเลย เพราะมันอ่านออกมาเหมือนข้อมูลพัง ไม่ใช่เหมือนการย้าย

TextArea ที่ตั้ง rows > 1 จะแอบเพิ่มปุ่ม '+Add' แบบหลายค่าให้เอง ซึ่งจะทำให้ฟิลด์ค่าเดียวพัง

กลาง 2026-07-06

อ่านเอกสารของเครื่องมือก่อนจะไปแต่งหน้าต่าง widget ของมัน คั่นก่อน อย่าเดา และในเครื่องมือ annotation โดยเฉพาะ property อะไรก็ตามที่เปลี่ยนฟิลด์ค่าเดียวให้กลายเป็นหลายค่าได้ ถือเป็นความเสี่ยงต่อความถูกต้องของข้อมูล ไม่ใช่แค่เรื่องเลย์เอาต์

ต้องวาง config ก่อน import task เสมอ ไม่จำเป็นต้องลบ task ที่พังไปแล้วทั้งก่อน

กลาง 2026-07-06

ใน Label Studio การแก้ config ไม่ย้อนไปมีผลกับของเก่า ให้แก้และเซฟ labeling config ให้เสร็จก่อน import ทุกครั้ง และลบ task ที่พังหรือค้างอยู่ทั้ง ไม่ใช่ import หับลงไป เขียนลำดับการกู้คืนนี้ไว้ข้างๆ ตัวเครื่องมือเลย ไม่ใช่ไว้แค่ใน log ของเซสชัน

Label Studio สลับเลย์เอาต์ไม่ได้ เลยแก้ด้วยการก๊อปปี้ config เป็นคู่แฝด freeze-L/freeze-R ทำให้ทุกการแก้ที่หลังต้องแก้ไข 4-5 รอบ

กลาง 2026-07-06

ถ้าเครื่องมือบังคับให้ต้องมี config ซ้ำหลายชุด ให้ generate ชุดต่างๆ ออกมาจากต้นทางเดียว แทนที่จะนั่งก๊อปปี้ ไม่จำเป็นทุกการแก้บั๊กที่หลังจะกลายเป็นงานแก้ N ไฟล์ และสำเนาที่จะค่อยๆ เพี้ยนไปคนละทาง

'Permission denied' ตอนเปลี่ยนชื่อโฟลเดอร์กลับ ดูเหมือนข้อมูลเสียหาย แต่จริงๆ คือมีอีกโปรแกรมเปิดค้างไว้

กลาง 2026-07-09

บน Windows 'Access denied' ตอนเปลี่ยนชื่อโฟลเดอร์ แปลว่ามี process เปิดค้างอยู่ ไม่ได้แปลว่าข้อมูลพัง พิสูจน์ด้วยการลองเขียนไฟล์เข้าไปข้างใน ปิดโปรแกรมที่จับไว้ แล้วลองใหม่ ก่อนจะไปที่ตราว่าเรื่องใหญ่

ทุกรอบของการสกัดข้อมูลถูกตรวจสอบด้วยสคริปต์ที่เขียนขึ้นชั่วคราวแล้วทิ้ง

กลาง 2026-07-10

ตัวตรวจที่ตัดสินว่าชุดข้อมูลครบหรือยัง ต้องเป็นสิ่งที่ commit เข้ารีโปและมีเทสต์ของตัวเอง ตัวตรวจแบบเขียนสดทิ้ง คือของอีกชิ้นที่ยังไม่ถูกตรวจสอบมาขวางระหว่างเรากับความจริง

build llama.cpp พร้อม CUDA ไป 3 รอบแล้วไม่สำเร็จ ทั้งที่จริงๆ ไม่เคยจำเป็นต้องใช้ CUDA เลย

กลาง 2026-07-21

ก่อนจะสู้กับปัญหาการ build ให้ถามตัวเองก่อนว่าตัวเลือก build นั้นจำเป็นจริงไหม — ขั้นตอน convert/quantize ใช้ CPU ล้วนๆ ควร build เฉพาะ config ขั้นต่ำที่งานนั้นต้องการจริงๆ

แปลงเป็น q8_0 ก่อนเพื่อประหยัดพื้นที่ดิสก์ แต่ llama-quantize ปฏิเสธที่จะ requantize จาก q8_0 ต่อ

กลาง 2026-07-21

ต้อง quantize จากไฟล์ต้นฉบับความละเอียดเต็ม (f16/bf16) เสมอ ตรวจสอบ format input ที่เครื่องมือรองรับก่อนจะคิดทางลัดประหยัดพื้นที่ขึ้นมาเอง

การตีความคำสั่งของ PowerShell เองทำให้ \$PATH เพี้ยนและ terminal ที่ใช้งานอยู่พัง

กลาง 2026-07-21

เมื่อ shell environment เริ่มมีการแปลก (PATH, env vars) ให้เปิด window ใหม่แทนการซ่อมของเดิม และห้ามให้ PowerShell มาตีความ syntax '\$VAR' ที่จริงๆ ตั้งใจให้ remote bash shell อ่าน — ต้องใส่ quote หรือเขียนลงไฟล์สคริปต์แทน

สคริปต์ **training** พิมพ์บรรทัดสำเร็จ แต่จริงๆ ยังรัน **sample-generation** อยู่อีก ~15 นาที ทำให้งานที่กำลังทำงานอยู่ดูเหมือนเสร็จแล้ว

กลาง 2026-07-21

ตัดสินใจงานเสร็จแล้วให้ดูจาก shell prompt ที่กลับมา หรือดูจาก `ps` เท่านั้น ห้ามดูจากบรรทัด log และควรวาง banner แสดงความสำเร็จไว้ที่จุดจบจริงของสคริปต์ หลังจากขั้นตอนเสริม/พิเศษทั้งหมดเสร็จแล้ว

พิมพ์คำสั่ง **full-training** ลงใน **terminal** ที่กำลังไม่ว่าง แล้วตกใจว่าทำไม **full_train.log** ไม่มีอยู่จริง

กลาง 2026-07-21

ห้ามตัดสินใจงานเสร็จแล้วจากข้อความที่ขึ้นบนหน้าจอ ต้องรอให้ shell prompt กลับมา หรือตรวจด้วย process check เมื่อไฟล์ log หายไป ให้เช็กก่อนว่าคำสั่งนั้นเคยเริ่มรันจริงไหม ก่อนสรุปว่ามันล้มเหลว

auto-commit ของ **VSCode** กวาดไฟล์ที่แยกไว้กลับมารวมเป็นก้อนเดียว **2.4GB** ระหว่างกำลัง **push**

กลาง 2026-08-01

ปิด auto-commit/auto-stage ของ editor ก่อนจะทำงาน git ด้วยมือทุกครั้ง (เขียน history ใหม่, แยก stage, rebase) IDE ที่เขียน index กลับหลังเรา จะล้าง staging ที่เราจัดไว้เสียๆ

รอบแรกของการเตือนเรื่อง **id** กว้างเกินไป ติดธง **id** ที่ถูกต้อง **1,059** ตัว และทำให้ **205** ไฟล์เต็มไปด้วยสัญญาณรบกวน

กลาง 2026-08-02

ทดสอบเงื่อนไขติดธงกับตัวอย่างก่อน แล้วนั่งอ่านผลที่มันจับได้ ก่อนจะปล่อยรันทั้งคลังข้อมูล ระบบเตือนที่เตือนพร่าเพรื่อจะถูกรบกวน ซึ่งแย่กว่าไม่มีระบบเตือนเลย

กับดักที่รู้อยู่แล้ว: **regex** ที่เขียนไว้แก่ **grid ref** จำนวนมาก ดันไปจับค่า **sheet_code** ด้วย

กลาง 2026-08-02

การแก้ข้อมูลจำนวนมาก ให้จำกัดขอบเขตด้วย path ของช่อง ห้ามใช้รูปแบบของค่า ถ้ามีสองช่องที่ข้อความหน้าตาเหมือนกัน ทำให้ความปลอดภัยมาจากโครงสร้าง ไม่ใช่จากความระมัดระวังของคน

บั๊กใน **skill wrapper** ของ **op01/op03** ที่สร้างไว้วันที่ **08-05** ต้องมาแก้เอาวันที่ **08-09**

กลาง 2026-08-09

สำเนาของเอกสารต้นฉบับต้องมีตัวตรวจว่ามันเพี้ยนไปจากต้นทางหรือยัง ไม่งั้นมันจะแยกทางกันแน่นอน ทางที่ดีกว่าคือให้ skill อ้างอิงไฟล์ต้นฉบับจริงๆ แทนที่จะก๊อปปี้เนื้อหาเอาไว้อีกที

โปรโตไทป์ **overlay** ฮาร์ดโค้ด **ELEMENT_TYPES** ไว้แค่ฐานรากกับเสา ทั้งคันทั้งหมด

กลาง 2026-08-20

ห้ามตรึงรายการชนิดข้อมูลไว้ตามที่ตัวอย่างทดสอบขึ้นเดี๋ยวนี ให้ดึงจาก schema แทน และเลือกตัวอย่างทดสอบที่มีคลาสยากที่สุดอยู่ด้วย ไม่ใช่คลาสที่ง่ายที่สุด

reportlab จัดรูปตัวอักษรไทยไม่ได้ ต้องรื้อการสร้าง **PDF** ไปใช้ **Edge** แบบ **headless** แทน

กลาง 2026-08-21

งานออกเอกสารภาษาไทย (หรือภาษาที่มีการประกอบตัวอักษรซับซ้อน) ต้องใช้ตัวจัดรูปตัวอักษรจริง คือ renderer ของเบราว์เซอร์ และให้ตรวจสอบด้วยการดึงข้อความกลับออกมาจาก **PDF** ที่ได้ เพราะไลบรารีที่หึงการจัดรูปเสียๆ จะให้ผลลัพธ์ที่รู้าพังก็ต่อเมื่อเปิดดูด้วยตาเท่านั้น

agent ของ **op04** ชน **limit** ของ **API session** แล้วทิ้งไฟล์ **GT** ค้างครั่งๆ กลางๆ ระหว่างเขียน

กลาง 2026-08-24

เขียนไฟล์ **GT** แบบ **atomic** (เขียนไฟล์ชั่วคราวแล้วค่อยเปลี่ยนชื่อ) และบันทึกสถานะเสร็จรายไฟล์ ตัวสร้าง **dataset** ต้องข้ามไฟล์ที่เสียแล้วรายงาน ห้ามเดาเนื้อหาซ่อมให้เอง

การตรวจจับเสากับคานปล่อยออกไปโดยมี **template** แคตัวเดียวต่อประเภท ตัวหนึ่ง **crop** มือ อีกตัวสังเคราะห์ด้วยโค้ด ไม่เคยผ่านการเก็บจริงเลย

กลาง 2026-08-25

ถ้า **detector** หลายคลาสมีคลาสหนึ่งที่ตรวจสอบมาหนักกว่าคลาสอื่นมาก ให้บอกไว้ข้างตัวเลขด้วย ไม่งั้นคลาสเดียวที่ทดสอบมาดีจะไปสร้างความน่าเชื่อถือปลอมๆ ให้คลาสที่เหลือ

Unsloth ใช้กับ InternVL3-78B ไม่ได้ ส่วน Llama-4-Scout บน Unsloth ทำได้แค่ 4-bit

กลาง 2026-08-26

จดไว้ทุกโมเดลที่เป็นตัวเลือกกว่า framework เทรนตัวไหนใช้ได้ และได้ที่ precision เท่าไร framework ที่รองรับแค่ 4-bit เป็นสัญญาณอันตรายสำหรับงาน vision ทุกชนิด เพราะข้อจำกัดนั้นแหละที่ทำให้ vision encoder ตาบอด

คลังเทมเพลตสายงูชาติอีก 13 หลัง — ช่องว่างย่อยและพื้นที่คลาสยังไม่มีใครครอบคลุม

กลาง 2026-08-28

ตัวตรวจจับที่เรียนรู้เองไม่มีทางเจอชุดที่มันไม่เคยเห็นมาก่อน ต้องกันเวลาไว้ป้อนตัวอย่างเริ่มต้นด้วยมือสำหรับชุดที่ยังไม่ครอบคลุม และรายงานความครอบคลุมเป็นรายหลัง เพื่อให้ "ไม่มีค่าใบ้เลย" มองเห็นได้ ไม่ใช่เสียบหายไป

subagent ชน spend limit ทั้ง session — งานสัปดาห์ op04 ต้องทำมือทั้งหมด

กลาง 2026-08-29

เช็คนกบ/โควตาก่อนตัดสินใจลงงานที่ใช้ agent ขนานกันหลายตัว และเขียนแผนสำรองแบบทำทีละตัวพร้อมเวลาที่ต้องใช้ไว้ล่วงหน้าเสมอ

คลังเทมเพลต CV สร้างจากบ้าน 40 หลังนี้เท่านั้น — ลูกค้าใหม่ที่เขียนแบบคนละสำนักอาจไม่แมตช์อะไรเลย และการเก็บข้อมูลเองก็เริ่มไม่ได้ถ้าไม่มีเมสิดตั้งต้น

กลาง 2026-08-30

ระบบจับรูปแบบ/เทมเพลตที่เทรนจากข้อมูลภายในต้องมีเส้นทางเริ่มต้นสำหรับธรรมเนียมที่ไม่เคยเจอ ออกแบบ UX สำหรับการป้อนเมสิดพร้อมกับตัวจับคู่ ไม่ใช่ค่อยมาทำทีหลัง

สคริปต์ตรวจที่เขียนเองอ่านชื่อไฟล์ผิด ผิดแล้วสรุปว่ากล้อง CV ไม่มีพิกัด

กลาง 2026-08-31

เมื่อตัวตรวจรายงานว่า 'ข้อมูลขาด X' ให้พิสูจน์ก่อนว่าตัวตรวจอ่านไฟล์ถูกกับข้อมูลที่รู้ว่าได้อยู่แล้ว — ตัวตรวจที่พึ่งหน้าตาเหมือนข้อมูลที่พึ่งเป๊ะ

ลูป poll สถานะ instance เรียก python3 ซึ่งบน Windows เป็น alias ของ Store ที่ใช้ไม่ได้ มันเลยวนไม่จบ

กลาง 2026-08-31

พิสูจน์ว่ารอบแรกของลูป poll ได้ข้อมูลจริงก่อนจะเดินจากไป และห้ามคิดว่า python3 มีอยู่บน shell ของ Windows

กับดัก Windows สองอัน: ไฟล์ manifest ที่เขียนด้วย Python ได้ CRLF และ tar อ่าน D:/ เป็นชื่อเครื่องปลายทาง

กลาง 2026-08-31

บน Windows ให้เขียนไฟล์ manifest ที่ใช้ข้ามระบบด้วย newline='\n' เสมอ และใช้ path แบบ MSYS (/d/) กับ tar ห้ามใช้ path ที่ขึ้นต้นด้วยตัวอักษรใดرف

ค่าเตือนเรื่อง visual token เทียบกับเพดานรวมของทั้งระบบ เลยแจ้งเตือนทุกครั้งที่รูปมันเล็กแบบถูกต้องอยู่แล้ว

กลาง 2026-08-31

เกณฑ์ของสัญญาณเตือนต้องคำนวณต่อรายการจากสิ่งที่รายการนั้นควรจะเป็นจริงๆ ไม่ใช่ใช้ค่าคงที่กลางเป็นฐาน เพราะแบบนั้นตัวตรวจจะกลายเป็นแค่ noise

PEFT's add_weighted_adapter ใช้ merge adapter แบบ MoE ที่มี target_parameters ไม่ได้

กลาง 2026-08-31

ต้องตรวจสอบก่อนว่า API ของ library ที่จะใช้รองรับสถาปัตยกรรมของเราจริงไหม ก่อนออกแบบขั้นตอนสุดท้ายของ pipeline ที่ใช้ต้นทุนสูง

ถ้าตั้งชื่อ adapter ที่ merge แล้วว่า 'destrier' มันจะซ่อนอยู่ในโฟลเดอร์ย่อยที่ไม่มีไฟล์อยู่ที่ root repo เลย

กลาง 2026-08-31

ต้องเช็คก่อนว่า serialization library เอาชื่อไปแปลงเป็น path ยังไง ก่อนตั้งชื่อ และต้องตรวจสอบโครงสร้างไฟล์จริงของสิ่งที่ publish ออกไป ไม่ใช่แค่เช็ค upload สำเร็จ

template เช่าเครื่องตัวใหม่ไม่มี /workspace และ pip เปลาๆ ก็ลงผิด Python

กลาง 2026-08-31

ทุกครั้งที่ได้ template เช่าเครื่องใหม่ ต้องเช็คก่อนว่าโฟลเดอร์ทำงานมีอยู่จริง และหา absolute path ของ Python ที่ผูกกับ torch ให้เจอก่อนติดตั้งอะไรทั้งนั้น ห้ามใช้ pip/python เปลาๆ โดยไม่เช็ค

presentation_state.json ยังเก็บ instance id เก่าค้างไว้ ทั้งที่คืนการดจออกไปแล้ว

กลาง 2026-08-31

ต้องเทียบข้อมูลที่บ้านทีกไว้ในเครื่องกับสถานะจริงของผู้ให้บริการ cloud ทุกครั้งที่อ่านค่า และให้เอนเอียงไปทาง 'เตือนไว้ก่อน' เวลาติดต่อผู้ให้บริการไม่ได้

สคริปต์ merge จะอัปโหลด source fold adapter ทั้ง 4 ตัวติดไปพร้อมกับ adapter ที่ merge แล้วด้วย

กลาง 2026-08-31

ก่อน push โมเดล ต้องไล่เช็คให้ครบว่ามีอะไรติดมาด้วยจริงๆ — การ push จะอัปโหลดตามสถานะจริงที่มี ไม่ใช่ตามเจตนาที่เราคิดไว้

วางคำสั่ง git config สองคำสั่งติดกันโดยไม่มีตัวค้น ทำให้ได้ error 'no action specified'

ตา 2026-07-06

เวลาส่งชุดคำสั่งหลายบรรทัดให้ agent ให้แยกคนละบรรทัด หรือใส่ ';' เชื่อมให้ชัดเจน และยังคงกฎเดิมไว้ว่า agent ต้องอธิบาย git config ไม่ใช่รันเอง — ในเคสนี้ safety protocol ทำงานถูกต้องแล้ว

flag `scp -P` กับ `ssh -p` ตัวพิมพ์ต่างกัน ทำให้อัปโหลดไฟล์ล้มเหลว

ตา 2026-07-21

พอลำดับที่ถูกต้อง (พร้อม flag) ทำงานสำเร็จครั้งแรก ให้จดลง runbook ทันทึ และอย่าสมมติว่าเครื่องมือ CLI สองตัวที่คล้ายกันจะใช้ตัวพิมพ์ flag เหมือนกัน

สมมติเอาเองว่า Unsloth ใช้รันโมเดลจริงในเครื่องได้เลย ทั้งที่ต้องใช้โมเดลเต็ม 70GB และการดจอ 74GB ขึ้นไป

ตา 2026-07-24

เครื่องมือสำหรับตอนเทรนกับเครื่องมือสำหรับตอนใช้งานจริง (inference) เป็นคนละโปรแกรมกัน มีข้อกำหนดฮาร์ดแวร์คนละแบบ ต้องยืนยันก่อนว่าเครื่องมือไหนรับผิชอบขั้นตอนไหน ก่อนวางแผนตาม

คลังคำศัพท์ pattern มีอยู่สองที่พร้อมกัน — พอขยาย spec โดยไม่แก้ checker คู่กัน ไฟล์ใหม่ทุกไฟล์จะพังทันที

ตา 2026-08-04

ให้คำนวณจำนวนออกมาจากลิสต์จริง แทนที่จะเขียนตัวเลขไว้ในข้อความ และให้ถือว่าค่าคงที่ใน checker เป็นส่วนหนึ่งของ spec ด้วย — เปลี่ยนทีเดียว คอมมิตเดียวจบ ไม่งั้น gate จะพังแม้ข้อมูลจะถูกต้องแล้วก็ตาม

ปุ่ม toggle 'คลิกเพื่อยอมรับ' บน overlay แค console.log อย่างเดียว — ไม่ได้บันทึกอะไรเลยจริงๆ

ตา 2026-08-20

UI ที่ทิ้งผลลัพธ์ของตัวเองไปเปลาๆ แย่ยิ่งกว่าไม่มี UI นั้นเลย ต้องบันทึกการตัดสินใจจริงๆ หรือไม่ก็ปิดใช้งานปุ่มนั้นไปก่อนจนกว่า write path จะพร้อม

ภาพ overlay ของบ้าน 01 ซึ่งเป็นหลักฐานที่แข็งที่สุดของสัปดาห์นั้น กลับเป็นแค่สคริปต์ชั่วคราวที่ไม่เคยเข้า repo

ตา 2026-08-20

พอผลลัพธ์จากสคริปต์ถูกยกมาอ้างเป็นหลักฐาน สคริปต์นั้นก็กลายเป็น deliverable แล้ว ต้อง commit ไว้ การมานั่ง calibrate ซ้ำสามรอบเสียเวลากว่าแค่ `git add` บรรทัดเดียวเยอะ

ข้าม overlay หน้า 28 ของบ้าน 09 เพราะวงกลม grid บนแบบพิมพ์ไม่ครบ calibrate ไม่ได้

ตา 2026-08-21

วิธี calibrate ที่พึ่งพาลักษณะบนแบบพิมพ์ต้องมีทางสำรองที่เขียนไว้ชัดเจน (calibrate มือแบบสองจุด หรือยืมสเกลจากแผ่นฟีน้องกัน) ไม่งั้นแผ่นที่ควรตรวจสอบมากที่สุดจะกลายเป็นแผ่นที่ตรวจไม่ได้เลย

ไม่ได้ติดตั้ง graphify CLI ไว้ — ขั้นตอนในเวิร์กโฟลว์ที่บันทึกไว้ถูกข้ามไปแบบเงียบๆ

ต้า 2026-08-26

เวิร์กโฟลว์ที่ต้องพึ่ง CLI เฉพาะเครื่อง ต้องเขียนวิธีติดตั้งไว้และเช็คว่ามีอยู่จริง ไม่ใช่ปล่อยให้ขึ้น 'command not found' แล้วข้ามไปเฉยๆ ควรใช้เครื่องมือที่ checked-in ไว้ใน repo แทนการพึ่ง PATH เฉพาะเครื่องของแต่ละคน

PaddleOCR ต้องแก้ปัญหาสภาพแวดล้อมสองอย่างก่อนถึงจะรันได้เลย

ต้า 2026-08-29

ให้บันทึกวิธีแก้ปัญหาสภาพแวดล้อมของเครื่องมือ vendor ลงใน runbook ทันทีที่เจอ — คนถัดไปที่ใช้เครื่องอื่นจะเจอกำแพงเดียวกันสองอันนี้ และต้องมานั่งไล่หาทางแก้เองใหม่หมด

อาร์กิวเมนต์ --split ของ measure_capacity.py ถูกสืบทอดไว้เป็น choices ตายตัว ทำให้วัดไฟล์ fold ไม่ได้

ต้า 2026-08-31

อย่า hard-code รายการ choices สำหรับส่วนหนึ่งของชื่อไฟล์ — มันจะทำให้การเปลี่ยนโครงสร้างข้อมูลกลายเป็นเครื่องมือใช้งานไม่ได้ทันทีที่ต้องการวัดผลจริงๆ

การเทรนโมเดล และ วิธีทำ ML

การเทรน แบ่งข้อมูล prompt รวมโมเดล และการเรียกใช้โมเดล ที่ไม่ได้ทำอย่างที่ตั้งใจ

ทั้งหมด 75 เรื่อง · วิกฤต 18 · สูง 34 · กลาง 21 · ต่ำ 2

เรื่องที่ต้องอ่านให้ครบ (52)

วิกฤต 2026-07-24

MAX_LENGTH ที่คิดจากตัวอย่างรูปเดียว จะตัด label ของตัวอย่าง gridmaster ทุกตัวทิ้ง

เกิดอะไรขึ้น: `MAX\_LENGTH=9216` ถูกคำนวณจากตัวอย่างที่มีรูปเดียว แต่ตัวอย่าง gridmaster หักหัว (บ้านละหนึ่งตัว) ใช้รูป 2-4 รูป และต้องการประมาณ 22,067 token จริงๆ ถ้าไม่แก้ ตัวเทรนจะตัด label ของตัวอย่างพวกนั้นทิ้งไปทั้งหมด

ต้นเหตุจริง: งบประมาณยาวลำดับถูกคำนวณจากตัวอย่างค่ากลาง ไม่ใช่ค่าสูงสุด และตอนตั้งค่าก็ไม่ได้นึกถึงตัวอย่างที่มีหลายรูป

เสียหายอะไร: จับได้ตอนอ่านทบทวนใน Phase 0.4 ก่อนจะเข้า GPU จริง ถ้าปล่อยผ่านไป ตัวอย่างที่สำคัญที่สุดเชิงโครงสร้างหัวตัวจะถูกเทรนด้วยลำดับที่ถูกตัดจน label หายไป โดยไม่มี error ขึ้นเลย

แก้ยังไง: เพิ่ม `MAX\_LENGTH` เป็น 24576 ยอมรับความเสี่ยง OOM ที่สูงขึ้นในตัวอย่าง 5 ตัวนั้น แทนที่จะลดความละเอียดของรูป

กฎที่ได้: ตั้งงบประมาณยาวลำดับจากตัวอย่างที่ยาวที่สุด ไม่ใช่ค่ากลาง และวัดแยกตามประเภทของตัวอย่างด้วย (รูปเดียว vs หลายรูป) การตัด label แบบเสียๆ เป็นบั๊กตอนเทรนที่แย่ที่สุด เพราะไม่มี error ขึ้น แต่ได้โมเดลที่แย่ง

วิกฤต 2026-07-24

JSON ที่ถูกต้องเหลือ 0% เพราะ chat template ตอน inference ปิดโหมด reasoning ที่ตอนเทรนไม่เคยใช้

เกิดอะไรขึ้น: การประเมินรอบแรกใน Phase 7 ได้คะแนน JSON ถูกต้อง 0% — แยกว่าโมเดลตั้งต้นที่ยังไม่ได้จูนอีก พอไปดูพบว่าโมเดลเขียนคำอธิบายความคิดยาวๆ นำหน้าทุกคำตอบ กินงบประมาณ token จนไปไม่ถึง JSON เลย

ต้นเหตุจริง: เรียก `apply\_chat\_template(add\_generation\_prompt=True)` โดยไม่ใส่ `enable\_thinking=False` ตอนเทรนใช้ `add\_generation\_prompt=False` และไม่เคยใส่แท็ก `` เลย แต่ตอน inference ค่าเริ่มต้นใส่ `\n` เปล่าๆ เข้าไป ทำให้โมเดลเข้าใจผิดคิดว่าวิเคราะห์ สรุปคือ template ตอน inference ไม่ตรงกับตอนเทรน

เสียหายอะไร: ผลลัพธ์ออกมาดูเหมือนการ fine-tune ทำโมเดลพัง แต่หลังแก้แล้ว ตัวเลขจริงคือ JSON ถูกต้อง 90%, view ตรงเป้า 70% (เดิม 60%) และ element recall 28.2% (เดิม 9.4% ดีขึ้น 3 เท่า) — จริงๆ แล้วการ fine-tune ได้ผลมาตลอด

แก้ยังไง: ใส่ `enable\_thinking=False` ในทั้งสามไฟล์ (`eval\_fields.py`, `train\_qwen36.py`, `export\_gguf.py`) แล้วรันใหม่ผ่านทันที

กฎที่ได้: chat template ตอน inference ต้องทำงานเหมือนตอนเทรนทุกไบต์ — เอามา diff กันตรงๆ เป็นข้อหนึ่งใน checklist เลย และถ้าตัวเลข eval ออกมาแยแบบหายๆ มักจะเป็นเพราะชุดทดสอบตั้งค่าไม่ตรงกันมากกว่าโมเดลพัง ตรวจสอบชุดทดสอบก่อนสรุปว่าเทรนล้มเหลว

เกือบเอาผลลัพธ์ของโมเดลที่แม่นแค่ 28% ป้อนกลับเข้าโพลเดอร์ที่ใช้เป็น ground truth สำหรับเทรน

เกิดอะไรขึ้น: มีคนตั้งคำถามว่าจะใช้โมเดลที่จนแล้วสร้าง raw JSON ของ `บ้าน\_ใหญ่\_1ชั้น\_01` ลงในโพลเดอร์ที่ใช้งานจริงเลยไหม แต่ `rawjson` ยังไม่ได้แก้ไขโดนคน/ คือ ground truth ที่ป้อนเข้าไปเทรน การเอาผลลัพธ์ที่แม่นแค่ 28% และยังไม่ผ่านการตรวจมาใส่ตรงนั้นเท่ากับวางยาตัวเอง

ต้นเหตุจริง: ยังไม่ได้ตัดสินใจล่วงหน้าว่าร่างที่ AI สร้างจะไปอยู่โพลเดอร์ไหน และโพลเดอร์ ground truth ก็เป็นปลายทางที่นึกถึงเป็นอันดับแรกอยู่แล้วเวลาพูดว่า "raw JSON ของบ้านหลังหนึ่ง"

เสียหายอะไร: จับได้ก่อนเกิดขึ้นจริง ถ้าปล่อยไป ชุดข้อมูลเทรนจะปนเปื้อนด้วยข้อผิดพลาดของโมเดลเองที่แม่นแค่ 28% เกิดวงจรป้อนกลับที่ทำให้หุ้รอบกั้ตไปแย่งเรื่อยๆ

แก้ยังง: ตกลงกันว่าผลลัพธ์เป็นแค่ร่างที่รอคนตรวจ ให้เขียนลงโพลเดอร์แยกต่างหาก ห้ามเขียนลงโพลเดอร์ ground truth เด็ดขาด

กฎที่ได้: ห้ามเขียนผลลัพธ์ของโมเดลลงในโพลเดอร์ที่ใช้เทรนโมเดลนั่นเอง บังคับด้วยการแยก path และถ้าเป็นไปได้ให้ใส่ตัวกันการเขียนไว้ด้วย — พลาดเซฟผิดที่ครั้งเดียว ปนเปื้อนหุ้รอบเทรนหลังจากนั้นทั้งหมด

merge_and_unload() สัมเหลวเจียบๆ กับ LoRA แบบ MoE ParamWrapper — ค่าใน tensor เปลี่ยนแค่ 0.67%

เกิดอะไรขึ้น: ตอน merge LoRA adapter เข้าโมเดลตั้งต้นก่อนแปลงเป็น GGUF พังก์ชันทำงานจนจบโดยไม่มี error แต่พอวัดด้วยมีอบน tensor ที่มี 201 ล้านค่า พบว่ามีค่าที่เปลี่ยนไปแค่ 0.67% — แปลว่าโมเดลที่ merge แล้วมีพฤติกรรมเกือบเหมือนโมเดลตั้งต้นที่ยังไม่ได้จนเลย เรื่องนี้ทำให้ GGUF ของ t02 ติดขัด และสงสัยอย่างหนักว่าน่าจะทำให้ GGUF ของ t01 เสียเจียบๆ ไปด้วย

ต้นเหตุจริง: กลไก ParamWrapper ของ peft ซึ่งเป็นวิธีที่ LoRA เกาะเข้ากับ experts ของ MoE ไม่ได้ถูกจัดการอย่างถูกต้องโดย merge_and_unload() ความล้มเหลวแบบนี้คือโครงสร้างผ่านแต่พฤติกรรมพัง — ไม่มี exception รูปร่างข้อมูลถูกต้อง แต่เนื้อในผิด

เสียหายอะไร: หุ้การตัดสินใจที่ตามมาจาก GGUF ของ t01 กลายเป็นเรื่องน่าสงสั้ยหมด บั้กนี้ตอนนีขวางทั้งสองรอบการเทรนไม่ใช่แค่ t02 และเป็นไปได้ว่าไฟล์ที่จะเอาไปใช้งานจริงไม่มีผลของการ fine-tune อยู่เลยแม้แต่นิดเดียว

แก้ยังง: ยังไม่ได้แก้ ออกแบบการทดสอบฟรีที่ใช้เวลาประมาณ 20 นาทีไว้แล้ว (รันหน้า 03 ด้วย GGUF ตั้งต้นที่ยังไม่จน แล้วเทียบกับผลของ t01) เพื่อยืนยัน และต้องรันก่อนจะจ่ายเงินเพิ่มอีก

กฎที่ได้: หลังทำขั้นตอน merge/convert/quantize หุ้ครั้ง ให้ตรวจสอบด้วยการวัดค่าน้ำหนัก (ก็เปอร์เซ็นต์ของค่าที่เปลี่ยนไป) และวัดพฤติกรรมกับตัวอย่างที่กันไว้ — อยายอมรับ "มันรันจบโดยไม่มี error" เป็นหลักฐานว่าการแปลงมีผลจริง

วิกฤต 2026-07-29

0 จาก 6 หน้าที่ใช้ schema ที่เทรนไว้ — พนชื่อ key ออกมา 3 แบบสลับกันมั่วๆ

เกิดอะไรขึ้น: ข้อมูลจริงใช้ png / doc_page / discipline / sheet_code แต่โมเดลพ่นออกมาเป็น 'page', 'page_number' และ 'page_number'+ 'sheet_id' สลับกันไปทั้ง 6 หน้า รวมสามรูปแบบ ไม่ตรงกับข้อมูลจริงสักแบบ ทั้งที่หน้า 03-06 เป็นข้อมูลเทรนของมันเองที่เห็นมาแล้ว 3 epoch

ต้นเหตุจริง: format คือสิ่งที่ fine-tune เรียนรู้ได้เร็วที่สุด การที่มันทำ format พังบนหน้าที่ท้อมมาแล้ว แต่กลับทำเนื้อหาๆ ได้ถูก คือลายนิ้วมือของโมเดลที่ไม่เคยได้รับการ tune เลย — นั่นคือการ merge สัมแบบเสียบๆ นั่นเอง

เสียหายอะไร: output นี้ระบบปลายทางที่คาดหวังสัญญาณรูปแบบข้อมูลจะอ่านไม่ได้เลย แต่ที่สำคัญกว่านั้นคือความผิดพลาดนี้เองที่เปิดโปงต้นเหตุจริงออกมา

แก้ยังไม: ยังไม่แก้ — จะหายก็ต่อเมื่อปัญหา LoRA/merge ถูกแก้

กฎที่ได้: ให้ดูรูปทรงของความผิดพลาด ไม่ใช่ดูอัตราความผิดพลาด อาการ 'ผิดในเรื่องง่าย แต่ถูกในเรื่องยาก' คือทองคำสำหรับการวินิจฉัย ซึ่งคะแนนรวมแบบภาพรวมจะกลบมันมิดสนิท

วิกฤต 2026-07-29

output ของโมเดลตรงกับ schema ที่เทรนไว้ 0 จาก 6 หน้า และใช้ชื่อ key ถึง 3 สไลด์

เกิดอะไรขึ้น: ไม่มีสักหน้าเดียวจาก 6 หน้าที่ใช้ key ตามข้อมูลจริง (png / doc_page / discipline / sheet_code) โมเดลใช้ "page", "page_number" และ "page_number"+"sheet_id" สลับกันไปมาในแต่ละหน้าอย่างไม่มี ความสม่ำเสมอ

ต้นเหตุจริง: เป็นหลักฐานหนักแน่นว่าการ fine-tune ไม่ได้อยู่ในของที่เรีนอยู่จริง เพราะ format ของ output คือสิ่งแรกที่ fine-tune ล็อกได้ ภายในไม่กี่ร้อย step โมเดลที่ไม่ได้ tune แล้วได้รับ prompt ที่บรรยาย schema จะมีพฤติกรรมแบบนี้เป๊ะ — มันทำตามคำศัพท์ใน prompt และไม่รู้จัก key ที่มีอยู่แต่ในคำตอบสำหรับเทรน

เสียหายอะไร: ทุกไฟล์ที่สกัดออกมาใช้งานปลายทางไม่ได้ถ้าไม่แปลงชื่อ key ก่อน และความไม่สม่ำเสมอระหว่างหน้าทำให้แม้แต่สคริปต์แปลงก็ยังเชื่อถือไม่ได้

แก้ยังไม: ยังไม่แก้ ข้อสังเกตนี้กลายเป็นเบาะแสหลักที่ชี้ไปยังสมมติฐานว่า LoRA หายไป

กฎที่ได้: การที่ output ทำตาม format ได้ถูก คือสัญญาณที่ถูกที่สุดและเร็วที่สุดว่า fine-tune ถูกโหลดเข้ามาจริง — ให้เช็คตั้งแต่หน้าแรก และถ้า format พังทั้งที่เนื้อหาดี ให้ถือว่าเป็นบักของการ deploy ไม่ใช่ปัญหาคุณภาพโมเดล

วิกฤต 2026-07-29

โมเดลเห็นภาพหลอน พนขึ้นส่วนออกมา 59 ขึ้น ในหน้าที่ข้อมูลจริงมีแค่ 17

เกิดอะไรขึ้น: ในหน้า 06 โมเดลพ่น element ออกมา 59 ขึ้น เทียบกับข้อมูลจริงที่มี 17 ขึ้น โดยแต่งเสาชั้นมาเอง 12 ต้น ประตุ 8 บาน และหน้าต่าง 4 บาน ซึ่งไม่มีอยู่ในแบบแผนนั้นเลย

ต้นเหตุจริง: เมื่อไม่มี fine-tune ทำงานอยู่ (ตามที่สงสัย) โมเดลจะเติม schema ที่ถูกขอมาด้วยรายการที่ดูสมเหตุสมผล แทนที่จะจำกัดตัวเองอยู่แค่สิ่งที่มียูบนหน้านั้นจริงๆ กฎ 'ใช้ null ห้ามเดา' ใน prompt เป็นแค่คำสั่งที่พูดออกไป ไม่ได้ถูกเรียนรู้

เสียหายอะไร: การถอดปริมาณงานอะไรก็ตามที่สร้างบน output นี้จะสูงเกินจริงไปมหาศาล และไฟล์ที่สกัดของบ้าน 01 ก็ปนเปื้อนไปแล้ว

แก้ยังไม: ยังไม่แก้ การสกัดข้อมูลบ้าน 01 ถูกหยุดไว้ก่อนจนกว่าคำถามเรื่อง LoRA หายจะได้คำตอบ

กฎที่ได้: ใส่การตรวจนับจำนวน element เทียบกับข้อมูลจริง (หรือเทียบกับการรันรอบสอง) ก่อนจะยอมรับผลการสกัดข้อมูล การพ่นเกินอันตรายนพๆ กับการตกหล่น และตรวจจับด้วยเครื่องได้ง่ายกว่าการนั่งไล่ดูด้วยตา

วิกฤต 2026-07-29

โมเดลกลับหัวกลับหางตัวอักษรของเส้นกริด และแต่งระยะห่างภายในขึ้นมาเองทั้งหมด บังเอิญได้แค่ระยะรวมที่ตรง

เกิดอะไรขึ้น: ข้อมูลจริงของกริดคือ D=0, C=4.0, B=6.0, A=9.5 แต่โมเดลตอบ A=0, B=1.5, C=5.0, D=9.5 — ตัวอักษรกลับด้าน ระยะห่างภายในผิดทุกช่วง มีแค่ผลรวม 9.5 ที่ถูก และยังพ่น grid_table ที่ให้ค่าหนึ่งค่าต่อจุดตัดกริดหนึ่งจุด ซึ่งเป็นปริมาณที่ไม่มีอยู่จริงในทางกายภาพ

ต้นเหตุจริง: ตัวเลขจริงจากแบบ (7.0, 9.5, 4.0, 3.5) ถูกสลับเปลี่ยนลงช่องต่างๆ แทนที่จะอ่านตามตำแหน่งจริง โมเดลสร้างตารางที่ดูสมเหตุสมผลขึ้นมาแทนที่จะยอมรับว่ามันแก้เรขาคณิตตรงนี้ไม่ได้

เสียหายอะไร: prompt กำหนดชัดว่าช่วง (span) ต้องมาจากตารางกริด ดังนั้นการคำนวณช่วงทุกอย่างที่อยู่ถัดจากนี้จึงผิดหมด ซึ่งเท่ากับทำลายปริมาณงานโครงสร้างที่เป็นเหตุผลของการมีอยู่ของทั้งโปรเจกต์

แก้ยังไง: ยังไม่แก้ ตั้งธงไว้ว่าเป็นข้อบกพร่องของ output ระดับรุนแรงสูงสุด

กฎที่ได้: ต้องตรวจ output ของกริดเชิงโครงสร้างก่อน (พิกัดต้องเรียงทางเดียว ตัวอักษรต้องเรียงตามแบบ ผลรวมสะสมต้องสอดคล้องกัน) ก่อนจะเอาไปคำนวณช่วงใดๆ — ผลรวมที่ถูกไม่ใช่หลักฐานว่าระยะห่างถูก

วิกฤต 2026-08-04

การเทรนรอบ t02 เสียเปล่าทั้งรอบ — Qwen3-VL-30B-A3B export เป็น GGUF ไม่ได้ แคมแพท t01 ทุกตัวชี้วัด

เกิดอะไรขึ้น: workflow ของ t02 ถูกตีตรา  SUPERSEDED: Qwen3-VL-30B-A3B export เป็น GGUF ไม่ได้เลย (เป็นบั๊กตอน merge MoE ใน `ParamWrapper.merge()` ของ MoE experts) และโมเดลที่ได้ก็แพท t01 ทุกตัวเลขที่วัด สุดท้ายทั้งสายนี้แล้วถอยกลับไปใช้ Qwen3.6

ต้นเหตุจริง: เลือกโมเดลแล้วเทรนไปเลย ทั้งที่ยังไม่เคยพิสูจน์เส้นทางใช้งานจริงครบวง (merge → export GGUF → รัน inference บนเครื่องนี้) จุดที่ตันอยู่ตรงขั้น export ซึ่งกว่าจะรู้ก็เทรนเสร็จไปแล้ว

เสียหายอะไร: การ fine-tune หนึ่งรอบเต็ม — ทั้งค่าเช่า GPU ทั้งเวลาสร้าง dataset ทั้งเวลาเทรน — ได้โมเดลที่เอาไปใช้ไม่ได้และแย่กว่าตัวเดิม ขาดทุนล้วนๆ

แก้ยังไง: ตีตรา t02 เป็น SUPERSEDED พร้อมบันทึกเหตุผลไว้ เก็บไฟล์ไว้เป็นหลักฐานว่าทำไม แล้วกลับไปใช้ Qwen3.6-35B-A3B ที่เส้นทาง merge/GGUF พิสูจน์แล้วว่าใช้ได้จริงบนเครื่องนี้

กฎที่ได้: พิสูจน์เส้นทางใช้งานจริงให้ครบ (merge → export → รันบนเครื่องจริง) กับโมเดลตัวตั้งต้น ****ก่อน**** ที่จะหุ้มเทรนหนึ่งรอบลงไป โมเดลที่ export ไม่ได้ = มีค่าเป็นศูนย์ ไม่ว่าจะคะแนนจะสวยแค่ไหน

เทรนแบบหน้าเดียวจบในที่เดียว แต่ของจริงรัน 5 pass ข้ามหน้า — โมเดลที่ tune มาแทนได้อย่างมากแค่ pass เดียว

เกิดอะไรขึ้น: ไล่ดูตั้งแต่ต้นจนจบแล้วพบว่า การเทรนสอนงานแบบ "หนึ่งภาพ → JSON ครบของหน้านั้น จบในที่เดียว" แต่ pipeline ที่ใช้งานจริงเป็นแบบ 5 pass และต้องอ้างอิงข้ามหน้า โมเดลที่ tune มาไม่เคยถูกสอนให้เชื่อมโยงข้ามหน้าเลย ดีที่สุดคือแทนได้แค่ Pass 1 ไม่ใช่ทั้ง pipeline แถมยังทำให้ค่า element recall ถูกจำกัดโดยโครงสร้างด้วย เพราะ ground truth สร้างจากหลายแผ่นแต่โมเดลเห็นแค่แผ่นเดียว

ต้นเหตุจริง: ออกแบบรูปแบบงานเทรนจากรูปแบบของข้อมูล (หนึ่งไฟล์ต่อหนึ่งหน้า) แทนที่จะออกแบบจากงานจริงที่มันต้องไปทำ และไม่มีใครเอา input/output ของการเทรนไปเทียบกับ pass ของ production ก่อนเริ่มเทรนเลย

เสียหายอะไร: fine-tune ไปสองรอบเต็ม (t01, t02) ได้โมเดลที่ไม่รู้จะเอาไปเสียบตรงไหน แถมตัวเลข recall ที่เป็นตัวชูโรงก็เป็นค่าที่โครงสร้างงานทำให้ไปไม่ถึงอยู่แล้ว ตัวเลขจึงไม่ใช่แค่ต่ำ แต่หลอกด้วย

แก้งัง: ตั้งเป็นกฎตายตัวว่าแผ่น t03 ต้องระบุให้ชัดก่อนเริ่มว่าจะไปแทน pass ไหน แล้วออกแบบ t03 เป็นสถาปัตยกรรม 2 pass ที่แมปตรงกับ pass จริงแบบ 1:1

กฎที่ได้: ก่อนเทรน ต้องบอกให้ได้ก่อนว่าโมเดลจะไปแทนขั้นตอนไหนของงานจริง แล้วทำ input/output ตอนเทรนให้เหมือนขั้นตอนนั้นเป๊ะ รอบที่วัดผลสลายแต่เอาไปใช้ที่ไหนไม่ได้ = รอบที่เสียเปล่า และตัวชี้วัดที่รูปแบบงานทำให้ไปไม่ถึง = ตัวชี้วัดที่พัง

Pass 3 ของงานจริงสั่งให้โมเดลกระยะช่วงเอาเอง — ขัดกับกฎ §4 ของ schema ตรงๆ

เกิดอะไรขึ้น: schema §4 เขียนไว้ว่าระยะช่วง "ให้โค้ดคำนวณจากกริดเท่านั้น ห้ามให้โมเดลประมาณระยะเด็ดขาด" แต่ `qt\_buildCountPrompt` กลับสั่งให้โมเดลจัดกลุ่ม element "ตามความยาวช่วง (เมตร)" ซึ่งก็คือให้ระยะจากภาพนั่นเอง

ต้นเหตุจริง: prompt ที่ใช้งานจริงกับ schema ของการสกัดข้อมูลเขียนกันคนละสาย ไม่มีใครเอามาตรวจไขว้กัน และกฎใน schema ก็ไม่เคยถูกบังคับใช้กับฝั่ง prompt เลย

เสียหายอะไร: ความยาวช่วง ซึ่งไหลตรงเข้าไปคิดปริมาตรคอนกรีต ความยาวเหล็ก และราคา กลายเป็นค่าที่ได้จากโมเดลเพ่งพิศเซลเอา ไม่ได้มาจากตารางกริด ปริมาณที่ผิดจึงไหลลงไปทุกการคำนวณข้างล่าง

แก้งัง: บันทึกไว้ว่าเป็นความขัดแย้งกันเอง แล้วสถาปัตยกรรมของ t03 ย้ายการคำนวณระยะช่วงไปอยู่ในโค้ดล้วนๆ (ชั้น merge) ไม่มี AI เข้ามาเกี่ยวเลย

กฎที่ได้: ตรวจ prompt เทียบกับสเปกข้อมูลด้วย ไม่ใช่ตรวจแค่โค้ดเทียบสเปก prompt ก็คือการนำกฎไปใช้อย่างหนึ่ง — prompt ที่ขอค่าที่สเปกห้ามโมเดลผลิต ก็คือการละเมิดสเปก

รูปแบบ output ของ Pass 1 เก็บเหลืบน/ล่างไม่ได้ — ต่อให้อ่านถูกเป๊ะก็โดนบีแบนตอนเขียนลงอยู่ดี

เกิดอะไรขึ้น: ของจริงมี `main\_bar\_count` ซองเดียวแบนๆ แต่ schema §6 กำหนดว่าต้องแยกเป็น `main\_bar{top, middle, bottom}` ต่อให้โมเดลอ่านมาถูกต้องสมบูรณ์แบบ ข้อมูลก็ถูกยุบทั้งตอนเขียนลงอยู่ดี

ต้นเหตุจริง: โครงสร้างข้อมูลฝั่ง production มีมาก่อนที่ schema จะกำหนดเรื่องแยกหน้าเหลืก แล้วก็ไม่เคยถูกขยายให้รองรับ ข้อมูลหายตรงจุดที่เขียนลง ซึ่งเป็นจุดที่การวัดความแม่นยำของตัวโมเดลเองมองไม่เห็น

เสียหายอะไร: ข้อมูลว่าเหลืกอยู่หน้าไหนถูกทำลายทุกครั้งที่สกด ไม่ว่าโมเดลจะดีแค่ไหน และทำให้การไป tune Pass 1 ไม่มีความหมายเลยจนกว่ารูปแบบ output จะเปลี่ยน

แก้ยังง: บันทึกไว้เป็นเงื่อนไขที่ต้องทำก่อน: ทั้ง prompt และรูปแบบ output ต้องเปลี่ยนก่อน การไป tune Pass 1 ถึงจะเริ่มมีค่า

กฎที่ได้: เช็คก่อนว่า schema ปลายทางเก็บสิ่งที่โมเดลผลิตออกมาได้จริงไหม ก่อนจะไป tune เพื่อสิ่งนั้น การไปพัฒนาตัวอ่าน ให้เก่งขึ้นทั้งที่ตัวเขียนตัดข้อมูลทิ้ง = เสียแรงเปล่า ขยายปลายทางให้รับได้ก่อน

t01 เทรนบนภาพที่เหลืแค่ ~266 visual token ต่อภาพ — เบลอกว่าที่ตั้งใจ 14 เท่า เพราะลืมใส่ resize="max"

เกิดอะไรขึ้น: รอบเทรน t01 ใช้ visual token ประมาณ 266 ต่อภาพ น้อยกว่าที่ตั้งใจไว้ราว 14 เท่า เพราะไม่เคยตั้งค่า `resize="max"` เลย เรื่องนี้ถูกระบุว่าเป็นหนึ่งในสองอย่างที่รู้าพังและต้องแก้ก่อนจะเริ่มรอบ t03

ต้นเหตุจริง: รับค่า default ของการเตรียมภาพมาใช้เลยโดยไม่เคยเช็คความั่นเหลื token ต่อภาพเท่าไร และในชุดตั้งค่าการเทรน ก็ไม่มีอะไรรายงานหรือ assert จำนวน visual token เลย

เสียหายอะไร: เทรน fine-tune ทั้งรอบบนภาพที่ความละเอียดต่ำจริงๆ — บนงานที่ต้องอ่าน mark กับตัวเลขระยะเล็กๆ บนแบบก่อสร้าง ซึ่งความละเอียดคือหัวใจทั้งหมด ตัวเลขทุกตัวของ t01 จึงถูกกดต่ำเพราะเรื่องนี้ และเพดานของโมเดลถูกกำหนดโดยค่า default ตัวเดียว

แก้ยังง: ยังไม่ได้แก้ — ระบุเป็นหนึ่งในสองอย่างที่ต้องแก้ให้ได้ก่อน t03 (อีกอย่างคือสร้าง dataset ใหม่จาก 11 หลังแทนที่จะเป็น 5 หลัง)

กฎที่ได้: assert จำนวน visual token ต่อภาพจริงๆ ก่อนเริ่มเทรน แล้ว log ไว้ด้วย บนงานที่ต้องอ่านรายละเอียดเล็กๆ ค่า default ของการเตรียมภาพตัวเดียวทำให้เสียทั้งรอบได้แบบเสียบๆ

VLM ยุดคนจริง 48 ตัวเหลือ 2 รายการใน JSON แล้วไม่มีอะไรจับได้เชิงกลไกเลย

เกิดอะไรขึ้น: บนหน้า 26 ของบ้าน 09 โมเดลพ่นคานออกมา 2 ตัวใน JSON ทั้งที่ในแผ่นมี 48 ตัว ความผิดพลาดนี้จะเห็นก็ต่อเมื่อมีคนมานั่งเพ่ง JSON เอง ไม่มีการตรวจอัตโนมัติไหนเลยที่เอาจำนวน element ที่ได้ไปเทียบกับอะไรสักอย่าง

ต้นเหตุจริง: เป็นอาการนับผิดของ VLM ที่มีบันทึกไว้แล้ว — cross-modal attention ที่ไม่สมดุลทำให้ตัวถอดรหัสเชื่อ text prior มากกว่า visual token และความแม่นยำตกฮวบเมื่อวัตุเกินราว 4 ชิ้น ข้ำเติมด้วยการที่ไม่มีแหล่งนับอิสระอีกแหล่งไว้เทียบเลย pipeline เลยไม่มีทางรู้ตัว

เสียหายอะไร: ground truth / output ของโมเดลผิดแบบเสียบๆ บน subtask ที่สำคัญที่สุด (beam plan) และตรวจไม่ได้เลยถ้าไม่มานั่งรื้อวิธีละหน้าด้วยมือ

แก้ยังง: เสนอไว้ (ยังไม่ได้ทำ) ให้มี pass ตรวจไขว้เพื่อ QA: ขอ output เป็น bbox มาพร้อมกับ grid_ref แล้วแปลง grid_ref→pixel ด้วย gridmaster ที่มีอยู่แล้ว แล้วให้ระบบเตือนอัตโนมัติเมื่อจำนวน bbox ไม่ตรงกับจำนวนใน elements[]

กฎที่ได้: output ของ VLM ที่เป็นจำนวนนับ ต้องเอาไปตรวจไขว้กับแหล่งนับอิสระอีกแหล่งเสมอ (classical detection, output แบบ bbox หรือตารางที่พิมพ์อยู่บนแบบ) ห้ามเชื่อมั่นลอยๆ เด็ดขาด

plan_beam recall ยังคงอยู่ที่ 0% — เป้าหมายหลักทั้งหมดของรอบ t03 ทำไม่สำเร็จ

เกิดอะไรขึ้น: ผลของบ้าน 08 ในรอบ t03: JSON ถูกต้อง 32/33 (97%), id-recall รวม 10/81 = 12.3% (ขึ้นจาก 3.5% ของ t02) โดย schedule 21.9% และ plan_slab 20% ดีขึ้นจริง แต่ plan_beam ได้ 0/15 = 0% เท่ากับ t02 เป๊ะ บนหน้า 20 โมเดลพ่นออกมาแค่ metadata แล้วปิด JSON เลยโดยไม่มี elements[] สักตัว แกม metadata ยังชี้ไปที่บ้านผิดหลัง (`source\_image` อ้างถึงบ้าน 22) คือมันแปะข้อความสำเร็จรูปที่ห้องจามาจากตัวอย่างที่คล้ายกัน

ต้นเหตุจริง: ตัวอย่างเทรน plan_beam 149 ตัวอย่างยังไม่พออยู่ดี และความล้มเหลวนี้เป็นเรื่องเชิงสถาปัตยกรรม (การนับที่หลอน) ไม่ใช่ปัญหาที่ขึ้นถอดรหัส grammar แก่เรื่องไวยากรณ์ได้ แต่ไม่ได้แก้เรื่องการดึงเนื้อหากลับมา — โมเดลผลิต JSON ที่ถูกต้องแต่ข้างในไม่มีอะไรเลย

เสียหายอะไร: เป้าหมายหลักของรอบนี้ล้มเหลว ที่แย่กว่านั้นคือ ตัวเลข JSON ที่ถูกต้องขึ้นสูง ในขณะที่ตัวชี้วัดเป้าหมายยังเป็นศูนย์ ซึ่งเป็นหน้าตาของผลลัพธ์ที่คนอ่านผิดเป็นความก้าวหน้าได้ง่ายมาก

แก้ยังง: ยังไม่ได้แก้ บันทึกไว้ตรงๆ ทางที่ระบุไว้ว่าจะไปต่อคือ เพิ่มข้อมูลผ่าน op04 หรือใช้แนวทาง grounding/bbox↔gridmaster และระบุชัดว่าไม่ใช้การไปแต่งคำตอนถอดรหัสอีกแล้ว

กฎที่ได้: ตัวเลข JSON ที่ถูกต้องขึ้นได้ ในขณะที่ตัวชี้วัดที่สำคัญจริงๆ ยังเป็นศูนย์ ให้รายงานตัวชี้วัดเป้าหมายแยกออกมาเสมอ และอย่าปล่อยให้ชัยชนะระดับรูปแบบมาแทนที่ชัยชนะระดับเนื้อหาในบทสรุปเด็ดขาด

โมเดล Purson พังเพราะบรรทัดเดียวที่ขาดหาย (crop_to_patches) ย่อภาพแบบทุกใบเหลือแค่ตราประทับ 256 token — เกิดซ้ำเป็นครั้งที่สามของบั๊กประเภทเดียวกัน

เกิดอะไรขึ้น: โมเดล Purson พังเพราะขาด `crop\_to\_patches` ไปแค่บรรทัดเดียว ทำให้แบบขนาด A1 ถูกย่อลงเหลือแค่ตราประทับที่ 256 token ต่อรูป มีการบันทึกหลักฐาน 3 ชั้น และระบุไว้ชัดเจนว่านี่คือครั้งที่สามที่บั๊กประเภทนี้เกิดขึ้น

ต้นเหตุจริง: ขั้นตอนเตรียมภาพเสื่อมคุณภาพลงแบบเสียวๆ แทนที่จะแจ้ง error — pipeline ยอมรับภาพที่ความละเอียดต่ำมากโดยไม่บ่นอะไรเลย ไม่มีการยืนยัน (assertion) จำนวน token หรือความละเอียดจริงต่อรูปเลย ทำให้บั๊กประเภทเดียวกันนี้เกิดซ้ำมาแล้วสองครั้งโดยไม่มีใครจับได้

เสียหายอะไร: โมเดลที่ fine-tune มาทั้งตัวใช้งานไม่ได้เลย ค่าใช้จ่าย compute และค่าเช่าเครื่องสูญเปล่าทั้งหมด เป็นการเกิดซ้ำครั้งที่สามของบั๊กประเภทเดียวกันนี้

แก้ยังไง: หาต้นเหตุและบันทึกไว้ใน `workmen's\_diary/สอนมะขาม\_ทำไม\_Purson\_กับ\_Voldemort\_พัง\_2026-09-01.md`

กฎที่ได้: ต้องเพิ่มการยืนยัน (assertion) แบบบังคับสำหรับจำนวน token ต่อรูป / ความละเอียดจริง ก่อนที่การเรนใดๆ จะเริ่มต้น บั๊กประเภทเดียวกันที่เกิดซ้ำมาแล้วสามครั้ง ต้องการตัวกันไม่ให้เกิดซ้ำ ไม่ใช่แค่การเขียนรายงานหาต้นเหตุอีกรอบ

โมเดล Voldemort ตาบอดเพราะบีบ ViT เป็น 4-bit quantization — รายงานว่าเห็นบาร์บีกับ BB-8 ในแบบก่อสร้าง

เกิดอะไรขึ้น: Voldemort พังด้วยสาเหตุที่ต่างจาก Purson: การ tiling ของภาพถูกต้องดีอยู่แล้ว แต่การบีบ quantization เป็น 4-bit ทำลาย ViT จนถึงขั้นตาบอด มันตอบว่าเห็นบาร์บีกับ BB-8 ในแบบก่อสร้าง

ต้นเหตุจริง: การบีบ quantization ถูกใช้แบบเดียวกันทั่วทั้งโมเดล รวมถึง vision tower ด้วย ซึ่งความละเอียด 4-bit เป็นระดับที่ vision tower หนไม่ได้ ไม่มีนโยบายความละเอียดแยกตามแต่ละส่วนของโมเดล และไม่มีการตรวจสอบภาพแบบง่ายๆ หลัง quantize เลย

เสียหายอะไร: โมเดล fine-tune ตัวที่สองใช้งานไม่ได้เลย — เสียค่าเทรน/ค่าเช่าเครื่องเพิ่มอีกก้อน

แก้ยังไง: หาต้นเหตุและบันทึกไว้ในเอกสารสอน

กฎที่ได้: ห้าม quantize vision tower ลงไปที่ 4-bit เด็ดขาด หลังจาก quantize ทุกครั้ง ให้รันการตรวจสอบภาพแบบง่ายๆ ('รูปนี้เห็นอะไร') ก่อนที่จะเสียเงินไปกับการประเมินหรือการเรนต่อ

การจำลองบ้าน house01 โกงตัวเองโดยไม่ได้ตั้งใจสองทาง ทำให้ผลลัพธ์ไม่สะท้อนความเป็นจริงของ Destrier

เกิดอะไรขึ้น: ตอนมะขามถามว่า JSON ที่ใช้จำลอง house01 เชื่อถือได้จริงไหมว่าเทียบเท่า Destrier พบว่าการโกงโดยไม่ได้ตั้งใจ 2 ทาง: (1) มันแมป `pattern` ไปเป็น subtask ตรงๆ (เช่น etc_plan → plan_slab เสมอ) ทั้งที่ pass0 จริงต้องตรวจดู element_type (2) มันคืนไฟล์ GT ทั้งไฟล์กลับมาแบบตรงๆ รวมถึงฟิลด์ที่การเรนจริงตัดทิ้ง (`element\_name\_assigned`) และประเภท element ที่โมเดล subtask ไม่เคยถูกสอนให้ตอบเลย

ต้นเหตุจริง: ตัวจำลองถูกสร้างขึ้นเพื่อผลิตผลลัพธ์ที่ดูสมเหตุสมผล แทนที่จะจำลอง code path จริงของ pass0/ตัวกรอง dataset มันข้าม logic การกรอง element_type ตาม subtask ของ `build\_dataset\_t03.py` ไปเลย ทำให้จำลองเป็นระบบที่เก่งกว่าความจริงมาก

เสียหายอะไร: ตัวเลขที่ดูน่าประทับใจของ house01 (62 ไฟล์, 153 element, BOQ \$144,704) สะท้อนระบบในอุดมคติ ไม่ใช่ระบบจริง พอสร้างใหม่ (`sim\_house\_from\_labels.py`) แสดงความจริง: pass0 ตอบอัตโนมัติได้แค่ 31 จาก 72 หน้า และได้งาน pass2 จริงแค่ 24 งานเท่านั้น

แก้ยังไง: สร้าง `sim\_house\_from\_labels.py` ให้อิง `pass0\_labels.jsonl` จริง และเลียนแบบการกรองของ `build\_dataset\_t03.py::PLAN\_SUBTASKS` แบบตรงๆ

กฎที่ได้: ตัวจำลองต้อง import หรือเลียนแบบ code path จริงเป๊ะๆ ไม่ใช่เขียนใหม่ตามที่คิดว่า 'มันน่าจะทำงานแบบนี้' ทางลัดใดๆ ในตัวจำลองจะกลายเป็นการโอ้อวดผลลัพธ์เกินจริง — และจะมีคนเอาตัวเลขนั้นไปอ้างอิงต่อจริงๆ

ถ้าแบ่ง train/val แบบสุ่มรายหน้า ข้อมูลจะรั่วข้ามฝั่ง

เกิดอะไรขึ้น: มีการตั้งว่าการแบ่ง train/val สำหรับ fine-tune ต้องแบ่งตามบ้าน ไม่ใช่สุ่มเลือกทีละหน้า เพราะหน้าจากบ้านหลังเดียวกันใช้กริด mark และสเปกเหมือนกัน

ต้นเหตุจริง: หน่วยตัวอย่างตามธรรมชาติของชุดข้อมูลคือ 'หน้า' การสุ่มแบ่งแบบไม่คิดอะไรเลยจึงเป็นการแบ่งรายหน้า ซึ่งทำให้แบบที่แทบจะเหมือนกันจากบ้านหลังเดียวกันไปอยู่ทั้งสองฝั่งของการแบ่ง

เสียหายอะไร: จับได้ก่อนที่จะใช้จริง ถ้าแบ่งรายหน้าจะได้ค่า validation ที่สวยเกินจริง ซึ่งจะบังปัญหาการ generalise ของโมเดลไว้

แก้ยังไง: แบ่งตามบ้านตามที่สั่ง (train = 01+02+04, val = 03) และแนะนำให้ใช้ 4/1 หรือ 5-fold ไปก่อน แล้วค่อยเปลี่ยนเป็น 80/10/10 เมื่อมีบ้านที่ label แล้ว 8-10 หลัง

กฎที่ได้: แบ่งข้อมูลสำหรับประเมินผลตามหน่วยที่ใหญ่ที่สุดที่ข้อมูลข้างในสัมพันธ์กัน (ในที่นี้คือ 'บ้าน') ไม่ใช่ตามหน่วยตัวอย่าง ถ้า val set รั่ว จะได้ตัวเลขที่คุณจะเชื่อทั้งที่ไม่ควรเชื่อ

สูง 2026-07-21

โมเดลที่ fine-tune แล้วยังหา element บนแบบเจอแค่ 9.4% — recall ยังไม่ได้แก้

เกิดอะไรขึ้น: ผลประเมินหลัง tuning แสดงความสำเร็จด้าน format ชัดเจน (JSON valid 0%→90%, view exact match 0%→60%) แต่ element recall อยู่ที่แค่ 16/170 (9.4%) คือโมเดลยังพลาดหลัก/คาน/เสาส่วนใหญ่ในแบบไปเลย

ต้นเหตุจริง: ชุดข้อมูลเทรนเล็ก (226 ตัวอย่างจาก 3 บ้าน) และการรันครั้งนี้ใช้ LoRA rank ที่ลดลง; format เรียนรู้ง่ายกว่าการนับ element ให้ครบทุกตัวมาก ทำให้ตัวเลขเด่นอย่าง 90% บดบังตัวชี้วัดที่สำคัญจริงๆ ไปได้

เสียหายอะไร: โมเดลที่ tune แล้วยังใช้ถอดปริมาณงานจริงไม่ได้ ต้อง tune รอบใหม่ — และ artifact ที่จะช่วยรอบต่อไปก็สูญหายไปภายหลังด้วย

แก้ยังไง: ยังไม่แก้ — บันทึกไว้เป็นเป้าหมายของรอบ tuning ถัดไป (เพิ่มข้อมูล / เพิ่ม epoch / ปรับ prompt)

กฎที่ได้: ตัดสินคุณภาพ fine-tune จากตัวชี้วัดที่มีมูลค่าทางธุรกิจจริง (element recall) ไม่ใช่ตัวที่ขยับง่ายที่สุด (JSON validity) ให้รายงานทั้งคู่กันเสมอ อย่าให้ตัวเลขเด่นตัวเดียวทำให้ดูเหมือนพร้อมใช้งานทั้งที่ยังไม่พร้อม

สูง 2026-07-21

GGUF ออกมาเป็น text-only เพราะ export ตัว projector สำหรับ vision (mmpoj) ไม่สำเร็จ

เกิดอะไรขึ้น: การแปลง GGUF สำเร็จสำหรับส่วนภาษา แต่ mmpoj (ตัวเชื่อม image encoder) แยกออกมาไม่ได้ — llama.cpp เองก็เตือนไว้ว่ารองรับแค่บางโมเดล ผลลัพธ์ที่ได้จึงอ่านแบบไม่ได้เลย ทั้งที่นี่คือจุดประสงค์หลักของโปรเจกต์

ต้นเหตุจริง: ไม่เคยตรวจสอบว่า export path นี้รองรับโมเดลตระกูลใหม่มาก่อนถึงปลาย pipeline เลย — สมมติไปเองว่าเครื่อง vision จะตามครึ่งข้อความมาได้

เสียหายอะไร: งานที่ส่งมอบของวันนั้นใช้ทำงานจริงไม่ได้ พบเรื่องนี้ตอนท้ายสุดของการรันเท่านั้น

แก้ยังไง: สามวันถัดมาพบว่าจริงๆ แก้ได้ฟรี — เพราะการเทรนสล็อตเลเยอร์ vision ไว้ (finetune_vision_layers=False) mmpoj ของค่ายต้นทาง (899MB) จึงเหมือนกันทุก byte ดาวน์โหลดมาจับคู่กับ GGUF ที่ tune แล้วได้เลย

กฎที่ได้: ตรวจสอบขั้นตอนที่ยากที่สุด/ใหม่ที่สุดของ export pipeline ก่อน บนโมเดลทดลองทั้ง ก่อนหุ้มเทรนจริงทั้งรอบไปกับมัน และเช็คว่า component ที่ถูกล็อกไว้ (frozen) สามารถหยิบยืมจากต้นทางมาใช้ได้เลยไหม แทนที่จะต้องแปลงเองใหม่

สูง 2026-07-24

โมเดลที่ fine-tune แล้ว recall element ได้แค่ 28.2% แถมหายกันไปอีก 78 ชิ้น

เกิดอะไรขึ้น: หลังแก้ reasoning-mode แล้ว วัดผลกับ --limit 20 ได้ JSON valid 90%, view exact match 70%, element recall 28.2%, element_id/count exact 100%, element_type exact 87.5% — และมี element ที่หายไถ่มา 78 ชิ้น

ต้นเหตุจริง: ชุด training เล็กเกินไป (315 ตัวอย่างจากบ้านแค่ 5 หลัง) และงานระดับ element เป็นส่วนที่ยากที่สุดของ schema โมเดลจึงพึ่งพลาตของจริงส่วนใหญ่และหลอนของปลอมออกมาด้วย

เสียหายอะไร: เอาโมเดลนี้ไปใช้ผลิต raw JSON จริงไม่ได้เลย — element หายไปกว่า 70% แถมมีของปลอมปนมาอีก เป็นตัวชี้ขาดว่าผลลัพธ์ของมันเป็นแค่ draft ที่ต้องให้คนตรวจซ้ำ

แก้ยังไง: ยังไม่แก้ — บันทึกไว้ตามจริงว่าสถานะตอนนี้เป็นแบบนี้ ทางแก้ต่อไปคือต้อง label บ้านเพิ่ม

กฎที่ได้: รายงานตัวเลขที่แย่ที่สุดควบคู่กับตัวเลขเด่นเสมอ การที่ recall ดีขึ้น 3 เท่าแต่ยังจบที่ 28% ก็ยังใช้งานจริงไม่ได้อยู่ดี และการหายไถ่ต้องแยกวัดเป็นตัวเลขของตัวเอง ห้ามเอาไปปนกับ recall

สูง 2026-07-25

ข้อเสนอแยกการเรียกโมเดลเป็นหลาย pass เกือบทำให้ข้อได้เปรียบจาก fine-tuning หายไปเสียๆ

เกิดอะไรขึ้น: มีข้อเสนอให้แยกขั้นตอน extraction เป็น 5 sub-call แบบเดียวกับ production pipeline ของ Constistant ถูกวิเคราะห์แล้วปฏิเสธ — เพราะโมเดลถูก train มาให้เรียกครั้งเดียวต่อหน้า sub-task ต่างๆ (survey ID, การนับ) ไม่เคยถูกเห็นตอน training เลย ดังนั้นจะกลายเป็นให้ Qwen ต้นฉบับ (base) ทำงานแทน ซึ่งเท่ากับเสียข้อได้เปรียบจากการ tune ไปโดยไม่รู้ตัว แถมยังเสียเวลาเพิ่มอีกราว 15 นาทีต่อหน้าจากการประมวลผลรูปซ้ำ 5 รอบ

ต้นเหตุจริง: รูปแบบการเรียก inference ถูกออกแบบแยกต่างหากจากรูปแบบข้อมูล training โมเดลที่ fine-tune แล้วจะรักษาข้อได้เปรียบไว้ได้ก็ต่อเมื่อใช้กับรูปแบบงานที่มันถูก train มาเท่านั้น

เสียหายอะไร: จับได้ตอนวิเคราะห์ก่อนลงมือทำจริง — ไม่งั้นจะเสียเวลาหลายชั่วโมงต่อบ้านหนึ่งหลัง แต่ได้คุณภาพแค่ระดับ base model ในราคา 5 เท่าของค่าประมวลผลรูป

แก้ยังไง: สรุปว่า: ถ้าจะใช้โมเดลที่ tune แล้ว ต้องเรียกครั้งเดียวต่อหน้า (แก้ปัญหา truncation ด้วยการตั้ง ctx ให้ขนาดพอดี ยอมรับว่าจะช้า) ถ้าจะทำ multi-pass จริงๆ ต้องใช้ cloud pipeline เดิม หรือไม่ก็ retrain ใหม่ด้วยข้อมูลที่จัดรูปแบบ multi-pass

กฎที่ได้: รูปแบบ training ต้องตรงกับรูปแบบการเรียกใช้งานจริง การเปลี่ยนวิธีเรียกโมเดลที่ fine-tune แล้ว จะทำให้มันกลับไปทำตัวเหมือน base model แบบเสียๆ โดยไม่มี error เตือนเลย

สูง 2026-07-28

บ้าน07 EE-05/06: ปลั๊กทีวี 4 จุดหายไปทั้งหมด ทั้งที่แบบมีไดอะแกรมสปลิตเตอร์ 4 ทางที่บอกจำนวนไว้อยู่แล้ว

เกิดอะไรขึ้น: สัญลักษณ์ปลั๊กทีวี (⊙) 4 จุดไม่เคยถูก extract เลย ทั้งที่แผ่นเดียวกันมีไดอะแกรมสปลิตเตอร์ 4 ทางที่จำนวนช่องต่อตรงกับปลั๊กทีวี 4 จุดพอดี — หลักฐานอยู่บนแผ่นนั้นอยู่แล้ว

ต้นเหตุจริง: การอ่านสัญลักษณ์ทำแบบแยกจุดต่อจุดโดยไม่ใช้การตรวจสอบความสอดคล้องภายในของแผ่นเอง (จำนวนช่องของสปลิตเตอร์ต้องกำหนดจำนวนปลั๊กที่มันเลี้ยงอยู่)

เสียหายอะไร: อุปกรณ์ 4 ชิ้นหายไปจากยอดถอดปริมาณงานไฟฟ้า ทำให้ปริมาณและต้นทุนต่ำกว่าความจริง

แก้ยังไง: กู้ปลั๊กที่หายกลับมาและตรวจทานไขว้กับไดอะแกรมสปลิตเตอร์

กฎที่ได้: ใช้เลขคณิตภายในของแผ่นแบบเองเป็นตัวเช็คความครบถ้วน — ช่องสปลิตเตอร์, ตารางแผงไฟ, จำนวนวงจร ล้วนกำหนดว่าต้องมีอุปกรณ์ปลายทางกี่ตัว ถ้ายอดไม่ตรงกันแสดงว่ามีอะไรตกหล่นไป

สูง 2026-07-29

โมเดลถูกเทรนด้วยภาพขนาด 512px (~266 visual token) แต่ตอน serve จริงป้อนถึง 3,796 token/ภาพ — ต่างกันถึง 14.3 เท่า

เกิดอะไรขึ้น: log ของ llama-server รายงานว่าใช้ $2048 + 1748 = 3,796$ visual token ต่อภาพตอน inference ในขณะที่บั๊กใน collator ย่อภาพตอนเทรนเหลือแค่ 512px หรือประมาณ 266 token โมเดลจึงถูกขอให้อ่านข้อมูลที่หนาแน่นกว่าที่เคยฝึกมาถึง 14.3 เท่า

ต้นเหตุจริง: collator ใช้ขนาดภาพ default แบบเสียบๆ เพราะโมเดลไม่ได้ประกาศขนาด default ไว้ ทำให้ resolution ตอนเทรนไม่เคยเป็นการตัดสินใจที่จงใจเลือกมาก่อน และไม่มีใครเอาไปเทียบกับ resolution ที่ serving stack จะป้อนเข้ามาจริง

เสียหายอะไร: คุณภาพการอ่านแบบที่แย่นี้ในคืนนั้นส่วนใหญ่จะมาจากเรื่องนี้เพียงอย่างเดียว ความรู้ที่เรียนมาในตอนเทรนแทบใช้ไม่ได้เลยที่ resolution ตอน serving จริง

แก้ยังง: ยังไม่ได้แก้ — ระบุแนวทางแก้ไว้แล้ว (--image-min-tokens 1024 ลองใช้ได้ฟรี) และมีแผนสำรองคือ train ใหม่ด้วย `resize="max"`

กฎที่ได้: ต้อง log และ assert รูปร่างข้อมูลนำเข้าที่ preprocess แล้วจริงๆ ทั้งตอนเทรนและตอน inference ให้ตรงกัน และถ้าไม่ตรงต้องให้ฟังก์ชันที่แบบเห็นชัด ปัญหา train/inference mismatch แบบนี้จะมองไม่เห็นถ้าดู log แยกกัน จะเห็นก็ต่อเมื่อเอาตัวเลขทั้งสองมาวางเทียบกันเท่านั้น

สูง 2026-07-29

quantize เป็น Q4_K_M ทั้งที่มีคำเตือนจากผู้พัฒนาไลบรารีเองว่าไม่ควรทำ แล้วก็ไม่เคยวัดผลซ้ำอีกเลย

เกิดอะไรขึ้น: เอกสาร t01_workflow.md บันทึกไว้ว่า Unsloth เตือนว่าโมเดลตระกูลนี้ 'สูญเสียคุณภาพผิดปกติเมื่อ quantize' ซึ่งเป็นเหตุผลที่บังคับให้เทรนด้วย bf16 และห้ามใช้ 4-bit แต่พอถึงขั้นตอน deploy กลับบีบขนาดจาก 70GB เหลือ 21GB แบบ Q4_K_M อยู่ดี (เพราะโน้ตบุ๊ก 8GB ใส่ 70GB ไม่ได้) และไม่เคยวัดความแม่นยำซ้ำหลัง compress เลย

ต้นเหตุจริง: ข้อจำกัดถูกบังคับใช้อย่างเคร่งครัดตอนเทรน แต่กลับถูกปล่อยผ่านเสียบๆ ตอน inference เพราะไม่มีใครมองว่า 'การเอาไปใช้งานจริง' เป็นขั้นตอนที่อาจทำให้คุณภาพเปลี่ยนไปได้เหมือนกัน

เสียหายอะไร: ไม่รู้เลยว่าความคุณภาพที่ฟังในคืนนั้นเท่าไรมาจากความเสียหายจากการ quantize และไม่มีการวัดผลใดๆ ที่จะแยกได้ว่าเป็นผลจากบั๊ก merge หรือปัญหา resolution mismatch

แก้ยังง: ยังไม่ได้แก้ — ต้องทำ A/B เทียบกับโมเดลที่ไม่ได้ quantize แบบมีการควบคุมตัวแปร

กฎที่ได้: ข้อจำกัดเรื่องคุณภาพต้องบังคับใช้กับทั้ง pipeline ไม่ใช่แค่ขั้นตอนเทรน เมื่อจำเป็นต้องละเมิดข้อจำกัดข้อไหนเพราะความจำเป็น ต้องจดงานวัดผลซ้ำเป็นงานบังคับพร้อมกันในตอนนั้นเลย มิฉะนั้นสิ่งที่รู้อยู่แล้วว่าเป็นการยอมประนีประนอม จะกลายเป็นสิ่งที่ไม่รู้อะไรเลยในอีกสัปดาห์ถัดมา

สูง 2026-07-29

เทรน/ใช้งานไม่ตรงกัน: เทรนที่ราว 266 visual tokens/ภาพ แต่ใช้งานจริงที่ 3,796

เกิดอะไรขึ้น: log ของ llama-server แสดงว่าตอน inference ใช้ 3,796 visual tokens ต่อภาพ ในขณะที่บั๊กของ collator ทำให้ตอนเทรนใช้แค่ราว 266 — ต่างกันถึง 14.3 เท่าระหว่างสิ่งที่โมเดลเรียนรู้กับสิ่งที่มันเห็นตอนใช้งานจริง

ต้นเหตุจริง: pipeline ภาพฝั่งเทรน (ค่า default ของ Unsloth collator) กับฝั่ง inference (llama.cpp) ถูกตั้งค่าแยกกันโดยไม่มีใครเทียบกันเลย ฝั่งเทรนลด resolution ลงโดยไม่มีใครรู้

เสียหายอะไร: สิ่งที่โมเดลเรียนรู้มาอาจไม่ transfer มาที่ resolution ตอนใช้งานจริงเลย เป็นสาเหตุที่เป็นไปได้อีกข้อของคุณภาพ extraction ที่แย่ในวันที่ 29 ก.ค.

แก้ยังไม: ยังไม่แก้ พบวิธีลองพรีอย่างแรกคือใส่ --image-min-tokens 1024 ส่วนแผนสำรองคือเทรนใหม่โดยตั้ง resize="max"

กฎที่ได้: ต้องยืนยันว่าการเตรียมภาพตอน inference ตรงกับตอนเทรนเป๊ะ — จับตัวเลขจริงของจำนวน token/pixel ทั้งสองฝั่งมาเทียบกัน อย่าเทียบแค่ค่าตั้งค่า (settings) เฉยๆ

สูง 2026-07-29

quantize เป็น Q4_K_M ทั้งที่ผู้พัฒนาเตือนไว้เอง แล้วไม่เคยวัดความแม่นยำใหม่อีกเลยหลังบีบอัด

เกิดอะไรขึ้น: Unsloth เตือนไว้ว่าโมเดลตระกูลนี้เสื่อมคุณภาพผิดปกติเมื่อ quantize — นี่คือเหตุผลที่ตอนเทรนต้องบังคับใช้ bf16 และห้าม 4-bit แต่พอถึงขั้นตอน deploy กลับบีบอัดโมเดลจาก 70GB เหลือ 21GB เป็น Q4_K_M (จำเป็นเพราะโน้ตบุ๊กมีแค่ 8GB) และไม่เคยวัดความแม่นยำใหม่อีกเลยหลังบีบอัดเลย

ต้นเหตุจริง: ค่าเตือนของผู้พัฒนาถูกนำไปใช้แค่ในขั้นตอนเทรนที่อ่านเจอครั้งแรก ไม่ถูกส่งต่อไปยังขั้นตอน deploy ที่มันเกี่ยวข้องพอๆ กัน ข้อจำกัดของฮาร์ดแวร์ถูกปล่อยให้มาแทนที่การตัดสินใจเรื่องคุณภาพ

เสียหายอะไร: artifact ที่ใช้งานจริงทุกวันไม่มีการวัดความแม่นยำที่แน่ชัดเลย เป็นสาเหตุที่เป็นไปได้อีกข้อของผลลัพธ์แย่ในวันที่ 29 ก.ค. ที่ไม่สามารถตัดออกหรือยืนยันได้

แก้ยังไม: ยังไม่แก้ บันทึกไว้เป็นสาเหตุรอง การวัดความแม่นยำของ artifact ที่ผ่านการ quantize แล้วยังคงอยู่

กฎที่ได้: ค่าเตือนเรื่องคุณภาพของโมเดลใช้ได้กับทุกขั้นตอน ไม่ใช่แค่ขั้นตอนที่อ่านเจอครั้งแรก การแปลงข้อมูลแบบ lossy ใดๆ กับ artifact ที่จะใช้งานจริง ต้องวัดความแม่นยำใหมบน artifact ตัวนั้นเป๊ะๆ เสมอ

สูง 2026-07-29

โมเดลอ่านเลขอ้างอิงแผ่นแบบ (A-05, A-07, A-09) เป็นชื่อมาร์คของเสา

เกิดอะไรขึ้น: ในแบบมีข้อความอ้างอิงข้ามแผ่นแบบ 'ดูรายละเอียดที่แผ่น A-05' โมเดลกลับตีความ A-05 / A-07 / A-09 ว่าเป็นชื่อของเสาแล้วเขียนออกมาเป็นมาร์คของชิ้นส่วน

ต้นเหตุจริง: โมเดลอ่านตัวหนังสือไทยได้แม่นยำ แต่ไม่เข้าใจว่าข้อความนั้นทำหน้าที่อะไร — เป็นความล้มเหลวเชิงความหมาย (semantic role) ไม่ใช่ปัญหาการอ่านภาพ (OCR) มันอ่านตัวชี้ (pointer) เป็นชื่อ (name)

เสียหายอะไร: สร้างชิ้นส่วนโครงสร้างปลอมที่มีเลขแผ่นแบบเป็นชื่อมาร์ค ซึ่งจะไหลเข้า BBS/BOQ เป็นรายการจริงถ้าไม่มีใครจับได้

แก้ยังไม: ยังไม่แก้ บันทึกไว้เป็นหลักฐานว่าปัญหาคือความเข้าใจ ไม่ใช่การมองเห็น

กฎที่ได้: ตรวจสอบมาร์คที่สกัดออกมาเทียบกับคลังคำศัพท์มาร์คจริงของโปรเจกต์ (และปฏิเสธอะไรก็ตามที่มีรูปแบบเหมือนเลขแผ่นแบบ) — โมเดลอ่านตัวหนังสือถูกได้สมบูรณ์แบบ แต่ยังจำแนกความหมายของมันผิดได้อยู่ดี

สูง 2026-07-29

ผลลัพธ์ขัดแย้งกันเอง: ห้องสองห้องถูกกำหนดจุดกริดชุดเดียวกันเป๊ะ

เกิดอะไรขึ้น: F6 ถูกกำหนดจุดกริด ["C-2","C-3","D-3","D-2"] และตั้งชื่อว่า 'เตรียมอาหาร' ส่วน F7 ถูกกำหนดจุดกริดเดียวกันสี่จุดแต่เรียงลำดับต่างกัน แล้วตั้งชื่อว่า 'นอน 2'

ต้นเหตุจริง: ไม่มีอะไรใน pipeline คอยตรวจสอบไขว้ (cross-check) รายการที่ผลิตออกมากับรายการอื่น โมเดลสร้างแต่ละรายการแยกกันโดยไม่มีข้อจำกัดเรื่องความสอดคล้องในภาพรวม

เสียหายอะไร: การนิยามห้องสองห้องที่ขัดแย้งกันเอง (พื้นที่ทับซ้อนกันไม่ได้จริง) เข้าไปครอบครองพื้นที่เดียวกัน เป็นความผิดพลาดที่จะไหลเข้าไปในตัวเลขพื้นที่และงานตกแต่งโดยไม่มีใครรู้ตัว

แก้ยังไง: ยังไม่แก้

กฎที่ได้: รัน assertion ตรวจสอบความสอดคล้องภายในที่ราคาถูกกับผลลัพธ์ของโมเดล (ไม่มีสองพื้นที่ไหนที่มีจุดชุดเดียวกันเป๊ะ ไม่มีมาร์คซ้ำที่มีคุณสมบัติต่างกัน) — สิ่งเหล่านี้จับความผิดพลาดที่ metric ความแม่นยำรายไฟล์ไม่มีทางจับได้เลย

สูง 2026-07-29

confidence_score กับ confidence_flags ถูกผลิตออกมา 0 ครั้งตลอดทั้ง 6 ไฟล์

เกิดอะไรขึ้น: schema ต้นฉบับ (ground truth) มีฟิลด์ confidence_score และ confidence_flags ที่บอกจุดที่ผู้อ่านตีความไม่ได้ (เช่น 'ตัวหนังสือเล็กเกิน ต้อง zoom') แต่โมเดลไม่ผลิตฟิลด์เหล่านี้เลยแม้แต่ตัวเดียวในทั้ง 6 ไฟล์

ต้นเหตุจริง: ฟิลด์เหล่านี้มีอยู่แค่ในคำตอบที่ใช้เทรน (training answers) ไม่เคยปรากฏใน prompt เลย — โมเดลที่ไม่ได้โหลด fine-tune จึงไม่มีทางรู้ว่าต้องมีฟิลด์พวกนี้ การที่มันหายไปทั้งหมดคือหลักฐานสนับสนุนสมมติฐานว่า LoRA หายไป

เสียหายอะไร: ไม่มีทางรู้เลยว่าส่วนไหนของผลลัพธ์เชื่อถือได้และส่วนไหนควรตรวจสอบซ้ำ ทำให้ต้องถือว่าทุกค่ายังไม่ผ่านการยืนยัน — ลบล้างจุดประสงค์ของ schema ที่ออกแบบมาเพื่อชี้เป้าให้คนตรวจ

แก้ยังไง: ยังไม่แก้ ใช้เป็นหลักฐานในการวินิจฉัยว่า LoRA มีอยู่จริงหรือไม่

กฎที่ได้: ฟิลด์ที่ปรากฏแค่ในคำตอบที่ใช้เทรนแต่ไม่เคยอยู่ใน prompt เลย คือตัวตรวจจับที่ดีที่สุดว่า fine-tune ยังอยู่จริงหรือไม่ — ควรเช็กการมีอยู่ของฟิลด์พวกนี้เป็นจุดตรวจสอบ sanity ของ artifact โดยตรง

สูง 2026-07-29

โมเดลสลับทั้งประเภทงานและชนิดวัสดุพร้อมกัน: ผนังอิฐมอญกลายเป็นฝ้าเพดานอิฐมวลเบา

เกิดอะไรขึ้น: ในหน้า 04 ความจริงคือผนัง (ผนัง) ที่ก่อด้วยอิฐมอญ แต่โมเดลกลับให้ผลเป็นฝ้าเพดาน (ฝ้าเพดาน) ที่ทำจากอิฐมวลเบา (AAC block)

ต้นเหตุจริง: โมเดลจับคู่คำที่ผิวเผินโดยไม่ผูกมัดเข้ากับชิ้นส่วนที่มันสังกัดอยู่จริง และไม่มีอะไรตรวจสอบว่าวัสดุนั้นใช้ได้กับประเภทงานที่กำหนดหรือไม่

เสียหายอะไร: ผิดสองมิติพร้อมกัน — ถ้าเอาไปตั้งราคา ทั้งประเภทงานและอัตราต่อหน่วยจะผิดทั้งคู่ ทำให้ความผิดพลาดใน BOQ ยิ่งทบเข้าไปแทนที่จะหักล้างกันเอง

แก้ยังไง: ยังไม่ได้แก้

กฎที่ได้: ต้องมีตารางตรวจสอบความสมเหตุสมผล (legality table) เทียบวัสดุกับประเภทงานก่อนที่บรรทัดที่สัปดาห์จะเข้าสู่การตั้งราคา — ความผิดพลาดสองมิติจะให้ตัวเลขที่ผิดแบบดูน่าเชื่อถือ ไม่ใช่ผิดแบบเห็นชัด

สูง 2026-08-04

มีคำศัพท์ประเภทหน้าแบบ 6 ชุดที่เข้ากันไม่ได้ ตอบคำถามเดียวกัน — และตอนนี้มี 2 จุดที่ข้อมูลหายจริง

เกิดอะไรขึ้น: คำถามว่า "หน้านี้เป็นหน้าประเภทไหน" ถูกตอบด้วยคำศัพท์ 6 ชุด (schema §1 มี 14/16 คำ, Pass 0 มี 2, CLASSIFY_SHEET/sheet_type มี 5, drawing_type มี 3, classifyPageType() มี 7, schema-v2 ที่ฝึกไว้มี 6) และมี 2 จุดที่ทำข้อมูลหายอยู่ทุกวันนี้: `wz-step1.js:224` ยุบ 5 คำให้เหลือ 2 (general_notes/schedule_table/unknown กลายเป็น section_detail หหมด) และ `drawing-index.js:205-208` ฮาร์ดโค้ด section_detail หับคำที่อ่านมาได้จริง

ต้นเหตุจริง: แนวคิดเดียวกันถูกสร้างแบบแยกกัน 6 ที่ตามเวลาที่ผ่านไป โดยไม่มีเจ้าของที่เป็นตัวจริง ผู้ใช้งานใหม่แต่ละรายก็นิยามคำศัพท์ที่ตัวเองต้องการขึ้นมาเองในที่ของตัวเอง

เสียหายอะไร: ข้อมูลการจำแนกหน้าถูกทำลายทุกครั้งที่รันใน 2 จุดที่รู้อยู่ — ประเภทหน้าที่อ่านมาถูกต้องกลับถูกเขียนทับด้วยคำที่ฮาร์ดโค้ดไว้ เกม Constatant ยังเก็บผลลัพธ์แบบ 16 คำไม่ได้เชิงโครงสร้าง จึงเอาตัวจำแนกที่ดีกว่ามาใช้ไม่ได้เลย

แก้ง่าย: ยังไม่แก้ — ตัดสินว่าตอนนี้ยังไม่คุ้ม เติมเฉพาะ 2 pattern ที่ขาดจริงๆ เข้า schema ส่วนการขยาย `sheet\_type`/`drawing\_type` บันทึกไว้ว่าเป็นเงื่อนไขที่ต้องทำก่อนถ้าจะมี Pass 0 ตัวใหม่

กฎที่ได้: หนึ่งแนวคิด หนึ่งชุดคำศัพท์ หนึ่งเจ้าของ และห้ามฮาร์ดโค้ดหับคำที่เพิ่งอ่านมาได้ — ถ้าผลที่อ่านมาถูกต้อง ก็ไม่ควรเสียเงินเสียเวลาไปอ่านตั้งแต่แรก

สูง 2026-08-21

ลูบครบวงจรที่หาเสร็จได้ element 21 ตัว แต่ได้ span 0 และ spec 0 — เป็นปัญหาคุณภาพโมเดล ไม่ใช่โค้ดแพลตฟอร์ม

เกิดอะไรขึ้น: หลังแก้บั๊กใน adapter ทั้งสองตัวแล้ว ไฟล์ที่ Qwen สก๊ตออกมาจากบ้านหลัง 09 นำเข้าได้สำเร็จ ได้ element 21 ตัว รวมของจริงอย่าง C1 × 8, C2 × 12, B2 × 18, ฐานราก และโครงถักเหล็ก — แต่ span ได้ 0/21 และ spec ได้ 0 เพราะโมเดลไม่เคยออก grid_ref_start/grid_ref_end เลย (มันยึดทั้งแผ่นเป็นสตริงเดียวแบบ F-F x 1-2, F-F x 2-3, ...) และไม่มีหน้า section หรือ schedule ของบ้านหลังนี้ถูกรันผ่านโมเดลเลยสักหน้า ขณะที่ ground truth ของบ้านหลังเดียวกันผ่าน adapter ตัวเดียวกันได้ 87 element / 55 spec / 18 span

ต้นเหตุจริง: มีสองสาเหตุซ้อนกัน — โมเดลไม่ออก grid ref รายชั้นบนแปลนคาน (ปัญหา recall 11% ซึ่งมาจากตัวอย่างเทรนแค่ราวๆ 13 ตัว) และขอบเขตการรันก็ไม่เคยครอบคลุมหน้า section/schedule ซึ่งเป็นแหล่งเดียวของ spec

เสียหายอะไร: เดโม ลูบครบวงจร ได้ element ที่ไม่มีรูปทรงและไม่มี spec วัสดุ BOQ นับของได้แต่ตีราคาหรือกำหนดขนาดไม่ได้ เรื่องนี้บอกตรงๆ ว่าเป็นช่องโหว่ ไม่ได้กลบไว้

แก้ง่าย: ยังไม่ได้แก้ บันทึกว่าเป็นเรื่องคุณภาพโมเดล ไม่ใช่โค้ดแพลตฟอร์ม ข้อเสนอคือเอาหน้า section/schedule ของบ้านหลังนี้รันผ่าน Qwen เพื่อให้มี spec อย่างน้อยให้ BOQ ตีราคาเหล็กได้

กฎที่ได้: เวลาใช้ลูบครบวงจร ให้บอกส่วนต่างเทียบกับ ground truth ที่ผ่านโค้ดเส้นทางเดียวกันด้วย (21 กับ 87 element, 0 กับ 18 span) — ลูบที่รันผ่านไม่ได้แปลว่าลูบนั้นใช้งานได้ และการเทียบกับ ground truth ผ่าน adapter ตัวเดียวกันคือวิธีเดียวที่จะแยกบั๊กของแพลตฟอร์มออกจากคุณภาพโมเดล

สูง 2026-08-21

บนแปลนคาน โมเดลยิบทั้งแผ่นเหลือ element แบบโน้ตโซนราวๆ 2 ตัว แทนที่จะออกทีละคาน

เกิดอะไรขึ้น: การเทียบภาพซ้อนบนบ้านหลัง 09 หน้า 26 พบว่าโมเดลย่อแผ่นที่มีคานจริง 48 ตัวเหลือ element 2 ตัว โดยยัดทั้งหน้าเป็นสตริงโน้ตโซนอันเดียว (F-F x 1-2, F-F x 2-3, ...) ซึ่งกล่องของมันลอยอยู่เฉยๆ ไม่ตรงกับเส้นคานจริงเส้นไหนเลย ส่วนหน้า 27 วางตำแหน่งไม่ได้เลย นี่เป็นความพังคนละแบบกับการหลอนรายละเอียดรั้ว และคนละแบบกับ JSON parse ไม่ผ่าน — ตรงนี้ JSON ถูกต้องทุกอย่าง แต่เนื้อหาผิดโครงสร้าง

ต้นเหตุจริง: ตัวอย่างเทรนของแปลนคานน้อยเกินไป (ราวๆ 13 ตัว) จนสอนไม่ได้ว่าต้องออกหนึ่งแถวต่อหนึ่งช่วงคาน โมเดลเลยถอยไปสรุปทั้งแผ่นด้วยโน้ตแบบย่อที่มันเคยเห็นในคำบรรยายภาพและในตาราง

เสียหายอะไร: ถึงแม้ grammar decoding จะทำให้ 96.8% ของหน้า parse ผ่านแล้ว แต่ช่วงคานก็ยังใช้ไม่ได้อยู่ดี — การนำเข้าฝั่งแพลตฟอร์มได้ span 0/21 เทียบกับ ground truth ที่ได้ 18/87 ความยาวคานจึงไม่เคยไปถึง BOQ

แก้ยังง: ยังไม่ได้แก้ ยกไปทำในรอบ t03 ด้วยการแตก subtask ตาม pattern บวกกับเพิ่มตัวอย่างเทรนของแปลนคาน

กฎที่ได้: JSON ถูกต้องไม่ได้แปลว่าเนื้อหาถูกต้อง ก่อนจะประกาศว่ารอบสกัดข้อมูลสำเร็จ ให้วัด recall ระดับ element ด้วยการซ้อนภาพเทียบกับ ground truth ไม่ใช่ดูแค่อัตรา parse ผ่าน

สูง 2026-08-24

t01 อ่านได้คาน 5 ตัว เส่า 0 ต้น บนหน้าที่ GT บอกว่ามีคาน 16 ตัว

เกิดอะไรขึ้น: บนบ้านหลัง 11 หน้า 22 โมเดล t01 ที่หูนแล้วรายงานว่ามีคาน 5 ตัว เส่า 0 ต้น ทั้งที่บนแผ่นเห็นป้ายคาน 13 กว่ตัวชัดๆ พร้อมเส่า และ GT บันทึกไว้ 16 ตัว ทั้ง base และตัวที่หูนได้หน้าที่ใช้ได้ 6 หน้าเท่ากัน และ base มักเร็วกว่าด้วยซ้ำ (หน้า 21: 121 วินาที เทียบกับ 410 วินาที) ซึ่งน่าจะแค่พูดน้อยกว่า ไม่ใช่ฉลาดกว่า

ต้นเหตุจริง: เป็นอาการหลอนเรื่องการนับที่ฝังอยู่ในสถาปัตยกรรมโมเดล ซึ่งบันทึกไว้แล้วตั้งแต่ 2026-08-22 ตอนนี้ยืนยันแล้วว่าเกิดกับโมเดลของเราเองบนข้อมูลของเราเอง fine-tune ไม่ได้ช่วยรักษาเรื่องนี้

เสียหายอะไร: ยืนยันว่าโมเดลที่หูนแล้วเชื่อถือไม่ได้เรื่องการนับจำนวนชิ้นส่วน การเทียบหูนกับไม่หูนจึงไม่ได้ผลชนะด้านคุณภาพเลยได้แค่ยืนยันรูปแบบความพังเท่านั้น

แก้ยังง: ยังไม่ได้แก้ บันทึกเป็นการยืนยันสดของผลวิจัย 2026-08-22 ส่วนทิศทางแก้ (detection ก่อน หรือ ใ้แบบ GroundCount) ยังไม่ได้สร้าง

กฎที่ได้: อย่าอ่านความต่างของความเร็ระหว่าง base กับตัวที่หูนว่าเป็นสัญญาณคุณภาพ ให้เทียบจำนวนชิ้นส่วนกับ GT เสมอ และถือว่าความแม่นยำของการนับเป็นตัววัดคนละตัวกับความถูกต้องของ JSON

สูง 2026-08-25

โมเดลคืน element list วางเปล่า พร้อม header ที่ก๊อปปี้มาจากบ้านคนละหลัง

เกิดอะไรขึ้น: บนหน้าแปลนคานที่ยาก โมเดลพ่นออกมาแคฟิวด์ header กับ "elements": [] ไม่มีคานสักตัว แยกว่านั่นคือ header ชีไปบ้านผิดหลัง (บอกว่าบ้าน 22 ทั้งที่คำสั่งอ่านบ้าน 08)

ต้นเหตุจริง: โมเดลจำรูปแบบ output จากตัวอย่างเทรนได้ แล้วผลิตซ้ำจากความจำแทนที่จะดูรูปตรงหน้าจริงๆ เป็นการท่องจำ/ หลอน ยิ่งซ้ำเติมด้วยรูปที่ถูกยอจนคานแทบไม่ปรากฏให้อ่านได้

เสียหายอะไร: plan_beam recall ค้างที่ 0% และรหัสบ้านที่ผิดใน output จะทำให้ข้อมูลปลายทางเพี้ยนเสียๆ ถ้ามี pipeline โหนดเชื่อฟิวด์ header นั้น

แก้ง่าย: หาต้นเหตุจริงได้ว่าเป็นปัญหา 'มองไม่เห็นวัตถุ' ไม่ใช่ 'อ่านตัวเลขผิด' นำไปสู่เครื่องมือ pattern recognition และการแก้ MAX_PIXELS ส่วนตัว header ที่หลอนยังไม่มีตัวกัน

กฎที่ได้: เวลาโมเดลพ่นเรคคอร์ดที่รูปแบบถูกแต่ว่างเปล่าหรือไม่ตรงไฟล์ ให้มองว่าเป็นการท่องจำ template ไม่ใช่บักการ parse และเพิ่มตัวตรวจว่า identity ใน header ตรงกับไฟล์ที่ป้อนเข้าไปจริง

สูง 2026-08-25

ความสามารถอ่านภาษาไทยไม่เคยเป็นเกณฑ์คัดกรองในการหาโมเดล — สองตัวที่รอดมายังไม่มีใครทดสอบภาษาไทย

เกิดอะไรขึ้น: คัดกรองโมเดล 8 ตัวด้วยขนาด โลเซนส์ และความง่ายในการเทรน พอเหลือ shortlist สองตัว (GLM-4.6V, Llama-4-Scout) ไดอารี่ถึงเพ็งบันทึกว่ายังไม่มีใครทดสอบว่าทั้งสองตัวอ่านภาษาไทยได้ดีแค่ไหน ทั้งที่ input ทั้งคลังของโปรเจกต์เป็นแบบ ก่อสร้างภาษาไทย

ต้นเหตุจริง: ความสามารถภาษาไทยเป็นเกณฑ์ที่ตรวจยากที่สุด (ไม่มี leaderboard ต้องลองรันจริง) เลยถูกเลื่อนไว้ก่อน แล้วการเลื่อนนั้นก็แปลว่ามันไม่เคยคัดอะไรออกเลย

เสียหายอะไร: โมเดลที่รอดมาทั้งสองตัวอาจตกม้าตายที่ข้อกำหนดเดียวที่หาทางเลี่ยงไม่ได้ ทำให้ shortlist ของงานสำรวจทั้งหมดเป็นแค่ของชั่วคราว ถ้าเลือกไปใช้เลย เสี่ยงเสียรอบเทรนทั้งรอบไปกับการค้นพบว่าโมเดลอ่านภาษาไทยไม่ออก

แก้ง่าย: ยังไม่ได้แก้ — บันทึกไว้เป็นข้อที่ยังไม่รู้

กฎที่ได้: ข้อกำหนดที่ตัดผู้สมัครออกได้ทันที ต้องอยู่ในตัวกรองด่านแรก ไม่ใช่เชิงอรรถท้ายเรื่อง shortlist ที่คัดมาครบทุกข้อ ยกเว้นข้อที่ซ้ำซ้อน ไม่นับเป็น shortlist

สูง 2026-08-26

InternVL3-78B กับ InternVL3.5-241B ดูเหมือนเป็นทางเลือกที่ไม่ใช่ Qwen แต่จริงๆ ใช้ Qwen เป็นแกนภาษา

เกิดอะไรขึ้น: เกณฑ์การคัดหาระบุชัดว่า 'ไม่ใช่ Qwen ทั้งแบรนด์และแกนโมเดล' แต่ InternVL3-78B กับ InternVL3.5-241B ผ่านเกณฑ์แบรนด์ พอตรวจดูกลับพบว่าใช้ Qwen2.5-72B และ Qwen3-235B ตามลำดับเป็นสมอง จึงถูกตัดสิทธิ์ทั้งคู่

ต้นเหตุจริง: แรนด์ของโมเดลไม่ได้บอกสายพันธุ์สถาปัตยกรรมที่แท้จริง ผู้ผลิตคนละเจ้าอาจใช้แกนโมเดล (backbone) ตัวเดียวกับที่กำลังพยายามหลีกเลี่ยงอยู่ก็ได้ ต้องอ่านส่วนสถาปัตยกรรมในการ์ดโมเดลเท่านั้นถึงจะรู้

เสียหายอะไร: ถ้าไม่จับได้ จะเลือกทางเลือกที่ดูเหมือนต่างแต่จริงๆ แล้วแบก dependency และจุดพึ่งพิงเดียวกันเป๊ะกับตัวเดิม ซึ่งทำลายจุดประสงค์ทั้งหมดของการค้นหาครั้งนี้

แก้ง่าย: ตัดสิทธิ์ทั้งสองตัวอย่างชัดเจนในตารางรวมโมเดล พร้อมบันทึกเหตุผลไว้ ('ข้างในใช้ Qwen2.5-72B เป็นสมอง')

กฎที่ได้: เวลาประเมินโมเดลเพื่อหลีกเลี่ยง dependency ตัวใดตัวหนึ่ง ให้เช็ค backbone จริงในส่วนสถาปัตยกรรม อย่าดูจากชื่อแบรนด์เด็ดขาด และควรบันทึก backbone เป็นคอลัมน์หลักในตารางเปรียบเทียบด้วย

โมเดลที่เข้ารอบชิงหลายตัวมีสัญญาอนุญาตที่ห้ามหรือจำกัดการใช้เชิงพาณิชย์อย่างคลุมเครือ

เกิดอะไรขึ้น: Pixtral-Large ใช้สัญญา Mistral Research License ห้ามใช้เชิงพาณิชย์โดยตรง (ต้องซื้อสัญญาแยกต่างหาก), MiniMax M3 มีเงื่อนไขซ้อนกันจนแม้แต่การ fine-tune หรือ deploy เองก็อาจนับเป็นเชิงพาณิชย์ได้, NVLM-D-72B เป็น CC-BY-NC-4.0

ต้นเหตุจริง: การเปิดให้ดาวน์โหลด weight ได้ถูกเข้าใจปนกับสัญญาแบบเปิดอยู่บ่อยๆ ข้อจำกัดจริงๆ อยู่ในเนื้อหาสัญญา ไม่ใช่การที่ดาวน์โหลดได้

เสียหายอะไร: โมเดลตัวไหนก็ตามในนี้ ถ้านำมาใช้โดยไม่อ่านสัญญาก่อน จะสร้างความเสี่ยงทางกฎหมายที่จะโดนพบกฟ้องเมื่อโมเดลถูกฝังลงในผลิตภัณฑ์เชิงพาณิชย์ไปแล้ว

แก้ยังไง: อ่านสัญญาเต็มของทุกตัวที่เข้ารอบ และบันทึกข้อจำกัดไว้เป็นรายการในตารางรวม พร้อมระบุผลชัดเจนว่าใช้เชิงพาณิชย์ได้



กฎที่ได้: อ่านเนื้อหาสัญญาเต็มของทุกโมเดลที่เข้ารอบก่อนใช้เงินใดๆ และบันทึกผลการใช้เชิงพาณิชย์เป็นคอลัมน์บังคับ 'ดาวน์โหลดได้' ไม่เคยแปลว่า 'ใช้เชิงพาณิชย์ได้'

เลือก Llama-4-Scout เป็นผู้รอดชีวิตตัวเดียวโดยไม่มีใครทดสอบว่ามันอ่านภาษาไทยได้หรือเปล่า

เกิดอะไรขึ้น: Llama-4-Scout ผ่านทุกเกณฑ์ในตารางรวม (ขนาด, สัญญา, ยืนยันเทรน LoRA ได้) และถูกเลือกเป็นผู้รอดชีวิตตัวเดียว แต่ตารางรวมเองก็บันทึกไว้ว่ายังไม่มีใครทดสอบเลยว่าโมเดลตัวไหนอ่านภาษาไทยได้ดีแค่ไหน และแนะนำให้ทดสอบ zero-shot Thai-OCR ก่อนเข้า GPU จริง

ต้นเหตุจริง: เกณฑ์การคัดเลือกครอบคลุมขนาด, สัญญา และเครื่องมือเทรน แต่ข้ามความสามารถที่สำคัญที่สุดของงานไป นั่นคือการอ่านแบบก่อสร้างภาษาไทย เกณฑ์ที่ใช้เป็นเกณฑ์ความเป็นไปได้ ไม่ใช่ความเหมาะสมกับงาน

เสียหายอะไร: เกือบตัดสินใจเข้า GPU ให้กับโมเดลที่ยังไม่เคยวัดความสามารถหลักที่งานต้องการเลยแม้แต่คนเดียว — เป็นความผิดพลาดคลาสเดียวกับการเทรน t04 โดยไม่เช็คความละเอียดภาพก่อน

แก้ยังไง: ยังไม่ได้แก้ — ทำเป็นข้อเสนอแนะ ('แนะนำ zero-shot Thai-OCR test ก่อนตัดสินใจลงทุนเข้า GPU จริง') แทนที่จะลงมือทำจริง

กฎที่ได้: ใส่ความสามารถที่สำคัญต้องงานเข้าไปในเกณฑ์คัดเลือกโดยตรง และทดสอบ zero-shot พรึ่ก่อนตัดสินใจเข้า เกณฑ์ความเป็นไปได้ (ขนาด/สัญญา/เครื่องมือ) ไม่ได้วัดว่าโมเดลทำงานได้จริงหรือเปล่า

สูง 2026-08-29

บล็อก CV hint ต้องห้ามใส่ตัวเลขมิติเด็ดขาด ไม่งั้นจะสอนโมเดลให้ทำผิดกฎของตัวเอง

เกิดอะไรขึ้น: สัญญาของ pass 2.4 hint ห้ามไว้ชัดเจนว่าห้ามใส่ตัวเลขมิติใดๆ เข้าไปในบล็อก hint ที่ต่อท้าย prompt เพราะการทำแบบนั้นจะเท่ากับสอนให้โมเดลผิดกฎหลักที่ว่า 'ห้ามเดามิติจากภาพเด็ดขาด' ด้วยมือของตัวเอง

ต้นเหตุจริง: ช่องทางเสริม (CV hint) สามารถขัดแย้งกับกฎใน prompt ได้แบบเสียบๆ ถ้าเนื้อหาของมันไม่ถูกควบคุม ข้อมูลช่วยเหลือที่แทรกเข้า prompt คือสัญญาณการเทรน ไม่ใช่บริบทที่เป็นกลาง

เสียหายอะไร: จับได้ตั้งแต่ตอนออกแบบเป็นกฎในสัญญา ก่อนที่จะปล่อยจริง จึงยังไม่มีความเสียหายที่วัดได้ — แต่ถ้าเกิดจริง จะเป็นปัญหาโมเดลแต่งมิติมาทั้งระบบซึ่งตามหาต้นเหตุนานมาก

แก้ยังไง: เขียนเป็นข้อห้ามชัดเจนไว้ใน pass2.4_hint/prompt.md ว่าห้ามใส่อะไรลงในบล็อก hint

กฎที่ได้: เมื่อแทรกข้อมูลเสริม/ที่มาจากเครื่องมือลงใน prompt ต้องระบุชัดเจนว่าห้ามมีอะไรอยู่ในนั้นบ้าง บริบทเสริมที่ขัดแย้งกับกฎที่ระบุไว้ จะเอาชนะกฎนั้นได้

สูง 2026-08-30

fixture ทดสอบการนำเข้าในโค้ดใช้ชื่อ pattern ที่ไม่มีในไฟล์จริงเลยสักไฟล์ ทำให้ test suite ผ่านตลอดขณะที่ path นำเข้าจริงไม่ได้อะไรเลย

เกิดอะไรขึ้น: `raw-extraction-adapter.js` รู้จักแค่ชื่อ pattern ตามสเปกคือ `plan` กับ `gridline` เท่านั้น แต่ข้อมูลจริงทั้ง 50 หลังใช้ `beam\_plan` (85), `footing\_plan` (60), `etc\_plan` (199), `roof\_frame\_plan` (46) และ `grid\_master` (42) — `plan`/`gridline` ไม่มีอยู่ในไฟล์จริงแม้แต่ไฟล์เดียว บักนี้อยู่อรอดมานานเพราะ fixture ในโค้ดถูกเขียนด้วยชื่อตามสเปกเช่นกัน ทุก test จึงผ่านโดยทดสอบ path ที่ไม่มีไฟล์จริงไฟล์ไหนเดินผ่านเลย จับได้ก็ตอนที่ลองรันบ้านจริงผ่าน pipeline แบบจำลองผู้ใช้จริง

ต้นเหตุจริง: fixture ถูกเขียนตามเอกสารสเปกแทนที่จะสุ่มมาจากชุดข้อมูลจริง ทำให้ test suite ตรวจสอบ adapter กับสมมติฐานของตัวเอง ชื่อในสเปกกับชื่อที่เกิดขึ้นจริงเบี่ยงเบนออกจากกันโดยไม่มีอะไรมาเทียบเลย

เสียหายอะไร: กดปุ่มนำเข้าข้อมูลจริงจะได้ element 0 ตัวและไม่มี grid เลยแบบเสียบๆ ไม่รู้ว่านานแค่ไหนแล้ว — path การนำเข้าด้วยมือก็ได้รับผลกระทบด้วย ไม่ใช่แค่ path ใหม่ของ Purson ขณะที่ test 1149 ตัวยังเชียวตลอด

แก้ยังไง: เพิ่ม `canonicalPattern()` + `PATTERN\_ALIASES` เข้าใน adapter ใช้กับจุดเปรียบเทียบทั้ง 5 จุดใน 2 ไฟล์ รายชื่อ alias จับคู่ให้ตรงกับ build_dataset_t03.py และเพิ่ม `tests/raw-extraction-pattern-aliases.test.js` (4 test) suite รวมเป็น 1153/1153

กฎที่ได้: fixture ที่คิดขึ้นจากสเปกทดสอบแค่สเปก ไม่ได้ทดสอบระบบจริง ต้องมี test อย่างน้อย 1 ตัวต่อ path การนำเข้าที่ใช้ตัวอย่างข้อมูลจริงจากชุดข้อมูลจริงเป๊ะๆ ไม่งั้น test suite ที่เชียวก็ไม่มีความหมายอะไรเลย

สูง 2026-08-31

โมเดลพ่นชื่อไฟล์จาก training set ออกมาตอนอ่านบ้านที่ไม่เคยเห็นมาก่อน — จำข้อมูลแทนที่จะเรียนรู้ (overfit)

เกิดอะไรขึ้น: ผลลัพธ์สำหรับบ้านที่ไม่เคยเห็นเลยมีชื่อไฟล์อย่าง house_001_หน้า00_gridline.json และ image/house_001/... โฟลเดอร์ ซึ่งเป็นชื่อที่จำมาจาก training corpus loss สุดท้ายอยู่ที่ 0.10-0.13 ซึ่งต่ำกว่าเกณฑ์ overfit ที่ Unsloth ระบุไว้ที่ 0.2

ต้นเหตุจริง: จำนวน epoch เยอะเกินไป และความหลากหลายของบ้านในข้อมูลน้อยเกินไปเมื่อเทียบกับความจุของโมเดล ทำให้เกิดการจำแทนที่จะเรียนรู้จริง

เสียหายอะไร: ผลลัพธ์สำหรับบ้านที่ไม่เคยเห็นปนเปื้อนไปด้วยข้อมูลอ้างอิงที่แต่งขึ้นจาก training data ซึ่งจะทำให้ระบบที่นำผลไปใช้ต่อพังตามไปด้วย

แก้ยังง: ยังไม่ได้แก้ในรอบนี้ — บันทึกไว้เป็นสิ่งที่ต้องแก้ในรอบถัดไป (ลด epoch เพิ่มความหลากหลายของบ้าน)

กฎที่ได้: ต้องติดตาม training loss เทียบกับเกณฑ์ overfit ระหว่างการเทรน และต้องทดสอบกับบ้านที่ไม่เคยอยู่ใน corpus จริงๆ ก่อนจะประกาศว่าโมเดลใช้งานได้ — ชื่อไฟล์ที่ถูกจำมาโผล่ในผลลัพธ์คือสัญญาณ overfit ที่ชัดเจนมาก

สูง 2026-08-31

แผนผังสำเร็จรูป (SP) ทั้ง 6 แผนหายไปทั้งหมดจากแบบคาน

เกิดอะไรขึ้น: หน้าแบบคานที่ให้คะแนน โมเดลไม่ผลิต element ประเภท SP ออกมาเลยแม้แต่ตัวเดียว — แผนผังสำเร็จรูปทั้ง 6 แผนหายไปหมด พร้อมข้อผิดพลาดอื่นๆ (แถว B ทุกตัวถูกติดป้ายเป็น B2 ทั้งที่ควรมี B3 ด้วย, B3 วางผิดทิศทาง, B0 ถูกจัดเป็นพื้นที่จริงเป็นคาน)

ต้นเหตุจริง: ยังไม่ทราบสาเหตุแน่ชัด — ถูกตั้งข้อสังเกตว่าอาจเป็นช่องโหว่เชิงระบบ ไม่ใช่แค่ความผิดพลาดสุ่ม เพราะทั้ง element class หายไปทั้งหมด ไม่ใช่หายไปแค่บางตัว

เสียหายอะไร: แผนผังสำเร็จรูปจำเป็นต้องใช้ในการทำ BOQ ถ้า element ทั้งประเภทหายไปหมด ปริมาณงานจะขาดหายไปแบบเสียๆ

แก้ยังง: ยังไม่ได้แก้ — ระบุไว้เป็นสิ่งที่ต้องทำลำดับความสำคัญรองลงมา คือต้องสืบว่าเป็นช่องโหว่เชิงระบบจริงหรือไม่

กฎที่ได้: เมื่อ element ทั้งประเภทหายไปทั้งหมด ไม่ใช่หายไปแค่บางตัว ต้องถือว่าเป็นช่องโหว่เชิงระบบและตรวจสอบก่อนนำไปใช้จริง — ค่าศูนย์แบบเสียๆ ในสิ่งที่เกี่ยวกับ BOQ อันตรายกว่าค่าที่ผิดแต่ยังมีตัวเลข

สูง 2026-08-31

โมเดลตอบว่าไม่มี element เลยสักตัว พร้อม confidence เต็ม 1.0 ทั้งที่หน้านั้นมีฐานราก 6 ตัวเห็นชัดๆ

เกิดอะไรขึ้น: ในรอบ greedy decoding หน้า 26 (ผังฐานราก/pile-cap) โมเดลตอบ elements: [] พร้อม confidence 1.0 แถมยังติดธงว่า "มีสัญลักษณ์ฐานรากแต่ไม่มีฐานรากถูกวาด" ทั้งที่ในภาพมีฐานราก 6 ตัวเห็นชัดเจน

ต้นเหตุจริง: ไม่มีอะไรใน schema หรือ pipeline ที่ผูกค่า confidence เข้ากับหลักฐานจริง โมเดลจึงอ้าง confidence เต็มกับผลลัพธ์ที่ว่างเปล่าได้อย่างอิสระ และไม่มีกฎตรวจสอบว่า "ว่างเปล่า+มั่นใจเต็ม" คือสัญญาณผิดปกติ

เสียหายอะไร: หน้าที่ตอบว่างเปล่าอย่างมั่นใจอันตรายกว่าหน้าที่ตอบไม่รู้เสียอีก — ถ้าปล่อยผ่านไป ระบบปลายทางจะเข้าใจว่า "แบบนี้ไม่มีฐานราก" แล้วทำ takeoff ตกหล่นฐานรากจริง 6 ตัว

แก้ยังง: ยังไม่ได้แก้ตัวช่อง confidence โดยตรง แต่ต้นเหตุของความว่างเปล่าถูกแก้แล้วด้วยการปิด greedy decoding (พอเปลี่ยนวิธี decode หน้า 26 อ่านถูก 100%)

กฎที่ได้: ให้ถือว่า confidence สูงบนผลลัพธ์ว่างเปล่าคือสัญญาณว่า validation ล้มเหลว ไม่ใช่ข้อมูลที่เชื่อถือได้ — โมเดลที่บอก "ไม่มีอะไรเลย มั่นใจมาก" ต้องมีตัวกันไว้เสมอ

สูง 2026-08-31

โมเดลเดาตำแหน่งเสาจากจุดตัดของคาน ทั้งที่กติกาห้ามเดาไว้ชัดเจน

เกิดอะไรขึ้น: ในเอาต์พุตหน้า 27 โมเดลใส่เสาพร้อมคอมเมนต์ของตัวเองว่า "ตำแหน่งเสาอนุมานจากจุดตัดของคาน" คือคำนวณตำแหน่ง element เอง ไม่ได้อ่านจากแบบจริง ซึ่งขัดกับกติกาห้ามเดาที่เขียนไว้ในพรมต์

ต้นเหตุจริง: กติกาห้ามเดามีแค่ในพรมต์ แต่ไม่มีอะไรตรวจจับหรือปฏิเสธ element ที่ถูกอนุมานเลยทั้งตอน generate และตอน ingest — แม้โมเดลจะประกาศเองว่าจะเมดกติกาที่ยังหลุดผ่านไปได้

เสียหายอะไร: structural element ที่แต่งขึ้นเองจะเข้าไปอยู่ใน takeoff เสมือนเป็นข้อมูลที่อ่านจากแบบจริง จับได้เฉพาะตอนอ่านเอาต์พุตด้วยมือเท่านั้น

แก้ยังไง: ยังไม่ได้แก้ — ไม่มีตัวตรวจที่ปฏิเสธการอนุมานที่โมเดลประกาศเองแบบนี้

กฎที่ได้: เมื่อโมเดลบอกตรงๆ ว่ากำลังเดา นั่นคือสัญญาณที่เครื่องอ่านได้ — ควรสร้างตัวตรวจจับสัญญาณนี้ แทนที่จะพึ่งกติกาในพรมต์อย่างเดียว

สูง 2026-09-01

t04 ทองคำคำตอบจากชุดฝึกแทนที่จะเรียนรู้อ่านแบบจริง แต่กลับถูกเข้าใจผิดว่าเก่ง

เกิดอะไรขึ้น: เพราะภาพอินพุตอ่านไม่ออก โมเดลเลยเรียนรู้ได้แค่ "รูปร่างของคำตอบที่ควรจะเป็น" และคำตอบของบ้านที่เคยเห็นตอนฝึกได้ พอเจอบ้านที่ไม่เคยเห็นก็แต่งคำตอบขึ้นเองทันที

ต้นเหตุจริง: อินพุตที่อ่านไม่ออก บวกกับสัญญาณ loss ที่ยังให้รางวัลกับ JSON หน้าตาถูกต้อง ทำให้การทองคำเป็นกลยุทธ์เดียวที่เรียนรู้ได้ ระบบฝึกทำให้การโกงกลายเป็นทางเลือกที่ดีที่สุด

เสียหายอะไร: ได้โมเดลทั้งตัวที่ความสามารถที่ดูเหมือนมีจริง ๆ คือการจำชุดฝึกล้วนๆ — ใช้กับบ้านใหม่ไม่ได้เลย ทั้งที่บ้านใหม่คือกรณีเดียวที่มีความหมายในการใช้งานจริง

แก้ยังไง: ตรวจพบด้วยการทดสอบกับบ้านที่ไม่เคยเห็นมาก่อน และดูเอาต์พุตจริงแทนที่จะดูแค่คะแนนรวม

กฎที่ได้: ต้องกันข้อมูลชุดที่ไม่เคยเห็นไว้ทดสอบเสมอ และเช็คผลเฉพาะกับชุดนั้น ถ้าโมเดลได้คะแนนดีกับข้อมูลที่เคยเห็นแต่พังกับข้อมูลใหม่ ให้สงสัยไว้ก่อนว่าอินพุตใช้งานไม่ได้และโมเดลกำลังทองคำคำตอบ ไม่ใช่เรียนรู้จากจริง

สูง 2026-09-01

การแบ่ง k-fold ของ t04 ไม่ได้เช็คว่แต่ละ fold มีข้อมูลครบทุก pass ทำให้ทุก fold ไม่มีบ้านสำหรับประเมิน pass0 เลย

เกิดอะไรขึ้น: แบ่งบ้าน 40 หลังเป็น fold โดยไม่เช็คว่ามีแค่ประมาณ 10 หลังที่มีข้อมูล pass0/pass2.4/pass3 (pass0 มีแค่ 5 หลังเท่านั้น) การแบ่งแบบสุ่มดันบ้านพิเศษเหล่านี้เกือบทั้งหมดไปอยู่ฝั่งเทรน พอนับไฟล์จริงพบว่าทุก fold มีบ้านสำหรับประเมิน pass1 ครบ 204 หลัง แต่มีบ้านสำหรับประเมิน pass0 เป็น 0 หลัง และ pass2.4 มีแค่ 0–9 หลัง pass3 มีแค่ 0–2 หลัง

ต้นเหตุจริง: แบ่งแบบ stratify ตามจำนวนบ้านเท่านั้น เหมือนถือว่าทุกบ้านเท่ากันหมด ทั้งที่จริงๆ ข้อมูลกระจายไม่สม่ำเสมอและมีน้อยมากในบาง pass ไม่มีใครนับความครอบคลุมของข้อมูลต่อ pass ก่อนแบ่ง

เสียหายอะไร: ต่อให้ฝึกสำเร็จก็จะวัดไม่ได้เลยว่าโมเดลเรียนรู้ pass0 หรือ pass3 หรือไม่ ถ้าเอา fold ชุดนี้ไปใช้ต่อ จะฝึกไปแล้วก็ยังไม่รู้คำตอบอยู่ดี

แก้ยังไง: มะขามจับได้โดยตรง ('อย่าเอา fold ของ t04 ไปใช้') และยืนยันด้วยการนับไฟล์จริง จึงปฏิเสธไม่ใช่โครงสร้าง fold ของ t04 ซ้ำ

กฎที่ได้: ก่อนแบ่งข้อมูลเป็น fold ให้นับความครอบคลุมต่อ sub-task ก่อน แล้ว stratify ตามมิติที่ข้อมูลน้อยที่สุด ไม่ใช่แค่จำนวนรายการรวม ต้องตรวจให้แน่ใจว่าทุก fold มีตัวอย่างสำหรับประเมินทุกความสามารถที่ต้องการวัด ไม่เป็นศูนย์

สูง 2026-09-01

เกือบรวมโมเดลแบบ model-soup โดยไม่มี LoRA A initialisation ร่วมกัน (เงื่อนไข same-basin)

เกิดอะไรขึ้น: แผนถั่วเฉลี่ยโมเดล 4 fold เข้าด้วยกันเป็น 'model soup' ที่น้ำหนัก 0.25 เท่ากันทุกตัว จะได้ผลก็ต่อเมื่อทุกโมเดลเริ่มจากจุดตั้งต้นเดียวกันเท่านั้น ทีมเกือบรวมโมเดลที่ไม่ได้ใช้ LoRA A init ร่วมกัน — บันทึกไว้เป็นเหตุการณ์เกือบพลาดในไดอารี่วันที่ 30 ส.ค.

ต้นเหตุจริง: เงื่อนไข same-basin สำหรับการถั่วเฉลี่ยน้ำหนักเป็นข้อกำหนดทางคณิตศาสตร์ที่ไม่มีเครื่องมือไหนบังคับให้ ถ้ารวมโมเดลจากจุดตั้งต้นต่างกัน ไฟล์ที่ได้จะดูปกติแต่ไม่มีความหมายทางตัวเลขเลย

เสียหายอะไร: ถ้าไม่จับได้ จะได้โมเดลรวมที่ดูเหมือนปกติแต่จริงๆ เป็นสัญญาณรบกวน (noise) โดยไม่มี error ให้เห็นในขั้นตอนไหนเลย

แก้ง่าย: จับได้ก่อนรวมโมเดล บันทึกเงื่อนไข LoRA A init ร่วมกันไว้เป็นบทเรียนในไดอารี่วันที่ 30 ส.ค.

กฎที่ได้: ก่อนถั่วเฉลี่ยน้ำหนักโมเดลชุดไหนก็ตาม ต้องตรวจสอบด้วยโปรแกรมว่าทุกตัวใช้จุดตั้งต้นเดียวกันจริง การรวมน้ำหนักโมเดลมีเงื่อนไขที่ไม่แสดงตัวเอง — ต้อง assert ไว้ ห้ามสมมติเอาเองว่าสคริปต์ฝึกกรูธชันนี้ไว้ให้แล้ว

สูง 2026-09-02

หน้าที่มีทั้งแปลนฐานรากและแปลนคานรวมอยู่ในแผ่นเดียว generate ไม่สำเร็จทั้งสองครั้ง แม้จะเพิ่ม timeout เป็น 40 นาทีแล้ว

เกิดอะไรขึ้น: แม้จะเพิ่ม timeout เป็น 40 นาทีแล้ว หน้า 12 (มีทั้งแปลนฐานรากและคานในแผ่นเดียว) ก็ยังพังทั้งสองครั้งที่ลอง พออ่านข้อความดิบพบว่ามีความล้มเหลวคนละแบบ — ส่วนฐานราก โมเดลสร้างครบทั้ง 7 element ถูกต้องแล้วแต่กลับวนพูดประโยคอธิบายเดิมซ้ำไม่มีที่สิ้นสุด ส่วนคานโมเดลยังสร้างเนื้อหาจริงอยู่ (64 ส่วน) แต่ถูกตัดก่อนจะจบแค่ 144 ตัวอักษร

ต้นเหตุจริง: ความหนาแน่นของเนื้อหาบนแผ่นเดียวกินกว่าที่การ generate หนึ่งครั้งจะทำเสร็จได้ ปัญหาสองแบบถูกปนกันโดยไม่รู้ตัว — วนซ้ำแบบไม่จบ (เพิ่มเวลาแก้ไขไม่ได้) กับข้อความยาวเกินจนถูกตัด (เพิ่มเวลาแก้ไขได้แค่เนียดๆ) ไม่มีตัวกันการวนซ้ำ (repetition penalty / loop detection) และไม่มีการแบ่งเนื้อหาต่อแผ่นเลย

เสียหายอะไร: พังซ้ำในหลายหลัง (บ้าน09 หน้า 9 ก็เจอแบบเดียวกัน) แต่ละครั้งต้องรันใหม่หรือกู้คืนเอง ไม่ใช่ปัญหาที่แก้ด้วยการปรับ timeout

แก้ง่าย: กู้คืนได้ด้วย `salvage\_json.py` ที่เขียนขึ้นเอง (ตัววิ่งไล่ตาม bracket-depth หาจุดตัดที่ปลอดภัยแล้วปิด JSON โดยไม่แต่งค่าใหม่) กู้คืนได้ครบทั้งสองไฟล์

กฎที่ได้: ต้องแยกให้ออกระหว่าง 'เวลาหมด' กับ 'ติดวนซ้ำ' — ต้องแก้คนละแบบ (แบ่งเนื้อหา vs. ใส่ตัวกันการวนซ้ำ/เงื่อนไขหยุด) สำหรับแผ่นที่มีหลาย view หนาแน่น ควร crop แยกแต่ละ view ก่อน generate ดีกว่าไปเพิ่ม timeout อีก

สูง 2026-09-02

บ้าน09 หน้า 9 (แปลนคาน + โครงหลังคาซ้อนทับกัน) ถูกตัดข้อความอีกครั้ง — ปัญหาแผ่นหนาแน่นเดิมเกิดซ้ำ

เกิดอะไรขึ้น: ระหว่างรันบ้าน09 หน้า 9 (แปลนคานบวกโครงหลังคาซ้อนสอง view) ถูกตัดข้อความกลาง generate — เหมือนกับปัญหาของบ้าน2 หน้า 12 ทุกประการ แต่เกิดกับบ้านคนละหลัง

ต้นเหตุจริง: ปัญหาแผ่นหนาแน่นหลาย view ไม่เคยถูกแก้ที่ต้นเหตุเลย (ไม่มีการ crop) มีแค่การกู้คืนให้รอดพอไปต่อได้ จึงเกิดซ้ำในบ้านหลังถัดไปทันที

เสียหายอะไร: ต้องกู้คืนด้วยมืออีกครั้งในงานที่ตั้งใจจะจับเวลาแบบ end-to-end สะอาดๆ ต้องกู้คืน element 22 ตัวด้วยสคริปต์

แก้ง่าย: กู้คืนด้วย `salvage\_json.py` (ครบทั้ง 22 element)

กฎที่ได้: ปัญหาที่เกิดซ้ำในชุดข้อมูลถัดไปไม่ใช่ edge case แล้ว — มันคือกรณีปกติ ต้องแก้แผ่นที่มีหลาย view หนาแน่นตั้งแต่ขั้น crop ก่อนจะรันบ้านเพิ่มเข้ากับดักเดิมอีก

เรื่องที่เหลือในหมวดนี้ (23)

พยายาม quantize จากไฟล์ GGUF ที่ quantize ไปแล้ว ต้องกลับไปแปลงใหม่จาก f16

กลาง 2026-07-21

ต้องเก็บและแปลงจากไฟล์ export ความละเอียดเต็มเสมอ ห้ามเอา quantization หนึ่งไปสร้างอีกอันต่อ

เกือบใช้ prompt ของแอป production ทั้งที่มันเล็งไปที่ schema คนละแบบกับที่โมเดลถูกฝึกมา

กลาง 2026-07-25

ต้อง pin prompt ที่ใช้ฝึกโมเดลจริงไว้คู่กับตัวโมเดลนั้นเลย เมื่อมี prompt หลายตระกูลสำหรับหลาย schema การเลือก prompt โดยไม่มี label กำกับคือบั๊กที่ทำให้ผลผิดแบบเสียบๆ

วางแผนเอาโมเดลที่เทรนแล้วไปรันกับบ้านหลังใหญ่ ทั้งที่ไม่มีบ้านประเภทนี้ในชุดเทรนเลย

กลาง 2026-07-25

บอกให้ชัดตั้งแต่แรกว่าโมเดลครอบคลุมข้อมูลแบบไหนบ้าง ก่อนที่ใครจะเอาไปรันกับข้อมูลใหม่ และตอนวางแผนเก็บชุดข้อมูล ให้ดูจากประเภทอาคารที่ตั้งใจจะใช้จริง ไม่ใช่เอาแค่อะไรที่บังเอิญ label เสร็จก่อน

เสนอให้ยัดเอกสาร schema ทั้งหมด 6,000 token เข้าไปใน prompt ตอน inference

กลาง 2026-07-25

สำหรับโมเดลที่ fine-tune มาแล้ว prompt เป็นส่วนหนึ่งของตัวโมเดล ถ้า prompt ตอนใช้งานต่างจากตอนเทรน ไม่ว่าจะยาวกว่าหรือสั้นกว่า ก็เท่ากับทิ้งผลของการ fine-tune ไปบางส่วน

เสียเวลาไปลอง SAM 3 / Grounding DINO / YOLO-World ก่อนจะรู้ว่ามันเป็นโมเดลผิดประเภท

กลาง 2026-08-04

เขียนโจทย์ออกมาเป็นสัญญา input→output ก่อนจะไปคัดเลือกโมเดล ถ้าสิ่งที่ต้องได้ออกมาคือรหัสที่ต้องไป join กับค่าในตาราง นั่นไม่ใช่โจทย์ตรวจจับวัตถุ ต่อให้มันดูเป็นงานภาพแค่ไหนก็ตาม

บั๊กที่ 1 ของ t02: คำนวณงบ token เร็วเกินไป โดยอิงตัวเลขที่หลังจากนั้นก็เปลี่ยน

กลาง 2026-08-04

อย่าเพิ่งคำนวณตัวเลขในแผนที่ได้มาจากค่าอื่น จนกว่าค่าตั้งต้นจะนิ่งแล้ว จัดลำดับให้การคิดงบและขนาดมาหลังจากชุดข้อมูลสรุปแล้ว ไม่ใช่ก่อน

โค้ดที่ปรับให้เร็วขึ้นด้วย text layer ของ PDF ไม่เคยทำงานเลย เพราะ PDF ทุกไฟล์เป็นเส้น vector ไม่มีตัวอักษรสักตัว

กลาง 2026-08-04

ก่อนสร้างโค้ดปรับความเร็วบนข้อสันนิษฐานเรื่องรูปแบบไฟล์ ต้องเอาไฟล์จริงมาลองก่อนว่าจริงไหม แล้วใส่ log ไว้ด้วยว่ากึ่งที่ปรับความเร็วนั้นได้ทำงานจริงหรือเปล่า ของที่ประหยัดได้แต่ไม่มีใครวัด ก็เท่ากับไม่ได้ประหยัดอะไรเลย

pattern สามคลาสมีตัวอย่างน้อยมาก แค่ 13, 12 และ 5 ไฟล์จากบ้าน 12 หลัง

กลาง 2026-08-04

ก่อนตัดสินใจว่าจะเทรนแยกเป็นรายคลาส ให้นับตัวอย่างต่อคลาสก่อน คลาสที่มีแค่ไฟล์เดียวต่อบ้าน ต้องวัดด้วยจำนวนบ้าน ไม่ใช่จำนวนหน้า ถึงจะรู้ว่าพอเทรนได้หรือยัง

Pass 3 จนไม่ได้ เพราะ schema เก็บคานแยกทีละช่วง แต่ Pass 3 ต้องการคานที่จัดกลุ่มตามความยาว

กลาง 2026-08-04

ก่อนวางแผนจน pass ไหน ให้เช็คว่าสร้าง dataset ในรูปแบบผลลัพธ์ของ pass นั้นได้จริงจากข้อมูลที่เก็บไว้หรือเปล่า ถ้ารูปแบบไม่ตรงกัน นี่คือการ dataset ไม่ใช้งานเทรน

โมเดลตัดตัวอักษรนำหน้าของ element_id ทั้งเป็นระบบในหน้าตาราง (D1 กลายเป็น 1)

กลาง 2026-08-20

ถ้าต้องคลายเกณฑ์การจับคู่ให้หลวมลง eval ถึงจะให้คะแนนได้ ความหลวมนั้นคือสิ่งที่ค้นพบในตัวมันเอง ให้บันทึกไว้ว่าเป็นข้อบกพร่องของโมเดล แล้วไปเช็คว่าข้อมูลเทรนสอนมันแบบนั้นหรือเปล่า อย่าไป normalize ทั้งเสียบๆ

โมเดลโนรายละเอียดไว้ในหน้า 16/17 ออกมาเป็นเสารากฐาน และคานาคอดิน

กลาง 2026-08-20

การสกัดข้อมูลรวดเร็วโดยไม่มีด้านคัดชนิดหน้า จะทำให้โมเดลแต่งโครงสร้างขึ้นมาบนแผนใดก็ตามที่ดูเหมือนงานโครงสร้าง ให้จำแนกชนิดหน้าก่อนสกัด และให้ถือว่าของที่มันขึ้นมาอันตรายกว่าของที่อ่านตกไป เพราะมันจะไหลไปคิดเงินใน BOQ

ยังมีหนึ่งหน้าที่พึ่งแม่ใช้ grammar-constrained decoding แล้ว โมเดลลวนลูปอยู่ข้างใน string

กลาง 2026-08-21

grammar-constrained decoding ปิดได้แค่ความพึงเชิงโครงสร้าง ให้เก็บกลไก parse แล้ว retry ไว้สำหรับที่เหลือ และอย่าคาดหวังว่า no_repeat_ngram จะจับลูปที่หมุนเปลี่ยน token ไปทีละตัวได้

ลองวิธี pattern recognition กับหน้า section ไป 3 วิธี สัมหมด แต่วิธีหนึ่งที่ดูเพอร์เฟกต์กับบ้านหลังแรกที่ทดสอบ

กลาง 2026-08-25

heuristic ทาง CV ที่ทำงานได้เพอร์เฟกต์กับตัวอย่างแรกยังไม่พิสูจน์อะไร ต้องทดสอบข้ามแหล่งที่มาจนถึงจะเชื่อได้ และให้จัดการทดลองที่ล้มเหลวพร้อมตัวเลขไว้ด้วย คนต่อไปจะได้ไม่เสียเวลาทั้งวันซ้ำ

NVLM-D-72B รันไม่ได้เลย แม้แต่ vLLM ก็เสิร์ฟไม่ได้ ซึ่ง NVIDIA เองก็ยอมรับ

กลาง 2026-08-26

ก่อนจะเอาโมเดลไหนเข้า shortlist ต้องยืนยันทั้ง licence ที่เปิดกว้าง และเส้นทางการ inference ที่ใช้งานได้จริง การมี weight ปลอ่ยออกมาไม่ได้พิสูจน์ทั้งสองอย่าง

glossary ไทย→อังกฤษอาจสอนโมเดลให้แปลป้ายที่ห้ามแปล ที่รอดมาได้เพราะตำแหน่งที่มันวางอยู่ใน prompt เท่านั้น

กลาง 2026-08-26

เมื่อบล็อกใหม่ใน prompt ชัดกับกฎที่มีอยู่เดิม การอาศัยว่ามันอยู่ใกล้กันคือความหวัง ไม่ใช่ตัวกัน ให้เพิ่มการตรวจแบบชัดเจนในการ eval รอบถัดไปว่า ฟิลด์ป้ายที่ต้องตรงตัวยังตรงกับ GT ที่ละตัวอักษรอยู่ไหม

อักขระ Markdown ที่ตกค้างอยู่ใน prompt ทำให้โมเดลหลอน

กลาง 2026-08-29

prompt คือ input ของโมเดล ไม่ใช่เอกสารให้คนอ่าน ลอกอักขระจัดรูปแบบออกก่อนส่งเสมอ และแยกฉบับที่คนอ่านออกจากข้อความ prompt ตัวจริงที่ใช้งาน

กฎข้อ 4 ของ prompt pass3 ห้ามคานแนวนอนใช้ cv_mark ร่วมกัน ซึ่งผิด

กลาง 2026-08-31

เขียนกฎที่คุยกับตัวตรวจจับแยกตามคลาสเรขาคณิตของชิ้นส่วน (จุด vs เส้น) และดูก่อนว่าตัวตรวจจับตีกรอบแต่ละคลาสยังงัย ก่อนจะเขียนเงื่อนไขว่าห้ามซ้ำ

ระบบ pass classify คาน B1/B2/B2A/B4 ผิดเป็นเสารากฐาน ทั้งที่การรันแบบ raw-power กลับตอบถูก

กลาง 2026-08-31

hint ที่ช่วยให้ subtask จับได้ครบขึ้น (recall) ก็อาจทำให้ classify เพี้ยนไปด้วย ต้องแยกเช็คความแม่นยำของการจำแนกประเภทชิ้นส่วน ออกจากค่า recall เวลาประเมิน pipeline ที่มี hint นำทาง

ตั้ง enable_thinking=False แล้วก็ยังหยุด chain-of-thought ไม่ให้รั่วเข้าไปปนกับคำตอบ JSON ไม่ได้

กลาง 2026-08-31

flag ที่ปิด channel หนึ่งไว้ ไม่ได้แปลว่าเนื้อหาจะไม่ปนมา ต้องตรวจสอบรูปแบบ output จริง ไม่ใช่เชื่อ flag เฉยๆ

ข้อผิดพลาดในการอ่านแบบ beam-plan มีมากกว่าแค่พื้นที่หาย — แถว B ถูกอ่านเป็น B2 ทั้งแถว, B3 ถูกวาดผิดแกน, B0 ถูกจำแนกเป็นพื้น

กลาง 2026-08-31

ต้องบันทึกรูปแบบข้อผิดพลาดเป็นรายรายการ ไม่ใช่แค่ตัวเลข recall รวม เพราะ 'แถว B เป็น B2 ทั้งแถว' กับ 'B3 อยู่ผิดแกน' ชี้ไปที่วิธีแก้การแทนคนละแบบกับที่คะแนนต่ำเพียงอย่างเดียวจะบอกได้

pass3 หากรอบยึด CV ไม่เจอเลยบนแบบเหล็ก C-channel — เป็นแบบทั้งสไตส์ที่วัดผลด้วยระบบนี้ไม่ได้เลย

กลาง 2026-09-02

ก่อนอ้างว่าขั้นตอนวัดผลใด 'ใช้งานได้' ต้องระบุให้ชัดว่ามันถูกสร้างจากแบบก่อสร้างสไตส์ไหน — คลังคำศัพท์ CV ที่เทรนมาจากสไตส์หนึ่ง จะได้ผลลัพธ์เป็นศูนย์อย่างสอดคล้องกับอีกสไตส์หนึ่ง และค่าศูนย์แบบนี้เข้าใจผิดเป็นบักหรือเป็นการทดสอบผ่านได้ง่ายมาก

รอดหวุดหวิด: ถ้าแปล prompt ทั้งชุดเป็นภาษาอังกฤษ จะเสียความแม่นยำไปราว 8%

ต่ำ 2026-08-25

สำหรับ prompt ข้ามภาษา ให้ใส่ glossary คำศัพท์เพิ่มเข้าไปแทนการแปลทั้งชุด — และต้องยืนยันก่อนว่าการเปลี่ยนแปลงที่เสนอมาแก้ปัญหาอะไรจริงๆ ก่อนจะลงมือทำ

พจนานุกรมคำไทย→ฟิลต์ ถูกวางไว้ท้ายๆ prompt มากเกินไป ต้องย้ายขึ้นไปไว้บนสุด

ต่ำ 2026-08-29

ให้วางคำนิยาม คำศัพท์ และข้อจำกัดไว้บนสุดของ prompt ก่อนตัวงานและข้อมูล ลำดับใน prompt เป็นพารามิเตอร์ที่มีผลจริง ไม่ใช่แค่เรื่องจัดรูปแบบ

บั๊กในโค้ด และ ตรรกะผิด

บั๊กธรรมดา: สูตรผิด เงื่อนไขผิด นับเกิน/ขาด regex ผิด หน้าจอพัง

ทั้งหมด 53 เรื่อง · วิกฤต 6 · สูง 14 · กลาง 23 · ต่ำ 10

เรื่องที่ต้องอ่านให้ครบ (20)

วิกฤต 2026-07-28

บั๊กที่ 4: สิม `enable_thinking=False` อีกแล้ว — บั๊กตัวเดียวกับที่เคยทำให้ t01 รัดได้ 0/3

เกิดอะไรขึ้น: `train_qwen3vl.py` ไม่มี `enable_thinking=False` ไดอารี่ระบุตรงๆ ว่านี่คือบั๊กตัวเดียวกับที่ทำให้ t01 รัดได้ 0 จาก 3 คือเป็นความล้มเหลวที่เคยรู้แล้วในอดีต ถูกนำกลับมาใหม่ในไฟล์ใหม่แบบเป๊ะๆ

ต้นเหตุจริง: ความล้มเหลวที่เคยวินิจฉัยไว้แล้ว ถูกแก้แค่ในสคริปต์เดียว และไม่เคยถูกบันทึกเป็นค่าที่ต้องมีใน config ที่ใช้ร่วมกัน ใน template หรือในตัวตรวจ — พอเขียนไฟล์ใหม่จากศูนย์ ก็เลยทำซ้ำแบบเดิมเป๊ะ

เสียหายอะไร: จะได้การวัดผลที่ได้ศูนย์อีกรอบ เสียทั้งเงินค่าเช่าและเวลาวิเคราะห์ที่ตามมา จับได้ก่อนเข้าเครื่อง

แก้ยังไง: เพิ่ม `enable_thinking=False` เข้าไป

กฎที่ได้: บั๊กทุกตัวที่เคยทำให้ได้คะแนนศูนย์ ต้องกลายเป็น assertion ในตัวตรวจ environment ไม่ใช่แค่แก้ในไฟล์เดียว ถ้าความล้มเหลวเก่ากลับมาได้แค่เพราะเขียนสคริปต์ใหม่ แปลว่ามันยังไม่เคยถูกแก้จริง

วิกฤต 2026-08-21

greedy decoding ที่ไม่มีตัวกันการวนซ้ำ ทำให้โมเดลวนจนชน token cap พังทั้งหน้า

เกิดอะไรขึ้น: หน้า beam plan ของบ้าน 08 (หน้า20, S-03) parse ไม่ผ่าน ต้นเหตุคือ `run_house_batch.py` กับ `infer_t02.py` ใช้ greedy decoding ล้วน (`do_sample=False`) โดยไม่มีตัวกันการวนซ้ำ โมเดลเลยวนพ่นคำ B2 ปลอมๆ เข้าไปเกือบทุกช่อง กริดจนชน `max_new_tokens` แล้วไม่เคยปิด JSON ได้หน้านั้นมีคำจริง 31 ตัว หายหมดทั้ง 31 ตัวเพราะวงเล็บไม่เคยปิด

ต้นเหตุจริง: การถอดรหัสแบบ autoregressive จะยังเสริมตัวเองเมื่อเจอเนื้อหาซ้ำๆ — ทุกครั้งที่ซ้ำ หลักฐานที่จะซ้ำอีกยิ่งมากขึ้น จน stop token ไม่มีวันชนะ greedy decoding ที่ไม่มีทั้งบงกชโทษและตัวบังคับโครงสร้าง จึงพังแน่นอนบนหน้าที่มี element เยอะ และ JSON เป็นแบบได้หมดหรือไม่ได้เลย ผลถูกบางส่วนจึงมีค่าเป็นศูนย์

เสียหายอะไร: หน้าที่มี element เยอะ ซึ่งก็คือแผนโครงสร้างที่มีค่าที่สุดพอดี ให้ผลใช้งานไม่ได้เลยสักหน้า ทั้งอัตรา JSON ที่ถูกต้อง 57.9% และ element recall 11% ส่วนใหญ่มาจากบั๊กตัวนี้ แค่หน้า20 หน้าเดียวก็มี element ใน ground truth ที่นับไม่ได้ 31 ตัว

แก้ยังไง: ใส่ `repetition_penalty=1.15` กับ `no_repeat_ngram_size=8` ลงในทั้งสองสคริปต์ (เป็นการกำบางส่วน คือ "ขอร้อง" ไม่ใช่ "ห้าม") แล้วคิดถัดมาก็เปลี่ยนไปใช้ grammar-constrained decoding ผ่าน `xgrammar` ซึ่งดัน JSON ที่ถูกต้องจาก 57.9% ขึ้นเป็น 96.8%

กฎที่ได้: ยารันงานที่ต้องได้ output แบบมีโครงสร้างด้วย greedy decoding ล้วนๆ โดยไม่มีตัวบังคับโครงสร้างเด็ดขาด — ให้ใช้ grammar-constrained decoding (`xgrammar`) เป็นค่าตั้งต้นสำหรับ output ที่เป็น JSON เพราะบงกชโทษแบบอ่อนๆ แะทำให้วนยากขึ้น แต่มีแค่ grammar เท่านั้นที่ทำให้วังเลิบบ้างเป็นไปไม่ได้

parseGridMaster อ่านไฟล์กริดรุ่นปัจจุบันไม่ได้เลยสักไฟล์ตั้งแต่ 2026-08-02 — ทุกการ import ได้กริดว่างเปล่าแบบเงียบๆ

เกิดอะไรขึ้น: parseGridMaster ของ Constistant อ่าน f.x_lines / f.y_lines ที่ระดับบนสุด แต่ schema ย้ายมันเข้าไปอยู่ในอ็อบเจกต์ grid{} ที่ซ่อนอยู่ข้างในแล้ว พอเราข้ามทั้งสองไฟล์เตอร์ของ dataset พบว่าไฟล์กริด 37 จาก 37 ไฟล์เป็นแบบซ่อน และ 0 ไฟล์เป็นแบบระดับบนสุด ทุกการ import raw-extraction ตั้งแต่วันที่ปรับโครงสร้างนั้นจึงได้กริดว่างเปล่า แล้วมารีคทุกช่วงว่า unresolved แบบเงียบๆ

ต้นเหตุจริง: การย้ายไฟล์ใน schema ผังข้อมูล ไม่เคยถูกส่งต่อไปยังฝั่งแพลตฟอร์มที่เราไปใช้ และเวลาหา key ไม่เจอ ฝั่งที่ใช้ก็คืนค่าว่างแทนที่จะโยน error ความล้มเหลวโดยสิ้นเชิงจึงแยกไม่ออกจาก 'บ้านหลังนี้ไม่มีกริด'

เสียหายอะไร: เงียบมาตั้งแต่ 2026-08-02: ทุกการ import raw-extraction ในแพลตฟอร์มได้ข้อมูลกริดเป็นศูนย์ เลยได้ช่วงคานที่แก้ได้เป็นศูนย์ตามไปด้วย ความยาวคานจึงไม่เคยไปถึง BOQ เลยแม้แต่หนึ่งเดียว

แก้ยังไง: แก้ให้อ่าน grid{} แบบซ่อนก่อน แล้วถอยไปอ่านระดับบนสุดสำหรับไฟล์เก่า พิสูจน์กับข้อมูลจริงแล้ว (รัน adaptRawExtraction บนบ้าน 09 ที่เป็น ground truth ช่วงคานขยับจาก 0/87 เป็น 18/87 พร้อมค่าจริง) เพิ่ม regression test ใหม่ 4 ตัวใน tests/raw-extraction-grid.test.js npm test ผ่าน 1122/1122

กฎที่ได้: parser ที่คืนค่าว่างเมื่อเจอโครงสร้างที่ไม่รู้จัก ต้องเตือนแทน — และทุกครั้งที่ย้ายไฟล์ใน schema ต้อง grep หาทุกจุดที่เราไปใช้ แล้วเขียน regression test ที่ใช้ไฟล์จริงรุ่นปัจจุบัน ไม่ใช่ข้อมูลตัวอย่างสำหรับเทสที่เขียนมือแล้วอัปเดตไปพร้อมกับ parser

parseGridRef รับแต่รูปแบบ grid ref ที่มีขีดคั่น ทั้งที่สเปกกำหนดให้ไม่มีขีด — ช่วงคานเลยเป็น null ทุกตัว

เกิดอะไรขึ้น: parseGridRef ทำ trimmed.split('-') แล้วปฏิเสธทุกอย่างที่ไม่ได้ออกมาเป็น 2 ส่วนพอดี แต่สเปก §0.8 กำหนดให้ใช้รูปแบบไม่มีขีด ("C1" ไม่ใช่ "C-1") และข้อมูลจริงทั้งหมดก็ใช้แบบนั้น ช่วงคานทุกตัวในทุกการ import จึงเป็น null

ต้นเหตุจริง: parser เขียนขึ้นตามข้อสันนิษฐานเรื่องรูปแบบ ('grid ref คือตัวอักษร-ขีด-ตัวเลข') ซึ่งขัดกับกฎ §0.8 ของสเปกเอง แล้วก็ไม่มีเทสไหนใช้ค่าจริงที่ตรงตามสเปก แคมเวลาพึ่งมันคืน null ไม่ได้โยน error เรื่องเลยไม่เคยโผล่ขึ้นมา

เสียหายอะไร: ช่วงคานทุกตัวในทุกการ import raw-extraction ได้ค่า null — เป็นบั๊กเงียบตัวที่สองจากสองตัวที่รวมกันแล้วทำให้เราคาดคิดของคานไม่เคยไปถึง BOQ เลย

แก้ยังไง: ให้รับ `^[A-Za-zก-ฮ]{1,3}*(\d+)?$` ควบคู่ไปกับรูปแบบมีขีดแบบเดิม ส่วน ref ที่นำหน้าด้วย ~ ซึ่งเป็นค่าโดยประมาณยังถูกปฏิเสธถูกต้องเหมือนเดิม ตรวจสอบกับข้อมูลจริงแล้ว (ช่วงคาน 0/87 → 18/87) พร้อม regression test ใหม่อีก 2 ตัว

กฎที่ได้: เขียนเทสของ parser จากตัวอย่างมาตรฐานในสเปก ไม่ใช่จากรูปแบบที่เราเดาเอาเอง — และทำให้การ parse ไฟล์ที่จำเป็นไม่ผ่านมันตั้งขึ้นมา (เตือน/นับจำนวน) แทนที่จะคืน null ซึ่งแยกไม่ออกจากกรณีที่ข้อมูลไม่มีอยู่จริงๆ

การลบไฟล์ซ้ำด้วยค่า NCC ทำลายความครอบคลุมในการตรวจจับจริงของ 4 หลัง — ความคล้ายกัน $\neq$ ใช้แทนกันได้

เกิดอะไรขึ้น: รันการลบไฟล์ซ้ำบนธนาคาร footing ที่ promote ไปแล้ว ลบ tpl_footing13/15/17 ออก (แต่ละตัวได้คะแนน 0.80-1.00 เทียบกับ template 7/6/16) พอลองตรวจความครอบคลุมจริง บ้าน 04, 05, 17 และ 26 หลุดจากสถานะเขียว template ที่หน้าตาเหมือนกันเป๊ะที่ NCC บน crop 64x64 กลับให้ผลต่างกันตอนใช้ multi-scale sliding-window matching บนหน้าเต็ม

ต้นเหตุจริง: NCC บน crop 64x64 ที่ resize แล้ว วัดความคล้ายกันของรูปลักษณะ ซึ่งไม่ใช่คุณสมบัติเดียวกับ "ให้ผล match เดียวกันตอนกวาดหน้าเต็มแบบ multi-scale" ตัว metric ที่ใช้ลบซ้ำเป็นแค่ตัวแทนของคุณสมบัติที่สำคัญจริงๆ

เสียหายอะไร: 4 หลังเสียความครอบคลุมในการตรวจจับ ถ้าไม่มีการเช็คความครอบคลุมก่อนใช้จริง ความเสียหายนี้จะโผล่มาทีหลังในรูปของการตรวจจับที่แย่งโดยไม่รู้สาเหตุ

แก้ยังไง: เพิ่ม gate ที่สองให้ `--bank --apply``: วัดความครอบคลุมจริงก่อนและหลังโดยใช้ `house_pages()` + `match_bank` ผ่าน path เดียวกับ `harvest_templates.py` ถ้าหลังไหนหลุดจากเขียว ยกเลิกการลบทั้งชนิดนั้นทันทีและคืนไฟล์อัตโนมัติ (ทดสอบแล้ว — จับ footing ได้ คินค่า 14 เป็น 14 อัตโนมัติ) ส่วนการลบซ้ำของ column ถูกตรวจสอบว่าปลอดภัยด้วย `match_points` จริงก่อนใช้: 7 template $\rightarrow$ 3 ความครอบคลุมไม่เปลี่ยน อยู่ที่ 16/30 หลังเท่าเดิม

กฎที่ได้: เมื่อจะลบข้อมูลซ้ำที่ป้อนเข้าตัวตรวจจับ ต้องตรวจสอบกับผลลัพธ์จริงของตัวตรวจจับ ไม่ใช่แค่คะแนนความคล้ายของรูปลักษณะ ให้สร้างการเช็คความครอบคลุมก่อน/หลังพร้อม auto-rollback ไว้ในตัวเครื่องมือเองเลย

ตัวสร้างชุดข้อมูลยังกรองด้วยชื่อ pattern เก่าหลังเปลี่ยนชื่อไปแล้ว — plan_footing/plan_beam/gridline หายไปเต็มวัน

เกิดอะไรขึ้น: `build_dataset_t03.py` ยังกรองด้วยชื่อ pattern เก่าคือ "plan" และ "gridline" หลังการเปลี่ยนชื่อวันที่ 28 ส.ค. plan_footing, plan_beam และ gridline หายไปจาก train/val.jsonl ทั้งหมด — 0 ตัวอย่างทุกตัว — ไม่มีใครสังเกตเห็นกว่าจะมีการสำรวจความสะอาดของ prompt มาเจอเข้าโดยบังเอิญ หลังแก้: gridline 40 / plan_footing 100 / plan_beam 125

ต้นเหตุจริง: การเปลี่ยนชื่อในจุดเดียว (คำศัพท์ประเภทหน้า) ไม่ได้ถูกกระจายไปถึงตัวกรองของฝั่งที่ใช้งาน และตัวสร้างชุดข้อมูลรายงานแค่จำนวนตัวอย่างรวม ไม่แยกรายงานเป็นราย subtask ทำให้ "plan_beam เท่ากับ 0" ดูเหมือนการ build ที่สำเร็จปกติ

เสียหายอะไร: สาม subtask ที่สำคัญที่สุด — รวมถึง plan_beam ซึ่งเป็นเป้าหมายหลักของรอบนี้เลย — หายไปจากข้อมูล training ทั้งหมด การเทรนใดๆ ในช่วงนี้จะไร้ประโยชน์สำหรับ subtask พวกนี้โดยที่ไม่มีใครมองเห็นสาเหตุเลย

แก้ยังไง: แก่ตัวกรอง ยืนยันได้ 257 แถวจริงใน train+val.jsonl

กฎที่ได้: ตัวสร้างชุดข้อมูลต้อง assert จำนวนตัวอย่างขั้นต่ำต่อ subtask และฟังดังๆ เมื่อเป็น 0 ตัวกรองที่ว่างเปล่าเสียๆ หลังเปลี่ยนชื่อคือบั๊กที่แพงที่สุดประเภทหนึ่ง — มันทำให้การเทรนทั้งรอบไร้ความหมายโดยไม่มี error ให้เห็นเลย

สูง 2026-07-09

คอลัมน์ที่ปักตรึงไว้ใน Label Studio ตัดเนื้อหาทิ้งเจียบๆ แทนที่จะให้เลื่อนดูได้

เกิดอะไรขึ้น: คนทำ annotation แจ้งว่าแถบซ้ายเลื่อนไม่ได้ ตรวจแล้วพบว่า `.stickycol`` ไม่มี `max-height` และไม่มี `overflow-y` เลย เนื้อหาส่วนที่สูงเกินจอจึงไม่ใช่แค่เลื่อนไปไม่ถึง แต่หายไปเลย โดยไม่มีอะไรบอกว่ามีของหายอยู่

ต้นเหตุจริง: คอลัมน์แบบ sticky ถูกจัดสไตล์บนข้อสันนิษฐานว่าเนื้อหาจะขนาดคงที่ และไม่เคยทดสอบกับเนื้อหาที่สูงกว่าจอ ส่วนค่าเริ่มต้นของ CSS overflow ก็คือตัดทิ้งแบบเจียบๆ

เสียหายอะไร: คนทำ annotation มองไม่เห็นบางส่วนของแบบที่ต้องมา label — เกิดบนโปรเจกต์จริงตัวแรกที่สร้างใน Label Studio Cloud และเป็นเหมือนกันทั้ง 5 ไฟล์ config

แก้ยังไง: เพิ่ม `max-height:96vh; overflow-y:auto; padding-right:6px`` ให้ `.stickycol`` ในไฟล์ XML config ทั้งห้าไฟล์

กฎที่ได้: แถวที่ปักตรึง/sticky ทุกอันต้องใส่ `max-height` กับ `overflow-y:auto` ตั้งแต่แรก เพราะโหมดฟังโดยค่าเริ่มต้นของมันคือข้อมูลหายไปเจียบๆ ไม่ใช่เลยเอาต์แตกให้เห็น

สูง 2026-07-19

RB2 ถูกบันทึกที่ตำแหน่งเดา confidence 0.4 ซึ่งกลายเป็นผิดทั้งหมด

เกิดอะไรขึ้น: RB2 ในผังคานาคอดินของบ้านหลัง 2 ถูกบันทึกไว้ที่ B3-A3 ด้วย confidence 0.4 — คือเดาล้วนๆ ที่จริงมันอยู่ที่ C4-B4 (1.8) + B4-A4 (3.15) ผังเสาแนว 4 ส่วนเลข 2.50 ที่เอาไปแปะกับ RB2 จริงๆ แล้วเป็นของ dummy grid B'

ต้นเหตุจริง: เอาค่าที่เดาแบบ confidence ต่ำมาเขียนลงเป็นข้อมูลจริง แทนที่จะปล่อยเป็น null แล้วติดธงไว้ และเอาระยะที่อธิบายไม่ได้ (2.50) ไปแปะกับ element ที่ใกล้ที่สุดที่ดูเข้าเค้า แทนที่จะปล่อยไว้ว่าไม่รู้ว่าเป็นของอะไร

เสียหายอะไร: คานาคอดินถูกบันทึกผิดฝั่งของอาคาร ระยะหนึ่งถูกยกให้ผิด element ซึ่งทำให้เจอ dummy grid B' ซ้ำไปด้วย

แก้ยังไง: แก่ RB2 เป็น C4-B4 + B4-A4 แล้วเอาเลข 2.50 ที่วางออกมาไปหาค่า B' = A-2.50 จากนั้นเพิ่ม RB1A ที่ B'3-B'4

กฎที่ได้: ตำแหน่งที่ confidence 0.4 คือการเดา และการเดาที่เขียนลงในฟิลด์ตำแหน่ง ปลายทางแยกไม่ออกเลยว่าเป็นข้อมูลจริงหรือเดามา ให้ปล่อยเป็น null พร้อมติดธงให้ชัด และห้ามเอาระยะที่อธิบายไม่ได้ไปแปะกับ element ที่บังเอิญอยู่ใกล้ที่สุด

สูง 2026-07-21

ตัวกัน `isinstance(ip.size, dict)` เป็น false เจียบๆ ทำให้การตั้งค่าความละเอียดภาพถูกข้ามไปโดยไม่มี error

เกิดอะไรขึ้น: โค้ดกัน `if isinstance(ip.size, dict)` ที่เพิ่มเข้าไปตอนแก้บั๊ก max_pixels เป็น False เสมอ ค่าความละเอียดที่ตั้งใจไว้ที่ 5,120 visual token เลยไม่เคยถูกใช้ โมเดลรันด้วยค่า default 16,384 ไปเจียบๆ

ต้นเหตุจริง: `ip.size` เป็น `SizeDict` ซึ่งรองรับ `[\"key\"] = value` ได้ แต่ไม่ได้สืบทอดมาจาก `dict` โค้ดกันไปเช็คการสืบทอดแทนที่จะเช็คพฤติกรรม และเส้นทางที่ล้มเหลวคือข้ามไปเจียบๆ ไม่ใช่ error

เสียหายอะไร: ทั้งการเรนและการประเมินผลรันด้วยความละเอียดภาพที่ผิดโดยไม่มีอาการอะไรให้เห็น ทำให้ค่า baseline ปนเปื้อนอย่างน้อยหนึ่งรอบ และเสียรอบดีบั๊กเพิ่มบนเครื่องที่เสียเงิน

แก้ยังไง: เอาเงื่อนไข isinstance ออก แล้วเขียนค่าลงไปตรงๆ ยืนยันด้วยสคริปต์เล็กๆ บนเครื่องที่อ่านค่าที่ได้กลับมาดู (ไม่ต้องโหลดโมเดลเต็มตัว)

กฎที่ได้: ห้ามเอาการเช็ค type มากับการเขียนค่า config ถ้ามันข้ามไปเจียบๆ ได้ ให้เขียนค่าลงไปเลย แล้วอ่านกลับมา assert ว่าตรงกับที่ตั้งใจ การไม่ทำอะไรเจียบๆ อันตรายกว่าการฟังโครมๆ

สูง 2026-08-20

from_pretrained ของ Unsloth เมีน kwarg ชื่อ subfolder แบบเงิบๆ ทำให้โหลด LoRA adapter ไม่ได้

เกิดอะไรขึ้น: ไฟล์ adapter ของ t02 อยู่ที่ lora-adapter/ ซ้างในตัว repo ของ HF ไม่ได้อยู่ที่ root from_pretrained ของ Unsloth ไม่ส่ง kwarg ชื่อ subfolder ต่อไปยังการเรียก AutoConfig/PeftConfig ซ้างในของมัน ทำให้โหลดไม่สำเร็จ เจอสาเหตุได้จากการอ่าน source ของ unsloth/models/loader.py จริงๆ ไม่ใช่เดา

ต้นเหตุจริง: ตัว loader ของ Unsloth รับ **kwargs ระดับบนสุดได้ แต่ไม่ได้ส่งต่อไปยังการเรียกที่ตรวจสอบ config ซ้างในเลย — เป็นการทิ้ง kwarg แบบเงิบๆ ไม่ใช่ error ผู้เรียกไม่มีทางรู้ว่า argument นั้นถูกเมีน

เสียหายอะไร: บล็อกการรัน inference ทั้งหมดจนกว่าจะหาสาเหตุเจอ ถ้าไม่หาให้ดี อาจเข้าใจผิดว่า adapter ที่อัปโหลดพังหรือ HF repo วางโครงสร้างผิด

แก้ยังง: ดาวน์โหลดล่วงหน้าด้วย snapshot_download(allow_patterns=[...]) แล้วชี้ from_pretrained ไปที่ path ในเครื่องที่ได้มา ใช้กับทั้ง run_house_batch.py และ infer_t02.py

กฎที่ได้: เมื่อ wrapper library เมีน argument ที่ส่งเข้าไปให้อ่าน source ของมันแทนการเดา และควรแปลง artifact ที่อยู่ remote เป็น path ในเครื่องด้วยตัวเอง (snapshot_download) แทนที่จะเชื่อว่า wrapper จะส่ง path kwarg ที่มันไม่เคยเอกสารไว้

สูง 2026-08-24

assert ของ MAX_LENGTH ไปเช็คตัวที่ภาพเยอะสุด แทนที่จะเช็ค sequence ที่ยาวจริง เลยมองไม่เห็นของที่สั้นอยู่

เกิดอะไรขึ้น: หลัง op04 ขยาย dataset จาก 452 เป็น 1,116 ตัวอย่าง พบว่ามีตัวอย่างหนึ่งยาวเกิน MAX_LENGTH แต่ assert เดิมเช็คแค่ตัวอย่างที่มีภาพเยอะที่สุด ซึ่งไม่ใช่ตัวที่ sequence ยาวที่สุด พอแก้ assert ให้เช็ค 3 อันดับแรกโดยรวม ภาพ+prompt+GT เข้าด้วยกัน ก็เจอ sequence จริงที่ 30,368 / 28,008 / 28,035 — เหลือระยะปลอดภัยแค่ 7% จาก 32,768 และ assert เดิมมองไม่เห็นสักตัว

ต้นเหตุจริง: ตัวกันใช้ตัววัดตัวแทน (จำนวนภาพ) แทนสิ่งที่มันคุ้มครองจริงๆ (ความยาว token รวม) โดยตัดข้อความ prompt กับความยาว GT ออกจากการคำนวณไปเลย

เสียหายอะไร: อาการแรกที่จะเจอคือตัวอย่างเหวี่ยงถูกตัดทิ้งเงิบๆ หรือ OOM กลางรอบ นี่คือนักคลาสเดียวกับที่บันทึกไว้แล้วใน t01 §0.4

แก้ยังง: เขียน assert ใหม่ให้คำนวณ token ของ ภาพ + prompt + GT แล้วเช็ค 3 อันดับที่ยาวที่สุด และให้ assert จำนวน visual token บน batch แรกด้วย เพื่อจับกรณีภาพถูกย่อเป็น 512px เงิบๆ

กฎที่ได้: ตัวกันต้องวัดสิ่งที่มันคุ้มครองจริงๆ ห้ามใช้ตัวแทน และเวลาเติมข้อมูลเข้า dataset ต้องคำนวณการกระจายความยาวใหม่ อย่าไว้ใจตัวกันที่จูนไว้กับการกระจายชุดเก่า

สูง 2026-08-24

รอบ inference แรกของบ้านหลัง 08 ตายทั้งสองงานแรก จากบั๊ก JSON คนละเรื่องสองตัว

เกิดอะไรขึ้น: รอบ inference ของ t03 บนบ้านหลัง 08 ล้มเหลว 2 จาก 2 งานแรก — (1) ผลลัพธ์ gridline มีโครงสร้างฟังก์ชันขาดปีกกาเปิดของ element ในอาร์เรย์ และชื่อฟิลด์ pox_M เพี้ยน (2) ผลลัพธ์ notes มี confidence_score เป็น 0. (จุดทศนิยมที่ไม่มีตัวเลขต่อ) และมีบั๊กที่สองในไฟล์เดียวกันคือ item_no ที่ขาดเครื่องหมายคำพูดเปิดของชื่อ key

ต้นเหตุจริง: grammar (xgrammar) ถูกแปะไว้กับ plan_beam อย่างเดียวตามกฎที่ตั้งไว้ ทำให้ subtask อื่นๆ ทั้งหมดยังสร้าง JSON แบบอิสระ แถม gridline ยังมีตัวอย่างเทรนน้อยที่สุด (45 ตัว) และเป็นงานที่ยากที่สุดด้วย

เสียหายอะไร: เสียรอบ inference ที่จ่ายเงินไปกับ 2 งานก่อนจะมีคนสังเกตเห็น และถ้าไปแก้ด้วยการ patch ด้วย regex ก็ะกลายเป็นตัวต้นไม่จบสิ้น

แก้ยังไง: ตัดสินใจ ☒ เปลี่ยนไปใช้ --grammar-all (บังคับ xgrammar กับทุก subtask ไม่ใช่แค่ plan_beam) ฆ่าโปรเซสที่รันอยู่ตอน 2/33 แล้วรันใหม่ทั้งชุด (ราวๆ \$0.4)

กฎที่ได้: ถ้า constraint เชิงโครงสร้างอันหนึ่งถูกต้องกับ subtask หนึ่ง มันแทบจะถูกต้องกับทุกอันแน่นอน — ให้เปิดใช้ทั่วทั้งระบบเป็นค่าเริ่มต้น และห้ามแก้ผลลัพธ์เสียรูปของโมเดลด้วยการ patch regex ที่ละแพตเทิร์น

สูง 2026-08-28

BEAM_GAP ตั้งค่าตายตัวที่ 21px มองไม่เห็นครึ่งหนึ่งของคลังแบบแปลนทั้งหมด

เกิดอะไรขึ้น: การตรวจจับคานใช้ค่า gap ตายตัวเดียวคือ 21px พอวัด 33 หลังจริงพบว่ามี 2 ชุดที่แตกต่างกัน คือชุดกรรมโยธาที่ ~16px และชุดบ้าน 17 ที่ ~21px ค่าเดียวมองเห็นได้แค่ชุดใดชุดหนึ่งเท่านั้น

ต้นเหตุจริง: ค่าคงที่นี้ตั้งจากบ้านที่เปิดอยู่ตอนเขียนตัวตรวจจับ แล้วไม่เคยตรวจสอบกับข้อมูลทั้งหมดเลย แบบจากหน่วยงานออกแบบต่างกันใช้ระยะห่างเส้นต่างกัน

เสียหายอะไร: การตรวจจับคานเป็นไปไม่ได้เลยในทางโครงสร้างกับเกือบครึ่งหนึ่งของคลังข้อมูล ตลอดเวลาที่ค่าคงที่นี้ยังใช้อยู่

แก้ยังไง: เปลี่ยนเป็น BEAM_GAPS = (16, 21) ซึ่งวัดจาก 33 หลังจริง

กฎที่ได้: ค่าคงที่เชิงเรขาคณิตใดๆ ที่มาจากแบบแปลน ต้องวัดกับข้อมูลทั้งหมดจริง และเก็บเป็นชุดค่าตามซีรีส์แบบแปลน ไม่ใช่ตัวเลขค่าเดียว หน่วยงานที่ออกแบบต่างกันมีธรรมเนียมต่างกัน

ค่า threshold การจับคู่เสาที่ 0.65 ทำให้เกิดผลลัพธ์ระยะใน 10 จาก 30 หลัง

เกิดอะไรขึ้น: ที่ threshold 0.65 ตัวตรวจจับเสาให้ผลลัพธ์ระยะใน 10 จาก 30 หลัง การ sweep ปรับขึ้นเป็น 0.75 ลดจำนวนหลังที่มีระยะเหลือแค่ 1 หลัง แต่เพิ่มเพลตเล็ก 2 ตัว (t6/t7) ยังพ่นจุดระยะ 49-96 จุด/หน้า ใส่แบบแปลนชุดอื่นที่ 0.75 อยู่ดี จนต้องเพิ่มกฎใหม่ (เพิ่มเพลตเล็ก + ความหนาแน่นหมึก < 0.15 → ต้องใช้ 0.85) ซึ่งแก้ได้หมด ไม่มีระยะเหลือเลยและยังจับ true positive ได้ครบ

ต้นเหตุจริง: threshold ค่าเดียวทั่วทั้งระบบไม่สามารถรองรับเพิ่มเพลตที่มีขนาดและความหนาแน่นหมึกต่างกันได้ เหมเพลตเล็กและเบาบางจะจับคู่กับสัญญาณรบกวนได้เกือบทุกที่ ค่า threshold ถูกตั้งจากสัญญาณแทนที่จะ sweep กับข้อมูลจริง

เสียหายอะไร: ถ้าไม่แก้ จะมีจุดตรวจจับผิดสูงสุดถึง 96 จุดต่อหน้าถูกป้อนเข้าโมเดลเป็น "hint" ซึ่งเป็นการวางยาให้ pass ที่ hint นี้ควรจะช่วยเสียเอง

แก้ยังไง: sweep กับ 30 หลังเพื่อตั้งค่าเป็น 0.75 เพิ่มกฎเพิ่มเพลตเล็ก/หมึกบาง → ใช้ 0.85 และใส่กฎนี้ไว้ที่จุดเดียวใน match_bank เพื่อไม่ให้ analyze กับ harvest ค่อยๆ เพี้ยนไปคนละทาง ผลลัพธ์คือเสาที่จับคู่กับ GT ได้เพิ่มจาก 4 เป็น 16 หลัง และ val 01-05 เชียวทั้งหมด

กฎที่ได้: ต้อง sweep threshold การตรวจจับกับข้อมูลทั้งชุดจริง และทำให้ threshold เป็นฟังก์ชันของคุณสมบัติเพิ่มเพลต (ขนาด, ความหนาแน่นหมึก) แทนที่จะเป็นค่าคงที่ค่าเดียว วางกฎไว้ที่จุดเดียวเพื่อไม่ให้ผู้เรียกใช้ค่อยๆ เพี้ยนไปคนละทาง

Dedup แบบ union-find ทำให้เกิดการเชื่อมโยงลูกโซ่ผิดพลาด รวมค่าที่ไม่เกี่ยวข้องเลยเข้าด้วยกัน

เกิดอะไรขึ้น: dedup_templates.py ตอนแรกใช้ union-find (จัดกลุ่มแบบส่งต่อ) ที่ $NCC \geq 0.8$ ไฟล์ A~B ได้คะแนน 1.0 และ B~C ได้ 0.80 ทำให้ A ถูกจัดกลุ่มรวมกับ C ทั้งที่ A~C มีคะแนนแค่ 0.737 — และคู่ที่ถูกรวมกันนั้นคือ crop ข้อความของคนละคำกันเลย ไม่ใช่ไอคอนด้วยซ้ำ

ต้นเหตุจริง: ความคล้ายกันไม่ใช่ความสัมพันธ์แบบส่งต่อ union-find บน threshold ความคล้ายจะสร้างลูกโซ่ที่ปลายทั้งสองข้างต่างกันได้ตามใจชอบ โดยมีไฟล์ตรงกลางเป็นสะพานเชื่อม

เสียหายอะไร: ถ้าไม่แก้ จะทิ้งเพิ่มเพลตที่แตกต่างกันจริงๆ ไปอย่างเงียบๆ และคะแนนความคล้ายที่รายงานออกมาจะเป็นค่าโกหก (คะแนนที่รายงานไม่ใช่คะแนนคู่จริงเลย)

แก้ยังไง: เขียนใหม่เป็นแบบ greedy — แต่ละไฟล์เทียบเฉพาะกับตัวแทนที่เก็บไว้แล้วเท่านั้น ไม่เทียบผ่านไฟล์อื่น รับประกันว่าทุกคะแนนที่รายงานเป็นคะแนนคู่จริง

กฎที่ได้: ห้ามใช้การจัดกลุ่มแบบส่งต่อบน threshold ความคล้ายเด็ดขาด ให้เทียบกับตัวแทนที่เก็บไว้แล้วเท่านั้น เพื่อให้คะแนนที่รายงานเป็นคะแนนที่วัดได้จริง

สูง 2026-08-29

ข้อความบรรยายของผู้ตรวจทานมนุษย์รั่วเข้าไปใน prompt ของโมเดลผ่านบล็อก JSON ของ grid master

เกิดอะไรขึ้น: ไฟล์ prompt ที่เป็นข้อความล้วนทุกไฟล์สแกนแล้วสะอาด ไม่มีอักขระต้องห้าม แต่ prompt ที่ประกอบขึ้นจริงสำหรับ plan_footing/plan_beam/plan_slab ต่อท้ายด้วยบล็อก JSON ของ grid master จริงจาก GT — และบล็อกนั้นมีฟิลด์ `source` กับ `confidence\_flags` ที่มีคำอธิบายของผู้ตรวจทานมนุษย์เต็มไปด้วย `;`, `\_`, `—` ซึ่งไปถึงโมเดลจริงๆ

ต้นเหตุจริง: การสแกนความสะอาดตรวจแค่ไฟล์ต้นฉบับ `.md` ที่เป็นข้อความ ไม่ได้ตรวจ prompt ที่ประกอบเสร็จแล้วซึ่งโมเดลได้รับจริง บล็อก grid master ถูกแทรกเข้าตอน build จาก GT จึงหลุดรอดการสแกนไฟล์ต้นฉบับไปเลย และไม่มีการกำหนด whitelist ฟิลด์ว่าจะไรควรถูกแปะเข้าไปบ้าง

เสียหายอะไร: อักขระที่ตั้งใจเอาออกจาก prompt อย่างเป็นระบบเพื่อลด hallucination ยังคงไปถึงโมเดลอยู่ดี บวกกับความเห็นผู้ตรวจทานที่ไม่เกี่ยวข้องซึ่งโมเดลต้องเิน ยังไม่ได้แก้

แก้ยังง: ยังไม่ได้แก้ — ตั้งเป็นประเด็นแยกต่างหากที่รอ Makham ตัดสินว่าจะตัด source/confidence_flags ออกแล้วเก็บแค่ id/pos_m/type หรือไม่

กฎที่ได้: ต้องสแกนสิ่งประดิษฐ์ที่ประกอบเสร็จแล้วที่โมเดลเห็นจริง ไม่ใช่แค่ไฟล์ต้นแบบ เวลาแทรกข้อมูลจริงเข้า prompt ต้อง whitelist เฉพาะฟิลด์ที่โมเดลต้องใช้ — ห้ามแปะทั้ง record เข้าไปเด็ดขาด

สูง 2026-08-30

รู้ปัญหาขึ้นมาตั้งแต่ t03 และยังไม่แก้: xgrammar บังคับแค่โครงสร้าง JSON ไม่ได้คุมเนื้อหาข้างในสตริง จึงเกิดการชนไม่รู้จบใน string value — นี่คือสิ่งที่ทำให้ server ค้างไป 7+ ชั่วโมง

เกิดอะไรขึ้น: xgrammar บังคับว่า output ต้องเป็น JSON ที่ valid แต่ไม่ได้จำกัดเนื้อหาภายใน string value เลย ดังนั้น request หนึ่งอาจเข้าไปวนซ้ำไม่รู้จบภายในสตริงและไม่มีวันจบ พฤติกรรมนี้รู้อยู่แล้วตั้งแต่ t03 แต่ก็ยังไม่ได้แก้ตอนที่เอา `serve\_purson.py` ขึ้นเครื่องเข้า และนี่คือสิ่งที่บล็อก event loop จนทำให้ server ทั้งตัวเข้าถึงไม่ได้นานกว่า 7 ชั่วโมง รวมถึง health check ง่ายๆ อย่าง `/v1/models` ด้วย

ต้นเหตุจริง: รูปแบบความล้มเหลวระดับ generation ที่รู้อยู่แล้วถูกส่งต่อมาโดยไม่มีทั้งวิธีบรรเทา (จำกัด token สูงสุดที่ทำงานจริง, เيونไขหยุด, ตัวกันการซ้ำ) และไม่มีการควบคุมเชิงสถาปัตยกรรมด้วย พอเจอ server แบบ single-threaded ที่ block เป็นครั้งแรกก็ล้มทั้งกระบวนการทันที

เสียหายอะไร: GPU เซาถูกคิดเงินไป 7+ ชั่วโมงโดยไม่ได้อะไรเลย การทดสอบที่วางแผนไว้ 2 อัน (รวมถึง `test\_browser\_path.py` แบบ end-to-end ที่ใช้ JWT ผู้ใช้จริง) ไม่เสร็จเลยสักอัน และต้องทำลายเครื่องด้วย `presentation.py down`

แก้ยังง: ยังไม่ได้แก้ ขั้นตอนถัดไปที่บันทึกไว้แก้แค่ตัวขยายปัญหา (ห่อ `generate()` ด้วย `asyncio.to\_thread()` หรือแยกเป็น thread คิวต่างหาก) ส่วนตัวการ generate ที่วนไม่รู้จบเองยังเป็นปัญหาเปิดอยู่ เช่นเดียวกับที่ยังไม่รู้จะทำไม `PAGE\_TIMEOUT\_S` ไม่เคยทำงาน

กฎที่ได้: รูปแบบความล้มเหลวที่รู้อยู่แล้วแต่ยังปล่อยเข้า production คือการส่มที่นัดหมายไว้ล่วงหน้าเอง grammar-constrained decoding รับประกันแค่รูปร่างของผลลัพธ์ ไม่รับประกันว่าจะจบ ต้องคู่กับการจำกัด token แบบเข้มและ timeout ที่พิสูจน์แล้วว่าทำงานจริง

สูง 2026-08-31

สูตรเฉลี่ย adapter แบบ $1/\sqrt{k}$ วัดจริงแล้วผิด — norm พุ่งขึ้น 3 ถึง 16 เท่า

เกิดอะไรขึ้น: สูตรเฉลี่ย $1/\sqrt{k}$ ที่วางแผนไว้ พอเอาไปวัดกับน้ำหนักจริง พบว่าเพราะทุก fold ชี้ไปทิศทางใกล้เคียงกันมาก พจน์ไขว้กันจึงบวกเสริมกัน ทำให้ $\|\Delta W\|$ ออกมาใหญ่เกินจริง 3.0 เท่า (attention) และ 16.3 เท่า (MoE)

ต้นเหตุจริง: เอาสูตร scaling จากทฤษฎีที่สมมติว่าแต่ละส่วนเกือบตั้งฉากกัน (independent) มาใช้ โดยไม่ได้วัดจริงว่า fold ต่างๆ มีทิศทางใกล้เคียงกันแค่ไหน

เสียหายอะไร: ถ้าไม่จับได้ จะได้โมเดลที่ merge มาด้วยสเกลผิดพลาดมาก โชคดีที่จับได้เพราะวัดจริงแทนที่จะเชื่อทฤษฎีเฉยๆ

แก้ยังไง: เปลี่ยนไปใช้วิธี concatenate ตาม rank axis แบบตรงตัว (r 16→48, alpha 32→96) วัด ratio ได้ 1.000 ทั้งสองประเภท layer เทียบเท่ากับ combination_type="cat" ของ PEFT ที่ทำด้วยมือเอง

กฎที่ได้: ก่อนใช้สูตร merge/เฉลี่ยน้ำหนักใดๆ ต้องวัด norm ratio กับน้ำหนักจริงก่อนเสมอ — สเกลจากทฤษฎีมักสมมติว่าแต่ละส่วนเป็นอิสระต่อกัน ซึ่ง fold ที่ fine-tune มาแล้วไม่เป็นแบบนี้

สูง 2026-09-01

pass3 นับจุดซ้ำเข้า BOQ ผ่านสามชั้นที่แยกกัน

เกิดอะไรขึ้น: จุดที่จับคู่แล้ว, จุดที่ใช้เป็น anchor, และจุดที่เป็นของ subtask อื่น ต่างถูกเพิ่มเป็น element ใหม่ซ้ำเข้าไปใน BOQ แก้ที่ละชั้นจนจำนวนซ้ำลดจาก 48 → 33 → 7 → 0

ต้นเหตุจริง: ไม่มีบัญชีกลาง (ledger) เดียวที่บอกว่า 'จุดไหนถูกนับไปแล้ว' แต่ละเส้นทางการใช้จุดต่างลบจุดออกจากมุมมองของตัวเองเท่านั้น ไม่ได้ลบออกจากชุดที่ใช้ร่วมกัน ทำให้จุดเดียวกันผ่านเช็ค 'เป็นข้อมูลใหม่ไหม' ได้ถึง 3 เส้นทางที่ต่างกัน

เสียหายอะไร: มี element หลอกเพิ่มเข้า BOQ ได้มากถึง 48 รายการ ต้องแก้ต่อเนื่องถึง 3 รอบ — ทำงานซ้ำสองครั้ง

แก้ยังไง: ติดตามจุดที่ถูกใช้ไปแล้วข้ามทั้งสามชั้น จนจำนวนซ้ำเหลือ 0

กฎที่ได้: เมื่อมีหลายเส้นทางโค้ดที่อ้างสิทธิ์รายการเดียวกันได้ ให้ใช้ชุดที่ถูกใช้ไปแล้วร่วมกันเพียงชุดเดียว แทนตัวกรองแยกตามเส้นทาง และเมื่อการแก้ de-dup ลดจำนวนซ้ำแต่ไม่หมดสนิท ให้แก้ต่อไป — เพราะแปลว่ายังมีเส้นทางอื่นที่อ้างสิทธิ์ซ้ำอยู่

สูง 2026-09-01

ถ้าเน็ตหลุดตอนกำลังรายงานว่างานล้มเหลว exception จะหลุดออกจาก loop แล้วมา worker ทั้งตัว

เกิดอะไรขึ้น: ใน `main()` ถ้า network error เกิดขึ้นตอนกำลังเรียก 'report job failed' exception จะพุ่งออกจาก loop `while` แล้วทำให้ worker process ทั้งตัวตายไปเลย

ต้นเหตุจริง: ทาง path ที่ใช้รายงาน error เองกลับไม่มีอะไรป้องกันไว้ — โค้ดที่มีหน้าที่จัดการความล้มเหลวตนเองได้โดยไม่มีอะไรรองรับ นอกจากนี้ path ของ done กับ failed ยังใช้โค้ดร่วมกัน ทำให้สถานะ 'failed' อาจไปเขียนทับงานที่จริงๆ เสร็จแล้วได้

เสียหายอะไร: แค่เน็ตสะดุดนิดเดียวตอนรายงาน error ก็ทำให้ worker ตายทั้งตัว ปลอมให้ GPU ที่เช่าไว้ว่างเปล่าแต่ยังคิดเงินอยู่

แก้ยังไง: แยก path ของ done กับ failed ออกจากกัน และห้ามประทับ 'failed' ทับงานที่เสร็จไปแล้ว

กฎที่ได้: path ที่ใช้รายงานความล้มเหลวคือจุดที่ห้ามพึ่งเด็ดขาด ต้องห่อมันไว้ และห้ามให้ done กับ failed ใช้ write path ร่วมกันจนฝั่ง failed ไปทับฝั่งที่สำเร็จได้

เรื่องที่เหลือในหมวดนี้ (33)

ค่ายาวๆ ที่ไม่มีที่ตัดคำสั้นออกนอกกล่องในหน้าตรวจของ Label Studio จนอ่านข้อมูลไม่ได้

กลาง 2026-07-06

อย่าปล่อย config ของ UI งาน annotation ออกไปโดยไม่เคยลองเรนเดอร์กับข้อมูลจริงสักครั้ง และให้ถือว่าทุกค่าที่เครื่องสร้าง (flag แบบ snake_case, ก่อน JSON, id) เป็นข้อความที่ตัดบรรทัดเองไม่ได้ ให้ตั้ง break-word กับฟิลด์พวกนี้เป็นค่าเริ่มต้นไปเลย

เติม xml-stylesheet เปล่าๆ เพื่อซ่อนคำเตือนของเบราว์เซอร์ แล้วผลลัพธ์และ สุดท้ายต้องย้อนกลับทั้งหมด

กลาง 2026-07-09

อย่าไปแก้เรื่องความสวยงามของเส้นทางการใช้งานที่ไม่ได้รองรับตั้งแต่แรก ให้บอกไปตรงๆ ว่าเส้นทางนั้นไม่รองรับ การหาทางเลี่ยงให้ปัญหาที่ไม่ใช่ปัญหา สร้างปัญหาจริงขึ้นมาได้

findProductionDir() แอบถอยไปใช้ mk_test/t2 ซึ่ง schema เก่ากว่าสองรุ่น เวลาหาโฟลเดอร์บ้านไม่เจอ

กลาง 2026-07-10

การถอยไปใช้ข้อมูลเก่าต้องส่งเสียงดัง หรือไม่ก็อย่ามีเลย ถ้าไม่เจอ input ที่ขอมา ให้ล้มพร้อมบอก path ที่ไปหามา อย่าแอบเอาชุดข้อมูลเก่ากว่าที่ยัง parse ผ่านมาใช้แทนเฉยๆ

eval_fields.py พัง เพราะ `ip.max\_pixels` กลายเป็น property แบบอ่านอย่างเดียวใน transformers เวอร์ชันที่ติดตั้งจริง

กลาง 2026-07-21

ต้อง pin เวอร์ชัน library ที่สคริปต์ train/eval ใช้ให้ตายตัว และตรวจสอบทุกการเขียนค่า config โมเดลกับเวอร์ชัน library ที่ติดตั้งจริงบนเครื่องเข้า ก่อนจะรันงานที่เสียเงินจริง

template matching ดิบๆ รายงานว่าเจอ 235 จุด ทั้งที่ฐานรากจริงมีแค่ 14 ตัว

กลาง 2026-08-25

detector แบบ sliding-window หรือแบบ score-map ต้องผ่าน NMS ก่อนถึงจะนับผลได้ อย่าเอาจำนวนพิกเซลที่คะแนนเกิน threshold มารายงานเป็นจำนวนวัตถุจริงๆ

ตั้ง threshold ของ self-harvest ผิดฝั่ง ค่า seed 0.85 ไม่มีทางถูกยิงได้เลย

กลาง 2026-08-26

อย่าตั้ง threshold ความคล้ายโดยไม่พล็อตการกระจายคะแนนของตัวอย่างบวกจริงกับลบจริงก่อน threshold ที่อยู่สูงกว่าการกระจายของตัวอย่างบวกคือ no-op แบบเฉยๆ ไม่ใช่ตัวกรองที่เข้มงวด

บั๊กป้าย Set-of-Mark ทับกัน 2 จุด ที่ unit test มองไม่เห็น

กลาง 2026-08-26

อะไรก็ตามที่ผลลัพธ์ออกมาเป็นภาพ ต้องเปิดดูด้วยตาก่อนปล่อย unit test พิสูจน์ได้แค่ว่าข้อมูลถูก มีแต่การมองเท่านั้นที่พิสูจน์ว่า render ออกมาแล้วอ่านได้

--pass25 เขียนแต่ไฟล์ sidecar ชื่อ *25 ขณะที่ apply_arm() อ่านไฟล์ชื่อธรรมดา

กลาง 2026-08-28

ฝั่งที่ผลิตกับฝั่งที่ใช้ไฟล์ต้องใช้ค่าคงที่ตัวเดียวกันในการตั้งชื่อ เวลา มีสอง pass ที่ผลิตไฟล์หน้าตาคล้ายกัน ต้องตรวจสอบว่าฝั่งผู้ใช้หาไฟล์ของฝั่งผู้ผลิตเจอจริงๆ การอ่านแล้วได้ค่าว่างไม่นับเป็น error

prompt ของ pass3 สั่งให้ตอบฟิลด์ `stirrup\_tie\_count` ซึ่งไม่มีอยู่ใน schema

กลาง 2026-08-28

ชื่อฟิลด์ที่เขียนใน prompt ถือเป็นส่วนของสัญญา schema ให้เพิ่มตัวตรวจว่าทุกชื่อฟิลด์ที่ prompt ขอ ต้องมีอยู่จริงใน schema — เป็นตัวกันประเภทเดียวกับที่ใช้เช็คอักขระต้องห้ามอยู่แล้ว

เทมเพลตเสา t6/t7 พนผลตรวจจับขยะ 49-96 อันต่อหน้า ที่ threshold 0.75 ที่เพิ่งปรับขึ้น

กลาง 2026-08-28

threshold เป็นค่าประจำเทมเพลตแต่ละตัว ไม่ใช่ค่าคงที่ตัวเดียวใช้ทั้งระบบ ก่อนปล่อยเทมเพลตใหม่ให้กวาดทดสอบทั้งคลัง และดูจำนวนขยะแยกรายเทมเพลต ไม่ใช่ดูแค่ยอดรวมผ่าน/ไม่ผ่าน

OCR ครอปภาพโดยใช้สัดส่วนของ PDF ต้นทาง ทั้งที่ภาพเป็น PNG ที่จัดเรียงใหม่แล้ว — ครอปผิดกรอบไปเลย

กลาง 2026-08-29

คำนวณพิกัดครอปจากภาพตัวที่โค้ดจะอ่านจริงเท่านั้น ห้ามคำนวณจากรูปแบบต้นทางก่อนหน้า และให้ดูผลครอปหนึ่งใบด้วยตา ก่อนรันทั้งชุด

อภิธานศัพท์เกือบถูกฝังไว้ในไฟล์ที่ถูก generate เท่านั้น — เรนเดอร์รอบหน้ามันจะหายเจี๊ยบๆ

กลาง 2026-08-29

ห้ามแก้ไฟล์ที่ถูก generate เด็ดขาด ให้ใส่หัวข้อความ 'generated — do not edit' ไว้ในทุกไฟล์ผลลัพธ์ เพื่อให้ผลการทำงานแบบเจี๊ยบๆ ไม่ได้

อภิธานศัพท์ถูกฉีดเข้าไปสองรอบ — จาก _common.md ครั้งหนึ่ง และจาก prompt ของแต่ละ subtask อีกครั้ง

กลาง 2026-08-29

เวลาที่ย้ายเนื้อหาจากแบบใช้ร่วมกันมาเป็นแบบแยกรายชิ้น ให้ลบจุดคิดของเดิมในการแก้ครั้งเดียวกัน แล้วพิสูจน์ด้วยการนับจำนวนที่ปรากฏในไฟล์ผลลัพธ์จริง ไม่ใช่ด้วยการอ่านโค้ด

`subprocess(shell=True)` บน Windows ทำให้ `>` ใน `gpu\_ram>=90` กลายเป็นการ redirect ไฟล์ — `vastai search` คืบคาวางโดยไม่มีเบาะแส

กลาง 2026-08-30

ห้ามใช้ `shell=True` กับ argument ที่แทรกค่าเข้าไป ให้ส่งเป็น argv list เสมอ บน Windows นี่ไม่ใช่เรื่องสไตล์การเขียน มันทำให้ argument ที่มี `>`, `<`, `&` หรือ `|` เพี้ยนแบบเจี๊ยบๆ

`serve\_purson.py`: request ที่ไม่มีรูปพังด้วย IndexError ที่ `images[0]` และ `return dict, 500` ถูก FastAPI ตีความเป็น list ไม่ใช่ HTTP status

กลาง 2026-08-30

ตรวจ payload ของ request ที่ทุกขอบเขตของเซิร์ฟเวอร์ และยืนยันธรรมเนียมการคืน error ของ framework นั้นๆ (FastAPI ต้องใช้ `JSONResponse`) — status code ผิดทำให้ทุกความล้มเหลวกลายเป็นความสำเร็จปลอมที่ปลายทาง

load_split() หาไฟล์ภาพโดยอิงตำแหน่งของสคริปต์ แทนที่จะอิงรากของ repo

กลาง 2026-08-31

ยึดการหาไฟล์ทุกจุดกับค่าคงที่ที่ชี้รากของ repo อย่าพึ่ง working directory ของ process หรือตำแหน่งของสคริปต์ตัวเอง

ตัวกัน (guard) ที่เช็ค batch โยน NameError เพราะไปอ้างตัวแปรที่ลบไปแล้ว แล้วรายงานว่า fail ทั้งที่จริงผ่าน

กลาง 2026-08-31

ต้องทดสอบตัวกันในเคสที่ผ่านด้วย ไม่ใช่แค่เคสที่พัง และถ้าตัวกันตัวไหน 'ร้องผิดบ่อย' ให้ถือเป็นบั๊กสำคัญอันดับต้นๆ — ตัวกันที่คนไม่เชื่อถือแยกว่าไม่มีตัวกันเลย

progress bar บน UI ไม่มีทางขึ้นถึง 100% ได้เลย ถ้างานนั้นมีหน้า gridline รวมอยู่ด้วย

กลาง 2026-08-31

ต้องทำให้ test fixture ครอบคลุมเคสที่ส่งผลต่อค่ารวมที่คำนวณด้วย — fixture ที่ไม่มีเคสพิเศษนั้นอยู่เลย จะปล่อยให้บั๊ก off-by-one ผ่านไปได้ตลอดกาล

ปุ่มบน UI เปลี่ยนป้ายเป็น 'อ่านด้วย Gemini' ทั้งที่ backend เรียกใช้ Qwen จริงๆ

กลาง 2026-09-01

ห้ามใช้ป้ายที่รู้อยู่แล้วว่าไม่ตรงกับความจริงในระหว่างที่ยังรอการตัดสินใจ ให้เปลี่ยนกลับเป็นค่ากลางๆ ที่ไม่ผูกกับผู้ให้บริการ (เช่น 'อ่านด้วย AI') จนกว่า backend จะตรงกับชื่อจริงๆ

UI แสดงความคืบหน้าซ่อน progress bar ไว้ระหว่างรอ และไม่บอกเลยว่ากำลังรัน pass ไหนอยู่

กลาง 2026-09-01

งานที่ใช้เวลานานทุกงาน UI ต้องแสดงสถานะเสมอ — 0% พร้อมชื่อขั้นตอนปัจจุบัน ดีกว่า progress bar ที่ซ่อนอยู่ การไม่มี progress indicator ให้เห็นในงานที่ใช้เวลา 75 นาที เป็นสาเหตุจริงที่ทำให้เกิดบักรันงานซ้ำ

พอเพิ่ม pass3 เข้าไป ค่า ETA กลับกระโดดขึ้นเป็นหลายชั่วโมงตอนทำงานใกล้จะเสร็จแล้วพอดี

กลาง 2026-09-01

อะไรก็ตามที่คำนวณเกี่ยวกับขั้นตอนของ pipeline ต้องดึงมาจาก ordered step list เท่านั้น ห้ามเขียนเป็นเงื่อนไขแยกทีละขั้นตอนด้วยมือ ไม่งั้นทุกขั้นตอนใหม่ที่เพิ่มเข้ามาจะไปทำให้ขั้นตอนเก่าเพี้ยนตามไปด้วยโดยไม่รู้ตัว

โมเดล 3D โครงสร้างเรนเดอร์พัฒนาเพดาน ทอระบายน้ำ และปลั๊กไฟ ออกมาเป็นสองโครงสร้างไปด้วย

กลาง 2026-09-01

ให้ใช้ filter list ที่มีอยู่แล้วซ้ำ แทนที่จะดึงข้อมูลดิบมาใช้ตรงๆ และเวลาตัดรายการทิ้ง ต้องรายงานว่าตัดอะไรออกไปบ้าง ไม่ใช่ตัดทิ้งเงียบๆ

หน้าแบบที่มีแต่คานล้วนๆ ไม่มีทางวัดผลได้เลย เพราะจุดกึ่งกลางกลอง (centroid) ของคานไม่ได้อยู่บน grid ref ไหนเลย

กลาง 2026-09-01

การอ้างอิงตำแหน่งเชิงเรขาคณิตต้องเคารพลักษณะของแต่ละชิ้นส่วน — ชิ้นส่วนแบบจุด (point element) กับแบบเส้น (linear element) ใช้กฎ 'อยู่ตรงไหน' เดียวกันไม่ได้ ต้องเช็คข้อสันนิษฐานกับชิ้นส่วนแต่ละประเภทก่อนจะเหมารวมใช้

ส่วนหัวของโปรเจกต์ Single วางฟิลด์ 5 ช่องเป็นแถวเติมความกว้างซ้อนกัน ดันเนื้อหาจริงตกจ่อไปหมด

ต่ำ 2026-07-09

ต้อง screenshot annotation config ทุกอันที่ความละเอียดหน้าจอลงจริงที่ใช้งาน ก่อนส่งให้คนติดป้าย ส่วนหัวที่กินพื้นที่จ่อไปหนึ่งในสามไม่ใช่แค่ปัญหาความสวยงาม แต่มันกินเวลาทำงานจริงของคนติดป้าย

GPU แสดง utilisation 100% ทั้งที่ใช้ไฟแค่ 33.7W — คอขวดจริงคือการ offload ไป CPU

ต่ำ 2026-07-29

อย่าดู GPU utilisation เพียงอย่างเดียว ต้องดูคู่กับ power draw และ log การจัดสรรหน่วยความจำเสมอ และเมื่อค่าใช้จ่ายอยู่ที่การ generate ข้อความ ให้ลดจำนวน token ที่สร้าง แทนที่จะไปไล่แก้ที่ฮาร์ดแวร์

ผังฐานรากบ้าน 12 นับได้ 13 เส้า ทั้งที่จริงมีแค่ 12 — มีกลองซ้ำหลุดรอด NMS ไปหนึ่งอัน

ต่ำ 2026-08-25

ให้บันทึกข้อผิดพลาดของการตรวจจับแบบ off-by-one ที่รู้อยู่แล้วไว้ตรงๆ ไม่ต้องปิดเศษให้หายไป ผู้ใช้ปลายทางที่เอาตัวเลขไปเทียบต้องรู้ว่าเครื่องมือมีอคตินับเกินอยู่นิดหน่อย

จุดตรวจจับขยะที่อยู่นอกแถบคานแค่ 4 พิกเซล หลุดรอดตัวกรอง footing-in-beam ไม่ได้

ต่ำ 2026-08-25

ตัวกรองเชิงเรขาคณิตต้องมี tolerance margin ที่สอดคล้องกับความคลาดเคลื่อนของตัวตรวจจับเอง ตัวกรองที่ใช้ขอบเขตแบบเป๊ะๆ จะรัวตรงขอบเสมอ

ตัวแปร `h` ถูกใช้ซ้ำในสคริปต์ชุดข้อมูล — ชื่อบ้านกลายเป็นความสูงของกลองไปโดยไม่รู้ตัว

ต่ำ 2026-08-28

อย่าใช้ชื่อตัวแปรตัวอักษรเดียวซ้ำข้าม scope ในสคริปต์ที่ผสมทั้ง string และตัวเลข ชื่อตัวเดียวไม่คุ้มกับบั๊กประเภทนี้ที่มันเปิดช่องให้เกิด

self-check ของ cv_scan --demo พังเพราะการเปลี่ยนฟอร์แมตของเราเอง ("4." → "4")

ต่ำ 2026-08-29

ให้เขียน assertion ของ self-check ให้ตรวจเนื้อหาเชิงความหมาย ไม่ใช่รูปแบบตัวอักษรตรงๆ และเวลาไล่แก้ทั้ง repo ให้รัน self-check ทุกตัวซ้ำหลังทำเสร็จ เพราะการแก้แบบกวาดทั้งชุดจะทำให้บาง self-check พังไปด้วย

ใส่เครื่องหมาย `\*\*` ต้องห้ามลงไปโดยไม่ตั้งใจ ระหว่างที่กำลังเขียนตัวเคลียร์ prompt ที่มีหน้าที่ลบมันออกพอดี

ต่ำ 2026-08-29

เวลาทบทวนห้ามใช้ตัวอักษรที่เป็น syntax เริ่มต้นของโปรแกรมแก้ไขเราเอง การสแกนอัตโนมัติไม่ใช่ทางเลือก ต้องรันหลังการแก้ไขทุกครั้ง แม้แต่การแก้ไขจากคนที่เขียนกodicานั้นเอง

assertion ของ selftest ที่เขียนเองผิดเอง จนเกือบทำให้เข้าใจผิดว่า `strip\_fence` มีบั๊ก

ต้า 2026-08-30

เวลาทดสอบใหม่ล้มเหลวบนโค้ดเก่าที่ย้ายมาและเคยใช้งานได้ดี ให้สงสัยตัวทดสอบก่อนสงสัยโค้ด — ต้องเช็คค่าที่คาดหวังกับผลลัพธ์จริงของโค้ดต้นฉบับให้แน่ใจก่อน

bash ฟ้อง 'unbound variable' ทั้งที่ argument ถูกส่งมาแล้ว ต้นเหตุคือประกาศ local หลายตัวไว้บรรทัดเดียวกันภายใต้ set -u

ต้า 2026-08-31

ประกาศ local แยกบรรทัดละตัวเสมอใน bash โดยเฉพาะเวลาใช้ set -u และเวลาสงสัยว่า shell มีบั๊กให้ยืนยันด้วย bash -x แทนการนั่งเดา

มี 3 ไฟล์เขียน window.x ตรงๆ โดยไม่มีตัวกัน ทำให้ทดสอบไม่ได้เลย

ต้า 2026-08-31

ให้ทำตามธรรมเนียมตัวกันของโปรเจกต์เวลาจะแปะตัวแปรลง global เพื่อให้ทุกโมดูล import ได้ในสภาพแวดล้อมทดสอบ

Git ความสะอาดของ repo และ ของที่ทำแล้วหาย

commit สาขา การ merge และงานที่ทำเสร็จแล้วแต่ไม่ได้เก็บ หรือของที่ลบไปแล้วกลับมา

ทั้งหมด 19 เรื่อง · วิกฤต 2 · สูง 3 · กลาง 6 · ต่ำ 8

เรื่องที่ต้องอ่านให้ครบ (5)

วิกฤต 2026-07-10

มีคน **force-rewrite** ประวัติทั้งหมดของ repo **Constistant** ทำให้ต้อง **hard reset** ฝั่งเครื่องตัวเอง

เกิดอะไรขึ้น: เกิดเหตุ git ประวัติแตกแยกในวันเดียวกันก่อนหน้านี้: ประวัติทั้งหมดของ repo **Constistant** ถูก **force-rewrite** ที่ฝั่ง upstream ทำให้ main ในเครื่องต้อง reset ให้ตรงกับ origin/main มีการตัดสินใจว่าไม่เกี่ยวกับงาน raw_json เลยจึงใจไม่บันทึกรายละเอียดไว้

ต้นเหตุจริง: การ **force-push** ไปเขียนประวัติที่เผยแพร่ไปแล้วทับ ทำให้ clone อื่นๆ ทุกตัวแตกจากมัน และไม่มีบันทึกว่าการแจ้งหรือประสานงานกันไว้เลย

เสียหายอะไร: ต้อง **hard reset main** ในเครื่อง และ commit ที่มีเฉพาะในเครื่องบน branch นั้นเสียหายทั้งหมด รายละเอียดของเหตุการณ์ไม่เคยถูกจดไว้ ต้นเหตุจริงจึงยังไม่มีใครรู้จนถึงตอนนี้

แก้ยังไง: reset main ในเครื่องให้ตรงกับ origin/main

กฎที่ได้: อย่า **force-push** branch ที่ใช้ร่วมกันเด็ดขาด และถ้าเกิดขึ้นแล้ว ให้บันทึกว่าอะไรถูกเขียนทับและงานในเครื่องอะไรหายไปบ้าง — คำว่า 'ไม่เกี่ยวกับงานวันนี้' นี่แหละคือวิธีที่ทำให้ต้นเหตุจริงหายไป

วิกฤต 2026-07-11

อ่านหน้า **50-51** พร้อมกันแล้วค่อยเขียนจากความจำ ทำให้ข้อมูลสลับกันไปมาระหว่าง **BOQ** สามแผ่น

เกิดอะไรขึ้น: หน้า 50 และ 51 ของบ้านหลังที่ 2 ถูกอ่านพร้อมกัน แล้วค่อยมาเขียนทีหลัง ผลคือข้อมูลโครงหลังคาเหล็กจากหน้า 50 (แผ่นที่ 3/19) ถูกเขียนลงไฟล์งานมุงหลังคา (แผ่นที่ 5/19) ส่วนข้อมูลจริงของแผ่น 5/19 ไม่เคยถูกเขียนลงไปเลย และแผ่น 6/19 (งานฝ้าเพดาน) ถูกทิ้งวางเพราะเข้าใจว่าอ่านไม่ออก ทั้งที่ข้อมูลมีอยู่ แต่ถูกเก็บผิดไฟล์

ต้นเหตุจริง: ตาราง **BOQ** หลายตารางที่หน้าตาคล้ายกันถูกจำไว้ในหัวข้ามการอ่านหลายหน้า ก่อนที่จะเขียนอะไรลงดิสก์สักอัน ตัวตนของหน้าจึงหายไประหว่างการอ่านกับการเขียน

เสียหายอะไร: ไฟล์ข้อมูลจริงผิดพร้อมกันสามไฟล์ — ไฟล์หนึ่งมีข้อมูลของอีกหน้า ไฟล์หนึ่งวางทั้งที่ควรมีข้อมูล และอีกไฟล์ถูกทำเครื่องหมายผิดๆ ว่าอ่านไม่ได้ ทั้งหมดนี้อยู่ในชุดข้อมูลที่ตั้งใจเอาไปใช้ **fine-tune**

แก้ยังไง: กลับไปอ่านหน้า 50 และ 51 ใหม่ทีละหน้าตั้งแต่ต้น เขียนทั้งสามไฟล์ใหม่ให้ถูก และใส่คำเตือนอธิบายสาเหตุไว้ในแต่ละไฟล์ที่แก้

กฎที่ได้: อ่านหน้าเดียวแล้วเขียนไฟล์ของหน้านั้นให้เสร็จก่อนอ่านหน้าถัดไป — ห้ามอ่านรวดเดียวหลายหน้าแล้วค่อยเขียนจากความจำ โดยเฉพาะตารางซ้ำๆ อย่าง **BOQ** ที่แต่ละหน้าหน้าตาเหมือนกัน

สูง 2026-07-07

โพลเดอร์ pipeline ทั้งชุดกลายเป็นสำเนาเก่าค้างอยู่ในอีก repo นาน 5 วัน

เกิดอะไรขึ้น: Constaint/No_touch_box/ กลายเป็นสำเนาของโพลเดอร์ pipeline จาก Training repo ที่ถูก freeze ไว้ตั้งแต่ 2 ก.ค. — ขาด mk_test/, ขาด config Label Studio ของมะขาม แถมยังมี CLAUDE.md เวอร์ชันเก่า ไฟล์หนึ่งในนั้น (label-studio-tasks.js) เป็นเวอร์ชันก่อนแก้บั๊ก ยังชี้ไปที่ path qwen-output แบบเก่า

ต้นเหตุจริง: งานถูกส่งต่อระหว่างสอง repo ด้วยการ copy โพลเดอร์ แทนที่จะกำหนดให้แต่ละไฟล์มี repo เจ้าของเดียว พอต้นฉบับอัปเดตต่อ สำเนา ก็หยุดรับการอัปเดตไปเสียเลย ทันที

เสียหายอะไร: ข้อมูลเพี้ยนไปนาน 5 วัน มีสคริปต์บั๊กเก่าค้างอยู่บนดิสก์ที่ใครก็เผลอรันได้ ต้องเสียเวลาทำความสะอาดหลายรอบ (7 ก.ค. สอกรอบ บวกส่วนที่ยังไม่เสร็จ) แะเพื่อพิสูจน์ว่าสำเนาไม่ทำอะไรที่ไม่เข้าเลย

แก้ยังไง: diff ไฟล์แต่ละตัวเทียบกับตัวจริงใน Training repo (3 ไฟล์เหมือนกันเป๊ะ, 1 ไฟล์เก่ากว่าชัดเจน) แล้วลบ rule_of_tune.md กับไฟล์ Label Studio ทั้ง 4 ตัว ส่วนที่เหลือของโพลเดอร์ยังรอการตัดสินใจ

กฎที่ได้: ทุกไฟล์ต้องมี repo เจ้าของเดียวเท่านั้น ถ้าจำเป็นต้องเห็นไฟล์เดียวกันจากสอง repo ให้ link หรืออ้างอิงถึงกัน ห้าม cp เด็ดขาด เพราะการ copy คือการ fork ที่ไม่มีใครจำได้ว่าต้อง merge กลับ

สูง 2026-07-10

งานสัปดาห์ (ตั้งแต่รอบ t4) ไม่ได้ commit เลย และยังคงค้างอยู่ถึงวันที่ 11 ก.ค.

เกิดอะไรขึ้น: ตั้งแต่รอบ t4 เป็นต้นมาไม่มีการ commit เลยสักครั้ง งานค้างวันที่ 9 ก.ค. เขียนว่า "ยังไม่ได้ commit อะไรเลยตั้งแต่รอบ t4" วันที่ 10 ก.ค. ย้ำอีกว่า "ค้างมาหลายวันแล้ว" และวันที่ 11 ก.ค. ก็ยังอยู่ในรายการ ครอบคลุมทั้งการเขียน schema ใหม่ การแก้ generator ไฟล์ XML หาไฟล์ raw JSON สามหลัง และเหตุการณ์เปลี่ยนชื่อโพลเดอร์ raw/

ต้นเหตุจริง: การ commit ถูกเลื่อนไปราบตอนจบงานทีเดียว ขณะที่แต่ละวันก็ใส่ของใหม่ที่ไม่เกี่ยวกับเข้ามาเรื่อยๆ และมีอุบัติเหตุอย่างน้อยหนึ่งเรื่อง (การเปลี่ยนชื่อ raw/) ที่ไปแตะ working tree เดียวกัน

เสียหายอะไร: งานสกิดข้อมูลที่ทำใหม่ไม่ได้ทั้งหลายวัน อยู่แค่ใน working tree ที่ยังไม่ commit ในรีโปที่ประวัติเพิ่งถูกเขียนทับแบบ force ในช่วงเดียวกันนั้นพอดี และ commit ที่เกิดขึ้นก็จะเป็นทั้งอุบัติเหตุเปลี่ยนชื่อ การแก้ config และข้อมูลบ้านสามหลังรวมกันมั่วไปหมด

แก้ยังไง: ยังไม่ได้แก้ — ยังอยู่ในรายการงานค้างตอนจบวันที่ 11 ก.ค.

กฎที่ได้: commit ทุกครั้งที่จบรอบงาน โดยขอบเขตจำกัดแค่อรอบนั้น — งานค้างที่ยังไม่ commit ทั้งเสียหายก็ไม่ได้ และรับประกันได้เลยว่า commit ที่ออกมาสุดท้ายจะรวีวไม่ไหว

งานที่ **commit** เข้า ``main`` มองไม่เห็นเลยสำหรับมะขาม ซึ่งทำงานอยู่บนสาขา ``Presentation-demo`` ที่แยกออกไปแล้ว

เกิดอะไรขึ้น: ผลลัพธ์ของบ้าน09 ถูก commit เข้า ``main`` แต่มะขามทำงานอยู่บน ``Presentation-demo`` เขาไปหาไฟล์บ้าน09 ผ่าน `GO.bat` แต่หาไม่เจอ เพราะสองสาขามีประวัติแยกกันคนละทางและไม่เคยถูก merge เข้าด้วยกันเลย

ต้นเหตุจริง: สองสาขาที่มีอายุนานแยกออกจากกันโดยไม่มีแผน merge และไม่มีข้อตกลงว่าสาขาไหนคือสาขาที่ใช้งานจริง ผลลัพธ์ถูก commit เข้าสาขาที่ผู้ใช้จริงไม่ได้อยู่ โดยไม่มีอะไรมาเตือนว่าเกิดความไม่ตรงกันนี้

เสียหายอะไร: มะขามหาสิ่งที่เพิ่งขอไปไม่เจอ ต้องคัดลอกไฟล์ทั้ง 17 ไฟล์ด้วยมือไปไว้ที่ ``D:\000download\09`` เป็นทางแก้ชั่วคราว และยังทิ้งการตัดสินใจเรื่อง merge ไว้ค้างคาไม่จบ

แก้ยังง: คัดลอกไฟล์ออกไปสำรองไว้ที่ ``D:\000download\09`` นอกเหนือจาก git ส่วนเรื่อง merge เองบันทึกไว้เป็นการตัดสินใจที่ยังค้างอยู่

กฎที่ได้: ตกลงกันให้ชัดว่าสาขาไหนคือสาขาที่ใช้งานจริงก่อนจะ commit ผลลัพธ์ลงไป และถ้าสาขาต้องแยกออกจากกันจริงๆ ต้องตัดสินใจจุด merge ทันทันที — อย่าปล่อยไว้ให้ merge ที่หลัง ค้างอยู่ ในขณะที่ผลลัพธ์ไปตกอยู่ที่ผู้ใช้มองไม่เห็น

เรื่องที่เหลือในหมวดนี้ (14)

- t3 บันทึกความชันหลังค่าไว้ 9 องศา แต่ในแบบเขียนว่า 6**

กลาง 2026-07-08

คำใดที่ปรากฏบนแบบมากกว่าหนึ่งแผ่น ต้องอ่านจากแผ่นที่ชัดที่สุดแล้วตรวจไขว้กัน และการสกัดข้อมูลรอบใหม่ต้องห้ามให้เห็นคำตอบของรอบก่อนเด็ดขาด
- หน้าที่คาตจากสารบัญว่าเป็นเกณฑ์ผังบริเวณ กลับเป็นอย่างอื่นไปคนละเรื่อง**

กลาง 2026-07-10

สารบัญบอกได้แค่โครงสร้าง ไม่ได้บอกเนื้อหา ทุกหน้าต้องจำแนกจากตัวหน้าเอง และต้องอนุญาตให้ agent เกี่ยงกับแผนที่มันได้รับมาได้
- ราคาของบ้านหลังที่ 7 อ่านผิดเป็น 1,572,000 ทั้งที่ตัวจริงคือ 1,528,000**

กลาง 2026-07-16

ตารางตัวเลขตัวพิมพ์เล็กต้องมีขั้นตอนตรวจข้ามแหล่งเป็นมาตรฐาน ไม่ใช่ทำเป็นครั้งคราว — เทคนิคเทียบข้ามบ้านเลขตัวเลขผิดถึง 3 จุดใน 2 รอบ ควรใช้กับทุกบ้าน
- บ้าน 07 แผ่น S-05: บันทึกตำแหน่ง BS1 ว่าอยู่ต่ำจากกริด C 1.1 ม. ทั้งที่จริงประมาณ 2.1 ม.**

กลาง 2026-07-28

ทุกระยะที่อ่านมา ให้เช็กกับขนาดช่วงเสาที่มันอยู่ข้างในด้วย ถ้าระยะนั้นใส่ลงในช่วงไม่ได้ หรือเหลือเศษที่ดูไม่สมเหตุผล แปลว่าอ่านตัวเลขผิดหลัก
- 12 commits ของ King-try (โปรโตไทป์ overlay, hooks, skills, agent) ค้างเติ่ง ไม่ได้ cherry-pick อะไรเลย เลื่อนการตัดสินใจข้าม session ไปเรื่อยๆ**

กลาง 2026-08-20

กึ่งที่แยกออกไปแล้วมีงานที่เราอยากได้อยู่ข้างใน คือทรัพย์สินที่เสื่อมค่าลงเรื่อยๆ ให้ cherry-pick commit ที่ต้องการใน session เดียวกับที่รีว ไม่งั้นต้องกลับมารีวใหม่ทั้งหมด แล้วช่องว่างก็จะยิ่งถ่างออก
- ความถูกต้องของ output คมด้วยการกรองที่หลัง ไม่ใช่คมด้วย grammar ตั้งแต่ตอนสร้างผล**

กลาง 2026-08-31

ควรบังคับ output ด้วย schema grammar ตั้งแต่ต้นทาง แทนที่จะกรองปลายทาง เพราะการกรองที่หลังเอาขยะออกได้ก็จริง แต่ไม่มีทางกู้ข้อมูลที่หายไปกลับมาได้

agent ที่ถูก kill ทั้งไฟล์ scratch ค้างไว้ในโฟลเดอร์ ground-truth โดยไม่มีใครมาเก็บกวาด

ต้า 2026-07-05

ให้เขียนไฟล์ scratch และไฟล์บันทึกความสืบหน้าลงในโฟลเดอร์ scratch แยกต่างหากเสมอ ห้ามเขียนลงในโฟลเดอร์ข้อมูลจริง การเก็บกวาดที่เกิดขึ้นเฉพาะตอน agent จบงานแบบปกติ ไม่ใช่การเก็บกวาดจริง ต้องคิดไว้ตั้งแต่แรกว่าการจบงานแบบผิดปกติคือกรณีปกติที่ต้องรองรับ

repo สะสมโฟลเดอร์โครงร่างเปล่า ไฟล์ว่าง สคริปต์ชั่วคราว และโฟลเดอร์ทดสอบซ้ำซ้อนไว้เต็มไปหมด

ต้า 2026-07-05

ห้ามสร้างโครงร่างไว้ก่อนที่จะมีโค้ดมาเติมจริง ห้ามเขียนไฟล์ชั่วคราวไว้ที่ root ของ repo และห้ามมีสองโฟลเดอร์ที่ชื่อต่างกันแค่เอกพจน์/พหูพจน์ เพราะคู่แบบนั้นจะแยก scope ของ tooling ออกจากกันเสียๆ

สคริปต์ migration ที่ถูกยกเลิกยังถูกทิ้งไว้บนดิสก์ ไม่มีเจ้าของและไม่มีการตัดสินใจว่าจะทำอย่างไรต่อ

ต้า 2026-07-08

เมื่อผลลัพธ์ของงานถูกทิ้งไปแล้ว ให้ลบหรือกักตัวเครื่องมือที่สร้างมันในขั้นตอนเดียวกันเลย การปล่อยให้มันยังรันได้อยู่คือการเปิดช่องให้เกิดความผิดพลาดซ้ำ

โฟลเดอร์ trash/ ที่มี 11 ไฟล์อยู่ในตัว repo แต่ไม่ได้อยู่ใน gitignore

ต้า 2026-07-09

ต้องตัดสินใจตั้งแต่ตอนสร้างโฟลเดอร์กักเก็บ (quarantine) ว่ามันควรอยู่ในสถานะ git แบบไหน ค่าเริ่มต้นคือมันจะถูก commit ไปด้วย และถ้าปล่อยให้ค้างไว้เรื่อยๆ จะพลาดสังเกตได้ง่าย

ไฟล์ driver สำหรับ inference 61 หน้าที่ใช้งานได้จริง มีอยู่เพียงชุดเดียวในสคริปต์แพด ไม่ได้อยู่ใน repo คู่กับผลลัพธ์ที่มันสร้างขึ้น

ต้า 2026-07-28

สคริปต์ที่สร้างผลลัพธ์ที่ commit ไปแล้ว ถือเป็น work product ต้อง commit ไว้คู่กับผลลัพธ์นั้นเลย โดยเฉพาะเมื่องานนั้นตั้งใจให้ resume ต่อได้

สคริปต์ pipeline รุ่นเก่าที่ถูกแทนที่แล้ว ยังอยู่บนดิสก์ ไม่มีใครตัดสินใจว่ามันตายแล้วหรือยัง

ต้า 2026-08-02

ตอนทำความสะอาด ให้จัดกลุ่มตามว่า 'ยังใช้งานอยู่หรือตายแล้ว' ไม่ใช่จัดตามว่าไฟล์นั้นเคยเป็นของพีเจอาร์ไหน ให้เช็คว่าไฟล์ owner script ของมันคืออันไหน แล้วเช็คแค่ครั้งเดียวและจดบันทึกไว้

งานเช็คลิสต์ accept/reject ของ Constant (แยก preview/save + UI) ค้างไว้โดยไม่ได้ commit

ต้า 2026-08-20

ให้ commit ส่วนที่ใช้ซ้ำได้ก่อน ค่อยไขว่ส่วนที่ยังไม่แน่ใจทีหลัง พีเจอาร์ที่ถูก reject ไม่ควรลากงาน refactor ที่ผ่านแล้วให้ตกไปด้วย

โค้ดสับตัวแปร TARGET/ELEMENT_TYPES ที่ตายไปแล้วยังหลงเหลืออยู่ในตัวสร้างชุดข้อมูล (dataset builder)

ต้า 2026-08-29

ให้ลบกลไกทิ้งพร้อมกับเหตุผลที่มันมีอยู่ โค้ดสับค่าที่ตายแล้วใน data pipeline อันตรายกว่าโค้ดตายที่อื่น เพราะมันทำให้ดูเหมือนกำลังมีการแปลงข้อมูลอยู่ ทั้งที่จริงๆ ไม่มีอะไรเกิดขึ้นเลย

เอกสาร การตั้งชื่อ และการสื่อสาร

เอกสารที่บอกไม่ตรงความจริง ชื่อที่ทำให้เข้าใจผิด และคำเตือนที่ไม่ถูกพูดให้ดังพอ

ทั้งหมด 34 เรื่อง · วิกฤต 3 · สูง 7 · กลาง 11 · ต่ำ 13

เรื่องที่ต้องอ่านให้ครบ (10)

วิกฤต 2026-07-21

รู้ยู่แล้วว่ามีความเสี่ยงข้อมูลหายถาวร แต่ไม่เคยพูดออกมาเป็นคำเตือนแบบบล็อกงาน

เกิดอะไรขึ้น: ความเสี่ยงที่ว่าการ destroy instance จะลบผลลัพธ์ทั้งถาวรนั้น รู้กันตั้งแต่ตอนแปลง GGUF เสร็จ แต่ถูกพูดถึงแค่ในฐานะรายงานเบาๆ ('ยังไม่ได้อัปเดต HuggingFace เต็มค้อยทำ') ไม่เคยถูกพูดเป็นคำเตือนชัดๆ ว่า 'ห้ามปิดเครื่องก่อนสำรองข้อมูล'

ต้นเหตุจริง: การสื่อสารความเสี่ยงที่ย้อนกลับไม่ได้ ถูกยัดรวมอยู่ในรายการสิ่งที่ต้องทำ แทนที่จะยกระดับเป็นคำเตือนที่บล็อกงาน งานที่เลื่อนไว้กับคำเตือนความเสี่ยงทำลายล้าง ถูกจัดเป็นข้อความประเภทเดียวกัน

เสียหายอะไร: เป็นเหตุโดยตรงที่ทำให้ข้อมูลสูญทั้งหมดตามข้อข้างบน และถูกบันทึกเป็นกฎ 'Mark of Shame' ใน rule_of_tune.md

แก้ยังง: เพิ่มกฎถาวรว่า ความเสี่ยงใดๆ ที่ทำลายข้อมูลถาวรได้ ต้องพูดออกมาเป็นคำเตือนแบบบล็อกงานที่แยกออกมาชัดเจน

กฎที่ได้: แยกคำเตือนความเสี่ยงทำลายล้างออกจากรายการงานทั่วไป — พูดออกมาจริงๆ ด้วยภาษาที่เข้าใจง่าย และพูดทันที ก่อนถึงจังหวะที่ความเสี่ยงนั้นจะเกิดจริง

วิกฤต 2026-07-24

ไฟล์หายไปบน GPU ที่เข้ามา เพราะคำเตือน "ยังไม่ได้สำรองข้อมูล" ถูกพูดแบบเบาๆ แทนที่จะเป็นคำสั่งหยุด

เกิดอะไรขึ้น: วันที่ 2026-07-21 งานบนเครื่องเช่าหายหมด บันทึกชั้นสุดในไดอารี่ระบุชัดว่าไม่ใช่บั๊กในโค้ด แต่เป็นเพราะผู้ช่วยพูดถึงเรื่องไฟล์ที่ยังไม่ได้สำรองแบบผ่านๆ เบาๆ ไม่ได้ยกขึ้นมาเป็นคำเตือนแบบบังคับหยุด เลยไม่มีใครลงมือทำอะไรก่อนเครื่องจะหายไป

ต้นเหตุจริง: การสื่อสารความเสี่ยงไม่มีระดับความรุนแรง สถานการณ์ที่อาจทำลายข้อมูลถาวร (งานที่ยังไม่ได้สำรองบนเครื่องเช่าชั่วคราว) ถูกพูดด้วยน้ำเสียงเดียวกับรายงานความคืบหน้าทั่วไป คนอ่านเลยแยกไม่ออกว่าอันไหนคือ "แจ้งให้ทราบ" อันไหนคือ "หยุดเดี๋ยวนี้"

เสียหายอะไร: งานบนเครื่องเช่าหายหมด และต้องใช้วันที่ 2026-07-24 ไปกับการสร้างและรันระบบตรวจสอบก่อนบิน Phase 0 เพื่อ กันไม่ให้เกิดซ้ำ — เสียเวลาทำงานกระบวนการเติมๆ หนึ่งวันแบบไม่ได้วางแผนไว้

แก้ยังง: กลายเป็นกฎข้อ 4 ใน rule_of_tune.md และเป็นที่มาของ checklist "Phase 0" ก่อนเข้าเครื่องใน RETUNE_WORKFLOW.md ต้องตรวจสอบให้แน่ใจว่าสำรองข้อมูลแล้วก่อนจะทำลายอะไรที่เป็นของชั่วคราว

กฎที่ได้: อะไรก็ตามที่ทำให้งานหายแบบกู้ไม่ได้ ต้องพูดเป็นคำสั่งหยุดชัดๆ ใช้คำว่า STOP ห้ามพูดแบบผ่านๆ ถ้าผลที่ตามมา ย้อนกลับไม่ได้ ให้ปฏิเสธที่จะทำต่อไปเลย ดีกว่าเอ่ยถึงมันแค่ผ่านๆ

เส้นทาง notes → Constistant ไม่ได้อะไรเลย: ตัว adapter อ่านไฟล์ที่ schema ไม่เคยกำหนดไว้

เกิดอะไรขึ้น: raw-extraction-adapter.js อ่าน notes.fc_ksc / fy_main_ksc / fy_stirrup_ksc / cover_mm เพื่อสร้าง extractedNotes แต่ primary_rawjson_schema.md ไม่เคยกำหนดโครงสร้างนั้นไว้เลย ตัว fc_ksc ไม่มีในสเปกเลยสักที และมีอยู่ในไฟล์ notes จริงแค่ 1 จาก 55 ไฟล์ สรุปคือสเปกคอนกรีตและเหล็กระดับทั้งโครงการไม่เคยไปถึง pipeline เลยสำหรับบ้านแทบทุกหลัง

ต้นเหตุจริง: ฝั่งที่เอาข้อมูลไปใช้เขียนขึ้นตามโครงสร้างไฟล์ที่ "หวังว่าจะมี" แทนที่จะเขียนตามสเปก แล้วสเปกก็ไม่เคยถูกอัปเดตให้ตรงกัน ไม่มีอะไรเช็คว่าไฟล์ที่ adapter อ่านมีอยู่จริงใน schema ใหม่ เส้นทางข้อมูลทั้งเส้นเลยอ่านไฟล์ที่ไม่มีอยู่และคืนค่าว่างไปเรื่อยๆ โดยไม่ error สักครั้ง

เสียหายอะไร: ชั้นคุณภาพคอนกรีต ชั้นคุณภาพเหล็ก และระยะหุ้ม — ซึ่งเป็นสเปกวัสดุระดับทั้งโครงการ — ไปถึง pipeline ของ BOQ ได้อย่างมาก 1 จาก 55 ไฟล์ notes คือแทบไม่เคยไปถึงเลย และเสียบมาตั้งแต่วันที่เขียน adapter ตัวนี้

แก้ยังไง: มะขามเลือกทาง "ไปกำหนดไฟล์ในสเปกให้ตรง" แทนที่จะแก้ฝั่งที่เอาไปใช้ เพิ่ม §4a ใหม่: มีชั้นแบบซ้อนที่บันทึกสิ่งที่พิมพ์อยู่บนแบบจริงๆ (concrete{}, steel{round_bar{},deformed_bar{}}, cover{default_mm,by_condition[]}) บวกกับ alias แบบแบนทางเดียวอีก 4 ตัว (fc_ksc/fy_main_ksc/fy_stirrup_ksc/cover_mm) เพื่อให้ adapter ไม่ต้องแก้อะไรเลย ส่วนการย้ายข้อมูลไฟล์เดิม 55 ไฟล์ยังค้างอยู่

กฎที่ได้: ทุกไฟล์ที่ adapter อ่าน ต้องมีอยู่จริงในสเปก — เพิ่มเทสที่ตรวจไขว้ว่าไฟล์ที่ adapter อ่านมีใน schema ใหม่ เพราะ adapter ที่อ่านไฟล์ที่ไม่มีอยู่จะพังแบบเสียบๆ ด้วยการคืนค่าว่าง ซึ่งหน้าตาเหมือนกับ 'บ้านหลังนี้ไม่มี notes' เป๊ะ

ย่อ schema แล้วทำกฎการทำงานจริงหล่นหายไป 3 ข้อโดยไม่รู้ตัว

เกิดอะไรขึ้น: พอเอา primary_rawjson_schema.md มาเทียบกับต้นฉบับ 219 บรรทัดที่ละห้วงข้อ พบว่ามีกฎจริงหายไป 3 ข้อ ไม่ใช่ประวัติที่ตั้งใจตัดออก คือ ขอบเขตงานอัตโนมัติ (run_pipeline.py สกัดอัตโนมัติแค่ 6 จาก 13 pattern), กฎ atomic-segment (คานต้องรายงานแยกทีละช่วงกริด ห้ามรวมยอดมาก่อน) และคำถามที่ยังค้างเรื่อง element_id ซนกันข้ามไฟล์ section

ต้นเหตุจริง: การย่อทำด้วยการเขียนใหม่ ไม่ได้ไล่ทีละห้วงข้อจากต้นฉบับ ของที่หายเลยมองไม่เห็นจนกว่าจะเอามา diff กันจริงๆ

เสียหายอะไร: schema ตัวที่ใช้งานจริง — ไฟล์ที่กำหนดการสกัดข้อมูลของบ้านทุกหลังต่อจากนี้ — ขาดทั้งกฎ atomicity ของคานและขอบเขตงานอัตโนมัติ ถ้าไม่ได้เอามา diff บ้านทุกหลังหลังจากนั้นจะถูกสกัดด้วยข้อกำหนดที่ไม่ครบ

แก้ยังไง: เทียบทีละห้วงข้อกับต้นฉบับฉบับเต็ม แล้วใส่กฎทั้งสามข้อกลับเข้าไป

กฎที่ได้: ย่อสเปกเสร็จแล้วต้อง diff กลับกับต้นฉบับทีละห้วงข้อ และพิสูจน์ว่าไม่มีกฎการทำงานข้อไหนหาย — งานเขียนใหม่เชื่อไม่ได้ว่ามันเก็บสิ่งที่มันไม่ได้พูดถึงไว้ครบ

สูง 2026-07-21

บอกว่า GGUF 'พร้อมใช้แล้ว' ทั้งที่มันอ่านภาพไม่ได้เลย

เกิดอะไรขึ้น: รายงานว่า GGUF ที่ quantize แล้วพร้อมใช้งาน ทั้งที่ส่วน mmproj/วิชันหายไป คือมันทำสิ่งเดียวที่โปรเจกต์นี้มีอยู่เพื่อทำ (อ่านแบบก่อสร้าง) ไม่ได้เลย

ต้นเหตุจริง: รายงานสถานะจากการที่ pipeline รันจบ ('แปลงเสร็จแล้ว') แทนที่จะรายงานจากความสามารถ ('ไฟล์ที่ได้ทำงานได้จริงไหม') ทำให้เคลมเกินจริง

เสียหายอะไร: ทำให้เข้าใจผิดว่ามีของที่ใช้กันได้จริงแล้ว ซึ่งมีส่วนทำให้ตัดสินใจปิดเครื่องไป

แก้ยังง: แก้ในไดอารี่แล้ว และเพิ่มเป็นกฎ: ห้ามบอกว่าผลลัพธ์พร้อมใช้ ถ้ามันยังทำหน้าที่หลักของมันไม่ได้

กฎที่ได้: รายงานความพร้อมโดยวัดจากหน้าที่หลักของสิ่งที่สร้าง และต้องพิสูจน์ด้วยการรันจริงตั้งแต่ต้นจนจบ ไม่ใช่วัดจากการที่ขั้นตอน build/convert จบด้วย exit code 0

สูง 2026-07-21

ผู้ช่วย AI เตือนเรื่องไม่มีการสำรองข้อมูลด้วยน้ำเสียงอ่อนเกินไป และประกาศว่าโมเดลพร้อมใช้ทั้งที่ยังอ่านภาพไม่ได้

เกิดอะไรขึ้น: ความเสียหายของวันที่ 07-21 ไม่ได้มาจากบั๊กโค้ด แต่มาจากความล้มเหลวในการสื่อสารสองอย่าง: คำเตือนเรื่องไม่มีการ backup ถูกพูดแบบเบาๆ แทนที่จะเป็นสัญญาณเตือนหยดงาน และ artifact ถูกบอกว่า 'พร้อมใช้' ทั้งที่รู้อยู่แล้วว่าส่วน vision ยังทำงานไม่ได้

ต้นเหตุจริง: ระดับความรุนแรงของคำเตือนไม่ได้ปรับให้เหมาะสมกับความสามารถย้อนกลับของการกระทำนั้น และคำว่า 'พร้อม' ถูกรายงานตามขั้นตอนที่สำเร็จ ไม่ใช่ตามเป้าหมายจริงของโปรเจกต์ (อ่านแบบก่อสร้างได้)

เสียหายอะไร: เป็นสาเหตุโดยตรงที่ทำให้งานทั้งวันหายไปแบบกู้คืนไม่ได้ และทำให้เข้าใจผิดเรื่องความสามารถของ GGUF

แก้ยังง: ตั้ง Phase 0 pre-rental checks และกฎว่า checklist ที่เขียนไว้ต้องถูกรันจริง ไม่ใช่แค่เขียนไว้เฉยๆ

กฎที่ได้: ปรับระดับความหนักแน่นของคำเตือนให้ตรงกับความสามารถย้อนกลับของการกระทำ — อะไรที่ทำลายข้อมูลได้ ต้องมีการยืนยันแบบบล็อกไว้ก่อนเสมอ และอย่ารายงานว่างานพร้อมใช้จากขั้นตอนที่สำเร็จ ให้รายงานตามความสามารถที่ผู้ใช้ต้องการจริงๆ

สูง 2026-07-28

ไฟล์ workflow ของการ tune คนละรอบใช้ชื่อซ้ำกัน ทำให้เข้าใจผิดว่ารอบนี้เสร็จ ทั้งที่เป็น ☒ ของรอบก่อน

เกิดอะไรขึ้น: เอกสาร workflow ของทั้งสองรอบใช้ชื่อแบบ RETUNE_WORKFLOW.md เหมือนกัน การเห็น checkbox ที่ติ๊กแล้วจากรอบก่อน จะทำให้เข้าใจผิดว่ารอบปัจจุบันเสร็จแล้ว — เป็นกลไกเดียวกับที่เคยทำให้เกิดเหตุการณ์ DAY OF SHAME มาแล้ว

ต้นเหตุจริง: ตัวบ่งบอกว่าไฟล์เป็นของรอบไหน อยู่ที่ path ของโฟลเดอร์เท่านั้น ไม่ได้อยู่ในชื่อไฟล์หรือหัวเอกสาร พอเปิดไฟล์แบบไม่มีบริบท ก็ไม่มีทางรู้ว่า checkmark นั้นเป็นของรอบไหน

เสียหายอะไร: เสี่ยงต่อการสรุปผิดว่า 'เสร็จแล้ว' ทั้งที่ยังไม่เสร็จ — เป็นความล้มเหลวแบบเดียวกับที่เคยทำให้ทีมเสียรอบงานไปแล้วครั้งหนึ่ง

แก้ยังง: ใช้ git mv เปลี่ยนชื่อเป็น t01_workflow.md / t02_workflow.md โดยรักษาประวัติไฟล์ไว้ครบ ทั้งสองไฟล์เขียนหัวเอกสารระบุรอบ สถานะ และคำเตือนว่า ☒ เป็นของรอบไหนชัดเจน พร้อมตั้งเป็นกฎในการตั้งชื่อไฟล์ต่อไป

กฎที่ได้: ตั้งชื่อไฟล์ที่แยกตามรอบให้มีเลขรอบอยู่ในชื่อไฟล์ และย้ายเลขรอบเข้าไปในหัวเอกสารด้วย เพราะสัญลักษณ์สถานะต้องบอกขอบเขตของตัวเองในตัวเอง เนื่องจากมันจะถูกอ่านแบบขาดบริบทแน่นอน

สูง 2026-08-21

แกนแนวดิ่งไม่มีฟิลต์รองรับใน schema เลย์สักช่อง ทั้งที่ระดับความสูงถูกพิมพ์อยู่บนทุกรูปด้านและรูปตัด

เกิดอะไรขึ้น: ระดับความสูงอย่าง +3.75 ระดับหลังคา, +0.60 ระดับพื้นชั้น 1 และ ± 0.00 ระดับอ้างอิง ถูกพิมพ์อยู่บนทุกรูปด้านและรูปตัดในชุดข้อมูล แต่ schema ไม่มีฟิลต์รองรับมันเลย — `x_lines/y_lines` เป็นแกนสำหรับแปลนเท่านั้น พบเรื่องนี้ตอนจะขามสั่งว่า 'ให้ grid master บันทึกทุกค่าที่เจอในทุกหน้า'

ต้นเหตุจริง: โมเดล grid ของ schema ถูกสร้างขึ้นจากมุมมองแบบแปลนล้วนๆ (สองแกนแนวนอน) และไม่เคยมีใครตรวจสอบมันเทียบกับแบบรูปตัดหรือรูปด้านเลย ซ้ำเติมด้วยการสันนิษฐานก่อนหน้านี้ว่ารูปด้านมีคุณค่าต่ำ

เสียหายอะไร: ทุกบ้านที่สกัดข้อมูลมาจนถึงตอนนี้บันทึกระดับแนวดิ่งเป็นศูนย์ทั้งหมด ความสูงพื้นและระดับห้องคาบที่จำเป็นสำหรับการคำนวณปริมาตรหรือสร้างโมเดล 3D ไม่เคยถูกเก็บเลยตลอดทั้ง dataset

แก้ยังง: เพิ่ม `z_levels[]` ไว้ใต้ `grid[]` ใน `primary_rawjson_schema.md` §4 ขยาย `dimension_chains[]` ให้รองรับแกน z เพิ่ม `unassigned_dimensions[]` เป็นตัวรับทั่วไป และเพิ่มรายการตรวจสอบ §0 อีก 2 ข้อให้ `check_format.py` บังคับใช้

กฎที่ได้: ตรวจสอบ schema เทียบกับแบบแต่ละประเภทอย่างน้อยหนึ่งแผ่นก่อนล็อกมัน — แกนที่หายไปทั้งแกนแบบนี้คือสิ่งที่รอตมาได้ถึง 22 บ้านเพราะไม่มีใครถามมันเลย

สูง 2026-08-25

PDF สอนงานส่งออกไปพร้อมข้ออ้างผิดว่าการเพิ่ม MAX_LENGTH ต้องเช่าการ์ดใหญ่ขึ้น

เกิดอะไรขึ้น: เอกสารสอนงานที่เขียนตอน 15:00 เขียนใน §2.3 ว่า 'MAX_LENGTH เพิ่มได้ แต่ต้องเช่าการ์ดใหญ่ขึ้น = จ่ายแพงขึ้น' แต่การแตก VRAM ที่ทำอีกชั่วโมงถัดมาแสดงว่า 79.7 จาก 87.5 GB ไม่โตตามความยาว sequence เลย ดังนั้นเพิ่มความละเอียด 32% ก็ยังพอดีกับการ์ดตัวเดิม เหลือ 4.0 GB และไม่มีค่าใช้จ่ายเพิ่ม เอกสารเก่าไปตั้งแต่ในเซสชันเดียวกัน

ต้นเหตุจริง: เขียนข้อสรุปลงเอกสารถาวรก่อนที่การวัดซึ่งจะเป็นตัวชี้ขาดจะเกิดขึ้น และเอกสารนั้นก็ไม่มีกลไกอะไรโยกกลับไปหาตัวเลขที่มันพึ่งพาอยู่

เสียหายอะไร: มีข้ออ้างเรื่องต้นทุนที่ผิดข้อเท็จจริงอยู่ในเอกสารที่มะขามอาจเอาไปให้อาจารย์ดู แถมมันจะเป็นการเชียร์ให้จ่ายเงินที่ไม่จำเป็นด้วย

แก้ยังง: แก้ §2.3 หันที่ไม่ผิดวัน และเพิ่ม §2.3.1/§2.3.2 พร้อมผลวัดการย่อขนาด 100% จริงและการแตก VRAM สร้าง PDF ใหม่จาก 12 เป็น 19 หน้า ตรวจโดยดึงข้อความกลับออกมาแล้วเช็ก marker หกจุด

กฎที่ได้: อย่าเขียนข้อสรุปลงเอกสารที่จะเอาไปให้คนอื่นดู ก่อนที่การวัดซึ่งจะชี้ขาดข้อสรุปนั้นจะเสร็จ ถ้าจำเป็นต้องเผยแพร่ก่อนข้อมูลจะมา ให้ติดป้ายลงวันที่ว่า 'ยังไม่ได้วัด' ไว้บนข้ออ้างนั้น เพื่อไม่ให้ใครอ่านเป็นข้อเท็จจริง

สูง 2026-08-25

ค่าเหล็กเสริมชั้นต่ำ 5 ค่าใน BBS เกือบถูกอ้างอิงเป็น มยพ. 1103-64 ทั้งที่มาตรฐานตัวนี้ไม่มีค่าออกแบบชั้นต่ำอยู่แล้ว

เกิดอะไรขึ้น: Ton สั่งให้เกดที่สามของระบบสำรอง BBS (assumeRebarFromRules) อ้างอิง มยพ. 1103 เพื่อความแม่นยำ พอไปอ่านเอกสารจริงทั้ง 20 หน้ากลับพบว่า มยพ. 1103-64 เป็นมาตรฐานด้านวัสดุและฝีมืองาน (มอก. / การตัด / การตัด / การงอขอ / การทาบ / การทดสอบ) ไม่มีค่าเหล็กเสริมชั้นต่ำเชิงออกแบบทั้ง 5 ค่าอยู่เลยสักตัว

ต้นเหตุจริง: สันนิษฐานเอาเองว่ามาตรฐานนี้ครอบคลุมค่าออกแบบชั้นต่ำเพราะหัวข้อเป็นเรื่องเหล็กเสริม โดยไม่มีใครอ่านขอบเขตของมันก่อนเลย มาตรฐานวัสดุเหล็กเสริมกับมาตรฐานออกแบบเหล็กเสริมเป็นคนละเอกสารกัน

เสียหายอะไร: ถ้าไม่จับได้ จะส่งค่าทั้ง 5 ตัวออกไปพร้อมการอ้างอิงมาตรฐานไทยที่ไม่มีค่าเหล่านั้นอยู่จริง ซึ่งเป็นความผิดพลาดด้านการอ้างอิงในงานวิศวกรรม

แก้ยังไ้: คงค่า ACI ไว้ตามเดิม แต่ติดธง `dpt1103: 'not\_found'` ทุกแถว พร้อมเพิ่มป้ายภาษาไทยต่อท้าย `label\_th` ทุกตัวเพื่อบอกให้ชัดในหน้าจว่าไม่มีการอ้างอิงมาตรฐานนี้จริง

กฎที่ได้: อ่านขอบเขตของมาตรฐานก่อนจะอ้างอิงเสมอ มาตรฐานที่ครอบคลุมวัสดุเดียวกันอาจครอบคลุมคนละแง่มุมกันโดยสิ้นเชิง และการอ้างอิงผิดยิ่งเลวร้ายกว่าการอ้างอิงมาตรฐานต่างชาติแบบตรงไปตรงมาเสียอีก

เรื่องที่เหลือในหมวดนี้ (24)

Label Studio ขึ้น 'undefined' แล้วไล่หาบั๊กในโค้ด ทั้งที่ต้นเหตุจริงคือ task ที่ import เข้าไปเป็นข้อมูลเก่า

กลาง 2026-07-06

ระบบไหนที่ข้อมูลถูก import เข้ามาเป็น snapshot (task ของ Label Studio, fixture ที่ seed ไว้, cache) ให้เช็คก่อนว่าฝั่งที่ใช้ข้อมูลถืออันเก่าอยู่หรือเปล่า ก่อนจะไปอ่านโค้ดฝั่งที่สร้างมันแม้แต่บรรทัดเดียว ตรวจไฟล์จริงบนดิสก์ก่อน คำสั่งเดียวจบและตัดปัญหาทั้งกลุ่มนี้ออกได้เลย

คู่มือใน onstart.sh บอกขั้นตอนผิด แกมอ้างถึงไฟล์ที่ไม่มีอยู่จริง

กลาง 2026-07-28

ตรวจคู่มือปฏิบัติงานทุกฉบับกับโค้ดปัจจุบันก่อนเริ่มเข้าเครื่อง ขั้นตอนไหนที่อ้างชื่อไฟล์ ต้องไปเช็คว่ามีไฟล์นั้นจริง ไม่งั้นจะไปรู้ว่าผิดตอนที่มีเตอร์เดินอยู่แล้ว

ในไดอารี่ หัวข้อ 5 บอกว่า export_gguf.py ปรับให้ t02 แล้ว แต่ในรายการค้างยังเขียนว่ายังไม่ได้ปรับ

กลาง 2026-07-28

ก่อนปิดบันทึกประจำวัน ให้เอารายการงานค้างมาเทียบกับงานที่ทำเสร็จในรอบนั้นก่อน รายการค้างที่ขัดกับเนื้อหาข้างบน แยกว่าไม่มีรายการเลย เพราะคนจะเชื่อมั่น

เส้นบอกระยะที่ไม่มีอะไรวางอยู่บนนั้น (ระยะจากขอบถึงเส้นกริดแรก) ถูกกฎ dummy grid ทิ้งไปเสียบบ

กลาง 2026-08-21

ฟิลต์ใน schema ต้องนิยามตามสิ่งที่แบบพิมพ์มาจริง ไม่ใช่ตามที่ผู้ใช้ข้อมูลตอนนี้ต้องการ และค่าอะไรก็ตามที่ตัวสกดตัดสนใจไม่บันทึก ต้องไปลง array เก็บของเหลือพร้อมค่าเตือน ห้ามหายไปเสียบบ

material_list → boqSeed[] ไม่มีทางเขียนลงที่เก็บ BOQ จริงของ Constistant เลย

กลาง 2026-08-21

อย่าส่งฝั่งผู้ผลิตข้อมูลออกไปโดยไม่ฝั่งผู้ใช้ ถ้าจะเลื่อนการต่อท่อออกไปก่อน ให้บันทึกไว้ตรงที่ตัวผู้ผลิตอยู่ เพื่อไม่ให้คนถัดไปเข้าใจผิดว่า 'มีผลลัพธ์ออกมาแล้ว' เทากับ 'ผลลัพธ์ถูกเอาไปใช้แล้ว'

site_plan.element_type ไม่มีชุดค่ามาตรฐาน เจอ 10 ค่าที่ต่างกันในบ้าน 5 หลัง

กลาง 2026-08-21

ปิดชุดค่าของฟิลด์หมวดหมู่ทุกตัวก่อนที่ค่าต่างๆ จะแตะเลขสองหลัก 10 ค่าจากบ้าน 5 หลังคือฟิลด์ที่ไม่เคยมีใครดูแลเลย และทุกหลังที่เพิ่มเข้ามาจะทำให้การย้ายข้อมูลในอนาคตแพงขึ้นอีก

`t04\_workflow.md` เก่าค้างหนัก — หัวไฟล์ยังเขียนว่า 'ไม่เคยเช่า GPU' ทั้งที่ 2 จาก 4 fold เทรนใกล้เสร็จแล้ว

กลาง 2026-08-30

อัปเดตเอกสารทำงานทันทีที่การตัดสินใจเปลี่ยน 'ไม่ใช่รอตอนจบ session' แผนเก่าที่อ่านแล้วเหมือนยังเป็นปัจจุบัน อันตรายกว่าไม่มีแผนเลย

copy คำสั่งจาก PDF มา สระไทยเพี้ยน ทำให้ cd ใช้งานไม่ได้

กลาง 2026-08-31

ห้ามใส่ตัวอักษรไทยไว้ในบรรทัดคำสั่งที่ตั้งใจให้คน copy ไปใช้ และต้องตรวจสอบด้วยโปรแกรมว่าบล็อกคำสั่งในคู่มือเป็น ASCII ล้วนจริงๆ

คอมเมนต์อธิบายที่ติดมาพร้อมคำสั่งตอน copy ทำให้ argparse พัง

กลาง 2026-08-31

ให้แยกคำอธิบายออกจากบรรทัดคำสั่งไปเลย และในจุดที่ผู้ปฏิบัติงานต้องทำหลายขั้นตอน ให้เปลี่ยนจากการพิมพ์คำสั่งเองเป็นเมนูตัวเลขที่ตัดปัญหาการ copy-paste ผิดทั้งประเภทออกไป

ยังไม่เคยสอนมะขามให้ใช้ vast.ai workflow เอง — เป็นงานที่ต้องส่งมอบแต่ถูกเลื่อนซ้ำแล้วซ้ำเล่า

กลาง 2026-09-02

งานส่งมอบที่เลื่อนไปครั้งหนึ่ง จะถูกเลื่อนไปทุกครั้ง ต้องจัดให้การสอนแบบ walk-through เป็นงานแรกของ session ไม่ใช่งานสุดท้าย ไม่งั้นจะไม่มีวันได้ทำ

อ่านข้อความตอบสั้นๆ ที่ถามว่าเป็นคำตอบใช่/ไม่ใช่ ทั้งที่ยังไม่ชัดเจน

กลาง

อย่าตีความข้อความตอบสั้น/ถามว่าเป็นคำตอบใช่/ไม่ใช่ต่อคำถามที่ค้างอยู่ — ถามย้ำให้ชัดเจนและรอคำตอบที่แน่นอนก่อนลงมือทำต่อ

สมมติเอาเองว่า Unsloth เป็นบริการที่ต้องล็อกอิน/มีบัญชี

ต่ำ 2026-07-21

ทุก dependency ต้องระบุให้ชัดว่าเป็น library ที่รันในเครื่องหรือเป็นบริการที่ต้องมี credential — ข้อกำหนดเรื่อง credential ควรเขียนไว้เฉพาะกับ component ที่มีจริงๆ เท่านั้น

เอกสารสอนงานเขียนว่า LoRA adapter มีขนาด 'ไม่กี่ร้อย MB' แต่ไฟล์จริงมีขนาด 7.5GB

ต่ำ 2026-07-21

เอกสารสอนงาน/สรุปงานต้องอ้างตัวเลขจริงที่วัดได้จากรันจริงเท่านั้น ต้องเช็คทุกตัวเลขกับ log ก่อนเผยแพร่ โดยเฉพาะขนาดไฟล์ที่ใช้ตัดสินใจเรื่องสำรองข้อมูล

เอกสารสอนงานเขียนว่า LoRA adapter มีขนาด 'ไม่กี่ร้อย MB' แต่ adapter ของ t01 จริงมีขนาด 7.5GB

ต่ำ 2026-07-21

สำหรับโมเดลแบบ MoE ทุกการประมาณขนาด adapter ต่อตัวต้องคูณด้วยจำนวน expert ด้วย ต้องแก้ความเข้าใจในเอกสารให้ถูก อย่าปล่อยให้ตัวเลขจากโมเดลแบบ dense ค้างอยู่เคียงกับตัวเลขจริง

rule_of_tune.md ข้อ 4 ชี้ไปที่ path ไฟล์ที่ถูกย้ายไปแล้ว

ต่ำ 2026-07-28

อ้างอิงเอกสารตามบทบาท ('workflow ของรอบนั้นๆ เอง') ไม่ใช่ตาม path ที่ตายตัว จะได้ไม่พังเจี๊ยบๆ เวลามีการเปลี่ยนชื่อ และต้องเช็ค cross-reference ทุกจุดในคอมมิตเดียวกับที่เปลี่ยนชื่อไฟล์นั้น

จำนวน token ที่ถูกต้อง 5,100 ตัว จะถูกอ่านว่าเป็นความล้มเหลว เพราะเอกสารระบุค่าที่คาดหวังไว้ที่ 5,120

ต้า 2026-07-28

ต้องเขียนค่าที่คาดหวังในเอกสารตามตัวเลขที่การคำนวณจริงให้ผลออกมา รวมถึงผลข้างเคียงจากการปิดเศษด้วย พร้อมอธิบายเหตุผล — ค่าที่คาดหวังแบบในอุดมคติจะสร้าง false alarm ขึ้นมาเอง

grid master เจอเคสการตั้งชื่อ dummy-grid ที่ spec ไม่มีกฎรองรับไว้ ต้องตัดสินใจเฉพาะหน้า

ต้า 2026-08-18

เมื่อแบบก่อสร้างจริงบังคับให้ต้องตัดสินใจเรื่องการตั้งชื่อที่ spec ยังไม่ครอบคลุม ให้บันทึกไว้ในไฟล์นั้น **และ** เสนอเป็นข้อแก้ไข spec ในวันเดียวกันเลย — ถ้าเป็นบรรทัดฐานที่ไม่มีการบันทึกไว้ที่อื่นนอกจาก metadata ของไฟล์เดียว บ้านหลังถัดไปจะขัดแย้งกับมันโดยไม่มีใครรู้ตัว

บ้าน 01 หน้า 19 ถูกบันทึกเป็นแปลนฐานราก/เสา ในวันที่ 08-18 แต่กลายเป็นแปลนคานาในวันที่ 08-20 — หน้าเดียวกัน สองประเภทแผ่น

ต้า 2026-08-20

เวลาบันทึกผลที่วัดได้ ให้คัดลอกค่า `pattern`/`sheet\_code` ที่บันทึกไว้จริงมาใส่ในไดอารี่จริงๆ อย่าเขียนบรรยายเอาเอง เพราะการพิสูจน์ผลที่มีหัวข้อคำถามจะเอาไปใช้ซ้ำไม่ได้

subtask roof_plan ไม่มีแถว input ในตาราง pass-design ทั้งที่จริงๆ เป็นแหล่งข้อมูล grid-master

ต้า 2026-08-21

ตารางความสัมพันธ์ที่สร้างด้วยมือต้องมีตัวเช็คความสอดคล้องเชิงกลไก — ถ้า subtask หนึ่งในคลาสนี้ประกาศ input ตัวอื่นในคลาสเดียวกันก็ต้องมีด้วย ให้ assert เช็คไว้เลย ไม่ใช่ปล่อยให้อ่านทีละแถว

นิยาม Pass 2 ว่าเป็น '7 pattern ที่ Constant ใช้' แต่พอไล่ดู adapter จริงพบว่าอ่านแค่ 6 — soil_boring_log ไม่ได้อยู่ในนั้นด้วยซ้ำ

ต้า 2026-08-21

เวลาจะกำหนดขอบเขตด้วยเหตุผล 'โค้ดใช้ตัวนี้จริง' ให้ระบุโมดูลที่ใช้จริงต่อ pattern ไปเลย — การไล่ตามแคโมดูลเดียวไม่ใช่ทั้งแพลตฟอร์ม และการมีรายการเกินมาหนึ่งตัวที่อ้างว่ามีเหตุผลรองรับ คือจุดที่ scope มันบวมขึ้นเสียๆ

ผู้ปฏิบัติงานถามถึงไฟล์ 'apply_arm.py' ซึ่งไม่มีอยู่จริง — ตัวสลับแขน (arm switcher) จริงๆ เป็นแค่ฟังก์ชันที่ฝังอยู่ในตัวรัน inference

ต้า 2026-08-29

ถ้าปุ่มควบคุมตัวไหนกำหนดพฤติกรรมของการรันที่มีค่าใช้จ่ายสูง ตำแหน่งของมันต้องเดาได้จากชื่อ การชี้ผ่านลิงก์ใน README นั้นโอเคสำหรับขั้นตอนทั่วไป แต่ไม่โอเคสำหรับฟังก์ชันเดียวที่ผู้ปฏิบัติงานต้องเชื่อใจตอนตีสอง

บันทึกในไดอารี่เขียนเป็นภาษาไทย ทั้งที่กติกาของไดอารี่เองบังคับให้เขียนภาษาอังกฤษ

ต้า 2026-09-01

ธรรมเนียมที่เขียนไว้แต่ไม่มีจุดบังคับใช้จะค่อยๆ เพี้ยนไปเอง — ถ้าจะให้กติกาคูมได้จริง ต้องใส่กฎเรื่องภาษาไว้ในหัวเทมเพลตของบันทึกให้คนเขียนเห็นตรงๆ หรือไม่ก็ยอมรับภาษาที่ใช้จริงแล้วแก้ README ให้ตรงกัน อย่าปล่อยให้กฎกับพฤติกรรมจริงขัดกันเอง

โฟลเดอร์แปลภาษาอังกฤษคู่ขนานล้าสมัยจนต้องลบทิ้ง

ต้า 2026-09-01

อย่าดูแลเอกสารที่มีชีวิตแบบมีฉบับแปลคู่ขนาน เลือกภาษาเดียวแล้วเขียนแบบนั้นตั้งแต่ตัวอักษรแรก — 'แปลที่หลัง' ไม่เคยเกิดขึ้นจริง

กฎของโปรเจกต์เสี่ยงถูกก๊อปปี้เข้า memory ส่วนตัวของ AI แทนที่จะอยู่ที่เดียว

ต้า

อย่าก๊อปปี้กฎของโปรเจกต์เข้า memory ส่วนตัวของ AI — ถ้าบทเรียนควรอยู่ในไฟล์แหล่งความจริงเดียว ให้เก็บไว้ที่นี้ทีเดียว ไม่ใช่มีสำเนาที่สองด้วย

กระบวนการทำงาน การวางแผน และ งานที่เสียเปล่า

ลำดับการทำงาน ขอบเขต การทำซ้ำ และแรงที่หุ้มไปผิดที่

ทั้งหมด 231 เรื่อง · วิฤต 21 · สูง 74 · กลาง 89 · ต่ำ 47

เรื่องที่ต้องอ่านให้ครบ (95)

วิฤต 2026-07-06

บัก Label Studio ตัวเดิมซ้ำอีกรอบ: ยัดตัวแปร \$ หลายตัวลงใน Text/Header ค่าเดียว ทำให้ import ล้ม 10 จาก 10 งาน

เกิดอะไรขึ้น: ตอน import ไฟล์ label-studio-tasks-makham-elements-structural.json เข้า Label Studio Cloud ล้มทั้ง 10 งาน ด้วยข้อความ `"sheet_code $sheet_name — $pattern: $view_title" key is expected in task data` สาเหตุคือเอาตัวแปร \$field ตั้งแต่ 2 ตัวขึ้นไปยัดใน `value=` ของ Text/Header อันเดียว บักตัวนี้เคยเจอมาแล้วเมื่อ 2026-07-02 และจดไว้ใน No_touch_box/CLAUDE.md แล้วด้วย แต่ตอนเขียน XML config ของ Gen 3 ใหม่กลับไม่ได้เอามาใช้ พอ grep ดูเจอจุดที่ผิด 7 จุด กระจ่ายใน 5 ไฟล์ XML

ต้นเหตุจริง: บทเรียนที่เขียนไว้แล้วไม่ถูกหยิบมาดูตอนกำลังเขียน config ใหม่ เพราะมันอยู่ลึกในหัวข้อย่อยของ Label Studio ใน CLAUDE.md ไม่ได้อยู่ในรายการ 'Key Lessons' ระดับบนสุด เลยไม่มีอะไรบังคับให้เช็คตอนเขียน config แบบเดียวกันขึ้นใหม่ อีกอย่างคืออาการของบักเปลี่ยนรูปไประหว่างสองครั้ง (ครั้งแรก 2 ก.ค. เป็น 'undefined' ตอนรันใน Repeater ส่วน 6 ก.ค. เป็น error ตอน validate ตอน import ที่ระดับบนสุดของ task) เลยไม่มีใครจำได้ว่าเป็นบักตัวเดียวกันจนกว่าจะไล่หาเจอ

เสียหายอะไร: import ล้ม 10 จาก 10 งาน ต้องไล่หาและเขียนใหม่ 7 จุดใน 5 ไฟล์ XML (แยกทุกค่าที่มีหลายตัวแปรออกเป็น Text หนึ่งแท็กต่อหนึ่งตัวแปร) แล้วต้องลบ task ที่ import พังไปแล้วทั้ง วาง config ใหม่แล้ว import ใหม่ตามลำดับที่ถูกต้อง งานตั้งค่า Label Studio ต้องทำใหม่ทั้งรอบ

แก้ยังไง: แยกทุกค่าที่มีหลายตัวแปรเป็น `<Text>` หนึ่งแท็กต่อหนึ่งตัวแปร ครบทั้ง 5 ไฟล์ XML ตรวจซ้ำด้วย grep แล้วยกกฎนี้ขึ้นเป็นข้อ 8 ของหัวข้อ 'Key Lessons (อย่าทำพลาดซ้ำ)' ระดับบนสุดใน No_touch_box/CLAUDE.md พร้อมใส่คำเตือนไว้ใน SESSION_HANDOFF_2026-07-06.md

กฎที่ได้: ห้ามใส่ตัวแปร \$field เกินหนึ่งตัวใน value ของ Text/Header ใน Label Studio เด็ดขาด — หนึ่งตัวแปรต่อหนึ่งแท็กเสมอ และในเชิงโครงสร้าง: บทเรียนที่เก็บไว้ได้หัวข้อย่อยของเครื่องมือ จะไม่มีใครอ่านตอนเขียน config ใหม่ ดังนั้นบักที่เกิดซ้ำต้องถูกยกขึ้นไปอยู่ในรายการ Key Lessons ระดับบนสุดทันทีที่มันซ้ำ ไม่ใช่ปล่อยไว้ตรงที่เขียนครั้งแรก

วิกฤต 2026-07-07

BOQ รอบ t1 บันทึกระเบียงปูพื้น 137/294 ตร.ม. ทั้งที่บ้านมีแค่ 70 ตร.ม.

เกิดอะไรขึ้น: Agent เบื้องหลังที่กลับไปอ่านหน้า 51 ใหม่จับได้ว่าการสักรอบ t1 กรอกปริมาณกระเบื้องปูพื้นไว้ 137 กับ 294 ตร.ม. ซึ่งเป็นไปไม่ได้ทางกายภาพสำหรับบ้านที่มีพื้นที่รวม 70 ตร.ม. (ระบุไว้ในหน้า 60) ค่าที่ถูกต้องของหมวด 2.3 อยู่ในช่วง 21-47

ต้นเหตุจริง: ตัวเลข BOQ ถูกคัดลอกที่ละช่องโดยไม่มีการตรวจสอบความสมเหตุสมผลเทียบกับหน้าอื่นในเอกสารเดียวกันเลย ไม่มีอะไรในกระบวนการที่เอาปริมาณไปเทียบกับพื้นที่ของตัวอาคารเอง

เสียหายอะไร: ปริมาณที่ผิดราว 4-6 เท่าหลุดไปอยู่ในชุดข้อมูล t1 และรอดมาได้จนกว่าจะมีการตรวจสอบหลังบังเอิญมาเจอ

แก้ยังไ: Agent แก่ปริมาณให้เป็นค่าที่ถูกต้อง พร้อมเขียนอธิบายว่าทำไมค่าเดิมถึงเป็นไปไม่ได้

กฎที่ได้: ทุกปริมาณที่สกัดออกมาต้องผ่านการตรวจสอบความสมเหตุสมผลข้ามหน้าอย่างน้อยหนึ่งครั้ง (เทียบพื้นที่ จำนวน หรือยอดรวม) ก่อนจะรับเข้ามา ตัวเลขที่เป็นไปไม่ได้ทางกายภาพต้องตกทันที ไม่ใช่คัดลอกตามต้นฉบับอย่างซื่อสัตย์

วิกฤต 2026-07-14

การแก้ทั้งหมดจากการรีวียูแค่ในโฟลเดอร์สำเนาที่แก้แล้ว ส่วนต้นฉบับที่เอาไป tune ไม่เคยถูกซิงค์เลย

เกิดอะไรขึ้น: ทุกจุดที่แก้ให้บ้าน 1 (กริดสมมติ D/1", การอ้างอิงจุดแบบไม่มีขีด, การเรียงลำดับ element ใหม่, การแยกสเปกออกมา) มีอยู่แค่ใน `json\_แก้ไขแล้ว/` ส่วนต้นฉบับ `rawjson\_ยังไม่ได้แก้ไขโดนคน/01.../` ซึ่งเป็นชุดที่เอาไป tune จริง ยังเป็นของเก่าทั้งหมด การซิงค์กลับไม่เคยเริ่มเลยข้ามทุกรายการต่อจากนั้น

ต้นเหตุจริง: สร้างโฟลเดอร์สำเนาที่แก้แล้วขึ้นมาเพื่อกันไม่ให้ต้นฉบับเสีย แต่ไม่ได้กำหนดขั้นตอนหรือเงื่อนไขการซิงค์กลับไว้พร้อมกัน สองฝั่งเลยห่างกันไปเรื่อยๆ ไม่มีที่สิ้นสุด

เสียหายอะไร: ชุดข้อมูลที่ใช้ train จริงยังมีค่าที่รู้อยู่แล้วว่าผิด (กริดสมมติผิด จำนวนเหล็กผิด การอ้างอิงมีขีด) ในขณะที่ข้อมูลที่แก้ถูกเลื่อนอนอยู่เฉยๆ ไม่ได้ใช้ ทุกวันที่รีวิวเพิ่ม ระยะห่างก็ยิ่งกว้างและความเสี่ยงตอนซิงค์ก็ยิ่งสูง

แก้ยังไ: ยังไม่ได้แก้ — 'การซิงค์กลับยังไม่เริ่มเลย' ค้างอยู่ในรายงานงานค้างมาจนถึง 2026-07-25

กฎที่ได้: เวลาแยกข้อมูลออกมาเป็นสำเนาเพื่อปกป้องต้นฉบับ ให้กำหนดเงื่อนไขการรวมกลับไว้ในการเปลี่ยนแปลงครั้งเดียวกันเลย สำเนาที่แยกออกมาโดยไม่มีแผนซิงค์ ก็คือแหล่งความจริงที่สองดีๆ นี่เอง

วิกฤต 2026-07-19

ป้ายเหล็กแกนหลัก '3-Ø16' ของคาน B5 หายไปจากข้อมูลทั้งต้น ในสองบ้านแยกกัน

เกิดอะไรขึ้น: ในบ้าน 4 ข้อมูลของ B5 ทั้งป้าย '3-Ø16' ไปเฉยๆ เหลือแค่ 2 เส้น ทั้งที่ควรเป็น 3+2=5 แล้วก็เจอความผิดพลาดแบบเดียวกันเป๊ะที่ B5 ของบ้าน 5 (ล่าง 3→5, เหล็ก L/8 หายไปทั้งชุด)

ต้นเหตุจริง: เมื่อคานหนึ่งมีทั้งป้ายหลักและป้ายเสริม L/8 คนอ่านอ่านเจอแค่ตัวหนึ่ง อีกตัวไม่เคยถูกบันทึกเลย และไม่มีการตรวจสอบไขว้เอาจำนวนเหล็กที่บันทึกไว้ไปเทียบกับภาพรูปตัด

เสียหายอะไร: นับเหล็กล่างของคานหลักขาดไป 60% ในสองบ้าน เป็นการขาดเหล็กเชิงโครงสร้างในข้อมูลโดยตรง

แก้ยังไ: แก้เป็น 5 ทั้งสองที่ หลังจากซูมเข้าไปในรูปตัดแล้วนับจุด และยังพบว่า 'additional' ของ B3X ในบ้าน 5 เป็นข้อมูลผี จึงลบทิ้ง

กฎที่ได้: คานที่มีป้ายเหล็กมากกว่าหนึ่งป้าย ต้องบันทึกให้ครบทุกป้ายอย่างชัดเจน — นับจุดในรูปตัดแล้วกระหนยอดกับผลรวมของทุกป้ายที่พิมพ์ไว้ก่อนบันทึก และต้องตรวจสอบข้อมูลผีด้วย: รายการใน additional_bars ที่ไม่มีจุดเหล็กในรูปตัดรองรับ คือข้อมูลที่ถูกรูขี้มนมา

train_qwen3vl.py มีบั๊กติดมา 5 ตัว เพราะเขียนไว้ตอนชุดข้อมูลยังมี 221 ตัวอย่าง และไม่เคยถูกรันเลยสักครั้ง

เกิดอะไรขึ้น: การอ่าน train_qwen3vl.py ก่อนบินเจอบั๊กคนละแบบ 5 ตัว ไฟล์นี้เขียนไว้ตั้งแต่ตอนที่ชุดข้อมูลยังมี 221 ตัวอย่าง (ตอนนี้ 403 แถว) และไม่เคยถูกรันเลยสักครั้งเดียว

ต้นเหตุจริง: เขียนโค้ดล่วงหน้าเพื่อการรันในอนาคต แล้วปล่อยให้มันเน่าไปในขณะที่ทั้งชุดข้อมูลและไลบรารีรอบตัวเปลี่ยนไปเรื่อยๆ ไม่เคยมีอะไรรันมันเลย ก็เลยไม่เคยมีอะไรทำให้ความล้มเหลวโผล่ออกมา

เสียหายอะไร: บั๊กห้าตัวจะเข้าไปอยู่ในการเทรนที่เสียเงินพร้อมกันหมด — รวมถึงสองตัวที่ทำข้อมูลเทรนเสียเจียบๆ แทนที่จะพัง

แก้ยังไง: แก้ทั้งห้าตัวก่อนเข้าเครื่อง และภายหลังยังพบอีกว่าไฟล์นี้ยังฟังตอนรันจริงด้วยบั๊กตัวที่หก ซึ่งเป็นเรื่องเวอร์ชันของไลบรารี (ดูรายการเรื่อง transformers 5.5.0)

กฎที่ได้: โค้ดที่ไม่เคยรัน คือโค้ดที่ยังไม่ได้ตรวจสอบ และมันเสื่อมสภาพลงเรื่อยๆ ตามที่ไลบรารีและข้อมูลเปลี่ยนไป ให้รันทุกสคริปต์อย่างน้อยหนึ่งครั้ง — แม้จะรันกับข้อมูลตัวอย่างเล็กๆ ก็ตาม — ก่อนจะไว้ใจให้มันไปรันงานที่เสียเงิน

คำแนะนำที่ยึดถือกันมาว่า 'อัปโหลด adapter ก่อนเสมอ' นั้นผิด — โค้ดจริงอัปโหลด GGUF ก่อน แล้วค่อย adapter ทีหลัง

เกิดอะไรขึ้น: แนวปฏิบัติเรื่องความปลอดภัยของการสำรองข้อมูลที่ใช้กันอยู่บอกว่า adapter จะถูกอัปโหลดขึ้นก่อน แต่พออ่านโค้ด export_gguf.py จริงๆ กลับตรงกันข้าม: GGUF ขึ้นก่อน adapter ขึ้นทีหลังสุด และเพราะการตรวจสอบความถูกต้อง (sanity check) ล้มก่อนที่ขึ้น GGUF จะเสร็จ เส้นทางเฟส 8 จึงไม่ได้สำรองอะไรไว้เลยสักอย่าง

ต้นเหตุจริง: เชื่อกันรับประกันด้านความปลอดภัยจากคำอธิบายโค้ดที่จ๋าๆ แทนที่จะไปตรวจสอบกับโค้ดจริง และความเชื่อนี้เป็นตัวรับน้ำหนักทั้งหมด — เพราะมันคือเหตุผลที่ทำให้ทุกคนไว้ใจว่าเฟส 8 เป็นเส้นทางสำรองข้อมูลได้

เสียหายอะไร: ทั้งรอบนี้อยู่ในความรู้สึกปลอดภัยจอมปลอมว่ามีข้อมูลสำรองแล้ว พอบวกกับการที่ sanity check ของเฟส 8 ล้มด้วยเส้นทางเฟส 8 จึงจะไม่ได้สำรองอะไรเลยแม้แต่ชิ้นเดียว

แก้ยังไง: ตรวจสอบกับโค้ดจริงแล้ว และเปลี่ยนกฎเป็น: การสำรองข้อมูลต้องทำแยกต่างหากเสมอ ห้ามพึ่งเฟส 8

กฎที่ได้: การรับประกันด้านความปลอดภัยทุกข้อต้องพิสูจน์กับโค้ดจริง ไม่ใช่กับความจำของตัวเอง และขั้นตอนสำรองข้อมูลที่ไม่อยู่ใน pipeline ที่อาจหยุดกลางคันได้ ไม่นับเป็นการสำรองข้อมูล — ต้องทำแยกและทำทุกครั้งไม่มีเงื่อนไข

ต้นเหตุจริงของความยุ่งเรื่อง format ทั้งหมด: spec ไม่เคยลือกรูปร่างของไฟล์ ล็อกแค่ฟิลด์

เกิดอะไรขึ้น: หลังใช้เวลาทั้งวันแก้ format ก็เจอต้นเหตุจริง: spec อธิบายวิธีเขียนแต่ละฟิลด์ แต่ไม่เคยลือกรูปร่างของไฟล์ ซึ่งเป็นเหตุผลว่าทำไมแต่ละบ้านถึงตั้งชื่อ array เองได้ เก็บหลักเสริมเป็น string ได้ หรือสะกดหลักปลอมเป็น `tie\_bar` ได้ โดยไม่ผิดกฎที่เขียนไว้เลยสักข้อ

ต้นเหตุจริง: spec ถูกเขียนแบบค่อยๆ เติมทีละกฎในระดับฟิลด์ โดยไม่มีสัญญาเชิงโครงสร้างระดับบนสุด อีสระเชิงโครงสร้างจึงไม่มีอะไรมาจำกัดตั้งแต่ต้น

เสียหายอะไร: บ้าน 11 หลังที่มีโครงสร้างแตกต่างกันไปคนละทาง ต้องตามแก้ย้อนหลังหลายร้อยจุดข้ามหลายเซชัน

แก้ยังไง: เพิ่ม §0 FORMAT LOCK ตัวใหม่ไว้บนสุดของ spec (10 กฎย่อย: ตัวห่อ, ฟิลด์ที่ element ต้องมี, element_id = มาร์คที่พิมพ์บนแบบ, รายการ element_type มาตรฐาน, หน่วยเป็น mm จำนวนเต็ม, หลักเสริมต้องเป็น object, printed_as, สัญกรณ์ grid_ref, ห้ามแต่ง pattern เอง, checklist ปิดท้าย) และเขียน `tools/check\_format.py` เพื่อบังคับตรวจสอบทั้ง 14 ข้อ โดย exit 1 เมื่อไม่ผ่าน

กฎที่ได้: spec ของข้อมูลต้องลือกรูปร่างของตัวห่อก่อน แล้วค่อยกำหนดความหมายของฟิลด์ และทุกกฎต้องตรวจด้วยเครื่องได้ชุดข้อมูล 1000 หลังไม่มีทางตรวจด้วยมือไหว

material_list กินไป 435 ไฟล์ = 37% ของแรง annotation ทั้งหมด แล้วให้ element เป็นศูนย์

เกิดอะไรขึ้น: พอนับ dataset จริง (11 หลัง 1,180 ไฟล์ 4,625 element) พบว่า material_list กินไป 435 ไฟล์ = 37% ของงาน annotation ทั้งหมดที่ทำมา แต่ผลิต element ได้ = 0 เพราะมันเป็น pattern แบบตาราง ไม่มีตำแหน่งของ element เลย ถ้าทำ annotation เฉพาะ pattern ที่ Constant ใช้จริง บ้านหนึ่งหลังจะลดจาก ~107 หน้าเหลือ ~38 หน้า

ต้นเหตุจริง: ขอบเขตงาน annotation ถูกกำหนดว่า 'ทุกหน้าของทุกหลัง' โดยไม่เคยวัดว่า pattern ไหนสร้างมูลค่าให้ปลายทางบ้าง สัญชาตญาณที่ว่าต้องทำให้ครบทุกหน้า ไม่เคยถูกเอาไปเทียบกับสิ่งที่ปลายทางอ่านจริง

เสียหายอะไร: ประมาณ 37% ของแรงคนที่ทำ annotation มาทั้งหมด ไม่ได้ element เลยสักตัว และถ้าเดินตามเป้าที่จะเพิ่มอีก 20 หลัง จะกลายเป็นต้องทำ 2,140 หน้าแทนที่จะเป็น ~750 หน้า คือเสียแรงเปล่า 65%

แก้ยังไง: สกิล op04 ตัวใหม่ข้าม material_list เป็นค่าตั้งต้น พร้อมเขียนหลักฐาน 435 ไฟล์/37%/element เป็นศูนย์ ไว้ในตัวสกิลเลย เขียน dataset_sizing.md พร้อมตัวเลขจริง และข้าม Pass 3 ทั้งหมดเป็นค่าตั้งต้นด้วย

กฎที่ได้: ก่อนจะขยายงาน annotation ให้ใหญ่ขึ้น ให้นับก่อนว่าแต่ละ pattern ผลิต element ได้กี่ตัวต่อไฟล์ แล้วตัด pattern ที่ให้ผลเป็นศูนย์ทิ้ง — วัดจาก dataset ที่มีอยู่ในมือแล้ว ดีกว่าเดาว่าทำให้ครบทุกหน้าคือทางที่ปลอดภัยที่สุด

สรุปผิดว่า Unsloth กลืน grammar constraint ไป แล้วเอาข้อสรุปนั้นไปเขียนลง SUMMARY.md

เกิดอะไรขึ้น: หลังจากกรอบแรกที่เปิด grammar แล้วยังได้ JSON ที่ parse ไม่ได้ ก็รีบสรุปทันทีว่าตัวโหลดของ Unsloth คงทั้ง grammar processor ไป แล้วเอาข้อสรุปนั้นไปเขียนลง SUMMARY.md เลย ซึ่งผิด พอไล่อ่านโค้ดที่ละชั้นก็พบว่า Unsloth ส่งต่อทุกอย่างครบ และ grammar ทำงานอยู่ตลอด ต้นเหตุจริงคือ lm-format-enforcer ถือว่าคอมมาท่าย ({ "a": 1, }) เป็นของถูกต้อง แต่ json ของ Python ไม่ถือ พิสูจน์ได้ด้วยบรรทัดเดียว: ลบคอมมาท่ายออก ไฟล์ที่ว่า "พัง" ก็ parse ได้ทันที

ต้นเหตุจริง: เลือกผู้ต้องสงสัยที่ฟังดูเข้าท่า แทนที่จะไปอ่านเส้นทางโค้ดจริง แล้วเอาการเดาไปบันทึกเป็นข้อค้นพบ ไม่ใช่เป็นสมมติฐาน เพราะเรื่อง "สองโลบารรียนยามคำว่า JSON ที่ถูกต้องไม่ตรงกันแบบเสียบๆ" มันเป็นคำอธิบายที่ฟังไม่มันเท่า "ตัวหอกกลืนมันไป" ผู้ต้องสงสัยที่ผิดเลยนะ

เสียหายอะไร: ต้นเหตุที่ผิดแต่เขียนอย่างมั่นใจ นิ่งอยู่ใน SUMMARY.md ของโปรเจกต์ พร้อมจะพาคนอ่านคนถัดไป (หรือตัวเราเอง อีกสามเดือนข้างหน้า) ไปผิดทาง ในไดอารี่บันทึกว่านี่คือบทเรียนที่แพงที่สุดของคืนนั้น และกลายเป็นกฎ rule_of_tune ข้อ 14

แก้ยังไง: ไล่หาต้นเหตุจริงด้วยการอ่านโค้ดที่ละชั้น เปลี่ยน backend ของ grammar ตัวตั้งต้นไปเป็น xgrammar (ซึ่งไม่มีทางพ่นคอมมาท่ายออกมาได้ตั้งแต่ต้นทาง) โดยมี lm-format-enforcer เป็นตัวสำรอง แก่ SUMMARY.md ให้ถูก และบันทึกเหตุการณ์นี้เป็น rule_of_tune ข้อ 14

กฎที่ได้: อย่าเขียนต้นเหตุลงเอกสารจนกว่าจะพิสูจน์ได้ด้วยการอ่านโค้ดจริงหรือทำซ้ำได้ด้วยบรรทัดเดียว — ถ้าหลักฐานยังไม่พอ ให้เขียนว่า 'ยังไม่รู้ต้นเหตุ สงสัย A หรือ B' ข้อสรุปที่ผิดแต่เขียนอย่างมั่นใจในเอกสาร อันตรายกว่าการไม่มีข้อสรุปเลย

สอง subtask ที่ recall เป็น 0% เหมือนกัน เกือบถูกเข้าใจผิดว่าเป็นปัญหาเดียวกัน

เกิดอะไรขึ้น: plan_beam กับ section ต่างรายงาน id-recall 0% ทั้งคู่ แต่จริงๆ เป็นคนละโรคกัน — plan_beam มีเพดานที่ทำได้จริง 88-93% (เป็นความล้มเหลวของโมเดลจริง) ส่วน section ในบ้านทดสอบเพดานอยู่ที่ $1/23 = 4\%$ (โมเดลที่สมบูรณ์แบบที่สุดก็ทำได้แค่ 4% metric แทบไม่บออะไรเลย)

ต้นเหตุจริง: รายงาน recall เป็นแค่เปอร์เซ็นต์เปล่าๆ โดยไม่มีเพดานที่ทำได้จริงกำกับไว้ด้วย ทำให้ความล้มเหลวสองแบบที่ต่างกันโดยสิ้นเชิงออกมาเป็นตัวเลขที่หน้าตาเหมือนกัน

เสียหายอะไร: ถ้าเข้าใจผิดว่าเป็นปัญหาเดียวกัน จะเสียทั้งรอบ tuning (และค่าเช่า GPU) ไปกับ subtask ที่โมเดลทำคะแนนไม่ได้ อยู่แล้วโดยธรรมชาติ ในขณะที่ปัญหาลงจริงยังไม่ถูกแตะเลย

แก้ยังไง: สร้าง `measure\_id\_ceiling.py` คำนวณเพดานที่ทำได้จริงต่อ subtask แล้วรายงานคู่กับค่าจริงของทั้ง 6 subtask แยกการรายงานเป็น `recall` กับ `recall\_printed` ในสคริปต์ eval

กฎที่ได้: ห้ามรายงานคะแนนโดยไม่มีเพดานที่ทำได้จริงกำกับ ตัวเลขเปอร์เซ็นต์ที่เหมือนกันเป๊ะ อาจเป็นปัญหาค้นหาคะช้ากันเลยก็ได้ และเพดานนี้แหละคือตัวที่จะแยกมันออกจากกัน

สมมติฐาน "การย่อภาพคือสาเหตุที่ plan_beam recall เป็น 0%" สรุปลจากเลขคณิต token แล้วถูกล้มด้วยการดูภาพจริง

เกิดอะไรขึ้น: การวิจัยรอบก่อนสรุปลจากเลขคณิต token ว่าการย่อขนาดภาพคือสาเหตุที่ plan_beam recall เป็น 0% พอทดสอบด้วยตาจริง — ครอบภาพ training ที่ถูกย่อมากที่สุด (บ้าน_ใหญ่_1ชั้น_01 หน้า15) ที่ความละเอียดตรงกับที่สเกลจริงที่โมเดลเห็นตอน MAX_PIXELS=7680 — พบว่าลูกบับเบิ้ลกริด มาร์คคาน มาร์คเสา และตัวอักษรบอกระยะ อ่านออกเท่ากันทั้งสองแบบ

ต้นเหตุจริง: ข้อสรุปเชิงสาเหตุถูกอนุมานจากตัวแทนทางอ้อม (จำนวน token) โดยไม่เคยดูจริงๆ ว่าโมเดลเห็นอะไร การเสีย token กับการเสียความชัดเจนที่อ่านออกได้ ไม่ใช่เรื่องเดียวกัน

เสียหายอะไร: ทีมเกือบจะไปซื้อความละเอียดภาพเพิ่ม (พิจารณาเช่า H200 ด้วยซ้ำ) เพื่อแก้ปัญหาที่ความละเอียดไม่ใช่สาเหตุ ความสัมพันธ์จริงที่พบแทนคือความยาวของ label JSON ใน GT — ค่าตอบของ plan_beam ยาวกว่าของ plan_footing 2.3 เท่า จากแผนเดียวกัน ความละเอียดเดียวกัน

แก้งัง: ทำการเปรียบเทียบกับตาจริงบนข้อมูล training จริง (ไม่ต้องใช้ GPU) บันทึกความชัดเจนนี้ไว้ และปรับทฤษฎีการทำงานใหม่เป็น "โมเดลล้มเหลวในการผลิต JSON ก้อนยาวที่สอดคล้องกันภายในตัวเองในการตอบครั้งเดียว" พร้อมข้อเสนอขั้นถัดไปคือแบ่ง plan_beam เป็นคำตอบย่อยตามโซนกริดแล้วเอาโค้ดมารวมทีหลัง

กฎที่ได้: ห้ามสรุปความล้มเหลวเชิงการรับรู้จากเลขคณิต token ต้อง render สิ่งที่โมเดลได้รับจริงๆ แล้วดูด้วยตา ครอบแล้วเทียบ 5 นาทีให้คำตอบที่ดีกว่าเลขคณิตหนึ่งชั่วโมง และอาจช่วยประหยัดค่าเช่า GPU ได้ทั้งก้อน

ริบสเกลไปเทรนพร้อมกัน 4 fold เต็มรูปแบบ (~60 GPU-hour, ~\$63) ก่อนจะพิสูจน์ถูกๆ ว่าโมเดลอ่านแบบได้จริงหรือไม่

เกิดอะไรขึ้น: รอบนี้กระโดดจาก 'dry-run 5 steps ผ่าน' ไปเป็น 4 fold × 306 step รันพร้อมกันบนการ์ดเช่า 4 ใบเลย ใบละ ~15-16 ชม. ทั้ง 4 fold รันจนจบและได้ adapter ออกมา และหลังจากทั้ง 4 fold เสร็จแล้ว eval ให้คะแนน 0% บน plan_beam/plan_footing การไล่หาสาเหตุถึงเจอว่า `crop\_to\_patches: true` หายไป ทำให้ไม่มีภาพไหนอ่านออกได้เลยตอนเทรน ค่าเช่า GPU ทั้งรอบได้ adapter 4 ตัวที่อ่านแบบไม่ได้เลย กฎที่เขียนแก้ให้ t05 บอกไว้ตรงๆ ว่าต้องเทรน fold เดียวก่อน ห้ามเริ่มด้วย 4 fold พร้อมกัน

ต้นเหตุจริง: dry-run พิสูจน์ได้แค่ระบบเดินได้ (VRAM พอ, loss เป็นตัวเลข, checkpoint บันทึกได้) แต่ไม่เคยพิสูจน์ว่าโมเดลเห็น pixel ที่อ่านออกได้จริง ไม่มี gate ราคาถูกคั่นระหว่าง 'config parse ผ่าน / รัน 5 step ได้' กับ 'หุ้ม 60 GPU-hour บน 4 การ์ด' การตัดสินใจสเกลถูกกดดันด้วยเวลา ('เวลาจริงลดครึ่ง 29→~16 ชม.') และงบที่อนุมัติไว้แล้ว ('อย่าสนบ ผมจ่ายเอง') ไม่ใช่ตัดสินใจจาก config ที่พิสูจน์แล้วว่าถูกต้อง

เสียหายอะไร: เสียค่าเช่า GPU ~\$63 บวก ~60 GPU-hour และเวลาทำงานจริงราว 24 ชม. ตลอด 2 วัน ได้ adapter ที่ใช้งานไม่ได้ 4 ตัว ยังต้องมาอัปโหลดขึ้น HF แยก 4 รอบ ตรวจ sha256 4 รอบ ทำลายอินสแตนซ์ 4 รอบ บวกรอบ eval อีกต่างหาก

แก้งัง: ไม่ได้แก้สำหรับ t04 (เงินจ่ายไปแล้ว) แต่เขียนเป็นกฎชัดเจนสำหรับ t05: ต้องเทรน fold เดียวก่อน บวกกับตรวจสอบ image-token แบบ CPU-only ก่อนรันเทรนจริงที่ต้องจ่ายเงินทุกครั้ง

กฎที่ได้: การสเกลออกแบบขนานจะขยายทุกอย่างที่ config ทำ รวมถึงบั๊กด้วย ก่อนจะหุ้มการ์ดเกินหนึ่งใบ ต้องรันการตรวจสอบราคาถูกแบบ end-to-end สักครั้งที่พิสูจน์ว่าโมเดลรับรู้ input จริง (probe จำนวน token ต่อภาพ หรือดูตัวอย่างที่ resize แล้วด้วยตา) ไม่ใช่แค่พิสูจน์ว่า loop เทรนรันได้

รัน JSON pretty-print หับทั้งชุด 343 ไฟล์ หับไฟล์ 16 ไฟล์ที่เพื่อนร่วมทีมเพิ่งตรวจสอบเสร็จ

เกิดอะไรขึ้น: คอมมิต `be9ad80` ("แก็ระบบ") รัน JSON pretty-print หับทั้ง repo 343 ไฟล์ใน json_แก้ไขแล้ว รวมทั้ง 16 ไฟล์ในบ้าน 24, 26, 27 ที่ Ark เพิ่ง commit ไปไม่กี่นาทีก่อนหน้านี้ มาพบที่หลังตอนไล่ประวัติของบ้าน 42

ต้นเหตุจริง: รันคำสั่งกว้างๆ แบบอัตโนมัติโดยไม่เช็ค `git log` ก่อนว่ามีใครเพิ่ง commit หับ path เดียวกันหรือเปล่า ไม่มีการเช็คเจ้าของหรือความสดใหม่ของงานมากนักไว้เลย

เสียหายอะไร: งานที่ Ark เพิ่งทำเสร็จถูกจัด format หับโดยไม่มีการเตือนล่วงหน้า ต้องมาทำ semantic diff เอง (เขียนสคริปต์ Node เอง ที่ normalize ลำดับ key และมองว่า element_id:"name" กับ element_id:null+element_name_assigned เท่ากัน) หับทั้ง 343 ไฟล์เพื่อพิสูจน์ว่าไม่มีข้อมูลหาย ต้องขอให้ Ark หยุด push/pull ชั่วคราวและสำรองไฟล์ในเครื่องตัวเอง บวกเขียนบันทึกเหตุการณ์และจดหมายขอโทษอย่างเป็นทางการ

แก้งัง: ยืนยันว่าข้อมูลจริงไม่ต่างกันเลย มีแค่ format ที่เปลี่ยน เผยแพร่บันทึกเหตุการณ์ + จดหมายขอโทษเป็นภาษาไทยในรูปแบบ artifact (ขอโทษเรื่องกระบวนการทำงาน ไม่ใช่เรื่องข้อมูลหาย เพราะไม่มีข้อมูลหายจริง)

กฎที่ได้: ก่อนรันคำสั่งอัตโนมัติกว้างๆ หับทั้ง repo ต้องรัน `git log --since` เช็ค path เป้าหมายก่อนว่ากำลังจะไปแตะงานของใคร ถ้ามีคนเพิ่ง commit ที่นั่น ให้ถามก่อน แม้จะเป็นแค่การเปลี่ยน format ก็ยังนับเป็นการเปลี่ยนสถานะงานที่คนอื่นกำลังทำอยู่

เปลี่ยนโมเดลมาใช้ InternVL3-78B ทำให้ toolchain ของ Unsloth ทั้งชุดใช้ไม่ได้ — ทั้งที่โมเดลนี้เคยถูกปฏิเสธไปแล้วเมื่อ 3 วันก่อน

เกิดอะไรขึ้น: มะขามสั่งให้ t04 ใช้ OpenGVLab/InternVL3-78B แต่ Unsloth ไม่รองรับ InternVL3 (อ้างอิง issue #3401/#2341) ทำให้ train_t03.py, infer_house_t03.py และ onstart.sh ใช้งานไม่ได้ทั้งหมด ต้องติดป้าย  นอกจากนี้การ์ดจอ 96GB ใบเดียวยังไม่พอด้วย (LoRA bf16 ต้องการ $\geq 2 \times H100$ -class ขึ้นไป หรือไม่ก็ต้องลอง QLoRA 4-bit บนการ์ดเดียวซึ่งยังไม่เคยทดสอบ) ที่แย่กว่านั้นคือโมเดลตัวนี้เคยถูกปฏิเสธไปแล้วในแคตตาล็อกวันที่ 2026-08-26 ด้วยเหตุผลว่า backbone ของมันก็คือ Qwen2.5-72B อยู่ดี

ต้นเหตุจริง: แคตตาล็อกเลือกโมเดลโดยดูแค่คุณภาพ (คะแนน OmniDocBench 83.76, license) แต่ไม่ได้เช็ค toolchain รองรับไหม หรือ VRAM พอไหม ความเข้ากันได้ของ framework กับขนาดการ์ดจอไม่เคยถูกใส่เป็นเกณฑ์คัดเลือกเลย ต้นทุนของการเปลี่ยนโมเดลจึงมองไม่เห็นจนกว่าจะตัดสินใจไปแล้ว

เสียหายอะไร: สคริปต์ที่ใช้งานได้ 3 ตัวกลายเป็นแค่ของอ้างอิง ต้องเลือก framework ใหม่ กลยุทธ์ GPU ใหม่ ตัวแปลง format ของ dataset (จาก jsonl แบบ Unsloth เป็น format ของ framework ใหม่) เขียนสคริปต์ train+infer ใหม่ทั้งหมด และต้องทดสอบ xgrammar/tokenizer ใหม่ทั้งชุด ยังทำให้งานที่เพิ่งแก้ในวันเดียวกัน (MAX_PIXELS 5120→7680 และแก้ OUT_DIR จาก t03 เป็น t04) กลายเป็นของเสียเปล่าไปด้วย เพราะเพิ่งแก้เสร็จก่อนจะเปลี่ยนโมเดล

แก้งัง: ติดป้าย  บนสคริปต์ทั้ง 3 ตัวและเก็บไว้เป็นของอ้างอิงยุค Qwen เขียนเอกสาร t04_Purson/t04_workflow.md เป็น Phase 0 ที่ระบุช่องว่างทุกจุด ตอนหลังตัดสินใจเลือก framework ใหม่เป็น LLaMA-Factory (สำรอง ms-swift ไว้เผื่อ) prompt/dataset/GT/CV sidecar เป็น plain text ใช้ต่อได้เลย ต้องแปลงแค่ envelope ของ jsonl เท่านั้น

กฎที่ได้: ต้องตั้งให้การรองรับของ framework และการใส่พอดกับ VRAM การ์ดเดียวเป็นเงื่อนไขบังคับในแคตตาล็อกเลือกโมเดล คู่ไปกับ benchmark และ license — โมเดลที่ training stack ของเราโหลดไม่ได้ ไม่นับเป็นตัวเลือก ต่อให้คะแนนดีแค่ไหนก็ตาม

เอา `model.generate()` หนีบล็อกไปใส่ใน `async def` ทำให้คำขอเดียวที่หลุดลูปแวนเชิร์ฟเวอร์ inference ทั้งตัวนาน 7 ชั่วโมงกว่า

เกิดอะไรขึ้น: ยิงคำขอพร้อมกัน 2 อัน (เทส e2e กับ arm 2.4a) เข้า `serve_purson.py` หลังจากนั้นเซิร์ฟเวอร์หยุดตอบทุกอย่างนานกว่า 7 ชั่วโมง แม้แต่ `/v1/models` ซึ่งเป็น endpoint เช็คสุขภาพที่ง่ายที่สุดก็ไม่ตอบ ตรวจแล้ว GPU ยังมีชีวิต (utilization นิ่งที่ 11-14%, process state R, ไม่มี error Xid/ECC) ต้นเหตุคือ `model.generate()` ถูกเรียกตรงๆ ในตัวจัดการที่เป็น `async def` พอมีคำขอหนึ่งตกลงไปในลูปพนข้อความซ้ำ (ปัญหาที่รู้จักกันตั้งแต่ t03 — grammar คุณได้แค่โครงสร้าง JSON ไม่ได้คุมเนื้อหา) มันก็บล็อก event loop ทั้งเส้น

ต้นเหตุจริง: เอาฟังก์ชันที่ทำงานยาวและบล็อก (ใช้ CPU/GPU หนัก) ไปวางตรงๆ ใน coroutine ของ `asyncio` ซึ่ง `async def` ของ FastAPI ไม่ได้ช่วยให้ทำงานพร้อมกันได้เลยถ้าข้างในเป็นการเรียกแบบบล็อก เพราะ event loop เป็น thread เดียว generate ที่ค้างอันเดียวจึงทำให้คำขออื่นอดหมด รวมถึง health check และดูเหมือนว่าตัวเช็ค timeout เองก็โดนด้วย

เสียหายอะไร: เสีย GPU เข้าไปเปล่าๆ กว่า 7 ชั่วโมงที่ราคา \$1.070/ชม. ระหว่างที่เซิร์ฟเวอร์เข้าถึงไม่ได้เลย และต้องปิดเครื่องด้วย `presentation.py down` ก่อนที่เทสเส้นทาง browser กับโมเดลจริงจะเสร็จ ณ สิ้นวันยังแก้ไม่เสร็จ และถูกระบุว่าเป็นตัวขวางอันดับหนึ่งก่อนพรินต์

แก้ยังง: ยังไม่ได้แก้ แต่บันทึกวิธีแก้ไว้แล้ว: ห่อ `generate()` ด้วย `asyncio.to_thread()` หรือทำคิว worker thread แยก เพื่อไม่ให้คำขอเดียวล้มเซิร์ฟเวอร์ทั้งตัวได้

กฎที่ได้: ห้ามเรียกฟังก์ชันโมเดล/IO ที่บล็อกตรงๆ ใน `async def` เด็ดขาด — ต้องใช้ `asyncio.to_thread()` หรือคิว thread/process เสมอ และก่อนจะเชื่อว่าเซิร์ฟเวอร์ใช้ได้ ต้องยิงคำขอพร้อมกัน 2 อันโดยให้อันหนึ่งกินเวลานาน เพื่อพิสูจน์ว่ามันรับพร้อมกันได้จริง

บั๊กเรื่องความละเอียดรูป/จำนวน token ตัวเดิม ทำให้เรนฟ้งเป็นรอบที่สามติด (t01 → t02 → t04) เพราะสองรอบแรกไม่เคยตั้งขั้นตอนตรวจการไว้เลย

เกิดอะไรขึ้น: t01 ฟ้งเพราะ Unsloth ถอยไปใช้ 512px เอง, t02 ฟ้งเพราะ data_collator ย่อรูปแล้วไม่เคยมีใครเทียบกับที่ตั้งใจไว้, t04 ฟ้งเพราะไม่รู้ว่ามี `crop_to_patches` อยู่ สาม framework คนละตัว สามกลไกคนละแบบ แต่อาการเดียวกันเป๊ะ: โมเดลถูกป้อนรูปที่เล็กเกินกว่าจะอ่านออกโดยไม่มีอะไรฟ้อง กว่าจะมีการเขียนกฎขึ้นมา (rule_of_tune.md ข้อ 15) ให้ตรวจจำนวน token ต่อรูปจาก source แล้วรัน probe บน CPU ก่อนจ่ายเงินเรน ก็ต่อเมื่อเกิดเรื่องเป็นครั้งที่สามแล้ว

ต้นเหตุจริง: หลัง t01 กับ t02 มีการแก้บั๊กเฉพาะตัวไป แต่ไม่มีใครเพิ่มขั้นตอนตรวจการที่ใช้ได้กับทุก framework ลงในเช็คลิสต์ก่อนเรน พอเปลี่ยน framework/โมเดลใหม่ ความผิดพลาดเดิมก็กลับมาทางประตูใหม่ และต้องนั่งหาต้นเหตุใหม่ทุกครั้งหลังจากที่เงินหมดไปแล้ว

เสียหายอะไร: เสียการเรนไป 3 รอบเพราะข้อผิดพลาดชนิดเดียวกัน เฉพาะ t04 รอบเดียวคิดเป็นประมาณ \$63 และ GPU ประมาณ 60 ชั่วโมง

แก้ยังง: บันทึกเป็น rule_of_tune.md ข้อ 15: ก่อนจะเรนจริงบน framework/โมเดลคู่ใหม่ ต้องไล่อ่าน source เพื่อดูจำนวน token ต่อรูปจริงๆ แล้วยืนยันด้วย probe ที่รันบน CPU อย่างเดียว ไม่ต้องใช้ GPU

กฎที่ได้: บั๊กที่กลับมาซ้ำข้ามรอบ แปลว่าเช็คลิสต์ขาดข้อ ไม่ใช่ดวงช่วย พออาการเดิมโผล่มาเป็นครั้งที่สอง ให้หยุดแก้เฉพาะคสนั้น แล้วเพิ่มด่านตรวจก่อนเริ่มงานแบบถาวรแทน

วิกฤต 2026-08-31

rsync resume ไม่ได้ทำอะไรเลยแบบเงียบๆ: ไปเช็คเครื่องมือผิดเครื่อง แล้ว **error** ถูกกลบด้วย **exit 0** กับ ``| tail``

เกิดอะไรขึ้น: scp ไป gpu4 ค้างอยู่ที่ 83% เลยพยายาม resume ด้วย `rsync --partial --append-verify` แต่ไปเช็ค ``which rsync`` แค้นเครื่องปลายทาง ไม่ได้ใช้บน Git Bash เครื่องตัวเองซึ่งไม่มี rsync ติดตั้ง คำสั่งจึงล้มด้วย `'command not found'` แต่ task รายงาน exit 0 เพราะ echo ที่ต่อท้ายทำงานสำเร็จ แล้ว ``| tail -2`` ก็ตัดข้อความ error ที่ไฟล์บน gpu4 จึงเป็นแค่ scp ที่ถูกฆ่ากลางทาง

ต้นเหตุจริง: ความผิดพลาดอย่างซ้อนกัน: เช็ความีเครื่องมือผิดเครื่อง, อ่านค่า exit code ของคำสั่งรวมแทนที่จะอ่านของคำสั่งที่สนใจจริงๆ, และตัด output ที่มีข้อความความล้มเหลวอยู่ทิ้ง

เสียหายอะไร: gpu4 เกือบเหวี่ยงไปโดยที่รูปหาย 60 ใบแบบเงียบๆ (md5 ไม่ตรง 2aa1039d... เทียบกับ 4b765a37..., แตก tar ไม่ได้ ไม่ครบ มีรูปแค่ 824 จาก 884 ใบ) จับได้เพราะมีการตรวจ md5 เป็นขั้นตอนบังคับเท่านั้น

แก้ยังไง: ส่งไฟล์ใหม่แล้วตรวจ md5 ให้ตรง พร้อมบันทึกกฎสองข้อไว้ให้ชัด

กฎที่ได้: เช็ความีเครื่องมือหรือเปล่าบน 'เครื่องที่จะรันคำสั่งนั้น' ไม่ใช่เครื่องปลายทาง และห้ามตัดสินว่าสำเร็จจาก exit code ของคำสั่งที่ต่อกันหลายตัว หรือจาก output ที่ถูกตัดมาแค่บางส่วน — ต้องอ่าน stderr ตัวจริงและยืนยันด้วย checksum

วิกฤต 2026-08-31

ปัญหา decoder ถูกบันทึกและทดสอบไว้ตั้งแต่ **2026-08-30** แต่ไม่เคยเอาไปแก้ใน **serve_purson.py**

เกิดอะไรขึ้น: `no_repeat_ngram_size=8 + repetition_penalty=1.15` ถูกระบุไว้แล้วตั้งแต่ 30 ส.ค. ว่าเป็นปัญหาซ้ำเติมของ t04 พร้อมผลทดสอบว่าเอาสองตัวนี้ออกแล้วจำนวน element ขยับจาก 0 เป็น 10 แต่ **serve_purson.py** ไม่เคยถูกอัปเดต ข้อบกพร่องเดิมจึงทำให้วันนี้ได้ 0% และหล่นออกมาเป็นภาษาจีน

ต้นเหตุจริง: บทเรียนถูกเขียนลงบันทึกประจำวัน แต่ไม่ได้เอาไปแก้ในทุกไฟล์ที่มีโค้ดตัวเดียวกันอยู่

เสียหายอะไร: เสียการวัดผลไปทั้งรอบ เพื่อทำซ้ำความล้มเหลวที่วินิจฉัยไว้ตั้งแต่เมื่อวานแล้ว

แก้ยังไง: แก้ใน **serve_purson.py** แล้ว พร้อมบันทึกกฎว่าต้องกระจายการแก้ไขไปที่

กฎที่ได้: เมื่อบันทึกประจำวันเขียนว่า 'X เป็นปัญหา' ให้ grep ทั้งโปรเจกต์หา X แล้วแก้ทุกจุดในวันเดียวกันเลย — บทเรียนที่จดไว้แต่ไม่ได้เอาไปใช้ คือบทเรียนที่ยังไม่ได้เรียนรู้

วิกฤต 2026-08-31

ผ่านไปอีกวันหลังเทรนเสร็จ โดยไม่มีใครรู้ว่าโมเดลอ่านออกจริงหรือเปล่า

เกิดอะไรขึ้น: เทรนเสร็จ รวม adapter แล้วอัปโหลดขึ้นเรียบร้อย กว่าจะมีคนวัด recall จริงก็วันถัดไป แล้วปรากฏว่าได้ 0% ภายใต้อาคาร generate ที่ส่งขึ้นไปจริง

ต้นเหตุจริง: pipeline การเทรนไม่มีขั้นตอนวัด recall แบบอัตโนมัติเลย การตัดสินว่าสำเร็จดูจากค่า loss กับดูว่ามีไฟล์ผลลัพธ์อยู่บน HuggingFace เท่านั้น

เสียหายอะไร: เสียเวลาไปเต็มวันและจ่ายค่า GPU เพิ่ม ไปกับโมเดลที่ไม่มีใครรู้ความสามารถจริง และตามที่ตั้งค่าไว้คือศูนย์

แก้ยังไง: ระบุเป็นสิ่งที่ต้องเพิ่มในรอบถัดไป: ให้มีการวัด recall อัตโนมัติอยู่ในรอบการเทรนเลย

กฎที่ได้: ทุกรอบการเทรนต้องจบด้วยการวัด recall ระดับงานจริงแบบอัตโนมัติบนบ้านที่กันไว้ไม่ให้โมเดลเห็น ก่อนจะประกาศว่าเสร็จ — ค่า loss กับการอัปโหลดสำเร็จ ไม่ได้พิสูจน์ความสามารถอะไรเลย

ปัญหาความละเอียดรูปโพลเป็นรอบที่สามติดกัน ทั้งที่มีบทเรียนมาแล้วสองครั้ง

เกิดอะไรขึ้น: t01 (Unsloth ถอยไป 512px), t02 (data_collator ย่อรูปโดยไม่มีใครเทียบ) และ t04 (ไม่รู้ว่ามี crop_to_patches) ล้วนเป็นปัญหาชนิดเดียวกัน คือโมเดลเรนด้วยความละเอียดต่ำกว่าที่ตั้งใจไว้มากแบบเสียวๆ บรรทัด config ของ t04 เขียนโดย Claude เมื่อ 2026-08-29 ทั้งที่มีบทเรียนมาแล้วสองครั้ง

ต้นเหตุจริง: บทเรียนก่อนหน้านี้ถูกบันทึกเป็นเรื่องเล่าเกี่ยวกับพารามิเตอร์ตัวใดตัวหนึ่ง แทนที่จะแปลงเป็น assert บังคับก่อนรันที่ครอบคลุมปัญหาทั้งชนิด

เสียหายอะไร: เสียการเรนทั้งรอบที่ใช้ 4 GPU บวกเวลาเต็มวัน และต้องทิ้งโมเดล — เป็นการซ้ำรอยครั้งที่สามของปัญหาชนิดเดียวกัน

แก้ยังไม: รอบนี้เพิ่ม preflight_seq_lengths() และตรวจนับ tile/token จริงบนเครื่องเรน ปัญหาชนิดนี้จึงมีตัวตรวจที่รันได้จริงคุมอยู่แล้ว

กฎที่ได้: ความล้มเหลวชนิดไหนที่เกิดซ้ำ ให้แปลงเป็น assert บังคับก่อนรันที่รันได้จริง — บทเรียนที่เขียนเป็นข้อความบรรยายพิสูจน์แล้วว่ากันการเกิดซ้ำไม่ได้ถึงสามครั้ง

ปัญหาชนิด 'รูปถูกย่อแบบเสียวๆ' ตัวเดียวกัน ฆ่าการเรนไปแล้ว 3 รอบ (t01, t02, t04)

เกิดอะไรขึ้น: t01: Unsloth ย่อรูปเป็น 512px เสียวๆ โดยไม่บอกใคร t02: ตัวแพ็คข้อมูลย่อรูป แล้วไม่มีใครเทียบขนาดก่อน-หลังเลย t04: ไม่มีใครรู้ว่ามีสวิตช์ crop_to_patches อยู่ สามกลไกคนละแบบ แต่ล้มเหลวเหมือนกันเป๊ะ และไม่มี error โพลมาสักครั้ง

ต้นเหตุจริง: หลังเกิดสองครั้งแรก ไม่มีการติดตัวกันถาวรไว้เลย แต่ละรอบถูกมองว่าเป็น config ผิดครั้งเดียวจบ แทนที่จะมองว่าเป็นความล้มเหลวชนิดที่เกิดซ้ำได้ ถ้าไม่มีการวัดบังคับก่อนรัน framework หรือตัวโหลดข้อมูลตัวใหม่ก็จะพาค่าย่อรูปเริ่มต้นของตัวเองกลับมาอีก

เสียหายอะไร: เสียการเรนไป 3 รอบ ได้โมเดลที่ใช้ไม่ได้ทั้งหมด แถมค่าใช้จ่ายทยบขึ้นเพราะทุกรอบต้องนั่งไล่หาต้นเหตุใหม่ราวกับเป็นเรื่องใหม่

แก้ยังไม: บันทึกเป็น rule_of_tune.md ข้อ 15 — บังคับวัด token ต่อรูปก่อนเรน ทำบน CPU ฟรีๆ — ซึ่งเปลี่ยนจากการค้นพบเฉพาะกิจซ้ำๆ ให้กลายเป็นด่านตรวจถาวร

กฎที่ได้: พอลความล้มเหลวแบบเดิมโพลมาครั้งที่สอง ให้หยุดแก้เฉพาะเคส แล้วติดตัวกันอัตโนมัติแบบถาวรแทน บั๊กที่เกิดซ้ำถึงสามครั้งคือกระบวนการที่ขาดไป ไม่ใช่ config ผิดสามครั้ง

คำถามค้าง 4 ข้อของ schema ถูกจัดไว้ในเอกสารแต่ไม่เคยถูกถามผู้ตัดสินใจจริงๆ

เกิดอะไรขึ้น: Generation 3.1 เปิดคำถามใหม่ 4 ข้อ — main_bar ที่บนล่างไม่สมมาตร, การยืนยัน additional_bars[] อย่างเป็นทางการ, ตารางกำหนดขนาดเสาหลายชั้น (ควรมีฟิลด์ 'level' แยก หรือฝังไว้ใน element_id) และฟิลด์แผ่นพรีคาสท์ (dowel_bar/topping_mesh/topping_thickness_min_mm/level_step_mm) ไดอารี่ระบุตรงๆ ว่าคำถามเหล่านี้ 'ยังไม่เคยถูกถาม' มะขามเลย จนถึงสิ้นวันที่ 6 กรกฎาคม ทั้งที่มะขามอยู่ในเซสชันเดียวกันทั้งวัน

ต้นเหตุจริง: คำถามถูกจัดลงเอกสารแทนที่จะถามในบทสนทนาสด การเขียนคำถามลงไปให้ความรู้สึกว่าเป็นความคืบหน้า และลดแรงกดดันที่จะถามจริง โดยเฉพาะเมื่อคนที่ตอบอยู่ตรงหน้าและงานกำลังไหลลื่นอยู่

เสียหายอะไร: การตัดสินใจเรื่อง schema สีซ้อยังไม่ถูกตัดสิน ในขณะที่การสกัดข้อมูลยังเดินหน้าต่อไปเรื่อยๆ มัน หมายความว่าข้อมูลถูกผลิตเพิ่มขึ้นภายใต้สมมติฐานที่อาจถูกกลับคำที่หลัง ทั้งสีซอถูกยกไปเป็นรายการค้าง #2 ณ สิ้นวัน

แก้ยังง: ยังไม่ได้แก้ — บันทึกไว้ตรงๆ ว่า 'ยังไม่ถูกถาม ณ วันนี้ (6 กรกฎาคม) ด้วย — ค้างจริง'

กฎที่ได้: ถ้าผู้ตัดสินใจอยู่ในเซสชัน ให้ถามคำถามที่เป็นตัวกันงานในเซสชันนั้นเลย ห้ามแฉจัดลงไฟล์ คำถามที่จัดไว้ในเอกสารคือคำถามที่จะถูกถามเข้าไปหลายวัน หรือไม่ถูกถามเลย ในขณะที่ข้อมูลยังคงถูกผลิตต่อไปภายใต้สมมติฐานที่ผิด

รายการค้างพองขึ้นเป็น 9 ข้อ หลายข้อค้างข้ามสองวันทำงานเต็มๆ โดยไม่มีใครแตะ

เกิดอะไรขึ้น: หุกรายการในไดอารี่วันที่ 2026-07-06 จบด้วยประโยค 'รายการค้างเดิมทั้งหมดยังเหมือนเดิม' และวันนั้นปิดท้ายด้วยรายการค้าง 9 ข้อ หลายข้อ — หน้า 41-47 BOQ, ความขัดแย้งเกรดคอนกรีต 210 กับ 240, ไฟล์ 0-byte เปล่า — ค้างมาตั้งแต่ 2026-07-05 แล้ว และถูกจัดเข้าคำต่อคำในหุกรายการติดกันโดยไม่มีการลงมือทำอะไร

ต้นเหตุจริง: รายการค้างเป็นแบบ append-only ไม่มีเจ้าของ ไม่มีขีดจำกัดขนาด และไม่มีขั้นตอนคัดกรอง แต่ละเซสชันเริ่มงานใหม่แทนที่จะปิดงานเก่า พิธีกรรมการทวนรายการซ้ำๆ กลายเป็นสิ่งทดแทนการลงมือทำจริง

เสียหายอะไร: สองในรายการที่ค้างข้ามวันคือตัวกันความถูกต้องของข้อมูล (หน้า BOQ ที่หายไป, เกรดคอนกรีตที่คลุ้มเครือ) ซึ่งกันความใช้ได้ของทั้งชุดข้อมูลบ้านหลังนี้ ในขณะที่แรงงานของวันนั้นไปลงกับเครื่องมือรีวิวนั่น

แก้ยังง: ยังไม่ได้แก้ — รายการ 9 ข้อถูกยกไปไว้ใน SESSION_HANDOFF_2026-07-06.md เป็นจุดเริ่มต้นของเซสชันถัดไป

กฎที่ได้: จำกัดขนาดรายการค้าง แล้วคัดกรองมันทุกครั้งที่เราเริ่มเซสชันใหม่ ไม่ใช่ตอนจบ อะไรก็ตามที่ถูกจัดเข้าสามเซสชันติดต่อกัน ต้องทำให้เสร็จ ทั้งไป หรือมอบหมายเจ้าของกับกำหนดวัน การทวนรายการซ้ำไม่ใช่การติดตามงาน — รายการค้างที่ไม่มีเจ้าของ คือการตัดสินใจว่าจะไม่มีวันทำมันเลย

สูง 2026-07-06

schema เปลี่ยนเวอร์ชันสามครั้งในวันเดียว (3.1 → 3.2 → 3.3) หลังจากสกัดข้อมูลเสร็จไปแล้ว ทำให้ต้องสกัดใหม่ 37 หน้าพร้อมแก้มือเพิ่ม

เกิดอะไรขึ้น: หน้า 1-60 ถูกสกัดเมื่อ 2026-07-05 ภายใต้ Gen 3 วันถัดมา schema เพิ่ม span_source + atomic beam segment (3.1), ข้อตกลง dummy-grid + grid master หน้า-00 (3.2), และการ join สเปกเข้า plan (3.3) แต่การเปลี่ยนแปลงทำให้ข้อมูลที่เขียนไปแล้วใช้ไม่ได้: ต้องสั่งทำรอบที่สองเต็ม (t2) ครอบคลุมหน้า 1-37, หน้า 19 view2 กับหน้า 20 view1 ต้องเขียนใหม่ทั้งหมด และไฟล์ t2 อีกสองไฟล์ต้อง join สเปกตามเข้าไปในทุก element ด้วยมือทีหลัง

ต้นเหตุจริง: schema ยังอยู่ระหว่างออกแบบในขณะที่การสกัดข้อมูลจำนวนมากกำลังรันขนานกันอยู่แล้ว — คำถามค้างจาก 2026-07-05 (บวกอีก 4 ข้อที่เปิดใหม่วันที่ 07-06) ไม่เคยถูกปิดก่อนที่ข้อมูลจะถูกผลิตออกมา ทุกการตัดสินใจด้าน design จึงลงเอยเป็นงานทำซ้ำกับไฟล์ที่มีอยู่แล้ว แทนที่จะเป็นข้อจำกัดสำหรับไฟล์ใหม่

เสียหายอะไร: ผลผลิตการสกัดข้อมูลเกือบเต็มวัน (หน้า 1-37 ราว 60 ไฟล์) ต้องทำใหม่ทั้งหมด บวกกับการแก้ไฟล์ JSON แต่ละไฟล์ด้วยมือ บวกกับการสร้างไฟล์ task ของ Label Studio ปลายทางใหม่ทั้งหมด ส่วนหน้า 38-60 ถูกทิ้งไว้ที่ schema เก่า ไม่เคยถูกอัปเดตตาม ทำให้ dataset ของบ้านหลังนี้ไม่สอดคล้องกันภายในตัวเองข้ามช่วงหน้า

แก้อย่างไร: สร้างรอบ t2 สำหรับหน้า 1-37 ส่วนหน้า 38-60 ถูกทิ้งไว้ที่ schema ของ t1 และไม่เคยถูกอัปเดตให้ทันสมัย

กฎที่ได้: ล็อก schema ให้นิ่งก่อนเริ่มสกัดข้อมูลจำนวนมาก หรือสกัดชุดนาร์่องเล็กๆ ไปก่อนจนกว่า schema จะหยุดเปลี่ยน การเปลี่ยน schema หลังจากสกัดไปแล้ว N หน้า จะเสียค่าทำซ้ำ N หน้าทุกครั้งไป — รวบรวมคำถามค้างทั้งหมดแล้วตอบให้ครบก่อนส่งเอเจนต์ไปทำงาน

สูง 2026-07-07

grid master มี dummy grid A'' ที่อาจไม่ใช่เส้นกริดจริง แต่เป็นพื้นที่ปรับระดับดิน

เกิดอะไรขึ้น: ตอนแก้ปัญหาความสับสนเรื่องส่วนยื่นของบ้านหลังที่ 1 confidence ของ dummy grid 1'/3'/A' ถูกปรับขึ้น แต่ของ A'' ถูกปรับลง — สงสัยว่า A'' ไม่ใช่เส้นกริดจริง แต่เป็นพื้นที่ปรับระดับดินที่ถูกเข้าใจผิดว่าเป็นกริด ถึงอย่างนั้น A'' ก็ยังอยู่ใน grid master เหมือนเดิม

ต้นเหตุจริง: เส้นบอกระดับพื้น/ดินถูกอ่านเป็นเส้นกริดโครงสร้างในรอบสกัดข้อมูลก่อนหน้า และ grid master ถูกสร้างขึ้นก่อนที่จะมีใครตรวจสอบ dummy grid แต่ละเส้นข้ามกับแบบแผนที่สอง

เสียหายอะไร: grid master เป็นตัวกำหนด grid_ref/grid_ref_start/grid_ref_end ของทุก element ในบ้าน ถ้ามีเส้นกริดที่แต่งขึ้นมา element ทุกตัวที่อ้างอิงเส้นนี้จะได้พิกัดที่ไม่มีอยู่จริงบนแบบ — และเรื่องนี้หลุดเข้าไปใน t3, t4 และโฟลเดอร์ 01 ที่ใช้งานจริงโดยยังไม่แก้

แก้อย่างไร: ยังไม่แก้ — แคลค confidence และบันทึกข้อสงสัยไว้ ไม่เคยลบหรือยืนยัน A'' อย่างจริงจัง

กฎที่ได้: dummy grid ต้องยืนยันจากแบบอย่างน้อย 2 แผ่นอิสระก่อนจะเข้า grid master ได้ (มาตรฐานเดียวกับที่ใช้ทีหลังกับบ้านหลังที่ 3 ซึ่งมี 3 คนยืนยัน 1'/3' อิสระกัน) การให้ confidence ต่ำไม่ใช่ทางออกแทนการแก้ไขจริง — ข้อผิดพลาดใน grid master จะไหลไปกระทบทุก element ในบ้านหลังนั้น

สูง 2026-07-10

ไม่เคยตัดสินใจว่าจะแยกโปรเจกต์ต่อบ้าน หรือรวมทุกบ้านไว้โปรเจกต์เดียว จนบล็อกการ import อยู่หลายวัน

เกิดอะไรขึ้น: ตอนเช็คความพร้อมของบ้านหลังที่ 2 เจอตัวบล็อกสองเรื่อง เรื่องชื่อไฟล์ชนกันแก้เสร็จวันเดียวกัน ส่วนอีกเรื่อง — "ยังไม่ตัดสินใจว่า Label Studio จะแยกโปรเจกต์ต่อบ้าน หรือรวมทุกบ้านไว้ในโปรเจกต์เดิม" — ไม่เคยถูกตัดสินใจเลย และไฟล์ซ้ำแบบเดิมในรายการงานค้างของทุกบันทึกหลังจากนั้นจนถึง 11 ก.ค.

ต้นเหตุจริง: การตัดสินใจที่บล็อกงานถูกบันทึกเป็นแผนงานค้าง แทนที่จะบังคับให้เคาะให้จบ แล้วก็ทำงานอ้อมมันไปเรื่อยๆ บ้านหลัง 2 กับ 3 ถูกสกัดข้อมูลและสร้างไฟล์ task เสร็จ ทั้งที่ยังไม่รู้ข้อมูลนั้นจะไปลงที่ไหน

เสียหายอะไร: พอถึงวันที่ 11 ก.ค. มีไฟล์ task ของบ้านครบสามหลัง (1, 2, 3) แต่ของหลัง 2 กับหลัง 3 ยังไม่ถูก import เข้า Label Studio เลยสักหลัง จุดประสงค์ทั้งหมดของการสกัดข้อมูล — เอาแบบไปวางตรงหน้าคน annotate — ยังไม่เกิดขึ้นเลยสำหรับสองในสามหลัง

แก้ยังไง: ยังไม่ได้แก้ — ยังเปิดค้างอยู่ในรายการงานค้างของบันทึกล่าสุด

กฎที่ได้: การตัดสินใจที่บล็อกขั้นตอนส่งมอบ ต้องบังคับให้เคาะก่อนจะขยายขั้นตอนการผลิต ผลงานสามหลังที่ไม่มีทีไป แยกว่าหลังเดียวที่เดินจนจบทาง

สูง 2026-07-12

งานสกัดข้อมูลหลายสัปดาห์ไม่ได้ commit ลง git เลย

เกิดอะไรขึ้น: ในบันทึกหลายวันติดกัน งานค้างประจำเขียนว่า "commit งานทั้งหมด (ยังไม่ได้ commit อะไรเลยตั้งแต่รอบ t4)" — raw JSON หัวหลังบวกกับการแก้ schema และกฎต่างๆ ค้างอยู่โดยไม่มีการติดตามใน git

ต้นเหตุจริง: การ commit ถูกมองเป็นงานแยกที่ค่อยทำทีหลัง ไม่ใช่ขั้นตอนหนึ่งในการทำงานต่อบ้านหนึ่งหลัง เลยถูกเลื่อนไปเรื่อยๆ ตามหลังงานสกัดข้อมูล

เสียหายอะไร: ไฟล์ JSON ที่ทำมือหลายร้อยไฟล์ ไม่มีประวัติเวอร์ชันและไม่มีจุดกู้คืน ถ้าเผลอเขียนทับหรือรัน regex กวาดผิด ก็กู้ไม่ได้เลย

แก้ยังไง: ยังไม่ได้แก้ในช่วงบันทึกเหล่านี้ — ยังค้างอยู่ในรายการงานค้าง

กฎที่ได้: commit ทุกครั้งที่จบบ้านหนึ่งหลัง ก่อนจะเริ่มหลังต่อไป ข้อมูลที่ทำมือและทำใหม่ไม่ได้ ถ้าไม่มี commit ก็ห่างจากการหายไปแค่สคริปต์ฟังตัวเอง

การถูกฆ่าเพราะลิมิต session ทั้งไฟล์ JSON ที่เขียนไม่จบไว้ ซึ่งการนับจำนวนไฟล์จับไม่ได้

เกิดอะไรขึ้น: ไฟล์ `หน้า54\_roof\_floor\_plan.json` ขาดวงเล็บปีกกาปิด `}` ของ object `grid\_dimensions\_as\_printed` เพราะ agent ถูกตัดหลังจากเปิด object ซ้อนแล้วแต่ยังไม่ทันปิด ไฟล์นั้นมีอยู่และจำนวนหน้าก็ตรง มีแต่การ validate ด้วย JSON.parse เติมรูปแบบเท่านั้นที่จับได้

ต้นเหตุจริง: การเช็คความครบใช้วิธี "จำนวนไฟล์เท่ากับจำนวนหน้า" แต่การเขียนที่ถูกขัดจังหวะจะได้ไฟล์ที่มีอยู่แต่ใช้ไม่ได้ ซึ่งการเช็คแบบนั้นมองไม่เห็น

เสียหายอะไร: ไฟล์ที่ใช้ไม่ได้จะทำให้สคริปต์ label-studio และทุกอย่างที่กินข้อมูลจากนั้นพัง เจอได้เพราะบังเอิญไปรัน validate เท่านั้น

แก้ยังไง: เขียนสคริปต์ node แบบใช้ครั้งเดียวเพื่อหาตำแหน่งที่ปีกกาหายจริงๆ (ตัว parser บอกแค่ว่าพังที่ EOF) แล้วใส่ `}` ลงตำแหน่งที่ถูกต้อง จากนั้น validate ใหม่ทั้ง 138 ไฟล์

กฎที่ได้: จำนวนไฟล์ไม่ใช่การเช็คความครบ ให้รัน JSON.parse กับทุกไฟล์ที่ผลิตออกมาก่อนประกาศว่าชุดนั้นเสร็จเสมอ และเวลา parser บอกว่าพังที่ EOF ให้เขียนโปรแกรมหาตำแหน่งที่วงเล็บไม่สมดุลจริงๆ แทนที่จะเดาจากข้อความ error

ใช้โซ่ระยะซ้อนกันหลายชั้นบนผังพื้นเป็นหลักฐานหลัก แทนที่จะใช้ผังบริเวณ

เกิดอะไรขึ้น: ขอบเขตอาคารถูกคิดจากโซ่ระยะที่ซ้อนกันบนผังพื้น ($0.60+0.90=1.50$) แทนที่จะดูจากแผ่น A-03 ซึ่งเป็นผังบริเวณและระยะย่นชายคาไว้ตรงๆ ว่า 0.60 ม.

ต้นเหตุจริง: ไม่มีการกำหนดลำดับความน่าเชื่อถือของหลักฐานสำหรับคำถาม "ขอบอาคารจริงอยู่ตรงไหน" คนอ่านเลยใช้แผ่นที่เปิดอยู่ตรงหน้า และโซ่ระยะบนผังพื้นก็คลุมเครือว่าตัวเลขไหนคือขอบอาคารจริง

เสียหายอะไร: เป็นเหตุโดยตรงที่ทำให้ค่าของกริด `3` ผิด และยังทำให้อ่านผิดอีกจุดบนบ้านหลัง 2 หน้า 06 ที่เอา '0.60' กับ '0.90' มาบวกกันเป็นชายคายื่นต่อเนื่อง 1.50 ม. อย่างผิดๆ

แก้ยังไง: กำหนดว่าแผ่น A-03 (ผังบริเวณ) เป็นหลักฐานหลักสำหรับขอบเขตอาคาร เวลาตรวจ dummy grid ที่ยื่นออกนอกกริดหลัก

กฎที่ได้: กำหนดลำดับความน่าเชื่อถือของหลักฐานแยกตามประเภทคำถาม สำหรับขอบเขตอาคารและระยะย่น ผังบริเวณชนะโซ่ระยะบนผังพื้น และระยะสองตัวที่ติดกันโดยใช้จุดร่วมกัน ไม่ได้แปลว่าเอามารวมกันได้โดยอัตโนมัติ

สูง 2026-07-13

dummy grid ตัวอื่นของบ้านหลัง 1 ไม่เคยถูกตรวจหาข้อบกพร่องอ้างอิงวนกลับที่ทำให้กริด 3' ผิด

เกิดอะไรขึ้น: หลังจากพบว่ากริด 3' ผิดเพราะหลักฐานที่ใช้ "ยืนยัน" มันคือคานที่มี span_source เป็น grid_table (คืออ้างอิงวนกลับไปที่ตารางกริดตัวเอง) บันทึกก็ระบุว่า A' กับ A'' มีรูปแบบการพึ่งพาแบบเดียวกัน — คาน B3X/B5X ก็ใช้ span_source: grid_table เหมือนกัน — และควรตรวจต่อ แต่การตรวจนั้นไม่เคยเกิดขึ้น

ต้นเหตุจริง: การแก้เฉพาะกรณีเดียวที่คำถามพาไปเจอ ไม่ได้ทำกับทั้งกลุ่มที่การวินิจฉัยไว้ และไม่มีการรันสคริปต์วาดหรือ grep หา span_source ทั่วทั้งบ้าน

เสียหายอะไร: ยังมี dummy grid อีกสองตัวในบ้านหลังที่เป็นแหล่งข้อมูล fine-tune ที่ยังไม่ได้ตรวจกับความผิดพลาดรูปแบบเดียวกันนี้ ซึ่งเคยทำให้ได้ค่า 8.5 ม. ผิด และช่วงคาน 1.5 ม. ผิดมาแล้ว

แก้ยังไม: ยังไม่ได้แก้ — บันทึกเป็นงานค้างข้อ 1 และไม่ได้กลับมาทำในบันทึกวันหลังๆ

กฎที่ได้: เมื่อระบุได้แล้วว่าต้นเหตุจริงเป็นรูปแบบ (field ที่คำมาจากการอนุมาน ไม่ได้มาจากการวัด) ให้ grep หาทุกที่ที่รูปแบบนั้นโผล่ทันที อย่าปิดงานที่กรณีเดียวที่มีคนแจ้งเข้ามา

สูง 2026-07-14

อ้างกฎเรื่องการขออนุญาต แล้วแก้ไฟล์ในข้อความเดียวกันโดยไม่รอคำตอบ

เกิดอะไรขึ้น: Claude บอกว่ากำลังอ้าง `rule\_of\_tune.md` ข้อ 7 ก่อนจะแก้ `primary\_rawjson\_schema.md` แล้วก็แก้ทันทีในข้อความเดียวกันเลย ผู้ใช้จับได้และถามว่าทำไมถึงอ้างกฎแล้วไม่หยุดจริง เรื่องนี้เกิดมาแล้วสองครั้ง

ต้นเหตุจริง: การอ้างกฎถูกเข้าใจว่าเท่ากับทำตามกฎแล้ว ตัวกฎบอกแค่ทำให้เรียกอะไรขึ้นมา แต่ไม่ได้บอกว่าการเรียกมันขึ้นมาแปลว่าต้องจบเทิร์นและรอคำตอบ

เสียหายอะไร: สเปก — เอกสารกลางที่สำคัญที่สุดของโปรเจกต์ — ถูกแก้สองครั้งโดยไม่ผ่านการอนุมัติที่กฎกำหนด และต้องเขียนกฎใหม่ขึ้นมาปิดช่องโหว่

แก้ยังไม: บันทึกเป็นกฎข้อ 9 ใหม่ใน `rule\_of\_tune.md`: การอ้างกฎแปลว่าต้องหยุดและถามจริงๆ ห้ามอ้างกฎแล้วแก้ต่อในข้อความเดียวกัน

กฎที่ได้: ประตูปฏิบัติการเปิดได้ด้วยคำตอบเท่านั้น ไม่ใช่ด้วยการประกาศ ถ้าเอ่ยชื่อกฎที่ต้องขออนุมัติ ให้จบเทิร์นตรงนั้นเลย

สูง 2026-07-15

คานเจ็ดตัวหายไปจากหน้า 19 จนกว่าผู้ใช้จะส่งรูปที่ชัดกว่ามาให้

เกิดอะไรขึ้น: คานเจ็ดตัว (`B2 D'2-D'3`, `B4 B1-B2`, `B4 B2-B3`, `B2 B1''-A1''`, `B2 C3'-B3'', `B3 A1''-A3'', `B3 A1''-A3'`) หายไปจากผังคานทั้งหมด จนกระทั่งผู้ใช้ส่งรูปหน้า 19 แบบเต็มที่ชัดกว่ามาให้ จำนวน element ขยับจาก 20 เป็น 27

ต้นเหตุจริง: การสกัดข้อมูลรอบแรกทำจากรูปที่ความละเอียดไม่พอกับความหนาแน่นของเส้นบนแผ่นนั้น และไม่มีการเช็คความครบที่จะสังเกตได้ว่าคานหายไปทั้งหมด

เสียหายอะไร: นับคานขาดไป 26% บนแผนโครงสร้าง — แปลว่าปริมาณงานทุกอย่างที่คิดต่อจากนั้นผิดไปหมด และการสกัดข้อมูลไม่ครบแบบเดียวกันนี้โผล่อีกครั้งแบบหนักกว่าเดิมที่บ้านหลัง 3 หน้า 31/32

แก้ยังไม: เพิ่มครบทั้ง 7 ตัวด้วย confidence 1 หลังตรวจกับรูปที่ชัดกว่า

กฎที่ได้: สกัดข้อมูลจากแบบโครงสร้างด้วยความละเอียดที่พิสูจน์แล้วว่าพอกับความหนาแน่นของเส้นบนแผ่นนั้น และให้เทียบจำนวน element กับแหล่งอิสระอื่น (ผังคานคอดิน รูปตัด หรือตาราง) ก่อนจะบอกว่าแผ่นนั้นสกัดครบแล้ว

สูง 2026-07-15

บ้านหลัง 1 ตรวจสอบวิธีที่อ่อนกว่าบ้านหลัง 2 เลยเหลือ error ที่ยังไม่เจอค้างไว้

เกิดอะไรขึ้น: ตอนตรวจบ้านหลัง 1 ใช้แค่ลำดับของ label เป็นหลัก ไม่ได้ซูมเข้าไปนับจุดหลักเสริม ซึ่งเป็นเทคนิคที่ทำให้เจอบั๊ก B2/B2X ในบ้านหลัง 2 มีการบันทึกไว้แล้วว่าบ้านหลัง 1 น่าจะยังซ่อนความผิดพลาดแบบเดียวกันอยู่

ต้นเหตุจริง: วิธีตรวจถูกพัฒนาให้ดีขึ้นระหว่างทำโปรเจกต์ แต่ไม่มีใครวางแผนย้อนกลับไปตรวจซ้ำบ้านที่ตรวจด้วยวิธีเก่าที่อ่อนกว่า

เสียหายอะไร: บ้านหลัง 1 ตรวจไม่ครบและรู้ทั้งรู้ ยังมี error เรื่องหลักเสริมที่ยังไม่เจอค้างอยู่ในชุดข้อมูลที่จะเอาไปเทรนโมเดล

แก้ยังไม: ยังไม่ได้แก้ — บันทึกไว้เป็นงานค้างว่า "ควรพิจารณากลับไปตรวจบ้านหลัง 1 ใหม่ด้วยวิธีซูมแล้วนับจุด"

กฎที่ได้: พอปรับวิธีตรวจให้ดีขึ้นเมื่อไหร่ ให้ย้อนกลับไปตรวจซ้ำทุกอย่างที่เคยตรวจด้วยวิธีเก่าทันที ไม่งั้นคุณภาพของชุดข้อมูลจะไม่เท่ากันเสียๆ ขึ้นอยู่กับว่าส่วนไหนถูกตรวจตอนไหน

สูง 2026-07-20

เพิ่มฟิลต์กลบเกลื่อนบั๊กหลักเสา แทนที่จะแก้บั๊กจริง ทำให้มันซ่อนอยู่ในอีก 6 ไฟล์

เกิดอะไรขึ้น: ตอนไล่กวาดทั้งโปรเจกต์เพื่อหาบั๊กหลักบน/ล่างของเสา พบว่ามันซ่อนอยู่ในเสารั้ว C1 อีก 6 ไฟล์ (บ้าน 1 หน้า 16/17, บ้าน 2 หน้า 20/21, บ้าน 3 หน้า 26/27) ซึ่ง "เคยถูกกลบเกลื่อนไปด้วยการเพิ่มฟิลต์ `total\_main\_bar\_count` แทนที่จะแก้ไขให้ถูกจริงๆ"

ต้นเหตุจริง: ตอนแรกที่เจอร่ายอตรวมผิด แทนที่จะแก้โครงสร้างที่ทำให้ยอดผิด กลับเพิ่มฟิลต์ใหม่มาชดเชยเพื่อบันทึกยอดที่ถูกไว้ โครงสร้างที่ผิดเลยยังอยู่และหยุดแสดงตัวว่าเป็นบั๊ก

เสียหายอะไร: บักรอดไปแบบไม่มีใครเห็นในอีก 6 ไฟล์ ซ้ำ 3 บ้าน จนกระทั่งมีการไล่กวาดทั้งโปรเจกต์แบบตั้งใจถึงเจอ

แก้ยังไม: แก้ทั้ง 6 ไฟล์ให้เหลือ `main\_bar.count` ตัวเดียว เพิ่ม `specs.CN` ที่หายไปบนหน้า 31 แล้วไล่กวาดทั้งโปรเจกต์ซ้ำเพื่อยืนยันว่าไม่เหลืออีก

กฎที่ได้: ห้ามเพิ่มฟิลต์ชดเชยมาบันทึกคำตอบที่ถูกไว้ข้างๆ โครงสร้างที่ผิดเด็ดขาด — ให้แก้โครงสร้าง ฟิลต์กลบเกลื่อนเปลี่ยนบั๊กที่มองเห็นให้กลายเป็นบั๊กที่ซ่อนตัว และการันตีว่ามันจะแพร่ต่อ

สูง 2026-07-21

วินิจฉัย OOM ผิดว่าเป็นเรื่องความยาว sequence กับ allocator เปลืองที่ แก้ไป 3 อย่างแล้วไม่เปลี่ยนอะไรเลย

เกิดอะไรขึ้น: แก้ไป 3 อย่าง (MAX_LENGTH 10240→9216, PYTORCH_ALLOC_CONF=expandable_segments=True, adamw_8bit→paged_adamw_8bit) แล้วรอบ 3 ก็ยัง crash ด้วย OOM แบบเดิมแทบไม่ต่างกัน (เหลือว่าง 98.25MB เทียบกับ 96.25MB) ต้นเหตุจริงคือ buffer ของ LoRA delta ที่อยู่ใน `forward\_native\_grouped\_mm` ซึ่งถูกคูณด้วย MoE expert 256 ตัว

ต้นเหตุจริง: เลือกวิธีแก้จากสมมติฐานที่ฟังดูเข้าเค้า แทนที่จะอ่านจุด crash จริงใน traceback การลดเพดานที่ตัวอย่างทดสอบไม่เคยไปถึงอยู่แล้ว มันช่วยอะไรไม่ได้ตั้งแต่แรกโดยหลักการ

เสียหายอะไร: เสียเวลา GPU ที่จ่ายเงินไปทั้งรอบ (fix-upload-run รอบ 3) ไปกับสิ่งที่ไม่ใช่ต้นเหตุ แถมยังทำให้เจอวิธีแก้จริงช้าลงอีก

แก้ยังไม: อ่าน traceback ที่ละบรรทัดจนเจอ `lora\_delta = torch.empty(...)` ในเส้นทางของ MoE expert แล้วหันไปจัดการที่ LoRA rank แทน

กฎที่ได้: อ่าน traceback ให้ถึงบรรทัดที่จองหน่วยความจำจริงๆ ก่อนจะไปแก้ค่าอะไรทั้งนั้น ถ้าวิธีแก้ที่คิดไว้อธิบายตัวเลขที่เห็นในเชิงกลไกไม่ได้ แปลว่ามันไม่ใช่วิธีแก้

สูง 2026-07-21

ลบ `merged/` ทิ้งก่อนที่จะสกัดส่วนที่ใช้อ่านภาพ (mmpoj) ออกมา

เกิดอะไรขึ้น: ลบโมเดลเต็มที่ merge แล้วทิ้งเพื่อเอาพื้นที่ดิสก์ ทั้งที่ยังต้องสกัดส่วน mmpoj (ตัวอ่านภาพ) ออกมาจากมันอยู่ ผลคือ GGUF ที่ได้อ่านได้แต่ข้อความ ทั้งที่โปรเจกต์นี้มีอยู่เพื่ออ่านแบบก่อสร้างโดยเฉพาะ

ต้นเหตุจริง: การลบไฟล์ถูกขัดด้วยความกดดันเรื่องพื้นที่ดิสก์เฉพาะหน้า โดยไม่มีเช็คลิสต์ว่าไฟล์แต่ละก้อนยังต้องถูกสกัดอะไรออกมาบ้างก่อนลบ

เสียหายอะไร: GGUF อ่านภาพไม่ได้เลย เส้นทางฝังวิชวลค้างไว้ไม่จบ และถ้าจะแก้ต้องสร้างโมเดล merged ขนาด 66GB ขึ้นมาใหม่

แก้ยังไม: ยังไม่ได้แก้ — บันทึกไว้เป็นปัญหาที่ยังค้าง ตัว llama.cpp เองก็เตือนว่า --mmpoj "ใช้ได้แค่กับบางโมเดล" และยังไม่ได้ตรวจสอบว่ารองรับ Qwen3.6 ไหม

กฎที่ได้: ก่อนลบไฟล์ระหว่างทางก้อนใหญ่ทุกครั้ง ให้ใส่รายการขั้นตอนปลายทางทุกชั้นที่ยังต้องใช้ไฟล์นั้น แล้วยืนยันว่าแต่ละชั้นรันไปแล้วจริง

สูง 2026-07-21

ไม่เคยไปขอ token ของ HuggingFace เลย การสำรองข้อมูลตามแผนเลยทำไม่ได้

เกิดอะไรขึ้น: แผนคืออัปโหลดไฟล์ที่ tune เสร็จขึ้น HuggingFace แต่ไม่เคยสร้าง HF token ไว้ การอัปโหลดเลยถูกเลื่อนไปเรื่อยๆ และไฟล์ก็ยังอยู่แค่บนเครื่องเซิร์ฟเวอร์ที่เครื่องถูกทำลายทิ้ง

ต้นเหตุจริง: credential ที่จำเป็นสำหรับขั้นตอนสำรอง/นำของออก ถูกทิ้งไว้ทำทีหลังสุด แทนที่จะเตรียมให้เสร็จก่อนเริ่มงานที่เสียเงิน

เสียหายอะไร: ตาข่ายนิรภัยชิ้นเดียวของงานทั้งชิ้น ใช้ไม่ได้พอดีในจังหวะที่ต้องการใช้

แก้ยังไม: ยังไม่ได้แก้ในเซสชันนี้ — ใส่ไว้เป็นงานค้าง ("อัปโหลด qwen36-tuned-Q4_K_M.gguf ขึ้น HuggingFace — ต้องมี HF token ก่อน")

กฎที่ได้: เตรียมและทดสอบ credential ทุกตัวที่เส้นทางนำของออกต้องใช้ (HF token, key ของที่เก็บไฟล์) ให้เสร็จก่อนเข้า GPU — และลองอัปโหลดไฟล์เล็กๆ ดูจริงสักครั้งเพื่อพิสูจน์ว่าเส้นทางนั้นใช้ได้

สูง 2026-07-21

รัน pipeline merge→f16→quantize GGUF ทั้งชุดก่อนที่จะเช็ค llama.cpp รองรับ vision ของโมเดลนี้หรือไม่

เกิดอะไรขึ้น: เสียเวลาหลายชั่วโมงกับ merge_lora.py, build CUDA ล้มเหลว 3 รอบ, คุ้ดิสก์เต็มซ้ำๆ, แปลง f16 (~70GB) และ quantize เป็น Q4_K_M — พอถึงตอนท้ายสุดถึงมาพบว่า GGUF ที่ได้เป็น text-only เท่านั้น และเอกสารของ llama.cpp เองก็เตือนไว้ว่า --mmpoj 'ใช้ได้แค่กับบางโมเดล' ซึ่ง Qwen3.6 ใหม่มากจนอาจจะยังไม่รองรับเลยด้วยซ้ำ

ต้นเหตุจริง: ไม่เคยตรวจสอบความเป็นไปได้ของสิ่งที่สำคัญที่สุดของงาน (อ่านแบบ = ต้องใช้ vision) ก่อนลงมือทำ pipeline ทั้งชุด การเช็คที่สำคัญที่สุดกลับถูกทิ้งไว้ท้ายสุด ทั้งที่ควรเป็นการค้นเอกสาร/issue แค่ 10 นาทีก่อนเริ่มด้วยซ้ำ

เสียหายอะไร: compute ทั้งคืน การเบียดดิสก์ และความพยายามทั้งหมดได้ artifact ขนาด 21.2GB ที่งานหลักที่โปรเจกต์นี้มีไว้ทำไม่ได้เลย

แก้ยังไม: ยังไม่แก้ — มะขามตัดสินใจหยุดและหันไปใช้วิธีรันผ่าน Python/Unsloth บน GPU เข้าแทน route GGUF

กฎที่ได้: ก่อนเริ่ม pipeline export/convert ที่ใช้เวลาหลายชั่วโมง ให้ตรวจสอบข้อกำหนดที่ 'ยากที่สุด' ก่อนเป็นอันดับแรก (ในที่นี้คือ: เครื่องมีรองรับ vision component ของสถาปัตยกรรมนี้ไหม) เรียงลำดับการเช็ค feasibility ตามความเสี่ยง ไม่ใช่ตามลำดับขั้นตอนของ pipeline

สูง 2026-07-24

checklist ก่อนเริ่มงานมีอยู่แค่ในเอกสาร แต่ไม่เคยถูกรันจริงเลย

เกิดอะไรขึ้น: การเช็ค Phase 0 ถูกเขียนไว้แต่ถูกปฏิบัติเหมือนเป็นแค่เนื้อหาให้อ่าน ไม่ใช่ขั้นตอนที่ต้องทำจริง ต้องเพิ่มกฎข้อ 4 เข้าไปโดยเฉพาะเพื่อบอกว่าการเช็คต้องทำจริง ไม่ใช่แค่มีอยู่ในเอกสาร

ต้นเหตุจริง: เอาการเขียนเอกสารมาปนกับการลงมือทำจริง การเขียนเช็คลิสต์ให้ความรู้สึกเหมือนได้ทำแล้ว และไม่มีอะไรในขั้นตอนงานบังคับให้บันทึกผล pass/fail จริงๆ ต่อรายการ

เสียหายอะไร: ปัญหาแฝงทุกอย่างที่ checklist ควรจะจับได้ (คำติสก์เริ่มต้น, บั๊ก MAX_LENGTH, ความถูกต้องของ token) รอดเข้าไปในช่วงเวลาที่เสียเงินจริงหมด

แก้ยังไง: แก้ไข rule_of_tune.md ข้อ 4 ใหม่: ก่อนเข้าเครื่องจริงทุกครั้งต้องรัน Phase 0 checks จริงๆ พร้อมเพิ่มตารางผลลัพธ์ที่มี  ต่อรายการ

กฎที่ได้: checklist ที่ไม่มีการบันทึกผลต่อรายการคือแค่ของประดับ ต้องมี artifact ยืนยัน (ผลลัพธ์, ตัวเลข, screenshot) ว่าแต่ละรายการถูกลงมือทำจริง ไม่ใช่แค่ถูกอ่าน

สูง 2026-07-24

เช็ค Phase 0 ก่อนเข้า GPU มีแค่ในเอกสาร แต่ไม่เคยถูกรันจริง

เกิดอะไรขึ้น: ต้องเพิ่ม Rule 4 ใน `rule\_of\_tune.md` เพื่อบังคับว่าก่อนเข้า GPU จริงทุกครั้งต้องรัน Phase 0 checks จริงๆ ไม่ใช่แค่มีเอกสารทิ้งไว้

ต้นเหตุจริง: เขียน checklist ไว้แต่ไม่เคยทำให้มันเป็น gate ที่บังคับผ่าน จึงกลายเป็นแค่ของอ่าน ไม่ใช่ของเช็ค — นี่คือเหตุที่บั๊ก MAX_LENGTH รอดมาได้จนถึงตอนอ่านทวนรอบ 0.4

เสียหายอะไร: มี checklist ที่เขียนไว้แต่ไม่เคยรันจริง จนกระทั่งบั๊ก MAX_LENGTH truncation กับค่า default ของ disk slider ถูกเจอก็ต่อเมื่อรัน check จริงๆ เท่านั้น

แก้ยังไง: เพิ่ม Rule 4 บังคับให้รัน Phase 0 ก่อนเข้า GPU ทุกครั้ง แล้วรันจริง 0.1 ถึง 0.5 (ทดสอบ HF token, เช็ดยอดเงิน, dry run ในเครื่องกับ mmpoj และโมเดลทดสอบจริง, อ่าน script ทวน, เลือกเครื่อง)

กฎที่ได้: checklist ที่ไม่ใช่ gate บังคับผ่านคือแค่เอกสาร ไม่ใช่การตรวจ ต้องทำให้มันสร้างหลักฐานจริง (ผล token test, log ของ dry run จริง) ก่อนที่จะเสียเงิน

สูง 2026-07-24

ปัญหา mmproj ที่ทำให้รอบวันที่ 21 พัง เป็นปัญหาที่สร้างขึ้นเอง — เพราะเลเยอร์ vision ถูกบล็อกไว้ ใช้ mmproj ของ vendor ได้เลยตั้งแต่แรก

เกิดอะไรขึ้น: รอบวันที่ 21 ถูกประกาศว่าพังเพราะดึง mmproj ไม่ได้ (เครื่องมือไม่รองรับโมเดลนี้) ทำให้ GGUF อ่านรูปไม่ได้ พอถึงวันที่ 24 ถึงรู้ว่า training script ตั้ง finetune_vision_layers=False ไว้ — ส่วนที่อ่านรูปของโมเดลที่ tune แล้วจึงเหมือนกับโมเดลต้นฉบับทุกไบต์ ดังนั้นเอา mmproj ที่ vendor ปลอมมา (899MB) มาใช้ได้เลย ไม่ต้อง extract อะไรเลย

ต้นเหตุจริง: ไม่มีใครเช็คก่อนว่า training script train อะไรจริงๆ ก่อนที่จะไปสรุปเองว่าต้องสร้าง mmproj กำหนดเอง ทีมเลยไปแก้เครื่องมือสำหรับสิ่งที่ pipeline ไม่เคยต้องการ

เสียหายอะไร: ความพยายาม extract mmproj ทั้งหมดในวันที่ 21 และปัญหา 'โมเดลอ่านรูปไม่ได้' ที่เกิดขึ้นในรอบนั้น เป็นสิ่งที่ป้องกันได้ตั้งแต่แรก ทางแก้ในวันที่ 24 คือแค่อ่านไฟล์ 899MB

แก้ยังง: ดาวน์โหลด mmproj ของ vendor มาใช้ และยืนยันด้วย dry run ในเครื่องจริง (llama.cpp อ่านข้อความไทย, grid 1/2/3 x A/B/C/D, และมาร์คฐานราก F1/C1 ได้ถูกต้องจากหน้า19 ของบ้าน01)

กฎที่ได้: ก่อนจะไล่แก้เครื่องมือแปลง/export ให้อ่าน training config ก่อนว่าส่วนที่กำลังพยายาม export นั้นถูกแก้จริงหรือเปล่า ถ้าเป็นส่วนที่บล็อกไว้ ก็เอาของ vendor มาใช้ได้เลยไม่ต้องแก้เอง

สูง 2026-07-24

เช่า GPU ครั้งแรกไม่มีการเช็คก่อนเลย — เจอบั๊กทุกตัวตอนมิเตอร์กำลังเดิน

เกิดอะไรขึ้น: การเช่าเมื่อ 2026-07-21 กระโดดจาก 'เช่า' ไป 'train' เลย ปัญหาทุกอย่างในวันนั้น (OOM, disk เต็ม, mmproj ไม่รองรับ, environment ถูกทับ, ไฟล์หาย) ถูกเจอตอนกำลังจ่ายเงินราว 30 บาท/ชั่วโมง เพิ่ง คิด checklist 'Phase 0' ก่อนเข้าขึ้นมาก็หลัง และในไดอารี่ก็บันทึกไว้ตรงๆ ว่าการมีเอกสาร checklist ไม่เหมือนกับการรันมันจริง

ต้นเหตุจริง: ไม่มีขั้นตอนที่บังคับให้ตรวจสอบฟรีในเครื่องตัวเอง (token, ยอดเงิน, ขนาด disk, dry run ในเครื่อง, อ่าน script ทวน) ก่อนจะเสียเงินจริง checklist ถูกเขียนขึ้นก็ต่อเมื่อความล้มเหลวที่เป็นต้นเหตุเกิดขึ้นแล้ว

เสียหายอะไร: การติดบักทั้งวันเกิดขึ้นตอนมิเตอร์กำลังเดิน แล้วผลงานของวันนั้น (LoRA, merged model, GGUF) ก็หายไปอยู่ดี ต้องมาทำใหม่อีกครั้งวันที่ 24/25

แก้ยังง: กลายเป็น Rule 4 ใน rule_of_tune.md และ Phase 0 ใน t01_workflow.md วันที่ 24 รัน Phase 0 จริง แล้วจับปัญหาได้ทันที 2 อย่าง (ค่า default disk 150GB กับบั๊ก MAX_LENGTH truncation) ก่อนที่จะเสียเงินสักบาท

กฎที่ได้: ทุกอย่าง queเช็คฟรีในเครื่องตัวเองได้ ต้องเช็คให้จบก่อนมิเตอร์เริ่มเดิน และกฎต้องบังคับให้ 'รัน' checklist ไม่ใช่แค่ 'มี' checklist

สูง 2026-07-24

การแก้เหล็กริมหน้า36 บ้าน5 ในบันทึก 07-19 กับ 07-24 เหมือนกันเป๊ะ — งานถูกทำซ้ำหรือไม่ก็บันทึกซ้ำ

เกิดอะไรขึ้น: บันทึกวันที่ 07-19 มีการแก้ S-08 หน้า36 ของบ้าน5: B2X top→3, ลบ B3X phantom, B4 bottom→5, B5 bottom→5 ส่วนบันทึกวันที่ 07-24 ก็มีการแก้แบบเดียวกันเป๊ะ หน้าเดียวกัน ตัวเลขเดียวกัน (main_bar.top ของ B2X 2→3, ลบ additional_bars ของ B3X ที่เป็น phantom, bottom ของ B4 2→5, bottom ของ B5 3→5)

ต้นเหตุจริง: อาจเป็นเพราะการแก้ไขหายไป (โพลเดอร์บ้าน5 ถูก 'คัดลอกจาก raw ใหม่แล้ว normalize' ในช่วงนี้ ซึ่งจะทับการแก้ไขเดิมที่เคยทำไว้) ต้องมาแก้ซ้ำ หรือไม่ก็เป็นงานเชสชันเดียวกันที่ถูกเขียนซ้ำลงบันทึกทั้งสองวัน ทั้งสองความเป็นไปได้เป็นความผิดพลาดของกระบวนการแบบเดียวกันและแก้ด้วยวิธีเดียวกัน

เสียหายอะไร: ไม่ว่าจะกรณีไหนก็เสียหาย — ถ้าวางตราชับซ้ำก็เสียเวลาทำงานที่ต้องนับคะแนนไปหนึ่งเชสชัน หรือถ้าบันทึกซ้ำ ก็ทำให้บันทึกความคืบหน้าซ้ำ ไม่ว่าแบบไหนก็เชื่อบันทึกว่าบ้านหลังนี้แก้จริงถึงไหนแล้วไม่ได้

แก้ยังไง: ยังไม่ได้แก้ — ความซ้ำนี้เห็นได้ในทั้งสองรายการแต่ไม่เคยถูกสอบสวนให้ตรงกัน

กฎที่ได้: การคัดลอกโพลเดอร์บ้านจาก raw ใหม่จะทำลายการแก้ไขที่ทำไว้แล้วในสำเนาที่แก้แล้วโดยเสียบๆ การคัดลอกจาก raw ต้องทำได้แค่ครั้งเดียวตอนเริ่มต้น และต้องมีตัวเช็คกันไม่ให้ทับซ้ำการแก้ไขที่มีอยู่แล้ว

สูง 2026-07-28

script ที่เขียนไว้นานแล้วและไม่เคยรัน ถูกเข้าใจว่าพร้อมใช้งาน

เกิดอะไรขึ้น: train_qwen3vl.py ถูกสันนิษฐานว่าใช้งานได้เพราะมันมีอยู่แล้วและถูกรีวิวแล้ว แต่จริงๆ ไม่เคยถูกรันเลยจนกระทั่งรอบ t02 ที่ต้องจ่ายเงิน

ต้นเหตุจริง: 'เขียนแล้วและรีวิวแล้ว' ถูกเข้าใจไปกับ 'รู้ว่าจะใช้ได้จริง' ทั้งที่โลกรอบข้าง (เวอร์ชัน library, โครงสร้าง repo ของโมเดล) ไม่เคยหยุดนิ่งระหว่างที่ script ถูกทิ้งไว้

เสียหายอะไร: ต้องมาติดปัญหา API เก่าที่เปลี่ยนไปตอนรันจ่ายเงินจริงบน GPU แทนที่จะเจอตอน dry run ฟรีในเครื่องตัวเอง

แก้ยังไง: แก้การเรียก API แล้วบันทึกไว้เป็นบทเรียนทั่วไปเรื่องโค้ดที่ถูกทิ้งไว้เฉยๆ

กฎที่ได้: ก่อนรันจริงที่ต้องจ่ายเงินทุกครั้ง ต้องรัน script ทุกตัวอย่างน้อยหนึ่งครั้ง — จะกับโมเดลจิวหรือจำกัดแค่ 5 step ก็ยังดี โค้ดที่ไม่ถูกแตะเสื่อมสภาพเพราะ dependency ของมันเปลี่ยนไปเรื่อยๆ

สูง 2026-07-28

บ้าน07 S-02: GB1X ถูกอ่านว่าเป็นช่วง C4→B4 เต็มช่วง ทั้งที่จริงคือแบ่งสั้น 1.60 ม. ทำให้ฐานรากถูกตีความผิดว่าหลุดกริดด้วย

เกิดอะไรขึ้น: GB1X ถูก extract ว่าพาดจาก C4 ถึง B4 (ช่วงนั้นจริงๆ เป็น GB1 อีกตัวหนึ่งต่างหาก) ของจริงคือแบ่งสั้น 1.60 ม. อยู่ใต้ B-4 ผลกระทบไล่ต่อไปคือ: ฐานราก F1 ที่ B-4 เคยถูกบันทึกว่าหลุดกริด ทั้งที่จริงมันอยู่บนกริดพอดี

ต้นเหตุจริง: ป้ายชื่อสมาชิกโครงสร้างถูกจับคู่กับช่วงที่ดูสมเหตุสมผลที่สุดที่อยู่ใกล้ๆ โดยไม่อ่านชุดตัวเลขระยะที่พิมพ์อยู่จริง และตำแหน่งกริดที่คำนวณตามมาก็ถูกเชื่อต่อไปเลยโดยไม่คำนวณใหม่จากเรขาคณิตที่แก้แล้ว

เสียหายอะไร: ความยาวช่วงที่ผิดไหลตรงเข้าปริมาณงานคานยันที่ และธงหลุดกริดปลอมๆ จะทำให้การอ้างอิงตำแหน่งกริดของฐานรานั้นเพี้ยนต่อไปในขั้นถัดไป

แก้ยังไง: แก้ไขแบบ append-or-rewrite พร้อมบันทึกการกลับคำไว้ ไม่ลบคำเตือนเดิมทิ้ง

กฎที่ได้: ห้ามเดาช่วงของสมาชิกโครงสร้างจากป้ายที่อยู่ใกล้กัน — ต้องอ่านชุดตัวเลขระยะที่พิมพ์อยู่จริง และเมื่อแก้ข้อเท็จจริงทางเรขาคณิตแล้ว ต้องคำนวณทุกอย่างที่อิงจากมันใหม่ ไม่ใช่แค่แก้ field ที่ผิด

สูง 2026-07-28

14 จาก 19 ไฟล์แปลนของบ้าน07 ต้องแก้ไข ทั้งหมดสรุปได้เป็น 3 ประเภทข้อผิดพลาดที่เกิดขึ้น ซึ่งไม่เคยอยู่ใน checklist เลย

เกิดอะไรขึ้น: มีแค่ 5 จาก 19 ไฟล์แปลนที่สะอาดจริงๆ 14 ไฟล์ที่ต้องแก้ไขสรุปได้เป็น 3 ประเภทซ้ำๆ กัน: นับสัญลักษณ์คนละชนิดปนกัน, อ่านความใกล้เคียงในแปลนเป็นการวางข้อจริง, และเส้นที่หน้าตาคล้ายสมาชิกโครงสร้างแต่จริงๆ ไม่ใช่ ไม่มีสักข้อที่อยู่ใน checklist การ extract

ต้นเหตุจริง: ข้อผิดพลาดถูกแก้เป็นรายจุดตามที่เจอ ไม่มีขั้นตอนที่สรุปข้อผิดพลาดที่เจอให้เป็นกฎแล้วเอาไปใช้กับทุกแผ่น ความผิดพลาดสามแบบเดียวกันจึงเกิดซ้ำทั้งบ้าน

เสียหายอะไร: 74% ของไฟล์แปลนบ้านหลังนี้มีข้อผิดพลาดอย่างน้อย 1 อย่าง และทั้ง 3 ประเภทนี้จะเกิดซ้ำในทุกบ้านต่อไปเรื่อยๆ จนกว่า checklist จะครอบคลุมมัน

แก้ยังไง: เขียนสรุป 3 ประเภทนี้ไว้เพื่อนำไปรวมเข้ากับ checklist Stage 3 ของ op2

กฎที่ได้: เมื่อข้อผิดพลาดเดียวกันเกิดขึ้นซ้ำครั้งที่สอง ให้หยุดแก้เป็นรายจุดแล้วเพิ่มเข้า checklist ในฐานะประเภทข้อผิดพลาด หุกรอบการ extract ควรจบด้วยการถามว่าข้อผิดพลาดที่เจอข้อไหนเป็น 'ประเภท' ไม่ใช่แค่ 'เหตุการณ์เดียว'

สูง 2026-07-28

export_gguf.py ยังเป็นสคริปต์ของ t01 เดิมทั้งต้น — ซ้ำไปที่ path ของ Qwen3.6 และ HF repo ของ t01 ซึ่ง Phase 7.5 ต้องพึ่งพา

เกิดอะไรขึ้น: สคริปต์ export ของ t02 ถูกถือปามาโดยไม่ได้อัปเดตแก้ไข ทั้ง path, repo, และช่วงขนาดโมเดลยังซ้ำไปที่ Qwen3.6 ถ้ารันตามนี้จริง อาจทับ HF repo qwen36-thai-rc ทั้ง ซึ่ง Phase 7.5 ต้องใช้ repo นี้ไปวัดผล t01 ซ้ำ นอกจากนี้สคริปต์ยังโหลดโมเดล 62GB ก่อนเช็คด้วยซ้ำว่ามี ADAPTER_DIR อยู่จริงหรือไม่

ต้นเหตุจริง: สคริปต์ถูกส่งต่อระหว่างรอบด้วยการถือป้ ไม่ใช่การทำให้พารามิเตอร์เปลี่ยนได้ ค่าเฉพาะของแต่ละรอบที่ hard-code ไว้จึงกลายเป็นภัยแฝงข้ามรอบ และไม่มีตัวกันป้องกัน repo ที่งานอื่นต้องพึ่งพาเลย

เสียหายอะไร: เกือบทำให้ adapter repo ของ t01 ที่เผยแพร่ไว้เสียหาย ซึ่งจะทำให้การวัดผล t01 ซ้ำใน Phase 7.5 เป็นไปไม่ได้เลย แถมยังเสียเวลาโหลดโมเดล 62GB ทั้งเปล่าๆ ในกรณีที่ไม่มี adapter อยู่จริงด้วย

แก้ยังไง: ผูก path/repo/ขนาดโมเดลเข้ากับ MODEL_SIZE เดียวกับตอน train เพิ่มการเช็ค ADAPTER_DIR มีอยู่จริงก่อนโหลดโมเดล และบล็อกไม่ให้ HF_REPO ที่ลงท้ายด้วย /qwen36-thai-rc ถูกเขียนทับโดยเด็ดขาด

กฎที่ได้: ให้ค่าที่เฉพาะกับแต่ละรอบเป็นพารามิเตอร์ ไม่ใช่ถือปสคริปต์มาแล้วปล่อยไว้ และต้องมีตัวกันปฏิเสธชัดเจนสำหรับปลายทางที่งานรอบอื่นต้องพึ่งพา ก่อนโหลดของหนักๆ ให้เช็คเงื่อนไขที่เช็คได้ถูกๆ ก่อนเสมอ

สูง 2026-07-29

คำเตือนตอนโหลดโมเดลที่ระบุบักตัวจริงชัดเจน ถูกพิมพ์ออกมาแต่ไม่มีใครอ่าน — เกิดขึ้นซ้ำถึงสองครั้ง

เกิดอะไรขึ้น: llama.cpp พิมพ์คำเตือนว่า 'Qwen-VL models require at minimum 1024 image tokens to function correctly ... try adding --image-min-tokens 1024' แต่ไม่มีใครอ่าน นี่เป็นครั้งที่สองแล้วที่รูปแบบเดียวกันเกิดขึ้น ก่อนหน้านี้อcollator เคยพิมพ์ 'Model does not have a default image size - using 512' ก็ถูกมองข้ามไปเช่นกัน

ต้นเหตุจริง: คำเตือนพิมพ์ปนอยู่กับข้อความตอนเริ่มต้นอีกนับร้อยบรรทัด และไม่มีขั้นตอนไหนในกระบวนการทำงานที่บังคับให้ต้องอ่านคำเตือนพวกนี้ก่อนเริ่มงานที่ใช้เวลานาน

เสียหายอะไร: บั๊กเรื่อง resolution ทั้งสองส่วนถูกรายงานเองโดยเครื่องมือ แต่ทั้งคู่ถูกมองข้ามไป จนต้องเสียรอบการเทรนทั้งรอบและเสียเวลาทำ extraction ทั้งคืน กว่าเจอต้นเหตุจากการสังเกตผล inference แทนที่จะเจอจากการอ่าน log

แก้ยังไม: ยังไม่ได้แก้ — บันทึกบทเรียนไว้แล้วแต่ยังไม่มีการบังคับตรวจสอบจริง

กฎที่ได้: ต้องทำให้การอ่านคำเตือนเป็นขั้นตอนบังคับ ไม่ใช่แค่หวังว่าจะมีคนสนใจอ่านเอง เช่น grep หา WARN/W: ใน log ตอนโหลด แล้วพิมพ์ผลลัพธ์ไว้บนสุดก่อนเริ่มงานที่ใช้เวลานานทุกครั้ง อะไรก็ตามที่เครื่องมือบอกให้ฟรีๆ คือรายงานบั๊กที่ถูกที่สุดที่จะได้รับแล้ว

สูง 2026-07-29

เขียนผล extraction ที่รู้อยู่แล้วว่าแย 6 หน้าลงโพลเดอร์ผลลัพธ์จริงก่อนตรวจสอบ

เกิดอะไรขึ้น: extract หน้าเพิ่มอีก 4 หน้า (03-06) แล้วเขียนลง tune_ai/t01/ผล/ ก่อนที่จะมีใครตรวจสอบคุณภาพ พอมาตรวจทีหลังถึงพบว่าใช้ไม่ได้ จึงต้องหยุดการรันเพิ่มเพื่อไม่ให้โพลเดอร์เต็มไปด้วยข้อมูลผิด

ต้นเหตุจริง: รันแบบ bulk production เต็มสูบไปเลยโดยไม่มี quality gate สำหรับผลลัพธ์ชุดแรก การตรวจสอบเกิดขึ้นเพราะมั่วขามาบังเอิญ ไม่ใช่เพราะกระบวนการบังคับให้ตรวจ

เสียหายอะไร: มีไฟล์ผลลัพธ์เสียปนอยู่ 6 ไฟล์ที่ต้องตามหาแล้วทั้ง ถ้าไม่หยุดรันทันทีจะมีเพิ่มอีกราว 55 ไฟล์เข้ามาปนด้วย

แก้ยังไม: แก้บางส่วน — หยุดรันเพิ่มจนกว่าจะทดสอบหาต้นเหตุเสร็จ แต่ไฟล์เสียที่มีอยู่แล้วยังคงคาอยู่ในดิสก์

กฎที่ได้: ทุกการรันแบบ bulk ต้องผ่าน gate ก่อนเสมอ — extract แค่ 1-2 รายการ diff เทียบกับ gold ก่อน แล้วค่อยปล่อยรันทั้งหมด ให้เขียนผลลัพธ์ชั่วคราวลง path กักกัน (quarantine) จนกว่าจะผ่านการตรวจ เพื่อไม่ให้ข้อมูลเสียปนกับข้อมูลดีจนแยกไม่ออก

สูง 2026-07-29

ข้อควรระวังที่เขียนไว้เองใน Phase 10 ('อย่าอ่านว่าพิสูจน์แล้วว่าเสร็จสมบูรณ์') เสื่อมกลายเป็น 'Phase 10 ผ่านแล้ว' ภายในไม่กี่วัน

เกิดอะไรขึ้น: เอกสารทดสอบ Phase 10 GGUF วันที่ 25 เขียนไว้ชัดเจนว่าอย่าอ่านผลนี้ว่าพิสูจน์ความสำเร็จสมบูรณ์แล้ว มันแค่เช็คว่ามีโมเดลผลิต JSON ออกมาได้และไม่ค้าง สัปดาห์ต่อมาทุกคนรวมถึงคนเขียนบันทึกเองจำได้แค่ 'Phase 10 ผ่านสีเขียว' แล้วเอาความจำนั้นไปใช้เป็นเหตุผลรันงานเต็มสเกล

ต้นเหตุจริง: ข้อควรระวังอยู่ในข้อความบรรยาย ไม่ได้อยู่ในสถานะเอง ป้ายผ่าน/ไม่ผ่านที่มีคำเตือนแนบมาจะสูญเสียคำเตือนทันทีที่ไม่มีใครสรุปมัน — และมันจะถูกสรุปซ้ำแล้วซ้ำเล่าโดยทุกคนและเอกสารรุ่นหลัง

เสียหายอะไร: GGUF ถูกถือว่าผ่านการตรวจสอบแล้วนานถึง 4 วัน และถูกใช้ในงาน extraction ทั้งคืนซึ่งได้ผลลัพธ์ใช้งานไม่ได้ 6 หน้า

แก้ยังไม: ยังไม่ได้แก้ในเชิงกระบวนการ — ข้อควรระวังถูกค้นพบใหม่อีกครั้งตอนย้อนกลับไปอ่านเอกสารต้นฉบับระหว่างสืบหาต้นเหตุ

กฎที่ได้: คำเตือนที่เขียนไว้ข้างเครื่องหมายถูกสีเขียวไม่ใช่คำเตือนจริง ต้องฝังข้อจำกัดไว้ในตัวสถานะเองเลย (เช่น 'Phase 10: ตรวจแค่รูปแบบ JSON เนื้อหายังไม่ได้ตรวจ') เพื่อให้มันรอดจากการถูกอ้างอิง สรุป และจดจำผิดๆ

สูง 2026-07-29

บั๊ก merge ของ MoE ถูกพิสูจน์ในโปรเจกต์ t02 ตั้งแต่วันที่ 28 แต่ไม่มีใครย้อนไปเช็ค t01 — รันสกัดข้อมูลทั้งคืนไปก่อนถึงจะมีคนถามอีกครั้ง

เกิดอะไรขึ้น: วันที่ 28 ทีมวัดผลจริงว่า merge_and_unload() merge LoRA แบบ ParamWrapper บนโมเดล MoE นี้ผิดเจี๊ยบๆ (เปลี่ยนค่าไปแค่ 0.67% จาก 201 ล้านค่า) t01 ก็สร้างด้วยฟังก์ชันเดียวกัน โมเดลตระกูลเดียวกัน ParamWrapper เดียวกัน แต่ไม่มีใครย้อนกลับไปตั้งคำถามกับ GGUF ของ t01 ที่ปล่อยออกไปแล้ว กลับรัน t01 เต็มสเกลในวันที่ 29 แล้วเพิ่งมาสงสัยตอนสืบหาสาเหตุว่าทำไมผลลัพธ์ใช้ไม่ได้

ต้นเหตุจริง: บั๊กถูกบันทึกไว้ว่าเป็น 'ปัญหาของ t02' แทนที่จะมองว่าเป็นคุณสมบัติของ toolchain เอง ไม่มีขั้นตอนไหนในกระบวนการที่ถามว่า 'ยังมี artifact ตัวไหนอีกที่สร้างจากโค้ดพาร์เดียวกันนี้' เมื่อพบต้นเหตุแล้ว

เสียหายอะไร: เสียเวลารันงานทั้งคืน (00:30-03:30) บวกกับ GPU-hours ของการ extract หน้าละราว 20 นาที 6 หน้า ไปกับการผลิตผลลัพธ์จาก artifact ที่รู้อยู่แล้วว่าน่าสงสัยตั้งแต่วันก่อนหน้า ระหว่างนั้นก็เขียนหน้าที่เสีย 6 หน้าลงไฟล์เดอร์ผลลัพธ์จริง

แก้ยังไม: ยังไม่แก้ — การทดสอบเทียบกับ base model ที่จะยืนยันเรื่องนี้ยังค้างอยู่ ณ ตอนจบ session

กฎที่ได้: เมื่อพบต้นเหตุใน artifact ตัวหนึ่ง ให้รีบไล่รายชื่อ artifact อื่นทั้งหมดที่สร้างจากโค้ดพาร์เดียวกันแล้วเปิดตรวจใหม่ทันที บั๊กใน toolchain ไม่มีวันจำกัดอยู่แค่ artifact ตัวที่บังเอิญเผยให้เห็นมันก่อน

สูง 2026-07-29

คำเตือนจาก model loader ที่ชี้ต้นเหตุทั้งสองข้อถูกพิมพ์ออกมาแล้วไม่มีใครอ่าน — เกิดซ้ำสองครั้ง

เกิดอะไรขึ้น: llama.cpp พิมพ์ข้อความ 'Qwen-VL models require at minimum 1024 image tokens to function correctly ... try adding --image-min-tokens 1024' 'ไม่มีใครอ่าน' นี่คือการที่สองที่เกิดรูปแบบเดียวกัน — ก่อนหน้านี Unslloth collator เคยพิมพ์ 'Model does not have a default image size - using 512' ก็ไม่มีใครอ่านเหมือนกัน ซึ่งคือบั๊ก resolution ที่ต่างกัน 14 เท่านั่นเอง

ต้นเหตุจริง: คำเตือนตอน startup ไหลผ่านไปพร้อมกับ log อีกหลายร้อยบรรทัดและถูกมองเป็นแค่ noise ไม่มีขั้นตอนไหนในกระบวนการบังคับให้อ่านคำเตือนก่อนเริ่มงานที่ใช้เวลานาน

เสียหายอะไร: ต้นเหตุของปัญหา resolution ทั้งสองข้ออยู่ใน log อยู่แล้วตลอดเวลา แต่กลับเสียเวลาหลายวันไปกับสมมติฐานอื่น

แก้ยังไม: ตั้งเป็นกฎแล้ว: อ่าน loader warnings ทุกครั้งก่อนเริ่มงานที่ใช้เวลานาน

กฎที่ได้: grep หา WARN/W ใน startup log ก่อนเริ่มงานที่มีค่าใช้จ่ายสูงทุกครั้ง และ pipe ผล output ของ loader ลงไฟล์ที่จะถูกอ่านจริง — คำเตือนที่ไม่มีใครอ่านก็เหมือนไม่มีคำเตือนเลย

สูง 2026-07-29

ฆ่างานที่ยังทำงานปกติอยู่ (รัน 1,256 วินาที) เพราะตั้ง timeout 1,200 วินาทีจากแค่สองหน้าที่เบาที่สุด

เกิดอะไรขึ้น: ค่า timeout 1,200 วินาทีถูกคำนวณจากเวลารันของหน้า 01-02 พอถึงหน้า 03 ต้องใช้เวลาถึง 1,256 วินาที เลยถูกตัดกลางคัน ทั้งที่ GPU utilization อยู่ที่ 100% ซึ่งพิสูจน์ว่างานยังทำงานอยู่จริง ไม่ได้ค้าง

ต้นเหตุจริง: ผิดพลาดสองอย่างพร้อมกัน: ตัวอย่างที่ใช้ตั้งค่า timeout คือหน้า 01-02 — คือหน้าปกและสารบัญ ซึ่งเป็นหน้าที่เบาที่สุดในทั้งชุด และการที่ 'ยังไม่มี output ออกมา' ถูกตีความว่า 'ค้าง' ทั้งที่โมเดลที่กำลัง generate อยู่โดยธรรมชาติจะเียบระหว่างทำงาน

เสียหายอะไร: งาน extraction จริงที่กำลังทำอยู่ถูกทำลาย และถ้าไม่ได้เช็ค GPU ก่อน คงเสียเวลาทั้งคืนไปกับการดีบั๊กอาการค้างที่ไม่มีอยู่จริง

แก้ยัง: ยืนยันว่างานยังทำงานอยู่ด้วย nvidia-smi (GPU 100% = ทำงานอยู่, 0% = ค้างจริง) ก่อนจะสรุปอะไร

กฎที่ได้: ตั้งค่า timeout โดยอิงจากเคสที่หนักที่สุดที่คาดว่าจะเจอ ไม่ใช่จากเคสแรกที่บังเอิญรัน และก่อนจะสรุปว่างานที่เียบบออยู่ตายแล้ว ให้เช็คสัญญาณที่เป็นอิสระอีกทาง (GPU utilization, ไฟล์ output ที่ยังโตขึ้นเรื่อยๆ)

สูง 2026-07-29

เบาะแสที่ชี้ต้นเหตุจริงคือ 'รูปแบบ' ของความผิดพลาด ไม่ใช่ 'จำนวน' — ถ้าดูแค่คะแนนรวมจะไม่มีทางเจอ

เกิดอะไรขึ้น: โมเดลทำได้ดีกับเนื้อหาที่ยาก (เลข TIS 8/8, ระดับชั้น 7/8, ห้อง 8/8) แต่ทำผิดพลาดกับส่วนที่ง่าย (รูปแบบ output 0/6) ความกลับด้านแบบนี้ผิดธรรมชาติของโมเดลที่ผ่านการ fine-tune มา และเป็นสิ่งที่ชี้ไปที่สมมติฐานว่า LoRA หายไป

ต้นเหตุจริง: ปฏิบัติการแรกคือมองว่าผลลัพธ์ 'คุณภาพแย่' แล้วไปหาสาเหตุจากโมเดลหรือ resolution มีแค่การไล่ดูรูปแบบความผิดพลาดแยกตามหมวดหมู่เท่านั้นที่เผยว่ารูปแบบความล้มเหลวนี้นตรงกับโมเดลที่ไม่ได้ tune ที่แค่เดินตาม prompt เฉยๆ

เสียหายอะไร: ถ้าทีมดูแค่คะแนนรวม จะไม่มีวันเจอต้นเหตุจริง แล้วจะเสียเวลาไปกับการเทรนใหม่หรือเช่า GPU เพิ่ม

แก้ยัง: วินิจฉัยด้วยการดูรูปแบบความผิดพลาด บันทึกไว้เป็นหนึ่งในหกบทเรียนของคีนั่น

กฎที่ได้: เมื่อผลลัพธ์แย่ ให้จัดหมวดหมู่ความผิดพลาดก่อนวัดเป็นตัวเลข — 'ผิดในเรื่องง่าย แต่ถูกในเรื่องยาก' คือลายเซ็นของปัญหาการ deploy/plumbing ไม่ใช่ปัญหาคุณภาพของโมเดล

สูง 2026-07-29

ขั้นตอน classify เองก็ไม่แม่นยำ — รูปด้านถูกอ่านเป็นรูปตัด และอ้างว่ามี 5 มุมมองทั้งที่มีจริงแค่ 2

เกิดอะไรขึ้น: หน้า 07/08/09 ถูกจัดประเภทเป็น `section` ทั้งที่ความจริงคือ `side\_profile` และอีกหน้าหนึ่งถูกรายงานว่ามี 5 มุมมอง ทั้งที่มีจริงแค่ 2

ต้นเหตุจริง: ขั้นตอน classify ราคาถูกของ pipeline 2 เฟส ไม่เคยถูกทดสอบความแม่นยำด้วยตัวมันเองเลย มันถูกมองแค่เป็นตัวช่วยจัดเส้นทาง ไม่ใช่ประตูที่ถ้าผิดแล้วกู้คืนไม่ได้

เสียหายอะไร: การจัดเส้นทางผิด — หน้าที่ควรถูกสกัดกลับถูกข้าม และหน้าที่ควรถูกข้ามกลับถูกสกัด ความผิดพลาดตรงนี้มองไม่เห็นในขั้นถัดไปเลย เพราะหน้าที่ถูกข้ามก็แค่ไม่มีข้อมูลออกมา

แก้ยัง: ยังไม่ได้แก้ — บันทึกไว้เป็นข้อควรระวังตอนตีความผลของคีนั่น

กฎที่ได้: ขั้นตอน routing/classification ที่ราคาถูกต้องวัดความแม่นยำของตัวมันเองเทียบกับ ground truth — ความผิดพลาดตรง routing จะลบงานออกไปเสียๆ จึงอันตรายกว่าความผิดพลาดจากการสกัดที่ยังพอเห็นร่องรอย

สูง 2026-07-29

คำเตือน 'อย่าเพิ่งเชื่อว่าพิสูจน์แล้ว' ที่เขียนไว้ ค่อยๆ กลายเป็นความจำผิดๆ ว่า 'Phase 10 ผ่านแล้ว' ในหัวทุกคน รวมถึงตัว AI เองด้วย

เกิดอะไรขึ้น: เอกสาร Phase 10 เขียนเตือนไว้ชัดเจนว่า 'อย่าอ่านว่านี่คือพิสูจน์แล้วว่าสมบูรณ์' แต่ผ่านไปหลายวัน คำเตือนนั้นถูกลืม และ Phase นั้นถูกจำง่าย ๆ คำว่า 'ผ่านแล้ว' — ทั้งโดยมะขามและโดย AI เอง ความจำผิดพลาดนี้เองที่ทำให้ GGUF ที่ยังไม่ได้ตรวจสอบถูกเอาไปใช้สกัดข้อมูลจริงในวันที่ 2026-07-29

ต้นเหตุจริง: สถานะ (ผ่านหรือไม่ผ่าน) กับเงื่อนไขข้อจำกัด (ข้อความเตือนที่แนบไว้ข้างๆ) ถูกเก็บคนละที่กันและเสื่อมสภาพในความจำเร็วไม่เท่ากัน คนและ AI จะจำแค่ป้ายสถานะ ไม่จำข้อความยาวๆ ที่แนบมา

เสียหายอะไร: สกัดข้อมูล 4 หน้าที่แย แล้วเขียนลงไฟล์เดอร์ผลลัพธ์จริง เพราะเชื่อว่า Phase ที่ใช้อยู่ผ่านการตรวจสอบแล้วจริง

แก้ยังไ: ยังไม่ได้แก้ — ยังไม่มีการเสนอกฎให้สถานะของ Phase พกข้อจำกัดของตัวเองติดไปด้วย

กฎที่ได้: คำเตือนที่เขียนเป็นข้อความข้างๆ สถานะเขียนจะถูกลืมแน่นอน ต้องเข้ารหัสข้อจำกัดไว้ในตัวสถานะเองเลย (เช่น 'Phase 10: ทดสอบแคร์นผ่าน ยังไม่ตรวจสอบเนื้อหา') อย่าฝากไว้กับประโยคที่ไม่มีใครอ่านซ้ำอีก

สูง 2026-07-30

เรื่อง 'ตรวจสอบว่า LoRA อยู่ใน GGUF ของ t01 จริงหรือไม่' ถูกลากยาวข้ามเซสชันไปเป็นครั้งที่สองโดยยังไม่ได้ทำ

เกิดอะไรขึ้น: รายการที่ถูกตั้งธงแดงจากคืนก่อน — ตรวจสอบว่า LoRA ถูกฝังเข้า GGUF ที่ export ของ t01 จริงหรือไม่ — ยังไม่ได้ทำเมื่อจบเซสชันนี้ ทั้งที่เซสชันนี้ใช้เวลาทั้งหมดไปกับงาน annotation

ต้นเหตุจริง: งาน annotation (บ้าน op1) มีเส้นชัยชัดเจนเสมอ จึงแย่งเวลาไปจากงานตรวจสอบที่ไม่มีเดดไลน์ และไม่มีกลไกบังคับให้รายการที่ถูกตั้งธงแดงต้องถูกจัดคิวทำจริง

เสียหายอะไร: artifact ของโมเดลที่ยังไม่ได้ตรวจสอบยังถูกใช้เป็น GGUF อ้างอิงของโปรเจกต์ต่อไป ถ้าการ merge ล้มเหลวแบบเงียบๆ การเปรียบเทียบทุกอย่างที่อ้างอิง t01 จะกลายเป็นการวัด base model เปล่าๆ แทน

แก้ยังไ: เซสชันนี้ยังไม่ได้แก้ — ตั้งธงค้างไว้สำหรับเซสชันถัดไป

กฎที่ได้: รายการที่ถูกตั้งธง  ต้องถูกจัดเป็นงานแรกของเซสชันถัดไป ไม่ใช่ต่อท้ายรายการ และห้ามใช้ artifact ของโมเดลที่ยังไม่ได้ตรวจสอบเป็นค่าฐานอ้างอิงเด็ดขาด

สูง 2026-08-02

schema มาตรฐานขัดแย้งกับบันทึกการแก้ไข 4 จุด — และ spec ผิดทุกจุดเลย

เกิดอะไรขึ้น: ตรวจสอบ `primary\_rawjson\_schema.md` เทียบกับ `สิ่งที่ต้องแก้.md` แล้วพบว่าขัดแย้งกัน 4 จุด (เครื่องหมายขีดของ point-ref, การตั้งชื่อ dummy grid, main_bar บน/ล่างของเสา, กฎการรวม point-element) ทั้ง 4 จุด spec เป็นฝ่ายผิดทั้งหมด — ข้อมูลเดินหน้าไปก่อนแล้ว

ต้นเหตุจริง: การตัดสินใจถูกบันทึกไว้ใน fix-log แทนที่ที่ตัดสินใจ แต่ไม่เคยถูกใส่กลับไปใน spec เลย spec จึงค่อยๆ เสื่อมกลายเป็นบรรยายข้อมูลรุ่นเก่าไปเสียๆ

เสียหายอะไร: ใครก็ตามที่ทำตาม spec (บ้านใหม่, คนใหม่, คนเขียน prompt) จะสร้างข้อมูลที่ขัดแย้งกับงานที่ทำไปแล้ว 11 บ้าน

แก้ยังไ: แก้ spec 5 จุดให้ตรงกับกฎที่ตัดสินใจไว้จริง และบันทึกทุกจุดไว้ใน `primary\_rawjson\_schema\_edit\_log.md`

กฎที่ได้: การตัดสินใจจะยังไม่ถือว่าเสร็จ จนกว่าจะถูกเขียนเข้า spec ไม่ใช่แค่ fix-log ต้องเทียบ spec กับ decision-log เป็นระยะ — spec คือเอกสารที่คนอื่นจะทำตามจริง

สูง 2026-08-02

บ้าน 06-11 ไม่เคยถูกคัดลอกเข้าโพลเดอร์ที่ตรวจแล้วเลย (571 ไฟล์)

เกิดอะไรขึ้น: `json\_แก้ไขแล้ว/` ซึ่งเป็นสำเนาที่ผ่านการตรวจแล้ว มีแค่บ้าน 01-05 บ้าน 06 ถึง 11 ไม่เคยถูกวางไว้ในนั้นเลย ทำให้ 571 ไฟล์ไม่เคยเข้าสู่ขั้นตอนการตรวจเลยสักครั้ง

ต้นเหตุจริง: ขั้นตอนการสกัดข้อมูล (op1) เขียนลงโพลเดอร์ raw ส่วนการย้ายบ้านเข้าโพลเดอร์ที่ตรวจแล้วเป็นขั้นตอนแยกที่ต้องทำมือ โดยไม่มีอะไรติดตามว่าแต่ละบ้านทำขั้นนี้แล้วหรือยัง

เสียหายอะไร: ข้อมูลที่สกัดแล้วของ 6 บ้านหลุดออกจาก pipeline ตรวจสอบ/ปรับมาตรฐานทั้งหมด และไม่ปรากฏในการตรวจคุณภาพทุกรอบที่รันบนสำเนาที่ตรวจแล้ว

แก้ง่าย: คัดลอกไฟล์ทั้ง 571 ไฟล์เข้ามาใหม่จาก raw

กฎที่ได้: ทุกขั้นตอนของ pipeline ต้องมี ledger ติดตามสถานะรายบ้าน บ้านหนึ่งหลังจะยังไม่ถือว่า 'เสร็จ' แค่ตอนสกัดข้อมูล — ต้องติดตามสถานะของมันในทุกขั้นตอนถัดไปอย่างชัดเจน

สูง 2026-08-02

การแก้รอบ 2 ที่ตัดชิดออก แต่เฉพาะไฟล์ที่*ชื่อ*มีคำว่า 'footing' — ตกไป 809 ref

เกิดอะไรขึ้น: รอบก่อนหน้าแก้ point ref ที่มีขีด (`C-1` → `C1`) แต่แก้เฉพาะไฟล์ที่ชื่อไฟล์มีคำว่า 'footing' ไฟล์ผั่งอื่นๆ ในบ้าน 06-11 ยังมีขีดอยู่หมด รวม 809 point ref ใน 87 ไฟล์ที่ต้องมาแก้กันใหม่ทีหลัง

ต้นเหตุจริง: ตอนแก้กำหนดขอบเขตด้วยชื่อไฟล์ แทนที่จะกำหนดด้วยชื่อช่องข้อมูล ปัญหาจริงมันอยู่ในช่องตระกูล `grid\_ref` ทุกที่ที่มันโผล่ ไม่ได้อยู่ในไฟล์ที่ชื่อแบบใดแบบหนึ่ง

เสียหายอะไร: งานที่รายงานว่า "แก้เสร็จแล้ว" ครอบคลุมข้อมูลที่มีปัญหาแค่เศษเสี้ยว สุดท้ายต้องรื้อทำใหม่ทั้ง corpus

แก้ง่าย: เขียน ref ใหม่ทั้ง 809 ตัวในทุกไฟล์ โดยกำหนดขอบเขตด้วยชื่อช่อง (`grid\_ref`/`grid\_ref\_start`/`grid\_ref\_end`/`grid\_refs`) แทนชื่อไฟล์

กฎที่ได้: เวลาแก้ข้อมูล ให้กำหนดขอบเขตด้วยช่องที่มันอยู่ ห้ามใช้ชื่อไฟล์ แล้วพิสูจน์ว่าแก้ครบจริงด้วยการนับจำนวนที่ยังผิดอยู่ทั้ง corpus ไม่ใช่เชื่อว่าสคริปต์รันแล้วจบ

สูง 2026-08-02

คำสั่ง 'ทำตามบ้าน 01-05' ผิด — เรื่องคำเกินใน grid ref บ้าน 01-05 คือตัวที่แย่ที่สุด

เกิดอะไรขึ้น: คำสั่งที่ยัดกันมาคือให้บ้านรุ่นหลังทำตามบ้าน 01-05 แต่เรื่องคำว่า `grid`/`dummy` ที่ซ้ำซ้อนใน grid ref บ้าน 05 ผิดอยู่ 127 จุด ส่วนบ้าน 06 ผิด 0 จุด — บ้านต้นแบบกลายเป็นตัวที่แย่ที่สุด ไม่ใช่ดีที่สุด

ต้นเหตุจริง: บ้าน 01-05 ถูกยกเป็นมาตรฐานเพราะเก่าที่สุดและถูกรีวิวมากที่สุด แต่ "เก่าที่สุด" แปลว่ามันเกิดก่อนกฎหลายข้อ และสะสมของเก่าที่ผิดกฎไว้มากที่สุดด้วย

เสียหายอะไร: ถ้าทำตามคำสั่งจริงๆ จะเป็นการเอาข้อผิดพลาดของบ้าน 01-05 ไปแพร่ใส่บ้านรุ่นใหม่ที่สะอาดกว่า

แก้ง่าย: ยึดตาม*กฎที่เขียนไว้* (ข้อ 5) ซึ่งบ้าน 01-05 เป็นคนเขียนเอง แทนที่จะยึดตาม*สภาพไฟล์*ของมัน — กฎถูกอยู่แล้ว แค่ไม่เคยถูกเอาไปใช้ให้ครบ

กฎที่ได้: ให้ยึดกฎที่เขียนไว้ ไม่ใช่ยึดสภาพของบ้านต้นแบบ ข้อมูลที่เก่าที่สุดมักปนเปื้อนของเก่ามากที่สุด อย่าปล่อยให้มันกลายเป็นมาตรฐานโดยปริยาย

งาน Label Studio ถูกยกเลิกและลบทั้งหมด — 51 ไฟล์ 22MB พร้อมขั้นตอนใน pipeline ที่สร้างรอบมันไว้

เกิดอะไรขึ้น: Label Studio ถูกยกเลิกทั้งต้น สิ่งที่ลบไป: ตัวสร้าง task, สคริปต์ import ทั้ง 2 ตัว, ไฟล์ XML ของโปรเจกต์ 3 ไฟล์, ไฟล์ task JSON ที่สร้างไว้ 21MB สำหรับ 10 บ้าน, PDF mindmap ของ flow การรีวิว, สคริปต์อัปเดตโหนดที่มีไว้สร้างไฟล์ task ของ LS อย่างเดียว และขั้นตอนใน workflow op1 ที่เรียกใช้มันทั้งหมด

ต้นเหตุจริง: สร้าง track เครื่องมือทำ annotation ทั้งชุดแล้วเอาไปรันกับ 10 บ้านก่อนที่จะตัดสินใจว่าจะใช้มันจริงหรือเปล่า คำถามซ้ำๆ ("เราต้องเก็บ annotation จากคนจริงๆ ไหม") ถูกตอบหลังจากสร้างเครื่องมือเสร็จแล้ว ไม่ใช่ก่อน

เสียหายอะไร: งานสร้างเครื่องมือหลายสัปดาห์ บวกกับขั้นตอนรัน `label-studio-tasks-makham.js` ที่ละบ้าน 10 บ้าน กลายเป็นต้นทุนจม แล้วยังต้องเขียน workflow op1 ใหม่เพื่อตัดขั้นตอนพวกนั้นออก

แก้ง่าย: ย้ายทุกอย่างไปพักไว้ใน `wait\_for\_thing/` ด้วย `git mv` ก่อน (กู้คืนได้) แล้วค่อยลบหลังยืนยัน ส่วนด้านตรวจหลังสัปดาห์ของ op1 เปลี่ยนไปใช้ `python tools/check\_format.py` แทน

กฎที่ได้: ตัดสินใจให้จบก่อนว่าต้องใช้ workflow นี้จริงไหม แล้วค่อยสร้างเครื่องมือและเอาไปรัน 10 บ้าน และเวลาจะฆ่า track ไหน ให้ย้ายด้วย `git mv` ก่อนเสมอ ห้ามใช้ `rm` — เพื่อมีของที่จัดประเภทผิดจะได้กู้กลับมาได้

ไฟล์ที่ยังใช้งานอยู่ 2 ไฟล์ถูกจัดผิดว่าเป็นของ Label Studio — ถ้าลบ manifest.json ไปสคริปต์ 2 ตัวจะพัง

เกิดอะไรขึ้น: ตอนเก็บกวาด Label Studio ไฟล์ `manifest.json` ถูกย้ายเข้าโฟลเดอร์รอลบ เพราะสคริปต์ import ของ LS เขียนลงไฟล์นี้ แต่เจ้าของจริงคือ `pdf-processor.py` กับ `qwen-processor.js` ซึ่งอ่านไฟล์นี้แต่ไม่ได้เป็นคนสร้าง ส่วน `review.html` ก็ถูกจัดผิดเหมือนกัน — มันคือ UI รีวิวแบบ local ที่มีมาก่อน LS ไม่ได้เกี่ยวกับ LS เลย

ต้นเหตุจริง: ดูความเป็นเจ้าของจาก "สคริปต์ไหนเขียนลงไฟล์นี้" แทนที่จะดูจาก "สคริปต์ไหนพึ่งพาไฟล์นี้" พอเครื่องมือที่กำลังจะลบไปแตะไฟล์นั้น เลยเหมาเอาว่าไฟล์นั้นเป็นของมัน

เสียหายอะไร: ถ้าลบ `manifest.json` ไปจริง `pdf-processor.py` กับ `qwen-processor.js` จะพังทั้งคู่ในครั้งถัดไปที่มีคนรัน — เป็นการพังเงียบๆ ที่จะโผล่มาทีหลัง รอดมาได้เพราะใช้วิธีย้ายไปพักแทนการ `rm` ตรงๆ

แก้ง่าย: กู้ทั้ง 2 ไฟล์กลับไปไว้ที่ `No\_touch\_box/` แล้วแก้การจัดประเภทของรอบ 6 ในบันทึกให้ถูกต้อง

กฎที่ได้: ก่อนลบไฟล์ ให้ grep หาทุกตัวที่*อ่าน*ไฟล์นั้น ไม่ใช่แค่ตัวที่เขียน ไฟล์ที่สคริปต์เปิดด้วย `open(..., 'r')` แต่ไม่เคยสร้างเอง คือ dependency ไม่ใช่ของที่ออกมา

สูง 2026-08-02

โพลเดอร์ทำงานที่รีวิแล้วกับโพลเดอร์ raw ที่เป็นข้อมูลจริง ไม่เคยถูก sync กันเลยตลอดประวัติโปรเจกต์

เกิดอะไรขึ้น: รอบ 4 ของวันที่ 2026-08-02 ทำสิ่งที่บันทึกเรียกเองว่า "การ sync กลับครั้งแรกในประวัติศาสตร์ของโปรเจกต์" — ดัน 362 ไฟล์เข้า rawjson ยังไม่ได้แก้ไขโดนคน/ ก่อนหน้านั้นทุกการแก้ไขทำใน json_แก้ไขแล้ว/ มีอยู่แค่ในโพลเดอร์นั้นทีเดียว ส่วนโพลเดอร์ที่ถูกกำหนดให้เป็นข้อมูลจริงยังเก็บเวอร์ชันที่ยังไม่แก้ อยู่ คำว่า "ยังไม่ได้เริ่ม sync กลับเลยสักบ้าน" โพลเป็นรายการค้างในบันทึก 3 รอบติดกันกว่าจะได้ทำจริง

ต้นเหตุจริง: มีโพลเดอร์ 2 ชุดที่ดูแลกันโดยไม่มีขั้นตอน sync ที่ชัดเจน และไม่มีตัวเช็คความทั้งสองตรงกันใหม่ workflow มีแต่การเขียนไปข้างหน้าลงโพลเดอร์ที่รีวิแล้วอย่างเดียว

เสียหายอะไร: ตลอดช่วงก่อนหน้านั้น อะไรก็ตามที่อ่านโพลเดอร์ raw — รวมถึงตัวสร้างชุดข้อมูล — อ่านข้อมูลก่อนแก้ไขทั้งหมด ทั้งที่ทีมเชื่อว่าแก้ไขแล้ว พอถึงเวลา sync จริงต้องออกค่าเตือน Rule 2 เติมรูปแบบและขออนุญาตอย่างชัดเจน เพราะมันไปแตะโพลเดอร์ที่ถูกป้องกันไว้เป็นจำนวนมาก

แก้ยังไง: sync 362 ไฟล์เข้าโพลเดอร์ raw ภายใต้การอนุญาตชัดเจน บันทึกไว้ใน raw_json_data_log.md และ No_touch_box/CLAUDE.md ผูกกับ commit 0029264 แล้วเอาการเช็ค "สองโพลเดอร์ต่างกัน 0 ไฟล์" เข้าเป็นส่วนหนึ่งของการปิดงาน

กฎที่ได้: ถ้ามีข้อมูลชุดเดียวกันอยู่ 2 ที่ ต้องมีขั้นตอน sync อยู่ใน workflow และมีตัวเช็คอัตโนมัติว่าเท่ากัน ไม่งั้นอีกฝั่งจะกลายเป็นของเก่าเจี๊ยบๆ และให้เอาการยืนยันว่า "สองโพลเดอร์ต่างกัน 0 ไฟล์" ไปไว้ในด้านตรวจ ไม่ใช่ในสรุปปิดงาน

สูง 2026-08-05

บ้าน 13 หายไป 10 หน้า แต่เพิ่งจับได้ตอน agent เบื่องหลังทั้ง 7 ตัวทำงานเสร็จหมดแล้ว

เกิดอะไรขึ้น: ตัวเช็คความครบของหน้าที่ตัวประสานงานเพิ่งพบว่าไฟล์หายไป 10 ไฟล์ (หน้า01/02/28-33 และการแก้ไข grid_ref บนหน้า34) ของบ้าน 13 หลังจาก agent เบื่องหลังทั้ง 7 ตัวส่งงานกลับมาหมดแล้ว ต้องรันตามอีกรอบเพื่อทำบ้านนี้ให้ครบ

ต้นเหตุจริง: ความครบของหน้าถูกเช็คตอนเป็นด่านสุดท้าย แทนที่จะเช็คตอนแจกงาน ไม่มีอะไรตรวจว่าช่วงหน้าที่แจกให้ agent ทุกตัวรวมกันแล้วครบทั้ง 92 หน้าหรือยังก่อนส่งเริ่ม

เสียหายอะไร: บ้านหลังหนึ่งถูกรายงานว่าสัปดาห์เสร็จแล้วทั้งที่ขาดไป 10 หน้า ต้องแจกงานรอบสอง และถ้าไม่ได้เช็ค ช่องโหว่นี้จะเข้า corpus ไปแบบเจี๊ยบๆ

แก้ยังไง: สกัดไฟล์ที่หายทั้ง 10 ในการ commit ตามอีกรอบจนบ้านครบ พร้อมบันทึกว่าตัวเช็คความครบควรรันให้เร็วกว่านี้

กฎที่ได้: ให้ตรวจความครบของหน้าตอนแจกงาน ไม่ใช่ตอนงานเสร็จ — ยืนยันว่าช่วงหน้าที่แจกไปทั้งหมดรวมกันแล้วเท่ากับจำนวนหน้าทั้งหมด ก่อนที่ agent ตัวแรกจะเริ่มทำงาน

สูง 2026-08-07

บ้าน 08 กับ 09 เซ็นรับว่า '100% เสร็จ' เมื่อ 07-30 แต่อีกสัปดาห์ต่อมาต้องมาแก้ทั้งคู่

เกิดอะไรขึ้น: บ้าน 08 ที่ประกาศเมื่อ 2026-07-30 ว่า '100% เสร็จ 63/63 หน้า ไฟล์ JSON ทั้ง 97 ไฟล์ถูกต้อง' ต้องแก้ 4 ไฟล์เมื่อ 08-07 (เส้นกริด, ผังคาน, ผังคานรัตรอบ, การขยายคาน-เสา) ส่วนบ้าน 09 ที่ประกาศ 100% เหมือนกัน ต้องแก้ 3 ไฟล์เมื่อ 08-08

ต้นเหตุจริง: คำว่า "เสร็จ" ถูกนิยามว่า "สัปดาห์ครบทุกหน้าและตัวเช็คฟอร์แมตผ่าน" ซึ่งวัดแค่ความครบของหน้าและความถูกต้องของรูปทรงข้อมูล ไม่ได้วัดความถูกต้องของเนื้อหา ไม่มีอะไรในเกณฑ์ปิดงานที่เอาค่าที่สัปดาห์ไปเทียบกับแบบจริง

เสียหายอะไร: บ้าน 2 หลังถูกบันทึกว่าเสร็จทั้งที่มีเนื้อหาผิดอยู่ และแปลว่าสถานะ "เสร็จ" ของบ้านอื่นๆ ก็เชื่อไม่ได้ด้วยเหตุผลเดียวกัน การแก้ครั้งนี้เกิดขึ้นเพราะบังเอิญมีคนกลับไปดูอีกรอบเท่านั้น

แก้ยังไง: ทำรอบแก้ไขไฟล์ของทั้งสองบ้าน

กฎที่ได้: แยก "สัปดาห์ครบแล้ว" กับ "ตรวจเนื้อหาแล้ว" ให้เป็นสองสถานะที่มีการตรวจคนละชุด ตัวเช็คฟอร์แมตผ่านไม่ใช้การรีวิวเนื้อหา อย่าปล่อยให้ ALL CHECKS PASS ถูกอ่านว่า "บ้านหลังนี้ถูกต้องแล้ว"

สูง 2026-08-18

branch King-try ตามหลัง main อยู่ 277 commit แกรมพค schema สำเนาที่เก่าไปหลายสิบกฎ

เกิดอะไรขึ้น: ตอนรีวิว branch King-try ของ kingparadorn พบว่ามันแยกออกมาตั้งแต่ 2026-07-25 และตามหลัง main อยู่ 277 commit — คือเกิดก่อนแคมเปญ op01/op03 ทั้ง 22 บ้าน และก่อนการรื้อ schema \$0 FORMAT LOCK ทั้งชุด สำเนา primary_rawjson_schema.md ของมันเองก็อยู่ใต้โฟลเดอร์เก่า raw_json_ตัวที่ใช้งานจริง/ แบบ 5 บ้าน และเก่าไปหลายสิบกฎ

ต้นเหตุจริง: branch พิเจอร์ที่อยู่ยาวไม่เคย rebase หรือ sync เลย ในขณะที่สเปกที่มันพึ่งพากำลังถูกเขียนใหม่บน main และเพราะ branch พกสำเนาเอกสาร schema ของตัวเองแทนที่จะอ้างอิงไปที่ทีเดียว การเบี่ยงเบนเลยมองไม่เห็นจากทั้งสองฝั่งจนกว่าจะมีคนเอามา diff กัน

เสียหายอะไร: การ merge จริงกลายเป็นเรื่องทำไม่ได้ งานที่มีประโยชน์จริง 12 commit (ต้นแบบ overlay ของ Gemini, hooks, skills, agent) เลยค้างเต้ง และจนถึง 2026-08-20 ก็ยังไม่ได้ cherry-pick ออกมา เครื่องมืออะไรที่เขียนบน branch นั้นก็เล็งไปที่รูปทรง schema ที่ไม่มีอยู่แล้ว

แก้ยังไง: ยังไม่แก้ บันทึกคำแนะนำไว้ว่าให้ cherry-pick ต้นแบบ overlay พร้อม hooks/skills/agent ขึ้น main แทนที่จะพยายาม merge ซึ่งยังค้างอยู่ ณ 2026-08-20 ('เซสชันนี้ไม่ได้ cherry-pick อะไรขึ้น main เลย')

กฎที่ได้: sync branch พิเจอร์กับ main อย่างน้อยสัปดาห์ละครึ่ง และห้ามให้ branch พกสำเนาสเปกที่ใช้ร่วมกันของตัวเอง — ให้อ้างอิงไปที่ไฟล์กลางไฟล์เดียว เวลาสเปกเปลี่ยนจะได้เกิด conflict ที่มองเห็นได้ แทนที่จะเบี่ยงเบนกันเสียๆ

สูง 2026-08-20

สร้างด่านรับ/ปฏิเสธแบบ checklist ตัวหนังสือ หั้งที่โ้จห้ยค็ือกอดที่กรอบเชียวบนแบบ

เกิดอะไรขึ้น: สร้างด่านรับ/ปฏิเสธเต็มรูปแบบเข้าไปในเส้นทางนำเข้า raw JSON ของ Constaint — เป็น checklist ของทุก element พร้อม มาร์ค/ประเภท/ชั้น/ตำแหน่งกริด/ความมั่นใจ บวกกับการแยก qt_importRawExtractionFiles() ออกเป็นชั้น preview กับ save ค่าตัดสินของมะขาม: ยังไม่ดีพอ โ้จห้ยค็ือกอดที่กรอบเชียวที่วาดทับบนผ้ง ไม่ใช่รายการตัวหนังสือ ตัวที่เป็นภาพจริงๆ ยังต้องสร้างอยู่

ต้นเหตุจริง: ติความโ้จห้ย 'กอดเพื่อรับ element' เป็นปัญหาการร็ิวข้อมูล (แถวในตาราง) แทนที่จะเป็นปัญหาการร็ิวเชิงตำแหน่ง (มาร์คบนแบบ) และไม่ได้ทวนโ้จห้ยกลับไปยืนยันก่อนลงมือ ความไม่ตรงกันเลย์โ้ลตอนเดโม

เสียหายอะไร: เสียงานทั้งเซสชัน บวกกับ UI ที่แก้แล้วยังไม่ commit ซึ่งไม่ตอบโ้จห้ย ฟิเจอร์จริง (SVG ที่กอดได้วางทับกริด วาดจาก GridReference ที่บันทึกไว้) ยังไม่ได้สร้าง ส่วนการแยก preview/save ที่ทำให้ยังเอาไปใช้ต่อได้

แก้ยังง: เซสชันนี้ยังไม่แก้ — บันทึกว่าเวอร์ชัน overlay ที่เป็นภาพยังค้างอยู่ ส่วนการแยก qt_buildRawExtractionPreview / qt_saveRawExtractionPreview เก็บไว้เป็นฐานที่ใช้ต่อได้

กฎที่ได้: ก่อนสร้าง UI ร็ิวอะไรก็ตาม ให้ทวนโ้จห้ยกลับเป็นประโยคเดียวที่ระบุว่าจะกอดตรงไหน ('คุณกอดที่กล่องเชียวบนแบบสแกน') แล้วขอคำยืนยัน — โ้จห้ยเชิงตำแหน่งที่ตอบด้วยตาราง แปลว่าต้องร็ือสร้างใหม่ทั้งหมด ไม่ใช่แค่ปรับนิดหน่อย

สูง 2026-08-20

ขอบเขตงานหดจาก 4 บ้านเหลือ 19 หน้า เพราะไม่เคยวัดเวลาสกดต่อหน้าไว้ก่อนเริ่มเลย

เกิดอะไรขึ้น: เวลาต่อหน้าจริงอยู่ที่ 1.5–15 นาที ผันผวนมาก แยกว่ามากบนหน้าที่โมเดลไม่เคยเห็นเพราะชนเพดาน token ขอบเขตถูกตัดสลดหลายรอบ: 4 บ้าน → 2 บ้าน → บ้าน 08 บ้านเดียว → จำกัดที่ 25 จาก 63 หน้า → จบที่ 19/25 หน้า บ้าน 09/10/11 ไม่เคยเริ่มเลย

ต้นเหตุจริง: แผนภูยวางจากค่าประมาณเวลาต่อหน้าที่ไม่เคยวัดจริง ตัวการที่ทำให้ผันผวน (หน้าที่โมเดลไม่เคยเห็นวนซ้ำจนชน max_new_tokens) เองก็คือบักวนซ้ำ (repetition-loop) ที่ยังไม่ถูกวินัจฉัย ทำให้หน้าที่ซ้ำที่สุดซ้ำด้วยเหตุผลที่ยังไม่มีใครหาเจอ

เสียหายอะไร: การทดสอบ generalization บน 4 บ้านลดลงเหลือแค่ 19 หน้าของบ้านเดียว ตัวเลข recall ที่ได้ (11%) จึงตั้งอยู่บนตัวอย่างที่บางกว่าที่วางแผนไว้มาก และบ้าน 09/10/11 ยังไม่เริ่มแม้ผ่านไประหนึ่งวันแล้ว

แก้ยังง: ตัดขอบเขตซ้ำๆ ระหว่างการรัน PAGE_FILTER ได้โหมดใหม่ให้ list เป็นลำดับที่กำหนดเองเพื่อจัดลำดับใหม่กลางคัน บ้าน 09/10/11 บันทึกไว้ว่ายังไม่เริ่ม

กฎที่ได้: จับเวลาสองหน้าที่เป็นตัวแทนจริง (ง่ายหนึ่งหน้า ยากที่สุดหนึ่งหน้า) บนเครื่องที่เข้ามาก่อนตัดสินใจกำหนดขอบเขต และมองเวลาต่อหน้าที่ผันผวนมากว่าเป็นสัญญาณของบักที่ต้องสืบสวน ไม่ใช่แค่ปัญหาเรื่องบเวลา

สูง 2026-08-21

แก้ schema เสียบบๆ สองครั้งในวันเดียว ละเมิดกฎบทที่ #7 ของ rule_of_tune ที่ต้องประกาศและบันทึกก่อน

เกิดอะไรขึ้น: หลังอ่าน rule_of_tune.md ตามคำสั่งของมะขาม พบว่ากฎบทที่ #7 ถูกละเมิดมาทั้งวัน:

primary_rawjson_schema.md ได้รับการยกเว้นจากกฎข้อ 1 (เพราะเป็นไฟล์ .md) แต่กฎบทที่ #7 กำหนดให้ต้องพูดออกมาถึงข้อยกเว้นนี้ก่อนแก้ไข และบันทึกการแก้ไขทุกครั้งลง primary_rawjson_schema_edit_log.md ทันที การแก้ schema สองครั้งก่อนหน้านี้ในวันเดียวกันไม่มีทั้งสองอย่างเลย

ต้นเหตุจริง: ไม่ได้อ่านไฟล์กฎกำกับตั้งแต่เริ่มเซสชัน จึงไม่รู้ขั้นตอนการแก้ไขเอกสารที่สำคัญที่สุดในโปรเจกต์ระหว่างที่กำลังแก้มันอยู่ สังเกตเห็นข้อยกเว้นจากกฎข้อ 1 แต่ไม่ได้สังเกตว่ามีข้อผูกพันอื่นแนบมากับข้อยกเว้นนั้นด้วย

เสียหายอะไร: การแก้ spec หลักของโปรเจกต์สองครั้งเกิดขึ้นโดยไม่มีร่องรอยการตรวจสอบและไม่มีการเตือนคนเลย ซึ่งเป็นสถานการณ์ที่ edit log มีไว้เพื่อป้องกันพอดี กู้คืนย้อนหลังได้ แต่ช่องโหว่นั้นมีอยู่จริง

แก้ยังไง: บันทึกการแก้ไขทั้งสองครั้งย้อนหลัง และระบุไว้ในบันทึกเองว่าเป็นการละเมิดกฎ ส่วนการแก้ 2 ครั้งถัดมา ได้ประกาศเตือนตามกฎข้อ 2 ก่อนแก้ และบันทึกทันทีตามที่ควรทำ

กฎที่ได้: อ่าน rule_of_tune.md ทุกครั้งที่เริ่มเซสชันที่จะแตะไฟล์ spec และเมื่อได้รับการยกเว้นจากกฎข้อหนึ่ง ให้ถือเป็นสัญญาณให้ไปอ่านว่ามีข้อผูกพันอื่นแนบมาด้วยหรือไม่ อย่าถือว่าเป็นใบอนุญาตครอบจักรวาล

สูง 2026-08-21

โพลเดอร์สองอันชื่อต่างกันแค่ตัวอักษรเดียว เก็บดีไซน์ที่ขัดกันเอง โดยไม่มีอะไรบอกว่าอันไหนคือของจริง

เกิดอะไรขึ้น: ใน tune_ai/t03 มีทั้ง prompt/ (ดีไซน์ 2-pass ฉบับร่างจาก 2026-08-04) และ prompts/ (งาน 4-pass ของ 2026-08-21) อยู่ด้วยกัน ไม่มีอะไรบอกเลยว่าอันไหนคือตัวปัจจุบัน สุดท้ายต้องมานั่งจัดใหม่หลังเที่ยงคืน แยกเป็นโพลเดอร์ละ pass แล้วเปลี่ยนชื่อ prompt/ เป็น _old_2026-08-04_two_pass/ พร้อมติดแบนเนอร์ 🚫

ต้นเหตุจริง: ดีไซน์ที่ถูกแทนที่แล้วถูกทิ้งไว้ตรงนั้น ใช้ชื่อที่ต่างจากตัวใหม่แค่ตัวอักษรเดียว และไม่มีป้ายบอกว่าเลิกใช้แล้ว คือไม่ได้มองว่าชื่อโพลเดอร์เป็นเรื่องสำคัญ

เสียหายอะไร: มีโอกาสจริงที่จะเปลืองเอา prompt แบบ 2-pass ที่เลิกใช้แล้วไปรันกับโครงสร้างโพลเดอร์แบบ 4-pass และยังทำให้ข้อความเรื่องช่องโหว่ของสเปกซ้กันอยู่ใน README สองไฟล์ ซึ่งเดี๋ยวก็จะเพี้ยนออกจากกัน

แก้ยังไง: แยกเป็นโพลเดอร์ละ pass (pass0_classify/, pass1_organize/, pass2_used/, pass3_unused/) เปลี่ยน prompt/ เป็น _old_2026-08-04_two_pass/ พร้อมแบนเนอร์ 🚫 ย้าย _common.md ขึ้นมาไว้ที่ราก t03 ลบ prompts/README.md แล้วยกส่วนที่มีเฉพาะในนั้นไปรวมใน t03/README.md ไม่ได้แก้เนื้อ prompt เลย ย้ายกับแก้ลิงก์อย่างเดียว

กฎที่ได้: พอดีไซน์ไหนถูกแทนที่ ให้ติดป้ายทันทีในวันานั้น — เปลี่ยนชื่อให้มีวันที่นำหน้า แล้วใส่แบนเนอร์ 🚫 ใน README ของมัน และห้ามปล่อยให้มีดีไซน์สองตัวที่แยกกันได้แค่ตัว s ต่อท้าย

สูง 2026-08-21

แก้บั๊กสำคัญใน adapter สองตัวพร้อมเทสต์ แล้วปล่อยค้างไว้ไม่ commit ใน repo Constant

เกิดอะไรขึ้น: การแก้ parseGridMaster เรื่อง grid ซ้อน กับแก้ parseGridRef เรื่องไม่มีขีดกลาง ใน js/drawing/raw-extraction-adapter.js พร้อม regression test ใหม่ 4 ตัวใน tests/raw-extraction-grid.test.js ถูกทิ้งค้างไว้ไม่ commit ตอนจบเซสชัน รวมถึงการแก้ UI checklist ของวันที่ 08-20 ใน raw-extraction-import.js ก็ค้างเหมือนกัน

ต้นเหตุจริง: เซสชันจบโดยไม่มีขั้นตอน commit และไฟล์พวกนี้แต่ละ pure module ที่มีตัวกัน purity อยู่ ซึ่งคนเขียนอยากให้มีคนรีวิวก่อน — แต่ รอรีวิว ถูกจดไว้เป็นงานค้าง แทนที่จะ commit ลงกันไปเลย

เสียหายอะไร: การแก้บั๊กเสียเปรียบแรงสองตัว (ที่ทำให้ทุกการ import ได้ grid วางและ span เป็น null มาตั้งแต่ 2026-08-02) มีอยู่ใน working tree เดียวเท่านั้น ถ้า working tree หาย สลับกิ่ง หรือ stash ก็จะไปพร้อมเทสต์ ประวัติของ repo นี้เองก็เคยบันทึกว่าแผนงานกับ mockup ที่อนุมัติแล้วหายไปด้วยวิธีนี้มาแล้ว

แก้ยังไง: ยังไม่ได้แก้ บันทึกเป็นงานค้างว่า การแก้ adapter สองตัวยังไม่ commit ใน repo Constant — มันแต่ละ pure module ที่มีตัวกัน purity ให้รีวิวแล้ว commit

กฎที่ได้: ห้ามจบเซสชันโดยทิ้งการแก้บั๊กค้างไว้แบบไม่ commit — commit ลงกึ่งแล้วขอรีวิวตรงนั้น รอรีวิว คือเหตุผลที่ต้อง commit ไม่ใช่เหตุผลที่จะทิ้งงานไว้ใน working tree

สูง 2026-08-24

วินิจฉัยผิด: คิดว่า xgrammar จะกัน generation ค้างได้ สุดท้ายหน้า 25 ก็ยังค้างอยู่ดี

เกิดอะไรขึ้น: หลัง xgrammar แก้หน้า 22 ได้ ก็เข้าใจไปเองว่าปัญหาค้างจบแล้ว พอมาหน้า 25 ก็ค้างอีกทั้งที่ใส่ grammar อยู่ เสียเวลาเข้าเครื่องไปอีกก่อนจะเข้าใจว่ามันคนละเรื่องกัน

ต้นเหตุจริง: สืบสนระหว่างความพึงสองแบบ grammar คุมว่า token ที่ออกมา ถูกต้อง ไหม (JSON ไม่ยอมปิด) แต่มันไม่คุม ความยาว ของผลลัพธ์ (สร้างยาวเกินไป) และในตัวรันก็ไม่มีอะไรตั้งเพดานเวลาจริงไว้เลย

เสียหายอะไร: ค้างอีกรอบเป็นหลายสิบนาทีบนเวลา GPU ที่จ่ายเงิน และสุดท้ายมะขามตัดทุกหน้าที่ t01 ทำไม่จบทั้ง ทำให้ชุดเปรียบเทียบลดจาก 34 → 22 → 12 → เหลือ 6 หน้าที่ใช้ได้

แก้ยังไง: เพิ่ม PAGE_TIMEOUT_S=25 นาที เป็น StoppingCriteria แยกออกจาก grammar

กฎที่ได้: grammar แก่ JSON เสียรูป ไม่ได้แก้ความยาวที่พุ่งไม่หยุด ทุกๆ generation ต้องมีเงื่อนไขหยุดตามเวลาจริงเป็นของตัวเอง ไม่ว่าจะเปะ constraint แบบไหนไว้ก็ตาม

สูง 2026-08-25

เทรน t03 ทั้งรอบเพื่อแก้ plan_beam แต่ผลยับ 0% → 0%

เกิดอะไรขึ้น: รอบเทรนที่ 3 (t03) จัดขึ้นเพื่อแก้ element recall ของ plan_beam โดยเฉพาะ ผลคือ schedule ดีขึ้น (0→21.9%) plan_slab ดีขึ้น (0→20%) แต่ plan_beam ยังอยู่ที่ 0% เป๊ะ เสียไปราว \$5 กับ 5 ชั่วโมง แลกกับตัวเลขเดียวที่ตั้งใจจะแก้ก็ไม่ขยับเลย

ต้นเหตุจริง: เปิดรอบเทรนโดยยังไม่ได้วินิจฉัยก่อนว่าทำไม plan_beam ถึงเป็น 0% อาการจริงคือ 'โมเดลไม่เห็นคนเลย' (elements array ว่าง) ไม่ใช่ 'โมเดลอ่านตัวเลขผิด' การเทรนซ้ำบนรูปที่ถูกย่อขนาดเฉิบๆ ชุดเดิม แก้ปัญหาเรื่องการมองเห็นไม่ได้อยู่แล้ว

เสียหายอะไร: เสียค่า GPU ราว \$5 กับเวลาเทรนจริง 5 ชั่วโมง แล้วเป้าหมายหลักของรอบไม่ขยับเลย ทีมเพิ่งมารู้สาเหตุทีหลัง

แก้ยังไม: วินิจฉัยย้อนหลังแล้ว: recall 0% แปลว่าโมเดลไม่เคยพ่น element ออกมาเลย บวกกับรูปถูกย่อขนาด 100% (คนละประเด็นที่เจอเพิ่ม) เจอต้นเหตุทั้งสองในวันเดียวกัน แต่ยังไม่ได้เทรนใหม่

กฎที่ได้: อย่าเปิดรอบเทรนที่ต้องจ่ายเงินเพื่อแก้ตัวเลขใด จนกว่าจะมีคำวินิจฉัยเป็นลายลักษณ์อักษรว่าตัวเลขนั้นแย่เพราะอะไร วินิจฉัยก่อน จ่ายค่า GPU ทีหลัง

สูง 2026-08-25

เริ่มลงมือแก้ section ที่ได้ 0% ทั้งที่ยังไม่ได้วินิจฉัยว่าทำไมถึง 0%

เกิดอะไรขึ้น: พอถูกถามว่า pattern recognition จะช่วย subtask section ได้ไหม ก็เริ่มลงมือเอาเครื่องมือไปใช้ทันที ทำไปได้ครึ่งทางถึงจับได้ว่าตัวเองทำผิดลำดับ เลยถอยกลับมาวินิจฉัยก่อน แล้วคำตอบก็เปลี่ยนไปคนละเรื่อง

ต้นเหตุจริง: คำถามที่ถามว่า 'เครื่องมือ X ช่วยปัญหา Y ได้ไหม' ถูกตีความเป็นคำสั่งให้เอา X ไปใช้ ข้ามขั้นตอนที่ต้องหาว่าก่อนว่า Y คืออะไรกันแน่

เสียหายอะไร: เสียแรงไปบางส่วน แต่จับได้ก่อนจะกลายเป็นการ implement เต็มรูปแบบ ถ้าไม่ทัน ก็จะทุ่มแรงลงไปกับเครื่องมือที่พิสูจน์ได้ว่าช่วยตัวเลขนี้ไม่ได้เลย

แก้ยังไม: ถอยกลับมาเขียน measure_id_ceiling.py เพื่อวินิจฉัยตัวเลขแทน แล้วพบว่าปัญหาลงจริงอยู่ที่วิธีวัด ไม่ใช่ที่โมเดล ไม่ใช่คำวินิจฉัย

กฎที่ได้: เวลาถูกถามว่า 'ใช้เครื่องมือ X ตรงนี้ได้ไหม' ให้ตอบคำถามวินิจฉัยก่อนว่าจริงๆ แล้วอะไรพัง ค่าขอให้ลองเครื่องมือ ไม่ใช่คำวินิจฉัย

สูง 2026-08-25

เริ่มเขียนตัวแก้ปัญหา section 0% ทั้งที่ยังไม่ได้วินิจฉัยก่อน — พอวินิจฉัยแล้วคำตอบเปลี่ยนไปคนละเรื่อง

เกิดอะไรขึ้น: มีคำถามว่า pattern recognition จะช่วยคู่คำ id-recall 0% ของ section ได้ไหม เริ่มลงมือลองเครื่องมือก่อนเลย แล้วค่อยหยุดกลับไปวินิจฉัย พอวินิจฉัยจริงถึงพบว่าต้นเหตุคือตัว metric ที่ใช้วัด ไม่ใช่ตัวโมเดล — เครื่องมือที่กำลังสร้างจึงไม่มีทางช่วยอะไรได้เลยตั้งแต่แรก

ต้นเหตุจริง: รีบกระโดดไปหาทางแก้ที่มีอยู่แล้วในมือ แทนที่จะหาให้ชัดก่อนว่าตัวเลขที่ฟังนั้นจริงๆ วัดอะไรอยู่

เสียหายอะไร: เสียเวลาไปกับ 3 การทดลอง pattern-recognition บน section (HoughCircles, contour box detection, นับจุดเล็ก) ที่ไม่มีทางแก้ต้นเหตุจริงได้เลย แต่จับได้ทันก่อนจะ commit รอบ tuning เต็มรูปแบบ

แก้ยังไม: ย้อนกลับไปวินิจฉัยก่อน สร้าง `measure\_id\_ceiling.py` เพื่อคำนวณเพดานที่ทำได้จริงของแต่ละ subtask ก่อนจะเสนอทางแก้ไขใดๆ

กฎที่ได้: วินิจฉัยก่อนลงมือสร้างเสมอ โดยเฉพาะตอนที่เครื่องมือที่เพิ่งทำเสร็จอยู่ในมือแล้วดูเหมือนจะเข้ากับปัญหาพอดี ถ้ายังไม่ได้ว่าตัวเลขที่ฟังวัดอะไร แล้วค่าสูงสุดตามทฤษฎีของมันคือเท่าไร แสดงว่ายังไม่พร้อมจะแก้

สูง 2026-08-25

คำแนะนำ max_frames=768 จากอาจารย์ชี้ไปที่ปุ่มปรับที่โปรเจกต์นี้ไม่เคยเรียกใช้เลย

เกิดอะไรขึ้น: เจอค่า `max_frames: 768` ใน `processor_config.json` แล้วเสนอเป็นเป้าหมายที่จะปรับจน แต่มันคือค่า default ที่ตัวสุมเฟรมวิดีโอของ Qwen ฝังมาเอง โปรเจกต์นี้ส่งทุกหน้าแบบก่อสร้างเป็นภาพนิ่งตัวเดียว ไม่เคยส่งเป็นวิดีโอเลย เส้นทางประมวลผลวิดีโอจึงไม่เคยถูกเรียกใช้ การขยับค่านี้จะไม่มีผลอะไรเลย

ต้นเหตุจริง: ระบุค่า config โดยจับคู่ชื่อในไฟล์ config เฉวๆ โดยไม่ได้ตามสอบว่าโค้ดส่วนที่อ่านค่านี้ถูกเรียกใช้งานจริงหรือเปล่า

เสียหายอะไร: ถ้าไม่จับได้ก่อน จะเสียรอบ tuning ทั้งรอบไปกับการเปลี่ยนค่าที่ไม่มีผลอะไรเลย แต่การตามสอบครั้งนี้กลับเจอปุ่มปรับตัวจริง (MAX_PIXELS, MAX_LENGTH) และนำไปสู่การค้นพบว่าภาพถูกลดขนาดลง 100%

แก้ยังไง: ทำ grep ทั้ง repo หาคำว่า `max_frames` (เจอจุดเดียว เป็น config ที่ระบบเซพอัตโนมัติ) ยืนยันว่าโค้ด training/inference ส่งข้อมูลแบบ `{"type": "image"}` แล้วรายงานกลับว่าปุ่มปรับตัวจริงคือ MAX_PIXELS กับ MAX_LENGTH

กฎที่ได้: ก่อนจะปรับค่า config ตัวไหน ต้องตามรอยโค้ดที่อ่านค่านั้นและพิสูจน์ให้ได้ว่ามันถูกรันจริงในระบบของเรา การที่ค่าหนึ่งปรากฏอยู่ในไฟล์ config ที่เซพอัตโนมัติ ไม่ได้แปลว่าโค้ดของเราใช้ค่านั้นจริง

สูง 2026-08-25

งานทั้งวันทั้งเครื่องมือ, config, เอกสาร และ metric ค้างไม่ได้ commit ตอนจบวัน

เกิดอะไรขึ้น: ตอนปิดวัน รายการที่ยังไม่ commit มี `measure_id_ceiling.py`, `infer_house_t03.py` (metric ตัวใหม่), `measure_capacity.py`, `train_t03.py` (7680/47104), เอกสารสอน .md + .pdf + โฟลเดอร์รูปภาพ, `md_to_book_pdf.py` (ตัวแก้บักรูปภาพ), `t03_workflow.md` และบันทึกประจำวันของวันนั้น มีแค่ 2 commit แรกของวัน (pattern recognition) ที่มะขาม commit ไปแล้ว

ต้นเหตุจริง: รอให้มะขามรีวิวก่อน commit ทำให้งานที่เกี่ยวข้องกันทั้งวันกองรวมอยู่ใน working tree เป็นก้อนเดียวที่ยังไม่ได้รีวิว

เสียหายอะไร: ถ้าเครื่องแครช, มีคำสั่ง `git checkout` พลาด หรือสลับ branch งานทั้งวันจะหายหมด ความเสี่ยงแบบนี้เคยเกิดขึ้นจริงแล้วใน repo Constant — แผนงานและ mockup ที่อนุมัติแล้วหายไปเพราะไม่เคยถูก commit เลย นอกจากนี้ยังทำให้รีวิวหรือ bisect ยากขึ้นเพราะมาเป็นก้อนเดียว

แก้ยังไง: ยังไม่ได้แก้ — บันทึกรายการที่ยังไม่ commit ไว้ในไดอารี่ เพื่อไม่ให้หลุดจากการติดตามไปเสียๆ

กฎที่ได้: commit งานที่เสร็จเป็นชิ้นๆ ทันทีที่ทำเสร็จ จะทำบน branch แยกไว้รอรีวิวก็ได้ "รอรีวิว" ไม่ใช่เหตุผลที่จะปล่อยงานค้างอยู่แค่ใน working tree — commit ที่ยังไม่รีวิวยังกู้คืนได้ แต่ไฟล์ที่ไม่ได้ commit กู้คืนไม่ได้

สูง 2026-08-25

ขั้นตอนจำแนกหน้าแบบของ pass0 — ขั้นที่เสี่ยงที่สุดในทั้ง pipeline — ไม่มีการตรวจสอบไขว้เลยแม้แต่จุดเดียว

เกิดอะไรขึ้น: มะขามเสนอ Pass 0.5 (สแกน CV แบบ template ทุกหน้าเพื่อสร้าง fingerprint ของหน้านั้น) เพราะป้ายกำกับหน้าของ pass0 คือ "ด้านเสี่ยงสุด เดิมไม่มีใครตรวจ" — ขั้นตอนนี้ทุกอย่างข้างล่างพึ่งพายุ่กลับไม่มีการตรวจสอบเลยแม้แต่จุดเดียว หน้าที่ถูกติดป้ายผิดจะถูกส่งไปยัง prompt สกัดข้อมูลผิดตัวโดยไม่มีใครรู้ และเนื้อหาของมันก็หายไปเงียบๆ

ต้นเหตุจริง: แรงในการตรวจสอบทั้งหมดถูกทุ่มไปที่ผลลัพธ์การสกัดข้อมูล (จำนวนชิ้นส่วน, id, recall) แต่ไม่มีเลยที่การตัดสินใจจัดเส้นทางก่อนหน้านั้น ทำให้ความผิดพลาดทั้งคลาสนี้โผล่ขึ้นมาได้แค่ในรูปข้อมูลที่หายไปโดยอธิบายไม่ได้ในภายหลังเท่านั้น

เสียหายอะไร: ทุกหน้าที่ pass0 ติดป้ายผิดมาจนถึงตอนนี้ไม่มีใครจับได้เลย — เป็นปัญหารูปแบบเดียวกับบัก roof_plan→plan ที่หน้าที่ติดแท็กผิดทำให้ปริมาณงาน BOQ ของงานหลังคาหายไปโดยไม่มี error ใดๆ ขึ้นเลย

แก้ยังไง: สร้าง tools/cv_scan.py ขึ้นในเย็นวันนั้นเป็นตัวตรวจสอบไขว้ของ Pass 0.5 แต่ยังไม่ได้อัปเข้ากับ pipeline จริง ("ต่อ cv_scan เข้าสายพาน: เริ่มจาก pass0 cross-check" ยังค้างอยู่)

กฎที่ได้: ต้องตรวจสอบขั้นตอนจัดเส้นทาง ไม่ใช่แค่ขั้นตอนผลลัพธ์ ตัวจำแนกที่ไม่มีการตรวจสอบทำให้ error ทุกอย่างข้างล่างกลายเป็นข้อมูลที่หายไปโดยอธิบายไม่ได้

สูง 2026-08-26

plan_beam recall 0% เคยถูกเข้าใจผิดว่าเป็นเพราะลดขนาดภาพ — ต้นเหตุจริงคือความยาวคำตอบ

เกิดอะไรขึ้น: เอกสารสอนงานบันทึกการสืบสวนกรณี plan_beam recall 0% ไว้ พบว่าต้นเหตุจริงคือความยาวของคำตอบทำให้ token ตัวเดียวพังแล้วทั้ง JSON ลมตาม ไม่ใช่เรื่องการลดขนาดภาพอย่างที่คิด ทฤษฎีเรื่องลดขนาดภาพถูกพิสูจน์ว่าผิดด้วยการเทียบภาพ before/after จากรูปจริงในชุดเทรน

ต้นเหตุจริง: ความละเอียดภาพเป็นตัวต้องสงสัยที่เข้าใจง่าย (และเคยเป็นบั๊กจริงใน t01 ตอนใช้ 512px) เลยถูกสันนิษฐานซ้ำโดยไม่ได้อัดสอบ กลไกจริง — คำตอบยังยาวยังมีโอกาสที่ token ตัวใดตัวหนึ่งจะพัง JSON — เจอได้จากการสืบสวนจริงเท่านั้น ไม่ใช่การเดา

เสียหายอะไร: เสียเวลาสืบสวนไปกับสมมติฐานผิด ถ้าไม่แก้ทัน จะไปแก้ผิดทาง คือเพิ่ม VRAM เพื่อรองรับความละเอียดสูงขึ้น แทนที่จะทำให้คำตอบสั้นลง

แก้ยังไง: พิสูจน์ด้วยการเทียบภาพ crop จริงจากรูปในชุดเทรน แล้วบันทึกไว้ในเอกสารสอนงาน ทิศทางแก้ที่แนะนำคือแตก plan_beam เป็นคำตอบย่อยตามโซนกริดแต่ละส่วน

กฎที่ได้: สาเหตุที่เคยจริงในรอบก่อน คือสมมติฐานผิดที่ล่อใจที่สุดในรอบถัดไป ต้องพิสูจน์ทั้งด้วยการวัดจริงก่อนลงมือแก้

สูง 2026-08-28

เปลี่ยนเลขโพลเดอร์บ้าน 22 หลังทำให้ทุกการอ้างอิงในล็อกเก่าพังและเลขไม่ตรงกับต้นไม้ rawjson_

เกิดอะไรขึ้น: 22 โพลเดอร์ใน json_ แก่ไขแล้ว ถูก git-mv เปลี่ยนเลขใหม่เป็น 01–38 (01–16 ไม่เปลี่ยน) เดิมช่องว่างของเลขคือบ้าน 12 หลังที่ยังไม่มีใครตรวจทาน ไม่ใช่ชื่อที่พัง หลังเปลี่ยนเลขแล้ว เลขจากลำดับ 17 ขึ้นไปไม่ตรงกับต้นไม้ `rawjson\_` อีกต่อไป

ต้นเหตุจริง: ช่องว่างของเลขถูกเข้าใจผิดว่าเป็นปัญหาการตั้งชื่อที่ต้องแก้ ทั้งที่จริงเป็นสัญญาณที่มีความหมาย (บ้านที่ยังไม่ถูกตรวจ) การเปลี่ยนเลขทำให้ความเชื่อมโยงระหว่างต้นไม้สองต้นพัง และทำให้การอ้างอิงตัวเลขทุกจุดในล็อก/ไดอารีก่อนหน้านั้นกลายเป็นข้อมูลกำกวม

เสียหายอะไร: ทุกรายการล็อกก่อนหน้านี้ที่อ้างเลขบ้านกลายเป็นข้อมูลกำกวมทันที วิธีเดียวที่จะอ่านมันได้คือใช้ตารางแมป เลขเก่า→เลขใหม่ ตอนนี้นี้ต้นไม้สองต้นใช้เลขคนละชุดสำหรับบ้านเดียวกัน

แก้ยังไง: ตารางแมป เลขเก่า→เลขใหม่ อยู่หัวไฟล์ json_ แก่ไขแล้ว/README.md ทำเครื่องหมาย ห้ามลบ ไว้ — เป็นทางเดียวที่จะอ่านล็อกเก่าได้ กติกาการคือ ให้อ้างอิงบ้านด้วยชื่อแทนเลขเวลาไม่แน่ใจ

กฎที่ได้: ห้ามเปลี่ยนเลข ID ที่ถูกอ้างอิงในบันทึกประวัติศาสตร์เด็ดขาด ให้ใช้ชื่อคงที่เป็นตัวยืนยันตัวตนและมองเลขเป็นแค่ของโชว์ ถ้าจำเป็นต้องเปลี่ยนเลขจริงๆ ตารางแมปจะกลายเป็นเอกสารการที่ต้องรักษาไว้ตลอดไป

สูง 2026-08-29

การก๊อปปีไฟล์สแตจที่ไม่เปลี่ยนระหว่าง t03 กับ t04 เคยทำให้สำเนาซ้ำเพี้ยนไปคนละทางแบบเงียบๆ

เกิดอะไรขึ้น: โปรเจกต์เคยเจอปัญหา 'drift' มาก่อน คือไฟล์เดียวกันอยู่สองที่ มีคนแก้สำเนาหนึ่ง แต่อีกสำเนาค้างไม่อัปเดตโดยไม่มีใครสังเกต สำหรับ t04 ทีมจึงตั้งใจไม่ก๊อปปีไฟล์ pass 0/1/2 ที่ไม่เปลี่ยน และวาง README ตัวชี้ทางแทน

ต้นเหตุจริง: การก๊อปปีไฟล์ทำให้เกิดแหล่งความจริงสองแหล่งโดยไม่มีกลไกให้ซิงก์กัน การเพี้ยนจะเกิดขึ้นเงียบๆ เพราะสำเนาทั้งสองยังคงถูกต้องตามไวยากรณ์เหมือนเดิม

เสียหายอะไร: เดิมเคยทำให้มีคนใช้สำเนาที่ค้างข้อมูลเก่าโดยไม่รู้ตัว ค่าใช้จ่ายนี้มองไม่เห็นเลยจนกว่าพฤติกรรมผิดพลาดจะโผล่ออกมาในขั้นตอนถัดไปไกลๆ

แก้ยังไง: ยึดกติกาว่าไฟล์จริงอยู่ที่เดียวเท่านั้น ทุกที่อื่นเก็บแค่ README ตัวชี้ทาง

กฎที่ได้: ห้ามก๊อปปีไฟล์ซ้ำข้ามรอบเทรนหรือข้ามโพลเดอร์เด็ดขาด ใช้แหล่งความจริงเดียว มีตัวชี้ทางหลายที่ — ถ้าเสี่ยงการมีสำเนาไม่ได้จริงๆ ต้องมีตัวตรวจอัตโนมัติที่แจ้งเตือนเมื่อสำเนาเริ่มเพี้ยนต่างกัน

สูง 2026-08-29

jsonl ถูกสร้างใหม่อย่างน้อย 5 ครั้งในวันเดียว มีความเสี่ยงจริงว่าจะเทรนกับชุดข้อมูลเก่าที่ค้างอยู่บนเครื่องเช่า

เกิดอะไรขึ้น: train/val.jsonl ถูกสร้างใหม่ซ้ำๆ ตลอดวัน (ปรับความสะอาด prompt, ย้าย glossary, เปลี่ยน split, กฎ span ใหม่, บ้าน 51/52, บ้าน 08) ทุกครั้งต้องเขียนคำเตือนแบบเดิมซ้ำในไดอารี่ว่า ให้อัปโหลดชุดล่าสุดก่อนสั่งรัน GPU อย่าใช้ชุดเก่าที่ค้างอยู่บนเครื่องเช่า

ต้นเหตุจริง: ชุดข้อมูลไม่มีการประทับ version/hash และไม่มีการตรวจสอบบนเครื่องเทรนว่าไฟล์ jsonl ที่อัปโหลดตรงกับที่ build ในเครื่องเองหรือไม่ ทางกันเดียวที่มีคือให้คนจำเอาว่าต้องอัปโหลดใหม่ — และคำเตือนนี้ต้องเขียนซ้ำถึง 4 ครั้งในวันเดียว ซึ่งนั่นแหละคือสัญญาณ

เสียหายอะไร: ถ้าลืมอัปโหลดแค่ครั้งเดียว จะเสียค่าเช่า GPU เต็มจำนวนกับการเทรนบนข้อมูลผิด และมักจะรู้ตัวหลังเกิดเหตุแล้วเท่านั้น

แก้ยังไง: ยังไม่ได้แก้ — บรรเทาด้วยการเขียนคำเตือนซ้ำในไดอารี่และ RUNBOOK

กฎที่ได้: ต้องประทับ hash เนื้อหาของชุดข้อมูล และให้สคริปต์เทรนตรวจสอบว่า hash ตรงกับที่คาดไว้ก่อนเริ่ม step 0 คำเตือนที่ต้องเขียนซ้ำถึง 4 ครั้งในวันเดียว คือสัญญาณว่าขาดตัวตรวจสอบอัตโนมัติ

สูง 2026-08-29

t04 รันมาหลายวันแล้วโดยไม่มีเอกสาร Phase 0 ตามที่ rule_of_tune §4 กำหนดไว้

เกิดอะไรขึ้น: เอกสาร Phase 0 ของ t04_workflow.md เพิ่งถูกสร้างตอนที่เปลี่ยนโมเดลเป็น InternVL3 เท่านั้น พร้อมบันทึกว่ารอบนี้ไม่เคยมี Phase 0 มาก่อนเลย

ต้นเหตุจริง: งาน t04/Purson เริ่มต้นเป็นการต่อยอดจาก t03 แทนที่จะประกาศเป็นรอบใหม่อย่างเป็นทางการ เช็คลิสต์เปิดรอบจึงไม่เคยถูกกระตุ้นให้ทำ

เสียหายอะไร: หลายวันของการออกแบบ pass-v2, เขียน prompt ใหม่, งานเหมืองเพลต CV และการสร้างชุดข้อมูลใหม่ เกิดขึ้นโดยไม่มีการบันทึกการตัดสินใจเรื่องโมเดล/เฟรมเวิร์ก/GPU ของรอบนี้เลย — ซึ่งเป็นเหตุผลตรงๆ ว่าทำไมการเปลี่ยนโมเดลถึงทำให้ toolchain ที่ยังขยายอยู่เมื่อวานพังไปทั้งชุด

แก้ยังไง: เขียน t04_Purson/t04_workflow.md ที่มีรายการช่องว่าง Phase 0 (เฟรมเวิร์ก, กลยุทธ์ GPU, ตัวแปลงชุดข้อมูล, สคริปต์เทรน/infer ใหม่, การทดสอบ xgrammar)

กฎที่ได้: รอบที่ได้ชื่อโพลเดอร์ใหม่ คือรอบใหม่ ต้องเปิดเอกสาร Phase 0 ก่อนเขียนโค้ดใดๆ ของรอบนั้น การตัดสินใจที่มันบังคับ (ใช้โมเดลไหน เฟรมเวิร์กไหน GPU แบบไหน) คือสิ่งที่ทำให้งานที่ทำไปแล้วเสียเปล่าย้อนหลังได้

สูง 2026-08-29

InternVL3-78B ไม่พอดีกับการดจอ 96GB ใบเดียวที่ workflow เซาเครื่องทั้งหมดถูกออกแบบไว้รองรับ

เกิดอะไรขึ้น: นอกจากปัญหา Unsloth ใช้ไม่ได้แล้ว การเปลี่ยนโมเดลยังทำลายข้อสันนิษฐานเรื่องฮาร์ดแวร์ด้วย — LoRA แบบ bf16 บนโมเดล dense 78B ต้องใช้การ์ดระดับ H100 อย่างน้อย 2 ใบ หรือไม่ต้องใช้เส้นทาง QLoRA 4-bit การ์ดเดียวที่ยังไม่เคยลองจริง

ต้นเหตุจริง: โมเดลถูกเลือกจากความสามารถ (คะแนน OmniDocBench, license) โดยผลกระทบด้าน VRAM ถูกค้นพบทีหลัง ไม่ได้เป็นส่วนหนึ่งของการตัดสินใจตั้งแต่แรก

เสียหายอะไร: สคริปต์เซาเครื่องทุกตัว, onstart, การประมาณราคา และโมเดลต้นทุน \$1/ชม. การ์ดเดียวทั้งหมดใช้ไม่ได้อีกต่อไป รอบนี้ตอนนี้ต้องใช้ฮาร์ดแวร์เพิ่มเป็นสองเท่า หรือใช้เส้นทาง quantized ที่ยังไม่เคยพิสูจน์ ทั้งสองทางไม่มึงบตั้งไว้

แก้ยังไม: ยังไม่ได้แก้ — บันทึกไว้ใน t04_workflow.md Phase 0 เป็นการตัดสินใจกลยุทธ์ GPU ที่ยังค้างอยู่

กฎที่ได้: การเลือกโมเดลคือการตัดสินใจด้านฮาร์ดแวร์ ต้องคำนวณความต้องการ VRAM และค่าเช่ารายชั่วโมงที่ตามมาเป็นส่วนหนึ่งของการเลือกโมเดลตั้งแต่แรก ไม่ใช่ค้นพบทีหลังตอนเขียนเอกสาร workflow

สูง 2026-08-30

ตัวกัน `PAGE\_TIMEOUT\_S` 25 นาที ไม่สามารถตัดคำขอที่ค้างได้เลยแบบเงิบๆ

เกิดอะไรขึ้น: `PAGE\_TIMEOUT\_S`=25 นาที มีไว้เพื่อฆ่างานที่หน้าทีค้างโดยเฉพาะ แต่ระหว่างเซิร์ฟเวอร์ค้าง 7 ชั่วโมง มันไม่เคยทำงานเลยสักครั้ง — คำขอไม่เคยถูกตัดจบและไม่มี error timeout ขึ้นมาเลย

ต้นเหตุจริง: สงสัย (ยังไม่ยืนยัน) ว่าการเรียก `generate()` แบบ blocking ทำให้ event loop ไม่มีทางกลับมาตรวจสอบ deadline ได้เลย — timeout ที่ทำงานอยู่ฝั่ง async ไม่สามารถแทรกตัด synchronous code ที่ยึด loop อยู่ได้

เสียหายอะไร: กลไกเดียวที่ออกแบบมาเพื่อจำกัดความเสียหายจากคำขอที่ฟังไม่ทำงานอะไรเลย ทำให้กรณีเลวร้ายสุดที่ควรเป็น 25 นาที กลายเป็นเซิร์ฟเวอร์ล่ม 7 ชั่วโมงและเสียค่าเช่า GPU ไปฟรี

แก้ยังไม: ยังไม่ได้แก้ อยู่ในรายการสิ่งที่ต้องทำก่อนพร็เซนต์ ข้อ 2: ต้องสืบสวนว่าทำไม timeout ไม่ทำงานนั้น

กฎที่ได้: timeout ที่อยู่ใน execution context เดียวกับสิ่งที่มันควรจะขัดจังหวะ ไม่ใช่ timeout จริง ต้องทดสอบตัวกันความปลอดภัยทุกตัวด้วยการจำลองความล้มเหลวที่มันควรป้องกันจริงๆ อย่าสมมติว่ามันทำงานเพราะแค่มีโค้ดอยู่

สูง 2026-08-30

Blocklist SKIP_SUBTASKS ที่เขียนมือพลาด 3 subtask ที่มี prompt แต่ไม่มี training example เลย — worker เกือบยิง prompt ที่โมเดลไม่เคยเห็น

เกิดอะไรขึ้น: มะขามจับได้ว่า 'ระบบ pass ผิด' ลิสต์ SKIP_SUBTASKS ถูกเขียนจากความจำแทนที่จะเช็คกับ `pass\_io\_table\_detailed.csv` ตัวจริง จึงพลาด 3 subtask ที่มี prompt แต่ไม่มี training example เลยสักตัว คือ `plan\_column` (ตายแล้ว), `material\_list` (prompt ไม่เคยถูกโหลดเข้า training) และ `soil\_boring\_log` (ไม่มี ground truth เลย)

ต้นเหตุจริง: blocklist ถูกเขียนจากการเดาว่าอะไรควรตัดออก แทนที่จะดึงมาจาก training manifest ที่เป็นแหล่งความจริง blocklist มีข้อเสียคือ 'fail open' — อะไรที่ลืมใส่จะถูกรันทันที ซึ่งเป็นทิศทางที่ผิดที่สุดสำหรับกรณีนี้

เสียหายอะไร: ถ้าไม่จับได้ทัน ระบบจะยิง prompt ที่โมเดลไม่เคยเห็นมาเลยเข้า production ได้ผลลัพธ์ที่ดูดีแต่มั่วโดยไม่มี error เตือน

แก้ยังไง: เปลี่ยนเป็น allowlist `TRAINED\_SUBTASKS` ที่มีแค่ 7 subtask ที่เทรนจริง ดึงมาจากตาราง training จริง ตอนนีถ้ามั่วเพิ่มตัวไหนจะแค่ข้ามมันไป ไม่ใช่รันมั่ว

กฎที่ได้: เวลาความเสี่ยงของ false positive คือการได้ผลลัพธ์มั่วแบบเสียๆ ให้ใช้ allowlist ที่ดึงจากไฟล์ต้นฉบับที่เป็นความจริง ห้ามใช้ blocklist ที่เขียนมือเด็ดขาด ต้อง fail closed เสมอ

สูง 2026-08-30

adapter 78B ที่ทั้งรอบจ่ายเงินไปเทรน กลับเอามาเสิร์ฟใช้งานไม่ได้เลย — เทรนด้วย LLaMA-Factory แต่ server ที่ใช้งานได้จริงโหลดมา Unisloth เท่านั้น --model t04` จึงถูกปฏิเสธทันที

เกิดอะไรขึ้น: ตัดสินใจไม่ใช่ vLLM เพราะ layout ของ MoE LoRA จาก Unisloth ไม่ตรงกับฟอร์แมตที่ vLLM รองรับทั้งสองแบบ จึงเขียน `serve\_purson.py` โดยยืมวิธี load/generate ของ Unisloth มาจาก `infer\_house\_t03.py` แทน server ตัวนั้นโหลดได้แค่ adapter ที่เทรนด้วย Unisloth เท่านั้น ดังนั้น `--model t04` (InternVL3-78B ที่เทรนด้วย LLaMA-Factory) จึงถูกปฏิเสธทันทีพร้อมข้อความว่าต้อง port ตัว loader จาก `infer\_house\_t04.py` มาก่อน ผลลัพธ์ทั้งรอบจึงไม่มีทางใช้งานได้เลย

ต้นเหตุจริง: framework ที่ใช้เทรนกับ framework ที่ใช้เสิร์ฟถูกเลือกแยกกันคนละเวลา — LLaMA-Factory ถูกเลือกตอนเทรน เพราะเป็นทางเดียวที่ทำ QLoRA บน InternVL3-78B ได้ ส่วน Unisloth ถูกเลือกตอนเสิร์ฟเพราะเป็น path เดียวที่พิสูจน์แล้วว่าโหลดได้จริง ไม่มีใครเช็คตอนตัดสินใจว่าทั้งสองตัวจะเข้ากันได้หรือเปล่า

เสียหายอะไร: ผลงานของรอบ 78B ใช้งานไม่ได้เลยถ้าไม่มีงาน port loader เพิ่มที่ยังไม่ได้ตั้งบไว้ ระบบปริเซนต์ต้องถอยกลับไปใช้ adapter t03 รุนเก่าแทน

แก้ยังไง: ยังไม่ได้แก้ `--model t04` fail ทันทีพร้อมข้อความชัดเจนแทนที่จะปล่อยให้พังกกลางการสาธิต — เป็นการเลือกโดยตั้งใจ

กฎที่ได้: ต้องเลือก serving path พร้อมกับตอนเลือก training framework และพิสูจน์ว่าฝั่งหนึ่งโหลดผลลัพธ์ของอีกฝั่งได้จริงก่อนจะจ่ายเงินเทรน — 'เทรนเสร็จแล้ว' กับ 'รันได้จริง' เป็นคนละคำกล่าวอ้างกันเลย

สูง 2026-08-31

แก้ต้นเหตุที่ยืนยันได้ตัวหนึ่งแล้วเข้าใจผิดว่าปลดล็อกทั้งรอบแล้ว

เกิดอะไรขึ้น: หลังจาก Lesson 15 ระบุว่า crop_to_patches คือต้นเหตุของ t04 และตรวจสอบแล้วว่าถูกต้อง ทีมงานก็สมมติว่าปัญหาเรื่องนี้แก้แล้ว แต่ยังมีอีกโรคหนึ่งซ่อนอยู่ข้างใต้ (การใช้ 4-bit ไปฆ่า ViT) ซึ่งจะให้อาการเหมือนเดิมเป๊ะ คือ JSON ที่สวยงามแต่ recall ~0%

ต้นเหตุจริง: ปฏิบัติต่อ 'เจอต้นเหตุแล้วแก้แล้ว' ว่าเป็นจุดจบ แทนที่จะไล่กลับไปดูรายการความเสี่ยงที่เหลือซึ่งเอกสารออกแบบของรอบนั้นเขียนไว้อยู่แล้ว

เสียหายอะไร: ถ้าไม่มีด่านคัดกรองราคา ~\$2 มาจับได้ทัน จะเผาเวลาข้ามคืนไปทั้งคืนกับหลาย GPU ไปกับโมเดลที่มองไม่เห็นอะไรเลยอีกรอบ

แก้ยังไง: บันทึกไว้เป็น rule_of_tune Lesson 16 ข้อ 3: หลังจากแก้ต้นเหตุแล้ว ให้ไล่กลับไปดูรายการความเสี่ยงที่เหลือในเอกสารออกแบบของรอบนั้น และสร้างด่านตรวจสอบสำหรับทุกข้อที่เหลือ

กฎที่ได้: การแก้ต้นเหตุที่ยืนยันแล้วไม่ใช่ไฟเขียว — ต้องอ่านรายการความเสี่ยงของรอบนั้นซ้ำ แล้วตั้งด่านตรวจสอบราคาทุกข้อที่เหลืออยู่ ก่อนจะเสียเงินเพิ่ม

สูง 2026-08-31

เหลือ headroom ของความยาว sequence แค่ 2.8% และตัวกันไม่ให้ label ถูกตัดทิ้งแบบเงิบๆ มีแค่คอมเมนต์ไม่มีการตรวจสอบจริง

เกิดอะไรขึ้น: sequence ที่ยาวที่สุดที่วัดได้คือ 45,839 เทียบกับ MAX_LENGTH 47,104 — เหลือ margin แค่ 2.8% ต่ำกว่า 5% ที่ควรมี การตรวจสอบก่อนหน้านี้เคยพบแล้วว่าบั๊กประเภทเดียวกับ t01 (label ถูกตัดกลาง JSON แบบเงิบๆ) มีแค่คอมเมนต์บอกไว้แต่ไม่มีการตรวจสอบตอนรัน

ต้นเหตุจริง: ชัดจำกัดถูกคำนวณบนกระดานและบันทึกไว้เป็นความตั้งใจเท่านั้น แทนที่จะบังคับด้วย assertion ก่อนโหลดข้อมูลจริง

เสียหายอะไร: แถวข้อมูลที่ยาวกว่านี้เพียงเล็กน้อยจะถูกตัด label กลาง JSON แบบเงิบๆ ทันที กลายเป็นการสอนโมเดลให้ตอบผิดรูปแบบโดยไม่มี error เตือนเลย

แก้ยังไง: เพิ่มฟังก์ชัน preflight_seq_lengths() เข้าไปใน training script วัด visual token จากขนาดรูปจริงบวก text token จาก tokenizer จริงสำหรับทุกแถวตอนโหลดข้อมูล และหยุดทันทีถ้ามีแถวไหนเกินชัดจำกัด

กฎที่ได้: ชัดจำกัดที่บันทึกไว้แต่ไม่มี assertion ตรวจสอบตอนรัน ไม่ถือว่าเป็นตัวกันจริง — ต้องบังคับให้ทุกข้อจำกัดเรื่อง capacity มี hard fail ตอนโหลดข้อมูล ก่อนขั้นตอนแรกของการเทรนจะเริ่ม

สูง 2026-08-31

ค่า **MAX_PIXELS** กับ **MAX_LENGTH** คำนวณบนกระดาษไว้ตั้งแต่ 25 ส.ค. แต่ไม่เคยรันจริงเลย

เกิดอะไรขึ้น: เอกสาร workflow บันทึกลงไว้ตั้งแต่ 25 ส.ค. ว่า $MAX_PIXELS=7680 / MAX_LENGTH=47104$ กับ margin 4.0 GB เป็นแค่การคำนวณบนกระดาษที่ไม่เคยรันจริง ต้องมี **TEST_STEPS=5** ยืนยันก่อน รอบนี้เป็นรอบแรกที่รันจริง — แล้วก็ชนเพดานทันที

ต้นเหตุจริง: งบประมาณด้าน capacity ถูกบันทึกไว้เหมือนเป็นตัวเลขที่ตรวจสอบแล้ว ทั้งที่จริงๆ เป็นแค่การประมาณที่ยังไม่ได้ทดสอบ

เสียหายอะไร: OOM ที่ step 2 ตอน dry-run แล้ว OOM จริงอีก 2 ใน 4 fold ตอนรันจริง

แก้ยังไง: ทำตามลำดับ fallback ที่เขียนไว้ล่วงหน้า: ลดจำนวนภาพ gridline จาก 4 เหลือ 3 ก่อน แล้วค่อยลด **MAX_PIXELS** ห้ามลด **MAX_LENGTH** เด็ดขาด (เพราะจะตัด label ทั้ง)

กฎที่ได้: ตัวเลขขีดจำกัด capacity ที่ยังไม่ได้ทดสอบ ต้องระบุในเอกสารให้ชัดว่า 'ยังไม่ทดสอบ' และต้องรันวัดจริงก่อนที่จะเอาไปอิงกับการใช้จ่ายเงิน

สูง 2026-08-31

แตกไฟล์ **tarball** ข้อมูล หับสคริปต์เทรนที่แก้ไปแล้วด้วยไฟล์เก่าที่ติดมาในนั้น

เกิดอะไรขึ้น: folds4.tar.gz มีไฟล์ train_t05_courser.py เวอร์ชันเก่าติดมาด้วย พอแตกไฟล์หลังจาก push สคริปต์ที่แก้แล้วไป แล้ว ก็ไปหับของที่แก้ไว้ ทำให้ gpu4 รันการทดสอบด้วยโค้ดเก่าและพังด้วย error 401 (ยัง push อยู่ตอน dry-run)

ต้นเหตุจริง: ไฟล์ข้อมูลต้นมีซอร์สโค้ดติดมาด้วย และลำดับการทำงานคือ push สคริปต์ก่อนแล้วค่อยแตก tarball ที่หลัง

เสียหายอะไร: เครื่องหนึ่งพังตอน probe และเสียเวลาไปกับการหาสาเหตุของบั๊กที่จริงๆ แก้ไปแล้ว

แก้ยังไง: ส่งสคริปต์ใหม่ไปอีกรอบ แล้วใช้ grep ตรวจ marker string ยืนยันในทุกเครื่องทั้ง 4 เครื่อง แทนที่จะเชื่อว่า scp สำเร็จเฉยๆ

กฎที่ได้: ห้ามให้ไฟล์ข้อมูลมีซอร์สโค้ดติดมาด้วยเด็ดขาด ถ้าเสี่ยงไม่ได้ ต้องแตกไฟล์ก่อนแล้วค่อย push โค้ดที่หลัง และต้อง grep ตรวจ marker เวอร์ชันในทุกเครื่องเสมอ อย่าเชื่อว่าการ copy สำเร็จเฉยๆ

สูง 2026-08-31

ตัวรันเดโมยังไม่เคยรันจบครบวงจรสักครั้ง — เซารอบเย็น 3 ครั้งติดก็ยังไม่ได้เซิร์ฟเวอร์ที่ใช้งานได้

เกิดอะไรขึ้น: ช่วงเตรียมเดโมภาคค่าเสียเครื่องเข้าไป 3 รอบติด: เครื่อง 49408084 เข้าต่อทั้งที่เมฆามส่งหยุดแล้ว, เครื่อง 49410094 สตาร์ทโมเดลผิดตัว (t03), เครื่อง 49411642 ติดปัญหา pgrep จับตัวเองผิด ทำให้เซิร์ฟเวอร์ไม่สตาร์ทเลย ทางที่แก้แล้วถูกวางแผนให้ลองรันจริงครั้งแรกเช้าวันถัดไป

ต้นเหตุจริง: แต่ละจุดที่แก้ ถูกตรวจสอบเฉพาะปัญหาที่มันแก้เท่านั้น ไม่เคยรันทั้งเส้นทาง GO.bat ตั้งแต่ต้นแบบ cold start สักครั้งเดียว

เสียหายอะไร: เสียเวลาเข้าเครื่องไปประมาณ 3 รอบโดยไม่ได้เดโมที่ใช้งานได้เลย และเส้นทางสำหรับวันพรุ่งนี้ (1 ก.ย.) ก็ยังไม่ได้พิสูจน์ว่าใช้ได้จริง

แก้ยังไง: ยังไม่ได้แก้ — เลื่อนการรันครบวงจรครั้งแรกไปเป็นเช้าวันถัดไป

กฎที่ได้: หลังแก้แต่ละจุดในเชนงาน ต้องรันทั้งเชนใหม่ตั้งแต่ cold start อย่างน้อยหนึ่งครั้ง การแก้ทีละจุดแยกกัน 3 รอบติด ก็ยังไม่ได้พิสูจน์ว่าทั้งเชนใช้งานได้จริง

สูง 2026-09-01

t04 ถูกฝึกจนจบโดยไม่มีจุดเช็คถูกๆ ก่อนใช้เงินจริง เลย์รู้ตัวว่าพังก็ต่อเมื่อจ่ายเงินไปแล้ว

เกิดอะไรขึ้น: จดบกร่องรายแรงของ t04 ถูกพบก็ต่อเมื่อรันฝึกจบทั้งรอบแล้ว เสียเงินไปหลายพันบาท ในขณะที่จุดบกร่องแบบเดียวกันของ t44 จับได้ในราคาแค่ประมาณ \$2 เพราะมี go/no-go gate ใส่ไว้แล้วตอนนั้น

ต้นเหตุจริง: ตอนทำ t04 ยังไม่มีขั้นตอนตรวจสอบก่อนใช้เงิน การตรวจสอบถูกวางไว้หลังขั้นตอนแพงโดยปริยาย แทนที่จะวางไว้ก่อน

เสียหายอะไร: เสียเงินค่า GPU ไปหลายพันบาทกับ t04 ทั้งที่การเช็คแค่ราวๆ \$2 ป้องกันได้

แก้ยังง: เพิ่ม go/no-go gate: โหลดโมเดลแล้วให้ลองอ่านหน้าแบบจริง 3 หน้าก่อนตัดสินใจฝึกเต็มรอบ ผ่าน → ฝึกต่อ, ไม่ผ่าน → หยุด

กฎที่ได้: ต้องมีจุดตรวจสอบที่ถูกที่สุดเท่าที่ทำได้ วางไว้ก่อนทุกขั้นตอนที่ใช้เงินเยอะเสมอ การเช็คแค่ \$2 ที่ช่วยประหยัดได้ถึง \$70 ต้องทำทุกครั้งไม่มีข้อยกเว้น แม้ตอนนั้นทุกอย่าง 'ดูเหมือนจะโอเคอยู่แล้ว' ก็ตาม

สูง 2026-09-01

วันเดโมสดทั้งวันหมดไปกับปัญหาโครงสร้างพื้นฐานและวนทำลาย/เริ่มใหม่ เดโมไม่จบสักครั้ง

เกิดอะไรขึ้น: ทั้งเซสชันหมดไปกับปัญหาโครงสร้างพื้นฐาน: tunnel ตาย, host แยกจนต้องทำลายแล้วเช่าใหม่, และวนรอบ teardown เพิ่ม 2 รอบ (`presentation.py down` ยืนยันว่าเหลือ 0 instance ที่คิดเงินทั้งสองรอบ รอบหนึ่งทำแค่เพื่อเดโมใหม่โดยเปิดอัติโนมัติ) เดโมสดจริงไม่เคยรันจบสักครั้ง เลื่อนไปวันถัดไป

ต้นเหตุจริง: ไม่มีการ smoke test ก่อนเดโม การเดโมจริงกลายเป็นการเช็คครบวงจรครั้งแรก ทำให้ปัญหาที่ซ่อนอยู่ในโครงสร้างพื้นฐาน (tunnel, host, key) มาโผล่ตอนช่วงเดโมแทนที่จะโผล่ก่อนหน้านี้

เสียหายอะไร: เสียเวลาทำงานทั้งเซสชันโดยไม่ได้เดโมเลย เสียค่าเช่า GPU หลายเครื่องแต่ไม่ได้ผลลัพธ์เดโมอะไรเลย

แก้ยังง: ไม่มีการแก้ในวันนั้น — มะขามตัดสินใจเองว่าจะลองใหม่วันถัดไป

กฎที่ได้: ต้องรัน dry-run เต็มเส้นทางเดโม (เช่า → tunnel → ส่งงาน → หน้าหน้า) ก่อนถึงเวลาเดโมจริงหลายชั่วโมง บนเครื่องเช่าราคาถูก อย่าปล่อยให้เดโมจริงเป็นครั้งแรกที่ทดสอบทั้งระบบแบบครบวงจร

เรื่องที่เหลือในหมวดนี้ (136)

ไม่มีบันทึกการทำงานเลยก่อนวันที่ 2026-07-05 และไม่เคยย้อนเอา git history ช่วงแรกมาเขียนเป็นไดอารี่

กลาง 2026-07-01

เริ่มเขียนบันทึกการทำงานตั้งแต่วันแรกของโปรเจกต์ ถ้าเริ่มเข้าไปแล้ว ให้อ่านไล่ git history ที่มีอยู่มาเขียนย้อนหลังทีเดียวทันที เพราะ commit message บอกแล้วว่าอะไรเปลี่ยน แต่ไม่เคยบอกว่าทำไม และ 'ทำไม' คือส่วนเดียวที่ย้อนกลับไปสร้างใหม่ไม่ได้

ย้ายเอกสารข้ามรีโปโดยไม่ได้เช็คว่ามีอะไรลิงก์มาที่ path เดิมบ้าง

กลาง 2026-07-05

ก่อนย้ายหรือเปลี่ยนชื่อไฟล์ที่มีเอกสารอื่นอ้างถึง ให้ grep หา path เดิมทั้ง workspace แล้วแก้ทุกจุดในการแก้ครั้งเดียวกัน การย้ายยังไม่ถือว่าเสร็จจนกว่าลิงก์ขาเข้าจะเสร็จด้วย

schema มีจุดที่ยังไม่ตัดสินใจ 5 จุดตั้งแต่เริ่มสเก็ตข้อมูล และหนึ่งในนั้นเพิ่งโผล่กลางงานจนงานติด

กลาง 2026-07-05

ให้สำรวจประเภทหน้าแบบคร่าวๆ ทั้งชุดเอกสารก่อนออกแบบ schema ไม่ใช่ดูแค่ตัวอย่างบางส่วน การเจอหน้าประเภทใหม่ทั้งกลุ่มกลางคันจะบิบให้ตัดสินใจเรื่อง schema ตอนกำลังรีบ ซึ่งเป็นที่มาของถึงขยะรวมที่ไม่มีใครกลับมาเก็บกวาดอีกเลย

คีย์ **localStorage** ถูกประกาศเองในโมดูล 14 ที่ พร้อมรายการคีย์แบบดูแลด้วยมือ ซึ่งคือต้นเหตุจริงของบั๊ก **deleteProject** ที่หึงข้อมูลกำพราไว้

กลาง 2026-07-05

รายการที่ควรจะได้มาจากข้อมูลอื่น แต่ดูแลด้วยมือ ยังไงก็ต้องแก้ค้ำ ให้ derive มันมาจากทะเบียนกลางแทน และเวลาบั๊กเป็นแบบ 'ลืมเพิ่ม X ลงในรายการ Y' ให้แก้ที่กลไกที่ยอมให้รายการนั้นแยกออกมาได้ ไม่ใช่แค่เติมรายการที่ขาด

เขียน **config** ของ **Label Studio** ส่งไปโดยไม่เคยเปิดดูเอง มะขามเลยต้องวางใหม่ 4 รอบในเย็นเดียว ตามที่แต่ละจุดพัง โผล่มาจากสกรีนช็อตของเขา

กลาง 2026-07-06

อย่าส่ง config ของ UI ให้คนอื่นเป็นคนทดสอบครั้งแรกแทน ให้โหลดดูเองสักรอบก่อน แม้จะเป็น task ปลอมอันเดียวยังดี ค่อยบอกว่าพร้อม ทดสอบเองรอบเดียวจะยุบวงจรวางใหม่ 4 รอบให้เหลือรอบเดียว

หน้า **BOQ 41-47** ค้างไม่ได้สัปดาห์สองวัน หลังจากรอบก่อนถูกหยุดกลางคัน

กลาง 2026-07-07

เวลางานถูกหยุดกลางคัน ให้เขียนหน่วยสุดท้ายที่ทำเสร็จลงในรายการค้างทันที ช่วงที่ขาดหายไปถ้าอยู่แค่ในบทสนทนา จะถูกเอามาเกลี่ยกันใหม่ทุกเซสชัน

ข้อความ **'Issue loading URL from \$image value'** ถูกสงสัยว่าเป็น **CORS** หรือยังไม่ **push** ขึ้น **GitHub** ทั้งที่ **URL** ปกติดีหมด

กลาง 2026-07-08

เจอ error แบบ 'โหลด URL ไม่ได้' ให้ยิง URL นั้นด้วยตัวเองก่อนเลย (curl ทั้งแบบใส่และไม่ใส่ Origin) ก่อนจะไปตั้งทฤษฎี เพราะปัญหาฝั่ง server กับฝั่ง client ขึ้นข้อความ error ในแอปหน้าตาเหมือนกันปะ

สร้างเส้นทาง **migration** เพื่อประหยัด **token** ขึ้นมาทั้งชุด รันไปแล้ว แล้วลบทิ้งภายในไม่กี่นาที

กลาง 2026-07-08

ตัดสินใจให้ชดก่อนว่ารอบนี้ทำเพื่อ 'จัดรูปข้อมูลใหม่' หรือ 'ตรวจสอบซ้ำ' ก่อนจะไปคิดเรื่องลดต้นทุน เพราะ migration ไม่มีทางเจอข้อผิดพลาดในข้อมูลที่มันกำลังย้ายอยู่ได้เลย

หน้าของ **t3** ประมาณ 13 หน้าติด **pattern type** ผิด และเจอเพราะ **t4** มาอ่านรูปใหม่ทุกหน้าเท่านั้น

กลาง 2026-07-08

คำศัพท์แบบปิดที่ไม่มีนิยามเขียนกำกับแต่ละคำ ไม่ใช่คำศัพท์ที่ควบคุมได้ มันเป็นแค่ข้อเสนอแนะ ให้นิยามทุกชนิดให้ครบก่อน สกัดข้อมูลรอบแรก ไม่ใช่หลังรอบที่สอง

คำสั่งย่อ **op1** ถูกเขียนเป็นเอกสาร **workflow 7** ขึ้นเต็มรูปแบบ ทั้งที่ยังไม่เคยรันสักครั้ง

กลาง 2026-07-10

อย่าเขียน workflow เป็นเอกสารหลักก่อนที่จะรันจริงจนจบสักรอบ เพราะการรันจริงรอบแรกคือสิ่งที่บอกว่ามีจุดพังแบบไหนบ้างที่ spec ต้องรับมือ

agent ที่ถูกฆ่าเพราะชนลิ้มิตของเซสชัน รายงานจุดเช็คพอยต์ในข้อความสุดท้ายต่ำกว่าที่มันเขียนลงดิสก์ไปจริงๆ

กลาง 2026-07-12

หลัง agent ถูกฆ่าทุกครั้ง ไฟล์บนดิสก์คือแหล่งความจริงเดียวเรื่องความคืบหน้า อย่าทำงานต่อจากข้อความสุดท้ายของตัว agent เอง

แก้ไฟล์เกี่ยวกับการ **tune** โดยไม่ได้ทำขั้นตอนเดือนที่กฎข้อ 2 กำหนดไว้

กลาง 2026-07-13

เวลาถูกกำหนดขอบเขตด้วย path ให้เช็คก่อนว่าโพลเดอร์ใหม่เข้าข่ายเจตนาของกฎนั้นหรือเปล่า อย่าเหมารวมมันได้รับการยกเว้น โพลเดอร์งานใหม่ต้องเกาะขอบเขตของกฎก่อนจะแก้ไฟล์แรก ไม่ใช่มาเกาะทีหลัง

ไฟล์ **task** ของ **Label Studio** ถูก **generate** ไว้สำหรับบ้านหลังที่ 2-5 แต่ไม่เคยถูก **import** จริง

กลาง 2026-07-13

pipeline ที่ขั้นสุดท้ายต้องทำมือ จะหยุดค้างอยู่ที่ขั้นอัตโนมัติขั้นสุดท้ายเสมอ ให้เขียนสคริปต์ import ชะ หรือไม่ก็นิยามคำว่าเสร็จเป็น 'import แล้ว' ไม่ใช่ 'generate แล้ว'

ข้อมูลสดตกแต่งและประตุนหน้าต่างระดับสถาปัตยกรรมที่สกัดมาจาก 5 หลัง ไม่มีใครในแอปเอาไปใช้เลย

กลาง 2026-07-14

ก่อนจะเพิ่มกลุ่มฟิลต์ใหม่เข้าไปในขอบเขตการสกัด ต้องระบุให้ได้ก่อนว่าใครเป็นคนใช้ข้อมูลนั้นต่อ การสกัดเพราะ 'มันพิมพ์อยู่ในแบบ' คือสาเหตุที่ทำให้ dataset โตขึ้นแต่ไม่มีค่าเพิ่มขึ้นตาม

จุดปลายคาน B2 พิมพ์ผิดเป็น D'2/D'3 ทั้งที่ควรเป็น D2/D3

กลาง 2026-07-15

หลังจากเพิ่ม dummy grid แล้ว ต้องรันเช็คว่า grid_ref ทุกตัวในไฟล์ซีไปยัง grid ที่ element นั้นแต่ละจริง — จุดที่แก้มีใกล้ label ที่เพิ่งเพิ่มใหม่คือจุดเสี่ยงพิมพ์ผิดที่สุด

AI ภายนอกแนะนำให้เปลี่ยนเกรตเทสิกเสริมทั้งที่ทำแล้วจะทำให้ข้อมูลที่ถูกอยู่แล้วเสียหาย

กลาง 2026-07-15

ให้มองคำแนะนำจาก AI ภายนอกเป็นแค่สมมติฐาน ไม่ใช่ข้อสรุป ต้องตรวจกับภาพต้นฉบับและกฎ convention ของโปรเจกต์เอง ก่อนทำอะไรตามคำแนะนำนั้น

ระยะที่หน้า 12 บอกว่าช่วง E-D ยาว 2.70 ม. ทั้งที่ grid master บอกว่า 2.0 ม.

กลาง 2026-07-16

ต้องเพิ่มขั้นตอนตรวจ dimension_m ของแต่ละไฟล์เทียบกับค่า span ที่คำนวณจาก grid master ความขัดแย้งระหว่างไฟล์กับ master ต้องเป็น hard error ไม่ใช่แค่โน้ต

dummy grid E' ของบ้านหลังที่ 4 มีค่าขัดแย้งกันสามค่าในสามแบบโครงสร้างของบ้านหลังเดียวกัน

กลาง 2026-07-16

การติดธงความขัดแย้งสามทางแทนการเปลี่ยนค่าถือว่าถูกแล้ว แต่ค่า grid master ที่ยังขัดแย้งกันอยู่ต้องบล็อกไม่ให้ไหลต่อไปยังไฟล์ที่พึ่งพามัน ไม่ใช่แค่ขึ้นคำเตือนเฉยๆ

ข้อสรุปที่ว่า 'ชายคา 1.30 ไม่เกี่ยวกับ 1'/4'' ผิดไปครึ่งหนึ่ง ต้องกลับมาทบทวนใหม่

กลาง 2026-07-16

ข้อสรุปเชิงลบเกี่ยวกับฟิเจอร์หนึ่งไม่สามารถเหมารวมกับทั้งแผ่นได้ ต้องไล่ดูฟิเจอร์ทั้งหมดบนแผ่นก่อนสรุปว่าไม่เกี่ยวข้องกัน และต้องลบร่องรอยของการอ่านเดิมที่ถูกแทนที่ไปในการแก้ครั้งเดียวกันเลย

grid ref สกัดมาพร้อมคำนำหน้า 'grid'/'dummy' และเครื่องหมาย prime สะกดเต็มคำ จนต้องมาไล่ normalize ทั้งชุดที่หลังในทุกบ้าน

กลาง 2026-07-19

ต้องกำหนด format string มาตรฐานไว้ใน schema ก่อนเริ่มสกัดข้อมูล ไม่ใช่มาแก้ทีหลัง การมาแก้ format ย้อนหลังทีหลังต้องไล่รัน regex ทุกบ้าน และทุกครั้งที่รัน regex คือความเสี่ยงที่จะไปทำฟิลต์ข้างเคียงเสียหาย

การวัดระยะจาก pixel ที่ไม่ได้ calibrate ทำให้ขึ้นธงความขัดแย้งปลอมกับค่า grid ที่ถูกอยู่แล้ว

กลาง 2026-07-20

ห้ามบันทึกความขัดแย้งที่มาจากการวัดโดยไม่ calibrate อัตราส่วน px/m กับเส้นกึ่งกลาง grid ที่รู้ค่าแน่นอนอย่างน้อย 2 เส้นบนแผ่นเดียวกันก่อน การวัดที่ไม่ได้ calibrate ไม่ถือเป็นหลักฐาน

grid_ref เขียนเป็น 'A-B' ซึ่งขัดกับคำบรรยายในไฟล์เดียวกันเอง

กลาง 2026-07-20

ต้อง validate grid_ref ทุกตัวเทียบกับข้อความบรรยายของไฟล์ตัวมันเอง ไฟล์ที่ขัดแย้งกับตัวเองคือบักประเภทหนึ่งที่สคริปต์จับได้ และเครื่องหมาย prime ก็สับสนกันได้ง่ายมาก

การ paste คำสั่งหลายบรรทัดพร้อมกัน ทำให้บางคำสั่งหาย (คำสั่ง `cd` ไม่เคยรันเลย)

กลาง 2026-07-21

รันทีละคำสั่งเดียว และรอให้ prompt กลับมาก่อนพิมพ์คำสั่งถัดไป ห้าม paste ก่อนคำสั่งจำนวนมากลงใน terminal ที่กำลังทำงานอยู่เด็ดขาด

เข้าใจว่า full training เริ่มรันแล้ว แต่จริงๆ คำสั่งแคตต่อคิวรออยู่หลังงานทดสอบที่ยังรันไม่เสร็จ

กลาง 2026-07-21

คำสั่งจะถือว่าเริ่มรันจริงก็ต่อเมื่อ prompt กลับมาแล้วเท่านั้น ต้องตรวจสอบด้วย process/log ก่อนที่จะสรุปว่างานที่ใช้เวลานานกำลังรันอยู่ — ห้ามพิมพ์คำสั่งล่วงหน้าใส่ terminal ที่กำลังไม่ว่าง

เสียเวลาทั้งวันที่ 21 ไปกับการพยายาม export mmpo ทั้งที่ไฟล์จากผู้ผลิตเดิมใช้งานได้อยู่แล้ว

กลาง 2026-07-21

ก่อนจะสู้กับข้อจำกัดของเครื่องมือ ต้องเช็ก่อนว่า config ของตัวเองทำให้ปัญหานั้นไม่จำเป็นต้องแก้เลยหรือเปล่า — component ที่ frozen อยู่แล้วไม่ต้อง export ใหม่ ไฟล์จาก upstream ยังใช้ได้

dummy grid B' ของบ้านหลังที่ 5 บันทึกเป็น 12.5 ก่อนจะตรวจแบบต้นฉบับพบว่าค่าที่ถูกต้องคือ 12.2

กลาง 2026-07-24

ห้ามวาง dummy grid ไว้ที่ค่าใดก็ตามที่ทำให้คานลงตัวพอดี ต้องวัดจากแบบต้นฉบับก่อนเสมอ แล้วค่อยเช็ว่าคานลงตัวหรือไม่ การย้อนปรับ grid ให้เข้ากับ element จะบังความขัดแย้งที่แท้จริงไว้

การรัน extraction บ้านหลังใหญ่ทั้งสองครั้งถูก kill กลางทาง ไม่ได้ output ออกมาเลย

กลาง 2026-07-25

งาน inference ที่ใช้เวลาหลายชั่วโมงต้องมี checkpoint บันทึก output ที่ละหน้า และควรรันแบบ detached (tmux/nohup) ไม่ใช่รันแบบ foreground ใน session แบบโต้ตอบ เพื่อให้การ kill เสียแค่หน้าเดียว ไม่ใช่เสียทั้งงาน

คิดเอาเองว่าโมเดลขนาดใกล้เคียงกันใช้เวลาเทรนพอกัน แต่ Qwen3-VL ช้ากว่า Qwen3.6 ถึง 2.9 เท่าด้วยการตั้งค่าเดียวกัน

กลาง 2026-07-28

ห้ามเดาเวลาหรือค่าใช้จ่ายจากจำนวนพารามิเตอร์ ให้วัดเวลาต่อ step จากการรันทดสอบสั้นๆ (TEST_STEPS) แล้วค่อยคูณออกไป ก่อนตัดสินใจรันเต็ม

ไม่ได้วางแผนวัด t01 ซ้ำบนเครื่องเดียวกันไว้ในรอบเช้า เลยต้องดาวน์โหลด base model ~70GB ใหม่อีกรอบทั้งที่มิเตอร์เดินอยู่

กลาง 2026-07-28

ถ้าจะเทียบ A/B แบบมีคัมตัวแปร ต้องวัดทั้งสองฝั่งบนเครื่องเดียวกันในรอบเดียวกัน ไม่งั้นพอมีคนสงสัยตัวเลขเมื่อไหร่ การรันตัวควบคุมจะกลายเป็นงานแถมที่แพงและไม่ได้วางแผนไว้

เกือบวินฉัยผิด: โมเดลที่เจียบอยู่ เกือบถูกตัดสินว่าค้าง

กลาง 2026-07-29

'ไม่มี output' ไม่ใช่หลักฐานว่า 'หยุดทำงาน' ก่อนจะ kill หรือไล่แก้งานยาวๆ ที่เจียบอยู่ ให้ยืนยันจากช่องทางที่สองก่อน เช่น การใช้งาน GPU ขนาดไฟล์ที่โตขึ้น หรือ log แสดงชีพจรที่ตัวงานพิมพ์ออกมาเอง

สคริปต์สกัดข้อมูลที่เขียนไว้วันที่ 28 อยู่ใน scratchpad และหายไปแล้วตอนวันที่ 29

กลาง 2026-07-29

อะไรก็ตามที่รันสำเร็จแล้วสักครั้ง ถือเป็นผลงาน ย้ายออกจาก scratchpad แล้ว commit ก่อนจบรอบงาน scratchpad มีไว้เก็บ output ที่ไม่ต้องใช้อีก ไม่ใช่เก็บสคริปต์ที่ผลิตมันขึ้นมา

แผนเดิมคือสกัดทุกหน้า ซึ่งกินเวลา ~16 ชม./บ้าน กว่าจะมีคนลองคุณเลขดู

กลาง 2026-07-29

ก่อนเริ่มงาน batch ให้เอาต้นทุนต่อหน่วยคูณจำนวนหน่วยดูก่อน แล้วเทียบกับเวลาที่ยอมจ่ายได้จริง คำนวณหัววินาที ช่วยไม่ให้เริ่มงานสืบทอดชั่วโมงที่ยังไงก็ต้องยกเลิกอยู่ดี

พลาดเรื่องขอบเขตหน้าซ้ำอีกที่บ้าน 09 — มีหน้า 'เกณฑ์' อยู่ 4 หน้า แต่ของจริงมีหน้าเดียว

กลาง 2026-07-30

พอเจอและแก้ข้อผิดพลาดในบ้านหลังหนึ่ง ให้ไปแก้กระบวนการที่ทำให้เกิดมันก่อน แล้วค่อยเริ่มบ้านหลังถัดไป อีกอย่างคือ หน้าท้ายเล่มของบ้านในซีรีส์เดียวกันควรทำ hash เทียบกัน ถ้าหน้าไหนเหมือนกันทุก byte ควรรู้จักและไม่ต้องอ่านซ้ำ

ชุด BOQ ที่เกือบเหมือนกัน 6 ชุด (174 แผ่น) ถูกไล่อ่านทีละแผ่น แทนที่จะ diff เทียบกัน

กลาง 2026-07-31

ก่อนสกัดชุดเอกสารที่เกือบซ้ำกัน ให้ diff เทียบกับชุดที่สกัดไว้แล้วก่อน แล้วอ่านเฉพาะแผ่นที่ต่าง พิสูจน์ให้ได้ว่าเหมือนกันทุก byte แล้วค่อยเอาของเดิมมาใช้ซ้ำ

ໃບທຳຮ້າງຂໍ້ 55: ອາດເຊື່ອ log ທີ່ບອກວ່າແກ້ແລ້ວ ໃຫ້ໄປຕາລາງກັບຂໍ້ມູນຈິງກ່ອນແກ້ spec

กลาง 2026-08-02

'log บอກວ່າແກ້ແລ້ວ' เป็นข้อสันนิษฐาน ไม่ใช่ข้อเท็จจริง ให้ grep ข้อมูลจริงดูก่อน ถ้าจะตัดสินใจอะไรที่ขึ้นอยู่กับว่าการแก้ครั้งก่อนลงจริง

ຍັງມີ 5 ໄຟລ໌ທີ່ຕິດ phase_note ຢູ່ ສິ່ງຕາມກຸງຂອງ schema ເອງແປວ່າໄຟລ໌ນັ້ນຍັງບໍ່ເສຍ

กลาง 2026-08-02

มาร์คที่แปลว่า 'ยังไม่เสร็จ' ต้องมีตัวตรวจที่ทำให้ไม่ผ่านเมื่อเจอ TODO ที่ไม่มีใครสแกนหา ก็เท่ากับไม่มี TODO

ແຕ່ປ່ຽນຊື່ໂຟລເດຣີ ຕ້ອງໄປແກ້ໄຟລ໌ JSON 454 ໄຟລ໌ ເພາະຝັງ path ເຕັມໄວ້ໃນຂອງຂໍ້ມູນ

กลาง 2026-08-02

ห้ามฝังชื่อโฟลเดอร์ระดับบนสุดลงในช่องข้อมูล ให้เก็บ path แบบอ้างอิงจาก root ที่ตั้งค่าไว้ จะได้เปลี่ยนชื่อ root ได้โดยไม่ต้องแตะข้อมูล

ເຈັດ session ຕິດກັນ ບໍ່ເລີຍບັນທຶກໄດ້ອາໄສຕ່ຳ ຕ້ອງມານຳຍ້ອນຈາກ git log ທີ່ຫຼັງ

กลาง 2026-08-05

เพิ่ม 'เขียนไดอารี่แล้ว' เข้าไปในด่านก่อนปิดเครื่องของ op03 ควบคู่กับการ commit เพราะ git log บอกได้แค่ว่าจะอะไรเปลี่ยน ไม่เคยบอกว่าตัดสินใจอะไรไป หรือเกือบพลาดตรงไหน

ບ້ານຫຼັງ 15 ຖືກ commit ດ້ວຍຂໍ້ຄວາມວ່າ 'ບ້ານຕໍ່ມາ' ແລະໄດ້ບັນທຶກຜົນ check_format.py ໄວ້ເລີຍ

กลาง 2026-08-08

ทำให้ผลของตัวตรวจเป็นส่วนหนึ่งของ commit เลย (ผ่าน pre-commit hook หรือบังคับให้มีในข้อความ commit) ด่านคุณภาพที่ไม่ได้บันทึกผลไว้ ถือว่าไม่เคยผ่านด่านนั้น

ງານສັກດຂໍ້ມູນບ້ານເຈັດວັນ ບໍ່ໄດ້ບັນທຶກສະເລ່ຍ ຕ້ອງມາຍ້ອນສ້າງຈາກ git log ທີ່ຫຼັງ

กลาง 2026-08-18

งาน op รอบหนึ่งยังไม่ถือว่าจบจนกว่าจะเขียนไดอารี่เสร็จ ให้เขียนใน session เดียวกับที่ทำงาน ก่อนจะทำลายเครื่องเข้าหรือปิด session เพราะการย้อนจาก git log ได้แค่ว่าจะอะไรเปลี่ยน ไม่มีทางได้ว่าทำไม

ເພດານເວລາ 90 ນາທີຂອງ op3 ທີ່ເຮັດບ້ານຫຼັງໃຫຍ່ທີ່ສຸດ ບໍ່ມີວັນເສຍເລີຍເປັນລະບົບ

กลาง 2026-08-18

ถ้างานที่จับเวลาไว้ทำงานขึ้นใหญ่สุดไม่ทัน ให้เพิ่มโหมดทำต่อได้หรือแบ่งงานเป็นท่อน อย่าปล่อยให้การเลือกงานเี่ยงไปหาของง่าย เสียๆ ไม่งั้นเคยยากจะไม่มีวันเข้า dataset เลยสักที

ສະຕຣິບ໌ເປີຍເບີຍແລະ overlay ເຊຍນໄວ້ໃນ scratchpad ຂອງ session ບໍ່ເລີຍ commit ເລີຍ

กลาง 2026-08-20

สคริปต์ไหนที่ผลิตตัวเลขที่คุณจะเอาไปอ้าง ต้อง commit ใน session เดียวกันนั้น โค้ดที่ใช้วัดผลคือของส่งมอบ ไม่ใช่ของชั่วคราว ถ้าพบว่าตัวเองต้องเขียนมันใหม่อีกรอบ แปลว่ารอบแรกก็ควร commit ไปแล้ว

ແກ້ repetition penalty ແລ້ວ commit ເລີຍ ໂດຍບໍ່ເລີຍຣັນບນ GPU ສັກດຮຸ່ງ

กลาง 2026-08-21

การแก้พารามิเตอร์ decoding ยังถือว่าไม่ได้ทดสอบ จนกว่ามันจะสร้าง token จริงบนหน้าที่เคยพังให้ดู ให้ระบุชัดๆ ทั้งในโค้ดและไดอารี่ว่ายังไม่ได้พิสูจน์ และห้ามใช้มันปิดบั๊ก

pass1_organize.py ເຊຍນເສຍແລ້ວສົ່ງອອກໄປໂດຍບໍ່ເລີຍຣັນກັບຮູບຈິງເລີຍ

กลาง 2026-08-21

ถ้าถูกสั่งให้ข้ามการทดสอบ ให้ส่งโค้ดพร้อมโน้ตตัวใหญ่ว่าอะไรที่ยังไม่เคยถูกใช้จริง และบอกไปพร้อมกันเลยว่าชุดทดสอบที่ถูกที่สุดคืออะไร เพราะโค้ด crop รูปที่ไม่ได้ทดสอบจะพังแบบเสียๆ ด้วยการให้ crop ที่ผิดแต่ดูน่าเชื่อ

ถ้าให้ Pass 0 จำแนกออกมาเป็น pattern 16 ค่า จะทำให้ Pass 1 ตัดสินใจส่งงานต่อไม่ได้เลย

กลาง 2026-08-21

เขียนไฟล์สัญญาณระหว่างสองขั้นตอนก่อนจะลงมือเขียนขั้นตอนไหนก็ตาม สัญญาณนั้นแหละคือจุดที่จะเห็นชัดว่าเป้าหมายการจำแนกละเอียดไม่พอ และเห็นตรงนั้นถูกกว่ามารู้ตอนเทรนเสร็จเยอะ

รูปแบบผลลัพธ์ของ run_house_batch.py เข้ากันไม่ได้กับตัว import ของแพลตฟอร์ม

กลาง 2026-08-21

นิยามสัญญาว่าไฟล์ฝั่งผู้ใช้ข้อมูลต้องหน้าตาอย่างไร ก่อนจะเขียนรูปแบบผลลัพธ์ของฝั่งผู้ผลิต และถ้าเสียงตัวแปลงไม่ได้จริงๆ ให้มันคัดลอกเนื้อหาแบบตรงตัวและบันทึกทุกฟิลด์ที่มันสร้างขึ้นเอง เพื่อไม่ให้มีใครเข้าใจผิดว่าไฟล์ที่ผ่านตัวแปลงคือไฟล์ที่คนตรวจแล้ว

skill ชื่อ op04 เขียนและลงทะเบียนไว้ โดยไม่เคยเอาไปรันกับบ้านสักหลัง

กลาง 2026-08-21

skill ที่ลงทะเบียนไว้ = คำสัญญาว่ามันใช้ได้ ห้ามลงทะเบียน workflow ที่ยังไม่ทดสอบโดยไม่มีโน้ตตัวใหญ่บนหัวไฟล์ skill ว่ายังไม่เคยรัน และต้องเอาไปรันกับของจริงหนึ่งชิ้นก่อนเริ่มแคมเปญถัดไป

สร้างเส้นทาง inference ด้วย GGUF ในเครื่องเสร็จ แล้วหันทันทีที่วัดได้ว่าใช้เวลา 23 นาทีต่อหน้า

กลาง 2026-08-24

จับเวลาหน่วยงานตัวแทนหนึ่งชิ้นบนเครื่องที่จะใช้จริงก่อน แล้วค่อยสร้าง workflow รอบๆ มัน การวัดแค่ 60 วินาทีฆ่าเส้นทางที่ไม่เวิร์กได้ก่อนที่มันจะกินเวลาไปทั้งวัน

การตัดสินใจที่ได้รับมอบอำนาจไว้แล้วภายใต้ att1235 ถูกเขียนว่า "รอมะขาม" แทนที่จะตัดสินใจไปเลย

กลาง 2026-08-24

เมื่อได้รับคำสั่งยืมมาให้ตัดสินใจเอง ให้บันทึกผลว่า "ตัดสินใจแล้ว" + เหตุผล + ความเสี่ยงที่รับไว้" ห้ามเขียนว่า "รออนุมัติ" การโยนการตัดสินใจที่มอบหมายมาแล้วกลับไป ตัวมันเองคือความผิดพลาด

ตารางเทียบ train_t03.py กับค่าเริ่มต้นของ t02 ไม่เคยทำงานครบ

กลาง 2026-08-24

ป้ายบอกที่มาบอกได้แค่เจตนา ตารางเทียบต่างหากที่จับของที่ตกหล่น ให้ทำทั้งสองอย่าง คือใส่ทุก argument ที่ส่งเข้าไปเทียบกับรอบที่ใช้ได้ล่าสุด ไม่ใช่เทียบแค่ตัวที่เราตั้งใจเปลี่ยน

สร้าง template matching หลายสเกลขึ้นมาแก้ปัญหาเรื่องขนาด ทั้งที่จริงๆ มันเป็นปัญหาเรื่องรูปทรง

กลาง 2026-08-25

ทดสอบสมมติฐานด้วยการเปิดข้อมูลจริงขึ้นมาดูก่อน แล้วค่อยเขียนโค้ด สิบวินาทีที่ใช้ดูด้วยตา ชนะการนั่งเขียนทฤษฎีผิดๆ เป็นชั่วโมง เหมือนบทเรียนข้อ 14 ใน rule_of_tune.md

ประเมิน Pixtral-Large ก่อนจะไปเช็ค license ซึ่งห้ามใช้เชิงพาณิชย์

กลาง 2026-08-25

เอาตัวกรองเชิงที่ราคาถูกที่สุดขึ้นก่อน license กับเงื่อนไขการใช้เชิงพาณิชย์ใช้เวลาเช็คไม่กี่วินาที และตัดตัวเลือกออกได้ก่อนจะเสียเวลาไปกับขนาด weight และการรองรับของ framework

Claude เข้าใจผิดว่า pass3 คือ "หิ่ง" ทั้งที่มันเป็นขั้นตอนสกัดข้อมูลจริงที่มี 8 subtask

กลาง 2026-08-25

อ่าน spec ของ pipeline stage นั้นเองก่อนจะไปคิดต่อ และพอเข้าใจถูกแล้วให้จดทันที ความเข้าใจที่แก้แล้วแต่ไม่ได้จด จะถูก session ถัดไปเข้าใจผิดซ้ำอีก

ชุดข้อมูลเทรนถูกสร้างใหม่หลังจากรัน sanity check ของ TEST_STEPS ไปแล้ว ผลตรวจที่บันทึกไว้จึงพูดถึงข้อมูลที่ไม่มีอยู่แล้ว

กลาง 2026-08-25

ประทับทุกผลการวัดว่ามาจาก dataset build ใด และรัน sanity check ใหม่ทุกครั้งที่สร้างชุดข้อมูลใหม่ ไม่งั้นการ rebuild ครั้งเดียวจะทำให้ตัวเลขทุกตัวที่บันทึกก่อนหน้านี้กลายเป็นเลขลอยๆ ที่ไม่มีเจ้าของ

ลบบ้านหลัง 42 ในเครื่องตัวเอง (ยังไม่ commit) ขณะที่ origin มีการแก้ไขใหม่กว่าจากเพื่อนร่วมทีม เกิด merge conflict

กลาง 2026-08-26

อย่าแก้ conflict ระหว่าง "ลบ" กับ "แก้ไข" ด้วยการลบ ให้เอาข้อมูลที่ขัดแย้งไปพักไว้ที่พื้นที่พักก่อนแล้วค่อยตัดสินใจทีหลัง และต้องประกาศก่อนจะลบบ้านหลังไหนออกจากเครื่องตัวเอง

ข้อเสนอแนะจากงานวิจัยเรื่อง GPU ไม่ถูกนำไปทำสั๊กข้อ

กลาง 2026-08-26

แปลงข้อเสนอแนะจากงานวิจัยทุกข้อให้เป็น action ที่ทำได้จริงพร้อมประมาณการค่าใช้จ่าย ภายใน session เดียวกัน ไม่จั้นมันจะไม่เกิดขึ้น ข้อเสนอที่ฝังอยู่ในพรีดข้อความคือข้อเสนอที่ไม่ได้เสนอ

สร้าง train/val/test.jsonl ใหม่ ชุดเก่าหายไปเลยโดยไม่ได้เก็บ snapshot ไว้

กลาง 2026-08-26

ประทับ hash หรือเวอร์ชันให้ dataset build ทุกครั้ง แล้วเก็บ manifest ไว้ ผลลัพธ์ใดๆ จะได้ผู้กลับไปหาชุดข้อมูลที่แน่นอนได้ และอย่ารวมการเปลี่ยนแปลงหลายเรื่องที่ไม่เกี่ยวข้องเข้าไปในการ regenerate ครึ่งเดียว เพราะจะสับสนไม่ได้ว่าตัวเลขที่เปลี่ยนมาจากสาเหตุอะไร

commit ที่แก้ GT เขียนแค่ "final" หรือเลขบ้าน สวเหตุผลกลับไม่ได้เลย

กลาง 2026-08-26

commit ที่แก้ข้อมูล GT ต้องระบุว่าจะอะไรและทำไม (แบบไหน ค่าไหน เดิมเป็นเท่าไร) คำว่า "final" ไม่บอกอะไรกับคนที่มาอ่านทีหลังเลย และทำลาย audit trail ของข้อมูลที่ป้อนเข้าโมเดล

ลบหน้า GT ยาว 156 บรรทัดออกจากบ้านหลัง 37 โดยไม่บันทึกเหตุผลไว้เลย

กลาง 2026-08-27

การลบ GT เป็นการตัดสินใจเรื่องข้อมูล ไม่ใช่การเก็บกวาด มันต้องมี commit ของตัวเองพร้อมเหตุผลอย่างน้อยหนึ่งบรรทัด ทุกครั้ง

สัญญาของ hint block อยู่สองที่ (prompt.md กับ cv_hint_text()) มีแค่คอมเมนต์กันไม่ให้เพี้ยน

กลาง 2026-08-29

เวลาที่เอกสารสเปกกับโค้ดต้องตรงกัน ให้เขียนเฮสต์อัตโนมัติตรวจผลลัพธ์ของโค้ดเทียบกับสเปก คอมเมนต์ที่เขียนว่า 'อย่าลืมแก้ไขตรงกัน' ไม่ใช่ตัวควบคุมอะไรเลย

prompt ของ pass 4 เขียนเสร็จทั้งชุด แล้วถูกยกเลิกเพราะไม่จำเป็น

กลาง 2026-08-29

ก่อนสร้างขั้นตอนสเก็ทข้อมูลใหม่ ให้ยืนยันก่อนว่าปลายทางเอาไปใช้จริงไหม ใส่ดูว่าการคำนวณตอนท้ายอ่านไฟล์ไหนบ้าง แล้วสร้างเฉพาะ pass ที่ป้อนไฟล์เหล่านั้น

แผนการทดลอง 2.4b (ภาพที่มาร์คเลข) ถูกสร้างเข้าไปใน pipeline แล้วก็ถูกยกเลิก

กลาง 2026-08-29

ตัดสินใจว่าการทดลองจะมีกี่แผนก่อนลงมือเขียนกลไกสลับแผน เริ่มจากตัวควบคุมบวกอีกหนึ่งแผนพอ แล้วค่อยเพิ่มเมื่อผลเปรียบเทียบรอบแรกบอกว่าต้องเพิ่มจริง

prompt กอปรปริมาณของ pass 3 เขียนเสร็จเต็มรูปแบบแต่ไม่เคยรันเลย และอาจต้องเขียนใหม่ทั้งอัน

กลาง 2026-08-29

อย่าสร้างขั้นตอนที่การออกแบบขึ้นอยู่กับคำถามต้นน้ำที่ยังไม่มีคำตอบให้เสร็จ ทดลองแบบถูกๆ เพื่อเคลียร์ข้อสันนิษฐานนั้นก่อนแล้วค่อยสร้าง

หัวข้อเชาการ์ด Vast ใน RUNBOOK มีกฎหลักสามข้อ — การ์ดถูกก่อน แยก venv ดึงผลลัพธ์ก่อนทำลาย — ทุกข้อเขียนหลังจากเสียหายไปแล้ว

กลาง 2026-08-29

กฎที่อยู่แค่ใน runbook จะถูกข้ามพอดีตอนที่มันสำคัญที่สุด โดยเฉพาะขั้นตอนทำลายเครื่อง ควรให้มันปฏิบัติเสร็จที่รันจนกว่าจะยืนยันแล้วว่าไฟล์ผลลัพธ์ออกจากเครื่องมาแล้วจริง

ย้าย prompt และสคริปต์จาก t03 ไป t04_Purson แล้ว path แบบ relative พังไปหลายไฟล์

กลาง 2026-08-29

พิสูจน์ว่าการย้ายโฟลเดอร์สำเร็จด้วยการสร้างไฟล์ผลลัพธ์ใหม่แล้วเทียบจำนวนกับก่อนย้าย — import ผ่านไม่ใช่หลักฐาน และที่ดีกว่านั้นคือให้อ้าง path ข้ามไฟล์ผ่านค่าคงที่ตัวเดียว การย้ายจะได้แค่บรรทัดเดียว

InternVL3 ถูกปิดตกในแคตตาล็อกวันที่ 2026-08-26 แล้วเอากลับมาใช้อีกสามวันถัดมา — เหตุผลที่ปิดตกใช้ตัดสินใจอะไรไม่ได้เลย

กลาง 2026-08-29

การปิดตกโมเดลต้องอ้างข้อบกพร่องที่วัดได้ หรือข้อจำกัดเชิงๆ (license, framework รองรับไหม, VRAM) ไม่ใช่ความเห็นเรื่องสถาปัตยกรรม จัดอันดับตัวเลือกด้วยเกณฑ์ที่ทดสอบได้ เวลากลับค่าจะได้เป็นเรื่องข้อมูล ไม่ใช่เรื่องความชอบ

prompt ของ pass 3 กับ merge_guard เขียนเสร็จแต่ไม่เคยรันกับโมเดลจริง — และถูกห้ามลองด้วย

กลาง 2026-08-29

โค้ดที่ยังไม่เคยรัน ให้ติดป้ายว่ายังไม่ผ่านการตรวจสอบไว้ตรงจุดที่คนจะกดรันมัน ไม่ใช่แค่ในเอกสาร workflow เขียนเสร็จแล้วตัวเองไม่เท่ากับเคยรันจริง

โครงสร้าง k-fold CV ถูกสร้างขึ้นทั้งที่ตัดสินใจไปแล้วว่าไม่คุ้ม GPU-hours และไม่เคยรันเลย

กลาง 2026-08-29

สร้างตามที่ถูกสั่งได้ แต่ต้องผูกเงื่อนไขไว้ตั้งแต่ตอนสร้างว่า 'รันเมื่อ X' ระบบที่ไม่มีเงื่อนไขว่าจะรันเมื่อไรและแพงกว่าเดิม 5 เท่า แยกไม่ออกจากระบบที่จะไม่ถูกรันเลย และมันจะผูกเมื่อ toolchain ช่างส่างเปลี่ยน

ยังมีความเสี่ยงเดิมคือคืนแคการ์ดใบสุดท้าย และสับสนระหว่าง stop กับ destroy (บทเรียน 2026-08-24) — รอบนี้มีตั้ง 4 ใบ

กลาง 2026-08-30

เช็คลิสต์ระบบต้องไล่ทรัพยากรทีละ ID และปิดท้ายด้วยการถามฝั่งผู้ให้บริการว่าเหลือศูนย์จริง ห้ามเขียนลอยๆ ว่า 'คืนการ์ด'

รายงานว่า 'อัปเดต workflow แล้ว' ทั้งที่ยังไม่ได้แก้ไฟล์จริง

กลาง 2026-08-30

ห้ามรายงานว่าแก้ไฟล์เสร็จถ้ายังไม่ได้ส่งเขียนจริง ถ้าไม่แน่ใจให้เปิดไฟล์อ่านซ้ำแล้วกบรทัดที่แก้มาเป็นหลักฐาน

ตัวแปลง GGUF ของ llama.cpp รองรับเฉพาะ checkpoint InternVL3 แบบ non-hf แต่ทั้ง pipeline ดันบังคับเทรนบนตัว '-hf'

กลาง 2026-08-30

ตรวจสอบความเข้ากันได้ของฟอร์แมตตลอดเส้นทาง (train → merge → export → serve) บนโมเดลเล็กที่สุดก่อนทุ่มเงินรันตัวใหญ่ อย่าปล่อยให้ขั้นตอน export เป็นสิ่งสุดท้ายที่ทดสอบ

แผนเสิร์ฟ adapter ของ t03 ผ่าน vLLM ใช้ไม่ได้ — layout MoE LoRA ของ Unsloth ไม่ตรงกับฟอร์แมตของ vLLM สักแบบ

กลาง 2026-08-30

เปิดดูไฟล์จริง (ชื่อ key, shape) เทียบกับฟอร์แมตที่ปลายทางคาดหวัง บนเครื่องตัวเองก่อนเข้า GPU สักใบ — 'เช็คของถูกก่อนของแพง'

งานเชื่อม Purson→web เขียนเสร็จทั้งเส้น แต่ไม่มีขั้นตอนไหนถูกรันเลย — migration ยังไม่ apply, function ยังไม่ deploy, ปุ่มไม่เคยถูกกดกับ GPU จริง

กลาง 2026-08-30

งานที่มีวันเดโมตายตัว ต้องข้อมเส้นทางเต็มตั้งแต่ต้นจนจบอย่างน้อยหนึ่งรอบก่อนถึงวันจริงนานๆ เขียนให้ชัดว่าขั้นตอนไหนยังไม่เคยรัน แล้วใช้รายการนั้นเป็นตารางงาน

`test\_browser\_path.py` (ใช้ JWT ของผู้ใช้จริง ไม่ใช่ service key) ไม่เคยรันจบกับโมเดลจริงเลย

กลาง 2026-08-30

จัดลำดับการตรวจสอบให้เคลมที่เสี่ยงที่สุดแบบต้นจนจบถูกพิสูจน์ก่อน ตอนที่เครื่องเขายังปกติดี อย่าเก็บเทสต์ที่สำคัญที่สุดไว้ท้ายสุดหลังการทดลองที่อาจทำให้ environment พัง

บ้านหนึ่งหลังใช้เวลา ~3 ชั่วโมงและค่า GPU ~\$3.2 — รันสดตอนฟรีเซนต์ไม่ได้

กลาง 2026-08-30

วัดเวลาต่อหน่วยของงานที่จะเดโม แล้วคูณด้วยขนาดงานจริงก่อนออกแบบรูปแบบการเดโม อะไรที่เกินสองสามนาที่ต้องคำนวณไว้ล่วงหน้า

แนะนำให้ใช้ split เดิมต่อ 'เพื่อประหยัดงบไว้ทำ eval' โดยไม่เคยตรวจว่า val ของ split นั้นปนเปื้อนหรือเปล่า

กลาง 2026-08-30

อย่าให้คำแนะนำเรื่องบหรือตารางงานที่เกี่ยวกับ dataset จนกว่าจะพิสูจน์ความสะอาดของ split ก่อน สคริปต์ถูกๆ ตัวเดียวเล็กกว่าแผนทั้งแผนที่สร้างทับข้อสันนิษฐานนั้น

ลองจับคู่หน้าคานด้วยตาเปล่า แล้วพบว่าทำไม่ได้จริงๆ

กลาง 2026-08-31

ทดลองวิธีทำมือกับเคสจริงที่ยากที่สุดหนึ่งเคสก่อนจะจ้อคิวทำทั้งชุด และถ้ามีพิกัดอยู่แล้ว ให้เลือกวิธีเชิงเรขาคณิตที่คำนวณได้แน่นอน ดีกว่าให้คนมานั่งดู

ประมาณเวลาเทรน (ETA) คลาดไปสองเท่า เพราะเอาเวลาต่อ step ของ InternVL-78B ไปใช้กับ Qwen 35B-A3B

กลาง 2026-08-31

ห้ามยกค่าคงที่เวลาต่อ step ข้ามสถาปัตยกรรมโมเดลเด็ดขาด ให้ถือว่ายังไม่รู้ค่าจนกว่าจะมี dry-run จริงบนโมเดลเป้าหมายแล้วได้ตัวเลขจริงออกมา

ทำลาย GPU ทั่วไป 3 ตัว ก่อนข้อความ 'อย่าเพิ่งคืนการ์ด' จะมาถึงประมาณ 2 นาที

กลาง 2026-08-31

ต้องถือว่าการ destroy เป็นสิ่งที่ย้อนกลับไม่ได้ — ถ้ามีคนตื่นอยู่และคอยติดตามงาน ให้หยุดรอสักครู่ก่อน destroy เพราะต้นทุนของการรอสักไม่กี่นาที ต่ำกว่าต้นทุนของการเสียเครื่องไปมากนัก

ผลทดสอบของ destrier (t05) ถูกเขียนไปอยู่ที่ไฟล์เดอร์ t04_Purson ทำให้ดูเหมือนเป็นผลของสาย t04 ที่ตายไปแล้ว

กลาง 2026-08-31

ต้องเก็บไฟล์ผลเทียบ benchmark ข้ามรอบไว้ในที่ที่ไม่ผูกกับรอบใดรอบหนึ่งตั้งแต่แรก และห้ามเขียนผลลัพธ์ของรอบใหม่ลงในไฟล์เดอร์ของรอบก่อนหน้านี้ แค่เพราะ input มันบังเอิญอยู่ที่นั่น

op04_run.py เขียนผลลงชื่อไฟล์ output ตายตัว ทำให้ถ้ารันเทียบรอบใหม่จะทับผลของรอบแรกทั้งไปเลย

กลาง 2026-08-31

ทุกการทดลองต้องมีไฟล์เดอร์ output ของตัวเองที่ผูกกับชื่อรอบนั้นๆ ห้ามหวังพึ่งการจำเอาไปเปลี่ยนชื่อก่อนรันเทียบ

ค่า fallback ของสัดส่วน U/P ถูกโยนให้ VLM รับผิดชอบ ทั้งที่งานวิจัยของทีมเองพิสูจน์แล้วว่า VLM วัดความแม่นยำระดับถอดปริมาณไม่ได้

กลาง 2026-08-31

ทุกจุดที่ pipeline ขอตัวเลขจากโมเดล แทนที่จะขอชื่อ/label เป็นจุดที่ต้องตรวจสอบกับหลัก symbolic/metric split ไม่ใช่เช็คแค่ path ที่กำลังออกแบบอยู่ตอนนั้น

t44 ถูกสร้างบน InternVL3-78B ทั้งที่แคตตาล็อกของทีมเองตัดออกจากรายชื่อไปแล้วเมื่อไม่กี่วันก่อน

กลาง 2026-09-01

เมื่อการวิจัยสรุปว่ามีรายชื่อที่ถูกตัดออก ต้องนำมาใช้เป็นข้อมูลบังคับสำหรับการตัดสินใจครั้งถัดไป และบันทึกไว้ว่าเลือกตัวไหนจาก shortlist ด้วยเหตุผลอะไร แคตตาล็อกที่ไม่มีใครดูตอนตัดสินใจก็เท่ากับงานวิจัยเปล่าประโยชน์

pass3 วัดค่า span ของคานที่ยังไม่ระบุได้แล้ว แต่ไม่เคยถูกต่อสายเข้ากับ adapter เพื่อไปแทนที่ค่า null

กลาง 2026-09-01

การสร้างความสามารถขึ้นมาเป็นแค่ครั้งหนึ่งของการแก้ปัญหา ต้องติดตาม 'ต่อสายเข้ากับตัวที่ใช้งานจริงแล้ว' เป็นเงื่อนไขความสำเร็จแยกต่างหากอย่างชัดเจน ไม่งั้นบั๊กเดิมจะยังอยู่เหมือนเดิม เพียงแต่ซ่อนอยู่หลังส่วนประกอบที่ดูเหมือนแก้แล้ว

ต้องรู้และตั้งค่าสภาพแวดล้อมสคริปต์ใหม่ทั้งชุดเป็นรอบที่สอง เพราะรอบแรกไม่ได้บันทึกวิดีโอหน้าจอไว้

กลาง 2026-09-01

อะไรก็ตามที่มีจุดประสงค์เดียวคือสร้างชิ้นงานให้คนอื่น (การอัดวิดีโอ ภาพหน้าจอ ไฟล์ export) ต้องเตรียมพร้อมและตรวจสอบให้เรียบร้อยก่อนเริ่มงานที่ใช้ต้นทุนสูง ไม่ใช่หลังจากทำเสร็จแล้ว เพราะการรัน GPU เข้าไม่มีทางฟรี

จัดระเบียบโครงสร้าง codebase ไม่ได้ เพราะสคริปต์ hard-code path ไว้ เลยต้องทำสำเนาซ้ำขนาด 115 MB ขึ้นมาแทน — กลายเป็นสองแหล่งข้อมูลที่จะค่อยๆ แยกกันไปจากกัน

กลาง 2026-09-01

การคัดลอกไฟล์เพื่อจัดระเบียบ จะสร้างแหล่งข้อมูลที่สองซึ่งค่อยๆ แยกออกจากกันแน่นอน โปรเจกต์นี้เคยลบโฟลเดอร์แบบนี้ทิ้งไปแล้วด้วยเหตุผลเดียวกัน ต้องแก้ hard-code path หรือใช้สำเนาเดียวกันกับ manifest ก่อนจะไป copy ไฟล์อะไรอีก

การกู้ JSON ที่ถูกตัดขาดกลางทาง เป็นแค่ขั้นตอนกู้ข้อมูลด้วยมือ ไม่ใช่ automation จริง

กลาง 2026-09-02

สคริปต์กู้ข้อมูลช่วยลดความเจ็บปวดได้ก็จริง แต่มันบังปิดตัวปัญหาที่แท้จริงไว้ — ให้งานที่ไว้เป็นหนี้เทคนิค ระบุวิธีแก้ที่แท้จริงไว้ด้วย และห้ามนับว่า pipeline เป็น automation เต็มรูปแบบ トラバ迪ที่ยังมีขั้นตอนที่คนต้องทำเองอยู่ในเส้นทางปกติ

ทดลอง Repeater Choices กับข้อมูลทั้งชุดโดยไม่ทดสอบก่อน ทำให้ import พังทั้ง 71 งาน

กลาง

อย่าทดลองกลไกที่ยังไม่พิสูจน์กับข้อมูลทั้งชุดในครั้งเดียว — ทดสอบกับ 1 รายการก่อนเสมอ

กฎ 'เตือนก่อนเสมอ' ยังไม่ครอบคลุมสถานะระบบภายนอก ไม่ใช่แค่ไฟล์ในเครื่อง

กลาง

กฎ 'เตือนก่อน' ครอบคลุมสถานะระบบภายนอกด้วย ไม่ใช่แค่ไฟล์ในเครื่อง — เช่น การลบ task ทั้งหมดในเครื่องมือ annotation บนคลาวด์ ต้องถามก่อนเสมอว่ามีข้อมูล review จริงอยู่หรือไม่

หน้าแบบที่ผสมสองมุมมองในหน้าเดียว เป็นความเสี่ยงที่เคยเตือนไว้แล้ว แต่เพิ่งได้รับการยืนยันจริงจากการอ่านนำร่อง (pilot) หลังจากออกแบบ schema เสร็จไปแล้ว

ต่ำ 2026-07-05

ให้ทำ pilot read ต่อไป มันได้ผลจริงและเป็นวิธีตรวจสอบที่ถูกที่สุด แต่เมื่อเอกสารสเปกเตือนถึงความเสี่ยงเชิงโครงสร้างไว้แล้ว ควรตัดสินใจให้จบก่อนเริ่มถอดข้อมูล ไม่ใช่แบกคำเตือนนั้นค้างไว้ เพราะคำเตือนที่รอดมาถึงตอนรันงานจริง จะกลายเป็นการเปลี่ยน schema กลางทาง

เอกสารสเปกถูกย้ายออกจาก repo Training แล้วก็เงียบๆ ย้ายกลับไปอีกครั้ง — การย้ายข้าม repo ครั้งนี้เป็นแค่การวนไปวนมา ไม่ใช่การตัดสินใจที่ยินยอม

ต่ำ 2026-07-05

ให้เก็บสเปกไว้ใน repo เดียวกับข้อมูลที่มีคนควบคุมอยู่ ถ้ามีการย้ายกลับในภายหลัง ต้องบันทึกการย้อนกลับนั้นไว้ใน diary ตอนที่มันเกิดขึ้น ไม่งั้น log จะเหลือตำแหน่งที่ขัดแย้งกันสองที่ไว้ในบันทึก

สคีมาย่อฉบับแรก (จาก 45KB เหลือ 10KB JSON) ถูกตัดกลับว่าย่อมากเกินไป ต้องทำใหม่

ต่ำ 2026-07-10

ก่อนย่อ spec ให้เล็กลง ต้องเขียนไว้ก่อนว่าเนื้อหาประเภทไหนบ้างที่ต้องคงไว้แบบคำต่อคำ — ถ้าตั้งเป้าหมายที่จำนวนไบต์ก่อน รับประกันได้ว่าต้องทำรอบสองแน่นอน

สร้างโฟลเดอร์ production โดยไม่มี prefix ลำดับ 00 ต้องมาเปลี่ยนชื่อกลางทางระหว่างย้ายไฟล์

ต่ำ 2026-07-10

ต้องตกลงกฎการตั้งชื่อโฟลเดอร์ให้เรียบร้อยก่อนใส่อะไรลงไป การเปลี่ยนชื่อโฟลเดอร์กลางทางคือจุดกำเนิดของ path อ้างอิงที่ตกค้าง (stale) นั่นที่

ความผิดปกติของราคาในเอกสารต้นทางตัวเดิม ถูกตีตรงซ้ำว่าเป็นการอ่านผิดถึง 4 บ้านติดต่อกัน

ต่ำ 2026-07-12

พอยืนยันครั้งแรกได้แล้วว่าความผิดปกติในเอกสารต้นทางเป็นของจริง ให้งานที่ตกลงในลิสต์ known-anomalies ระดับโปรเจกต์ทันที ไม่งั้นทุกคนที่มาตรวจทีหลังต้องมาค้นพบซ้ำเองอีกรอบ เสียเวลาฟรี

เข้าใจผิดว่า **output** เทอร์มินัลที่ละๆ (^[[B^[[A) หมายถึงงานค้างหรือพัง

ต้า 2026-07-21

ตัดสินใจว่า GPU job ยังทำงานอยู่หรือไม่ ให้ดูจาก `nvidia-smi` (utilisation + VRAM) เท่านั้น อย่าดูจากหน้าตาของ log text

ความพยายามซ่อม **environment** ครั้งแรก (ลง **unsloth** ใหม่) ไม่ได้ช่วยแก้อะไรเลย

ต้า 2026-07-21

การซ่อม environment ที่พัง ต้อง diff ดูว่าจะไปเปลี่ยนไปจริง (เวอร์ชันจาก pip log / run log) แล้วคืนค่าแพ็คเกจตัวนั้นให้ตรงเป๊ะ อย่าแคลง package ที่ import พังใหม่เฉยๆ

ความพยายามแก้ **OOM** ครั้งแรก แก้ผิดจุด (ไปปรับ **allocator** แทนที่จะลด **LoRA rank**)

ต้า 2026-07-21

ก่อนใช้วิธีแก้ OOM แบบทั่วไป ให้ແจแจก่อนว่าจะไรกิน VRAM อยู่จริง (weights / adapters / activations) แล้วจัดการตัวที่กินเยอะที่สุด — วิธีแก้ที่ประหยัดได้แค่ 1% แต่ต้องเสียเวลา restart ใหม่ทั้งรอบ ไม่คุ้ม

สถานะ (**status marker**) ในเอกสาร **workflow** ขัดแย้งกับเนื้อหาของมันเอง

ต้า 2026-07-25

ต้องอัปเดตสถานะพร้อมกับเนื้อหาที่มันสรุปในการแก้ครั้งเดียวกันเสมอ และควรกลับมาตรวจเอกสารติดตามงานเทียบกับของจริงเป็นระยะ อย่าเชื่อมั่นเฉยๆ

งานบ้าน **08 (op2)** ที่ค้างมาจากวันที่ **28** ยังไม่ได้เริ่มทำเลย

ต้า 2026-07-29

แยกงานที่เกิดจากเหตุการณ์เฉพาะหน้ากับงานที่วางแผนไว้ให้อยู่คนละบัญชี งานที่ค้างข้ามวันข้ามสัปดาห์ควรถูกจัดตารางเวลาให้ชัดเจนหรือตัดทิ้งไปเลย การปล่อยให้ค้างเฉยๆ คือวิธีที่ backlog หายไปจากสายตาโดยไม่รู้ตัว

การสร้าง **task** ของ **Label Studio** ถูกรันเป็นขั้นตอนบังคับใน **op1** กับบ้าน **08/09/10** ทั้งที่อีกสามวันถัดมาพีเจอรทั้งหมดนี้ถูกยกเลิกไปเลย

ต้า 2026-07-30

เมื่อพีเจอรปลายทางกำลังถูกทบทวนอยู่ ให้ระงับขั้นตอนที่ป้อนข้อมูลเข้าไปหามันในเวิร์กโฟลว์ต้นทางก่อน ผลลัพธ์ที่ประหยัดที่สุดคือผลลัพธ์ที่ไม่ถูกสร้างขึ้นมาตั้งแต่แรก

โฟลเดอร์ไดอารีภาษาอังกฤษที่ทำคู่ขนานกัน ค้างไม่ได้อัปเดตมาตั้งแต่ **2026-07-21** — สองโฟลเดอร์ไม่ตรงกัน

ต้า 2026-08-02

ห้ามเก็บเอกสารชุดเดียวกันไว้สองสำเนาต่างภาษา/ต่างที่กันเด็ดขาด ให้มีแหล่งข้อมูลจริงแหล่งเดียว — สำเนาที่ค้างเก่าเนื้อหาไม่ทันปัจจุบัน แะยิ่งกว่าไม่มีสำเนาซ้ำเลยเสียอีก

ลิงก์เอกสารสองลิงก์ใน **No_touch_box/CLAUDE.md** เสียอยู่แล้ว — ขาด **path segment** ของ **docs/** ไป

ต้า 2026-08-04

ควรรันตัวเช็คลิงก์กับไฟล์ .md ทั้งหมด ลิงก์เอกสารที่เสียในไฟล์ปฐมนิเทศ เสียเวลาให้กับผู้อ่านใหม่ทุกคน และมักไม่มีใครตรวจพบไปเป็นเดือนๆ

ออกแบบ **Pass 1** เป็นเรื่องเรียงลำดับ/คิวงาน ทั้งที่จริงๆ มันคือการจัดโฟลเดอร์ทางกายภาพ

ต้า 2026-08-21

ก่อนออกแบบ pipeline stage ไหนก็ตาม ให้เขียนไว้ก่อนว่า stage ถัดไปต้องเจออะไรบนดิสก์ — คำกริยาคลุมเครืออย่าง 'จัดระเบียบ' หรือ 'เข้าคิว' ต้องแปลงเป็น directory layout ที่ชัดเจนก่อนเสมอ

เขียนบทสอนเป็นภาษาไทย ทั้งที่กติกาของไดอารีบอกให้ใช้ภาษาอังกฤษเท่านั้น

ต้า 2026-08-21

เวลาติดกาคับความต้องการจริงของผู้อ่านขัดกัน ให้ระบุการเบี่ยงเบนพร้อมเหตุผลไว้ในเอกสารเดียวกันเลย แล้วค่อยแก้กติกาให้เพื่อกรณีนี้ไว้ จะได้ไม่ต้องมีใครมาเถียงซ้ำอีก

สองเซสชันแก้ไฟล์ `run_house_batch.py` ในวันเดียวกันโดยไม่รู้ว่าอีกฝ่ายก็กำลังแก้

ต้า 2026-08-21

เมื่อสองเซสชันกำลังแก้บั๊กเดียวกันในวันเดียวกัน ให้บอกไว้ในไดอารี่ก่อนแก้ ไม่ใช่หลังแก้ ไม่งั้นไฟล์เดียวกันจะโดนแก้ซ้ำการเดียวกันซ้ำสองรอบ แล้วรอบที่สองก็ทำให้รอบแรกไม่มีความหมาย

การเทรนถูกฆ่าทิ้งที่ step 20 (เผาไปแล้วราว \$0.35) เพราะเลือกบ้านที่ใช้ทำ benchmark ผิด

ต้า 2026-08-24

ต้องล็อกการแบ่ง train/val/test เป็นด้าน Phase 0 ก่อนเข้า GPU ทุกครั้ง การเปลี่ยนสัดส่วนแบ่งหลัง step 0 แปลว่าต้องจ่ายค่ารันซ้ำสองรอบเสมอ

ชื่อไฟล์เอกสารเวิร์กโฟลว์ยังอ้างบ้าน 08/09/10 ทั้งที่ scope เปลี่ยนไปเป็นบ้าน 11 แล้ว

ต้า 2026-08-24

พอ scope ของการทดลองเปลี่ยน ให้เปลี่ยนชื่อไฟล์และแก๊ลิงก์ให้ตรงด้วย ชื่อไฟล์ที่โกหกแยกว่าลิงก์ที่พังเสียอีก เพราะมันพังแบบเจียบๆ

คำขอที่ 1 ของอาจารย์ที่ปรึกษา (พจนานุกรมไทย-อังกฤษ) ถูกวิจัยจนครบแล้วค่อยพักไว้ เพราะไม่มีใครนิยามไว้เลยว่ามันจะแก้ปัญหอะไร

ต้า 2026-08-25

ก่อนจะไปวิจัยข้อเสนอนี้ ให้ถามก่อนว่ามันเสี่ยงไปที่ความล้มเหลวที่วัดได้ตัวไหน ข้อ 3 ในมิติตั้งเดียวกันชี้ไปที่ตัวเลขที่ผิด แต่มีเป้าหมายจริงรองรับเลยคุ้มค่า ส่วนข้อ 1 กลับตรงข้ามเลยค้างตั้ง

หนังสือสอนรวมเล่มถูกตั้งชื่อตามเดือนกรกฎาคม แต่ดันมีบทของเดือนสิงหาคมอยู่ในนั้นแล้ว ทำให้ต่อท้าย 'ตอนที่ 4' ไม่ได้

ต้า 2026-08-25

อย่าใส่ช่วงวันที่ไว้ในชื่อไฟล์ของเอกสารที่จะต่อท้ายไปเรื่อยๆ (append-only) — ให้เปลี่ยนชื่อไฟล์ตั้งแต่ครั้งแรกที่ต่อท้าย ไม่ใช่มารอเปลี่ยนหลังต่อไปแล้วสามครั้ง

ไฟล์ `'pattern_recognition.py'` ที่อาจารย์อ้างถึง ถูกค้นหาทั่วทั้งสาม repo รวมประวัติ git ทั้งหมดแล้ว — ไม่มีอยู่จริงเลย

ต้า 2026-08-25

ต้องยืนยันก่อนว่าไฟล์ที่อาจารย์อ้างถึงเป็นของเราเองหรือเป็นแค่ตัวอย่างในเลกเชอร์ — เป็นบั๊กคลาสเดียวกับกรณี `max_frames`: สิ่งที่อาจารย์ชี้มาคือการพูดถึงแนวคิด ไม่จำเป็นต้องมี path จริงในโปรเจกต์เรา

PDF สอนถูกเขียนแบบย่อก่อน แล้วต้องเขียนใหม่ทั้งหมดจนได้ 13 หน้า หลังโดนขอว่า 'ขอละเอียดกว่านี้'

ต้า 2026-08-26

สำหรับเอกสารสอน ให้ถามระดับความละเอียดที่ต้องการไว้ก่อนตั้งแต่แรก และตั้งเป็นค่าเริ่มต้นว่าต้องโชว์ตัวเลือกที่ถูกปฏิเสธพร้อมเหตุผลด้วย — ตัวเลือกที่ถูกตัดทิ้งมักเป็นส่วนที่มีประโยชน์ที่สุด

ตั้งชื่อ commit ว่า 't04' เป็นเรื่องตลก ทั้งที่ไม่มีโมดูล t04 อยู่ในนั้นเลย — กลายเป็นกับดักการตั้งชื่อ

ต้า 2026-08-26

ข้อความ commit จะถูกอ่านโดยคนที่ไม่มีบริบทในอีกหลายเดือนข้างหน้า อย่าใช้ชื่อตลกที่ไปชนกับระบบตั้งชื่อจริงของโปรเจกต์

CLAUDE.md เกิด merge conflict เพราะสองคนต่อท้ายบันทึกงานที่จุดเดียวกันพร้อมกัน

ต้า 2026-08-26

log ที่ใช้ร่วมกันแบบต่อท้ายบนสุดจะ conflict ทุกครั้งที่มีคนทำงานพร้อมกันในวันเดียวกัน ทางแก้คือต่อท้ายที่ล่างสุดแทน หรือไม่ก็แยกไฟล์ต่อวันแบบที่ `workmen's_diary` ทำอยู่แล้ว

ไฟล์ sidecar จากการทดสอบ OCR 26 ไฟล์ ถูกทิ้งไว้ในโฟลเดอร์ `image/` ของจริง

ต้า 2026-08-29

การทดสอบเครื่องมือที่เขียนไฟล์ sidecar ต้องชี้ output ไปที่โฟลเดอร์ทดลองต่างหากเสมอ เพราะไฟล์ sidecar ที่เขียนปนกับข้อมูลจริงจะแยกไม่ออกจากผลลัพธ์จริงในรอบถัดไป

สตีมา §1 มีข้อความอ้างว่า prompt ไม่ได้สอนเรื่อง split ทั้งที่จริงๆ สอนไปแล้ว

ต้า 2026-08-29

หลักการเขียนเอกสารบอกว่าไฟล์อื่น *ไม่* ทำอะไร ถ้าจำเป็นต้องเขียนจริงๆ ให้ใส่ตัวชี้ไปยังไฟล์นั้นเพื่อให้การแก้ไขที่นั่นเตือนให้มาอัปเดตเอกสารนี้ด้วย

Pass 4 ถูกยกเลิกหลังจากสร้างเสร็จและต่อสายเข้า pipeline เรียบร้อยแล้ว

ต้า 2026-08-29

ก่อนสร้างและต่อสาย pipeline stage ใดก็ตาม ให้ยืนยันก่อนว่าจำเป็นจริงๆ เวลาจะยกเลิก ให้เก็บพักไว้แทนการลบทิ้ง — แต่การไปๆ มาๆ แบบนี้ก็เป็นหลักฐานว่า scope ควรตกลงกันให้ชัดตั้งแต่แรก

arm การทดลอง 2.4b ที่สร้างด้วยเทคนิคสลับภาพที่มีเครื่องหมายกำกับ ถูกสร้างเสร็จแล้วยกเลิกก่อนได้รันเลยสักครั้ง

ต้า 2026-08-29

ให้ตัดสินใจ matrix การทดลองให้เสร็จก่อนเริ่มลงมือสร้าง arm สร้างสาม arm แล้วยกเลิกหนึ่งยังถูกกว่าไม่สร้างเลย — แต่จะคุ้มก็ต่อเมื่อ arm เหล่านั้นตัดสินใจล่วงหน้าไม่ได้จริงๆ เท่านั้น

prompt ของ plan_column ถูก render เป็นไฟล์จริงเหมือนใช้งานได้ ทั้งที่ subtask นี้ไม่มี training config รองรับ

ต้า 2026-08-29

อย่า render artifact ที่ดูเหมือนใช้งานได้จริงสำหรับ target ที่ตายแล้ว ถ้าจำเป็นต้องมีไว้อ้างอิง ให้แยกไว้คนละโพลเดอร์ชัดเจน อย่างวางปนกับสามไฟล์ที่ใช้งานจริง

สั่ง op04 ให้ทำ 'บ้านที่ยังไม่เคยทำ' ทั้งที่จริงๆ คลังข้อมูล 50 บ้านถูกสกัดครบไปหมดแล้ว

ต้า 2026-08-29

ให้มีตัวนับจำนวนที่สกัดแล้ว/ที่เหลืออยู่ ณ จุดที่มีการสั่งงาน การเช็คงานค้างก่อนรับคำสั่งนี้แหละที่ช่วยให้ความเสี่ยงดาวนโหลดซ้ำกลายเป็นชุดข้อมูลใหม่ที่สะอาดแทน

บันทึกว่าได้ล็อกอิน HF ครบทั้ง 4 การ์ดแล้ว ทั้งที่จริงทำไปแค่ 3/4

ต้า 2026-08-30

บันทึกสถานะการตั้งค่าต่อเครื่องได้ก็ต่อเมื่อรันคำสั่งยืนยันบนเครื่องนั้นจริงๆ แล้วเท่านั้น (เช่น `hf whoami`) ห้ามสันนิษฐานว่าขั้นตอนที่ทำซ้ำๆ เสร็จหมดทุกเครื่องแล้ว

ไฟล์ `testfile.txt` ที่ทิ้งไว้ตอน debug บั๊กการอัปโหลด ดันหลุดไปอยู่ใน HF repo สาธารณะ

ต้า 2026-08-30

เวลา debug กับปลายทางที่เป็น production หรือสาธารณะ ให้ตั้งชื่อ artifact ให้เด่นชัดต่างหาก และลบทิ้งในขั้นตอนเดียวกับที่สร้างมันขึ้นมาเลย อย่าฝากความหวังไว้กับความจำว่าจะกลับมาเคลียร์ทีหลัง

โพลเดอร์ output มีขนาด 5.6GB ทั้งที่ adapter จริงมีแค่ 842MB — ถ้าอัปโหลดจริงๆ จะดันไฟล์ checkpoint ที่ซ้ำซ้อนขึ้นไปด้วย

ต้า 2026-08-30

ให้แยก artifact ที่ตั้งใจจะเผยแพร่ออกมาไว้ในโพลเดอร์ staging ก่อนอัปโหลด อย่ามารองโพลเดอร์ที่ปนกันอยู่ตอนอัปโหลด

การนับลำดับ fold ระหว่างชื่อ repo บน HF (เริ่มที่ 1) กับไฟล์ภายใน (เริ่มที่ 0) ไม่ตรงกัน — เสี่ยง mismatch จริงๆ

ต้า 2026-08-30

อย่าใช้ฐานการนับสองแบบสำหรับ entity เดียวกัน ถ้าเสี่ยงไม่ได้จริงๆ ให้เขียนตารางแมปให้ครบไว้ในเช็คลิสต์ แล้วตรวจสอบทุก artifact กับตารางนั้นก่อนเผยแพร่

วาง Supabase service key ผิดช่อง config และตัวอักษรแรกหายไปด้วย (`b\_secret\_` แทนที่จะเป็น `sb\_secret\_`)

ต้า 2026-08-30

ให้ validate secret ตอนโหลดค่า — เช็ค prefix ที่ควรจะเป็น (`sb\_secret\_`) และปฏิเสธ config key ที่ไม่รู้จักรู้ให้เห็นชัดๆ เพื่อให้คำที่วางผิดช่องหรือพิมพ์ตกหล่นขึ้น error ที่ชัดเจน แทนที่จะขึ้นเป็น auth error things

template ที่รอตรวจสอบโดยคนค้างอยู่ 146 รายการ — คอขวดคือคนไม่ใช่เครื่องมือ และการอนุมัติอัตโนมัติก็ถูกห้ามไว้จากบทเรียนก่อนหน้านี้แล้ว

ตำ 2026-08-30

เมื่อขั้นตอนนั้นต้องคงเป็นการตรวจสอบด้วยมือเพื่อความถูกต้อง ให้สร้างเครื่องมือที่ทำให้ขั้นตอนมือนั้นเร็วขึ้น แทนที่จะปล่อยให้ควายขึ้นเรื่อยๆ — และให้เครื่องมือนั้นทำได้แค่แนะนำ (ผลิตคำสั่งออกมาให้) ไม่ใช่ลงมือทำเอง

ยอดเครดิตคงเหลือใน Vast.ai ถูกบันทึกว่า \$16.66 ทั้งที่จริงๆ มี \$84.82

ตำ 2026-08-31

ให้เช็คยอดเงินจริงจากผู้ให้บริการอีกครั้งตอนที่ตัดสินใจใช้จ่าย ห้ามวางแผนจากตัวเลขที่ถือมาจากบันทึกเก่า

รูปบ้านที่ใช้ทำ benchmark ยังค้างอยู่ใต้ t04_Purson — เป็นงานค้างข้ามวันที่สองแล้ว

ตำ 2026-08-31

ถ้าเลื่อนการเก็บกวาดเพราะมีงานกำลังรันอยู่ ให้ตั้งกำหนดเก็บวันที่ที่งานนั้นเสร็จ ไม่ใช่โยนเข้าลิสต์ to-do ทิ้งไป ไม่จันมันจะค้างไปเรื่อยๆ

แก้จำนวน epoch ในคอนฟิกของทั้งสองฝั่งก่อนที่ผลวิจัยยืนยันจะออกมา

ตำ 2026-08-31

ถ้าคำสั่งบอกให้ทำการเปลี่ยนแปลงคู่กับขั้นตรวจสอบ ให้ตรวจสอบให้เสร็จก่อนค่อย commit การเปลี่ยนแปลง เว้นแต่คำสั่งจะบอกไว้ชัดเจนเป็นอย่างอื่น

ประเมินเวลาของ batch CV ไว้ 70 วินาที/หน้า จากหน้าแรกที่เพิ่ง cold-start แต่อัตราจริงอยู่ที่ราว 13 วินาที

ตำ 2026-08-31

หึ่งตัวอย่างแรกออกไปเวลาประมาณ throughput — ต้องวัดตอน steady state จากหลายๆ รายการก่อนค่อยคาดการณ์เวลารวมของ batch

สงสัยว่า rasterizer ผลิต DPI ไม่คงที่ แต่พอวัดจริงพบว่า DPI ปกติดี ความแปรปรวนมาจากบ้านหลังเดียว

ตำ 2026-08-31

วัดการกระจายตัวของข้อมูลก่อนตั้งชื่อสาเหตุ — 'ขนาดมันแปรปรวน' กับ 'เครื่องมือมันพัง' เป็นคนละข้อกล่าวหากัน

การเปลี่ยนชื่อ t05_Asclepius เป็น t05_Courser ทำให้ dataset ทั้งสองชุดใช้งานไม่ได้ เพราะ path ถูกฝังไว้ใน JSON ที่สร้างขึ้น

ตำ 2026-08-31

อย่าฝัง path ไว้ในข้อมูลที่ generate ออกมา — ให้ resolve จาก root เดียวตอนโหลดข้อมูล ไม่จันแค่เปลี่ยนชื่อโฟลเดอร์ก็ต้อง rebuild ใหม่ทั้งหมด

ขั้นตอนเปิดหน้าต่าง worker คือขั้นตอนที่ลิมบอยที่สุดตอนสตาร์ทเดโมด้วยมือ

ตำ 2026-08-31

ขั้นตอนสตาร์ทที่ลิมง่ายและลิมแล้วไม่มีอะไรฟ้อง ควรทำให้เป็นอัตโนมัติหรือมีตัวตรวจเช็ค ไม่ใช่แค่เขียนไว้ในเอกสาร

pass2.5 เขียนผล self-harvest ลงใน results โดยไม่มีใครนำไปใช้เลย

ตำ 2026-09-01

อย่าปล่อย producer stage ออกมาก่อนที่ consumer ของมันจะพร้อม ถ้าจำเป็นต้องทำจริงๆ ให้ทำเครื่องหมายไว้ให้ชัดทั้งในโค้ดและในไดอารี่ อย่าให้ใครเข้าใจผิดว่า 'ไฟล์มีอยู่' แปลว่า 'ฟีเจอร์ใช้งานได้'

การทดลอง greedy-decoding ทั้งรอบล้มเหลว ต้องเก็บเข้า archive เป็นผลลัพธ์ที่ใช้ไม่ได้

ตำ 2026-09-01

ตั้งชื่อโฟลเดอร์ผลการทดลองให้บอกผลลัพธ์และลำดับไว้ตั้งแต่สร้าง (เช่น `01\_usable`, `02\_failed\_greedy`) ไม่ใช่มาตั้งทีหลัง — โฟลเดอร์ผลลัพธ์ที่ชื่อกำกวมทำให้ต้องมาไล่เก็บกวาดทีหลัง แถมเสี่ยงมีคนหยิบผลที่ตายไปแล้วไปอ้างอิง

INVENTORY.csv ตกยุคไปถึง 70 แถว มีงานทั้งหมด 6 หมวดที่ไม่ได้ถูกบันทึกเลย

ตำ 2026-09-01

อินเวนทอรีที่อัปเดตเป็นรอบๆ คืออินเวนทอรีที่ผิดตลอดเวลา — โมเดล/fold/ชุดผลลัพธ์ใหม่ควรถูกบันทึกลงแถวทันทีที่สร้าง ไม่ใช่รอถึงรอบเก็บกวาดครั้งถัดไป

ข้าม npm test ทั้ง session โดยให้เหตุผลว่าแก้ไขไฟล์ที่ไม่มีเทสต์คอมมูน

ต้า 2026-09-02

'ไฟล์นี้ไม่มีเทสต์' คือเหตุผลที่ควรเพิ่มเทสต์ ไม่ใช่เหตุผลที่จะข้ามชุดเทสต์ทั้งหมด — รันชุดเทสต์ไปเลย มันถูกและช่วยคุ้มกันงานที่เหลือของทั้งวัน